API docs

目錄

1. List available models
2. Chat completions (supports streaming and tool calls)
3. Generate text embeddings

AI Gateway provides OpenAI-compatible endpoints, allowing existing OpenAI SDKs or tools to call models through MaiAgent by changing only the URL and API key. These endpoints are available only under `/api/v1/`, and their paths do not end with `/`. Authentication uses `Authorization: Bearer` (the key is your MaiAgent API Key, but Gateway endpoints accept only the `Bearer` prefix; using the `Api-Key` prefix returns `401`). MaiAgent must enable AI Gateway for your organization. Administrators configure which models each key can call in API Key management. For full details, see the technical manual: AI Gateway: OpenAI-compatible API integration.

List available models

GET `/api/v1/models`

Returns the model IDs and pricing for models this key can call. For model ID conventions, see Model IDs in the technical manual.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/models"

  -H "Authorization: Bearer YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/models", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/models"
headers = {
    "Authorization": "Bearer YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/models", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

Chat completions (supports streaming and tool calls)

POST `/api/v1/chat/completions`

The request and response formats match OpenAI chat/completions (model, messages, stream, tools, etc.); the platform schema does not list these fields. For parameters and examples, see Chat completion parameters and Minimal examples in the technical manual.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/chat/completions"

  -H "Authorization: Bearer YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/chat/completions", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/chat/completions"
headers = {
    "Authorization": "Bearer YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/chat/completions", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

Generate text embeddings

POST `/api/v1/embeddings`

The request and response formats match OpenAI embeddings (model, input). For examples, see Embeddings in the technical manual.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/embeddings"

  -H "Authorization: Bearer YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/embeddings", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/embeddings"
headers = {
    "Authorization": "Bearer YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/embeddings", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

Example API Call (Shell)

目錄

1. Create a New AI Assistant
2. List Embedding Models
3. List Reranker Models
4. List Large Language Models
5. List AI Assistants
6. Retrieve a Specific AI Assistant
7. Update an AI Assistant
8. Partially Update an AI Assistant
9. Delete AI Assistant
10. Duplicate AI Assistant
11. Enable or Disable Evaluation
12. List Hooks for the Organization
13. Create a Hook from a Template
14. Retrieve a Specific Hook
15. Partially Update a Hook
16. Delete Hook
17. List Hooks Attached to an AI Assistant
18. Replace All Hooks Attached to an AI Assistant
19. List Schedules
20. Create Schedule
21. Retrieve a Specific Schedule
22. Partially Update a Schedule
23. Delete a Schedule
24. Run a Schedule Immediately
25. Enable or Disable a Schedule

Example API Call (Shell)

Create a New AI Assistant

`POST` `/api/v1/chatbots/`

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Chatbot name. In Agent mode it carries semantic meaning; in other modes it only

Field	Type	Required	Description
			distinguishes chatbots.
largeLanguageModel	string (uuid)	Yes	Large language model used by the chatbot to generate responses (including custom AI Gateway models created by the organization)
embeddingModel	string (uuid)	No	Embedding model used to vectorize text; optional
rerankerModel	string (uuid)	No	Model used for reranking; optional
instructions	string	No	Role instructions describing the chatbot's role and behavior.
knowledgeBases	array[IdName]	No	List of knowledge bases the chatbot can access
databases	array[IdName]	No	List of SQL databases the chatbot can access (for Text-to-SQL)
organization	string (uuid)	No	Organization the chatbot belongs to; if empty, it is a personal chatbot
replyMode	object	No	Reply mode: normal or streaming responses `normal` : Normal ; `template` : Template ; `hybrid` : Hybrid ; `agent` : Agent;
template	string	No	Template used by Template and Hybrid modes.
unanswerableTemplate	string	No	Template used by Template and Hybrid modes when the question cannot be answered.
totalWordsCount	integer (int64)	No	Accumulated total word count used.
outputMode	object	No	Output mode: text, table, or custom format `text` : Text ; `json_schema` : JSON Schema;
rawOutputFormat	object	No	JSON structure definition for the custom output format
groups	array[IdName]	No	List of groups the chatbot can access
tools	array[ToolSummary]	No	List of tools the chatbot can use
skills	array[SkillSummary]	No	List of skills associated with the chatbot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode `normal` : Normal ; `canvas` : Canvas ; `team` : Team;
numberOfRetrievedChunks	integer	No	Number of reference chunks to retrieve; default: 12, minimum: 1

Field	Type	Required	Description
defaultLabelRelations	object	No	Default knowledge base label filters at the assistant level, using the same format as query_metadata.label_relations; null means no filtering. Applied only as the lowest-priority fallback when the caller does not provide query_metadata.
useFullRetrievalRange	boolean	No	When enabled, ignores LLM top_k truncation and returns the complete reranker results
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enables inline citations, inserting citation markers such as [1][2] into responses.
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to run code in an isolated container.
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial outpu...
customMaxLlmOutputTokens	integer	No	Custom maximum number of LLM output tokens; minimum: 512
voiceConfig	object	No	
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type.
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider.
voiceConfig.sttConfig	object	No	STT configuration parameters (JSON).
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider.
voiceConfig.ttsConfig	object	No	TTS configuration parameters (JSON).
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider.
voiceConfig.realtimeConfig	object	No	Realtime configuration parameters (JSON).

Field	Type	Required	Description
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty
voiceConfig.turnHandling	object	No	Turn-taking and interruption settings for the voice agent.
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Reasoning level: null (model default) or {"reasoning_effort": "off
enableToolSearching	boolean	No	Enables dynamic tool searching, allowing the agent to discover and load tools dynamically through tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
requireTextWithFlexCards	boolean	No	When enabled, a turn that would end with only Flex Message cards and no text for the user is re-issued to the LLM once, asking it to write the text first and then send the card. Each retry incurs a se...
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversati...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat hist...
longTermMemoryTopK	integer	No	Number of long-term memory nodes returned per retrieval (similarity_top_k for vector retrieval); default: 10, range: 1–50
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other

Field	Type	Required	Description
			conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search...
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Example Request Schema

```
{
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate
responses (including custom AI Gateway models created by the organization)
  "embeddingModel"?: string (uuid) // Embedding model used to vectorize text; optional
(Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking; optional (Optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior.
(Optional)
  "knowledgeBases"?: [ // List of knowledge bases the chatbot can access (Optional)
    {
      "id": string (uuid)
    }
  ]
  "databases"?: [ // List of SQL databases the chatbot can access (for Text-to-SQL) (Optional)
    {
      "id": string (uuid)
```

```
    }
  ]
  "organization"?: string (uuid) // Organization the chatbot belongs to; if empty, it is a
personal chatbot (Optional)
  "replyMode"?:  // Reply mode: normal or streaming responses

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON structure definition for the custom output format
(Optional)
  "groups"?: [ // List of groups the chatbot can access (Optional)
    {
      "id": string (uuid)
    }
  ]
  "tools"?: [ // List of tools the chatbot can use (Optional)
    {
      "id": string (uuid)
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
    }
  ]
  "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to retrieve; default: 12,
minimum: 1 (Optional)
  "defaultLabelRelations"?: object // Default knowledge base label filters at the assistant
level, using the same format as query_metadata.label_relations; null means no filtering.
Applied only as the lowest-priority fallback when the caller does not provide query_metadata.
```

```
(Optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores LLM top_k truncation and returns the complete reranker results (Optional)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers such as [1][2] into responses. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial output already rendered. Each retry incurs a second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum: 512 (Optional)
  "voiceConfig"?: { // Optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent. (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and does not speak (Optional)
  }
  }
  "thinkingConfig"?: object // Reasoning level: null (model default) or {"reasoning_effort": "off|minimal|low|medium|high|xhigh|max"} (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool searching, allowing the agent to discover and load tools dynamically through tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100, default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200, default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default. (Optional)
  "requireTextWithFlexCards"?: boolean // When enabled, a turn that would end with only Flex Message cards and no text for the user is re-issued to the LLM once, asking it to write the text first and then send the card. Each retry incurs a second billing for the same prompt. (Optional)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversation search results keep their 100-character cut either way. Reduces memory bloat from large tool outputs (code interpreter bash stdout, web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
```

```
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
  "longTermMemoryTopK"?: integer // Number of long-term memory nodes returned per retrieval
(similarity_top_k for vector retrieval); default: 10, range: 1–50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
}
```

Example Request Values

```
{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
```

```
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "defaultLabelRelations": null,
  "useFullRetrievalRange": true,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
  "enableBrowserTool": true,
  "enableImageSearch": true,
  "enableClaudeStreamRetry": true,
  "customMaxLlmOutputTokens": 123,
  "voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
  },
  "thinkingConfig": null,
  "enableToolSearching": true,
  "maxAgentIterations": 123,
  "agentTimeoutSeconds": 123,
  "agentTimeoutMessage": "Example String",
  "requireTextWithFlexCards": true,
  "enableSizeBasedToolResultTruncation": true,
  "toolResultMaxChars": 123,
  "enableVectorMemory": true,
  "longTermMemoryTopK": 123,
  "enableCrossConversationSearch": true,
  "crossConversationLookbackDays": 123,
  "useGatewayFallback": true,
```

```
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": true,
  "enableGetMessageContextTool": true,
  "enableReadAttachmentFullTextTool": true,
  "enableParseAttachmentContentTool": true,
  "enableListAvailableParsersTool": true,
  "enableGetDownloadLinkTool": true
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "databases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "tools": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "skills": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "connectors": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
```

```
      "numberOfRetrievedChunks": 123,
      "defaultLabelRelations": null,
      "useFullRetrievalRange": true,
      "enableEvaluation": true,
      "enableInlineCitations": true,
      "enableCodeInterpreter": true,
      "enableBrowserTool": true,
      "enableImageSearch": true,
      "enableClaudeStreamRetry": true,
      "customMaxLlmOutputTokens": 123,
      "voiceConfig": {
        "enabled": true,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": null,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": null,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": null,
        "realtimeInstructions": "Example String",
        "turnHandling": null,
        "enableVoiceReply": true
      },
      "thinkingConfig": null,
      "enableToolSearching": true,
      "maxAgentIterations": 123,
      "agentTimeoutSeconds": 123,
      "agentTimeoutMessage": "Example String",
      "requireTextWithFlexCards": true,
      "enableSizeBasedToolResultTruncation": true,
      "toolResultMaxChars": 123,
      "enableVectorMemory": true,
      "longTermMemoryTopK": 123,
      "enableCrossConversationSearch": true,
      "crossConversationLookbackDays": 123,
      "useGatewayFallback": true,
      "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
      "enableSearchMessagesTool": true,
      "enableGetMessageContextTool": true,
      "enableReadAttachmentFullTextTool": true,
      "enableParseAttachmentContentTool": true,
      "enableListAvailableParsersTool": true,
      "enableGetDownloadLinkTool": true
    }'

  # Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example string",
    "knowledgeBases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "databases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example string",
    "unanswerableTemplate": "Example string",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "tools": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "skills": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "connectors": [
      {
```

```
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": null,
    "useFullRetrievalRange": true,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
      "enabled": true,
      "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
      "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
      "sttConfig": null,
      "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
      "ttsConfig": null,
      "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
      "realtimeConfig": null,
      "realtimeInstructions": "Example string",
      "turnHandling": null,
      "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example string",
    "requireTextWithFlexCards": true,
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": true,
    "enableGetMessageContextTool": true,
    "enableReadAttachmentFullTextTool": true,
    "enableParseAttachmentContentTool": true,
    "enableListAvailableParsersTool": true,
    "enableGetDownloadLinkTool": true
  };

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
```

```
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example string",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example string",
    "unanswerableTemplate": "Example string",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": None,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
```

```
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": None,
    "useFullRetrievalRange": True,
    "enableEvaluation": True,
    "enableInlineCitations": True,
    "enableCodeInterpreter": True,
    "enableBrowserTool": True,
    "enableImageSearch": True,
    "enableClaudeStreamRetry": True,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
        "enabled": True,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": None,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": None,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": None,
        "realtimeInstructions": "Example string",
        "turnHandling": None,
        "enableVoiceReply": True
    },
    "thinkingConfig": None,
    "enableToolSearching": True,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example string",
    "requireTextWithFlexCards": True,
    "enableSizeBasedToolResultTruncation": True,
    "toolResultMaxChars": 123,
    "enableVectorMemory": True,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": True,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": True,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": True,
    "enableGetMessageContextTool": True,
    "enableReadAttachmentFullTextTool": True,
    "enableParseAttachmentContentTool": True,
    "enableListAvailableParsersTool": True,
    "enableGetDownloadLinkTool": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'largeLanguageModel' => '550e8400-e29b-41d4-a716-446655440000',
            'embeddingModel' => '550e8400-e29b-41d4-a716-446655440000',
            'rerankerModel' => '550e8400-e29b-41d4-a716-446655440000',
            'instructions' => 'Example string',
            'knowledgeBases' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'databases' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'organization' => '550e8400-e29b-41d4-a716-446655440000',
            'replyMode' => new \stdClass(),
            'template' => 'Example string',
            'unanswerableTemplate' => 'Example string',
            'totalWordsCount' => 123,
            'outputMode' => new \stdClass(),
            'rawOutputFormat' => null,
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'tools' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'skills' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'connectors' => [
                [
```

```
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'agentMode' => new \stdClass(),
            'numberOfRetrievedChunks' => 123,
            'defaultLabelRelations' => null,
            'useFullRetrievalRange' => true,
            'enableEvaluation' => true,
            'enableInlineCitations' => true,
            'enableCodeInterpreter' => true,
            'enableBrowserTool' => true,
            'enableImageSearch' => true,
            'enableClaudeStreamRetry' => true,
            'customMaxLlmOutputTokens' => 123,
            'voiceConfig' => [
                'enabled' => true,
                'voiceAgentType' => '550e8400-e29b-41d4-a716-446655440000',
                'sttProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'sttConfig' => null,
                'ttsProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'ttsConfig' => null,
                'realtimeProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'realtimeConfig' => null,
                'realtimeInstructions' => 'Example string',
                'turnHandling' => null,
                'enableVoiceReply' => true
            ],
            'thinkingConfig' => null,
            'enableToolSearching' => true,
            'maxAgentIterations' => 123,
            'agentTimeoutSeconds' => 123,
            'agentTimeoutMessage' => 'Example string',
            'requireTextWithFlexCards' => true,
            'enableSizeBasedToolResultTruncation' => true,
            'toolResultMaxChars' => 123,
            'enableVectorMemory' => true,
            'longTermMemoryTopK' => 123,
            'enableCrossConversationSearch' => true,
            'crossConversationLookbackDays' => 123,
            'useGatewayFallback' => true,
            'gatewayFallbackGroup' => '550e8400-e29b-41d4-a716-446655440000',
            'enableSearchMessagesTool' => true,
            'enableGetMessageContextTool' => true,
            'enableReadAttachmentFullTextTool' => true,
            'enableParseAttachmentContentTool' => true,
            'enableListAvailableParsersTool' => true,
            'enableGetDownloadLinkTool' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
```

```
}
?>
```

Response

Status code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // The large language model used by the chatbot to generate answers (including organization-built AI Gateway models)
  "llm":  // The bound model's id, name, and retirement information, read directly from the FK; returned even if the organization has disabled the model
  {
    "id": string (uuid)
    "name": string
    "retiresAt": string (date) // Provider retirement date; after it passes, a daily job switches the assistant to the replacement and disables this model. null means no retirement is scheduled
    "replacement":  // The model that takes over after retirement (id and name); null means unspecified, and nothing is migrated automatically on expiry
    {
      "id": string (uuid)
      "name": string
    }
  }
  "embeddingModel"?: string (uuid) // The embedding model used to vectorize text，optional (optional)
  "rerankerModel"?: string (uuid) // The model used for reranking，optional (optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior. (optional)
  "knowledgeBases"?: [ // List of knowledge bases the chatbot can access (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"?: [ // List of SQL databases the chatbot can access (used for the Text-to-SQL feature) (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
```

```
  "updatedAt": string (timestamp)
  "organization"?: string (uuid) // The organization the chatbot belongs to; if empty, it is a
personal chatbot (optional)
  "replyMode"?:  // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (optional)
  "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition for the custom output format (optional)
  "groups"?: [ // List of groups the chatbot can access (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "tools"?: [ // List of tools the chatbot can use (optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (optional)
  {
```

```
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to retrieve. Default is 12, minimum is 1 (optional)
  "defaultLabelRelations"?: object // Assistant-level default knowledge base label filter, in the same format as query_metadata.label_relations; null means no filtering. Applied only as the lowest-priority fallback when the caller does not pass query_metadata. (optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores the LLM's top_k truncation and returns the full reranker results (optional)
  "enableEvaluation"?: boolean // optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers such as [1][2] into responses. (optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run code in an isolated container. (optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely. (optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial output already rendered. Each retry incurs a second billing for the same prompt. (optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens. Minimum is 512 (optional)
  "voiceConfig"?: { // optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle. (optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty (optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent. (optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and does not speak (optional)
  }
  }
  "thinkingConfig"?: object // Thinking level: null (model default) or {"reasoning_effort": "off|minimal|low|medium|high|xhigh|max"} (optional)
  "enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to discover and load tools dynamically through tool_searching_tool (optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100, default: 20) (optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200, default: 600) (optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default. (optional)
  "requireTextWithFlexCards"?: boolean // When enabled, a turn that would end with only Flex Message cards and no text for the user is re-issued to the LLM once, asking it to write the text first and then send the card. Each retry incurs a second billing for the same prompt. (optional)
  "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
```

```
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversation search results keep their 100-character cut either way. Reduces memory bloat from large tool outputs (code interpreter bash stdout, web search results, etc.). (optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat history window is used for multi-turn context. (optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes retrieved per long-term memory lookup (similarity_top_k of the vector search). Default is 10, range 1–50 (optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (optional)
  "gatewayFallbackGroup"?: string (uuid) // optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links for produced content. AGENT mode only. (optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example string",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example string",
    "retiresAt": "Example string",
    "replacement": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string"
```

```
    }
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string"
    }
  ],
  "updatedAt": "Example string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example string",
  "unanswerableTemplate": "Example string",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string",
      "description": "Example string",
      "displayName": "Example string",
      "toolType": "Example string"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string",
      "description": "Example string"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 456,
  "defaultLabelRelations": null,
  "useFullRetrievalRange": false,
```

```
  "enableEvaluation": false,
  "enableInlineCitations": false,
  "enableCodeInterpreter": false,
  "enableBrowserTool": false,
  "enableImageSearch": false,
  "enableClaudeStreamRetry": false,
  "customMaxLlmOutputTokens": 456,
  "voiceConfig": {
    "enabled": false,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example string",
    "turnHandling": null,
    "enableVoiceReply": false
  },
  "thinkingConfig": null,
  "enableToolSearching": false,
  "maxAgentIterations": 456,
  "agentTimeoutSeconds": 456,
  "agentTimeoutMessage": "Example string",
  "requireTextWithFlexCards": false,
  "isMultimodalLlm": false,
  "enableSizeBasedToolResultTruncation": false,
  "toolResultMaxChars": 456,
  "enableVectorMemory": false,
  "longTermMemoryTopK": 456,
  "enableCrossConversationSearch": false,
  "crossConversationLookbackDays": 456,
  "useGatewayFallback": false,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": false,
  "enableGetMessageContextTool": false,
  "enableReadAttachmentFullTextTool": false,
  "enableParseAttachmentContentTool": false,
  "enableListAvailableParsersTool": false,
  "enableGetDownloadLinkTool": false,
  "isLocked": false,
  "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
  "canUpdate": false,
  "canDelete": false
}
```


List Embedding Models

`GET` `/api/v1/embedding-models/`

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/embedding-models/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/embedding-models/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/embedding-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/embedding-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
[
  {
    "id": string (uuid)
    "name": string
    "isDefault"?: boolean // Optional
    "isMultimodal"?: boolean // Whether this embedding model supports multimodal (text + image)
embeddings (Optional)
    "icon":
    {
      "id": string (uuid)
      "name": string
      "image": string (uri)
    }
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isDefault": false,
    "isMultimodal": false,
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "image": "https://example.com/file.jpg"
    }
  }
]
```

List Reranker Models

GET `/api/v1/reranker-models/`

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/reranker-models/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/reranker-models/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/reranker-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/reranker-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
[
  {
    "id": string (uuid)
    "name": string
    "isDefault"?: boolean // Optional
    "icon":
    {
      "id": string (uuid)
      "name": string
      "image": string (uri)
    }
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isDefault": false,
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "image": "https://example.com/file.jpg"
    }
  }
]
```

List Large Language Models

GET `/api/v1/large-language-models/`

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/large-language-models/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/large-language-models/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/large-language-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/large-language-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
[
  {
    "id": string (uuid)
    "name": string
    "retiresAt": string (date) // Provider retirement date; after it passes, a daily job
switches assistants to the replacement and disables this model. null means no retirement is
scheduled
    "replacement":  // Model that takes over after retirement (id and name); null means none is
specified, so nothing is migrated automatically at expiry
    {
      "id": string (uuid)
      "name": string
    }
    "contextWindow"?: integer // Used only by OpenAILike or Ollama models; other models ignore
this value. (Optional)
    "isDefault"?: boolean // Optional
    "provider": string
    "servedBy": string // Who runs inference, which is where prompts are sent (for example
OpenRouter, AWS Bedrock); models on a customer credential show Customer credential.
    "icon":
    {
      "id": string (uuid)
      "name": string
      "image": string (uri)
    }
    "isMultiModalSupport"?: boolean // Optional
    "isFunctionalCallingAvailable": boolean // Deprecated: same value as
isToolCallingAvailable; use isToolCallingAvailable.
    "isToolCallingAvailable"?: boolean // Optional
    "isTemplateAvailable": boolean
    "isOutputFormatAvailable": boolean
    "isThinkingAvailable"?: boolean // When checked, this model shows the thinking depth
setting and requests carry thinking parameters (thinking_budget for Gemini, reasoning_effort
for OpenAI-compatible hosts). Unchecking stops sending them; saved thinking settings are kept
but have no effect. Check only for models known to support thinking parameters, otherwise the
provider returns 4xx. (Optional)
    "defaultThinkingLevel"?: // May have different types (Optional)
    string (enum: off, minimal, low, medium, high, xhigh, max) // The thinking level sent when
neither the conversation nor the assistant sets one. Empty sends no thinking parameter, so the
provider default applies. A level the model does not accept is lowered to the nearest one it
does. Takes effect only when Supports Thinking is checked.

* `off` - off
* `minimal` - minimal
* `low` - low
* `medium` - medium
* `high` - high
* `xhigh` - xhigh
```

```
* `max` - max (Optional)
    "thinkingConfigSchema": { // Thinking settings form for this model; null means this model
has no thinking settings
    {
      "type"?: string // Form type, always reasoning_effort (Optional)
      "fields"?: object // reasoning_effort field: options lists the thinking levels this model
accepts (weakest to strongest); default is always null (Optional)
      "canDisable"?: boolean // Whether this model can turn thinking off; when true, options
includes off (Optional)
      "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model thinks on its own, or the platform default level enables thinking
(Optional)
      "unsetLevel"?: string // Level the platform actually sends when none is set; null means
no thinking parameter is sent and the model's own default applies (Optional)
    }
    }
    "replyMode": [
      string
    ]
    "outputMode": [
      string
    ]
    "agentMode": [
      string
    ]
    "maxTokens"?: integer // Optional
    "minTokens": integer
    "isCustomModel": boolean // Whether the row is organization-owned (materialized from an AI
Gateway custom model).
    "originCountry": string // ISO 3166-1 alpha-2 code of the country the model developer is
based in (e.g. US, CN, FR); null when unknown. Follows the model, not who runs inference.
    "originRegion": // May have different types
    string (enum: cn, us, other) // Origin group for labels and filters: cn / us / other; null
when unknown (no label).

* `cn` - China
* `us` - United States
* `other` - Other countries
    "originRegionDisplay": string // Translated origin group name (e.g. China / United States /
Other countries); null when unknown.
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "retiresAt": "Response string",
    "replacement": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    },
```

```
    "contextWindow": 456,
    "isDefault": false,
    "provider": "Response string",
    "servedBy": "Response string",
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "image": "https://example.com/file.jpg"
    },
    "isMultiModalSupport": false,
    "isFunctionalCallingAvailable": false,
    "isToolCallingAvailable": false,
    "isTemplateAvailable": false,
    "isOutputFormatAvailable": false,
    "isThinkingAvailable": false,
    "defaultThinkingLevel": "off",
    "thinkingConfigSchema": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    },
    "replyMode": [
      "Response string"
    ],
    "outputMode": [
      "Response string"
    ],
    "agentMode": [
      "Response string"
    ],
    "maxTokens": 456,
    "minTokens": 456,
    "isCustomModel": false,
    "originCountry": "Response string",
    "originRegion": "cn",
    "originRegionDisplay": "Response string"
  }
]
```


List AI Assistants

GET `/api/v1/chatbots/`

Parameters

Parameter Name	Required	Type	Description

`largeLanguageModel`	X	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`pagination`	X	string	Whether to paginate (true/false)
`query`	X	string	
`replyMode`	X	string	
`source`	X	string	`self_built`: self_built ; `built_in`: built_in ; `all`: all;

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes
it only distinguishes chatbots.
      "groups": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "largeLanguageModel": { // Base LLM serializer: identification fields plus retirement
information, allowing the frontend to warn when a model is about to retire.
```

```
      {
        "id": string (uuid)
        "name": string
        "retiresAt": string (date) // Provider retirement date; once this date is reached, the
daily scheduled job switches assistants to replacement and disables this model. null means no
retirement is scheduled
        "replacement":  // The replacement model after retirement (id and name); null means no
replacement is specified, so no automatic migration occurs on the retirement date
        {
          "id": string (uuid)
          "name": string
        }
      }
      }
      "embeddingModel": {
      {
        "id": string (uuid)
        "name": string
        "isDefault"?: boolean // Optional
        "isMultimodal"?: boolean // Whether this embedding model supports multimodal (text +
image) embeddings (Optional)
        "icon":
        {
          "id": string (uuid)
          "name": string
          "image": string (uri)
        }
      }
      }
      "rerankerModel": {
      {
        "id": string (uuid)
        "name": string
        "isDefault"?: boolean // Optional
        "icon":
        {
          "id": string (uuid)
          "name": string
          "image": string (uri)
        }
      }
      }
      "knowledgeBases": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "tools": [
        {
          "id": string (uuid)
          "name": string
          "description": string
          "displayName": string
          "toolType": string
        }
      ]
```

```
      "skills": [
        {
          "id": string (uuid)
          "name": string
          "description": string
        }
      ]
      "agentMode"?:  // Agent mode: Canvas or Normal. Canvas mode generates answers on a
canvas; Normal mode generates regular answers.

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
      {
      }
      "replyMode"?:  // Reply mode: Normal, Template, Hybrid, or Agent.

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
      {
      }
      "updatedAt": string (timestamp)
      "enableEvaluation"?: boolean // Optional
      "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
      "enableToolSearching"?: boolean // Enables dynamic tool search, letting the agent
discover and load tools via tool_searching_tool. (Optional)
      "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
run code in an isolated container. (Optional)
      "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
      "enableImageSearch"?: boolean // Provide the image_search tool when any attached
knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
(Optional)
      "numberOfRetrievedChunks"?: integer // Optional
      "useFullRetrievalRange"?: boolean // When enabled, the retrieval tool ignores the top_k
set by the LLM and returns the full reranker result up to the maximum number of retrieved
chunks. Useful for knowledge bases with long tables that need a wider retrieval range.
(Optional)
      "isMultimodalLlm": boolean // Return whether the chatbot's LLM supports multimodal.
      "voiceConfig":
      {
        "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by
every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a
per-platform toggle. (Optional)
      }
      "createdBy":
      {
        "id": string (uuid)
        "name": string
      }
      "isLocked"?: boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified (Optional)
      "sourceBuildInAgent"?: string (uuid) // Points to the official Build-in Agent blueprint
when instantiated by a rental (Optional)
      "canUpdate": boolean // Return whether the current user can update this resource.
```

```
      "canDelete": boolean // Return whether the current user can delete this resource.
    }
  ]
}
```

Example Response Values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "groups": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "largeLanguageModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "retiresAt": "Response string",
        "replacement": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      },
      "embeddingModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isDefault": false,
        "isMultimodal": false,
        "icon": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
          "image": "https://example.com/file.jpg"
        }
      },
      "rerankerModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isDefault": false,
        "icon": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
          "image": "https://example.com/file.jpg"
        }
      },
      "knowledgeBases": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
```

```
        }
      ],
      "tools": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
          "description": "Response string",
          "displayName": "Response string",
          "toolType": "Response string"
        }
      ],
      "skills": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
          "description": "Response string"
        }
      ],
      "agentMode": {},
      "replyMode": {},
      "updatedAt": "Response string",
      "enableEvaluation": false,
      "enableInlineCitations": false,
      "enableToolSearching": false,
      "enableCodeInterpreter": false,
      "enableBrowserTool": false,
      "enableImageSearch": false,
      "numberOfRetrievedChunks": 456,
      "useFullRetrievalRange": false,
      "isMultimodalLlm": false,
      "voiceConfig": {
        "enabled": false
      },
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "isLocked": false,
      "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
      "canUpdate": false,
      "canDelete": false
    }
  ]
}
```


Retrieve a Specific AI Assistant

GET `/api/v1/chatbots/{id}`

Parameters

Parameter	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Response Structure Example

```
{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate answers (including organization-built AI Gateway models)
  "llm":  // ID, name, and retirement information of the bound model, read directly from the FK; returned even if the organization has disabled the model
  {
    "id": string (uuid)
    "name": string
    "retiresAt": string (date) // Provider retirement date; after it passes, the daily job switches the assistant to the replacement and disables this model. null means no retirement is scheduled
    "replacement":  // Model that takes over after retirement (id and name); null means none is specified, so nothing is migrated automatically on expiry
    {
      "id": string (uuid)
      "name": string
    }
  }
  "embeddingModel"?: string (uuid) // Embedding model used to vectorize text; optional (Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking; optional (Optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior. (Optional)
  "knowledgeBases"?: [ // List of knowledge bases the chatbot can access (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"?: [ // List of SQL databases the chatbot can access (used for the Text-to-SQL feature) (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization"?: string (uuid) // Organization the chatbot belongs to; if empty, it is a personal chatbot (Optional)
  "replyMode"?:  // Reply mode: normal or streaming responses

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
```

```
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition of the custom output format (Optional)
  "groups"?: [ // List of groups the chatbot can access (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "tools"?: [ // List of tools the chatbot can use (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to retrieve; defaults to
12, minimum 1 (Optional)
  "defaultLabelRelations"?: object // Assistant-level default knowledge base label filter, in
the same format as query_metadata.label_relations; null means no filtering. Applied as the
lowest-priority fallback only when the caller does not provide query_metadata. (Optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores the LLM's top_k truncation and
returns the full reranker results (Optional)
  "enableEvaluation"?: boolean // Optional
```

```
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count; minimum 512
(Optional)
  "voiceConfig"?: { // Optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
  }
  }
  "thinkingConfig"?: object // Thinking level: null (model default) or {"reasoning_effort":
"off|minimal|low|medium|high|xhigh|max"} (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool searching, allowing the agent to
discover and load tools dynamically via tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
  "requireTextWithFlexCards"?: boolean // When enabled, a turn that would end with only Flex
Message cards and no text for the user is re-issued to the LLM once, asking it to write the
text first and then send the card. Each retry incurs a second billing for the same prompt.
(Optional)
  "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are kept as head+tail (with the identifiers found in the cut part) when
written to conversation history; knowledge base, SQL and conversation search results keep their
100-character cut either way. Reduces memory bloat from large tool outputs (code interpreter
bash stdout, web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
```

```
archived messages are discarded and only the active chat history window is used for multi-turn context. (Optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes retrieved per long-term memory lookup (similarity_top_k of the vector search); defaults to 10, range 1–50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links for produced content. AGENT mode only. (Optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "retiresAt": "Response string",
    "replacement": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
```

```
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "updatedAt": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response string",
  "unanswerableTemplate": "Response string",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "displayName": "Response string",
      "toolType": "Response string"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 456,
  "defaultLabelRelations": null,
  "useFullRetrievalRange": false,
  "enableEvaluation": false,
  "enableInlineCitations": false,
  "enableCodeInterpreter": false,
  "enableBrowserTool": false,
  "enableImageSearch": false,
  "enableClaudeStreamRetry": false,
  "customMaxLlmOutputTokens": 456,
  "voiceConfig": {
    "enabled": false,
```

```
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Response string",
    "turnHandling": null,
    "enableVoiceReply": false
  },
  "thinkingConfig": null,
  "enableToolSearching": false,
  "maxAgentIterations": 456,
  "agentTimeoutSeconds": 456,
  "agentTimeoutMessage": "Response string",
  "requireTextWithFlexCards": false,
  "isMultimodalLlm": false,
  "enableSizeBasedToolResultTruncation": false,
  "toolResultMaxChars": 456,
  "enableVectorMemory": false,
  "longTermMemoryTopK": 456,
  "enableCrossConversationSearch": false,
  "crossConversationLookbackDays": 456,
  "useGatewayFallback": false,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": false,
  "enableGetMessageContextTool": false,
  "enableReadAttachmentFullTextTool": false,
  "enableParseAttachmentContentTool": false,
  "enableListAvailableParsersTool": false,
  "enableGetDownloadLinkTool": false,
  "isLocked": false,
  "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
  "canUpdate": false,
  "canDelete": false
}
```


Update an AI Assistant

PUT `/api/v1/chatbots/{id}`

Parameters

Parameter	Required	Type	Description

`id`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
largeLanguageModel	string (uuid)	Yes	Large language model used by the chatbot to generate responses (including custom AI Gateway models created by the organization)
embeddingModel	string (uuid)	No	Embedding model used to vectorize text; optional
rerankerModel	string (uuid)	No	Model used for reranking; optional
instructions	string	No	Role instructions describing the chatbot's role and behavior.
knowledgeBases	array[IdName]	No	List of knowledge bases the chatbot can access
databases	array[IdName]	No	List of SQL databases the chatbot can access (for Text-to-SQL)
organization	string (uuid)	No	Organization the chatbot belongs to; if empty, it is a personal chatbot
replyMode	object	No	Reply mode: normal or streaming responses `normal` : Normal ; `template` : Template ; `hybrid` : Hybrid ; `agent` : Agent;
template	string	No	Template used by Template and Hybrid modes.
unanswerableTemplate	string	No	Template used by Template and Hybrid modes when the question cannot be answered.
totalWordsCount	integer (int64)	No	Accumulated total word count used.
outputMode	object	No	Output mode: text, table, or custom format `text` : Text ; `json_schema` : JSON Schema;
rawOutputFormat	object	No	JSON structure definition for the custom output format
groups	array[IdName]	No	List of groups the chatbot can access

Field	Type	Required	Description
tools	array[ToolSummary]	No	List of tools the chatbot can use
skills	array[SkillSummary]	No	List of skills associated with the chatbot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode `normal` : Normal ; `canvas` : Canvas ; `team` : Team;
numberOfRetrievedChunks	integer	No	Number of reference chunks to retrieve; default: 12, minimum: 1
defaultLabelRelations	object	No	Default knowledge base label filters at the assistant level, using the same format as query_metadata.label_relations; null means no filtering. Applied only as the lowest-priority fallback when the caller does not provide query_metadata.
useFullRetrievalRange	boolean	No	When enabled, ignores LLM top_k truncation and returns the complete reranker results
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enables inline citations, inserting citation markers such as [1][2] into responses.
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to run code in an isolated container.
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial outpu...
customMaxLlmOutputTokens	integer	No	Custom maximum number of LLM output tokens; minimum: 512
voiceConfig	object	No	
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.

Field	Type	Required	Description
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type.
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider.
voiceConfig.sttConfig	object	No	STT configuration parameters (JSON).
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider.
voiceConfig.ttsConfig	object	No	TTS configuration parameters (JSON).
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider.
voiceConfig.realtimeConfig	object	No	Realtime configuration parameters (JSON).
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty
voiceConfig.turnHandling	object	No	Turn-taking and interruption settings for the voice agent.
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Reasoning level: null (model default) or {"reasoning_effort": "off
enableToolSearching	boolean	No	Enables dynamic tool searching, allowing the agent to discover and load tools dynamically through tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
requireTextWithFlexCards	boolean	No	When enabled, a turn that would end with only Flex Message cards and no text for the user is re-issued to the LLM once, asking it to write the text first and then send the card. Each retry incurs a se...
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversati...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.

Field	Type	Required	Description
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat hist...
longTermMemoryTopK	integer	No	Number of long-term memory nodes returned per retrieval (similarity_top_k for vector retrieval); default: 10, range: 1–50
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search...
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Example Request Schema

```
{
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate responses (including custom AI Gateway models created by the organization)
```

```
  "embeddingModel"?: string (uuid) // Embedding model used to vectorize text; optional
(Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking; optional (Optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior.
(Optional)
  "knowledgeBases"?: [ // List of knowledge bases the chatbot can access (Optional)
    {
      "id": string (uuid)
    }
  ]
  "databases"?: [ // List of SQL databases the chatbot can access (for Text-to-SQL) (Optional)
    {
      "id": string (uuid)
    }
  ]
  "organization"?: string (uuid) // Organization the chatbot belongs to; if empty, it is a
personal chatbot (Optional)
  "replyMode"?:  // Reply mode: normal or streaming responses

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON structure definition for the custom output format
(Optional)
  "groups"?: [ // List of groups the chatbot can access (Optional)
    {
      "id": string (uuid)
    }
  ]
  "tools"?: [ // List of tools the chatbot can use (Optional)
    {
      "id": string (uuid)
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
    }
```

```
  ]
  "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to retrieve; default: 12,
minimum: 1 (Optional)
  "defaultLabelRelations"?: object // Default knowledge base label filters at the assistant
level, using the same format as query_metadata.label_relations; null means no filtering.
Applied only as the lowest-priority fallback when the caller does not provide query_metadata.
(Optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores LLM top_k truncation and returns
the complete reranker results (Optional)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum:
512 (Optional)
  "voiceConfig"?: { // Optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
  }
  }
  "thinkingConfig"?: object // Reasoning level: null (model default) or {"reasoning_effort":
"off|minimal|low|medium|high|xhigh|max"} (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool searching, allowing the agent to
discover and load tools dynamically through tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
```

```
default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
  "requireTextWithFlexCards"?: boolean // When enabled, a turn that would end with only Flex
Message cards and no text for the user is re-issued to the LLM once, asking it to write the
text first and then send the card. Each retry incurs a second billing for the same prompt.
(Optional)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are kept as head+tail (with the identifiers found in the cut part) when
written to conversation history; knowledge base, SQL and conversation search results keep their
100-character cut either way. Reduces memory bloat from large tool outputs (code interpreter
bash stdout, web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
  "longTermMemoryTopK"?: integer // Number of long-term memory nodes returned per retrieval
(similarity_top_k for vector retrieval); default: 10, range: 1–50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
}
```

Example Request Values

```
{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
```

```
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "defaultLabelRelations": null,
  "useFullRetrievalRange": true,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
  "enableBrowserTool": true,
  "enableImageSearch": true,
  "enableClaudeStreamRetry": true,
  "customMaxLlmOutputTokens": 123,
  "voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
  },
```

```
  "thinkingConfig": null,
  "enableToolSearching": true,
  "maxAgentIterations": 123,
  "agentTimeoutSeconds": 123,
  "agentTimeoutMessage": "Example String",
  "requireTextWithFlexCards": true,
  "enableSizeBasedToolResultTruncation": true,
  "toolResultMaxChars": 123,
  "enableVectorMemory": true,
  "longTermMemoryTopK": 123,
  "enableCrossConversationSearch": true,
  "crossConversationLookbackDays": 123,
  "useGatewayFallback": true,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": true,
  "enableGetMessageContextTool": true,
  "enableReadAttachmentFullTextTool": true,
  "enableParseAttachmentContentTool": true,
  "enableListAvailableParsersTool": true,
  "enableGetDownloadLinkTool": true
}
```


Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X PUT "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "databases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "tools": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "skills": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "connectors": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
```

```
      "numberOfRetrievedChunks": 123,
      "defaultLabelRelations": null,
      "useFullRetrievalRange": true,
      "enableEvaluation": true,
      "enableInlineCitations": true,
      "enableCodeInterpreter": true,
      "enableBrowserTool": true,
      "enableImageSearch": true,
      "enableClaudeStreamRetry": true,
      "customMaxLlmOutputTokens": 123,
      "voiceConfig": {
        "enabled": true,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": null,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": null,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": null,
        "realtimeInstructions": "Example String",
        "turnHandling": null,
        "enableVoiceReply": true
      },
      "thinkingConfig": null,
      "enableToolSearching": true,
      "maxAgentIterations": 123,
      "agentTimeoutSeconds": 123,
      "agentTimeoutMessage": "Example String",
      "requireTextWithFlexCards": true,
      "enableSizeBasedToolResultTruncation": true,
      "toolResultMaxChars": 123,
      "enableVectorMemory": true,
      "longTermMemoryTopK": 123,
      "enableCrossConversationSearch": true,
      "crossConversationLookbackDays": 123,
      "useGatewayFallback": true,
      "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
      "enableSearchMessagesTool": true,
      "enableGetMessageContextTool": true,
      "enableReadAttachmentFullTextTool": true,
      "enableParseAttachmentContentTool": true,
      "enableListAvailableParsersTool": true,
      "enableGetDownloadLinkTool": true
    }'

  # Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example string",
    "knowledgeBases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "databases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example string",
    "unanswerableTemplate": "Example string",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "tools": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "skills": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "connectors": [
      {
```

```
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": null,
    "useFullRetrievalRange": true,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
      "enabled": true,
      "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
      "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
      "sttConfig": null,
      "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
      "ttsConfig": null,
      "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
      "realtimeConfig": null,
      "realtimeInstructions": "Example string",
      "turnHandling": null,
      "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example string",
    "requireTextWithFlexCards": true,
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": true,
    "enableGetMessageContextTool": true,
    "enableReadAttachmentFullTextTool": true,
    "enableParseAttachmentContentTool": true,
    "enableListAvailableParsersTool": true,
    "enableGetDownloadLinkTool": true
  };

axios.put(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
```

```
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example string",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example string",
    "unanswerableTemplate": "Example string",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": None,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
```

```
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": None,
    "useFullRetrievalRange": True,
    "enableEvaluation": True,
    "enableInlineCitations": True,
    "enableCodeInterpreter": True,
    "enableBrowserTool": True,
    "enableImageSearch": True,
    "enableClaudeStreamRetry": True,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
        "enabled": True,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": None,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": None,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": None,
        "realtimeInstructions": "Example string",
        "turnHandling": None,
        "enableVoiceReply": True
    },
    "thinkingConfig": None,
    "enableToolSearching": True,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example string",
    "requireTextWithFlexCards": True,
    "enableSizeBasedToolResultTruncation": True,
    "toolResultMaxChars": 123,
    "enableVectorMemory": True,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": True,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": True,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": True,
    "enableGetMessageContextTool": True,
    "enableReadAttachmentFullTextTool": True,
    "enableParseAttachmentContentTool": True,
    "enableListAvailableParsersTool": True,
    "enableGetDownloadLinkTool": True
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example Name',
            'largeLanguageModel' => '550e8400-e29b-41d4-a716-446655440000',
            'embeddingModel' => '550e8400-e29b-41d4-a716-446655440000',
            'rerankerModel' => '550e8400-e29b-41d4-a716-446655440000',
            'instructions' => 'Example string',
            'knowledgeBases' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'databases' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'organization' => '550e8400-e29b-41d4-a716-446655440000',
            'replyMode' => new \stdClass(),
            'template' => 'Example string',
            'unanswerableTemplate' => 'Example string',
            'totalWordsCount' => 123,
            'outputMode' => new \stdClass(),
            'rawOutputFormat' => null,
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'tools' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'skills' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'connectors' => [
                [
```

```
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'agentMode' => new \stdClass(),
            'numberOfRetrievedChunks' => 123,
            'defaultLabelRelations' => null,
            'useFullRetrievalRange' => true,
            'enableEvaluation' => true,
            'enableInlineCitations' => true,
            'enableCodeInterpreter' => true,
            'enableBrowserTool' => true,
            'enableImageSearch' => true,
            'enableClaudeStreamRetry' => true,
            'customMaxLlmOutputTokens' => 123,
            'voiceConfig' => [
                'enabled' => true,
                'voiceAgentType' => '550e8400-e29b-41d4-a716-446655440000',
                'sttProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'sttConfig' => null,
                'ttsProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'ttsConfig' => null,
                'realtimeProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'realtimeConfig' => null,
                'realtimeInstructions' => 'Example string',
                'turnHandling' => null,
                'enableVoiceReply' => true
            ],
            'thinkingConfig' => null,
            'enableToolSearching' => true,
            'maxAgentIterations' => 123,
            'agentTimeoutSeconds' => 123,
            'agentTimeoutMessage' => 'Example string',
            'requireTextWithFlexCards' => true,
            'enableSizeBasedToolResultTruncation' => true,
            'toolResultMaxChars' => 123,
            'enableVectorMemory' => true,
            'longTermMemoryTopK' => 123,
            'enableCrossConversationSearch' => true,
            'crossConversationLookbackDays' => 123,
            'useGatewayFallback' => true,
            'gatewayFallbackGroup' => '550e8400-e29b-41d4-a716-446655440000',
            'enableSearchMessagesTool' => true,
            'enableGetMessageContextTool' => true,
            'enableReadAttachmentFullTextTool' => true,
            'enableParseAttachmentContentTool' => true,
            'enableListAvailableParsersTool' => true,
            'enableGetDownloadLinkTool' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
```

```
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate answers (including organization-owned AI Gateway models)
  "llm":  // ID, name, and retirement info of the bound model, read directly from the FK; returned even if the model has been disabled by the organization
  {
    "id": string (uuid)
    "name": string
    "retiresAt": string (date) // Provider retirement date; after it passes, a daily job switches the assistant to the replacement and disables this model. null means no retirement is scheduled
    "replacement":  // Model that takes over after retirement (id and name); null means none is specified and nothing is migrated automatically on expiry
    {
      "id": string (uuid)
      "name": string
    }
  }
  "embeddingModel"?: string (uuid) // Embedding model used to vectorize text; optional (Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking; optional (Optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior. (Optional)
  "knowledgeBases"?: [ // List of knowledge bases the chatbot can access (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"?: [ // List of SQL databases the chatbot can access (used for the Text-to-SQL feature) (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
```

```
  "updatedAt": string (timestamp)
  "organization"?: string (uuid) // Organization the chatbot belongs to; if empty, it is a
personal chatbot (Optional)
  "replyMode"?:  // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON structure definition for the custom output format
(Optional)
  "groups"?: [ // List of groups the chatbot can access (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "tools"?: [ // List of tools the chatbot can use (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
```

```
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks retrieved; default is 12, minimum is 1 (Optional)
  "defaultLabelRelations"?: object // Assistant-level default knowledge base label filter, in the same format as query_metadata.label_relations; null means no filtering. Applied only as the lowest-priority fallback when the caller does not provide query_metadata. (Optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores the LLM's top_k truncation and returns the full reranker results (Optional)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers such as [1][2] into responses. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial output already rendered. Each retry incurs a second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum LLM output tokens; minimum is 512 (Optional)
  "voiceConfig"?: { // Optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent. (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and does not speak (Optional)
  }
  }
  "thinkingConfig"?: object // Thinking level: null (model default) or {"reasoning_effort": "off|minimal|low|medium|high|xhigh|max"} (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to discover and load tools dynamically via tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100, default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200, default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default. (Optional)
  "requireTextWithFlexCards"?: boolean // When enabled, a turn that would end with only Flex Message cards and no text for the user is re-issued to the LLM once, asking it to write the text first and then send the card. Each retry incurs a second billing for the same prompt. (Optional)
```

```
  "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversation search results keep their 100-character cut either way. Reduces memory bloat from large tool outputs (code interpreter bash stdout, web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat history window is used for multi-turn context. (Optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes retrieved per long-term memory lookup (similarity_top_k of the vector search); default is 10, range 1–50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links for produced content. AGENT mode only. (Optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}
```

Response Example Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "retiresAt": "Response string",
    "replacement": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "name": "Response string"
    }
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "updatedAt": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response string",
  "unanswerableTemplate": "Response string",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "displayName": "Response string",
      "toolType": "Response string"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 456,
  "defaultLabelRelations": null,
```

```
  "useFullRetrievalRange": false,
  "enableEvaluation": false,
  "enableInlineCitations": false,
  "enableCodeInterpreter": false,
  "enableBrowserTool": false,
  "enableImageSearch": false,
  "enableClaudeStreamRetry": false,
  "customMaxLlmOutputTokens": 456,
  "voiceConfig": {
    "enabled": false,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Response string",
    "turnHandling": null,
    "enableVoiceReply": false
  },
  "thinkingConfig": null,
  "enableToolSearching": false,
  "maxAgentIterations": 456,
  "agentTimeoutSeconds": 456,
  "agentTimeoutMessage": "Response string",
  "requireTextWithFlexCards": false,
  "isMultimodalLlm": false,
  "enableSizeBasedToolResultTruncation": false,
  "toolResultMaxChars": 456,
  "enableVectorMemory": false,
  "longTermMemoryTopK": 456,
  "enableCrossConversationSearch": false,
  "crossConversationLookbackDays": 456,
  "useGatewayFallback": false,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": false,
  "enableGetMessageContextTool": false,
  "enableReadAttachmentFullTextTool": false,
  "enableParseAttachmentContentTool": false,
  "enableListAvailableParsersTool": false,
  "enableGetDownloadLinkTool": false,
  "isLocked": false,
  "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
  "canUpdate": false,
  "canDelete": false
}
```


Partially Update an AI Assistant

PATCH `/api/v1/chatbots/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
largeLanguageModel	string (uuid)	No	Large language model used by the chatbot to generate responses (including custom AI Gateway models created by the organization)
embeddingModel	string (uuid)	No	Embedding model used to vectorize text; optional
rerankerModel	string (uuid)	No	Model used for reranking; optional
instructions	string	No	Role instructions describing the chatbot's role and behavior.
knowledgeBases	array[IdName]	No	List of knowledge bases the chatbot can access
databases	array[IdName]	No	List of SQL databases the chatbot can access (for Text-to-SQL)
organization	string (uuid)	No	Organization the chatbot belongs to; if empty, it is a personal chatbot
replyMode	object	No	Reply mode: normal or streaming responses `normal` : Normal ; `template` : Template ; `hybrid` : Hybrid ; `agent` : Agent;
template	string	No	Template used by Template and Hybrid modes.
unanswerableTemplate	string	No	Template used by Template and Hybrid modes when the question cannot be answered.

Field	Type	Required	Description
totalWordsCount	integer (int64)	No	Accumulated total word count used.
outputMode	object	No	Output mode: text, table, or custom format `text` : Text ; `json_schema` : JSON Schema;
rawOutputFormat	object	No	JSON structure definition for the custom output format
groups	array[IdName]	No	List of groups the chatbot can access
tools	array[ToolSummary]	No	List of tools the chatbot can use
skills	array[SkillSummary]	No	List of skills associated with the chatbot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode `normal` : Normal ; `canvas` : Canvas ; `team` : Team;
numberOfRetrievedChunks	integer	No	Number of reference chunks to retrieve; default: 12, minimum: 1
defaultLabelRelations	object	No	Default knowledge base label filters at the assistant level, using the same format as query_metadata.label_relations; null means no filtering. Applied only as the lowest-priority fallback when the caller does not provide query_metadata.
useFullRetrievalRange	boolean	No	When enabled, ignores LLM top_k truncation and returns the complete reranker results
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enables inline citations, inserting citation markers such as [1][2] into responses.
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to run code in an isolated container.
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial outpu...

Field	Type	Required	Description
customMaxLlmOutputTokens	integer	No	Custom maximum number of LLM output tokens; minimum: 512
voiceConfig	object	No	
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type.
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider.
voiceConfig.sttConfig	object	No	STT configuration parameters (JSON).
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider.
voiceConfig.ttsConfig	object	No	TTS configuration parameters (JSON).
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider.
voiceConfig.realtimeConfig	object	No	Realtime configuration parameters (JSON).
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty
voiceConfig.turnHandling	object	No	Turn-taking and interruption settings for the voice agent.
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Reasoning level: null (model default) or {"reasoning_effort": "off
enableToolSearching	boolean	No	Enables dynamic tool searching, allowing the agent to discover and load tools dynamically through tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
requireTextWithFlexCards	boolean	No	When enabled, a turn that would end with only Flex Message cards and no text for the user is re-issued to the LLM once, asking it to write the text first and then send the card. Each retry incurs a se...

Field	Type	Required	Description
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversati...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat hist...
longTermMemoryTopK	integer	No	Number of long-term memory nodes returned per retrieval (similarity_top_k for vector retrieval); default: 10, range: 1–50
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search...
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.

Field	Type	Required	Description
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Example Request Schema

```
{
  "name"?: string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots. (Optional)
  "largeLanguageModel"?: string (uuid) // Large language model used by the chatbot to generate responses (including custom AI Gateway models created by the organization) (Optional)
  "embeddingModel"?: string (uuid) // Embedding model used to vectorize text; optional (Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking; optional (Optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior. (Optional)
  "knowledgeBases"?: [ // List of knowledge bases the chatbot can access (Optional)
    {
      "id": string (uuid)
    }
  ]
  "databases"?: [ // List of SQL databases the chatbot can access (for Text-to-SQL) (Optional)
    {
      "id": string (uuid)
    }
  ]
  "organization"?: string (uuid) // Organization the chatbot belongs to; if empty, it is a personal chatbot (Optional)
  "replyMode"?:  // Reply mode: normal or streaming responses (Optional)

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?:  // Output mode: text, table, or custom format (Optional)

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON structure definition for the custom output format (Optional)
  "groups"?: [ // List of groups the chatbot can access (Optional)
    {
      "id": string (uuid)
    }
  ]
```

```
  "tools"?: [ // List of tools the chatbot can use (Optional)
    {
      "id": string (uuid)
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
    }
  ]
  "agentMode"?:  // Agent mode: normal, SQL, or workflow mode (Optional)

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to retrieve; default: 12,
minimum: 1 (Optional)
  "defaultLabelRelations"?: object // Default knowledge base label filters at the assistant
level, using the same format as query_metadata.label_relations; null means no filtering.
Applied only as the lowest-priority fallback when the caller does not provide query_metadata.
(Optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores LLM top_k truncation and returns
the complete reranker results (Optional)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum:
512 (Optional)
  "voiceConfig"?: { // Optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
```

```
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent. (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and does not speak (Optional)
  }
  }
  "thinkingConfig"?: object // Reasoning level: null (model default) or {"reasoning_effort": "off|minimal|low|medium|high|xhigh|max"} (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool searching, allowing the agent to discover and load tools dynamically through tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100, default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200, default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default. (Optional)
  "requireTextWithFlexCards"?: boolean // When enabled, a turn that would end with only Flex Message cards and no text for the user is re-issued to the LLM once, asking it to write the text first and then send the card. Each retry incurs a second billing for the same prompt. (Optional)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversation search results keep their 100-character cut either way. Reduces memory bloat from large tool outputs (code interpreter bash stdout, web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat history window is used for multi-turn context. (Optional)
  "longTermMemoryTopK"?: integer // Number of long-term memory nodes returned per retrieval (similarity_top_k for vector retrieval); default: 10, range: 1–50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
```

```
for produced content. AGENT mode only. (Optional)
}
```

Request Example Values


```
{
  "name": "Example name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example string",
  "unanswerableTemplate": "Example string",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "defaultLabelRelations": null,
  "useFullRetrievalRange": true,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
```

```
  "enableBrowserTool": true,
  "enableImageSearch": true,
  "enableClaudeStreamRetry": true,
  "customMaxLlmOutputTokens": 123,
  "voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example string",
    "turnHandling": null,
    "enableVoiceReply": true
  },
  "thinkingConfig": null,
  "enableToolSearching": true,
  "maxAgentIterations": 123,
  "agentTimeoutSeconds": 123,
  "agentTimeoutMessage": "Example string",
  "requireTextWithFlexCards": true,
  "enableSizeBasedToolResultTruncation": true,
  "toolResultMaxChars": 123,
  "enableVectorMemory": true,
  "longTermMemoryTopK": 123,
  "enableCrossConversationSearch": true,
  "crossConversationLookbackDays": 123,
  "useGatewayFallback": true,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": true,
  "enableGetMessageContextTool": true,
  "enableReadAttachmentFullTextTool": true,
  "enableParseAttachmentContentTool": true,
  "enableListAvailableParsersTool": true,
  "enableGetDownloadLinkTool": true
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "databases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "tools": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "skills": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "connectors": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
```

```
      "numberOfRetrievedChunks": 123,
      "defaultLabelRelations": null,
      "useFullRetrievalRange": true,
      "enableEvaluation": true,
      "enableInlineCitations": true,
      "enableCodeInterpreter": true,
      "enableBrowserTool": true,
      "enableImageSearch": true,
      "enableClaudeStreamRetry": true,
      "customMaxLlmOutputTokens": 123,
      "voiceConfig": {
        "enabled": true,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": null,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": null,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": null,
        "realtimeInstructions": "Example String",
        "turnHandling": null,
        "enableVoiceReply": true
      },
      "thinkingConfig": null,
      "enableToolSearching": true,
      "maxAgentIterations": 123,
      "agentTimeoutSeconds": 123,
      "agentTimeoutMessage": "Example String",
      "requireTextWithFlexCards": true,
      "enableSizeBasedToolResultTruncation": true,
      "toolResultMaxChars": 123,
      "enableVectorMemory": true,
      "longTermMemoryTopK": 123,
      "enableCrossConversationSearch": true,
      "crossConversationLookbackDays": 123,
      "useGatewayFallback": true,
      "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
      "enableSearchMessagesTool": true,
      "enableGetMessageContextTool": true,
      "enableReadAttachmentFullTextTool": true,
      "enableParseAttachmentContentTool": true,
      "enableListAvailableParsersTool": true,
      "enableGetDownloadLinkTool": true
    }'

  # Replace YOUR_API_KEY and review the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "databases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "tools": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "skills": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "connectors": [
      {
```

```
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": null,
    "useFullRetrievalRange": true,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
      "enabled": true,
      "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
      "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
      "sttConfig": null,
      "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
      "ttsConfig": null,
      "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
      "realtimeConfig": null,
      "realtimeInstructions": "Example String",
      "turnHandling": null,
      "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example String",
    "requireTextWithFlexCards": true,
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": true,
    "enableGetMessageContextTool": true,
    "enableReadAttachmentFullTextTool": true,
    "enableParseAttachmentContentTool": true,
    "enableListAvailableParsersTool": true,
    "enableGetDownloadLinkTool": true
  };

axios.patch(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
```

```
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": None,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
```

```
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "defaultLabelRelations": None,
    "useFullRetrievalRange": True,
    "enableEvaluation": True,
    "enableInlineCitations": True,
    "enableCodeInterpreter": True,
    "enableBrowserTool": True,
    "enableImageSearch": True,
    "enableClaudeStreamRetry": True,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
        "enabled": True,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": None,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": None,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": None,
        "realtimeInstructions": "Example String",
        "turnHandling": None,
        "enableVoiceReply": True
    },
    "thinkingConfig": None,
    "enableToolSearching": True,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example String",
    "requireTextWithFlexCards": True,
    "enableSizeBasedToolResultTruncation": True,
    "toolResultMaxChars": 123,
    "enableVectorMemory": True,
    "longTermMemoryTopK": 123,
    "enableCrossConversationSearch": True,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": True,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
    "enableSearchMessagesTool": True,
    "enableGetMessageContextTool": True,
    "enableReadAttachmentFullTextTool": True,
    "enableParseAttachmentContentTool": True,
    "enableListAvailableParsersTool": True,
    "enableGetDownloadLinkTool": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example Name',
            'largeLanguageModel' => '550e8400-e29b-41d4-a716-446655440000',
            'embeddingModel' => '550e8400-e29b-41d4-a716-446655440000',
            'rerankerModel' => '550e8400-e29b-41d4-a716-446655440000',
            'instructions' => 'Example String',
            'knowledgeBases' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'databases' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'organization' => '550e8400-e29b-41d4-a716-446655440000',
            'replyMode' => new \stdClass(),
            'template' => 'Example String',
            'unanswerableTemplate' => 'Example String',
            'totalWordsCount' => 123,
            'outputMode' => new \stdClass(),
            'rawOutputFormat' => null,
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'tools' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'skills' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'connectors' => [
                [
```

```
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'agentMode' => new \stdClass(),
            'numberOfRetrievedChunks' => 123,
            'defaultLabelRelations' => null,
            'useFullRetrievalRange' => true,
            'enableEvaluation' => true,
            'enableInlineCitations' => true,
            'enableCodeInterpreter' => true,
            'enableBrowserTool' => true,
            'enableImageSearch' => true,
            'enableClaudeStreamRetry' => true,
            'customMaxLlmOutputTokens' => 123,
            'voiceConfig' => [
                'enabled' => true,
                'voiceAgentType' => '550e8400-e29b-41d4-a716-446655440000',
                'sttProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'sttConfig' => null,
                'ttsProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'ttsConfig' => null,
                'realtimeProvider' => '550e8400-e29b-41d4-a716-446655440000',
                'realtimeConfig' => null,
                'realtimeInstructions' => 'Example String',
                'turnHandling' => null,
                'enableVoiceReply' => true
            ],
            'thinkingConfig' => null,
            'enableToolSearching' => true,
            'maxAgentIterations' => 123,
            'agentTimeoutSeconds' => 123,
            'agentTimeoutMessage' => 'Example String',
            'requireTextWithFlexCards' => true,
            'enableSizeBasedToolResultTruncation' => true,
            'toolResultMaxChars' => 123,
            'enableVectorMemory' => true,
            'longTermMemoryTopK' => 123,
            'enableCrossConversationSearch' => true,
            'crossConversationLookbackDays' => 123,
            'useGatewayFallback' => true,
            'gatewayFallbackGroup' => '550e8400-e29b-41d4-a716-446655440000',
            'enableSearchMessagesTool' => true,
            'enableGetMessageContextTool' => true,
            'enableReadAttachmentFullTextTool' => true,
            'enableParseAttachmentContentTool' => true,
            'enableListAvailableParsersTool' => true,
            'enableGetDownloadLinkTool' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
```

```
}
?>
```

Response

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // The large language model used by the chatbot to generate answers (including organization-built AI Gateway models)
  "llm":  // The id, name, and retirement information of the bound model, read directly from the FK; returned even if the organization has disabled the model
  {
    "id": string (uuid)
    "name": string
    "retiresAt": string (date) // Provider retirement date; after it passes, a daily job switches the assistant to the replacement and disables this model. null means no retirement is scheduled
    "replacement":  // The model that takes over after retirement (id and name); null means none is specified, and no automatic migration happens at expiry
    {
      "id": string (uuid)
      "name": string
    }
  }
  "embeddingModel"?: string (uuid) // The embedding model used to vectorize text; optional (optional)
  "rerankerModel"?: string (uuid) // The model used for reranking; optional (optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior. (optional)
  "knowledgeBases"?: [ // List of knowledge bases the chatbot can access (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"?: [ // List of SQL databases the chatbot can access (used for the Text-to-SQL feature) (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
```

```
  "updatedAt": string (timestamp)
  "organization"?: string (uuid) // The organization the chatbot belongs to; if empty, it is a
personal chatbot (optional)
  "replyMode"?:  // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (optional)
  "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (optional)
  {
  }
  "rawOutputFormat"?: object // JSON structure definition for the custom output format
(optional)
  "groups"?: [ // List of groups the chatbot can access (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "tools"?: [ // List of tools the chatbot can use (optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "skills"?: [ // List of skills associated with the chatbot (optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (optional)
```

```
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to retrieve; defaults to
12, minimum 1 (optional)
  "defaultLabelRelations"?: object // Assistant-level default knowledge base label filter, in
the same format as query_metadata.label_relations; null means no filtering. Applied as the
lowest-priority fallback only when the caller does not pass query_metadata. (optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores the LLM's top_k truncation and
returns the full reranker results (optional)
  "enableEvaluation"?: boolean // optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum
512 (optional)
  "voiceConfig"?: { // optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent.
(optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (optional)
  }
  }
  "thinkingConfig"?: object // Thinking level: null (model default) or {"reasoning_effort":
"off|minimal|low|medium|high|xhigh|max"} (optional)
  "enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to
discover and load tools dynamically through tool_searching_tool (optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (optional)
  "requireTextWithFlexCards"?: boolean // When enabled, a turn that would end with only Flex
Message cards and no text for the user is re-issued to the LLM once, asking it to write the
text first and then send the card. Each retry incurs a second billing for the same prompt.
(optional)
```

```
  "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversation search results keep their 100-character cut either way. Reduces memory bloat from large tool outputs (code interpreter bash stdout, web search results, etc.). (optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat history window is used for multi-turn context. (optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes returned per long-term memory retrieval (similarity_top_k of the vector search); defaults to 10, range 1–50 (optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (optional)
  "gatewayFallbackGroup"?: string (uuid) // optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links for produced content. AGENT mode only. (optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "retiresAt": "Response String",
    "replacement": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "name": "Response String"
    }
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "updatedAt": "Response String",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response String",
  "unanswerableTemplate": "Response String",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String",
      "displayName": "Response String",
      "toolType": "Response String"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 456,
  "defaultLabelRelations": null,
```

```
  "useFullRetrievalRange": false,
  "enableEvaluation": false,
  "enableInlineCitations": false,
  "enableCodeInterpreter": false,
  "enableBrowserTool": false,
  "enableImageSearch": false,
  "enableClaudeStreamRetry": false,
  "customMaxLlmOutputTokens": 456,
  "voiceConfig": {
    "enabled": false,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Response String",
    "turnHandling": null,
    "enableVoiceReply": false
  },
  "thinkingConfig": null,
  "enableToolSearching": false,
  "maxAgentIterations": 456,
  "agentTimeoutSeconds": 456,
  "agentTimeoutMessage": "Response String",
  "requireTextWithFlexCards": false,
  "isMultimodalLlm": false,
  "enableSizeBasedToolResultTruncation": false,
  "toolResultMaxChars": 456,
  "enableVectorMemory": false,
  "longTermMemoryTopK": 456,
  "enableCrossConversationSearch": false,
  "crossConversationLookbackDays": 456,
  "useGatewayFallback": false,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": false,
  "enableGetMessageContextTool": false,
  "enableReadAttachmentFullTextTool": false,
  "enableParseAttachmentContentTool": false,
  "enableListAvailableParsersTool": false,
  "enableGetDownloadLinkTool": false,
  "isLocked": false,
  "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
  "canUpdate": false,
  "canDelete": false
}
```


Delete AI Assistant

DELETE `/api/v1/chatbots/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Duplicate AI Assistant

POST `/api/v1/chatbots/{id}/duplicate/`

Parameters

Parameter Name	Required	Type	Description

id	V	string

Request Content

Request Parameters

Field	Type	Required	Description
name	string	Yes	

Example Request Structure

```
{
  "name": string
}
```

Example Request Values

```
{
  "name": "Example name"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name"
  }'

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "name": "Example name"
  };

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "name": "Example name"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 201

Example Response Structure

```
{
  "id": string (uuid)
  "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it
only distinguishes chatbots.
  "largeLanguageModel": string (uuid) // Large language model the bot uses to generate answers
(including models from the organization's own AI Gateway)
  "llm":  // ID, name, and retirement information of the bound model, read directly from the
FK; returned even if the organization has disabled the model
  {
    "id": string (uuid)
    "name": string
    "retiresAt": string (date) // Provider retirement date; after it passes, a daily job
switches assistants to the replacement and disables this model. null means no retirement is
```

```
scheduled
    "replacement":  // Model that takes over after retirement (id and name); null means none is
specified, so nothing is migrated automatically at expiry
    {
      "id": string (uuid)
      "name": string
    }
  }
  "embeddingModel"?: string (uuid) // Embedding model used to vectorize text; optional
(Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking; optional (Optional)
  "instructions"?: string // Role instructions describing the chatbot's role and behavior.
(Optional)
  "knowledgeBases"?: [ // List of knowledge bases the bot can access (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"?: [ // List of SQL databases the bot can access (used for the Text-to-SQL
feature) (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization"?: string (uuid) // Organization the bot belongs to; if empty, it is a personal
bot (Optional)
  "replyMode"?:  // Reply mode: standard reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used by Template and Hybrid modes. (Optional)
  "unanswerableTemplate"?: string // Template used by Template and Hybrid modes when the
question cannot be answered. (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count used. (Optional)
  "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON structure definition of the custom output format
(Optional)
  "groups"?: [ // List of groups the bot can access (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "tools"?: [ // List of tools the bot can use (Optional)
    {
```

```
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "skills"?: [ // List of skills associated with the bot (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
  {
  }
  "numberOfRetrievedChunks"?: integer // Number of reference chunks retrieved; default is 12,
minimum is 1 (Optional)
  "defaultLabelRelations"?: object // Assistant-level default knowledge base label filter, in
the same format as query_metadata.label_relations; null means no filtering. Applied as the
lowest-priority fallback only when the caller does not send query_metadata. (Optional)
  "useFullRetrievalRange"?: boolean // When enabled, ignores the LLM's top_k truncation and
returns the complete reranker results (Optional)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enables inline citations, inserting citation markers
such as [1][2] into responses. (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to run
code in an isolated container. (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum number of LLM output tokens; minimum
is 512 (Optional)
  "voiceConfig"?: { // Optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type. (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider. (Optional)
    "sttConfig"?: object // STT configuration parameters (JSON). (Optional)
```

```
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider. (Optional)
    "ttsConfig"?: object // TTS configuration parameters (JSON). (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider. (Optional)
    "realtimeConfig"?: object // Realtime configuration parameters (JSON). (Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Turn-taking and interruption settings for the voice agent. (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and does not speak (Optional)
  }
  }
  "thinkingConfig"?: object // Thinking level: null (model default) or {"reasoning_effort": "off|minimal|low|medium|high|xhigh|max"} (Optional)
  "enableToolSearching"?: boolean // Enables dynamic tool search, allowing the agent to discover and load tools dynamically through tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100, default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200, default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default. (Optional)
  "requireTextWithFlexCards"?: boolean // When enabled, a turn that would end with only Flex Message cards and no text for the user is re-issued to the LLM once, asking it to write the text first and then send the card. Each retry incurs a second billing for the same prompt. (Optional)
  "isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the character threshold are kept as head+tail (with the identifiers found in the cut part) when written to conversation history; knowledge base, SQL and conversation search results keep their 100-character cut either way. Reduces memory bloat from large tool outputs (code interpreter bash stdout, web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat history window is used for multi-turn context. (Optional)
  "longTermMemoryTopK"?: integer // Number of memory nodes retrieved per long-term memory lookup (similarity_top_k of the vector search); default is 10, range 1–50 (Optional)
  "enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (Optional)
  "crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (Optional)
  "useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (Optional)
  "gatewayFallbackGroup"?: string (uuid) // Optional
  "enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (Optional)
  "enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (Optional)
  "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (Optional)
```

```
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "retiresAt": "Response string",
    "replacement": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "updatedAt": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response string",
  "unanswerableTemplate": "Response string",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
```

```
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "displayName": "Response string",
      "toolType": "Response string"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 456,
  "defaultLabelRelations": null,
  "useFullRetrievalRange": false,
  "enableEvaluation": false,
  "enableInlineCitations": false,
  "enableCodeInterpreter": false,
  "enableBrowserTool": false,
  "enableImageSearch": false,
  "enableClaudeStreamRetry": false,
  "customMaxLlmOutputTokens": 456,
  "voiceConfig": {
    "enabled": false,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Response string",
    "turnHandling": null,
    "enableVoiceReply": false
  },
  "thinkingConfig": null,
  "enableToolSearching": false,
  "maxAgentIterations": 456,
  "agentTimeoutSeconds": 456,
  "agentTimeoutMessage": "Response string",
  "requireTextWithFlexCards": false,
  "isMultimodalLlm": false,
  "enableSizeBasedToolResultTruncation": false,
  "toolResultMaxChars": 456,
  "enableVectorMemory": false,
```

```
  "longTermMemoryTopK": 456,
  "enableCrossConversationSearch": false,
  "crossConversationLookbackDays": 456,
  "useGatewayFallback": false,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": false,
  "enableGetMessageContextTool": false,
  "enableReadAttachmentFullTextTool": false,
  "enableParseAttachmentContentTool": false,
  "enableListAvailableParsersTool": false,
  "enableGetDownloadLinkTool": false,
  "isLocked": false,
  "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
  "canUpdate": false,
  "canDelete": false
}
```


Enable or Disable Evaluation

PATCH `/api/v1/chatbots/{id}/enable-evaluation/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
enableEvaluation	boolean	No	

Example Request Schema

```
{
  "enableEvaluation"?: boolean // Optional
}
```

Example Request Values

```
{
  "enableEvaluation": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "enableEvaluation": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "enableEvaluation": true
  };

axios.patch(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "enableEvaluation": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'enableEvaluation' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "enableEvaluation"?: boolean // Optional
}
```

Example Response Values

```
{
  "enableEvaluation": false
}
```

List Hooks for the Organization

GET `/api/v1/hooks/`

Hooks are created from Hook templates. The template determines the trigger point and handling behavior; these cannot be specified when creating a Hook. Once attached to an AI assistant, the Hook runs on incoming and outgoing messages. For details, see Hook Message Interception in the user manual.

Parameters

Parameter Name	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`search`	X	string	A search term.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/hooks/?page=1&pageSize=1&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/hooks/?page=1&pageSize=1&search=example",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/hooks/?page=1&pageSize=1&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/hooks/?
page=1&pageSize=1&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "organization": string (uuid)
      "templateId": string (uuid)
      "templateName": string
      "templateTypeLabel": string
      "scope": string
      "name": string
      "description": string
      "triggerPoint": string
      "action": string
```

```
      "handlerType": string
      "config": object
      "blockMessage": string
      "enabled": boolean
      "createdBy": object
      "chatbots": [ // Attached AI assistants for the list column. Uses the prefetched
``chatbot_attachments`` (see ``HookViewSet.get_queryset``) to avoid N+1.
        object
      ]
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "templateId": "550e8400-e29b-41d4-a716-446655440000",
      "templateName": "Response string",
      "templateTypeLabel": "Response string",
      "scope": "Response string",
      "name": "Response string",
      "description": "Response string",
      "triggerPoint": "Response string",
      "action": "Response string",
      "handlerType": "Response string",
      "config": null,
      "blockMessage": "Response string",
      "enabled": false,
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "chatbots": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response name",
          "description": "Example response description"
        }
      ],
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
```

```
  ]
}
```

Create a Hook from a Template

POST `/api/v1/hooks/`

Request Body

Request Parameters

Field	Type	Required	Description
templateId	string (uuid)	Yes	
name	string	Yes	
description	string	No	
config	object	No	
blockMessage	string	No	
enabled	boolean	No	

Example Request Schema

```
{
  "templateId": string (uuid)
  "name": string
  "description"?: string // Optional
  "config"?: object // Optional
  "blockMessage"?: string // Optional
  "enabled"?: boolean // Optional
}
```

Example Request Values

```
{
  "template_id": "00000000-0000-0000-0000-000000000001",
  "name": "Customer PII Redaction",
  "description": "Mask phone numbers and emails in inbound messages.",
  "config": {
    "data": {
      "patterns": [
        "phone",
```

```
        "email"
      ],
      "mode": "mask"
    }
  },
  "block_message": "",
  "enabled": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/hooks/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "template_id": "00000000-0000-0000-0000-000000000001",
    "name": "Customer PII Redaction",
    "description": "Mask phone numbers and emails in inbound messages.",
    "config": {
      "data": {
        "patterns": [
          "phone",
          "email"
        ],
        "mode": "mask"
      }
    },
    "block_message": "",
    "enabled": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "template_id": "00000000-0000-0000-0000-000000000001",
    "name": "Customer PII Redaction",
    "description": "Mask phone numbers and emails in inbound messages.",
    "config": {
      "data": {
        "patterns": [
          "phone",
          "email"
        ],
        "mode": "mask"
      }
    },
    "block_message": "",
    "enabled": true
  };

axios.post("https://api.maiagent.ai/api/v1/hooks/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/hooks/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "template_id": "00000000-0000-0000-0000-000000000001",
    "name": "Customer PII Redaction",
    "description": "Mask phone numbers and emails in inbound messages.",
    "config": {
        "data": {
            "patterns": [
                "phone",
                "email"
            ],
            "mode": "mask"
        }
    },
    "block_message": "",
    "enabled": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/hooks/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'template_id' => '00000000-0000-0000-0000-000000000001',
            'name' => 'Customer PII Redaction',
            'description' => 'Mask phone numbers and emails in inbound messages.',
            'config' => [
                'data' => [
                    'patterns' => [
                        'phone',
                        'email'
                    ],
                    'mode' => 'mask'
                ]
            ],
            'block_message' => '',
            'enabled' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "organization": string (uuid)
  "templateId": string (uuid)
  "templateName": string
  "templateTypeLabel": string
  "scope": string
  "name": string
  "description": string
  "triggerPoint": string
  "action": string
  "handlerType": string
  "config": object
  "blockMessage": string
  "enabled": boolean
  "createdBy": object
  "chatbots": [ // Attached AI assistants for the list column. Uses the prefetched
``chatbot_attachments`` (see ``HookViewSet.get_queryset``) to avoid N+1.
    object
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "templateId": "550e8400-e29b-41d4-a716-446655440000",
  "templateName": "Response string",
  "templateTypeLabel": "Response string",
  "scope": "Response string",
  "name": "Response string",
  "description": "Response string",
  "triggerPoint": "Response string",
  "action": "Response string",
  "handlerType": "Response string",
  "config": null,
  "blockMessage": "Response string",
  "enabled": false,
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Retrieve a Specific Hook

GET `/api/v1/hooks/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "organization": string (uuid)
  "templateId": string (uuid)
  "templateName": string
  "templateTypeLabel": string
  "scope": string
  "name": string
  "description": string
  "triggerPoint": string
  "action": string
  "handlerType": string
  "config": object
  "blockMessage": string
  "enabled": boolean
  "createdBy": object
```

```
  "chatbots": [ // Attached AI assistants for the list column. Uses the prefetched
``chatbot_attachments`` (see ``HookViewSet.get_queryset``) to avoid N+1.
    object
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "templateId": "550e8400-e29b-41d4-a716-446655440000",
  "templateName": "Response string",
  "templateTypeLabel": "Response string",
  "scope": "Response string",
  "name": "Response string",
  "description": "Response string",
  "triggerPoint": "Response string",
  "action": "Response string",
  "handlerType": "Response string",
  "config": null,
  "blockMessage": "Response string",
  "enabled": false,
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Partially Update a Hook

`PATCH` `/api/v1/hooks/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	
description	string	No	
config	object	No	
blockMessage	string	No	
enabled	boolean	No	

Example Request Schema

```
{
  "name"?: string // Optional
  "description"?: string // Optional
  "config"?: object // Optional
  "blockMessage"?: string // Optional
  "enabled"?: boolean // Optional
}
```

Example Request Values

```
{
  "name": "Example name",
  "description": "Example string",
  "config": null,
  "blockMessage": "Example string",
  "enabled": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string",
    "config": null,
    "blockMessage": "Example string",
    "enabled": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string",
    "config": null,
    "blockMessage": "Example string",
    "enabled": true
  };

axios.patch("https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "config": None,
    "blockMessage": "Example string",
    "enabled": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string',
            'config' => null,
            'blockMessage' => 'Example string',
            'enabled' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "organization": string (uuid)
  "templateId": string (uuid)
  "templateName": string
  "templateTypeLabel": string
  "scope": string
  "name": string
```

```
  "description": string
  "triggerPoint": string
  "action": string
  "handlerType": string
  "config": object
  "blockMessage": string
  "enabled": boolean
  "createdBy": object
  "chatbots": [ // Attached AI assistants for the list column. Uses the prefetched
``chatbot_attachments`` (see ``HookViewSet.get_queryset``) to avoid N+1.
    object
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "templateId": "550e8400-e29b-41d4-a716-446655440000",
  "templateName": "Response string",
  "templateTypeLabel": "Response string",
  "scope": "Response string",
  "name": "Response string",
  "description": "Response string",
  "triggerPoint": "Response string",
  "action": "Response string",
  "handlerType": "Response string",
  "config": null,
  "blockMessage": "Response string",
  "enabled": false,
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Delete Hook

DELETE `/api/v1/hooks/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/hooks/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

List Hooks Attached to an AI Assistant

GET `/api/v1/chatbots/{chatbotId}/hooks/`

Parameters

Parameter Name	Required	Type	Description

chatbotId	V	string

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "results": [
    {
      "id": string (uuid)
      "name": string
      "scope": string
      "templateName": string
      "templateTypeLabel": string
      "triggerPoint": string
      "action": string
      "enabled": boolean
      "order": integer
    }
  ]
}
```

Example Response Values

```
{
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "scope": "Response string",
      "templateName": "Response string",
      "templateTypeLabel": "Response string",
      "triggerPoint": "Response string",
      "action": "Response string",
      "enabled": false,
      "order": 456
    }
  ]
}
```

Replace All Hooks Attached to an AI Assistant

PUT `/api/v1/chatbots/{chatbotId}/hooks/`

Parameters

Parameter Name	Required	Type	Description
chatbotId	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
hookIds	array[string]	Yes	

Example Request Schema

```
{
  "hookIds": [
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "hookIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X PUT "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "hookIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "hookIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.put(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "hookIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'hookIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "results": [
    {
      "id": string (uuid)
      "name": string
      "scope": string
      "templateName": string
      "templateTypeLabel": string
      "triggerPoint": string
      "action": string
      "enabled": boolean
```

```
      "order": integer
    }
  ]
}
```

Example Response Values

```
{
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "scope": "Response string",
      "templateName": "Response string",
      "templateTypeLabel": "Response string",
      "triggerPoint": "Response string",
      "action": "Response string",
      "enabled": false,
      "order": 456
    }
  ]
}
```

List Schedules

GET `/api/v1/schedules/`

A schedule runs a prompt with an AI assistant at the specified time; the system automatically provisions a dedicated contact and conversation when the schedule is created.

Parameters

Parameter Name	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/schedules/?page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/schedules/?page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/schedules/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/schedules/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "prompt": string
      "chatbot": string (uuid)
      "chatbotName": string
      "executionMode": string (enum: in_context, isolated) // * `in_context` - In Context
* `isolated` - Isolated
      "scheduleType": string (enum: cron, interval, one_shot) // * `cron` - Cron
* `interval` - Interval
* `one_shot` - One Shot
      "cronExpression"?: string // Optional
      "intervalSeconds"?: integer // Optional
      "oneShotAt"?: string (timestamp) // Optional
      "deliveryConversationIds"?: [ // Optional
        string (uuid)
      ]
      "deliveryWebhookUrls"?: [ // Optional
        string (uri)
      ]
      "saveResultToContext"?: boolean // Optional
      "enabled"?: boolean // Optional
      "lastExecutedAt"?: string (timestamp) // Optional
      "executionCount"?: integer // Optional
      "maxExecutions"?: integer // Optional
      "countsTowardLimit": boolean
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
```

```
      "prompt": "Response string",
      "chatbot": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotName": "Response string",
      "executionMode": "in_context",
      "scheduleType": "cron",
      "cronExpression": "Response string",
      "intervalSeconds": 456,
      "oneShotAt": "Response string",
      "deliveryConversationIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "deliveryWebhookUrls": [
        "https://example.com/file.jpg"
      ],
      "saveResultToContext": false,
      "enabled": false,
      "lastExecutedAt": "Response string",
      "executionCount": 456,
      "maxExecutions": 456,
      "countsTowardLimit": false,
      "createdAt": "Response string"
    }
  ]
}
```


Create Schedule

POST `/api/v1/schedules/`

Request Body

Request Parameters

Field	Type	Required	Description
chatbot	string (uuid)	Yes	
name	string	Yes	
executionMode	string (enum: in_context, isolated)	Yes	`in_context` : In Context ; `isolated` : Isolated;
prompt	string	Yes	
contextVariables	object	No	
inbox	string (uuid)	Yes	
saveResultToContext	boolean	No	

Field	Type	Required	Description
deliveryConversationIds	array[string]	No	
deliveryWebhookUrls	array[string]	No	
scheduleType	string (enum: cron, interval, one_shot)	Yes	`cron` : Cron ; `interval` : Interval ; `one_shot` : One Shot;
cronExpression	string	No	
intervalSeconds	integer	No	
oneShotAt	string (timestamp)	No	
timezone	string	No	
enabled	boolean	No	
maxExecutions	integer	No	

Example Request Schema

```
{
  "chatbot": string (uuid)
  "name": string
  "executionMode": string (enum: in_context, isolated) // * `in_context` - In Context
* `isolated` - Isolated
  "prompt": string
  "contextVariables"?: object // Optional
  "inbox": string (uuid)
  "saveResultToContext"?: boolean // Optional
  "deliveryConversationIds"?: [ // Optional
    string (uuid)
  ]
  "deliveryWebhookUrls"?: [ // Optional
    string (uri)
  ]
  "scheduleType": string (enum: cron, interval, one_shot) // * `cron` - Cron
* `interval` - Interval
* `one_shot` - One Shot
  "cronExpression"?: string // Optional
  "intervalSeconds"?: integer // Optional
  "oneShotAt"?: string (timestamp) // Optional
  "timezone"?: string // Optional
  "enabled"?: boolean // Optional
  "maxExecutions"?: integer // Optional
}
```

Example Request Values

```
{
  "chatbot": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example name",
  "executionMode": "in_context",
  "prompt": "Example string",
```

```
  "contextVariables": null,
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "saveResultToContext": true,
  "deliveryConversationIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "deliveryWebhookUrls": [
    "https://example.com/file.jpg"
  ],
  "scheduleType": "cron",
  "cronExpression": "Example string",
  "intervalSeconds": 123,
  "oneShotAt": "Example string",
  "timezone": "Example string",
  "enabled": true,
  "maxExecutions": 123
}
```


Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/schedules/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "chatbot": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "executionMode": "in_context",
    "prompt": "Example string",
    "contextVariables": null,
    "inbox": "550e8400-e29b-41d4-a716-446655440000",
    "saveResultToContext": true,
    "deliveryConversationIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
      "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": true,
    "maxExecutions": 123
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "chatbot": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "executionMode": "in_context",
    "prompt": "Example string",
    "contextVariables": null,
    "inbox": "550e8400-e29b-41d4-a716-446655440000",
    "saveResultToContext": true,
    "deliveryConversationIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
      "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": true,
    "maxExecutions": 123
  };

axios.post("https://api.maiagent.ai/api/v1/schedules/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/schedules/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "chatbot": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "executionMode": "in_context",
    "prompt": "Example string",
    "contextVariables": None,
    "inbox": "550e8400-e29b-41d4-a716-446655440000",
    "saveResultToContext": True,
    "deliveryConversationIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
        "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": True,
    "maxExecutions": 123
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/schedules/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'chatbot' => '550e8400-e29b-41d4-a716-446655440000',
            'name' => 'Example name',
            'executionMode' => 'in_context',
            'prompt' => 'Example string',
            'contextVariables' => null,
            'inbox' => '550e8400-e29b-41d4-a716-446655440000',
            'saveResultToContext' => true,
            'deliveryConversationIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'deliveryWebhookUrls' => [
                'https://example.com/file.jpg'
            ],
            'scheduleType' => 'cron',
            'cronExpression' => 'Example string',
            'intervalSeconds' => 123,
            'oneShotAt' => 'Example string',
            'timezone' => 'Example string',
            'enabled' => true,
            'maxExecutions' => 123
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "chatbot": string (uuid)
  "organization": string (uuid)
  "name": string
  "executionMode": string (enum: in_context, isolated) // * `in_context` - In Context
* `isolated` - Isolated
  "prompt": string
  "contextVariables"?: object // Optional
  "inbox": string (uuid)
  "saveResultToContext"?: boolean // Optional
  "dedicatedContact": string (uuid)
  "dedicatedConversation": string (uuid)
  "deliveryConversationIds"?: [ // Optional
    string (uuid)
  ]
  "deliveryWebhookUrls"?: [ // Optional
    string (uri)
  ]
  "scheduleType": string (enum: cron, interval, one_shot) // * `cron` - Cron
* `interval` - Interval
* `one_shot` - One Shot
  "cronExpression"?: string // Optional
  "intervalSeconds"?: integer // Optional
  "oneShotAt"?: string (timestamp) // Optional
  "timezone"?: string // Optional
  "periodicTask": integer
  "enabled"?: boolean // Optional
  "lastExecutedAt": string (timestamp)
  "executionCount": integer
  "maxExecutions"?: integer // Optional
  "countsTowardLimit": boolean
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbot": "550e8400-e29b-41d4-a716-446655440000",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "executionMode": "in_context",
  "prompt": "Response string",
  "contextVariables": null,
```

```
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "saveResultToContext": false,
  "dedicatedContact": "550e8400-e29b-41d4-a716-446655440000",
  "dedicatedConversation": "550e8400-e29b-41d4-a716-446655440000",
  "deliveryConversationIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "deliveryWebhookUrls": [
    "https://example.com/file.jpg"
  ],
  "scheduleType": "cron",
  "cronExpression": "Response string",
  "intervalSeconds": 456,
  "oneShotAt": "Response string",
  "timezone": "Response string",
  "periodicTask": 456,
  "enabled": false,
  "lastExecutedAt": "Response string",
  "executionCount": 456,
  "maxExecutions": 456,
  "countsTowardLimit": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Retrieve a Specific Schedule

GET `/api/v1/schedules/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Schedule.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "chatbot": string (uuid)
  "organization": string (uuid)
  "name": string
  "executionMode": string (enum: in_context, isolated) // * `in_context` - In Context
* `isolated` - Isolated
  "prompt": string
  "contextVariables"?: object // Optional
  "inbox": string (uuid)
  "saveResultToContext"?: boolean // Optional
  "dedicatedContact": string (uuid)
  "dedicatedConversation": string (uuid)
  "deliveryConversationIds"?: [ // Optional
    string (uuid)
  ]
  "deliveryWebhookUrls"?: [ // Optional
    string (uri)
  ]
  "scheduleType": string (enum: cron, interval, one_shot) // * `cron` - Cron
* `interval` - Interval
* `one_shot` - One Shot
  "cronExpression"?: string // Optional
  "intervalSeconds"?: integer // Optional
  "oneShotAt"?: string (timestamp) // Optional
  "timezone"?: string // Optional
  "periodicTask": integer
  "enabled"?: boolean // Optional
  "lastExecutedAt": string (timestamp)
  "executionCount": integer
  "maxExecutions"?: integer // Optional
  "countsTowardLimit": boolean
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbot": "550e8400-e29b-41d4-a716-446655440000",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "executionMode": "in_context",
  "prompt": "Response string",
  "contextVariables": null,
```

```
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "saveResultToContext": false,
  "dedicatedContact": "550e8400-e29b-41d4-a716-446655440000",
  "dedicatedConversation": "550e8400-e29b-41d4-a716-446655440000",
  "deliveryConversationIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "deliveryWebhookUrls": [
    "https://example.com/file.jpg"
  ],
  "scheduleType": "cron",
  "cronExpression": "Response string",
  "intervalSeconds": 456,
  "oneShotAt": "Response string",
  "timezone": "Response string",
  "periodicTask": 456,
  "enabled": false,
  "lastExecutedAt": "Response string",
  "executionCount": 456,
  "maxExecutions": 456,
  "countsTowardLimit": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Partially Update a Schedule

PATCH `/api/v1/schedules/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Schedule.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	
prompt	string	No	

Field	Type	Required	Description
contextVariables	object	No	
saveResultToContext	boolean	No	
deliveryConversationIds	array[string]	No	
deliveryWebhookUrls	array[string]	No	
scheduleType	string (enum: cron, interval, one_shot)	No	`cron` : Cron ; `interval` : Interval ; `one_shot` : One Shot;
cronExpression	string	No	
intervalSeconds	integer	No	
oneShotAt	string (timestamp)	No	
timezone	string	No	
enabled	boolean	No	
maxExecutions	integer	No	

Example Request Schema

```
{
  "name"?: string // Optional
  "prompt"?: string // Optional
  "contextVariables"?: object // Optional
  "saveResultToContext"?: boolean // Optional
  "deliveryConversationIds"?: [ // Optional
    string (uuid)
  ]
  "deliveryWebhookUrls"?: [ // Optional
    string (uri)
  ]
  "scheduleType"?: string (enum: cron, interval, one_shot) // * `cron` - Cron
* `interval` - Interval
* `one_shot` - One Shot (Optional)
  "cronExpression"?: string // Optional
  "intervalSeconds"?: integer // Optional
  "oneShotAt"?: string (timestamp) // Optional
  "timezone"?: string // Optional
  "enabled"?: boolean // Optional
  "maxExecutions"?: integer // Optional
}
```

Example Request Values

```
{
  "name": "Example name",
  "prompt": "Example string",
  "contextVariables": null,
  "saveResultToContext": true,
```

```
  "deliveryConversationIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "deliveryWebhookUrls": [
    "https://example.com/file.jpg"
  ],
  "scheduleType": "cron",
  "cronExpression": "Example string",
  "intervalSeconds": 123,
  "oneShotAt": "Example string",
  "timezone": "Example string",
  "enabled": true,
  "maxExecutions": 123
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "prompt": "Example string",
    "contextVariables": null,
    "saveResultToContext": true,
    "deliveryConversationIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
      "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": true,
    "maxExecutions": 123
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "Example name",
    "prompt": "Example string",
    "contextVariables": null,
    "saveResultToContext": true,
    "deliveryConversationIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
      "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": true,
    "maxExecutions": 123
  };

axios.patch("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example name",
    "prompt": "Example string",
    "contextVariables": None,
    "saveResultToContext": True,
    "deliveryConversationIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
        "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": True,
    "maxExecutions": 123
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'prompt' => 'Example string',
            'contextVariables' => null,
            'saveResultToContext' => true,
            'deliveryConversationIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'deliveryWebhookUrls' => [
                'https://example.com/file.jpg'
            ],
            'scheduleType' => 'cron',
            'cronExpression' => 'Example string',
            'intervalSeconds' => 123,
            'oneShotAt' => 'Example string',
            'timezone' => 'Example string',
            'enabled' => true,
            'maxExecutions' => 123
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "chatbot": string (uuid)
  "organization": string (uuid)
  "name": string
  "executionMode":
  {
  }
  "prompt": string
  "contextVariables"?: object // Optional
  "inbox": string (uuid)
  "saveResultToContext"?: boolean // Optional
  "dedicatedContact": string (uuid)
  "dedicatedConversation": string (uuid)
  "deliveryConversationIds"?: [ // Optional
    string (uuid)
  ]
  "deliveryWebhookUrls"?: [ // Optional
    string (uri)
  ]
  "scheduleType": string (enum: cron, interval, one_shot) // * `cron` - Cron
* `interval` - Interval
* `one_shot` - One Shot
  "cronExpression"?: string // Optional
  "intervalSeconds"?: integer // Optional
  "oneShotAt"?: string (timestamp) // Optional
  "timezone"?: string // Optional
  "periodicTask": integer
  "enabled"?: boolean // Optional
  "lastExecutedAt": string (timestamp)
  "executionCount": integer
  "maxExecutions"?: integer // Optional
  "countsTowardLimit": boolean
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbot": "550e8400-e29b-41d4-a716-446655440000",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "executionMode": {},
  "prompt": "Response string",
  "contextVariables": null,
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "saveResultToContext": false,
  "dedicatedContact": "550e8400-e29b-41d4-a716-446655440000",
```

```
  "dedicatedConversation": "550e8400-e29b-41d4-a716-446655440000",
  "deliveryConversationIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "deliveryWebhookUrls": [
    "https://example.com/file.jpg"
  ],
  "scheduleType": "cron",
  "cronExpression": "Response string",
  "intervalSeconds": 456,
  "oneShotAt": "Response string",
  "timezone": "Response string",
  "periodicTask": 456,
  "enabled": false,
  "lastExecutedAt": "Response string",
  "executionCount": 456,
  "maxExecutions": 456,
  "countsTowardLimit": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Delete a Schedule

DELETE `/api/v1/schedules/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Schedule.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Run a Schedule Immediately

`POST` `/api/v1/schedules/{id}/run-now/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Schedule.

Request Body

Request Parameters

Field	Type	Required	Description
chatbot	string (uuid)	Yes	
name	string	Yes	
executionMode	string (enum: in_context, isolated)	Yes	`in_context` : In Context ; `isolated` : Isolated;
prompt	string	Yes	
contextVariables	object	No	
inbox	string (uuid)	Yes	
saveResultToContext	boolean	No	
deliveryConversationIds	array[string]	No	
deliveryWebhookUrls	array[string]	No	
scheduleType	string (enum: cron, interval, one_shot)	Yes	`cron` : Cron ; `interval` : Interval ; `one_shot` : One Shot;
cronExpression	string	No	

Field	Type	Required	Description
intervalSeconds	integer	No	
oneShotAt	string (timestamp)	No	
timezone	string	No	
enabled	boolean	No	
maxExecutions	integer	No	

Example Request Schema

```
{
  "chatbot": string (uuid)
  "name": string
  "executionMode": string (enum: in_context, isolated) // * `in_context` - In Context
* `isolated` - Isolated
  "prompt": string
  "contextVariables"?: object // Optional
  "inbox": string (uuid)
  "saveResultToContext"?: boolean // Optional
  "deliveryConversationIds"?: [ // Optional
    string (uuid)
  ]
  "deliveryWebhookUrls"?: [ // Optional
    string (uri)
  ]
  "scheduleType": string (enum: cron, interval, one_shot) // * `cron` - Cron
* `interval` - Interval
* `one_shot` - One Shot
  "cronExpression"?: string // Optional
  "intervalSeconds"?: integer // Optional
  "oneShotAt"?: string (timestamp) // Optional
  "timezone"?: string // Optional
  "enabled"?: boolean // Optional
  "maxExecutions"?: integer // Optional
}
```

Example Request Values

```
{
  "chatbot": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example name",
  "executionMode": "in_context",
  "prompt": "Example string",
  "contextVariables": null,
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "saveResultToContext": true,
  "deliveryConversationIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "deliveryWebhookUrls": [
    "https://example.com/file.jpg"
```

```
  ],
  "scheduleType": "cron",
  "cronExpression": "Example string",
  "intervalSeconds": 123,
  "oneShotAt": "Example string",
  "timezone": "Example string",
  "enabled": true,
  "maxExecutions": 123
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/run-now/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "chatbot": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "executionMode": "in_context",
    "prompt": "Example string",
    "contextVariables": null,
    "inbox": "550e8400-e29b-41d4-a716-446655440000",
    "saveResultToContext": true,
    "deliveryConversationIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
      "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": true,
    "maxExecutions": 123
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "chatbot": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "executionMode": "in_context",
    "prompt": "Example string",
    "contextVariables": null,
    "inbox": "550e8400-e29b-41d4-a716-446655440000",
    "saveResultToContext": true,
    "deliveryConversationIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
      "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": true,
    "maxExecutions": 123
  };

axios.post("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/run-now/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-446655440000/run-
now/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "chatbot": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "executionMode": "in_context",
    "prompt": "Example string",
    "contextVariables": None,
    "inbox": "550e8400-e29b-41d4-a716-446655440000",
    "saveResultToContext": True,
    "deliveryConversationIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "deliveryWebhookUrls": [
        "https://example.com/file.jpg"
    ],
    "scheduleType": "cron",
    "cronExpression": "Example string",
    "intervalSeconds": 123,
    "oneShotAt": "Example string",
    "timezone": "Example string",
    "enabled": True,
    "maxExecutions": 123
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-
41d4-a716-446655440000/run-now/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'chatbot' => '550e8400-e29b-41d4-a716-446655440000',
            'name' => 'Example name',
            'executionMode' => 'in_context',
            'prompt' => 'Example string',
            'contextVariables' => null,
            'inbox' => '550e8400-e29b-41d4-a716-446655440000',
            'saveResultToContext' => true,
            'deliveryConversationIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'deliveryWebhookUrls' => [
                'https://example.com/file.jpg'
            ],
            'scheduleType' => 'cron',
            'cronExpression' => 'Example string',
            'intervalSeconds' => 123,
            'oneShotAt' => 'Example string',
            'timezone' => 'Example string',
            'enabled' => true,
            'maxExecutions' => 123
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status code: 202

Example Response Schema

```
{
  "status"?: string // Optional
}
```

Example Response Values

```
{
  "status": "scheduled"
}
```

Enable or Disable a Schedule

POST `/api/v1/schedules/{id}/toggle/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Schedule.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/toggle/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/toggle/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-41d4-a716-
446655440000/toggle/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/schedules/550e8400-e29b-
41d4-a716-446655440000/toggle/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "chatbot": string (uuid)
  "organization": string (uuid)
  "name": string
  "executionMode": string (enum: in_context, isolated) // * `in_context` - In Context
* `isolated` - Isolated
  "prompt": string
  "contextVariables"?: object // Optional
  "inbox": string (uuid)
  "saveResultToContext"?: boolean // Optional
  "dedicatedContact": string (uuid)
  "dedicatedConversation": string (uuid)
  "deliveryConversationIds"?: [ // Optional
    string (uuid)
  ]
  "deliveryWebhookUrls"?: [ // Optional
    string (uri)
  ]
  "scheduleType": string (enum: cron, interval, one_shot) // * `cron` - Cron
* `interval` - Interval
* `one_shot` - One Shot
  "cronExpression"?: string // Optional
  "intervalSeconds"?: integer // Optional
  "oneShotAt"?: string (timestamp) // Optional
  "timezone"?: string // Optional
  "periodicTask": integer
  "enabled"?: boolean // Optional
  "lastExecutedAt": string (timestamp)
  "executionCount": integer
  "maxExecutions"?: integer // Optional
  "countsTowardLimit": boolean
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbot": "550e8400-e29b-41d4-a716-446655440000",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "executionMode": "in_context",
  "prompt": "Response string",
  "contextVariables": null,
```

```
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "saveResultToContext": false,
  "dedicatedContact": "550e8400-e29b-41d4-a716-446655440000",
  "dedicatedConversation": "550e8400-e29b-41d4-a716-446655440000",
  "deliveryConversationIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "deliveryWebhookUrls": [
    "https://example.com/file.jpg"
  ],
  "scheduleType": "cron",
  "cronExpression": "Response string",
  "intervalSeconds": 456,
  "oneShotAt": "Response string",
  "timezone": "Response string",
  "periodicTask": 456,
  "enabled": false,
  "lastExecutedAt": "Response string",
  "executionCount": 456,
  "maxExecutions": 456,
  "countsTowardLimit": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Status code: 400

Example Response Schema

```
{
  "detail"?: string // Optional
}
```

Example Response Values


```
{
  "detail": "Response string"
}
```

Prerequisites

目錄

1. Query Credit Balance
2. Get Wallet Details
3. Get Wallet Balance
4. Query Expiring Credits
5. Query Top-up History
6. List Credit Transactions
7. Retrieve a Credit Transaction
8. Query Credit Transaction History
9. Query Consumption Totals
10. Query Credit Usage by Category
11. Retrieve Credit Usage for a Single Category
12. Retrieve Member Usage for a Billable Item
13. Retrieve Billable Item Usage for a Single Member
14. List Current Member Quotas
15. List Current Role Quotas
16. List Current Conversation Platform Quotas
17. List Member Quota Settings
18. Retrieve a Member Quota Setting
19. Set a Member Quota
20. Update Member Quota Settings
21. List Role Quota Settings
22. Retrieve a Role Quota Setting
23. Retrieve Role Member Quota Statistics
24. Distribute Quotas by Role in Batches
25. Query your own quota
26. Query the quotas of your roles
27. List pricing rules
28. Retrieve a pricing rule
29. List Credit Items
30. Retrieve a Credit Item

Prerequisites

The Credit category provides APIs for querying and allocating organization Credits: balances, expiring Credits, top-up history, consumption details and totals, usage statistics, current allocations for members, roles, and conversation platforms, pricing rules and pricing items, as well as setting allocations for individual members and assigning allocations in bulk by role.

Prerequisites

Authentication and Path Parameters

All APIs except “Pricing Rules” and “Pricing Items” require an API Key in the HTTP header:

```
Authorization: Api-Key {API_KEY}
```

The system automatically identifies the organization and member from the API Key. APIs starting with `/api/v1/credit/` do not require a separate organization identifier. Obtain the IDs required in the path as follows:

- `organizationPk`: Call `GET /api/v1/organizations/` and use the organization’s `id` from the response.
- Wallet `id`: Call `GET /api/v1/organizations/{id}/` and use `creditWallet.id` from the response.

Rate Limits

For APIs that require an API Key, the rate limit is **the number of requests per minute configured individually for each API Key**. Organization administrators configure “Requests per Minute (RPM)” under “Organization Settings → API Keys”. Requests made with the same key to all APIs on the platform count toward the same limit. Keys with no limit configured are not subject to this restriction. Requests that exceed the limit are rejected with HTTP 429. Wait until the next minute before retrying.

“Pricing Rules” and “Pricing Items” do not require an API Key. Their rate limits are calculated separately by source IP; see the notes for each API.

Permissions and Billing Modes

Each API lists its required permissions and billing mode prerequisites below. Requests made with an API Key use the identity of the member who created it, and permissions are determined by that member’s role. Requests are rejected with HTTP 403 in either of the following cases:

- **Insufficient permissions**: The role of the member who owns the key lacks the permission listed for the API (“Credit Management”, “Usage Statistics”, or “Organization Overview” permission). Ask an organization administrator to enable the corresponding permission for the member’s role under “Organization Settings → Roles”.
- **The organization does not use Credit billing**: APIs marked “The organization must use Credit billing” are available only to organizations using Credit billing. Call `GET /api/v1/organizations/{id}/` to check: these APIs are available if `organizationPlan.billingMode`

in the response is `credit` (or if the organization has no plan yet and `organizationPlan` is empty). To switch to Credit billing, contact your MaiAgent representative.

Query Credit Balance

`GET` `/api/v1/credit/wallets/`

Required permission: Credit Management. Billing mode: Unrestricted; organizations using any billing mode can call this API.

Parameters

Parameter Name	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/wallets/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/wallets/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/wallets/?
page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/wallets/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```
  "previous"?: string (uri) // Optional
  "results": [
    {
      "totalCredits": string (decimal)
      "autoTopup": object // Format auto-topup configuration.
      "balanceAlert": object // Format balance alert configuration.
      "nextGrant": object // Format the next grant, or None when nothing
will actually be granted.

Amount-first: a missing/zero ``amount`` always wins over any schedule
(stale ``next_grant_at`` row, Stripe period_end with zero-grant plan),
so the frontend never receives a "scheduled in X days for 0 credits"
entry — the API decides whether the block renders at all.
      "lastGrant": object // Format the most recent PLAN_GRANT topup, or
None when wallet has none.
      "consumptionHint": object // Return 7-day daily average consumption
hint, or None if insufficient data.
      "billingProvider": string // Return the organization's billing
provider so the Credit page can pick its card layout.
      "periodCreditGrant": string // Return current-period credit grant
total; None unless AWS Marketplace billed.
      "periodMeteredUsage": string // Return current-period accepted
metered usage; None unless AWS Marketplace billed.
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "totalCredits": "Response string",
      "autoTopup": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "balanceAlert": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
```

```
        "description": "Example response description"
      },
      "nextGrant": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "lastGrant": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "consumptionHint": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "billingProvider": "Response string",
      "periodCreditGrant": "Response string",
      "periodMeteredUsage": "Response string"
    }
  ]
}
```


Get Wallet Details

GET `/api/v1/credit/wallets/{id}`

Required permission: Credit Management. Billing mode: Unrestricted; organizations using any billing mode can call this API.

Parameters

Parameter Name	Required	Type	Description

id	V	string	A UUID string identifying this Credit Wallet.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/wallets/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/wallets/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/wallets/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/wallets/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "totalCredits": string (decimal)
  "autoTopup": object // Format auto-topup configuration.
```

```
  "balanceAlert": object // Format balance alert configuration.
  "nextGrant": object // Format the next grant, or None when nothing will
actually be granted.

Amount-first: a missing/zero ``amount`` always wins over any schedule
(stale ``next_grant_at`` row, Stripe period_end with zero-grant plan),
so the frontend never receives a "scheduled in X days for 0 credits"
entry — the API decides whether the block renders at all.
  "lastGrant": object // Format the most recent PLAN_GRANT topup, or None
when wallet has none.
  "consumptionHint": object // Return 7-day daily average consumption
hint, or None if insufficient data.
  "billingProvider": string // Return the organization's billing provider
so the Credit page can pick its card layout.
  "periodCreditGrant": string // Return current-period credit grant
total; None unless AWS Marketplace billed.
  "periodMeteredUsage": string // Return current-period accepted metered
usage; None unless AWS Marketplace billed.
}
```

Example Response Values

```
{
  "totalCredits": "Response string",
  "autoTopup": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "balanceAlert": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "nextGrant": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "lastGrant": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
```

```
  "consumptionHint": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "billingProvider": "Response string",
  "periodCreditGrant": "Response string",
  "periodMeteredUsage": "Response string"
}
```


Get Wallet Balance

GET `/api/v1/credit/wallets/{id}/balance/`

Required permission: Credit Management. Billing mode: Unrestricted; organizations using any billing mode can call this API.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Credit Wallet.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/wallets/550e8400-
e29b-41d4-a716-446655440000/balance/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/wallets/550e8400-
e29b-41d4-a716-446655440000/balance/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/wallets/550e8400-e29b-41d4-a716-446655440000/balance/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/wallets/550e8400-e29b-
41d4-a716-446655440000/balance/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "totalCredits": string (decimal)
  "autoTopup": object // Format auto-topup configuration.
```

```
  "balanceAlert": object // Format balance alert configuration.
  "nextGrant": object // Format the next grant, or None when nothing will
actually be granted.

Amount-first: a missing/zero ``amount`` always wins over any schedule
(stale ``next_grant_at`` row, Stripe period_end with zero-grant plan),
so the frontend never receives a "scheduled in X days for 0 credits"
entry — the API decides whether the block renders at all.
  "lastGrant": object // Format the most recent PLAN_GRANT topup, or None
when wallet has none.
  "consumptionHint": object // Return 7-day daily average consumption
hint, or None if insufficient data.
  "billingProvider": string // Return the organization's billing provider
so the Credit page can pick its card layout.
  "periodCreditGrant": string // Return current-period credit grant
total; None unless AWS Marketplace billed.
  "periodMeteredUsage": string // Return current-period accepted metered
usage; None unless AWS Marketplace billed.
}
```

Example Response Values

```
{
  "totalCredits": "Response string",
  "autoTopup": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "balanceAlert": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "nextGrant": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "lastGrant": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
```

```
  "consumptionHint": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "billingProvider": "Response string",
  "periodCreditGrant": "Response string",
  "periodMeteredUsage": "Response string"
}
```


Query Expiring Credits

GET `/api/v1/credit/wallets/{id}/expiring-credits/`

Required permission: Credit Management. Billing mode: Unrestricted; organizations using any billing mode can call this API. Only Credits with an expiration date are listed. Credits without an expiration date are excluded, so the results do not represent the organization’ s entire balance. The days parameter specifies how many days ahead to check for expiring Credits (default: 30; pass all to list all Credits with an expiration date).

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Credit Wallet.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/wallets/550e8400-
e29b-41d4-a716-446655440000/expiring-credits/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/wallets/550e8400-
e29b-41d4-a716-446655440000/expiring-credits/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/wallets/550e8400-e29b-41d4-a716-446655440000/expiring-credits/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/wallets/550e8400-e29b-
41d4-a716-446655440000/expiring-credits/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "totalCredits": string (decimal)
  "autoTopup": object // Format auto-topup configuration.
```

```
  "balanceAlert": object // Format balance alert configuration.
  "nextGrant": object // Format the next grant, or None when nothing will
actually be granted.

Amount-first: a missing/zero ``amount`` always wins over any schedule
(stale ``next_grant_at`` row, Stripe period_end with zero-grant plan),
so the frontend never receives a "scheduled in X days for 0 credits"
entry — the API decides whether the block renders at all.
  "lastGrant": object // Format the most recent PLAN_GRANT topup, or None
when wallet has none.
  "consumptionHint": object // Return 7-day daily average consumption
hint, or None if insufficient data.
  "billingProvider": string // Return the organization's billing provider
so the Credit page can pick its card layout.
  "periodCreditGrant": string // Return current-period credit grant
total; None unless AWS Marketplace billed.
  "periodMeteredUsage": string // Return current-period accepted metered
usage; None unless AWS Marketplace billed.
}
```

Example Response Values

```
{
  "totalCredits": "Response string",
  "autoTopup": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "balanceAlert": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "nextGrant": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "lastGrant": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
```

```
  "consumptionHint": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "billingProvider": "Response string",
  "periodCreditGrant": "Response string",
  "periodMeteredUsage": "Response string"
}
```


Query Top-up History

GET `/api/v1/credit/wallets/{id}/topup-history/`

Required permission: Credit Management. Billing mode: Unrestricted; organizations using any billing mode can call this API.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Credit Wallet.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/wallets/550e8400-
e29b-41d4-a716-446655440000/topup-history/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/wallets/550e8400-
e29b-41d4-a716-446655440000/topup-history/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/wallets/550e8400-e29b-41d4-a716-446655440000/topup-history/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/wallets/550e8400-e29b-
41d4-a716-446655440000/topup-history/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "totalCredits": string (decimal)
  "autoTopup": object // Format auto-topup configuration.
```

```
  "balanceAlert": object // Format balance alert configuration.
  "nextGrant": object // Format the next grant, or None when nothing will
actually be granted.

Amount-first: a missing/zero ``amount`` always wins over any schedule
(stale ``next_grant_at`` row, Stripe period_end with zero-grant plan),
so the frontend never receives a "scheduled in X days for 0 credits"
entry — the API decides whether the block renders at all.
  "lastGrant": object // Format the most recent PLAN_GRANT topup, or None
when wallet has none.
  "consumptionHint": object // Return 7-day daily average consumption
hint, or None if insufficient data.
  "billingProvider": string // Return the organization's billing provider
so the Credit page can pick its card layout.
  "periodCreditGrant": string // Return current-period credit grant
total; None unless AWS Marketplace billed.
  "periodMeteredUsage": string // Return current-period accepted metered
usage; None unless AWS Marketplace billed.
}
```

Example Response Values

```
{
  "totalCredits": "response string",
  "autoTopup": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "balanceAlert": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "nextGrant": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "lastGrant": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
```

```
  "consumptionHint": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "billingProvider": "response string",
  "periodCreditGrant": "response string",
  "periodMeteredUsage": "response string"
}
```


List Credit Transactions

GET `/api/v1/credit/transactions/`

Required permission: Credit management permission. Billing mode: Any. Organizations in any billing mode can call this endpoint.

Parameters

Parameter	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/transactions/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/transactions/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/transactions/?
page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/transactions/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e-
>getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```
{
  "count": integer
```

```
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "eventDatetime": string (timestamp)
      "createdAt": string (timestamp)
      "transactionType": string
      "creditItem": object
      "amount": string (decimal)
      "balanceAfter": string (decimal)
      "quantity": string (decimal)
      "provider": string
      "modelCode": string
      "unitPriceSnapshot": string (decimal)
      "member": object
      "description": string
      "metadata": object
      "targetObject": object // Return frontend deep-link target for this
transaction's GFK content.

``{kind: 'chat.message', objectId, chatbotId}`` opens the AgentOps record
modal;
``{kind: 'dataset.evaluation', objectId, isMultiTurn}`` opens the
evaluation results page.
Returns None when no GFK is attached, the content type is unsupported,
or the target no longer resolves inside this organization.
    }
  ]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "eventDatetime": "response string",
      "createdAt": "response string",
      "transactionType": "response string",
      "creditItem": {
```

```
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response example name",
        "description": "response example description"
      },
      "amount": "response string",
      "balanceAfter": "response string",
      "quantity": "response string",
      "provider": "response string",
      "modelCode": "response string",
      "unitPriceSnapshot": "response string",
      "member": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response example name",
        "description": "response example description"
      },
      "description": "response string",
      "metadata": null,
      "targetObject": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response example name",
        "description": "response example description"
      }
    }
  ]
}
```


Retrieve a Credit Transaction

GET `/api/v1/credit/transactions/{id}`

Required permission: Credit management permission. Billing mode: Any. Organizations in any billing mode can call this endpoint.

Parameters

Parameter	Required	Type	Description

`id`	V	string	A UUID string identifying this Credit Transaction.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/credit/transactions/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/transactions/550e84
00-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/transactions/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/transactions/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e-
>getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
  "eventDatetime": string (timestamp)
  "createdAt": string (timestamp)
  "transactionType": string
  "creditItem": object
  "amount": string (decimal)
  "balanceAfter": string (decimal)
  "quantity": string (decimal)
  "provider": string
  "modelCode": string
  "unitPriceSnapshot": string (decimal)
  "member": object
  "description": string
  "metadata": object
  "targetObject": object // Return frontend deep-link target for this
transaction's GFK content.

``{kind: 'chat.message', objectId, chatbotId}`` opens the AgentOps record
modal;
``{kind: 'dataset.evaluation', objectId, isMultiTurn}`` opens the
evaluation results page.
Returns None when no GFK is attached, the content type is unsupported,
or the target no longer resolves inside this organization.
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "eventDatetime": "response string",
  "createdAt": "response string",
  "transactionType": "response string",
  "creditItem": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  },
  "amount": "response string",
  "balanceAfter": "response string",
  "quantity": "response string",
  "provider": "response string",
  "modelCode": "response string",
  "unitPriceSnapshot": "response string",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "response example name",
    "description": "response example description"
  },
  "description": "response string",
  "metadata": null,
  "targetObject": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  }
}
```


Query Credit Transaction History

`GET` `/api/v1/credit/transactions/history/`

Required permission: Credit management permission. Billing mode: Any. Organizations in any billing mode can call this endpoint.

Parameters

Parameter	Required	Type	Description
`category`	X	string	Credit item category (e.g. llm_token, platform_fee)
`endDate`	X	string	End date (inclusive, whole day in the platform timezone Asia/Taipei), format: YYYY-MM-DD
`limit`	X	integer	Rows per page, capped at 100
`page`	X	integer	1-based page number
`search`	X	string	Case-insensitive match against transaction description and metadata

`startDate`	X	string	Start date (inclusive, whole day in the platform timezone Asia/Taipei), format: YYYY-MM-DD
`type`	X	string	Which rows to include: all ledger entries, consumption only, or credited only

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/credit/transactions/history/?
category=example&endDate=example&limit=1&page=1&search=example&start
Date=example&type=all"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/transactions/histor
y/?
category=example&endDate=example&limit=1&page=1&search=example&start
Date=example&type=all", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/transactions/history/?
category=example&endDate=example&limit=1&page=1&search=example&start
Date=example&type=all"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/transactions/history/?
category=example&endDate=example&limit=1&page=1&search=example&start
Date=example&type=all", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e-
>getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "eventDatetime": string (timestamp)
  "createdAt": string (timestamp)
  "transactionType": string
  "creditItem": object
  "amount": string (decimal)
  "balanceAfter": string (decimal)
  "quantity": string (decimal)
  "provider": string
  "modelCode": string
  "unitPriceSnapshot": string (decimal)
  "member": object
  "description": string
  "metadata": object
  "targetObject": object // Return frontend deep-link target for this
transaction's GFK content.

``{kind: 'chat.message', objectId, chatbotId}`` opens the AgentOps record
modal;
``{kind: 'dataset.evaluation', objectId, isMultiTurn}`` opens the
evaluation results page.
Returns None when no GFK is attached, the content type is unsupported,
or the target no longer resolves inside this organization.
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "eventDatetime": "response string",
  "createdAt": "response string",
  "transactionType": "response string",
  "creditItem": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  },
  "amount": "response string",
  "balanceAfter": "response string",
  "quantity": "response string",
  "provider": "response string",
  "modelCode": "response string",
```

```
  "unitPriceSnapshot": "response string",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  },
  "description": "response string",
  "metadata": null,
  "targetObject": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  }
}
```


Query Consumption Totals

GET `/api/v1/credit/transactions/history-summary/`

Required permissions: Credit management permission. Billing mode: Unrestricted; organizations using any billing mode can call this endpoint.

Parameters

Parameter Name	Required	Type	Description
`category`	X	string	Credit item category (e.g. llm_token, platform_fee)
`endDate`	X	string	End date (inclusive, whole day in the platform timezone Asia/Taipei), format: YYYY-MM-DD
`search`	X	string	Case-insensitive match against transaction description and metadata

`startDate`	X	string	Start date (inclusive, whole day in the platform timezone Asia/Taipei), format: YYYY-MM-DD
`type`	X	string	Which rows to include: all ledger entries, consumption only, or credited only

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/credit/transactions/history-
summary/?
category=example&endDate=example&search=example&startDate=example&ty
pe=all"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/transactions/history-summary/?category=example&endDate=example&search=example&startDate=example&type=all", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/transactions/history-summary/?category=example&endDate=example&search=example&startDate=example&type=all"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/transactions/history-
summary/?
category=example&endDate=example&search=example&startDate=example&ty
pe=all", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "consumedTotal": string (decimal)
  "creditedTotal": string (decimal)
  "count": integer
}
```

Example Response Values

```
{
  "consumedTotal": "response string",
  "creditedTotal": "response string",
  "count": 456
}
```

Query Credit Usage by Category

`GET` `/api/v1/organizations/{organizationPk}/usage-statistics/credit-categories/`

Required permissions: Usage statistics permission. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this organization ID
`endDate`	X	string	End date, format: YYYY-MM-DD
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

`startDate`	V	string	Start date, format: YYYY-MM-DD
`timeGranularity`	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/credit-categories/?
endDate=example&page=1&pageSize=1&startDate=example&timeGranularity=
day"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/credit-categories/?
endDate=example&page=1&pageSize=1&startDate=example&timeGranularity=
day", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/credit-categories/?
endDate=example&page=1&pageSize=1&startDate=example&timeGranularity=
day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/credit-categories/?
endDate=example&page=1&pageSize=1&startDate=example&timeGranularity=
day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    object
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ]
}
```

Retrieve Credit Usage for a Single Category

GET `/api/v1/organizations/{organizationPk}/usage-statistics/credit-categories/{id}`

Required permissions: Usage statistics permission. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID
`endDate`	X	string	End date, format: YYYY-MM-DD
`startDate`	V	string	Start date, format: YYYY-MM-DD
`timeGranularity`	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/credit-
categories/550e8400-e29b-41d4-a716-446655440000/?
endDate=example&startDate=example&timeGranularity=day"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/credit-
categories/550e8400-e29b-41d4-a716-446655440000/?
endDate=example&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/credit-categories/550e8400-
e29b-41d4-a716-446655440000/?
endDate=example&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/credit-categories/550e8400-
e29b-41d4-a716-446655440000/?
endDate=example&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
object
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
```

Retrieve Member Usage for a Billable Item

GET `/api/v1/organizations/{organizationPk}/usage-statistics/credit-categories/{itemPk}/members/`

Required permissions: Usage statistics permission. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`itemPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID
`endDate`	X	string	End date, format: YYYY-MM-DD
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

`startDate`	V	string	Start date, format: YYYY-MM-DD
`timeGranularity`	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/credit-
categories/{itemPk}/members/?
endDate=example&page=1&pageSize=1&startDate=example&timeGranularity=
day"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/credit-
categories/{itemPk}/members/?
endDate=example&page=1&pageSize=1&startDate=example&timeGranularity=
day", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/credit-categories/{itemPk}/members/?endDate=example&page=1&pageSize=1&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/credit-
categories/{itemPk}/members/?
endDate=example&page=1&pageSize=1&startDate=example&timeGranularity=
day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    object
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ]
}
```

Retrieve Billable Item Usage for a Single Member

GET `/api/v1/organizations/{organizationPk}/usage-statistics/credit-categories/{itemPk}/members/{id}`

Required permissions: Usage statistics permission. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`itemPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID
`endDate`	X	string	End date, format: YYYY-MM-DD
`startDate`	V	string	Start date, format: YYYY-MM-DD
`timeGranularity`	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/credit-
categories/{itemPk}/members/550e8400-e29b-41d4-a716-446655440000/?
endDate=example&startDate=example&timeGranularity=day"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/credit-
categories/{itemPk}/members/550e8400-e29b-41d4-a716-446655440000/?
endDate=example&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/credit-categories/{itemPk}/members/550e8400-e29b-41d4-a716-446655440000/?endDate=example&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/credit-
categories/{itemPk}/members/550e8400-e29b-41d4-a716-446655440000/?
endDate=example&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
object
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
```

List Current Member Quotas

GET `/api/v1/organizations/{organizationPk}/credit-quota-members/`

Required permissions: Credit management permission. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this organization ID
`ordering`	X	string	Sort field (default '-created_at')
`page`	X	integer	Page number
`pageSize`	X	integer	Rows per page (max 100)
`quotaStatus`	X	string	Filter by quota status

`search`	X	string	Filter by member name, email, or member ID (case-insensitive substring)

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/credit-quota-members/?ordering=-
created_at&page=1&pageSize=1&quotaStatus=allocated&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/credit-quota-members/?ordering=-
created_at&page=1&pageSize=1&quotaStatus=allocated&search=example",
config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/credit-quota-members/?ordering=-created_at&page=1&pageSize=1&quotaStatus=allocated&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/credit-quota-members/?ordering=-
created_at&page=1&pageSize=1&quotaStatus=allocated&search=example",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "count"?: integer // Optional
      "next"?: string (uri) // Optional
      "previous"?: string (uri) // Optional
      "results"?: [ // Optional
        {
          "id"?: string (uuid) // Optional
          "name"?: string // Optional
          "email"?: string // Optional
          "credit_quota_required"?: boolean // Optional
          "active_quota"?: { // Most recent quota summary (active first);
same shape as the members API active_quota. (Optional)
          {
            "id"?: string (uuid) // Optional
            "quota_limit"?: string // Optional
            "consumed_amount"?: string // Optional
            "remaining"?: string // Optional
            "valid_from"?: string (date-time) // Optional
            "valid_until"?: string (date-time) // Optional
            "is_active"?: boolean // Optional
            "expired_at"?: string (date-time) // Optional
            "note"?: string // Optional
          }
          }
        }
      ]
      "stats"?: { // Organization-wide member counts, unaffected by
search or quota_status filter. (Optional)
      {
        "total"?: integer // Optional
        "allocated"?: integer // Optional
        "expired"?: integer // Optional
        "unallocated"?: integer // Optional
      }
      }
    }
  ]
}
```

Example Response Values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "count": 456,
      "next": "https://example.com/file.jpg",
      "previous": "https://example.com/file.jpg",
      "results": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
          "email": "Response string",
          "credit_quota_required": false,
          "active_quota": {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response name",
            "description": "Example response description"
          }
        }
      ],
      "stats": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    }
  ]
}
```


List Current Role Quotas

`GET` `/api/v1/organizations/{organizationPk}/credit-quota-groups/`

Required permissions: Credit management permission. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this organization ID
`ordering`	X	string	Sort field (default '-created_at')
`page`	X	integer	Page number
`pageSize`	X	integer	Rows per page (max 100)
`quotaStatus`	X	string	Filter by quota status
`search`	X	string	Filter by group name or group ID (case-insensitive substring)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/credit-quota-groups/?ordering=-
created_at&page=1&pageSize=1&quotaStatus=allocated&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/credit-quota-groups/?ordering=-
created_at&page=1&pageSize=1&quotaStatus=allocated&search=example",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/credit-quota-groups/?ordering=-
created_at&page=1&pageSize=1&quotaStatus=allocated&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/credit-quota-groups/?ordering=-
created_at&page=1&pageSize=1&quotaStatus=allocated&search=example",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "count"?: integer // Optional
      "next"?: string (uri) // Optional
      "previous"?: string (uri) // Optional
      "results"?: [ // Optional
        {
          "id"?: string (uuid) // Optional
          "name"?: string // Optional
          "type"?: string (enum: owner, default, custom) // Optional
          "credit_quota_required"?: boolean // Optional
          "active_quota"?: { // Most recent quota summary (active first);
same shape as the member/inbox variants. (Optional)
          {
            "id"?: string (uuid) // Optional
            "quota_limit"?: string // Optional
            "consumed_amount"?: string // Optional
            "remaining"?: string // Optional
            "valid_from"?: string (date-time) // Optional
            "valid_until"?: string (date-time) // Optional
            "is_active"?: boolean // Optional
            "expired_at"?: string (date-time) // Optional
            "note"?: string // Optional
          }
          }
        }
      ]
      "stats"?: { // Organization-wide group counts, unaffected by search
or quota_status filter. (Optional)
      {
        "total"?: integer // Optional
        "allocated"?: integer // Optional
        "expired"?: integer // Optional
        "unallocated"?: integer // Optional
      }
      }
    }
  ]
}
```

Example Response Values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "count": 456,
      "next": "https://example.com/file.jpg",
      "previous": "https://example.com/file.jpg",
      "results": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
          "type": "owner",
          "credit_quota_required": false,
          "active_quota": {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response name",
            "description": "Example response description"
          }
        }
      ],
      "stats": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    }
  ]
}
```


List Current Conversation Platform Quotas

GET `/api/v1/organizations/{organizationPk}/credit-quota-inboxes/`

Required permissions: Credit management permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this organization ID
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`search`	X	string	A search term.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/credit-quota-inboxes/?
ordering=example&page=1&pageSize=1&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/credit-quota-inboxes/?
ordering=example&page=1&pageSize=1&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/credit-quota-inboxes/?
ordering=example&page=1&pageSize=1&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/credit-quota-inboxes/?
ordering=example&page=1&pageSize=1&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "channelType":
      {
      }
      "channelTypeLabel": string
      "webChatAccessType": string // Sub-type for web channels (web =
public embed, admin = internal Q&A, api = completions).

A chatbot ships with three same-named web inboxes, so the frontend needs
this to tell them apart. None for non-web channels. Reads the prefetched
generic ``channel`` relation (no extra query on the list path).
      "chatbot": object
      "creditQuotaRequired": boolean // When True, this inbox must have
an active credit quota for the AI to reply
      "activeQuota": object // Return the most recent quota summary
(active first), via prefetch to avoid N+1.
      "canUpdate": boolean // Return whether the current user can update
this resource.
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "channelType": {},
      "channelTypeLabel": "Response string",
      "webChatAccessType": "Response string",
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
```

```
      },
      "creditQuotaRequired": false,
      "activeQuota": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "canUpdate": false
    }
  ]
}
```


List Member Quota Settings

GET `/api/v1/organizations/{organizationPk}/members/{memberPk}/credit-quotas/`

Required permissions: Credit management permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`memberPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/{memberPk}/credit-quotas/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/{memberPk}/credit-quotas/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/{memberPk}/credit-quotas/?
page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/{memberPk}/credit-quotas/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "member": object
      "quotaLimit": string (decimal) // Maximum credits this member may
consume
      "consumedAmount": string (decimal) // Credits consumed so far
(updated via F() for concurrency safety)
      "remaining": string (decimal)
      "validFrom": string (timestamp)
      "validUntil": string (timestamp) // NULL means permanent
      "resetPeriod":  // Cycle on which consumed_amount is automatically
zeroed; "none" keeps the one-shot ceiling

* `none` - No Reset
* `daily` - Daily
* `weekly` - Weekly
* `monthly` - Monthly
      {
      }
      "nextResetAt": string (timestamp) // Local-midnight boundary of the
next automatic reset; NULL when reset_period is "none"
      "isActive": boolean
      "expiredAt": string (timestamp) // Actual deactivation time (cron
or manual)
      "createdBy": object
      "note": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "member": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "quotaLimit": "Response string",
      "consumedAmount": "Response string",
      "remaining": "Response string",
      "validFrom": "Response string",
      "validUntil": "Response string",
      "resetPeriod": {},
      "nextResetAt": "Response string",
      "isActive": false,
      "expiredAt": "Response string",
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "note": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Retrieve a Member Quota Setting

GET `/api/v1/organizations/{organizationPk}/members/{memberPk}/credit-quotas/{id}`

Required permissions: Credit management permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`memberPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/{memberPk}/credit-
quotas/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/{memberPk}/credit-
quotas/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/{memberPk}/credit-quotas/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/{memberPk}/credit-quotas/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
```

```
  "member": object
  "quotaLimit": string (decimal) // Maximum credits this member may
consume
  "consumedAmount": string (decimal) // Credits consumed so far (updated
via F() for concurrency safety)
  "remaining": string (decimal)
  "validFrom": string (timestamp)
  "validUntil": string (timestamp) // NULL means permanent
  "resetPeriod":  // Cycle on which consumed_amount is automatically
zeroed; "none" keeps the one-shot ceiling

* `none` - No Reset
* `daily` - Daily
* `weekly` - Weekly
* `monthly` - Monthly
  {
  }
  "nextResetAt": string (timestamp) // Local-midnight boundary of the
next automatic reset; NULL when reset_period is "none"
  "isActive": boolean
  "expiredAt": string (timestamp) // Actual deactivation time (cron or
manual)
  "createdBy": object
  "note": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "quotaLimit": "Response string",
  "consumedAmount": "Response string",
  "remaining": "Response string",
  "validFrom": "Response string",
  "validUntil": "Response string",
  "resetPeriod": {},
  "nextResetAt": "Response string",
```

```
  "isActive": false,
  "expiredAt": "Response string",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "note": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Set a Member Quota

POST `/api/v1/organizations/{organizationPk}/members/{memberPk}/credit-quotas/`

Required permissions: Credit management permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`memberPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/{memberPk}/credit-quotas/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{}'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/{memberPk}/credit-quotas/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/{memberPk}/credit-quotas/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/{memberPk}/credit-quotas/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => new \stdClass()
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "member": object
  "quotaLimit": string (decimal) // Maximum credits this member may
consume
  "consumedAmount": string (decimal) // Credits consumed so far (updated
via F() for concurrency safety)
  "remaining": string (decimal)
  "validFrom": string (timestamp)
  "validUntil": string (timestamp) // NULL means permanent
  "resetPeriod":  // Cycle on which consumed_amount is automatically
zeroed; "none" keeps the one-shot ceiling

* `none` - No Reset
* `daily` - Daily
* `weekly` - Weekly
* `monthly` - Monthly
  {
  }
  "nextResetAt": string (timestamp) // Local-midnight boundary of the
next automatic reset; NULL when reset_period is "none"
  "isActive": boolean
  "expiredAt": string (timestamp) // Actual deactivation time (cron or
manual)
  "createdBy": object
  "note": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "quotaLimit": "Response string",
  "consumedAmount": "Response string",
  "remaining": "Response string",
  "validFrom": "Response string",
```

```
  "validUntil": "Response string",
  "resetPeriod": {},
  "nextResetAt": "Response string",
  "isActive": false,
  "expiredAt": "Response string",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "note": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Update Member Quota Settings

PATCH `/api/v1/organizations/{organizationPk}/members/{memberPk}/credit-quotas/{id}`

Required permissions: Credit management permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`memberPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/members/{memberPk}/credit-quotas/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{}'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {};

axios.patch("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/{memberPk}/credit-
quotas/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/{memberPk}/credit-quotas/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/{memberPk}/credit-quotas/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => new \stdClass()
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "member": object
  "quotaLimit": string (decimal) // Maximum credits this member may
consume
  "consumedAmount": string (decimal) // Credits consumed so far (updated
via F() for concurrency safety)
  "remaining": string (decimal)
  "validFrom": string (timestamp)
  "validUntil": string (timestamp) // NULL means permanent
  "resetPeriod":  // Cycle on which consumed_amount is automatically
zeroed; "none" keeps the one-shot ceiling

* `none` - No Reset
* `daily` - Daily
* `weekly` - Weekly
* `monthly` - Monthly
  {
  }
  "nextResetAt": string (timestamp) // Local-midnight boundary of the
next automatic reset; NULL when reset_period is "none"
  "isActive": boolean
  "expiredAt": string (timestamp) // Actual deactivation time (cron or
manual)
  "createdBy": object
  "note": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "quotaLimit": "Response string",
  "consumedAmount": "Response string",
  "remaining": "Response string",
  "validFrom": "Response string",
```

```
  "validUntil": "Response string",
  "resetPeriod": {},
  "nextResetAt": "Response string",
  "isActive": false,
  "expiredAt": "Response string",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "note": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


List Role Quota Settings

`GET` `/api/v1/organizations/{organizationPk}/groups/{groupPk}/credit-quotas/`

Required permissions: Credit management permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID
`page`	X	integer	A page number within the paginated result set.

`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/{groupPk}/credit-quotas/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/{groupPk}/credit-quotas/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/{groupPk}/credit-quotas/?
page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/{groupPk}/credit-quotas/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "group": object
      "quotaLimit": string (decimal) // Maximum credits members of this
group may consume
      "consumedAmount": string (decimal) // Credits consumed so far
(updated via F() for concurrency safety)
      "remaining": string (decimal)
      "validFrom": string (timestamp)
      "validUntil": string (timestamp) // NULL means permanent
      "resetPeriod":  // Cycle on which consumed_amount is automatically
zeroed; "none" keeps the one-shot ceiling

* `none` - No Reset
* `daily` - Daily
* `weekly` - Weekly
* `monthly` - Monthly
      {
      }
      "nextResetAt": string (timestamp) // Local-midnight boundary of the
next automatic reset; NULL when reset_period is "none"
      "isActive": boolean
      "expiredAt": string (timestamp) // Actual deactivation time (cron
or manual)
      "createdBy": object
      "note": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "group": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "quotaLimit": "Response string",
      "consumedAmount": "Response string",
      "remaining": "Response string",
      "validFrom": "Response string",
      "validUntil": "Response string",
      "resetPeriod": {},
      "nextResetAt": "Response string",
      "isActive": false,
      "expiredAt": "Response string",
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "note": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Retrieve a Role Quota Setting

`GET` `/api/v1/organizations/{organizationPk}/groups/{groupPk}/credit-quotas/{id}`

Required permissions: Credit management permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`groupPk`	V	string	
`id`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/{groupPk}/credit-quotas/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/{groupPk}/credit-quotas/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/{groupPk}/credit-quotas/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/{groupPk}/credit-quotas/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
  "group": object
  "quotaLimit": string (decimal) // Maximum credits members of this group
may consume
  "consumedAmount": string (decimal) // Credits consumed so far (updated
via F() for concurrency safety)
  "remaining": string (decimal)
  "validFrom": string (timestamp)
  "validUntil": string (timestamp) // NULL means permanent
  "resetPeriod":  // Cycle on which consumed_amount is automatically
zeroed; "none" keeps the one-shot ceiling

* `none` - No Reset
* `daily` - Daily
* `weekly` - Weekly
* `monthly` - Monthly
  {
  }
  "nextResetAt": string (timestamp) // Local-midnight boundary of the
next automatic reset; NULL when reset_period is "none"
  "isActive": boolean
  "expiredAt": string (timestamp) // Actual deactivation time (cron or
manual)
  "createdBy": object
  "note": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "group": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "quotaLimit": "Response string",
  "consumedAmount": "Response string",
  "remaining": "Response string",
  "validFrom": "Response string",
  "validUntil": "Response string",
  "resetPeriod": {},
  "nextResetAt": "Response string",
```

```
  "isActive": false,
  "expiredAt": "Response string",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "note": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Retrieve Role Member Quota Statistics

GET `/api/v1/organizations/{organizationPk}/groups/{id}/credit-quota-stats/`

Required permissions: Credit management permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Group.
`organizationPk`	V	string	A UUID string identifying this organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/credit-quota-stats/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/credit-quota-stats/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/credit-quota-stats/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/credit-quota-stats/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "activeQuotaMembersCount": integer
```

```
  "quotaRequiredMembersCount": integer
}
```

Example Response Values

```
{
  "activeQuotaMembersCount": 456,
  "quotaRequiredMembersCount": 456
}
```

Distribute Quotas by Role in Batches

POST `/api/v1/organizations/{organizationPk}/groups/{id}/batch-distribute-credit-quota/`

Required permissions: Credit management and organization overview permissions. Billing mode: The organization must use Credit billing.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Group.
`organizationPk`	V	string	A UUID string identifying this organization ID

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	
maigptCanCodeInterpreter	boolean	No	Whether members of this role may use the code interpreter built-in tool in MaiGPT.
maigptCanRag	boolean	No	Whether members of this role may use the knowledge base RAG built-in tool in MaiGPT.
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.
maigptAllLlms	boolean	No	Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none.
maigptAllTools	boolean	No	Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound

Field	Type	Required	Description
			tools; an empty binding then means none.
maigptAllConnectors	boolean	No	Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none.
maigptAllSkills	boolean	No	Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none.

Example request structure

```
{
  "name": string
  "canCreateChatbot"?: boolean // Optional
  "canCreateKnowledgeBase"?: boolean // Optional
  "canCreateInbox"?: boolean // Optional
  "canCreateDatabase"?: boolean // Optional
  "canCreateTool"?: boolean // Optional
  "canCreateAgentUiTool"?: boolean // Optional
  "canCreateSkill"?: boolean // Optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role may use the code interpreter built-in tool in MaiGPT. (Optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the knowledge base RAG built-in tool in MaiGPT. (Optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this switch too. (Optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may use the browser built-in tool in MaiGPT. (Optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none. (Optional)
  "maigptAllTools"?: boolean // Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none. (Optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may
```

```
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (Optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (Optional)
}
```

Example request values

```
{
  "name": "Example name",
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/batch-distribute-credit-quota/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true
  }'

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/batch-distribute-credit-quota/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
```

```
  console.error(error.response?.data || error.message);
});
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/batch-distribute-credit-quota/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "canCreateChatbot": True,
    "canCreateKnowledgeBase": True,
    "canCreateInbox": True,
    "canCreateDatabase": True,
    "canCreateTool": True,
    "canCreateAgentUiTool": True,
    "canCreateSkill": True,
    "maigptCanCodeInterpreter": True,
    "maigptCanRag": True,
    "maigptCanDeepResearch": True,
    "maigptCanBrowserTool": True,
    "maigptAllLlms": True,
    "maigptAllTools": True,
    "maigptAllConnectors": True,
    "maigptAllSkills": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/batch-distribute-credit-quota/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'canCreateChatbot' => true,
            'canCreateKnowledgeBase' => true,
            'canCreateInbox' => true,
            'canCreateDatabase' => true,
            'canCreateTool' => true,
            'canCreateAgentUiTool' => true,
            'canCreateSkill' => true,
            'maigptCanCodeInterpreter' => true,
            'maigptCanRag' => true,
            'maigptCanDeepResearch' => true,
            'maigptCanBrowserTool' => true,
            'maigptAllLlms' => true,
            'maigptAllTools' => true,
            'maigptAllConnectors' => true,
            'maigptAllSkills' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
```

```
    }
    ?>
```

Response body

Status code: 200

Example response structure

```
{
  "id": string (uuid)
  "name": string
  "type":
  {
  }
  "permissions": [ // Returns the list of group permissions

Processing steps:
1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission is missing its parent permission, automatically
load the missing parent from the database
4. Serialize parent permissions and their corresponding child
permissions, then sort by order
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Returns the number of remaining members
(after subtracting the 3 shown in the list) for the frontend +N badge

Unlike resource counts:
<ul>
  <li>members_count: Returns total - 3 (for the +N badge); None means the
total is <= 3</li>
  <li>chatbots_count/knowledge_bases_count/inboxes_count: Returns the
```

```
total count</li>
</ul>
  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]
  "databasesCount": integer
  "databasesPreview": [
    string
  ]
  "canCreateChatbot"?: boolean // Optional
  "canCreateKnowledgeBase"?: boolean // Optional
  "canCreateInbox"?: boolean // Optional
  "canCreateDatabase"?: boolean // Optional
  "canCreateTool"?: boolean // Optional
  "canCreateAgentUiTool"?: boolean // Optional
  "canCreateSkill"?: boolean // Optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (Optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (Optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (Optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (Optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (Optional)
  "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (Optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (Optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use
```

```
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (Optional)
  "createdAt": string (timestamp)
}
```

Example response values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "membersPreview": [
    "Response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "Response string"
  ],
  "knowledgeBasesCount": 456,
  "knowledgeBasesPreview": [
    "Response string"
  ],
  "inboxesCount": 456,
  "inboxesPreview": [
    "Response string"
  ],
  "databasesCount": 456,
  "databasesPreview": [
    "Response string"
  ],
  "canCreateChatbot": false,
  "canCreateKnowledgeBase": false,
  "canCreateInbox": false,
  "canCreateDatabase": false,
  "canCreateTool": false,
  "canCreateAgentUiTool": false,
```

```
  "canCreateSkill": false,
  "maigptCanCodeInterpreter": false,
  "maigptCanRag": false,
  "maigptCanDeepResearch": false,
  "maigptCanBrowserTool": false,
  "maigptAllLlms": false,
  "maigptAllTools": false,
  "maigptAllConnectors": false,
  "maigptAllSkills": false,
  "createdAt": "Response string"
}
```


Query your own quota

GET `/api/v1/organizations/{organizationPk}/me/credit-quota/`

Required permissions: No additional permissions are required. Any organization member can query their own quota. Billing mode: The organization must use Credit billing.

Parameters

Parameter name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/me/credit-quota/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/me/credit-quota/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/me/credit-quota/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/me/credit-quota/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

Query the quotas of your roles

GET `/api/v1/organizations/{organizationPk}/me/group-credit-quotas/`

Required permissions: No additional permissions are required. Any organization member can query their own quota. Billing mode: The organization must use Credit billing.

Parameters

Parameter name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/me/group-credit-quotas/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/me/group-credit-quotas/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/me/group-credit-quotas/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/me/group-credit-quotas/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

List pricing rules

`GET` `/api/v1/credit/pricing-rules/`

No authentication required: This endpoint can be called without an API Key. Rate limit: 60 requests per minute per source IP. Exceeding the limit returns HTTP 429.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/pricing-rules/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/pricing-rules/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/pricing-rules/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/pricing-rules/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response structure

```
[
  {
    "id": string (uuid)
    "creditItem": string (uuid)
```

```
    "provider": string
    "modelCode": string
    "creditPerUnit": string (decimal)
    "priority": integer // Higher priority = more specific (model-
specific=20, provider-level=10)
    "effectiveFrom": string (timestamp)
    "effectiveUntil": string (timestamp)
    "isActive": boolean
    "notes": string // Internal notes about this pricing rule
  }
]
```

Example response values


```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "creditItem": "550e8400-e29b-41d4-a716-446655440000",
    "provider": "Response string",
    "modelCode": "Response string",
    "creditPerUnit": "Response string",
    "priority": 456,
    "effectiveFrom": "Response string",
    "effectiveUntil": "Response string",
    "isActive": false,
    "notes": "Response string"
  }
]
```


Retrieve a pricing rule

`GET` `/api/v1/credit/pricing-rules/{id}`

No authentication required: This endpoint can be called without an API Key. Rate limit: 60 requests per minute per source IP. Exceeding the limit returns HTTP 429.

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this Credit Pricing Rule.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/pricing-
rules/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/pricing-
rules/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/pricing-rules/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/pricing-rules/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "creditItem": string (uuid)
```

```
  "provider": string
  "modelCode": string
  "creditPerUnit": string (decimal)
  "priority": integer // Higher priority = more specific (model-
specific=20, provider-level=10)
  "effectiveFrom": string (timestamp)
  "effectiveUntil": string (timestamp)
  "isActive": boolean
  "notes": string // Internal notes about this pricing rule
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "creditItem": "550e8400-e29b-41d4-a716-446655440000",
  "provider": "response string",
  "modelCode": "response string",
  "creditPerUnit": "response string",
  "priority": 456,
  "effectiveFrom": "response string",
  "effectiveUntil": "response string",
  "isActive": false,
  "notes": "response string"
}
```

List Credit Items

GET `/api/v1/credit/items/`

No login required: Call this endpoint without an API Key. Rate limit: 60 requests per minute per source IP. Exceeding this limit returns HTTP 429.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/items/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/items/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/items/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/items/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
[
  {
    "id": string (uuid)
    "code": string // Unique identifier like llm_input_token
```

```
    "name": string
    "category":
    {
    }
    "unit": string // token, character, minute, etc.
    "creditPerUnit": string (decimal)
    "baseCreditPerUnit": string (decimal)
    "effectiveFrom": string (timestamp)
    "isActive": boolean
    "description": string
    "displayGroup": string // Items with the same display_group are
merged into one expandable row on the pricing page. Empty means the item
stands alone.
    "displayOrder": integer // Controls display ordering. Lower numbers
appear first.
    "attachmentRetentionDays": integer // Current Config retention
setting, so the pricing-page tooltip day count follows admin changes.
    "gatewayTrafficFeeDefaultPercent": string // System default AI
Gateway traffic-fee percent, for pricing pages without an org context.

This endpoint is AllowAny with no organization, so it can only surface
the system default ("for reference only" on the frontend); logged-in
pricing pages read the organization's actual percent from the
organization detail API instead.
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "code": "response string",
    "name": "response string",
    "category": {},
    "unit": "response string",
    "creditPerUnit": "response string",
    "baseCreditPerUnit": "response string",
    "effectiveFrom": "response string",
    "isActive": false,
    "description": "response string",
    "displayGroup": "response string",
    "displayOrder": 456,
    "attachmentRetentionDays": 456,
```

```
    "gatewayTrafficFeeDefaultPercent": "response string"
  }
]
```

Retrieve a Credit Item

GET `/api/v1/credit/items/{id}`

No login required: Call this endpoint without an API Key. Rate limit: 60 requests per minute per source IP. Exceeding this limit returns HTTP 429.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Credit Item.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/credit/items/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/credit/items/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/credit/items/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/credit/items/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "code": string // Unique identifier like llm_input_token
```

```
  "name": string
  "category":
  {
  }
  "unit": string // token, character, minute, etc.
  "creditPerUnit": string (decimal)
  "baseCreditPerUnit": string (decimal)
  "effectiveFrom": string (timestamp)
  "isActive": boolean
  "description": string
  "displayGroup": string // Items with the same display_group are merged into one expandable row on the pricing page. Empty means the item stands alone.
  "displayOrder": integer // Controls display ordering. Lower numbers appear first.
  "attachmentRetentionDays": integer // Current Config retention setting, so the pricing-page tooltip day count follows admin changes.
  "gatewayTrafficFeeDefaultPercent": string // System default AI Gateway traffic-fee percent, for pricing pages without an org context.

This endpoint is AllowAny with no organization, so it can only surface the system default ("for reference only" on the frontend); logged-in pricing pages read the organization's actual percent from the organization detail API instead.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "code": "response string",
  "name": "response string",
  "category": {},
  "unit": "response string",
  "creditPerUnit": "response string",
  "baseCreditPerUnit": "response string",
  "effectiveFrom": "response string",
  "isActive": false,
  "description": "response string",
  "displayGroup": "response string",
  "displayOrder": 456,
  "attachmentRetentionDays": 456,
```

```
  "gatewayTrafficFeeDefaultPercent": "response string"
}
```

API Call Example (Shell)

目錄

1. Get Service Provider Configuration (for IdP Discovery)
2. List or Query Members (Supports `filter`, `startIndex`, and `count`)
3. Create Member
4. Get a Specific Member
5. Replace Member
6. Partially Update Member (Deactivate, Rename, or Change externalId)
7. Deactivate Member
8. List Groups (Mapped to MaiAgent Roles)
9. Create Group
10. Retrieve a specific group
11. Replace a group
12. Partially update a group (rename, add or remove members)
13. Delete a group

API Call Example (Shell)

SCIM 2.0 endpoints let your identity provider (Entra ID, Okta, etc.) automatically provision, update, and deactivate organization members. The base URL is https://api.maiagent.ai/api/v1/scim/{orgUuid}, with no trailing `/`. Authentication uses `Authorization: Bearer` (obtained from the identity synchronization settings in the admin console, rather than an API Key), with `Content-Type: application/scim+json` and a limit of 100 requests per minute per organization. For field mappings and deactivation semantics, see Member Provisioning in the technical manual.

Get Service Provider Configuration (for IdP Discovery)

`GET` `/api/v1/scim/{orgUuid}/ServiceProviderConfig`

Parameters

Parameter Name	Required	Type	Description
orgUuid	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/ServiceProviderConfig"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

axios.get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/ServiceProviderConfig", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/ServiceProviderConfig"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/ServiceProviderConfig", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
200	No response body

List or Query Members (Supports `filter`, `startIndex`, and `count`)

GET `/api/v1/scim/{orgUuid}/Users`

Parameters

Parameter Name	Required	Type	Description
`orgUuid`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Users"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

axios.get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Users", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Users"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Users", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
200	No response body

Create Member

`POST` `/api/v1/scim/{orgUuid}/Users`

Parameters

Parameter Name	Required	Type	Description
`orgUuid`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-
41d4-a716-446655440000/Users"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

// Request Body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Users", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Users"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Users", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
200	No response body

Get a Specific Member

GET `/api/v1/scim/{orgUuid}/Users/{userId}`

Parameters

Parameter Name	Required	Type	Description
`orgUuid`	V	string	
`userId`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

axios.get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Users/550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
200	No response body

Replace Member

PUT `/api/v1/scim/{orgUuid}/Users/{userId}`

Parameters

Parameter Name	Required	Type	Description
`orgUuid`	V	string	
`userId`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

// Request Body (payload)
const data = null;

axios.put("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.put(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Users/550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
200	No response body

Partially Update Member (Deactivate, Rename, or Change externalId)

`PATCH` `/api/v1/scim/{orgUuid}/Users/{userId}`

Parameters

Parameter Name	Required	Type	Description
`orgUuid`	V	string	
`userId`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-
41d4-a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

// Request Body (payload)
const data = null;

axios.patch("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.patch(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Users/550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
200	No response body

Deactivate Member

DELETE `/api/v1/scim/{orgUuid}/Users/{userId}`

Parameters

Parameter Name	Required	Type	Description
`orgUuid`	V	string	
`userId`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-
41d4-a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-
41d4-a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Users/550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
204	No response body

List Groups (Mapped to MaiAgent Roles)

GET `/api/v1/scim/{orgUuid}/Groups`

Parameters

Parameter Name	Required	Type	Description
`orgUuid`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

axios.get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Groups"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Groups", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
200	No response body

Create Group

POST `/api/v1/scim/{orgUuid}/Groups`

Parameters

Parameter Name	Required	Type	Description
`orgUuid`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-
41d4-a716-446655440000/Groups"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

// Request Body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Groups"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Groups", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

Retrieve a specific group

GET `/api/v1/scim/{orgUuid}/Groups/{groupId}`

Parameters

Parameter name	Required	Type	Description
`groupId`	V	string	
`orgUuid`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

axios.get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Groups/550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

Replace a group

PUT `/api/v1/scim/{orgUuid}/Groups/{groupId}`

Parameters

Parameter name	Required	Type	Description
`groupId`	V	string	
`orgUuid`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

// Request body (payload)
const data = null;

axios.put("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.put(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Groups/550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

Partially update a group (rename, add or remove members)

`PATCH` `/api/v1/scim/{orgUuid}/Groups/{groupId}`

Parameters

Parameter name	Required	Type	Description
`groupId`	V	string	
`orgUuid`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-
41d4-a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

// Request body (payload)
const data = null;

axios.patch("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Groups/550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.patch(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-
446655440000/Groups/550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
200	No response body

Delete a group

DELETE `/api/v1/scim/{orgUuid}/Groups/{groupId}`

Parameters

Parameter name	Required	Type	Description
`groupId`	V	string	
`orgUuid`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-
41d4-a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Bearer YOUR_SCIM_TOKEN"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Bearer YOUR_SCIM_TOKEN'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-
41d4-a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Bearer YOUR_SCIM_TOKEN"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/scim/550e8400-e29b-41d4-
a716-446655440000/Groups/550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Bearer YOUR_SCIM_TOKEN'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
204	No response body

Example API call (Shell)

目錄

1. Create Web Chat
2. List Web Chats
3. Retrieve a Specific Web Chat
4. Update Web Chat
5. Partially Update Web Chat
6. Set Up Contact Credentials
7. Get Voice Agent Token
8. Upload a Batch QA File
9. Export Batch QA to Excel
10. Get LLM Usage Statistics
11. Get AI Assistant Statistics

Example API call (Shell)

Create Web Chat

`POST` `/api/v1/web-chats/`

Request Body

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	

Field	Type	Required	Description
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	`full` : Full ; `compact` : Compact ; `hidden` : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a row of buttons above the input field, which remains usable. When disabled, the existing overlay panel is used. This gradual rollout switch can be disabled at any time to revert to the previous behavior.
enableConversationHandoff	boolean	No	When enabled, the webchat for this conversation platform displays the handoff UI (notification bar, summary card, and continuation button) and registers handoff tools. It is still governed by the global handoff switch in system settings: disabling the global switch disables handoff everywhere. MaiGPT always displays the handoff UI regardless of this switch. The context usage percentage bar at the bottom is

Field	Type	Required	Description
			controlled separately by "Display Context Usage". Disabled by default, so customer-facing webchats show no handoff UI. This gradual rollout switch can be disabled to revert immediately without rolling back the release.
enableDisplayContextUsage	boolean	No	When enabled, the webchat for this conversation platform displays a context usage percentage bar below the input field. It is independent of conversation handoff: handoff switches (including the global handoff switch) do not affect it, and it does not affect the handoff notification bar. MaiGPT always displays the context usage bar regardless of this switch. Disabled by default; disabling it reverts the behavior immediately without rolling back the release.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, ranging from 3-60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Minutes after the last AI reply before the survey appears, ranging from 1-60 minutes

Field	Type	Required	Description
satisfactionSurveyPrompt	string	No	Custom survey title text. Leave blank to use the default localized text in the frontend
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. `narrative` shows per-stage copy; `brand_animation` plays the uploaded asset and hides every system narration line. Defaults to `narrative` , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds `loading_long_wait_threshold_seconds` , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays `loading_animation_shared_asset_url` ,

Field	Type	Required	Description
			so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Example Request Schema

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
```

```
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of buttons above the input field, which remains usable. When disabled, the existing overlay panel is used. This gradual rollout switch can be disabled at any time to revert to the previous behavior. (Optional)
  "enableConversationHandoff"?: boolean // When enabled, the webchat for this conversation platform displays the handoff UI (notification bar, summary card, and continuation button) and registers handoff tools. It is still governed by the global handoff switch in system settings: disabling the global switch disables handoff everywhere. MaiGPT always displays the handoff UI regardless of this switch. The context usage percentage bar at the bottom is controlled separately by "Display Context Usage". Disabled by default, so customer-facing webchats show no handoff UI. This gradual rollout switch can be disabled to revert immediately without rolling back the release. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, the webchat for this conversation platform displays a context usage percentage bar below the input field. It is independent of conversation handoff: handoff switches (including the global handoff switch) do not affect it, and it does not affect the handoff notification bar. MaiGPT always displays the context usage bar regardless of this switch. Disabled by default; disabling it reverts the behavior immediately without rolling back the release. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3-60 minutes (Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey appears, ranging from 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text. Leave blank to use the default localized text in the frontend (Optional)
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "domain": string
  }
```

```
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "conversationStartersDisplayCount"?: integer // Optional
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
}
```

Example Request Values

```
{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
```

```
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableConversationHandoff": true,
  "enableDisplayContextUsage": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
  "hookMarkerTextInputBlock": "Example string",
  "hookMarkerTextOutputBlock": "Example string",
  "enableShareConversation": true,
  "enableLocation": true,
  "enableShowPoweredBy": true,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": true,
  "enableDisplayPreviousConversation": true,
  "enableDisplayConversationHistory": true,
  "enableConversationTimer": true,
  "conversationTimerDuration": 123,
  "enableAutoNewConversationOnTimeout": true,
  "enableSatisfactionSurvey": true,
  "satisfactionSurveyDelayMinutes": 123,
  "satisfactionSurveyPrompt": "Example string",
  "enableThemeModeToggle": true,
  "enableDisplayFontSizeSwitch": true,
  "defaultThemeMode": {},
  "enableDownloadCitations": true,
  "customDomain": {
    "domain": "Example string"
  },
  "enableDisplayChatbotAvatar": true,
  "enableDisplayChatbotName": true,
  "buttonIcon": "https://example.com/file.jpg",
  "conversationStartersDisplayCount": 123,
  "preChatFormEnabled": true,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
```

```
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 123,
  "loadingAnimationUseSharedAsset": true,
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": true,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```


Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "darkTheme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableConversationHandoff": true,
    "enableDisplayContextUsage": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
```

```
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "darkTheme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableConversationHandoff": true,
    "enableDisplayContextUsage": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
```

```
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

axios.post("https://api.maiagent.ai/api/v1/web-chats/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": True,
    "theme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": True,
        "userMessageBackgroundShadowEnabled": True,
        "chatbotMessageGradientEnabled": True,
        "userMessageGradientEnabled": True
    },
    "darkTheme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": True,
        "userMessageBackgroundShadowEnabled": True,
        "chatbotMessageGradientEnabled": True,
        "userMessageGradientEnabled": True
    },
    "enableFileUpload": True,
    "enableDisplayCitations": True,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": True,
    "enableConversationHandoff": True,
    "enableDisplayContextUsage": True,
    "enableDisplayThinking": True,
    "enableUserThinkingControl": True,
    "enableDisplayQueryMetadata": True,
    "enableDisplayHookMarkers": True,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
```

```
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": True,
    "enableLocation": True,
    "enableShowPoweredBy": True,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": True,
    "enableDisplayPreviousConversation": True,
    "enableDisplayConversationHistory": True,
    "enableConversationTimer": True,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": True,
    "enableSatisfactionSurvey": True,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": True,
    "enableDisplayFontSizeSwitch": True,
    "defaultThemeMode": {},
    "enableDownloadCitations": True,
    "customDomain": {
        "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": True,
    "enableDisplayChatbotName": True,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": True,
    "preChatFormFields": None,
    "preChatFormTitle": None,
    "preChatFormSubtitle": None,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": None,
    "loadingLongWaitCopyByLocale": None,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": True,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": None,
    "loadingTypingShowCopy": True,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'avatar' => 'https://example.com/file.jpg',
            'logo' => 'https://example.com/file.jpg',
            'description' => 'Example string',
            'cover' => 'https://example.com/file.jpg',
            'backUrl' => 'Example string',
            'isActive' => true,
            'theme' => [
                'primaryColor' => 'Example string',
                'navbarTextColor' => 'Example string',
                'conversationBackgroundColor' => 'Example string',
                'chatbotMessageTextColor' => 'Example string',
                'userMessageTextColor' => 'Example string',
                'chatbotMessageBackgroundColor' => 'Example string',
                'userMessageBackgroundColor' => 'Example string',
                'chatbotMessageBackgroundShadowEnabled' => true,
                'userMessageBackgroundShadowEnabled' => true,
                'chatbotMessageGradientEnabled' => true,
                'userMessageGradientEnabled' => true
            ],
            'darkTheme' => [
                'primaryColor' => 'Example string',
                'navbarTextColor' => 'Example string',
                'conversationBackgroundColor' => 'Example string',
                'chatbotMessageTextColor' => 'Example string',
                'userMessageTextColor' => 'Example string',
                'chatbotMessageBackgroundColor' => 'Example string',
                'userMessageBackgroundColor' => 'Example string',
                'chatbotMessageBackgroundShadowEnabled' => true,
                'userMessageBackgroundShadowEnabled' => true,
                'chatbotMessageGradientEnabled' => true,
                'userMessageGradientEnabled' => true
            ],
            'enableFileUpload' => true,
            'enableDisplayCitations' => true,
            'toolDisplayMode' => 'full',
            'enableAskOptionButtons' => true,
            'enableConversationHandoff' => true,
            'enableDisplayContextUsage' => true,
            'enableDisplayThinking' => true,
            'enableUserThinkingControl' => true,
            'enableDisplayQueryMetadata' => true,
            'enableDisplayHookMarkers' => true,
```

```
            'hookMarkerTextInputModify' => 'Example string',
            'hookMarkerTextOutputModify' => 'Example string',
            'hookMarkerTextInputBlock' => 'Example string',
            'hookMarkerTextOutputBlock' => 'Example string',
            'enableShareConversation' => true,
            'enableLocation' => true,
            'enableShowPoweredBy' => true,
            'poweredByLogo' => 'https://example.com/file.jpg',
            'enableAnonymous' => true,
            'enableDisplayPreviousConversation' => true,
            'enableDisplayConversationHistory' => true,
            'enableConversationTimer' => true,
            'conversationTimerDuration' => 123,
            'enableAutoNewConversationOnTimeout' => true,
            'enableSatisfactionSurvey' => true,
            'satisfactionSurveyDelayMinutes' => 123,
            'satisfactionSurveyPrompt' => 'Example string',
            'enableThemeModeToggle' => true,
            'enableDisplayFontSizeSwitch' => true,
            'defaultThemeMode' => new \stdClass(),
            'enableDownloadCitations' => true,
            'customDomain' => [
                'domain' => 'Example string'
            ],
            'enableDisplayChatbotAvatar' => true,
            'enableDisplayChatbotName' => true,
            'buttonIcon' => 'https://example.com/file.jpg',
            'conversationStartersDisplayCount' => 123,
            'preChatFormEnabled' => true,
            'preChatFormFields' => null,
            'preChatFormTitle' => null,
            'preChatFormSubtitle' => null,
            'loadingDisplayMode' => new \stdClass(),
            'loadingStageCopyByLocale' => null,
            'loadingLongWaitCopyByLocale' => null,
            'loadingLongWaitThresholdSeconds' => 123,
            'loadingAnimationUseSharedAsset' => true,
            'loadingAnimationSharedAssetUrl' => 'https://example.com/file.jpg',
            'loadingAnimationAssetUrlsByStage' => null,
            'loadingTypingShowCopy' => true,
            'toolCardRunningIconUrl' => 'https://example.com/file.jpg',
            'toolCardCompletedIconUrl' => 'https://example.com/file.jpg'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input field, which remains usable. When disabled, the current overlay panel is
retained. This gradual rollout toggle can be disabled at any time to revert the behavior. (Optional)
```

```
  "enableConversationHandoff"?: boolean // When enabled, this conversation platform's webchat
displays the handoff UI (notification bar, summary card, and continue button) and registers handoff
tools. It remains subject to the global handoff toggle in system settings: disabling the global
toggle disables handoff everywhere. MaiGPT always displays this UI regardless of this toggle. The
context usage percentage bar at the bottom is controlled separately by "Display context usage".
Disabled by default, so customers' public-facing webchats show no handoff UI. This gradual rollout
toggle can be disabled to revert the behavior without rolling back the release. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, this conversation platform's webchat
displays a context usage percentage bar below the input field. This is independent of conversation
handoff: handoff toggles (including the global handoff toggle) do not affect it, and it does not
affect the handoff notification bar. MaiGPT always displays this bar regardless of this toggle.
Disabled by default; disabling it reverts the behavior without rolling back the release. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3-60 minutes
(Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI reply before the
survey appears, ranging from 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the frontend's
default localized text (Optional)
  "accessType":
  {
  }
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "id": string (uuid)
    "domain": string
    "status":
    {
    }
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
  "voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
  "conversationStartersDisplayCount"?: integer // Optional
  "includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
```

```
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
  "tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped — SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
    object
  ]
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
  "inbox": object
  "quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Response string",
  "cover": "https://example.com/file.jpg",
```

```
  "backUrl": "Response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "Response string"
  ],
  "theme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
    "userMessageBackgroundColor": "Response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "darkTheme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
    "userMessageBackgroundColor": "Response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "enableFileUpload": false,
  "enableDisplayCitations": false,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": false,
  "enableConversationHandoff": false,
  "enableDisplayContextUsage": false,
  "enableDisplayThinking": false,
  "enableUserThinkingControl": false,
  "enableDisplayQueryMetadata": false,
  "enableDisplayHookMarkers": false,
  "hookMarkerTextInputModify": "Response string",
  "hookMarkerTextOutputModify": "Response string",
  "hookMarkerTextInputBlock": "Response string",
  "hookMarkerTextOutputBlock": "Response string",
  "enableShareConversation": false,
  "enableLocation": false,
  "enableShowPoweredBy": false,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": false,
  "enableDisplayPreviousConversation": false,
  "enableDisplayConversationHistory": false,
  "enableConversationTimer": false,
  "conversationTimerDuration": 456,
  "enableAutoNewConversationOnTimeout": false,
  "enableSatisfactionSurvey": false,
  "satisfactionSurveyDelayMinutes": 456,
  "satisfactionSurveyPrompt": "Response string",
  "accessType": {},
```

```
  "enableThemeModeToggle": false,
  "enableDisplayFontSizeSwitch": false,
  "defaultThemeMode": {},
  "enableDownloadCitations": false,
  "customDomain": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "domain": "Response string",
    "status": {}
  },
  "enableDisplayChatbotAvatar": false,
  "enableDisplayChatbotName": false,
  "buttonIcon": "https://example.com/file.jpg",
  "enableCodeInterpreter": false,
  "voiceAgentEnabled": false,
  "conversationStartersDisplayCount": 456,
  "includeGreetingInContext": false,
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "preChatFormEnabled": false,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 456,
  "loadingAnimationUseSharedAsset": false,
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": false,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg",
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "quickMenu": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```


List Web Chats

GET `/api/v1/web-chats/`

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
[
  {
    "id": string (uuid)
    "avatar"?: string (uri) // Optional
    "name": string
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "avatar": "https://example.com/file.jpg",
    "name": "Response string"
  }
]
```

Retrieve a Specific Web Chat

GET `/api/v1/web-chats/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this web chat.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of buttons above the input field, which remains usable. When disabled, the existing overlay panel is retained. This gradual rollout switch can be disabled at any time to revert the behavior. (Optional)
  "enableConversationHandoff"?: boolean // When enabled, this conversation platform’s webchat displays the handoff UI (notification bar, summary card, and resume button) and registers handoff tools. The global handoff switch in system settings still applies: disabling it disables handoff everywhere. MaiGPT always displays this UI regardless of this switch. The context usage percentage
```

```
bar at the bottom is controlled separately by “Display context usage”. Disabled by default, so customer-facing webchats display no handoff UI. This gradual rollout switch can be disabled to revert the behavior without rolling back the release. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, this conversation platform’s webchat displays a context usage percentage bar below the input field. This is independent of conversation handoff: handoff switches, including the global handoff switch, do not affect it, and it does not affect the handoff notification bar. MaiGPT always displays the bar regardless of this switch. Disabled by default; disabling it reverts the behavior without rolling back the release. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, from 3 to 60 minutes (Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI reply before the survey appears, from 1 to 60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the frontend’s default localized text (Optional)
  "accessType":
  {
  }
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "id": string (uuid)
    "domain": string
    "status":
    {
    }
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
  "voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
  "conversationStartersDisplayCount"?: integer // Optional
  "includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM context so the agent can resolve short replies like "1" to the matching starter option.
  "tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
```

```
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped — SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
    object
  ]
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
  "inbox": object
  "quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}
```

Example Response Values

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "name": "Test doc",
  "avatar": "https://example.com/static/images/default-chatbot-avatar.png",
  "logo": null,
  "description": "",
  "cover": null,
  "backUrl": "",
  "isActive": true,
  "enableSpeech": false,
  "displayGreeting": null,
  "displayConversationStarters": [],
```

```
  "theme": {
    "primaryColor": "#3854d8",
    "dialogColor": "#ddeaf6",
    "navbarTextColor": "#ffffff"
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "enableShareConversation": false,
  "enableLocation": false,
  "enableShowPoweredBy": true,
  "poweredByLogo": null,
  "accessType": "web",
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Test doc",
    "signAuth": {
      "id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
      "signSource": {
        "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
        "accessType": "keycloak"
      },
      "signParams": {
        "keycloak": {
          "url": "example.com",
          "realm": "example",
          "clientId": "example"
        }
      }
    }
  }
}
```


Update Web Chat

PUT `/api/v1/web-chats/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this web chat.

Request Content

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	

Field	Type	Required	Description
toolDisplayMode	string (enum: full, compact, hidden)	No	`full` : Full ; `compact` : Compact ; `hidden` : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a row of buttons above the input box, which remains usable. When disabled, the current overlay panel is retained. This gradual rollout toggle can be disabled at any time to revert to the previous behavior.
enableConversationHandoff	boolean	No	When enabled, this conversation platform's webchat displays the handoff UI (notification bar, summary card, and continue button) and registers handoff tools. It remains subject to the global handoff toggle in system settings: disabling the global toggle disables handoff everywhere. MaiGPT always displays the UI regardless of this toggle. The context usage percentage bar at the bottom is controlled separately by "Display context usage". Disabled by default, so customer-facing webchats display no handoff UI. This gradual rollout toggle can be disabled to revert immediately without rolling back the release.
enableDisplayContextUsage	boolean	No	When enabled, this conversation platform's webchat displays a context usage percentage bar below the input box. It is independent of conversation handoff: handoff toggles, including the global handoff toggle, do not affect it, and it does not affect the handoff notification bar. MaiGPT always displays the bar regardless of this toggle. Disabled by default; disabling it reverts immediately without rolling back the release.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	

Field	Type	Required	Description
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, ranging from 3-60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Number of minutes after the last AI reply before the survey appears, ranging from 1-60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text; leave blank to use the frontend's default localized text
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (contains 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	

Field	Type	Required	Description
loadingDisplayMode	object	No	Waiting-experience style. `narrative` shows per-stage copy; `brand_animation` plays the uploaded asset and hides every system narration line. Defaults to `narrative`, which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds `loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays `loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Example Request Structure

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme": {
  {
```

```
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input box, which remains usable. When disabled, the current overlay panel is
retained. This gradual rollout toggle can be disabled at any time to revert to the previous behavior.
(Optional)
  "enableConversationHandoff"?: boolean // When enabled, this conversation platform's webchat
displays the handoff UI (notification bar, summary card, and continue button) and registers handoff
tools. It remains subject to the global handoff toggle in system settings: disabling the global
toggle disables handoff everywhere. MaiGPT always displays the UI regardless of this toggle. The
context usage percentage bar at the bottom is controlled separately by "Display context usage".
Disabled by default, so customer-facing webchats display no handoff UI. This gradual rollout toggle
can be disabled to revert immediately without rolling back the release. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, this conversation platform's webchat
displays a context usage percentage bar below the input box. It is independent of conversation
handoff: handoff toggles, including the global handoff toggle, do not affect it, and it does not
affect the handoff notification bar. MaiGPT always displays the bar regardless of this toggle.
Disabled by default; disabling it reverts immediately without rolling back the release. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
```

```
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3-60 minutes
(Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI reply before the
survey appears, ranging from 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the frontend's
default localized text (Optional)
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "domain": string
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "conversationStartersDisplayCount"?: integer // Optional
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
```

```
  "toolCardCompletedIconUrl"?: string (uri) // Optional
}
```

Example Request Values

```
{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableConversationHandoff": true,
  "enableDisplayContextUsage": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
  "hookMarkerTextInputBlock": "Example string",
  "hookMarkerTextOutputBlock": "Example string",
  "enableShareConversation": true,
  "enableLocation": true,
  "enableShowPoweredBy": true,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": true,
  "enableDisplayPreviousConversation": true,
```

```
  "enableDisplayConversationHistory": true,
  "enableConversationTimer": true,
  "conversationTimerDuration": 123,
  "enableAutoNewConversationOnTimeout": true,
  "enableSatisfactionSurvey": true,
  "satisfactionSurveyDelayMinutes": 123,
  "satisfactionSurveyPrompt": "Example string",
  "enableThemeModeToggle": true,
  "enableDisplayFontSizeSwitch": true,
  "defaultThemeMode": {},
  "enableDownloadCitations": true,
  "customDomain": {
    "domain": "Example string"
  },
  "enableDisplayChatbotAvatar": true,
  "enableDisplayChatbotName": true,
  "buttonIcon": "https://example.com/file.jpg",
  "conversationStartersDisplayCount": 123,
  "preChatFormEnabled": true,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 123,
  "loadingAnimationUseSharedAsset": true,
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": true,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "darkTheme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableConversationHandoff": true,
    "enableDisplayContextUsage": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
```

```
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "darkTheme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableConversationHandoff": true,
    "enableDisplayContextUsage": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
```

```
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

axios.put("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": True,
    "theme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": True,
        "userMessageBackgroundShadowEnabled": True,
        "chatbotMessageGradientEnabled": True,
        "userMessageGradientEnabled": True
    },
    "darkTheme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": True,
        "userMessageBackgroundShadowEnabled": True,
        "chatbotMessageGradientEnabled": True,
        "userMessageGradientEnabled": True
    },
    "enableFileUpload": True,
    "enableDisplayCitations": True,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": True,
    "enableConversationHandoff": True,
    "enableDisplayContextUsage": True,
    "enableDisplayThinking": True,
    "enableUserThinkingControl": True,
    "enableDisplayQueryMetadata": True,
    "enableDisplayHookMarkers": True,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
```

```
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": True,
    "enableLocation": True,
    "enableShowPoweredBy": True,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": True,
    "enableDisplayPreviousConversation": True,
    "enableDisplayConversationHistory": True,
    "enableConversationTimer": True,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": True,
    "enableSatisfactionSurvey": True,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": True,
    "enableDisplayFontSizeSwitch": True,
    "defaultThemeMode": {},
    "enableDownloadCitations": True,
    "customDomain": {
        "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": True,
    "enableDisplayChatbotName": True,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": True,
    "preChatFormFields": None,
    "preChatFormTitle": None,
    "preChatFormSubtitle": None,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": None,
    "loadingLongWaitCopyByLocale": None,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": True,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": None,
    "loadingTypingShowCopy": True,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'avatar' => 'https://example.com/file.jpg',
            'logo' => 'https://example.com/file.jpg',
            'description' => 'Example string',
            'cover' => 'https://example.com/file.jpg',
            'backUrl' => 'Example string',
            'isActive' => true,
            'theme' => [
                'primaryColor' => 'Example string',
                'navbarTextColor' => 'Example string',
                'conversationBackgroundColor' => 'Example string',
                'chatbotMessageTextColor' => 'Example string',
                'userMessageTextColor' => 'Example string',
                'chatbotMessageBackgroundColor' => 'Example string',
                'userMessageBackgroundColor' => 'Example string',
                'chatbotMessageBackgroundShadowEnabled' => true,
                'userMessageBackgroundShadowEnabled' => true,
                'chatbotMessageGradientEnabled' => true,
                'userMessageGradientEnabled' => true
            ],
            'darkTheme' => [
                'primaryColor' => 'Example string',
                'navbarTextColor' => 'Example string',
                'conversationBackgroundColor' => 'Example string',
                'chatbotMessageTextColor' => 'Example string',
                'userMessageTextColor' => 'Example string',
                'chatbotMessageBackgroundColor' => 'Example string',
                'userMessageBackgroundColor' => 'Example string',
                'chatbotMessageBackgroundShadowEnabled' => true,
                'userMessageBackgroundShadowEnabled' => true,
                'chatbotMessageGradientEnabled' => true,
                'userMessageGradientEnabled' => true
            ],
            'enableFileUpload' => true,
            'enableDisplayCitations' => true,
            'toolDisplayMode' => 'full',
            'enableAskOptionButtons' => true,
            'enableConversationHandoff' => true,
            'enableDisplayContextUsage' => true,
            'enableDisplayThinking' => true,
            'enableUserThinkingControl' => true,
            'enableDisplayQueryMetadata' => true,
```

```
            'enableDisplayHookMarkers' => true,
            'hookMarkerTextInputModify' => 'Example string',
            'hookMarkerTextOutputModify' => 'Example string',
            'hookMarkerTextInputBlock' => 'Example string',
            'hookMarkerTextOutputBlock' => 'Example string',
            'enableShareConversation' => true,
            'enableLocation' => true,
            'enableShowPoweredBy' => true,
            'poweredByLogo' => 'https://example.com/file.jpg',
            'enableAnonymous' => true,
            'enableDisplayPreviousConversation' => true,
            'enableDisplayConversationHistory' => true,
            'enableConversationTimer' => true,
            'conversationTimerDuration' => 123,
            'enableAutoNewConversationOnTimeout' => true,
            'enableSatisfactionSurvey' => true,
            'satisfactionSurveyDelayMinutes' => 123,
            'satisfactionSurveyPrompt' => 'Example string',
            'enableThemeModeToggle' => true,
            'enableDisplayFontSizeSwitch' => true,
            'defaultThemeMode' => new \stdClass(),
            'enableDownloadCitations' => true,
            'customDomain' => [
                'domain' => 'Example string'
            ],
            'enableDisplayChatbotAvatar' => true,
            'enableDisplayChatbotName' => true,
            'buttonIcon' => 'https://example.com/file.jpg',
            'conversationStartersDisplayCount' => 123,
            'preChatFormEnabled' => true,
            'preChatFormFields' => null,
            'preChatFormTitle' => null,
            'preChatFormSubtitle' => null,
            'loadingDisplayMode' => new \stdClass(),
            'loadingStageCopyByLocale' => null,
            'loadingLongWaitCopyByLocale' => null,
            'loadingLongWaitThresholdSeconds' => 123,
            'loadingAnimationUseSharedAsset' => true,
            'loadingAnimationSharedAssetUrl' => 'https://example.com/file.jpg',
            'loadingAnimationAssetUrlsByStage' => null,
            'loadingTypingShowCopy' => true,
            'toolCardRunningIconUrl' => 'https://example.com/file.jpg',
            'toolCardCompletedIconUrl' => 'https://example.com/file.jpg'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status code: `200`

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input field, which remains usable. When disabled, the existing overlay panel is
retained. This gradual rollout switch can be disabled at any time to revert the behavior. (Optional)
```

```
  "enableConversationHandoff"?: boolean // When enabled, this conversation platform's webchat
displays the handoff UI (notification bar, summary card, and resume button) and registers handoff
tools. It remains subject to the global handoff switch in system settings (when the global switch is
off, handoff is disabled everywhere). MaiGPT always displays the UI regardless of this switch. The
context usage percentage bar at the bottom is controlled separately by “Display context usage”.
Disabled by default, so customer-facing webchat displays no handoff UI. This gradual rollout switch
reverts the behavior immediately when disabled, without a version rollback. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, this conversation platform's webchat
displays a context usage percentage bar below the input field. It is independent of conversation
handoff: handoff switches (including the global handoff switch) do not affect it, and it does not
affect the handoff notification bar. MaiGPT always displays the bar regardless of this switch.
Disabled by default; disabling it immediately reverts the behavior without a version rollback.
(Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3 to 60 minutes
(Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI reply before the
survey appears, ranging from 1 to 60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend's
default localized text (Optional)
  "accessType":
  {
  }
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "id": string (uuid)
    "domain": string
    "status":
    {
    }
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
  "voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
  "conversationStartersDisplayCount"?: integer // Optional
```

```
  "includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
  "tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped — SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
    object
  ]
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
  "inbox": object
  "quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Response string",
```

```
  "cover": "https://example.com/file.jpg",
  "backUrl": "Response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "Response string"
  ],
  "theme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
    "userMessageBackgroundColor": "Response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "darkTheme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
    "userMessageBackgroundColor": "Response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "enableFileUpload": false,
  "enableDisplayCitations": false,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": false,
  "enableConversationHandoff": false,
  "enableDisplayContextUsage": false,
  "enableDisplayThinking": false,
  "enableUserThinkingControl": false,
  "enableDisplayQueryMetadata": false,
  "enableDisplayHookMarkers": false,
  "hookMarkerTextInputModify": "Response string",
  "hookMarkerTextOutputModify": "Response string",
  "hookMarkerTextInputBlock": "Response string",
  "hookMarkerTextOutputBlock": "Response string",
  "enableShareConversation": false,
  "enableLocation": false,
  "enableShowPoweredBy": false,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": false,
  "enableDisplayPreviousConversation": false,
  "enableDisplayConversationHistory": false,
  "enableConversationTimer": false,
  "conversationTimerDuration": 456,
  "enableAutoNewConversationOnTimeout": false,
  "enableSatisfactionSurvey": false,
  "satisfactionSurveyDelayMinutes": 456,
  "satisfactionSurveyPrompt": "Response string",
```

```
  "accessType": {},
  "enableThemeModeToggle": false,
  "enableDisplayFontSizeSwitch": false,
  "defaultThemeMode": {},
  "enableDownloadCitations": false,
  "customDomain": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "domain": "Response string",
    "status": {}
  },
  "enableDisplayChatbotAvatar": false,
  "enableDisplayChatbotName": false,
  "buttonIcon": "https://example.com/file.jpg",
  "enableCodeInterpreter": false,
  "voiceAgentEnabled": false,
  "conversationStartersDisplayCount": 456,
  "includeGreetingInContext": false,
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example Response Name",
      "description": "Example Response Description"
    }
  ],
  "preChatFormEnabled": false,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 456,
  "loadingAnimationUseSharedAsset": false,
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": false,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg",
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example Response Description"
  },
  "quickMenu": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example Response Description"
  }
}
```


Partially Update Web Chat

PATCH `/api/v1/web-chats/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this web chat.

Request Body

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	No	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	No	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	

Field	Type	Required	Description
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	`full` : Full ; `compact` : Compact ; `hidden` : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a row of buttons above the input box, which remains usable. When disabled, the existing overlay panel is retained. This rollout toggle can be disabled at any time to revert the behavior.
enableConversationHandoff	boolean	No	When enabled, this conversation platform’ s webchat displays the handoff UI (notification bar, summary card, and continue button) and registers handoff tools. It remains subject to the global handoff toggle in system settings: disabling the global toggle disables handoff everywhere. MaiGPT always displays it regardless of this platform toggle. The context usage percentage bar at the bottom is controlled separately by “Display context usage”. Disabled by default, so customer-facing webchats show no handoff UI. This rollout toggle can be disabled to revert the behavior without rolling back the release.
enableDisplayContextUsage	boolean	No	When enabled, this conversation platform’ s webchat displays a context usage percentage bar below the input box. This is independent of conversation handoff: handoff toggles, including the global handoff toggle, do not affect it, and it does not affect the handoff notification bar. MaiGPT always displays it regardless of this toggle. Disabled by default; disable it to revert the behavior without rolling back the release.
enableDisplayThinking	boolean	No	

Field	Type	Required	Description
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, ranging from 3-60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Number of minutes after the last AI reply before the survey appears, ranging from 1-60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text; leave blank to use the frontend’ s default localized text
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (contains 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	

Field	Type	Required	Description
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. `narrative` shows per-stage copy; `brand_animation` plays the uploaded asset and hides every system narration line. Defaults to `narrative` , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds `loading_long_wait_threshold_seconds` , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays `loading_animation_shared_asset_url` , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme"?: { // Optional
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme"?: { // Optional
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input box, which remains usable. When disabled, the existing overlay panel is
retained. This rollout toggle can be disabled at any time to revert the behavior. (Optional)
  "enableConversationHandoff"?: boolean // When enabled, this conversation platform’s webchat
displays the handoff UI (notification bar, summary card, and continue button) and registers handoff
tools. It remains subject to the global handoff toggle in system settings: disabling the global
toggle disables handoff everywhere. MaiGPT always displays it regardless of this platform toggle. The
context usage percentage bar at the bottom is controlled separately by “Display context usage”.
Disabled by default, so customer-facing webchats show no handoff UI. This rollout toggle can be
disabled to revert the behavior without rolling back the release. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, this conversation platform’s webchat
displays a context usage percentage bar below the input box. This is independent of conversation
handoff: handoff toggles, including the global handoff toggle, do not affect it, and it does not
affect the handoff notification bar. MaiGPT always displays it regardless of this toggle. Disabled by
default; disable it to revert the behavior without rolling back the release. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
```

```
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3-60 minutes
(Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI reply before the
survey appears, ranging from 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the frontend's
default localized text (Optional)
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "domain": string
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "conversationStartersDisplayCount"?: integer // Optional
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
```

```
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected. (Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
}
```

Example Request Values

```
{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableConversationHandoff": true,
  "enableDisplayContextUsage": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
```

```
  "hookMarkerTextInputBlock": "Example string",
  "hookMarkerTextOutputBlock": "Example string",
  "enableShareConversation": true,
  "enableLocation": true,
  "enableShowPoweredBy": true,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": true,
  "enableDisplayPreviousConversation": true,
  "enableDisplayConversationHistory": true,
  "enableConversationTimer": true,
  "conversationTimerDuration": 123,
  "enableAutoNewConversationOnTimeout": true,
  "enableSatisfactionSurvey": true,
  "satisfactionSurveyDelayMinutes": 123,
  "satisfactionSurveyPrompt": "Example string",
  "enableThemeModeToggle": true,
  "enableDisplayFontSizeSwitch": true,
  "defaultThemeMode": {},
  "enableDownloadCitations": true,
  "customDomain": {
    "domain": "Example string"
  },
  "enableDisplayChatbotAvatar": true,
  "enableDisplayChatbotName": true,
  "buttonIcon": "https://example.com/file.jpg",
  "conversationStartersDisplayCount": 123,
  "preChatFormEnabled": true,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 123,
  "loadingAnimationUseSharedAsset": true,
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": true,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "darkTheme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableConversationHandoff": true,
    "enableDisplayContextUsage": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
```

```
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "darkTheme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableConversationHandoff": true,
    "enableDisplayContextUsage": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
```

```
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

axios.patch("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": True,
    "theme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": True,
        "userMessageBackgroundShadowEnabled": True,
        "chatbotMessageGradientEnabled": True,
        "userMessageGradientEnabled": True
    },
    "darkTheme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": True,
        "userMessageBackgroundShadowEnabled": True,
        "chatbotMessageGradientEnabled": True,
        "userMessageGradientEnabled": True
    },
    "enableFileUpload": True,
    "enableDisplayCitations": True,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": True,
    "enableConversationHandoff": True,
    "enableDisplayContextUsage": True,
    "enableDisplayThinking": True,
    "enableUserThinkingControl": True,
    "enableDisplayQueryMetadata": True,
    "enableDisplayHookMarkers": True,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
```

```
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": True,
    "enableLocation": True,
    "enableShowPoweredBy": True,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": True,
    "enableDisplayPreviousConversation": True,
    "enableDisplayConversationHistory": True,
    "enableConversationTimer": True,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": True,
    "enableSatisfactionSurvey": True,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": True,
    "enableDisplayFontSizeSwitch": True,
    "defaultThemeMode": {},
    "enableDownloadCitations": True,
    "customDomain": {
        "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": True,
    "enableDisplayChatbotName": True,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": True,
    "preChatFormFields": None,
    "preChatFormTitle": None,
    "preChatFormSubtitle": None,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": None,
    "loadingLongWaitCopyByLocale": None,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": True,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": None,
    "loadingTypingShowCopy": True,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'avatar' => 'https://example.com/file.jpg',
            'logo' => 'https://example.com/file.jpg',
            'description' => 'Example string',
            'cover' => 'https://example.com/file.jpg',
            'backUrl' => 'Example string',
            'isActive' => true,
            'theme' => [
                'primaryColor' => 'Example string',
                'navbarTextColor' => 'Example string',
                'conversationBackgroundColor' => 'Example string',
                'chatbotMessageTextColor' => 'Example string',
                'userMessageTextColor' => 'Example string',
                'chatbotMessageBackgroundColor' => 'Example string',
                'userMessageBackgroundColor' => 'Example string',
                'chatbotMessageBackgroundShadowEnabled' => true,
                'userMessageBackgroundShadowEnabled' => true,
                'chatbotMessageGradientEnabled' => true,
                'userMessageGradientEnabled' => true
            ],
            'darkTheme' => [
                'primaryColor' => 'Example string',
                'navbarTextColor' => 'Example string',
                'conversationBackgroundColor' => 'Example string',
                'chatbotMessageTextColor' => 'Example string',
                'userMessageTextColor' => 'Example string',
                'chatbotMessageBackgroundColor' => 'Example string',
                'userMessageBackgroundColor' => 'Example string',
                'chatbotMessageBackgroundShadowEnabled' => true,
                'userMessageBackgroundShadowEnabled' => true,
                'chatbotMessageGradientEnabled' => true,
                'userMessageGradientEnabled' => true
            ],
            'enableFileUpload' => true,
            'enableDisplayCitations' => true,
            'toolDisplayMode' => 'full',
            'enableAskOptionButtons' => true,
            'enableConversationHandoff' => true,
            'enableDisplayContextUsage' => true,
            'enableDisplayThinking' => true,
            'enableUserThinkingControl' => true,
            'enableDisplayQueryMetadata' => true,
```

```
            'enableDisplayHookMarkers' => true,
            'hookMarkerTextInputModify' => 'Example string',
            'hookMarkerTextOutputModify' => 'Example string',
            'hookMarkerTextInputBlock' => 'Example string',
            'hookMarkerTextOutputBlock' => 'Example string',
            'enableShareConversation' => true,
            'enableLocation' => true,
            'enableShowPoweredBy' => true,
            'poweredByLogo' => 'https://example.com/file.jpg',
            'enableAnonymous' => true,
            'enableDisplayPreviousConversation' => true,
            'enableDisplayConversationHistory' => true,
            'enableConversationTimer' => true,
            'conversationTimerDuration' => 123,
            'enableAutoNewConversationOnTimeout' => true,
            'enableSatisfactionSurvey' => true,
            'satisfactionSurveyDelayMinutes' => 123,
            'satisfactionSurveyPrompt' => 'Example string',
            'enableThemeModeToggle' => true,
            'enableDisplayFontSizeSwitch' => true,
            'defaultThemeMode' => new \stdClass(),
            'enableDownloadCitations' => true,
            'customDomain' => [
                'domain' => 'Example string'
            ],
            'enableDisplayChatbotAvatar' => true,
            'enableDisplayChatbotName' => true,
            'buttonIcon' => 'https://example.com/file.jpg',
            'conversationStartersDisplayCount' => 123,
            'preChatFormEnabled' => true,
            'preChatFormFields' => null,
            'preChatFormTitle' => null,
            'preChatFormSubtitle' => null,
            'loadingDisplayMode' => new \stdClass(),
            'loadingStageCopyByLocale' => null,
            'loadingLongWaitCopyByLocale' => null,
            'loadingLongWaitThresholdSeconds' => 123,
            'loadingAnimationUseSharedAsset' => true,
            'loadingAnimationSharedAssetUrl' => 'https://example.com/file.jpg',
            'loadingAnimationAssetUrlsByStage' => null,
            'loadingTypingShowCopy' => true,
            'toolCardRunningIconUrl' => 'https://example.com/file.jpg',
            'toolCardCompletedIconUrl' => 'https://example.com/file.jpg'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status code: `200`

Example response schema

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input field, which remains usable. When disabled, the existing overlay panel is
retained. This gradual rollout toggle can be disabled at any time to revert the behavior. (Optional)
```

```
  "enableConversationHandoff"?: boolean // When enabled, this conversation platform's webchat
displays the handoff UI (notification bar, summary card, and continuation button) and registers
handoff tools. The global handoff toggle in system settings still applies: disabling it disables
handoff everywhere. MaiGPT always displays the UI regardless of this toggle. The context usage
percentage bar at the bottom is controlled separately by "Display Context Usage". Disabled by
default, so customer-facing webchat displays no handoff UI. This gradual rollout toggle lets you
revert the behavior by disabling it without rolling back the release. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, this conversation platform's webchat
displays a context usage percentage bar below the input field. This is independent of conversation
handoff: handoff toggles, including the global handoff toggle, do not affect it, and it does not
affect the handoff notification bar. MaiGPT always displays it regardless of this toggle. Disabled by
default; disabling it reverts the behavior without rolling back the release. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3-60 minutes
(Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
appears, ranging from 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the frontend's
default localized text (Optional)
  "accessType":
  {
  }
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "id": string (uuid)
    "domain": string
    "status":
    {
    }
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
  "voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
  "conversationStartersDisplayCount"?: integer // Optional
  "includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
```

```
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
  "tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped — SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
    object
  ]
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
  "inbox": object
  "quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Response string",
  "cover": "https://example.com/file.jpg",
```

```
  "backUrl": "Response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "Response string"
  ],
  "theme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
    "userMessageBackgroundColor": "Response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "darkTheme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
    "userMessageBackgroundColor": "Response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "enableFileUpload": false,
  "enableDisplayCitations": false,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": false,
  "enableConversationHandoff": false,
  "enableDisplayContextUsage": false,
  "enableDisplayThinking": false,
  "enableUserThinkingControl": false,
  "enableDisplayQueryMetadata": false,
  "enableDisplayHookMarkers": false,
  "hookMarkerTextInputModify": "Response string",
  "hookMarkerTextOutputModify": "Response string",
  "hookMarkerTextInputBlock": "Response string",
  "hookMarkerTextOutputBlock": "Response string",
  "enableShareConversation": false,
  "enableLocation": false,
  "enableShowPoweredBy": false,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": false,
  "enableDisplayPreviousConversation": false,
  "enableDisplayConversationHistory": false,
  "enableConversationTimer": false,
  "conversationTimerDuration": 456,
  "enableAutoNewConversationOnTimeout": false,
  "enableSatisfactionSurvey": false,
  "satisfactionSurveyDelayMinutes": 456,
  "satisfactionSurveyPrompt": "Response string",
  "accessType": {},
```

```
  "enableThemeModeToggle": false,
  "enableDisplayFontSizeSwitch": false,
  "defaultThemeMode": {},
  "enableDownloadCitations": false,
  "customDomain": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "domain": "Response string",
    "status": {}
  },
  "enableDisplayChatbotAvatar": false,
  "enableDisplayChatbotName": false,
  "buttonIcon": "https://example.com/file.jpg",
  "enableCodeInterpreter": false,
  "voiceAgentEnabled": false,
  "conversationStartersDisplayCount": 456,
  "includeGreetingInContext": false,
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "preChatFormEnabled": false,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 456,
  "loadingAnimationUseSharedAsset": false,
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": false,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg",
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "quickMenu": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```


Set Up Contact Credentials

POST `/api/v1/web-chats/{id}/setup-contact-credentials/`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this web chat.

Request Body

Request Parameters

Field	Type	Required	Description
sourceId	string	Yes	Unique identifier for the contact
name	string	No	Contact display name
mcpCredentials	object (contains 2 properties: toolId, headers)	No	MCP credentials configuration
mcpCredentials.toolId	string (uuid)	Yes	MCP Tool ID
mcpCredentials.headers	object	No	Authentication headers
embedOrigin	string	No	Origin of the embedding page (forwarded by the web chat frontend) for allowlist validation
ts	integer	No	Unix timestamp (seconds, UTC) the signature was computed with. Required when the inbox login settings enable Source ID access with signature verification.
sig	string	No	Hex HMAC-SHA256 of "{webChatId}.{sourceId}.{ts}" computed by the embedding site backend

Request Schema Example

```
{
  "sourceId": string // Unique identifier for the contact
  "name"?: string // Contact display name (Optional)
  "mcpCredentials"?:  // MCP credentials configuration (Optional)
  {
    "toolId": string (uuid) // MCP Tool ID
    "headers"?: object // Authentication headers (Optional)
  }
  "embedOrigin"?: string // Origin of the embedding page (forwarded by the web chat frontend) for allowlist validation (Optional)
  "ts"?: integer // Unix timestamp (seconds, UTC) the signature was computed with. Required when the inbox login settings enable Source ID access with signature verification. (Optional)
  "sig"?: string // Hex HMAC-SHA256 of "{webChatId}.{sourceId}.{ts}" computed by the embedding site backend (Optional)
}
```

Request Example Values

```
{
  "sourceId": "Example string",
  "name": "Example name",
  "mcpCredentials": {
    "toolId": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  },
  "embedOrigin": "Example string",
  "ts": 123,
  "sig": "Example string"
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-
446655440000/setup-contact-credentials/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "sourceId": "Example string",
    "name": "Example name",
    "mcpCredentials": {
      "toolId": "550e8400-e29b-41d4-a716-446655440000",
      "headers": {}
    },
    "embedOrigin": "Example string",
    "ts": 123,
    "sig": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "sourceId": "Example string",
    "name": "Example name",
    "mcpCredentials": {
      "toolId": "550e8400-e29b-41d4-a716-446655440000",
      "headers": {}
    },
    "embedOrigin": "Example string",
    "ts": 123,
    "sig": "Example string"
  };

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "sourceId": "Example string",
    "name": "Example name",
    "mcpCredentials": {
        "toolId": "550e8400-e29b-41d4-a716-446655440000",
        "headers": {}
    },
    "embedOrigin": "Example string",
    "ts": 123,
    "sig": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'sourceId' => 'Example string',
            'name' => 'Example name',
            'mcpCredentials' => [
                'toolId' => '550e8400-e29b-41d4-a716-446655440000',
                'headers' => new \stdClass()
            ],
            'embedOrigin' => 'Example string',
            'ts' => 123,
            'sig' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "contactId": string (uuid) // The created or existing contact ID
  "created": boolean // Whether the contact was created by this call. False means an existing contact matched the source_id; in that case the request name is not applied (update names via PATCH /contacts/{id}/).
}
```

Response Example Values


```
{
  "contactId": "550e8400-e29b-41d4-a716-446655440000",
  "created": false
}
```


Status Code: 400 - No response body

Get Voice Agent Token

POST `/api/v1/web-chats/{id}/voice-agent/token/`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this web chat.

Request Body

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	

Field	Type	Required	Description
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	`full` : Full ; `compact` : Compact ; `hidden` : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a row of buttons above the input field, which remains usable. When disabled, the existing overlay panel is retained. This gradual rollout switch can be turned off at any time to revert the behavior.
enableConversationHandoff	boolean	No	When enabled, this conversation platform's webchat displays the handoff UI (notification bar, summary card, and continuation button) and registers handoff tools. The global handoff switch in system settings still applies: when it is off, handoff is disabled everywhere. MaiGPT always displays the handoff UI regardless of this switch. The context

Field	Type	Required	Description
			usage percentage bar at the bottom is controlled separately by "Display context usage". Disabled by default, so customers' public webchats show no handoff UI. This gradual rollout switch can be turned off to revert the behavior without rolling back the release.
enableDisplayContextUsage	boolean	No	When enabled, this conversation platform's webchat displays a context usage percentage bar below the input field. It is independent of conversation handoff: handoff switches, including the global handoff switch, do not affect it, and it does not affect the handoff notification bar. MaiGPT always displays the bar regardless of this switch. Disabled by default; turn it off to revert the behavior without rolling back the release.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, from 3 to 60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Number of minutes after the last AI reply before the survey appears, from 1 to 60 minutes

Field	Type	Required	Description
satisfactionSurveyPrompt	string	No	Custom survey title text; leave blank to use the frontend's default localized text
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (contains 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. `narrative` shows per-stage copy; `brand_animation` plays the uploaded asset and hides every system narration line. Defaults to `narrative` , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds `loading_long_wait_threshold_seconds` , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays `loading_animation_shared_asset_url` , so the admin only uploads once. False

Field	Type	Required	Description
			expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
```

```
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input box, which remains usable. When disabled, the existing overlay panel is used.
This gradual rollout toggle can be disabled at any time to revert the behavior. (Optional)
  "enableConversationHandoff"?: boolean // When enabled, this conversation platform's webchat
displays the handoff UI (notification bar, summary card, and continuation button) and registers
handoff tools. The global handoff toggle in system settings still applies: disabling it disables
handoff everywhere. MaiGPT always displays the UI regardless of this toggle. The context usage
percentage bar at the bottom is controlled separately by "Display Context Usage". Disabled by
default, so customer-facing webchats display no handoff UI. This gradual rollout toggle reverts the
behavior when disabled, without rolling back the release. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, this conversation platform's webchat
displays a context usage percentage bar below the input box. This is independent of conversation
handoff: handoff toggles, including the global toggle, do not affect it, and it does not affect the
handoff notification bar. MaiGPT always displays the bar regardless of this toggle. Disabled by
default; disabling it reverts the behavior without rolling back the release. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3-60 minutes
(Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
appears, ranging from 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the frontend's
default localized text (Optional)
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "domain": string
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
```

```
  "conversationStartersDisplayCount"?: integer // Optional
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
}
```

Example Request Values

```
{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "Example string",
  "isActive": true,
  "theme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
```

```
  "darkTheme": {
    "primaryColor": "Example string",
    "navbarTextColor": "Example string",
    "conversationBackgroundColor": "Example string",
    "chatbotMessageTextColor": "Example string",
    "userMessageTextColor": "Example string",
    "chatbotMessageBackgroundColor": "Example string",
    "userMessageBackgroundColor": "Example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableConversationHandoff": true,
  "enableDisplayContextUsage": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "Example string",
  "hookMarkerTextOutputModify": "Example string",
  "hookMarkerTextInputBlock": "Example string",
  "hookMarkerTextOutputBlock": "Example string",
  "enableShareConversation": true,
  "enableLocation": true,
  "enableShowPoweredBy": true,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": true,
  "enableDisplayPreviousConversation": true,
  "enableDisplayConversationHistory": true,
  "enableConversationTimer": true,
  "conversationTimerDuration": 123,
  "enableAutoNewConversationOnTimeout": true,
  "enableSatisfactionSurvey": true,
  "satisfactionSurveyDelayMinutes": 123,
  "satisfactionSurveyPrompt": "Example string",
  "enableThemeModeToggle": true,
  "enableDisplayFontSizeSwitch": true,
  "defaultThemeMode": {},
  "enableDownloadCitations": true,
  "customDomain": {
    "domain": "Example string"
  },
  "enableDisplayChatbotAvatar": true,
  "enableDisplayChatbotName": true,
  "buttonIcon": "https://example.com/file.jpg",
  "conversationStartersDisplayCount": 123,
  "preChatFormEnabled": true,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 123,
  "loadingAnimationUseSharedAsset": true,
```

```
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": true,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-
446655440000/voice-agent/token/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "darkTheme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableConversationHandoff": true,
    "enableDisplayContextUsage": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
```

```
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": true,
    "theme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "darkTheme": {
      "primaryColor": "Example string",
      "navbarTextColor": "Example string",
      "conversationBackgroundColor": "Example string",
      "chatbotMessageTextColor": "Example string",
      "userMessageTextColor": "Example string",
      "chatbotMessageBackgroundColor": "Example string",
      "userMessageBackgroundColor": "Example string",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true,
      "chatbotMessageGradientEnabled": true,
      "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableConversationHandoff": true,
    "enableDisplayContextUsage": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
```

```
    "hookMarkerTextInputModify": "Example string",
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-
agent/token/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "Example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "Example string",
    "isActive": True,
    "theme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": True,
        "userMessageBackgroundShadowEnabled": True,
        "chatbotMessageGradientEnabled": True,
        "userMessageGradientEnabled": True
    },
    "darkTheme": {
        "primaryColor": "Example string",
        "navbarTextColor": "Example string",
        "conversationBackgroundColor": "Example string",
        "chatbotMessageTextColor": "Example string",
        "userMessageTextColor": "Example string",
        "chatbotMessageBackgroundColor": "Example string",
        "userMessageBackgroundColor": "Example string",
        "chatbotMessageBackgroundShadowEnabled": True,
        "userMessageBackgroundShadowEnabled": True,
        "chatbotMessageGradientEnabled": True,
        "userMessageGradientEnabled": True
    },
    "enableFileUpload": True,
    "enableDisplayCitations": True,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": True,
    "enableConversationHandoff": True,
    "enableDisplayContextUsage": True,
    "enableDisplayThinking": True,
    "enableUserThinkingControl": True,
    "enableDisplayQueryMetadata": True,
    "enableDisplayHookMarkers": True,
    "hookMarkerTextInputModify": "Example string",
```

```
    "hookMarkerTextOutputModify": "Example string",
    "hookMarkerTextInputBlock": "Example string",
    "hookMarkerTextOutputBlock": "Example string",
    "enableShareConversation": True,
    "enableLocation": True,
    "enableShowPoweredBy": True,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": True,
    "enableDisplayPreviousConversation": True,
    "enableDisplayConversationHistory": True,
    "enableConversationTimer": True,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": True,
    "enableSatisfactionSurvey": True,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "Example string",
    "enableThemeModeToggle": True,
    "enableDisplayFontSizeSwitch": True,
    "defaultThemeMode": {},
    "enableDownloadCitations": True,
    "customDomain": {
        "domain": "Example string"
    },
    "enableDisplayChatbotAvatar": True,
    "enableDisplayChatbotName": True,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": True,
    "preChatFormFields": None,
    "preChatFormTitle": None,
    "preChatFormSubtitle": None,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": None,
    "loadingLongWaitCopyByLocale": None,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": True,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": None,
    "loadingTypingShowCopy": True,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'avatar' => 'https://example.com/file.jpg',
            'logo' => 'https://example.com/file.jpg',
            'description' => 'Example string',
            'cover' => 'https://example.com/file.jpg',
            'backUrl' => 'Example string',
            'isActive' => true,
            'theme' => [
                'primaryColor' => 'Example string',
                'navbarTextColor' => 'Example string',
                'conversationBackgroundColor' => 'Example string',
                'chatbotMessageTextColor' => 'Example string',
                'userMessageTextColor' => 'Example string',
                'chatbotMessageBackgroundColor' => 'Example string',
                'userMessageBackgroundColor' => 'Example string',
                'chatbotMessageBackgroundShadowEnabled' => true,
                'userMessageBackgroundShadowEnabled' => true,
                'chatbotMessageGradientEnabled' => true,
                'userMessageGradientEnabled' => true
            ],
            'darkTheme' => [
                'primaryColor' => 'Example string',
                'navbarTextColor' => 'Example string',
                'conversationBackgroundColor' => 'Example string',
                'chatbotMessageTextColor' => 'Example string',
                'userMessageTextColor' => 'Example string',
                'chatbotMessageBackgroundColor' => 'Example string',
                'userMessageBackgroundColor' => 'Example string',
                'chatbotMessageBackgroundShadowEnabled' => true,
                'userMessageBackgroundShadowEnabled' => true,
                'chatbotMessageGradientEnabled' => true,
                'userMessageGradientEnabled' => true
            ],
            'enableFileUpload' => true,
            'enableDisplayCitations' => true,
            'toolDisplayMode' => 'full',
            'enableAskOptionButtons' => true,
            'enableConversationHandoff' => true,
            'enableDisplayContextUsage' => true,
            'enableDisplayThinking' => true,
            'enableUserThinkingControl' => true,
            'enableDisplayQueryMetadata' => true,
```

```
            'enableDisplayHookMarkers' => true,
            'hookMarkerTextInputModify' => 'Example string',
            'hookMarkerTextOutputModify' => 'Example string',
            'hookMarkerTextInputBlock' => 'Example string',
            'hookMarkerTextOutputBlock' => 'Example string',
            'enableShareConversation' => true,
            'enableLocation' => true,
            'enableShowPoweredBy' => true,
            'poweredByLogo' => 'https://example.com/file.jpg',
            'enableAnonymous' => true,
            'enableDisplayPreviousConversation' => true,
            'enableDisplayConversationHistory' => true,
            'enableConversationTimer' => true,
            'conversationTimerDuration' => 123,
            'enableAutoNewConversationOnTimeout' => true,
            'enableSatisfactionSurvey' => true,
            'satisfactionSurveyDelayMinutes' => 123,
            'satisfactionSurveyPrompt' => 'Example string',
            'enableThemeModeToggle' => true,
            'enableDisplayFontSizeSwitch' => true,
            'defaultThemeMode' => new \stdClass(),
            'enableDownloadCitations' => true,
            'customDomain' => [
                'domain' => 'Example string'
            ],
            'enableDisplayChatbotAvatar' => true,
            'enableDisplayChatbotName' => true,
            'buttonIcon' => 'https://example.com/file.jpg',
            'conversationStartersDisplayCount' => 123,
            'preChatFormEnabled' => true,
            'preChatFormFields' => null,
            'preChatFormTitle' => null,
            'preChatFormSubtitle' => null,
            'loadingDisplayMode' => new \stdClass(),
            'loadingStageCopyByLocale' => null,
            'loadingLongWaitCopyByLocale' => null,
            'loadingLongWaitThresholdSeconds' => 123,
            'loadingAnimationUseSharedAsset' => true,
            'loadingAnimationSharedAssetUrl' => 'https://example.com/file.jpg',
            'loadingAnimationAssetUrlsByStage' => null,
            'loadingTypingShowCopy' => true,
            'toolCardRunningIconUrl' => 'https://example.com/file.jpg',
            'toolCardCompletedIconUrl' => 'https://example.com/file.jpg'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input box, which remains usable. When disabled, the existing overlay panel is used.
This gradual rollout toggle can be disabled at any time to revert the behavior. (Optional)
```

```
  "enableConversationHandoff"?: boolean // When enabled, webchat on this conversation platform
displays the handoff UI (notification bar, summary card, and continue button) and registers handoff
tools. The global handoff toggle in system settings still applies: when it is off, handoff is
disabled everywhere. MaiGPT always displays the UI regardless of this toggle. The context usage
percentage bar at the bottom is controlled separately by "Display context usage". Disabled by
default, so customer-facing webchat shows no handoff UI. This gradual rollout toggle can be disabled
to revert the behavior without rolling back the release. (Optional)
  "enableDisplayContextUsage"?: boolean // When enabled, webchat on this conversation platform
displays a context usage percentage bar below the input box. This is independent of conversation
handoff: handoff toggles (including the global handoff toggle) do not affect it, and it does not
affect the handoff notification bar. MaiGPT always displays the bar regardless of this toggle.
Disabled by default; disabling it reverts the behavior without rolling back the release. (Optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, ranging from 3-60 minutes
(Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Number of minutes after the last AI reply before the
survey appears, ranging from 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave blank to use the default
localized text in the frontend (Optional)
  "accessType":
  {
  }
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?:  // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?:  // Optional
  {
    "id": string (uuid)
    "domain": string
    "status":
    {
    }
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
  "voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
  "conversationStartersDisplayCount"?: integer // Optional
  "includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
```

```
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
  "tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped — SDK
should never show an unnamed tool to a contact. Quick Menu tools are
static input-bar UI, never an LLM tool, so masking one is meaningless.
    object
  ]
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
  "inbox": object
  "quickMenu": object // Mounted Quick Menu Agent UI config for the web chat input bar; null when
absent (read-only)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "Response string",
  "cover": "https://example.com/file.jpg",
```

```
  "backUrl": "Response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "Response string"
  ],
  "theme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
    "userMessageBackgroundColor": "Response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "darkTheme": {
    "primaryColor": "Response string",
    "navbarTextColor": "Response string",
    "conversationBackgroundColor": "Response string",
    "chatbotMessageTextColor": "Response string",
    "userMessageTextColor": "Response string",
    "chatbotMessageBackgroundColor": "Response string",
    "userMessageBackgroundColor": "Response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "enableFileUpload": false,
  "enableDisplayCitations": false,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": false,
  "enableConversationHandoff": false,
  "enableDisplayContextUsage": false,
  "enableDisplayThinking": false,
  "enableUserThinkingControl": false,
  "enableDisplayQueryMetadata": false,
  "enableDisplayHookMarkers": false,
  "hookMarkerTextInputModify": "Response string",
  "hookMarkerTextOutputModify": "Response string",
  "hookMarkerTextInputBlock": "Response string",
  "hookMarkerTextOutputBlock": "Response string",
  "enableShareConversation": false,
  "enableLocation": false,
  "enableShowPoweredBy": false,
  "poweredByLogo": "https://example.com/file.jpg",
  "enableAnonymous": false,
  "enableDisplayPreviousConversation": false,
  "enableDisplayConversationHistory": false,
  "enableConversationTimer": false,
  "conversationTimerDuration": 456,
  "enableAutoNewConversationOnTimeout": false,
  "enableSatisfactionSurvey": false,
  "satisfactionSurveyDelayMinutes": 456,
  "satisfactionSurveyPrompt": "Response string",
  "accessType": {},
```

```
  "enableThemeModeToggle": false,
  "enableDisplayFontSizeSwitch": false,
  "defaultThemeMode": {},
  "enableDownloadCitations": false,
  "customDomain": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "domain": "Response string",
    "status": {}
  },
  "enableDisplayChatbotAvatar": false,
  "enableDisplayChatbotName": false,
  "buttonIcon": "https://example.com/file.jpg",
  "enableCodeInterpreter": false,
  "voiceAgentEnabled": false,
  "conversationStartersDisplayCount": 456,
  "includeGreetingInContext": false,
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "preChatFormEnabled": false,
  "preChatFormFields": null,
  "preChatFormTitle": null,
  "preChatFormSubtitle": null,
  "loadingDisplayMode": {},
  "loadingStageCopyByLocale": null,
  "loadingLongWaitCopyByLocale": null,
  "loadingLongWaitThresholdSeconds": 456,
  "loadingAnimationUseSharedAsset": false,
  "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
  "loadingAnimationAssetUrlsByStage": null,
  "loadingTypingShowCopy": false,
  "toolCardRunningIconUrl": "https://example.com/file.jpg",
  "toolCardCompletedIconUrl": "https://example.com/file.jpg",
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "quickMenu": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```


Upload a Batch QA File

POST `/api/v1/web-chats/{webchatPk}/batch-qas/`

Parameters

Parameter Name	Required	Type	Description
webchatPk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
file	string (uri)	Yes	

Example Request Schema

```
{
  "file": string (uri)
}
```

Example Request Values

```
{
  "file": "https://example.com/file.jpg"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-
446655440000/batch-qas/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "file": "https://example.com/file.jpg"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "file": "https://example.com/file.jpg"
  };

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-
qas/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "file": "https://example.com/file.jpg"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'file' => 'https://example.com/file.jpg'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "file": string (uri)
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "file": "https://example.com/file.jpg",
```

```
  "createdAt": "Response string"
}
```

Export Batch QA to Excel

GET `/api/v1/web-chats/{webchatPk}/batch-qas/{id}/export-excel/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Batch QA File.
`webchatPk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-
446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
200	Excel File Download

Get LLM Usage Statistics

GET `/api/v1/chatbots/{chatbotPk}/llm-usage-statistics/`

Parameters

Parameter Name	Required	Type	Description
`chatbotPk`	V	string	A UUID string identifying this Chatbot ID
`endDate`	X	string	End date, format: YYYY-MM-DD

`startDate`	V	string	Start date, format: YYYY-MM-DD
`timeGranularity`	V	string	Time granularity (day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
[
  {
    "llmDetails": [
      {
        "chatbotId": string (uuid)
        "wordsCount": integer
        "inputTokenUsage": integer
        "outputTokenUsage": integer
        "cacheCreationInputTokens": integer
        "cacheReadInputTokens": integer
        "totalTokenUsage": integer
        "llmModelId": string (uuid)
        "llmModelName": string
        "llmModelProvider": string
      }
    ]
    "date"?: string (date) // Date when time_granularity=day (optional)
    "month"?: string (date) // Month when time_granularity=month (first day of the month) (optional)
    "totalWordsCount": integer
    "totalInputTokenUsage": integer
    "totalOutputTokenUsage": integer
    "totalCacheCreationInputTokens": integer
    "totalCacheReadInputTokens": integer
    "totalTokenUsage": integer
  }
]
```

Example Response Values


```
[
  [
    {
      "totalWordsCount": 511,
      "totalInputTokenUsage": 1200,
      "totalOutputTokenUsage": 300,
      "totalCacheCreationInputTokens": 0,
      "totalCacheReadInputTokens": 0,
      "totalTokenUsage": 1500,
      "month": "2025-03-01",
      "llmDetails": [
        {
          "chatbotId": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
          "wordsCount": 511,
          "inputTokenUsage": 1200,
          "outputTokenUsage": 300,
          "cacheCreationInputTokens": 0,
          "cacheReadInputTokens": 0,
          "totalTokenUsage": 1500,
          "llmModelId": "d4282719-f50e-4a82-88d0-953e202a3383",
          "llmModelName": "GPT-4o-2024-08-06",
          "llmModelProvider": "openai"
        }
      ]
```

```
    }
  ]
]
```


Status Code: 400

Example Response Schema


```
{
  "detail"?: string // Optional
  "valid_types"?: [ // Optional
    string
  ]
}
```


Example Response Values


```
[
  [
    {
      "totalWordsCount": 511,
      "totalInputTokenUsage": 1200,
      "totalOutputTokenUsage": 300,
      "totalCacheCreationInputTokens": 0,
      "totalCacheReadInputTokens": 0,
      "totalTokenUsage": 1500,
      "month": "2025-03-01",
      "llmDetails": [
        {
          "chatbotId": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
          "wordsCount": 511,
          "inputTokenUsage": 1200,
          "outputTokenUsage": 300,
          "cacheCreationInputTokens": 0,
          "cacheReadInputTokens": 0,
          "totalTokenUsage": 1500,
          "llmModelId": "d4282719-f50e-4a82-88d0-953e202a3383",
          "llmModelName": "GPT-4o-2024-08-06",
          "llmModelProvider": "openai"
        }
      ]
    }
  ]
]
```


Get AI Assistant Statistics

GET `/api/v1/chatbots/{chatbotPk}/statistics/`

Parameters

Parameter Name	Required	Type	Description
`chatbotPk`	V	string	A UUID string identifying this Chatbot ID
`endDate`	X	string	End date (YYYY-MM-DD), defaults to today
`startDate`	X	string	Start date (YYYY-MM-DD), defaults to today
`timeGranularity`	V	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
[
  {
    "datetime": string // Timestamp (milliseconds, string)
    "messagesCount": integer // Message count
    "conversationsCount": integer // Conversation count
    "avgMessagesPerConversation": number (double) // Average messages per conversation
    "avgLikesPerFeedback": number (double) // Satisfaction: likes / (likes + dislikes), 0 when there is no feedback
    "likesCount": integer // Like count (excluding feedback on opening messages; daily/monthly values are totals for the period)
    "dislikesCount": integer // Dislike count (excluding feedback on opening messages; daily/monthly values are totals for the period)
  }
]
```

Example Response Values

```
[
  {
    "datetime": "Response string",
    "messagesCount": 456,
    "conversationsCount": 456,
    "avgMessagesPerConversation": 456,
    "avgLikesPerFeedback": 456,
    "likesCount": 456,
    "dislikesCount": 456
  }
]
```

API call example (Shell)

目錄

1. Send Message (Streaming)
2. Send Message (Create)
3. Send Proactive Message
4. Create New Conversation
5. List messages
6. Retrieve a specific message
7. Get Conversation List
8. Get a Specific Conversation
9. Rename a conversation
10. Delete Conversation
11. Stop AI Response
12. Submit Conversation Satisfaction
13. List Conversation Events
14. Assign a conversation to a member
15. Claim Conversation
16. Transfer a Conversation to Another Member
17. Return to AI Replies
18. List Conversation Output Files
19. Share Conversation
20. Quickly Share a Conversation
21. List Shared Conversations
22. Retrieve a Specific Shared Conversation
23. Update a Shared Conversation
24. Delete a Shared Conversation
25. Fork a Shared Conversation
26. Get My Shared Conversations
27. Create Message Feedback
28. Update Message Feedback
29. Delete Message Feedback
30. List Conversation Records
31. Retrieve a Specific Conversation Record
32. Create a Conversation Record Export Request
33. Get Conversation Record Export Status

API call example (Shell)

Send Message (Streaming)

`POST` `/api/v1/chatbots/{chatbotId}/completions/`

Parameters

Parameter Name	Required	Type	Description
`chatbotId`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	No	Unique conversation identifier; if empty, a new conversation is created (optional)
message	object (with 8 properties: content, contentPayload, queryMetadata...)	Yes	Message content to send
message.content	string	Yes	Text content of the message; can be an empty string if attachments are provided
message.contentPayload	object	No	Additional message content payload in JSON format (optional)
message.queryMetadata	object	No	Query metadata in JSON format (optional)
message.metadata	object	No	Message environment metadata, such as timezone information, in JSON format (optional)
message.attachments	array[AttachmentInput]	No	Message attachment list (optional)
message.toolIds	array[string]	No	List of tool IDs to enable for this message (optional)
message.clientTools	array[any]	No	Tool definitions provided by the client (such as a Chrome extension) for this message (optional). Each entry must contain name(str), description(str), and input_schema(dict). When the LLM calls a tool, the backend returns the clientToolCall content and ends the turn without executing the tool.
message.sender	string (uuid)	No	Sender Contact ID (optional)
isStreaming	boolean	No	Whether to respond in streaming mode; defaults to false (optional)
waitForAttachments	boolean	No	Whether to wait for attachment processing to complete before entering the agent workflow; defaults to true (optional)

Request Schema Example

```
{
  "conversation"?: string (uuid) // Unique conversation identifier; if empty, a new
conversation is created (optional) (Optional)
  "message":  // Message content to send
  {
    "content": string // Text content of the message; can be an empty string if attachments are
provided
    "contentPayload"?: object // Additional message content payload in JSON format (optional)
(Optional)
    "queryMetadata"?: object // Query metadata in JSON format (optional) (Optional)
    "metadata"?: object // Message environment metadata, such as timezone information, in JSON
format (optional) (Optional)
    "attachments"?: [ // Message attachment list (optional) (Optional)
      {
        "id": string (uuid) // Unique attachment identifier
        "type":  // Attachment type. Options: image (image), video (video, under development
and not yet supported), audio (audio), sticker (sticker, under development and not yet
supported), other (other)

* `image` - Image
* `video` - Video
* `audio` - Audio
* `sticker` - Sticker
* `other` - Other
        {
        }
        "filename": string // Attachment filename
        "file": string (uri) // Attachment file URL
      }
    ]
    "toolIds"?: [ // List of tool IDs to enable for this message (optional) (Optional)
      string (uuid)
    ]
    "clientTools"?: [ // Tool definitions provided by the client (such as a Chrome extension)
for this message (optional). Each entry must contain name(str), description(str), and
input_schema(dict). When the LLM calls a tool, the backend returns the clientToolCall content
and ends the turn without executing the tool. (Optional)
      object
    ]
    "sender"?: string (uuid) // Sender Contact ID (optional) (Optional)
  }
  "isStreaming"?: boolean // Whether to respond in streaming mode; defaults to false (optional)
(Optional)
  "waitForAttachments"?: boolean // Whether to wait for attachment processing to complete
before entering the agent workflow; defaults to true (optional) (Optional)
}
```

Example Request Values

```
{
  "message": {
    "content": "Hello, please introduce yourself"
```

```
  },
  "is_streaming": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "message": {
      "content": "Hello, please introduce yourself"
    },
    "is_streaming": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "message": {
      "content": "Hello, please introduce yourself"
    },
    "is_streaming": true
  };

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "message": {
        "content": "Hello, please introduce yourself"
    },
    "is_streaming": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'message' => [
                'content' => 'Hello, please introduce yourself'
            ],
            'is_streaming' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	Conversation response content; an event stream in streaming mode
400	Invalid request parameters; possible causes include a Contact ID that does not belong to the organization

Send Message (Create)

POST `/api/v1/messages/`

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	Yes	
type	string	No	
content	string	No	
contentPayload	object	No	
attachments	array[AttachmentCreateInput]	No	
canvas	object	No	
canvas.name	string	Yes	
canvas.canvasType	object	Yes	
canvas.title	string	Yes	
canvas.content	string	Yes	
slideTemplate	object (with 3 properties: slug, displayName, thumbnailUrls)	No	
slideTemplate.slug	string	Yes	
slideTemplate.displayName	string	Yes	
slideTemplate.thumbnailUrls	array[string]	No	
queryMetadata	object	No	Query metadata that controls the scope of knowledge available for querying; see each endpoint description for details
queryMetadata.knowledgeBases	array[object]	No	Knowledge bases available for querying; an empty array or null prevents all queries (no permissions), while omitting this field imposes no restriction
queryMetadata.labelRelations	object	No	Label filters; an empty conditions array means no label filtering (not a lack of

Field	Type	Required	Description
			permissions). Must be provided together with knowledgeBases. When provided alone, label filtering is not applied to knowledge base retrieval (equivalent to no restriction)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
metadata	object	No	Message metadata, such as timezone and other environment information
broadcast	boolean	No	
skipCopilotTrigger	boolean	No	
suggestedForId	string (uuid)	No	
skillIds	array[string]	No	Per-message selected skill IDs from the WebChat client (camelCase: skillIds).
toolIds	array[string]	No	Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
templateId	string (uuid)	No	Document/sheet template ID for file-level compilation (camelCase: templateId).

Request Schema Example

```
{
  "conversation": string (uuid)
  "type"?: string // Optional
  "content"?: string // Optional
  "contentPayload"?: object // Optional
  "attachments"?: [ // Optional
    {
      "type"?:  // Optional
      {
```

```
      }
      "filename": string
      "file": string (uri)
      "conversation"?: string (uuid) // Optional
    }
  ]
  "canvas"?: { // Optional
  {
    "name": string
    "canvasType":
    {
    }
    "title": string
    "content": string
  }
  }
  "slideTemplate"?:  // Optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // Optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge available for querying; see each endpoint description for details (Optional)
  {
    "knowledgeBases"?: [ // Knowledge bases available for querying; an empty array or null prevents all queries (no permissions), while omitting this field imposes no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // File ID list (excluded or exclusively selected depending on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // File ID list (the new name for chatbotFileIds; this field takes precedence when both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // FAQ ID list (excluded or exclusively selected depending on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true selects all knowledge base content except the listed items; false selects only the listed items (Optional)
      }
    ]
    "labelRelations"?: { // Label filters; an empty conditions array means no label filtering (not a lack of permissions). Must be provided together with knowledgeBases. When provided alone, label filtering is not applied to knowledge base retrieval (equivalent to no restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions combinations (Optional)
```

```
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Message metadata, such as timezone and other environment information
(Optional)
  "broadcast"?: boolean // Optional
  "skipCopilotTrigger"?: boolean // Optional
  "suggestedForId"?: string (uuid) // Optional
  "skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (Optional)
    string (uuid)
  ]
  "toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(Optional)
    string (uuid)
  ]
  "templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (Optional)
}
```

Example Request Values

```
{
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ],
    "metadata": null
  },
  "feedback": "like",
  "createdAt": "1748314926000",
  "attachments": [],
  "citations": [],
  "citationNodes": [],
  "canvas": {
    "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
    "name": "system-architecture-diagram",
    "canvasType": "markdown",
```

```
    "title": "System Architecture Diagram",
    "content": "~<del>mermaid\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API
Layer]\n    C --> D[Database]\n</del>~",
    "createdAt": "1748314926000"
  }
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
    "sender": {
      "id": 2.992461161002561e+38,
      "name": "jessiekuo",
      "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
    },
    "type": "incoming",
    "content": "This is a test message",
    "contentPayload": {
      "content": "This is a test message",
      "items": [
        {
          "type": "text",
          "text": "This is a test message",
          "startTimestamp": null,
          "citations": []
        }
      ],
      "metadata": null
    },
    "feedback": "like",
    "createdAt": "1748314926000",
    "attachments": [],
    "citations": [],
    "citationNodes": [],
    "canvas": {
      "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
      "name": "system-architecture-diagram",
      "canvasType": "markdown",
      "title": "System Architecture Diagram",
      "content": "~mermaid\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C --> D[Database]\n~",
      "createdAt": "1748314926000"
    }
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
    "sender": {
      "id": 2.992461161002561e+38,
      "name": "jessiekuo",
      "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
    },
    "type": "incoming",
    "content": "This is a test message",
    "contentPayload": {
      "content": "This is a test message",
      "items": [
        {
          "type": "text",
          "text": "This is a test message",
          "startTimestamp": null,
          "citations": []
        }
      ],
      "metadata": null
    },
    "feedback": "like",
    "createdAt": "1748314926000",
    "attachments": [],
    "citations": [],
    "citationNodes": [],
    "canvas": {
      "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
      "name": "system-architecture-diagram",
      "canvasType": "markdown",
      "title": "System Architecture Diagram",
      "content": "~mermaid\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C --> D[Database]\n~",
      "createdAt": "1748314926000"
    }
  };

axios.post("https://api.maiagent.ai/api/v1/messages/", data, config)
  .then(response => {
    console.log('Response received successfully:');
```

```
  console.log(response.data);
})
.catch(error => {
  console.error('Request error:');
  console.error(error.response?.data || error.message);
});
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
    "sender": {
        "id": 2.992461161002561e+38,
        "name": "jessiekuo",
        "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
    },
    "type": "incoming",
    "content": "This is a test message",
    "contentPayload": {
        "content": "This is a test message",
        "items": [
            {
                "type": "text",
                "text": "This is a test message",
                "startTimestamp": None,
                "citations": []
            }
        ],
        "metadata": None
    },
    "feedback": "like",
    "createdAt": "1748314926000",
    "attachments": [],
    "citations": [],
    "citationNodes": [],
    "canvas": {
        "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
        "name": "system-architecture-diagram",
        "canvasType": "markdown",
        "title": "System Architecture Diagram",
        "content": "~~mermaid\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C --> D[Database]\n~~",
        "createdAt": "1748314926000"
    }
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
```

```
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/messages/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'conversation' => 'c96a0fb9-d106-4b16-8706-3906533bafa2',
            'sender' => [
                'id' => 2.992461161002561e+38,
                'name' => 'jessiekuo',
                'avatar' => 'https://media-dev.maiagent.ai/static/images/default-user-
avatar.png'
            ],
            'type' => 'incoming',
            'content' => 'This is a test message',
            'contentPayload' => [
                'content' => 'This is a test message',
                'items' => [
                    [
                        'type' => 'text',
                        'text' => 'This is a test message',
                        'startTimestamp' => null,
                        'citations' => []
                    ]
                ],
                'metadata' => null
            ],
            'feedback' => 'like',
            'createdAt' => '1748314926000',
            'attachments' => [],
            'citations' => [],
            'citationNodes' => [],
            'canvas' => [
                'id' => '034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd',
                'name' => 'system-architecture-diagram',
                'canvasType' => 'markdown',
                'title' => 'System Architecture Diagram',
                'content' => '~~mermaid
graph TD
    A[User] --> B[Frontend Interface]
    B --> C[API Layer]
    C --> D[Database]
~~',
                'createdAt' => '1748314926000'
            ]
```

```
            ]
        ]);

        $data = json_decode($response->getBody(), true);
        echo "Response received successfully:\n";
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error: ' . $e->getMessage();
    }
    ?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email"?: string (email) // Optional
    "phoneNumber"?: string // Optional
  }
  "type"?: string // Optional
  "content"?: string // Optional
  "contentPayload"?: object // Optional
  "feedback":
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // Optional
    "updatedAt": string (timestamp)
  }
  "createdAt": string (timestamp)
  "attachments"?: [ // Optional
    {
      "id": string (uuid)
      "type"?:  // Optional
      {
      }
      "filename": string
```

```
      "file": string (uri)
      "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

For conversation-bound attachments: conversation.last_message_created_at + retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
      "conversation"?: string (uuid) // Optional
    }
  ]
  "citations": [
    {
      "id": string (uuid)
      "filename": string // Filename
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?:  // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John) (Optional)
          "displayName": string
        }
      ]
      "vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
      "chunksCount": integer // Number of chunks/nodes generated from this file
      "waitingTime": number (double)
      "processingTime": number (double)
      "processingTimeDetails": object
      "previewUrl": string // For presentations (pptx/ppt), returns the URL of a previewable
derived pdf file (displayed with the PDF viewer on the frontend); otherwise, returns None.
```

```
Use the same CustomizedFileFieldSerializer as the `file` field to generate URLs and ensure a
consistent format (including each
storage provider's presign/host rules). Do not use get_absolute_url(), which generates URLs in
an incorrect format.
      "createdAt": string (timestamp)
    }
  ]
  "citationNodes": [
    {
      "chatbotTextNode": {
      {
        "id": string (uuid)
        "charactersCount": integer
        "hitsCount": integer
        "text": string
        "updatedAt": string (timestamp)
        "filename": string
        "chatbotFile": object // Return complete ChatbotFile information, including the file
URL and type, to support image previews on the frontend
        "knowledgeBaseFile": object // Same as get_chatbot_file, for backward compatibility
        "pageNumber": integer // Backward-compatible alias of ``page_start``.
        "pageStart": integer
        "pageEnd": integer
        "citationTitle": string // Source title shown on the inline citation hover card.
        "citationDescription": string // Source description shown on the inline citation hover
card.
        "citationQuote": string // Quoted text snippet shown on the inline citation hover card.
        "hasImage": boolean // Determine whether an image is present (check the file type or
Markdown images in text)
        "imageUrl": string // Extract the image URL (prefer the file URL; otherwise extract it
from Markdown)
        "displayText": string // Return the full text with Markdown image markup removed
        "displayTitle": string // Return the display title (fallback: citation_title ->
filename)
        "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via ``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
        "labels": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "rawUserDefineMetadata"?: object // Metadata keys and values defined by the user.
(Optional)
        "metadataEnabledForSearch": object // Map each user-defined metadata key on this node
to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is True *and*
the node carries a value for it. Since this maps over ``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the node's
knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.
        "startCharIdx": integer
```

```
        "endCharIdx": integer
      }
      }
      "score"?: number (double) // Optional
      "displayScore": integer
      "citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply
content (Optional)
      "highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (Optional)
    }
  ]
  "canvas"?: { // Optional
  {
    "id": string (uuid)
    "name": string
    "canvasType":
    {
    }
    "title": string
    "content": string
    "createdAt": string (timestamp)
  }
  }
  "slideTemplate"?:  // Optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // Optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the scope of searchable knowledge; see
the description of each endpoint (Optional)
  {
    "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no access), while omission means no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true selects all knowledge base content except the
listed items; false selects only the listed items (Optional)
      }
    ]
    "labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (it does not mean no access). Must be provided together with knowledgeBases; when
provided alone, label filtering is not applied to knowledge base retrieval (equivalent to no
```

```
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions, supporting nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Message metadata, such as timezone and other environment information
(Optional)
  "recordId": string // Return the ChatbotRecord ID associated with the Message

Note:
    For an unsaved Message (such as a virtual Message during streaming), return None directly,
    to avoid triggering a DB query for the OneToOneField reverse relation (the N+1 problem).
  "broadcast"?: boolean // Optional
  "skipCopilotTrigger"?: boolean // Optional
  "activeRevisionNumber": integer
  "revisions": [
    {
      "id": string (uuid)
      "revisionNumber": integer
      "content": string
      "contentPayload": object
      "isActive": boolean
      "createdAt": string (timestamp)
    }
  ]
  "suggestedForId"?: string (uuid) // Optional
  "suggestionStatus": string
  "suggestionTriggerSource": string
  "suggestionError": string
  "retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
  "skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (Optional)
    string (uuid)
  ]
  "toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(Optional)
    string (uuid)
  ]
  "templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (Optional)
  "videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery;
``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
  "isGreeting": boolean // Whether this message is a greeting message.
}
```

Example Response Values

```
{
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ],
    "metadata": null
  },
  "feedback": "like",
  "createdAt": "1748314926000",
  "attachments": [],
  "citations": [],
  "citationNodes": [],
  "canvas": {
    "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
    "name": "system-architecture-diagram",
    "canvasType": "markdown",
    "title": "System Architecture Diagram",
    "content": "~<del>mermaid\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API
Layer]\n    C --> D[Database]\n</del>~",
    "createdAt": "1748314926000"
  }
}
```


Send Proactive Message

POST `/api/v1/messages/outgoing/`

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	Yes	
type	string	No	
content	string	No	
contentPayload	object	No	
attachments	array[AttachmentCreateInput]	No	
canvas	object	No	
canvas.name	string	Yes	
canvas.canvasType	object	Yes	
canvas.title	string	Yes	
canvas.content	string	Yes	
slideTemplate	object (with 3 properties: slug, displayName, thumbnailUrls)	No	
slideTemplate.slug	string	Yes	
slideTemplate.displayName	string	Yes	
slideTemplate.thumbnailUrls	array[string]	No	
queryMetadata	object	No	Query metadata that controls the scope of searchable knowledge; see the description of each endpoint
queryMetadata.knowledgeBases	array[object]	No	List of searchable knowledge bases; an empty array or null means none are searchable (no access), while omission means no restriction
queryMetadata.labelRelations	object	No	Label filter criteria; an empty conditions array means no label filtering (it does not mean no access). Must be provided together with knowledgeBases; when provided alone, label filtering is not applied to knowledge base retrieval (equivalent to no restriction)

Field	Type	Required	Description
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions, supporting nested operator/conditions combinations
metadata	object	No	Message metadata, such as timezone and other environment information
broadcast	boolean	No	
skipCopilotTrigger	boolean	No	
suggestedForId	string (uuid)	No	
skillIds	array[string]	No	Per-message selected skill IDs from the WebChat client (camelCase: skillIds).
toolIds	array[string]	No	Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
templateId	string (uuid)	No	Document/sheet template ID for file-level compilation (camelCase: templateId).

Example Request Schema

```
{
  "conversation": string (uuid)
  "type"?: string // Optional
  "content"?: string // Optional
  "contentPayload"?: object // Optional
  "attachments"?: [ // Optional
    {
      "type"?:  // Optional
      {
      }
      "filename": string
      "file": string (uri)
      "conversation"?: string (uuid) // Optional
    }
  ]
  "canvas"?: { // Optional
  {
    "name": string
```

```
    "canvasType":
    {
    }
    "title": string
    "content": string
  }
  }
  "slideTemplate"?:  // Optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // Optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the scope of searchable knowledge; see
the description of each endpoint (Optional)
  {
    "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no access), while omission means no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true selects all knowledge base content except the
listed items; false selects only the listed items (Optional)
      }
    ]
    "labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (it does not mean no access). Must be provided together with knowledgeBases; when
provided alone, label filtering is not applied to knowledge base retrieval (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions, supporting nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Message metadata, such as timezone and other environment information
(Optional)
  "broadcast"?: boolean // Optional
  "skipCopilotTrigger"?: boolean // Optional
```

```
  "suggestedForId"?: string (uuid) // Optional
  "skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (Optional)
    string (uuid)
  ]
  "toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(Optional)
    string (uuid)
  ]
  "templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (Optional)
}
```

Example Request Values


```
{
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "type": "Example string",
  "content": "Hello! I would like to learn about the product.",
  "contentPayload": null,
  "attachments": [
    {
      "type": {},
      "filename": "document.pdf",
      "file": "https://example.com/file.jpg",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "canvas": {
    "name": "Example name",
    "canvasType": {},
    "title": "Example name",
    "content": "Hello! I would like to learn about the product."
  },
  "slideTemplate": {
    "slug": "Example string",
    "displayName": "Example string",
    "thumbnailUrls": [
      "Example string"
    ]
  },
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
```

```
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "metadata": null,
  "broadcast": true,
  "skipCopilotTrigger": true,
  "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
  "skillIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "toolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "templateId": "550e8400-e29b-41d4-a716-446655440000"
}
```


Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/outgoing/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Example string",
    "content": "Hello! I would like to learn about the product.",
    "contentPayload": null,
    "attachments": [
      {
        "type": {},
        "filename": "document.pdf",
        "file": "https://example.com/file.jpg",
        "conversation": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "canvas": {
      "name": "Example name",
      "canvasType": {},
      "title": "Example name",
      "content": "Hello! I would like to learn about the product."
    },
    "slideTemplate": {
      "slug": "Example string",
      "displayName": "Example string",
      "thumbnailUrls": [
        "Example string"
      ]
    },
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
```

```
          "conditions": [
            null
          ]
        }
      },
      "metadata": null,
      "broadcast": true,
      "skipCopilotTrigger": true,
      "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
      "skillIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "toolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "templateId": "550e8400-e29b-41d4-a716-446655440000"
    }'

  # Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Example string",
    "content": "Hello! I would like to learn about the product.",
    "contentPayload": null,
    "attachments": [
      {
        "type": {},
        "filename": "document.pdf",
        "file": "https://example.com/file.jpg",
        "conversation": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "canvas": {
      "name": "Example name",
      "canvasType": {},
      "title": "Example name",
      "content": "Hello! I would like to learn about the product."
    },
    "slideTemplate": {
      "slug": "Example string",
      "displayName": "Example string",
      "thumbnailUrls": [
        "Example string"
      ]
    },
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
```

```
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "metadata": null,
    "broadcast": true,
    "skipCopilotTrigger": true,
    "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
    "skillIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "toolIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "templateId": "550e8400-e29b-41d4-a716-446655440000"
  };

axios.post("https://api.maiagent.ai/api/v1/messages/outgoing/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/outgoing/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Example string",
    "content": "Hello! I would like to learn about the product.",
    "contentPayload": None,
    "attachments": [
        {
            "type": {},
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg",
            "conversation": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "canvas": {
        "name": "Example name",
        "canvasType": {},
        "title": "Example name",
        "content": "Hello! I would like to learn about the product."
    },
    "slideTemplate": {
        "slug": "Example string",
        "displayName": "Example string",
        "thumbnailUrls": [
            "Example string"
        ]
    },
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": True
            }
        ],
```

```
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                None
            ]
        }
    },
    "metadata": None,
    "broadcast": True,
    "skipCopilotTrigger": True,
    "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
    "skillIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "toolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "templateId": "550e8400-e29b-41d4-a716-446655440000"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/messages/outgoing/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'conversation' => '550e8400-e29b-41d4-a716-446655440000',
            'type' => 'Example string',
            'content' => 'Hello! I would like to learn about the product.',
            'contentPayload' => null,
            'attachments' => [
                [
                    'type' => new \stdClass(),
                    'filename' => 'document.pdf',
                    'file' => 'https://example.com/file.jpg',
                    'conversation' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'canvas' => [
                'name' => 'Example name',
                'canvasType' => new \stdClass(),
                'title' => 'Example name',
                'content' => 'Hello! I would like to learn about the product.'
            ],
            'slideTemplate' => [
                'slug' => 'Example string',
                'displayName' => 'Example string',
                'thumbnailUrls' => [
                    'Example string'
                ]
            ],
            'queryMetadata' => [
                'knowledgeBases' => [
                    [
                        'knowledgeBaseId' => '550e8400-e29b-41d4-a716-446655440000',
                        'chatbotFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'knowledgeBaseFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'faqIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'hasUserSelectedAll' => true
```

```
                    ]
                ],
                'labelRelations' => [
                    'operator' => 'AND',
                    'conditions' => [
                        null
                    ]
                ]
            ],
            'metadata' => null,
            'broadcast' => true,
            'skipCopilotTrigger' => true,
            'suggestedForId' => '550e8400-e29b-41d4-a716-446655440000',
            'skillIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'toolIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'templateId' => '550e8400-e29b-41d4-a716-446655440000'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email"?: string (email) // Optional
    "phoneNumber"?: string // Optional
  }
```

```
  "type"?: string // Optional
  "content"?: string // Optional
  "contentPayload"?: object // Optional
  "feedback":
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // Optional
    "updatedAt": string (timestamp)
  }
  "createdAt": string (timestamp)
  "attachments"?: [ // Optional
    {
      "id": string (uuid)
      "type"?:  // Optional
      {
      }
      "filename": string
      "file": string (uri)
      "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

For conversation-bound attachments: conversation.last_message_created_at + retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
      "conversation"?: string (uuid) // Optional
    }
  ]
  "citations": [
    {
      "id": string (uuid)
      "filename": string // File name
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?:  // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
```

```
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John) (Optional)
          "displayName": string
        }
      ]
      "vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
      "chunksCount": integer // Number of chunks/nodes generated from this file
      "waitingTime": number (double)
      "processingTime": number (double)
      "processingTimeDetails": object
      "previewUrl": string // For presentations (pptx/ppt), returns the URL of a previewable
derived pdf file (displayed by the frontend PDF viewer); otherwise, returns None.

Generate the URL with the same CustomizedFileFieldSerializer used for the `file` field to
ensure a consistent format (including each
storage provider's presign/host rules). Do not use get_absolute_url(): it produces incorrectly
formatted URLs.
      "createdAt": string (timestamp)
    }
  ]
  "citationNodes": [
    {
      "chatbotTextNode": {
      {
        "id": string (uuid)
        "charactersCount": integer
        "hitsCount": integer
        "text": string
        "updatedAt": string (timestamp)
        "filename": string
        "chatbotFile": object // Returns complete ChatbotFile information, including the file
URL and type, to support frontend image previews
        "knowledgeBaseFile": object // Same as get_chatbot_file, for backward compatibility
        "pageNumber": integer // Backward-compatible alias of ``page_start``.
        "pageStart": integer
        "pageEnd": integer
        "citationTitle": string // Source title shown on the inline citation hover card.
        "citationDescription": string // Source description shown on the inline citation hover
card.
        "citationQuote": string // Quoted text snippet shown on the inline citation hover card.
        "hasImage": boolean // Determines whether an image is included (checks the file type or
Markdown images in text)
        "imageUrl": string // Extracts the image URL (prefers the file URL; otherwise, extracts
it from Markdown)
        "displayText": string // Returns the full text with Markdown image markup removed
        "displayTitle": string // Returns the display title (fallback: citation_title ->
filename)
        "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.
```

```
Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via ``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
        "labels": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "rawUserDefineMetadata"?: object // Metadata keys and values defined by the user.
(Optional)
        "metadataEnabledForSearch": object // Map each user-defined metadata key on this node
to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is True *and*
the node carries a value for it. Since this maps over ``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the node's
knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.
        "startCharIdx": integer
        "endCharIdx": integer
      }
      }
      "score"?: number (double) // Optional
      "displayScore": integer
      "citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply
content (Optional)
      "highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (Optional)
    }
  ]
  "canvas"?: { // Optional
  {
    "id": string (uuid)
    "name": string
    "canvasType":
    {
    }
    "title": string
    "content": string
    "createdAt": string (timestamp)
  }
  }
  "slideTemplate"?:  // Optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // Optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the scope of searchable knowledge; see
each endpoint description for details (Optional)
  {
    "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no access), while omission means no restriction (Optional)
      {
```

```
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true selects all knowledge base content except the
listed items; false selects only the listed items (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array means no label
filtering (it does not mean no access). Must be provided together with knowledgeBases; when
provided alone, label filtering is not applied to knowledge base retrieval (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions, supporting nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Message metadata, such as timezone and other environment information
(Optional)
  "recordId": string // Returns the ChatbotRecord ID corresponding to the Message

Note:
    For an unsaved Message (such as a virtual Message during streaming), returns None directly,
    avoiding a DB query through the OneToOneField reverse relation (the N+1 problem).
  "broadcast"?: boolean // Optional
  "skipCopilotTrigger"?: boolean // Optional
  "activeRevisionNumber": integer
  "revisions": [
    {
      "id": string (uuid)
      "revisionNumber": integer
      "content": string
      "contentPayload": object
      "isActive": boolean
      "createdAt": string (timestamp)
    }
  ]
  "suggestedForId"?: string (uuid) // Optional
  "suggestionStatus": string
  "suggestionTriggerSource": string
  "suggestionError": string
  "retryCount": integer // How many times the LLM call was auto-retried during this message
```

```
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
  "skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (Optional)
    string (uuid)
  ]
  "toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(Optional)
    string (uuid)
  ]
  "templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (Optional)
  "videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery;
``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
  "isGreeting": boolean // Whether this message is a greeting message.
}
```

Example response values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "sender": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
    "phoneNumber": "Response string"
  },
  "type": "Response string",
  "content": "Response string",
  "contentPayload": null,
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "Response string",
    "updatedAt": "Response string"
  },
  "createdAt": "Response string",
  "attachments": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "type": {},
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "expiresAt": "Response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "citations": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "fileType": "Response string",
```

```
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "provider": {},
        "isTimestampSttProvider": false
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "speakerLabels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "originalLabel": "Response string",
          "customName": "Response string",
          "displayName": "Response string"
        }
      ],
      "vectorStorageSize": 456,
      "chunksCount": 456,
      "waitingTime": 456,
      "processingTime": 456,
      "processingTimeDetails": null,
      "previewUrl": "Response string",
      "createdAt": "Response string"
    }
  ],
  "citationNodes": [
    {
      "chatbotTextNode": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "charactersCount": 456,
        "hitsCount": 456,
        "text": "Response string",
        "updatedAt": "Response string",
        "filename": "Response string",
        "chatbotFile": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response name",
          "description": "Example response description"
        },
        "knowledgeBaseFile": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response name",
          "description": "Example response description"
        },
        "pageNumber": 456,
```

```
        "pageStart": 456,
        "pageEnd": 456,
        "citationTitle": "Response string",
        "citationDescription": "Response string",
        "citationQuote": "Response string",
        "hasImage": false,
        "imageUrl": "Response string",
        "displayText": "Response string",
        "displayTitle": "Response string",
        "highlightedText": "Response string",
        "labels": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ],
        "rawUserDefineMetadata": null,
        "metadataEnabledForSearch": {
          "key1": "value1",
          "key2": "value2",
          "createdAt": "2025-01-01T00:00:00.000Z"
        },
        "startCharIdx": 456,
        "endCharIdx": 456
      },
      "score": 456,
      "displayScore": 456,
      "citationNumber": 456,
      "highlightedText": "Response string"
    }
  ],
  "canvas": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "canvasType": {},
    "title": "Response string",
    "content": "Response string",
    "createdAt": "Response string"
  },
  "slideTemplate": {
    "slug": "Response string",
    "displayName": "Response string",
    "thumbnailUrls": [
      "Response string"
    ]
  },
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "recordId": "Response string",
  "broadcast": false,
  "skipCopilotTrigger": false,
  "activeRevisionNumber": 456,
```

```
  "revisions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "revisionNumber": 456,
      "content": "Response string",
      "contentPayload": null,
      "isActive": false,
      "createdAt": "Response string"
    }
  ],
  "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
  "suggestionStatus": "Response string",
  "suggestionTriggerSource": "Response string",
  "suggestionError": "Response string",
  "retryCount": 456,
  "skillIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "toolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "templateId": "550e8400-e29b-41d4-a716-446655440000",
  "videoGeneration": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "isGreeting": false
}
```


Create New Conversation

POST `/api/v1/conversations/`

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{}'

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {};

axios.post("https://api.maiagent.ai/api/v1/conversations/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/conversations/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => new \stdClass()
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 201

Example Response Structure

```
{
  "id": string (uuid)
  "contact":
  {
    "id": string (uuid)
    "name": string
    "sourceId"?: string // Optional
  }
  "lastMessageCreatedAt": string (timestamp)
  "autoReplyEnabled": boolean
  "thinkingConfigSchema": { // Thinking settings form for this model; null means this model has
no thinking settings
  {
    "type"?: string // Form type, always reasoning_effort (Optional)
```

```
    "fields"?: object // reasoning_effort field: options lists the thinking levels this model
accepts (weakest to strongest); default is always null (Optional)
    "canDisable"?: boolean // Whether this model can turn thinking off; when true, options
includes off (Optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model thinks on its own, or the platform default level enables thinking
(Optional)
    "unsetLevel"?: string // Level the platform actually sends when none is set; null means no
thinking parameter is sent and the model's own default applies (Optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // May have different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled = platform switch; a continuation
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
  "title": string
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "sourceId": "Response string"
```

```
  },
  "lastMessageCreatedAt": "Response string",
  "autoReplyEnabled": false,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "displayContextUsage": false,
  "title": "Response string"
}
```


List messages

`GET` `/api/v1/messages/`

Parameters

Parameter name	Required	Type	Description
`conversation`	V	string	Conversation ID
`cursor`	X	string	The pagination cursor value.
`pageSize`	X	integer	Number of results to return per page.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response content

Status code: `200`

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "conversation": string (uuid)
      "sender":
      {
        "id": string (uuid)
        "name": string
        "avatar": string
        "email"?: string (email) // Optional
        "phoneNumber"?: string // Optional
      }
      "type"?: string // Optional
      "content"?: string // Optional
      "contentPayload"?: object // Optional
      "feedback":
      {
        "id": string (uuid)
        "type":
        {
        }
        "suggestion"?: string // Optional
        "updatedAt": string (timestamp)
      }
      "createdAt": string (timestamp)
      "attachments"?: [ // Optional
        {
          "id": string (uuid)
          "type"?:  // Optional
          {
          }
          "filename": string
          "file": string (uri)
          "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

For conversation-bound attachments: conversation.last_message_created_at + retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
          "conversation"?: string (uuid) // Optional
        }
      ]
      "citations": [
        {
```

```
          "id": string (uuid)
          "filename": string // Filename
          "file": string (uri) // File to upload
          "fileType": string
          "knowledgeBase"?:  // Optional
          {
            "id": string (uuid)
            "name": string
          }
          "size": integer
          "status":
          {
          }
          "parser":
          {
            "id": string (uuid)
            "name": string
            "provider":
            {
            }
            "isTimestampSttProvider": boolean
          }
          "labels"?: [ // Optional
            {
              "id": string (uuid)
              "name": string
            }
          ]
          "rawUserDefineMetadata"?: object // Optional
          "speakerLabels": [
            {
              "id": string (uuid)
              "originalLabel": string // Original speaker label from diarization (e.g.,
SPEAKER_00)
              "customName"?: string // User-defined speaker name (e.g., John) (Optional)
              "displayName": string
            }
          ]
          "vectorStorageSize": integer // Size of vectors for this file in Elasticsearch
(bytes)
          "chunksCount": integer // Number of chunks/nodes generated from this file
          "waitingTime": number (double)
          "processingTime": number (double)
          "processingTimeDetails": object
          "previewUrl": string // Returns the URL of a previewable PDF derivative for
presentations (pptx/ppt), displayed in the frontend PDF viewer; otherwise returns None.

Generate the URL using the same CustomizedFileFieldSerializer as the `file` field to ensure a
consistent format (including each
storage provider's presign/host rules). Do not use get_absolute_url(): it produces incorrectly
formatted URLs.
          "createdAt": string (timestamp)
        }
      ]
      "citationNodes": [
        {
```

```
          "chatbotTextNode": {
          {
            "id": string (uuid)
            "charactersCount": integer
            "hitsCount": integer
            "text": string
            "updatedAt": string (timestamp)
            "filename": string
            "chatbotFile": object // Returns complete ChatbotFile information, including the
file URL and type, to support frontend image previews
            "knowledgeBaseFile": object // Same as get_chatbot_file, for backward compatibility
            "pageNumber": integer // Backward-compatible alias of ``page_start``.
            "pageStart": integer
            "pageEnd": integer
            "citationTitle": string // Source title shown on the inline citation hover card.
            "citationDescription": string // Source description shown on the inline citation
hover card.
            "citationQuote": string // Quoted text snippet shown on the inline citation hover
card.
            "hasImage": boolean // Determines whether an image is present (checks the file type
or Markdown images in text)
            "imageUrl": string // Extracts the image URL (prefers the file URL; otherwise
extracts it from Markdown)
            "displayText": string // Returns the full text with Markdown image markup removed
            "displayTitle": string // Returns the display title (fallback: citation_title ->
filename)
            "highlightedText": string // Return text with matched terms wrapped in ``<mark>``
tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via ``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
            "labels": [
              {
                "id": string (uuid)
                "name": string
              }
            ]
            "rawUserDefineMetadata"?: object // Metadata keys and values defined by the user.
(Optional)
            "metadataEnabledForSearch": object // Map each user-defined metadata key on this
node to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is True *and*
the node carries a value for it. Since this maps over ``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the node's
knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.
            "startCharIdx": integer
            "endCharIdx": integer
          }
          }
          "score"?: number (double) // Optional
          "displayScore": integer
          "citationNumber"?: integer // Citation number matching the [1], [2] markers in the
reply content (Optional)
```

```
          "highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags
(Optional)
        }
      ]
      "canvas"?: { // Optional
      {
        "id": string (uuid)
        "name": string
        "canvasType":
        {
        }
        "title": string
        "content": string
        "createdAt": string (timestamp)
      }
      }
      "slideTemplate"?:  // Optional
      {
        "slug": string
        "displayName": string
        "thumbnailUrls"?: [ // Optional
          string
        ]
      }
      "queryMetadata"?: { // Query metadata controlling the scope of searchable knowledge; see
each endpoint description for details (Optional)
      {
        "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null
means none are searchable (no access), while omission means no restriction (Optional)
          {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected
according to hasUserSelectedAll) (Optional)
              string (uuid)
            ]
            "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds;
this field takes precedence when both are provided) (Optional)
              string (uuid)
            ]
            "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected according to
hasUserSelectedAll) (Optional)
              string (uuid)
            ]
            "hasUserSelectedAll"?: boolean // true selects all knowledge base content except
the listed items; false selects only the listed items (Optional)
          }
        ]
        "labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not a lack of access). Must be provided together with knowledgeBases; when provided
alone, knowledge base retrieval does not apply label filtering (equivalent to no restriction)
(Optional)
        {
          "operator": string (enum: AND, OR) // How label conditions are combined
          "conditions"?: [ // List of label conditions, supporting nested operator/conditions
combinations (Optional)
            object
```

```
          ]
        }
        }
      }
      }
      "metadata"?: object // Message metadata, such as timezone and other environment
information (Optional)
      "recordId": string // Returns the ChatbotRecord ID associated with the Message

Note:
    For unsaved Messages (such as virtual Messages during streaming), return None directly,
    to avoid triggering a DB query for the OneToOneField reverse relation (the N+1 problem).
      "broadcast"?: boolean // Optional
      "skipCopilotTrigger"?: boolean // Optional
      "activeRevisionNumber": integer
      "revisions": [
        {
          "id": string (uuid)
          "revisionNumber": integer
          "content": string
          "contentPayload": object
          "isActive": boolean
          "createdAt": string (timestamp)
        }
      ]
      "suggestedForId"?: string (uuid) // Optional
      "suggestionStatus": string
      "suggestionTriggerSource": string
      "suggestionError": string
      "retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
      "skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (Optional)
        string (uuid)
      ]
      "toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase:
toolIds). (Optional)
        string (uuid)
      ]
      "templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (Optional)
      "videoGeneration": object // Return the latest GeneratedVideo state for FE reload
recovery;
``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
      "isGreeting": boolean // Whether this message is a greeting message.
    }
  ]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
```

```
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "conversation": "550e8400-e29b-41d4-a716-446655440000",
      "sender": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "avatar": "Response string",
        "email": "response@example.com",
        "phoneNumber": "Response string"
      },
      "type": "Response string",
      "content": "Response string",
      "contentPayload": null,
      "feedback": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "type": {},
        "suggestion": "Response string",
        "updatedAt": "Response string"
      },
      "createdAt": "Response string",
      "attachments": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "type": {},
          "filename": "Response string",
          "file": "https://example.com/file.jpg",
          "expiresAt": "Response string",
          "conversation": "550e8400-e29b-41d4-a716-446655440000"
        }
      ],
      "citations": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "filename": "Response string",
          "file": "https://example.com/file.jpg",
          "fileType": "Response string",
          "knowledgeBase": {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          },
          "size": 456,
          "status": {},
          "parser": {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string",
            "provider": {},
            "isTimestampSttProvider": false
          },
          "labels": [
            {
              "id": "550e8400-e29b-41d4-a716-446655440000",
              "name": "Response string"
            }
          ],
```

```
      "rawUserDefineMetadata": null,
      "speakerLabels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "originalLabel": "Response string",
          "customName": "Response string",
          "displayName": "Response string"
        }
      ],
      "vectorStorageSize": 456,
      "chunksCount": 456,
      "waitingTime": 456,
      "processingTime": 456,
      "processingTimeDetails": null,
      "previewUrl": "Response string",
      "createdAt": "Response string"
    }
  ],
  "citationNodes": [
    {
      "chatbotTextNode": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "charactersCount": 456,
        "hitsCount": 456,
        "text": "Response string",
        "updatedAt": "Response string",
        "filename": "Response string",
        "chatbotFile": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response name",
          "description": "Example response description"
        },
        "knowledgeBaseFile": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response name",
          "description": "Example response description"
        },
        "pageNumber": 456,
        "pageStart": 456,
        "pageEnd": 456,
        "citationTitle": "Response string",
        "citationDescription": "Response string",
        "citationQuote": "Response string",
        "hasImage": false,
        "imageUrl": "Response string",
        "displayText": "Response string",
        "displayTitle": "Response string",
        "highlightedText": "Response string",
        "labels": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ],
        "rawUserDefineMetadata": null,
        "metadataEnabledForSearch": {
```

```
            "key1": "value1",
            "key2": "value2",
            "createdAt": "2025-01-01T00:00:00.000Z"
          },
          "startCharIdx": 456,
          "endCharIdx": 456
        },
        "score": 456,
        "displayScore": 456,
        "citationNumber": 456,
        "highlightedText": "Response string"
      }
    ],
    "canvas": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "canvasType": {},
      "title": "Response string",
      "content": "Response string",
      "createdAt": "Response string"
    },
    "slideTemplate": {
      "slug": "Response string",
      "displayName": "Response string",
      "thumbnailUrls": [
        "Response string"
      ]
    },
    "queryMetadata": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    },
    "metadata": null,
    "recordId": "Response string",
    "broadcast": false,
    "skipCopilotTrigger": false,
    "activeRevisionNumber": 456,
    "revisions": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "revisionNumber": 456,
        "content": "Response string",
        "contentPayload": null,
        "isActive": false,
        "createdAt": "Response string"
      }
    ],
    "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
    "suggestionStatus": "Response string",
    "suggestionTriggerSource": "Response string",
    "suggestionError": "Response string",
    "retryCount": 456,
    "skillIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
```

```
      "toolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "templateId": "550e8400-e29b-41d4-a716-446655440000",
      "videoGeneration": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "isGreeting": false
    }
  ]
}
```


Retrieve a specific message

GET `/api/v1/messages/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this Message.
`conversation`	V	string	Conversation ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-
446655440000/?conversation=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-446655440000/?
conversation=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-446655440000/?
conversation=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-
a716-446655440000/?conversation=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: `200`

Example response schema

```
{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email"?: string (email) // Optional
    "phoneNumber"?: string // Optional
  }
  "type"?: string // Optional
  "content"?: string // Optional
  "contentPayload"?: object // Optional
  "feedback":
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // Optional
    "updatedAt": string (timestamp)
  }
  "createdAt": string (timestamp)
  "attachments"?: [ // Optional
    {
      "id": string (uuid)
      "type"?:  // Optional
      {
      }
      "filename": string
      "file": string (uri)
      "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

For conversation-bound attachments: conversation.last_message_created_at + retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
      "conversation"?: string (uuid) // Optional
    }
  ]
  "citations": [
    {
      "id": string (uuid)
      "filename": string // Filename
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?:  // Optional
```

```
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John) (Optional)
          "displayName": string
        }
      ]
      "vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
      "chunksCount": integer // Number of chunks/nodes generated from this file
      "waitingTime": number (double)
      "processingTime": number (double)
      "processingTimeDetails": object
      "previewUrl": string // Returns the URL of a previewable PDF derivative for presentations
(pptx/ppt), displayed in the frontend PDF viewer; otherwise returns None.

Generate the URL using the same CustomizedFileFieldSerializer as the `file` field to ensure a
consistent format (including each
storage provider's presign/host rules). Do not use get_absolute_url(): it produces incorrectly
formatted URLs.
      "createdAt": string (timestamp)
    }
  ]
  "citationNodes": [
    {
      "chatbotTextNode": {
      {
        "id": string (uuid)
        "charactersCount": integer
        "hitsCount": integer
        "text": string
        "updatedAt": string (timestamp)
```

```
        "filename": string
        "chatbotFile": object // Returns complete ChatbotFile information, including the file
URL and type, to support frontend image previews
        "knowledgeBaseFile": object // Same as get_chatbot_file, for backward compatibility
        "pageNumber": integer // Backward-compatible alias of ``page_start``.
        "pageStart": integer
        "pageEnd": integer
        "citationTitle": string // Source title shown on the inline citation hover card.
        "citationDescription": string // Source description shown on the inline citation hover
card.
        "citationQuote": string // Quoted text snippet shown on the inline citation hover card.
        "hasImage": boolean // Determines whether an image is present (checks the file type or
Markdown images in text)
        "imageUrl": string // Extracts the image URL (prefers the file URL; otherwise extracts
it from Markdown)
        "displayText": string // Returns the full text with Markdown image markup removed
        "displayTitle": string // Returns the display title (fallback: citation_title ->
filename)
        "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via ``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
        "labels": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "rawUserDefineMetadata"?: object // Metadata keys and values defined by the user.
(Optional)
        "metadataEnabledForSearch": object // Map each user-defined metadata key on this node
to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is True *and*
the node carries a value for it. Since this maps over ``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the node's
knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.
        "startCharIdx": integer
        "endCharIdx": integer
      }
      }
      "score"?: number (double) // Optional
      "displayScore": integer
      "citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply
content (Optional)
      "highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (Optional)
    }
  ]
  "canvas"?: { // Optional
  {
    "id": string (uuid)
    "name": string
    "canvasType":
    {
```

```
    }
    "title": string
    "content": string
    "createdAt": string (timestamp)
  }
  }
  "slideTemplate"?:  // Optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // Optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each endpoint description for details (Optional)
  {
    "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this field takes precedence when both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all knowledge base content except the listed items; false = select only the listed items (Optional)
      }
    ]
    "labelRelations"?: { // Label filter criteria; an empty conditions array means no label filtering (not a lack of permission). Must be provided together with knowledgeBases; when provided alone, label filtering is not applied to knowledge base retrieval (equivalent to no restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How label criteria are combined
      "conditions"?: [ // List of label criteria; supports nested operator/conditions combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Message metadata, such as timezone and other environment information (Optional)
  "recordId": string // Return the ChatbotRecord ID associated with the Message

Note:
```

```
    For an unsaved Message (such as a virtual Message during streaming), return None directly
    to avoid triggering a DB query for the OneToOneField reverse relation (the N+1 problem).
  "broadcast"?: boolean // Optional
  "skipCopilotTrigger"?: boolean // Optional
  "activeRevisionNumber": integer
  "revisions": [
    {
      "id": string (uuid)
      "revisionNumber": integer
      "content": string
      "contentPayload": object
      "isActive": boolean
      "createdAt": string (timestamp)
    }
  ]
  "suggestedForId"?: string (uuid) // Optional
  "suggestionStatus": string
  "suggestionTriggerSource": string
  "suggestionError": string
  "retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
  "skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (Optional)
    string (uuid)
  ]
  "toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(Optional)
    string (uuid)
  ]
  "templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (Optional)
  "videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery;
``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
  "isGreeting": boolean // Whether this message is a greeting message.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "sender": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
    "phoneNumber": "Response string"
  },
  "type": "Response string",
  "content": "Response string",
  "contentPayload": null,
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
```

```
      "suggestion": "Response string",
      "updatedAt": "Response string"
    },
    "createdAt": "Response string",
    "attachments": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "type": {},
        "filename": "Response string",
        "file": "https://example.com/file.jpg",
        "expiresAt": "Response string",
        "conversation": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "citations": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "filename": "Response string",
        "file": "https://example.com/file.jpg",
        "fileType": "Response string",
        "knowledgeBase": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        },
        "size": 456,
        "status": {},
        "parser": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
          "provider": {},
          "isTimestampSttProvider": false
        },
        "labels": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ],
        "rawUserDefineMetadata": null,
        "speakerLabels": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "originalLabel": "Response string",
            "customName": "Response string",
            "displayName": "Response string"
          }
        ],
        "vectorStorageSize": 456,
        "chunksCount": 456,
        "waitingTime": 456,
        "processingTime": 456,
        "processingTimeDetails": null,
        "previewUrl": "Response string",
        "createdAt": "Response string"
      }
    ],
```

```
  "citationNodes": [
    {
      "chatbotTextNode": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "charactersCount": 456,
        "hitsCount": 456,
        "text": "Response string",
        "updatedAt": "Response string",
        "filename": "Response string",
        "chatbotFile": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response name",
          "description": "Example response description"
        },
        "knowledgeBaseFile": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response name",
          "description": "Example response description"
        },
        "pageNumber": 456,
        "pageStart": 456,
        "pageEnd": 456,
        "citationTitle": "Response string",
        "citationDescription": "Response string",
        "citationQuote": "Response string",
        "hasImage": false,
        "imageUrl": "Response string",
        "displayText": "Response string",
        "displayTitle": "Response string",
        "highlightedText": "Response string",
        "labels": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ],
        "rawUserDefineMetadata": null,
        "metadataEnabledForSearch": {
          "key1": "value1",
          "key2": "value2",
          "createdAt": "2025-01-01T00:00:00.000Z"
        },
        "startCharIdx": 456,
        "endCharIdx": 456
      },
      "score": 456,
      "displayScore": 456,
      "citationNumber": 456,
      "highlightedText": "Response string"
    }
  ],
  "canvas": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "canvasType": {},
    "title": "Response string",
```

```
    "content": "Response string",
    "createdAt": "Response string"
  },
  "slideTemplate": {
    "slug": "Response string",
    "displayName": "Response string",
    "thumbnailUrls": [
      "Response string"
    ]
  },
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "recordId": "Response string",
  "broadcast": false,
  "skipCopilotTrigger": false,
  "activeRevisionNumber": 456,
  "revisions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "revisionNumber": 456,
      "content": "Response string",
      "contentPayload": null,
      "isActive": false,
      "createdAt": "Response string"
    }
  ],
  "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
  "suggestionStatus": "Response string",
  "suggestionTriggerSource": "Response string",
  "suggestionError": "Response string",
  "retryCount": 456,
  "skillIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "toolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "templateId": "550e8400-e29b-41d4-a716-446655440000",
  "videoGeneration": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "isGreeting": false
}
```


Get Conversation List

`GET` `/api/v1/conversations/`

Parameters

Parameter Name	Required	Type	Description
`assignee`	X	string	
`chatbot`	X	string	
`contact`	X	string	
`contactTags`	X	array	Multiple values may be separated by commas.
`cursor`	X	string	The pagination cursor value.
`endDate`	X	string	
`externalConversationId`	X	string	
`externalSource`	X	string	
`inbox`	X	string	
`isPinned`	X	string	
`mine`	X	boolean	
`mode`	X	string	`chat`: Chat ; `cowork`: Cowork;
`pageSize`	X	integer	Number of results to return per page.
`startDate`	X	string	
`status`	X	array	`open`: Open ; `resolved`: Resolved ; `queued`: Queued;
`tags`	X	array	Multiple values may be separated by commas.
`unassigned`	X	boolean	
`updatedAfter`	X	string	

Code Examples

Shell/Bash

```shell
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-a716-446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=example&externalSource=example&inbox=550e8400-e29b-41d4-a716-446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=example&tags=example&unassigned=true&updatedAfter=2025-01-01T00:00:00.000Z"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```javascript
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-a716-446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=example&externalSource=example&inbox=550e8400-e29b-41d4-a716-446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=example&tags=example&unassigned=true&updatedAfter=2025-01-01T00:00:00.000Z", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-
446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-
a716-
446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=ex
ample&externalSource=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=exa
mple&tags=example&unassigned=true&updatedAfter=2025-01-01T00:00:00.000Z"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/?
assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-
446655440000&contact=550e8400-e29b-41d4-a716-
446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=ex
ample&externalSource=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=exa
mple&tags=example&unassigned=true&updatedAfter=2025-01-01T00:00:00.000Z", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "contact": { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields required by the List view; inboxes, mcp_credentials, and
```

```
api_credentials are removed
to avoid N+1 query problems. email / phone_number / metadata are scalar fields on the Contact
itself,
needed by the frontend conversation sidebar, and incur no N+1 cost.
      {
        "id": string (uuid)
        "name": string
        "avatar"?: string // Stores our own media as a relative storage key, or an external CDN
URL as-is. (Optional)
        "email"?: // May have different types (Optional)
        string (email) // Optional
        "phoneNumber"?: string // Optional
        "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
        "sourceId"?: string // Optional
        "tags": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
      }
      }
      "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
      {
        "id": string (uuid)
        "name": string
        "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
        "unreadConversationsCount"?: integer // Optional
        "signAuth"?:  // Optional
        {
          "id": string (uuid)
          "signSource": string (uuid)
          "signParams": {
          {
            "keycloak":
            {
              "clientId": string
```

```
              "url": string
              "realm": string
            }
            "line":
            {
              "liffId": string
            }
            "ad":
            {
              "saml": string
            }
            "google"?:  // Optional
            {
              "enabled": boolean
            }
          }
          }
          "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a
Source ID (verified per the verify mode below) may chat without going through the login flow.
Disabled keeps every existing behavior unchanged. (Optional)
          "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must
send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID
can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
          {
          }
          "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
          {
          }
          "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a
visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API
tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak
sign source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (Optional)
          "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com"
wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
            string
          ]
        }
        "enableAnalysis"?: boolean // Optional
      }
      }
      "chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already
eager-loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
```

```
      "title": string
      "lastMessage"?: { // Optional
      {
        "id": string (uuid)
        "type"?: string // Optional
        "content"?: string // Optional
        "createdAt": string (timestamp)
      }
      }
      "callRecord":
      {
        "id": string (uuid)
        "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
        "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
        "isActive": boolean
        "canHangup": boolean
      }
      "lastMessageCreatedAt": string (timestamp)
      "unreadMessagesCount": integer
      "autoReplyEnabled": boolean
      "isAutoReplyNow": boolean
      "lastReadAt": string (timestamp)
      "createdAt": string (timestamp)
      "isGroupChat": boolean
      "enableGroupMention"?: boolean // Optional
      "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint's description for details (Optional)
      {
        "knowledgeBases"?: [ // List of knowledge bases that can be queried; empty array or
null = none can be queried (no permission), not provided = unrestricted (Optional)
          {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected,
depending on hasUserSelectedAll) (Optional)
              string (uuid)
            ]
            "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; this
field takes precedence when both are provided) (Optional)
              string (uuid)
            ]
            "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
              string (uuid)
            ]
            "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base
and exclude the items in the list; false = select only the items in the list (Optional)
          }
```

```
        ]
        "labelRelations"?: { // Tag filter conditions; an empty conditions array = no tag
filtering (not "no permission"). Must be provided together with knowledgeBases — when provided
alone, knowledge base retrieval does not apply tag filtering (equivalent to unrestricted)
(Optional)
        {
          "operator": string (enum: AND, OR) // How tag conditions are combined
          "conditions"?: [ // List of tag conditions; supports nested operator/conditions
combinations (Optional)
            object
          ]
        }
        }
      }
      }
      "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
      "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
      "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
      "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
      {
        "type"?: string // Form type, always reasoning_effort (Optional)
        "fields"?: object // reasoning_effort field: options lists the thinking levels this
model accepts (from weakest to strongest); default is always null (Optional)
        "canDisable"?: boolean // Whether this model can turn off thinking; when true, options
includes off (Optional)
        "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model thinks on its own, or the platform default level enables thinking
(Optional)
        "unsetLevel"?: string // The level the platform actually sends when no level is set;
null means no thinking parameter is sent and the model's own default applies (Optional)
      }
      }
      "effectiveThinkingConfig": object // The level the next reply sends on this
conversation's model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
      "llm": string (uuid) // The model the next reply runs on: the conversation's pin, else
its assistant's current model.

TC-0205: returning the stored ``conversation.llm`` would show a model an unpinned conversation
no
longer uses once its assistant switches model, while ``thinking_config_schema`` already follows
the
effective model. Read-only: displaying it never marks the conversation as explicitly selected;
only the update-llm endpoint does.
      "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
```

```
* `failed` - Failed
      {
      }
      "deepResearchOutputFormat": // May have different types
      string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set
only when the user clicks a format in the pre-research choice card; reset to NULL each time
Deep Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and
falls back to Canvas at render time, but stays distinct from an explicit canvas pick so the
frontend can lock the card only after a real selection. To roll back to fixed-Canvas behavior,
force this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
      "ipAddress": string
      "ipCountryCode": string
      "ipRecordedAt": string (timestamp)
      "assignee":
      {
        "id": string (uuid)
        "name": string
        "userId": string (uuid)
      }
      "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
      "tags": [
        {
          "id": string (uuid)
          "name": string
          "memberIds"?: [ // Optional
            string (uuid)
          ]
        }
      ]
      "progressStatus": string
      "firstResponseAt"?: string (timestamp) // Optional
      "queuedAt"?: string (timestamp) // Optional
      "assignedAt"?: string (timestamp) // Optional
      "transferReason"?: string // Optional
      "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale
(1=very dissatisfied, 5=very satisfied)
      "satisfactionComment": string
      "satisfactionSubmittedAt": string (timestamp)
      "latestAnalysisSummary": string // The conversation's current analysis summary (its
active summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
      "clientPlatform":  // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
      {
```

```
      }
      "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
      "slideProjectId": string
      "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
      "matchedSnippet": string // Keyword-centred excerpt of the matched message, with
ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
      "matchedMessageId": string
      "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
      "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped
into. Null means unclassified (shown under the time groups). Deleting the folder resets this to
null without deleting the conversation.
      "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
      "callerOrganizationId": string // Bound caller organization (stamped at conversation
start); null when unbound
      "handoff": object // Handoff information: enabled = platform switch; the continuation
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
      "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
    }
  ]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "contact": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "avatar": "Response string",
        "email": "response@example.com",
        "phoneNumber": "Response string",
        "metadata": null,
        "sourceId": "Response string",
        "tags": [
          {
```

```
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Response string",
          "url": "Response string",
          "realm": "Response string"
        },
        "line": {
          "liffId": "Response string"
        },
        "ad": {
          "saml": "Response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "Response string"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "title": "Response string",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Response string",
    "content": "Response string",
    "createdAt": "Response string"
  },
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
```

```
    "canHangup": false
  },
  "lastMessageCreatedAt": "Response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Response string",
  "createdAt": "Response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response string",
  "firstResponseAt": "Response string",
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
```

```
      "satisfactionSubmittedAt": "Response string",
      "latestAnalysisSummary": "Response string",
      "clientPlatform": {},
      "workingDirectory": "Response string",
      "slideProjectId": "Response string",
      "matchedField": "Response string",
      "matchedSnippet": "Response string",
      "matchedMessageId": "Response string",
      "pinnedAt": "Response string",
      "folder": "550e8400-e29b-41d4-a716-446655440000",
      "hasActiveBrowserSession": false,
      "callerOrganizationId": "Response string",
      "handoff": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "displayContextUsage": false
    }
  ]
}
```


Get a Specific Conversation

GET `/api/v1/conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "contact":
  {
    "id": string (uuid)
    "name": string
    "sourceId"?: string // optional
  }
  "lastMessageCreatedAt": string (timestamp)
  "autoReplyEnabled": boolean
  "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
  {
    "type"?: string // Form type, always reasoning_effort (optional)
    "fields"?: object // reasoning_effort field: options lists the thinking levels this model
accepts (from weakest to strongest); default is always null (optional)
    "canDisable"?: boolean // Whether thinking can be turned off for this model; when true,
options includes off (optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model decides on its own, or the platform default level enables thinking
(optional)
    "unsetLevel"?: string // Level the platform actually sends when none is set; null means no
thinking parameter is sent and the model default applies (optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // May be of different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.
```

```
* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled = platform switch; a continuation
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
  "title": string
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "sourceId": "Response string"
  },
  "lastMessageCreatedAt": "Response string",
  "autoReplyEnabled": false,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "displayContextUsage": false,
  "title": "Response string"
}
```


Rename a conversation

PATCH `/api/v1/conversations/{id}/rename/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Request Content

Request Parameters

Field	Type	Required	Description
contact	object	No	Lightweight Contact Serializer used by the Conversation List view. It includes only the fields the List view needs and removes inboxes, mcp_credentials, and api_credentials to avoid N+1 query problems. email / phone_number / metadata are scalar fields on the Contact itself that the frontend conversation sidebar needs to display, so they add no N+1 cost.
contact.name	string	Yes	
contact.avatar	string	No	Stores our own media as a relative storage key, or an external CDN URL as-is.
contact.email	object	No	
contact.phoneNumber	string	No	
contact.metadata	object	No	Custom customer information shown in the customer panel of the conversation page
contact.sourceId	string	No	
inbox	object	No	Inbox summary embedded in a conversation payload. `unread_conversations_count` stays here because it is part of the documented REST conversation API. It is an org-

Field	Type	Required	Description
			wide denormalized counter, so WS em...
inbox.name	string	Yes	
inbox.channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)	Yes	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` : Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
inbox.unreadConversationsCount	integer	No	
inbox.signAuth	object (with 8 properties: id, signSource, signParams...)	No	
inbox.signAuth.signSource	string (uuid)	Yes	
inbox.signAuth.signParams	object	Yes	
inbox.signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
inbox.signAuth.signParams.line	object (with 1 properties: liffId)	Yes	
inbox.signAuth.signParams.ad	object (with 1 properties: saml)	Yes	
inbox.signAuth.signParams.google	object (with 1 properties: enabled)	No	
inbox.signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
inbox.signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC

Field	Type	Required	Description
			signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
inbox.signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
inbox.signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-to...
inbox.signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
inbox.enableAnalysis	boolean	No	
lastMessage	object	No	
lastMessage.type	string	No	
lastMessage.content	string	No	
enableGroupMention	boolean	No	
queryMetadata	object	No	Query metadata that controls the scope of knowledge that can be queried; see each endpoint description for details
queryMetadata.knowledgeBases	array[object]	No	List of knowledge bases that can be queried; empty array or null = none can be queried (no permission); not provided = unrestricted
queryMetadata.labelRelations	object	No	Label filter conditions; empty conditions array = no label filtering (not "no permission"). Must be provided together with knowledgeBases — when provided alone, knowledge base retrieval does not apply

Field	Type	Required	Description
			label filtering (equivalent to unrestricted)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How the label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
toolMask	object	No	Enabled/disabled state per tool: {tool_id: bool}
connectorMask	object	No	Enabled/disabled state per connector: {connector_id: bool}
thinkingConfig	object	No	None = not set (inherit the chatbot setting); otherwise {"reasoning_effort": }
status	string (enum: open, resolved, queued)	No	`open` : Open ; `resolved` : Resolved ; `queued` : Queued;
firstResponseAt	string (timestamp)	No	
queuedAt	string (timestamp)	No	
assignedAt	string (timestamp)	No	
transferReason	string	No	

Example Request Structure

```
{
  "contact"?: { // Lightweight Contact Serializer used by the Conversation List view

Includes only the fields the List view needs and removes inboxes, mcp_credentials, and
api_credentials
to avoid N+1 query problems. email / phone_number / metadata are scalar fields on the Contact
itself,
which the frontend conversation sidebar needs to display, so they add no N+1 cost. (optional)
  {
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (optional)
    "email"?: // May be of different types (optional)
    string (email) // optional
    "phoneNumber"?: string // optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
```

```
conversation page (optional)
    "sourceId"?: string // optional
  }
  }
  "inbox"?: { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``. (optional)
  {
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // optional
    "signAuth"?:  // optional
    {
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source
ID (verified per the verify mode below) may chat without going through the login flow. Disabled
keeps every existing behavior unchanged. (optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send
```

```
an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act
as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor
logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on
their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign
source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // optional
  }
  }
  "lastMessage"?: { // optional
  {
    "type"?: string // optional
    "content"?: string // optional
  }
  }
  "enableGroupMention"?: boolean // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint description for details (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; empty array or null =
none can be queried (no permission); not provided = unrestricted (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; when
both are provided, this field takes precedence) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (optional)
          string (uuid)
        ]
```

```
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the items in the list; false = select only the items in the list (optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; empty conditions array = no label
filtering (not "no permission"). Must be provided together with knowledgeBases — when provided
alone, knowledge base retrieval does not apply label filtering (equivalent to unrestricted)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (optional)
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (optional)
  "firstResponseAt"?: string (timestamp) // optional
  "queuedAt"?: string (timestamp) // optional
  "assignedAt"?: string (timestamp) // optional
  "transferReason"?: string // optional
}
```

Example Request Values

```
{
  "contact": {
    "name": "Example name",
    "avatar": "Example string",
    "email": null,
    "phoneNumber": "Example string",
    "metadata": null,
    "sourceId": "Example string"
  },
  "inbox": {
    "name": "Example name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Example string",
          "url": "Example string",
          "realm": "Example string"
        },
```

```
        "line": {
          "liffId": "Example string"
        },
        "ad": {
          "saml": "Example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "Example string"
      ]
    },
    "enableAnalysis": true
  },
  "lastMessage": {
    "type": "Example string",
    "content": "Hello! I would like to learn about your products."
  },
  "enableGroupMention": true,
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "status": "open",
  "firstResponseAt": "Example string",
  "queuedAt": "Example string",
  "assignedAt": "Example string",
```

```
  "transferReason": "Example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/rename/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
    },
    "lastMessage": {
      "type": "Example string",
```

```
      "content": "Hello! I would like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  }'

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
```

```
    },
    "lastMessage": {
      "type": "Example string",
      "content": "Hello! I'd like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  };

axios.patch("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/rename/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while sending the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/rename/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "contact": {
        "name": "Example name",
        "avatar": "Example string",
        "email": None,
        "phoneNumber": "Example string",
        "metadata": None,
        "sourceId": "Example string"
    },
    "inbox": {
        "name": "Example name",
        "channelType": "line",
        "unreadConversationsCount": 123,
        "signAuth": {
            "signSource": "550e8400-e29b-41d4-a716-446655440000",
            "signParams": {
                "keycloak": {
                    "clientId": "Example string",
                    "url": "Example string",
                    "realm": "Example string"
                },
                "line": {
                    "liffId": "Example string"
                },
                "ad": {
                    "saml": "Example string"
                },
                "google": {
                    "enabled": True
                }
            },
            "sourceIdAccessEnabled": True,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": True,
            "keycloakTokenPassthroughAllowedHosts": [
                "Example string"
            ]
        },
        "enableAnalysis": True
    },
```

```
    "lastMessage": {
        "type": "Example string",
        "content": "Hello! I'd like to learn about your products."
    },
    "enableGroupMention": True,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": True
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                None
            ]
        }
    },
    "toolMask": None,
    "connectorMask": None,
    "thinkingConfig": None,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while sending the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/rename/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'contact' => [
                'name' => 'Example name',
                'avatar' => 'Example string',
                'email' => null,
                'phoneNumber' => 'Example string',
                'metadata' => null,
                'sourceId' => 'Example string'
            ],
            'inbox' => [
                'name' => 'Example name',
                'channelType' => 'line',
                'unreadConversationsCount' => 123,
                'signAuth' => [
                    'signSource' => '550e8400-e29b-41d4-a716-446655440000',
                    'signParams' => [
                        'keycloak' => [
                            'clientId' => 'Example string',
                            'url' => 'Example string',
                            'realm' => 'Example string'
                        ],
                        'line' => [
                            'liffId' => 'Example string'
                        ],
                        'ad' => [
                            'saml' => 'Example string'
                        ],
                        'google' => [
                            'enabled' => true
                        ]
                    ],
                    'sourceIdAccessEnabled' => true,
                    'sourceIdVerifyMode' => new \stdClass(),
                    'sourceIdSignatureTtlMinutes' => new \stdClass(),
                    'keycloakTokenPassthrough' => true,
                    'keycloakTokenPassthroughAllowedHosts' => [
                        'Example string'
                    ]
                ],
```

```
                'enableAnalysis' => true
            ],
            'lastMessage' => [
                'type' => 'Example string',
                'content' => 'Hello! I'd like to learn about your products.'
            ],
            'enableGroupMention' => true,
            'queryMetadata' => [
                'knowledgeBases' => [
                    [
                        'knowledgeBaseId' => '550e8400-e29b-41d4-a716-446655440000',
                        'chatbotFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'knowledgeBaseFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'faqIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'hasUserSelectedAll' => true
                    ]
                ],
                'labelRelations' => [
                    'operator' => 'AND',
                    'conditions' => [
                        null
                    ]
                ]
            ],
            'toolMask' => null,
            'connectorMask' => null,
            'thinkingConfig' => null,
            'status' => 'open',
            'firstResponseAt' => 'Example string',
            'queuedAt' => 'Example string',
            'assignedAt' => 'Example string',
            'transferReason' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while sending the request: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact serializer used by the Conversation List view

Includes only the fields the List view needs; inboxes, mcp_credentials, and api_credentials are
removed
to avoid N+1 query problems. email / phone_number / metadata are scalar fields of Contact
itself,
which the frontend conversation sidebar needs to display, at no N+1 cost.
  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // Multiple types possible (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
```

```
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at
```

```
login, never the contact Source ID. (Optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
  "title": string
  "lastMessage"?: { // Optional
  {
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
  }
  }
  "callRecord":
  {
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint's description for details (Optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an empty array or null
= nothing can be queried (no permission), omitted = no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected, depending
```

```
on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name of chatbotFileIds; this
field takes precedence if both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the items in the lists; false = select only the items in the lists (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array = no label
filtering (not a lack of permission). Must be provided together with knowledgeBases; if
provided alone, knowledge base retrieval does not apply label filtering (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; nested operator/conditions combinations are
supported (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
  {
    "type"?: string // Form type, always reasoning_effort (Optional)
    "fields"?: object // reasoning_effort field: options lists the thinking levels this model
accepts (from weakest to strongest); default is always null (Optional)
    "canDisable"?: boolean // Whether this model can turn thinking off; when true, options
includes off (Optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model decides how to think, or the platform's default level enables
thinking (Optional)
    "unsetLevel"?: string // The level the platform actually sends when none is set; null means
no thinking parameter is sent and the model's own default applies (Optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
  "llm": string (uuid) // The model the next reply runs on: the conversation's pin, else its
```

```
assistant's current model.

TC-0205: returning the stored ``conversation.llm`` would show a model an unpinned conversation
no
longer uses once its assistant switches model, while ``thinking_config_schema`` already follows
the
effective model. Read-only: displaying it never marks the conversation as explicitly selected;
only the update-llm endpoint does.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // Multiple types possible
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"?: [ // Optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
```

```
  "satisfactionComment": string
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
  "clientPlatform":  // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled = platform switch; the continuation
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
```

```
    "email": "response@example.com",
    "phoneNumber": "Response string",
    "metadata": null,
    "sourceId": "Response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Response string",
          "url": "Response string",
          "realm": "Response string"
        },
        "line": {
          "liffId": "Response string"
        },
        "ad": {
          "saml": "Response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "Response string"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "title": "Response string",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Response string",
    "content": "Response string",
    "createdAt": "Response string"
```

```
  },
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "Response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Response string",
  "createdAt": "Response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response string",
```

```
  "firstResponseAt": "Response string",
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
  "satisfactionSubmittedAt": "Response string",
  "latestAnalysisSummary": "Response string",
  "clientPlatform": {},
  "workingDirectory": "Response string",
  "slideProjectId": "Response string",
  "matchedField": "Response string",
  "matchedSnippet": "Response string",
  "matchedMessageId": "Response string",
  "pinnedAt": "Response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "displayContextUsage": false
}
```


Delete Conversation

DELETE `/api/v1/conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Stop AI Response

POST `/api/v1/conversations/{conversationId}/stop-response/`

Parameters

Parameter Name	Required	Type	Description
conversationId	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/stop-response/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/stop-response/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/stop-response/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/stop-response/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	No response body

Submit Conversation Satisfaction

POST `/api/v1/conversations/{id}/satisfaction/`

Each conversation can only be submitted once.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Request body

Request parameters

Field	Type	Required	Description
contact	object	Yes	Lightweight Contact Serializer for the Conversation List view. Includes only the fields required by the List view, removing inboxes, mcp_credentials, and api_credentials to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself, needed for display in the conversation sidebar, with no N+1 cost.
contact.name	string	Yes	
contact.avatar	string	No	Stores our own media as a relative storage key, or an external CDN URL as-is.
contact.email	object	No	
contact.phoneNumber	string	No	
contact.metadata	object	No	Custom customer information shown in the customer panel of the conversation page
contact.sourceId	string	No	
inbox	object	Yes	Inbox summary embedded in a conversation payload. `unread_conversations_count` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS em...
inbox.name	string	Yes	
inbox.channelType	string (enum: line, telegram, teams, web,	Yes	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` :

Field	Type	Required	Description
	messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)		Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
inbox.unreadConversationsCount	integer	No	
inbox.signAuth	object (with 8 properties: id, signSource, signParams...)	No	
inbox.signAuth.signSource	string (uuid)	Yes	
inbox.signAuth.signParams	object	Yes	
inbox.signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
inbox.signAuth.signParams.line	object (with 1 properties: liffId)	Yes	
inbox.signAuth.signParams.ad	object (with 1 properties: saml)	Yes	
inbox.signAuth.signParams.google	object (with 1 properties: enabled)	No	
inbox.signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
inbox.signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
inbox.signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime:

Field	Type	Required	Description
			once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
inbox.signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-to...
inbox.signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
inbox.enableAnalysis	boolean	No	
lastMessage	object	No	
lastMessage.type	string	No	
lastMessage.content	string	No	
enableGroupMention	boolean	No	
queryMetadata	object	No	Query metadata that controls the scope of knowledge that can be queried; see each endpoint's description for details
queryMetadata.knowledgeBases	array[object]	No	List of knowledge bases that can be queried; an empty array or null = none can be queried (no permission); omitted = unrestricted
queryMetadata.labelRelations	object	No	Label filter conditions; an empty conditions array = no label filtering (not "no permission"). Must be provided together with knowledgeBases — if provided alone, knowledge base retrieval does not apply label filtering (equivalent to unrestricted)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How the label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested

Field	Type	Required	Description
			operator/conditions combinations
toolMask	object	No	Enabled/disabled state per tool: {tool_id: bool}
connectorMask	object	No	Enabled/disabled state per connector: {connector_id: bool}
thinkingConfig	object	No	None = not set (inherit the chatbot setting); otherwise {"reasoning_effort": }
status	string (enum: open, resolved, queued)	No	`open` : Open ; `resolved` : Resolved ; `queued` : Queued;
firstResponseAt	string (timestamp)	No	
queuedAt	string (timestamp)	No	
assignedAt	string (timestamp)	No	
transferReason	string	No	

Request Schema Example

```
{
  "contact": { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields required by the List view, removing inboxes, mcp_credentials, and
api_credentials
to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself,
needed for display in the conversation sidebar, with no N+1 cost.
  {
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May have different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
```

```
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source
ID (verified per the verify mode below) may chat without going through the login flow. Disabled
keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send
an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act
as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the
```

```
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor
logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on
their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign
source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (Optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "lastMessage"?: { // Optional
  {
    "type"?: string // Optional
    "content"?: string // Optional
  }
  }
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint's description for details (Optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an empty array or null
= none can be queried (no permission); omitted = unrestricted (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; if both
are provided, this field takes precedence) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the items in the list; false = select only the items in the list (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array = no label
filtering (not "no permission"). Must be provided together with knowledgeBases — if provided
alone, knowledge base retrieval does not apply label filtering (equivalent to unrestricted)
(Optional)
```

```
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
}
```

Example Request Values

```
{
  "contact": {
    "name": "Example Name",
    "avatar": "Example string",
    "email": null,
    "phoneNumber": "Example string",
    "metadata": null,
    "sourceId": "Example string"
  },
  "inbox": {
    "name": "Example Name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Example string",
          "url": "Example string",
          "realm": "Example string"
        },
        "line": {
          "liffId": "Example string"
        },
        "ad": {
          "saml": "Example string"
        },
        "google": {
          "enabled": true
```

```
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "Example string"
      ]
    },
    "enableAnalysis": true
  },
  "lastMessage": {
    "type": "Example string",
    "content": "Hello! I would like to learn about your products."
  },
  "enableGroupMention": true,
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "status": "open",
  "firstResponseAt": "Example string",
  "queuedAt": "Example string",
  "assignedAt": "Example string",
  "transferReason": "Example string"
}
```


Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/satisfaction/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "contact": {
      "name": "Example Name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example Name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
    },
    "lastMessage": {
      "type": "Example string",
```

```
      "content": "Hello! I would like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "contact": {
      "name": "Example Name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example Name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
```

```
    },
    "lastMessage": {
      "type": "Example string",
      "content": "Hello! I would like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  };

axios.post("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/satisfaction/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred with the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/satisfaction/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "contact": {
        "name": "Example Name",
        "avatar": "Example string",
        "email": None,
        "phoneNumber": "Example string",
        "metadata": None,
        "sourceId": "Example string"
    },
    "inbox": {
        "name": "Example Name",
        "channelType": "line",
        "unreadConversationsCount": 123,
        "signAuth": {
            "signSource": "550e8400-e29b-41d4-a716-446655440000",
            "signParams": {
                "keycloak": {
                    "clientId": "Example string",
                    "url": "Example string",
                    "realm": "Example string"
                },
                "line": {
                    "liffId": "Example string"
                },
                "ad": {
                    "saml": "Example string"
                },
                "google": {
                    "enabled": True
                }
            },
            "sourceIdAccessEnabled": True,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": True,
            "keycloakTokenPassthroughAllowedHosts": [
                "Example string"
            ]
        },
        "enableAnalysis": True
    },
```

```
    "lastMessage": {
        "type": "Example string",
        "content": "Hello! I would like to learn about your products."
    },
    "enableGroupMention": True,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": True
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                None
            ]
        }
    },
    "toolMask": None,
    "connectorMask": None,
    "thinkingConfig": None,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred with the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/satisfaction/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'contact' => [
                'name' => 'Example name',
                'avatar' => 'Example string',
                'email' => null,
                'phoneNumber' => 'Example string',
                'metadata' => null,
                'sourceId' => 'Example string'
            ],
            'inbox' => [
                'name' => 'Example name',
                'channelType' => 'line',
                'unreadConversationsCount' => 123,
                'signAuth' => [
                    'signSource' => '550e8400-e29b-41d4-a716-446655440000',
                    'signParams' => [
                        'keycloak' => [
                            'clientId' => 'Example string',
                            'url' => 'Example string',
                            'realm' => 'Example string'
                        ],
                        'line' => [
                            'liffId' => 'Example string'
                        ],
                        'ad' => [
                            'saml' => 'Example string'
                        ],
                        'google' => [
                            'enabled' => true
                        ]
                    ],
                    'sourceIdAccessEnabled' => true,
                    'sourceIdVerifyMode' => new \stdClass(),
                    'sourceIdSignatureTtlMinutes' => new \stdClass(),
                    'keycloakTokenPassthrough' => true,
                    'keycloakTokenPassthroughAllowedHosts' => [
                        'Example string'
                    ]
                ],
```

```
                'enableAnalysis' => true
            ],
            'lastMessage' => [
                'type' => 'Example string',
                'content' => 'Hello! I would like to learn about the product.'
            ],
            'enableGroupMention' => true,
            'queryMetadata' => [
                'knowledgeBases' => [
                    [
                        'knowledgeBaseId' => '550e8400-e29b-41d4-a716-446655440000',
                        'chatbotFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'knowledgeBaseFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'faqIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'hasUserSelectedAll' => true
                    ]
                ],
                'labelRelations' => [
                    'operator' => 'AND',
                    'conditions' => [
                        null
                    ]
                ]
            ],
            'toolMask' => null,
            'connectorMask' => null,
            'thinkingConfig' => null,
            'status' => 'open',
            'firstResponseAt' => 'Example string',
            'queuedAt' => 'Example string',
            'assignedAt' => 'Example string',
            'transferReason' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields required by the List view, removing inboxes, mcp_credentials, and
api_credentials
to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself,
needed for display in the conversation sidebar, with no N+1 cost.
  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May have different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
```

```
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
```

```
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
  "title": string
  "lastMessage"?: { // Optional
  {
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
  }
  }
  "callRecord":
  {
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint's description for details (Optional)
  {
    "knowledgeBases"?: [ // Knowledge bases available for querying; an empty array or null
prevents all queries (no permissions), while omitting this field imposes no restriction
(Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // File ID list (excluded or exclusively selected depending on
```

```
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // File ID list (the new name for chatbotFileIds; this field
takes precedence when both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // FAQ ID list (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true selects all knowledge base content except the
listed items; false selects only the listed items (Optional)
      }
    ]
    "labelRelations"?: { // Label filters; an empty conditions array means no label filtering
(not a lack of permissions). Must be provided together with knowledgeBases. When provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
  {
    "type"?: string // Form type; always reasoning_effort (Optional)
    "fields"?: object // reasoning_effort field: options lists the thinking levels this model
accepts (from weakest to strongest), and default is always null (Optional)
    "canDisable"?: boolean // Whether this model can disable thinking; when true, options
includes off (Optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model decides on its own, or the platform default level enables thinking
(Optional)
    "unsetLevel"?: string // The level the platform actually sends when no level is set; null
means no thinking parameter is sent and the model's own default applies (Optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
  "llm": string (uuid) // The model the next reply runs on: the conversation's pin, else its
```

```
assistant's current model.

TC-0205: returning the stored ``conversation.llm`` would show a model an unpinned conversation
no
longer uses once its assistant switches model, while ``thinking_config_schema`` already follows
the
effective model. Read-only: displaying it never marks the conversation as explicitly selected;
only the update-llm endpoint does.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // May have different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"?: [ // Optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
```

```
  "satisfactionComment": string
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
  "clientPlatform":  // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled = platform switch; the continuation
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "avatar": "Response String",
```

```
    "email": "response@example.com",
    "phoneNumber": "Response String",
    "metadata": null,
    "sourceId": "Response String",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response String"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Response String",
          "url": "Response String",
          "realm": "Response String"
        },
        "line": {
          "liffId": "Response String"
        },
        "ad": {
          "saml": "Response String"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "Response String"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "title": "Response String",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Response String",
    "content": "Response String",
    "createdAt": "Response String"
```

```
  },
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "Response String",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Response String",
  "createdAt": "Response String",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response String",
  "ipCountryCode": "Response String",
  "ipRecordedAt": "Response String",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response String",
```

```
  "firstResponseAt": "Response String",
  "queuedAt": "Response String",
  "assignedAt": "Response String",
  "transferReason": "Response String",
  "satisfactionScore": 456,
  "satisfactionComment": "Response String",
  "satisfactionSubmittedAt": "Response String",
  "latestAnalysisSummary": "Response String",
  "clientPlatform": {},
  "workingDirectory": "Response String",
  "slideProjectId": "Response String",
  "matchedField": "Response String",
  "matchedSnippet": "Response String",
  "matchedMessageId": "Response String",
  "pinnedAt": "Response String",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response String",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "displayContextUsage": false
}
```


List Conversation Events

GET `/api/v1/conversations/{id}/events/`

Event records for the human handoff process (event_type: transfer_triggered／human_assigned／returned_to_ai／queue_timeout), which can be reconciled with Webhook events. For the workflow, see Customer Service System Integration in the technical manual.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Code Examples

Shell/Bash

```shell
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/events/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```javascript
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/events/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/events/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-
41d4-a716-446655440000/events/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields required by the List view; removes inboxes, mcp_credentials, and
api_credentials
to avoid N+1 query problems. email / phone_number / metadata are Contact's own scalar fields,
which the frontend conversation sidebar needs to display, with no N+1 cost.
  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May be of different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
```

```
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
```

```
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
  "title": string
  "lastMessage"?: { // Optional
  {
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
  }
  }
  "callRecord":
  {
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint's description for details (Optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an empty array or null
= none can be queried (no permission); omitted = no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected, depending
on hasUserSelectedAll) (Optional)
```

```
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; if
both are provided, this field takes precedence) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the knowledge base and
exclude the items in the list; false = select only the items in the list (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array = no label
filtering (not a lack of permission). Must be provided together with knowledgeBases — when
provided alone, knowledge base retrieval does not apply label filtering (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
  {
    "type"?: string // Form type; always reasoning_effort (Optional)
    "fields"?: object // reasoning_effort field: options lists the thinking levels this model
accepts (from weakest to strongest); default is always null (Optional)
    "canDisable"?: boolean // Whether thinking can be turned off for this model; when true,
options includes off (Optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model thinks on its own, or the platform default level turns thinking on
(Optional)
    "unsetLevel"?: string // The level the platform actually sends when no level is set; null
means no thinking parameter is sent and the model's own default applies (Optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
  "llm": string (uuid) // The model the next reply runs on: the conversation's pin, else its
assistant's current model.
```

```
TC-0205: returning the stored ``conversation.llm`` would show a model an unpinned conversation
no
longer uses once its assistant switches model, while ``thinking_config_schema`` already follows
the
effective model. Read-only: displaying it never marks the conversation as explicitly selected;
only the update-llm endpoint does.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // May be of different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"?: [ // Optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
  "satisfactionComment": string
```

```
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
  "clientPlatform":  // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled = platform switch; the continuing
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
```

```
    "phoneNumber": "Response string",
    "metadata": null,
    "sourceId": "Response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Response string",
          "url": "Response string",
          "realm": "Response string"
        },
        "line": {
          "liffId": "Response string"
        },
        "ad": {
          "saml": "Response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "Response string"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "title": "Response string",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Response string",
    "content": "Response string",
    "createdAt": "Response string"
  },
```

```
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "Response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Response string",
  "createdAt": "Response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response string",
  "firstResponseAt": "Response string",
```

```
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
  "satisfactionSubmittedAt": "Response string",
  "latestAnalysisSummary": "Response string",
  "clientPlatform": {},
  "workingDirectory": "Response string",
  "slideProjectId": "Response string",
  "matchedField": "Response string",
  "matchedSnippet": "Response string",
  "matchedMessageId": "Response string",
  "pinnedAt": "Response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "displayContextUsage": false
}
```


Assign a conversation to a member

PATCH `/api/v1/conversations/{id}/assign/`

Sets the member taking over and disables AI auto-reply; a queued conversation changes to in progress.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Request Content

Request Parameters

Field	Type	Required	Description
contact	object	No	Lightweight Contact Serializer for the Conversation List view. Includes only the fields required by the List view; removes inboxes, mcp_credentials, and api_credentials to avoid N+1 query problems. email / phone_number / metadata are Contact's own scalar fields, which the frontend conversation sidebar needs to display, with no N+1 cost.
contact.name	string	Yes	
contact.avatar	string	No	Stores our own media as a relative storage key, or an external CDN URL as-is.
contact.email	object	No	
contact.phoneNumber	string	No	
contact.metadata	object	No	Custom customer information shown in the customer panel of the conversation page
contact.sourceId	string	No	
inbox	object	No	Inbox summary embedded in a conversation payload. `unread_conversations_count` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS em...
inbox.name	string	Yes	
inbox.channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)	Yes	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` : Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
inbox.unreadConversationsCount	integer	No	

Field	Type	Required	Description
inbox.signAuth	object (with 8 properties: id, signSource, signParams...)	No	
inbox.signAuth.signSource	string (uuid)	Yes	
inbox.signAuth.signParams	object	Yes	
inbox.signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
inbox.signAuth.signParams.line	object (with 1 property: liffId)	Yes	
inbox.signAuth.signParams.ad	object (with 1 property: saml)	Yes	
inbox.signAuth.signParams.google	object (with 1 property: enabled)	No	
inbox.signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
inbox.signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
inbox.signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
inbox.signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools

Field	Type	Required	Description
			on their behalf, so tools do not require a separate per-to...
inbox.signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
inbox.enableAnalysis	boolean	No	
lastMessage	object	No	
lastMessage.type	string	No	
lastMessage.content	string	No	
enableGroupMention	boolean	No	
queryMetadata	object	No	Query metadata that controls the scope of knowledge that can be queried; see each endpoint's description for details
queryMetadata.knowledgeBases	array[object]	No	List of knowledge bases that can be queried; an empty array or null = none can be queried (no permission); omitted = no restriction
queryMetadata.labelRelations	object	No	Label filter conditions; an empty conditions array = no label filtering (not a lack of permission). Must be provided together with knowledgeBases — when provided alone, knowledge base retrieval does not apply label filtering (equivalent to no restriction)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How the label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
toolMask	object	No	Enabled/disabled state per tool: {tool_id: bool}
connectorMask	object	No	Enabled/disabled state per connector: {connector_id: bool}
thinkingConfig	object	No	None = not set (inherit the chatbot setting); otherwise

Field	Type	Required	Description
			{"reasoning_effort": }
status	string (enum: open, resolved, queued)	No	`open` : Open ; `resolved` : Resolved ; `queued` : Queued;
firstResponseAt	string (timestamp)	No	
queuedAt	string (timestamp)	No	
assignedAt	string (timestamp)	No	
transferReason	string	No	

Request Schema Example

```
{
  "contact"?: { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields required by the List view; removes inboxes, mcp_credentials, and
api_credentials
to avoid N+1 query problems. email / phone_number / metadata are Contact's own scalar fields,
which the frontend conversation sidebar needs to display, with no N+1 cost. (Optional)
  {
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May be of different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
  }
  }
  "inbox"?: { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``. (Optional)
  {
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
```

```
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own
```

```
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (Optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "lastMessage"?: { // Optional
  {
    "type"?: string // Optional
    "content"?: string // Optional
  }
  }
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls which knowledge can be queried; see each
endpoint's description (Optional)
  {
    "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array or null = nothing
is queryable (no permission); omitted = no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; if both
are provided, this field takes precedence) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the listed items; false = select only the listed items (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array = no label
filtering (not a lack of permission). Must be provided together with knowledgeBases — when
provided alone, knowledge base retrieval does not apply label filtering (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
```

```
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
}
```

Example Request Values

```
{
  "contact": {
    "name": "Example name",
    "avatar": "Example string",
    "email": null,
    "phoneNumber": "Example string",
    "metadata": null,
    "sourceId": "Example string"
  },
  "inbox": {
    "name": "Example name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Example string",
          "url": "Example string",
          "realm": "Example string"
        },
        "line": {
          "liffId": "Example string"
        },
        "ad": {
          "saml": "Example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "Example string"
      ]
    },
    "enableAnalysis": true
  },
```

```
  "lastMessage": {
    "type": "Example string",
    "content": "Hello! I'd like to learn about your products."
  },
  "enableGroupMention": true,
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "status": "open",
  "firstResponseAt": "Example string",
  "queuedAt": "Example string",
  "assignedAt": "Example string",
  "transferReason": "Example string"
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/assign/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
    },
    "lastMessage": {
      "type": "Example string",
```

```
        "content": "Hello! I'd like to learn about your products."
      },
      "enableGroupMention": true,
      "queryMetadata": {
        "knowledgeBases": [
          {
            "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
            "chatbotFileIds": [
              "550e8400-e29b-41d4-a716-446655440000"
            ],
            "knowledgeBaseFileIds": [
              "550e8400-e29b-41d4-a716-446655440000"
            ],
            "faqIds": [
              "550e8400-e29b-41d4-a716-446655440000"
            ],
            "hasUserSelectedAll": true
          }
        ],
        "labelRelations": {
          "operator": "AND",
          "conditions": [
            null
          ]
        }
      },
      "toolMask": null,
      "connectorMask": null,
      "thinkingConfig": null,
      "status": "open",
      "firstResponseAt": "Example string",
      "queuedAt": "Example string",
      "assignedAt": "Example string",
      "transferReason": "Example string"
    }'

  # Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
```

```
    },
    "lastMessage": {
      "type": "Example string",
      "content": "Hello! I'd like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  };

axios.patch("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/assign/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred with the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/assign/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "contact": {
        "name": "Example name",
        "avatar": "Example string",
        "email": None,
        "phoneNumber": "Example string",
        "metadata": None,
        "sourceId": "Example string"
    },
    "inbox": {
        "name": "Example name",
        "channelType": "line",
        "unreadConversationsCount": 123,
        "signAuth": {
            "signSource": "550e8400-e29b-41d4-a716-446655440000",
            "signParams": {
                "keycloak": {
                    "clientId": "Example string",
                    "url": "Example string",
                    "realm": "Example string"
                },
                "line": {
                    "liffId": "Example string"
                },
                "ad": {
                    "saml": "Example string"
                },
                "google": {
                    "enabled": True
                }
            },
            "sourceIdAccessEnabled": True,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": True,
            "keycloakTokenPassthroughAllowedHosts": [
                "Example string"
            ]
        },
        "enableAnalysis": True
    },
```

```
    "lastMessage": {
        "type": "Example string",
        "content": "Hello! I'd like to learn about your products."
    },
    "enableGroupMention": True,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": True
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                None
            ]
        }
    },
    "toolMask": None,
    "connectorMask": None,
    "thinkingConfig": None,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("An error occurred with the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/assign/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'contact' => [
                'name' => 'Example name',
                'avatar' => 'Example string',
                'email' => null,
                'phoneNumber' => 'Example string',
                'metadata' => null,
                'sourceId' => 'Example string'
            ],
            'inbox' => [
                'name' => 'Example name',
                'channelType' => 'line',
                'unreadConversationsCount' => 123,
                'signAuth' => [
                    'signSource' => '550e8400-e29b-41d4-a716-446655440000',
                    'signParams' => [
                        'keycloak' => [
                            'clientId' => 'Example string',
                            'url' => 'Example string',
                            'realm' => 'Example string'
                        ],
                        'line' => [
                            'liffId' => 'Example string'
                        ],
                        'ad' => [
                            'saml' => 'Example string'
                        ],
                        'google' => [
                            'enabled' => true
                        ]
                    ],
                    'sourceIdAccessEnabled' => true,
                    'sourceIdVerifyMode' => new \stdClass(),
                    'sourceIdSignatureTtlMinutes' => new \stdClass(),
                    'keycloakTokenPassthrough' => true,
                    'keycloakTokenPassthroughAllowedHosts' => [
                        'Example string'
                    ]
                ],
```

```
                'enableAnalysis' => true
            ],
            'lastMessage' => [
                'type' => 'Example string',
                'content' => 'Hello! I'd like to learn about your products.'
            ],
            'enableGroupMention' => true,
            'queryMetadata' => [
                'knowledgeBases' => [
                    [
                        'knowledgeBaseId' => '550e8400-e29b-41d4-a716-446655440000',
                        'chatbotFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'knowledgeBaseFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'faqIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'hasUserSelectedAll' => true
                    ]
                ],
                'labelRelations' => [
                    'operator' => 'AND',
                    'conditions' => [
                        null
                    ]
                ]
            ],
            'toolMask' => null,
            'connectorMask' => null,
            'thinkingConfig' => null,
            'status' => 'open',
            'firstResponseAt' => 'Example string',
            'queuedAt' => 'Example string',
            'assignedAt' => 'Example string',
            'transferReason' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred with the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields required by the List view; removes inboxes, mcp_credentials, and
api_credentials
to avoid N+1 query problems. email / phone_number / metadata are Contact's own scalar fields,
which the frontend conversation sidebar needs to display, with no N+1 cost.
  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May be of different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
```

```
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
```

```
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
  "title": string
  "lastMessage"?: { // Optional
  {
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
  }
  }
  "callRecord":
  {
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint's description for details (Optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an empty array or null
= nothing can be queried (no permission), omitted = no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected, depending
on hasUserSelectedAll) (Optional)
```

```
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name of chatbotFileIds; if both
are provided, this field takes precedence) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the items in the list; false = select only the items in the list (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array = no label
filtering (this does not mean no permission). Must be provided together with knowledgeBases —
when provided alone, knowledge base retrieval does not apply label filtering (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
  {
    "type"?: string // Form type; always reasoning_effort (Optional)
    "fields"?: object // The reasoning_effort field: options lists the thinking levels this
model accepts (from weakest to strongest), and default is always null (Optional)
    "canDisable"?: boolean // Whether thinking can be turned off for this model; when true,
options includes off (Optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model decides on its own, or the platform default level enables thinking
(Optional)
    "unsetLevel"?: string // The level the platform actually sends when no level is set; null
means no thinking parameter is sent and the model's own default applies (Optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
  "llm": string (uuid) // The model the next reply runs on: the conversation's pin, else its
assistant's current model.
```

```
TC-0205: returning the stored ``conversation.llm`` would show a model an unpinned conversation
no
longer uses once its assistant switches model, while ``thinking_config_schema`` already follows
the
effective model. Read-only: displaying it never marks the conversation as explicitly selected;
only the update-llm endpoint does.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // May have different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"?: [ // Optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
  "satisfactionComment": string
```

```
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
  "clientPlatform":  // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled = platform switch; a continuation
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
```

```
    "phoneNumber": "Response string",
    "metadata": null,
    "sourceId": "Response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Response string",
          "url": "Response string",
          "realm": "Response string"
        },
        "line": {
          "liffId": "Response string"
        },
        "ad": {
          "saml": "Response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "Response string"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "title": "Response string",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Response string",
    "content": "Response string",
    "createdAt": "Response string"
  },
```

```
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "Response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Response string",
  "createdAt": "Response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response string",
  "firstResponseAt": "Response string",
```

```
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
  "satisfactionSubmittedAt": "Response string",
  "latestAnalysisSummary": "Response string",
  "clientPlatform": {},
  "workingDirectory": "Response string",
  "slideProjectId": "Response string",
  "matchedField": "Response string",
  "matchedSnippet": "Response string",
  "matchedMessageId": "Response string",
  "pinnedAt": "Response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "displayContextUsage": false
}
```


Claim Conversation

PATCH `/api/v1/conversations/{id}/claim/`

Uses the caller as the member taking over the conversation, with the same effect as "Assign Conversation".

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Request Body

Request Parameters

Field	Type	Required	Description
contact	object	No	Lightweight Contact Serializer for the Conversation List view. Includes only the fields needed by the List view, omitting inboxes, mcp_credentials, and api_credentials to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself, needed for display in the frontend conversation sidebar, with no N+1 cost.
contact.name	string	Yes	
contact.avatar	string	No	Stores our own media as a relative storage key, or an external CDN URL as-is.
contact.email	object	No	
contact.phoneNumber	string	No	
contact.metadata	object	No	Custom customer information shown in the customer panel of the conversation page
contact.sourceId	string	No	
inbox	object	No	Inbox summary embedded in a conversation payload. `unread_conversations_count` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS em...
inbox.name	string	Yes	
inbox.channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)	Yes	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` : Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
inbox.unreadConversationsCount	integer	No	

Field	Type	Required	Description
inbox.signAuth	object (with 8 properties: id, signSource, signParams...)	No	
inbox.signAuth.signSource	string (uuid)	Yes	
inbox.signAuth.signParams	object	Yes	
inbox.signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
inbox.signAuth.signParams.line	object (with 1 properties: liffId)	Yes	
inbox.signAuth.signParams.ad	object (with 1 properties: saml)	Yes	
inbox.signAuth.signParams.google	object (with 1 properties: enabled)	No	
inbox.signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
inbox.signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
inbox.signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
inbox.signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools

Field	Type	Required	Description
			on their behalf, so tools do not require a separate per-to...
inbox.signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
inbox.enableAnalysis	boolean	No	
lastMessage	object	No	
lastMessage.type	string	No	
lastMessage.content	string	No	
enableGroupMention	boolean	No	
queryMetadata	object	No	Query metadata that controls the scope of knowledge that can be queried; see each endpoint's description for details
queryMetadata.knowledgeBases	array[object]	No	List of knowledge bases that can be queried; an empty array or null = nothing can be queried (no permission), omitted = no restriction
queryMetadata.labelRelations	object	No	Label filter conditions; an empty conditions array = no label filtering (this does not mean no permission). Must be provided together with knowledgeBases — when provided alone, knowledge base retrieval does not apply label filtering (equivalent to no restriction)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How the label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
toolMask	object	No	Enabled/disabled state per tool: {tool_id: bool}
connectorMask	object	No	Enabled/disabled state per connector: {connector_id: bool}
thinkingConfig	object	No	None = not set (inherit the chatbot setting); otherwise

Field	Type	Required	Description
			{"reasoning_effort": }
status	string (enum: open, resolved, queued)	No	`open` : Open ; `resolved` : Resolved ; `queued` : Queued;
firstResponseAt	string (timestamp)	No	
queuedAt	string (timestamp)	No	
assignedAt	string (timestamp)	No	
transferReason	string	No	

Request Schema Example

```
{
  "contact"?: { // Lightweight Contact Serializer for the Conversation List view.

Includes only the fields needed by the List view, omitting inboxes, mcp_credentials, and
api_credentials
to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself,
needed for display in the frontend conversation sidebar, with no N+1 cost. (Optional)
  {
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May have different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
  }
  }
  "inbox"?: { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``. (Optional)
  {
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
```

```
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own
```

```
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (Optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "lastMessage"?: { // Optional
  {
    "type"?: string // Optional
    "content"?: string // Optional
  }
  }
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint's description for details (Optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an empty array or null
= nothing can be queried (no permission), not provided = no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected, depending
on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name for chatbotFileIds; if both
are provided, this field takes precedence) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // FAQ ID list (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true= select all content in the knowledge base except
the items in the list; false = select only the items in the list (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array = no label
filtering (not a lack of permission). Must be provided together with knowledgeBases — when
provided alone, knowledge base retrieval does not apply label filtering (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
```

```
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
}
```

Request Example

```
{
  "contact": {
    "name": "Example name",
    "avatar": "Example string",
    "email": null,
    "phoneNumber": "Example string",
    "metadata": null,
    "sourceId": "Example string"
  },
  "inbox": {
    "name": "Example name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Example string",
          "url": "Example string",
          "realm": "Example string"
        },
        "line": {
          "liffId": "Example string"
        },
        "ad": {
          "saml": "Example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "Example string"
      ]
    },
    "enableAnalysis": true
  },
```

```
  "lastMessage": {
    "type": "Example string",
    "content": "Hello! I'd like to learn about your products."
  },
  "enableGroupMention": true,
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "status": "open",
  "firstResponseAt": "Example string",
  "queuedAt": "Example string",
  "assignedAt": "Example string",
  "transferReason": "Example string"
}
```


Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/claim/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
    },
    "lastMessage": {
      "type": "Example string",
```

```
      "content": "Hello! I'd like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  }'

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload (payload)
const data = {
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
```

```
    },
    "lastMessage": {
      "type": "Example string",
      "content": "Hello! I'd like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  };

axios.patch("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/claim/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/claim/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload (payload)
data = {
    "contact": {
        "name": "Example name",
        "avatar": "Example string",
        "email": None,
        "phoneNumber": "Example string",
        "metadata": None,
        "sourceId": "Example string"
    },
    "inbox": {
        "name": "Example name",
        "channelType": "line",
        "unreadConversationsCount": 123,
        "signAuth": {
            "signSource": "550e8400-e29b-41d4-a716-446655440000",
            "signParams": {
                "keycloak": {
                    "clientId": "Example string",
                    "url": "Example string",
                    "realm": "Example string"
                },
                "line": {
                    "liffId": "Example string"
                },
                "ad": {
                    "saml": "Example string"
                },
                "google": {
                    "enabled": True
                }
            },
            "sourceIdAccessEnabled": True,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": True,
            "keycloakTokenPassthroughAllowedHosts": [
                "Example string"
            ]
        },
        "enableAnalysis": True
    },
```

```
    "lastMessage": {
        "type": "Example string",
        "content": "Hello! I'd like to learn about your products."
    },
    "enableGroupMention": True,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": True
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                None
            ]
        }
    },
    "toolMask": None,
    "connectorMask": None,
    "thinkingConfig": None,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/claim/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'contact' => [
                'name' => 'Example name',
                'avatar' => 'Example string',
                'email' => null,
                'phoneNumber' => 'Example string',
                'metadata' => null,
                'sourceId' => 'Example string'
            ],
            'inbox' => [
                'name' => 'Example name',
                'channelType' => 'line',
                'unreadConversationsCount' => 123,
                'signAuth' => [
                    'signSource' => '550e8400-e29b-41d4-a716-446655440000',
                    'signParams' => [
                        'keycloak' => [
                            'clientId' => 'Example string',
                            'url' => 'Example string',
                            'realm' => 'Example string'
                        ],
                        'line' => [
                            'liffId' => 'Example string'
                        ],
                        'ad' => [
                            'saml' => 'Example string'
                        ],
                        'google' => [
                            'enabled' => true
                        ]
                    ],
                    'sourceIdAccessEnabled' => true,
                    'sourceIdVerifyMode' => new \stdClass(),
                    'sourceIdSignatureTtlMinutes' => new \stdClass(),
                    'keycloakTokenPassthrough' => true,
                    'keycloakTokenPassthroughAllowedHosts' => [
                        'Example string'
                    ]
                ],
```

```
                'enableAnalysis' => true
            ],
            'lastMessage' => [
                'type' => 'Example string',
                'content' => 'Hello! I'd like to learn about your products.'
            ],
            'enableGroupMention' => true,
            'queryMetadata' => [
                'knowledgeBases' => [
                    [
                        'knowledgeBaseId' => '550e8400-e29b-41d4-a716-446655440000',
                        'chatbotFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'knowledgeBaseFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'faqIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'hasUserSelectedAll' => true
                    ]
                ],
                'labelRelations' => [
                    'operator' => 'AND',
                    'conditions' => [
                        null
                    ]
                ]
            ],
            'toolMask' => null,
            'connectorMask' => null,
            'thinkingConfig' => null,
            'status' => 'open',
            'firstResponseAt' => 'Example string',
            'queuedAt' => 'Example string',
            'assignedAt' => 'Example string',
            'transferReason' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer, used for the Conversation List view

Only includes the fields required by the List view; inboxes, mcp_credentials and
api_credentials are removed
to avoid N+1 query problems. email / phone_number / metadata are scalar fields of the Contact
itself;
the frontend conversation sidebar needs to display them, with no N+1 cost.
  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May be of different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
```

```
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at
```

```
login, never the contact Source ID. (Optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
  "title": string
  "lastMessage"?: { // Optional
  {
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
  }
  }
  "callRecord":
  {
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge available for
querying; see each endpoint description for details (Optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases available for querying; an empty array or
null means nothing can be queried (no permission), while omitting this field means no
restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
```

```
        "chatbotFileIds"?: [ // List of file IDs (treated as exclusions or as the only
selections, depending on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; if
both are provided, this field takes precedence) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (treated as exclusions or as the only selections,
depending on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the items in the list; false = select only the items in the list (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array means no label
filtering is applied (not a lack of permission). It must be provided together with
knowledgeBases — when provided alone, knowledge base retrieval does not apply label filtering
(equivalent to no restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; nested operator/conditions combinations are
supported (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
  {
    "type"?: string // Form type, always reasoning_effort (Optional)
    "fields"?: object // reasoning_effort field: options lists the thinking levels this model
accepts (from weakest to strongest), and default is always null (Optional)
    "canDisable"?: boolean // Whether this model can turn thinking off; when true, options
includes off (Optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model thinks on its own, or the platform default level enables thinking
(Optional)
    "unsetLevel"?: string // The level the platform actually sends when none is set; null means
no thinking parameter is sent and the model's own default applies (Optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
```

```
  "llm": string (uuid) // The model the next reply runs on: the conversation's pin, else its
assistant's current model.

TC-0205: returning the stored ``conversation.llm`` would show a model an unpinned conversation
no
longer uses once its assistant switches model, while ``thinking_config_schema`` already follows
the
effective model. Read-only: displaying it never marks the conversation as explicitly selected;
only the update-llm endpoint does.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // May have different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"?: [ // Optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
```

```
dissatisfied, 5=very satisfied)
  "satisfactionComment": string
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
  "clientPlatform":  // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled = platform switch; a continuing
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
```

```
    "avatar": "Response string",
    "email": "response@example.com",
    "phoneNumber": "Response string",
    "metadata": null,
    "sourceId": "Response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Response string",
          "url": "Response string",
          "realm": "Response string"
        },
        "line": {
          "liffId": "Response string"
        },
        "ad": {
          "saml": "Response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "Response string"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "title": "Response string",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Response string",
    "content": "Response string",
```

```
    "createdAt": "Response string"
  },
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "Response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Response string",
  "createdAt": "Response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
```

```
  "progressStatus": "Response string",
  "firstResponseAt": "Response string",
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
  "satisfactionSubmittedAt": "Response string",
  "latestAnalysisSummary": "Response string",
  "clientPlatform": {},
  "workingDirectory": "Response string",
  "slideProjectId": "Response string",
  "matchedField": "Response string",
  "matchedSnippet": "Response string",
  "matchedMessageId": "Response string",
  "pinnedAt": "Response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "displayContextUsage": false
}
```


Transfer a Conversation to Another Member

PATCH `/api/v1/conversations/{id}/transfer/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Request Body

Request Parameters

Field	Type	Required	Description
contact	object	No	Lightweight Contact serializer used by the Conversation List view. It includes only the fields the List view needs and removes inboxes, mcp_credentials, and api_credentials to avoid N+1 query problems. email / phone_number / metadata are scalar fields on the Contact itself that the frontend conversation sidebar needs to display, with no N+1 cost.
contact.name	string	Yes	
contact.avatar	string	No	Stores our own media as a relative storage key, or an external CDN URL as-is.
contact.email	object	No	
contact.phoneNumber	string	No	
contact.metadata	object	No	Custom customer information shown in the customer panel of the conversation page
contact.sourceId	string	No	
inbox	object	No	Inbox summary embedded in a conversation payload. `unread_conversations_count` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS em...
inbox.name	string	Yes	
inbox.channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)	Yes	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` : Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
inbox.unreadConversationsCount	integer	No	

Field	Type	Required	Description
inbox.signAuth	object (with 8 properties: id, signSource, signParams...)	No	
inbox.signAuth.signSource	string (uuid)	Yes	
inbox.signAuth.signParams	object	Yes	
inbox.signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
inbox.signAuth.signParams.line	object (with 1 properties: liffId)	Yes	
inbox.signAuth.signParams.ad	object (with 1 properties: saml)	Yes	
inbox.signAuth.signParams.google	object (with 1 properties: enabled)	No	
inbox.signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
inbox.signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
inbox.signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
inbox.signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools

Field	Type	Required	Description
			on their behalf, so tools do not require a separate per-to...
inbox.signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
inbox.enableAnalysis	boolean	No	
lastMessage	object	No	
lastMessage.type	string	No	
lastMessage.content	string	No	
enableGroupMention	boolean	No	
queryMetadata	object	No	Query metadata that controls the scope of knowledge available for querying; see each endpoint description for details
queryMetadata.knowledgeBases	array[object]	No	List of knowledge bases available for querying; an empty array or null means nothing can be queried (no permission), while omitting this field means no restriction
queryMetadata.labelRelations	object	No	Label filter conditions; an empty conditions array means no label filtering is applied (not a lack of permission). It must be provided together with knowledgeBases — when provided alone, knowledge base retrieval does not apply label filtering (equivalent to no restriction)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How the label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; nested operator/conditions combinations are supported
toolMask	object	No	Enabled/disabled state per tool: {tool_id: bool}
connectorMask	object	No	Enabled/disabled state per connector: {connector_id: bool}
thinkingConfig	object	No	None = not set (inherit the chatbot setting); otherwise

Field	Type	Required	Description
			{"reasoning_effort": }
status	string (enum: open, resolved, queued)	No	`open` : Open ; `resolved` : Resolved ; `queued` : Queued;
firstResponseAt	string (timestamp)	No	
queuedAt	string (timestamp)	No	
assignedAt	string (timestamp)	No	
transferReason	string	No	

Request Schema Example

```
{
  "contact"?: { // Lightweight Contact serializer used by the Conversation List view.

It includes only the fields the List view needs and removes inboxes, mcp_credentials, and
api_credentials
to avoid N+1 query problems. email / phone_number / metadata are scalar fields on the Contact
itself
that the frontend conversation sidebar needs to display, with no N+1 cost. (Optional)
  {
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May have different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
  }
  }
  "inbox"?: { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``. (Optional)
  {
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
```

```
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign
```

```
source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (Optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "lastMessage"?: { // Optional
  {
    "type"?: string // Optional
    "content"?: string // Optional
  }
  }
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge that can be
queried; see each endpoint description for details (Optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an empty array or null
= nothing can be queried (no permission), omitted = unrestricted (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected, depending
on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name of chatbotFileIds; if both
are provided, this field takes precedence) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the items in the lists; false = select only the items in the lists (Optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions array = no label
filtering (not "no permission"). Must be provided together with knowledgeBases — when provided
alone, knowledge base retrieval does not apply label filtering (equivalent to unrestricted)
(Optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
```

```
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
}
```

Example Request Values

```
{
  "contact": {
    "name": "Example name",
    "avatar": "Example string",
    "email": null,
    "phoneNumber": "Example string",
    "metadata": null,
    "sourceId": "Example string"
  },
  "inbox": {
    "name": "Example name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Example string",
          "url": "Example string",
          "realm": "Example string"
        },
        "line": {
          "liffId": "Example string"
        },
        "ad": {
          "saml": "Example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "Example string"
      ]
    },
    "enableAnalysis": true
```

```
  },
  "lastMessage": {
    "type": "Example string",
    "content": "Hello! I would like to learn about the product."
  },
  "enableGroupMention": true,
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "status": "open",
  "firstResponseAt": "Example string",
  "queuedAt": "Example string",
  "assignedAt": "Example string",
  "transferReason": "Example string"
}
```


Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/transfer/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
    },
    "lastMessage": {
      "type": "Example string",
```

```
      "content": "Hello! I would like to learn about the product."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
```

```
    },
    "lastMessage": {
      "type": "Example string",
      "content": "Hello! I would like to learn about the product."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  };

axios.patch("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/transfer/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/transfer/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "contact": {
        "name": "Example name",
        "avatar": "Example string",
        "email": None,
        "phoneNumber": "Example string",
        "metadata": None,
        "sourceId": "Example string"
    },
    "inbox": {
        "name": "Example name",
        "channelType": "line",
        "unreadConversationsCount": 123,
        "signAuth": {
            "signSource": "550e8400-e29b-41d4-a716-446655440000",
            "signParams": {
                "keycloak": {
                    "clientId": "Example string",
                    "url": "Example string",
                    "realm": "Example string"
                },
                "line": {
                    "liffId": "Example string"
                },
                "ad": {
                    "saml": "Example string"
                },
                "google": {
                    "enabled": True
                }
            },
            "sourceIdAccessEnabled": True,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": True,
            "keycloakTokenPassthroughAllowedHosts": [
                "Example string"
            ]
        },
        "enableAnalysis": True
    },
```

```
    "lastMessage": {
        "type": "Example string",
        "content": "Hello! I would like to learn about the product."
    },
    "enableGroupMention": True,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": True
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                None
            ]
        }
    },
    "toolMask": None,
    "connectorMask": None,
    "thinkingConfig": None,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/transfer/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'contact' => [
                'name' => 'Example name',
                'avatar' => 'Example string',
                'email' => null,
                'phoneNumber' => 'Example string',
                'metadata' => null,
                'sourceId' => 'Example string'
            ],
            'inbox' => [
                'name' => 'Example name',
                'channelType' => 'line',
                'unreadConversationsCount' => 123,
                'signAuth' => [
                    'signSource' => '550e8400-e29b-41d4-a716-446655440000',
                    'signParams' => [
                        'keycloak' => [
                            'clientId' => 'Example string',
                            'url' => 'Example string',
                            'realm' => 'Example string'
                        ],
                        'line' => [
                            'liffId' => 'Example string'
                        ],
                        'ad' => [
                            'saml' => 'Example string'
                        ],
                        'google' => [
                            'enabled' => true
                        ]
                    ],
                    'sourceIdAccessEnabled' => true,
                    'sourceIdVerifyMode' => new \stdClass(),
                    'sourceIdSignatureTtlMinutes' => new \stdClass(),
                    'keycloakTokenPassthrough' => true,
                    'keycloakTokenPassthroughAllowedHosts' => [
                        'Example string'
                    ]
                ],
```

```
                'enableAnalysis' => true
            ],
            'lastMessage' => [
                'type' => 'Example string',
                'content' => 'Hello! I would like to learn about the product.'
            ],
            'enableGroupMention' => true,
            'queryMetadata' => [
                'knowledgeBases' => [
                    [
                        'knowledgeBaseId' => '550e8400-e29b-41d4-a716-446655440000',
                        'chatbotFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'knowledgeBaseFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'faqIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'hasUserSelectedAll' => true
                    ]
                ],
                'labelRelations' => [
                    'operator' => 'AND',
                    'conditions' => [
                        null
                    ]
                ]
            ],
            'toolMask' => null,
            'connectorMask' => null,
            'thinkingConfig' => null,
            'status' => 'open',
            'firstResponseAt' => 'Example string',
            'queuedAt' => 'Example string',
            'assignedAt' => 'Example string',
            'transferReason' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer for the Conversation List view

It includes only the fields the List view needs and removes inboxes, mcp_credentials, and
api_credentials
to avoid N+1 query problems. email / phone_number / metadata are scalar fields on the Contact
itself
that the frontend conversation sidebar needs to display, so they add no N+1 cost.
  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May have different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
```

```
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at
```

```
login, never the contact Source ID. (Optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
  "title": string
  "lastMessage"?: { // Optional
  {
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
  }
  }
  "callRecord":
  {
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata controlling which knowledge can be queried; see each
endpoint description for details (Optional)
  {
    "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array or null means
none can be queried (no access), while omitting it imposes no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected, depending
```

```
on hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name of chatbotFileIds; if both
are provided, this field takes precedence) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true selects all content of the knowledge base and
excludes the items in the list; false selects only the items in the list (Optional)
      }
    ]
    "labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not no access). Must be provided together with knowledgeBases; when provided alone,
label filtering is not applied to knowledge base retrieval (equivalent to no restriction)
(Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions, supporting nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
  {
    "type"?: string // Form type, always reasoning_effort (Optional)
    "fields"?: object // The reasoning_effort field: options lists the thinking levels this
model accepts (from weakest to strongest), and default is always null (Optional)
    "canDisable"?: boolean // Whether this model can turn thinking off; when true, options
includes off (Optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off means no thinking (fastest);
model_default means the model thinks on its own, or the platform default level enables thinking
(Optional)
    "unsetLevel"?: string // The level the platform actually sends when no level is set; null
means no thinking parameter is sent, leaving it to the model's own default (Optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
  "llm": string (uuid) // The model the next reply runs on: the conversation's pin, else its
```

```
assistant's current model.

TC-0205: returning the stored ``conversation.llm`` would show a model an unpinned conversation
no
longer uses once its assistant switches model, while ``thinking_config_schema`` already follows
the
effective model. Read-only: displaying it never marks the conversation as explicitly selected;
only the update-llm endpoint does.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // May have different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"?: [ // Optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
```

```
  "satisfactionComment": string
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
  "clientPlatform":  // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled is the platform toggle; the continuation
conversation includes source_conversation_id/source_title; the original conversation being
handed off includes continued_to_conversation_id/continued_to_title and
latest_summary_message_id
  "displayContextUsage": boolean // Whether to display the context usage percentage bar (a
conversation platform toggle, always true in MaiGPT; independent of the handoff toggle)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
```

```
    "avatar": "response string",
    "email": "response@example.com",
    "phoneNumber": "response string",
    "metadata": null,
    "sourceId": "response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "response string",
          "url": "response string",
          "realm": "response string"
        },
        "line": {
          "liffId": "response string"
        },
        "ad": {
          "saml": "response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "response string"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "title": "response string",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "response string",
    "content": "response string",
```

```
    "createdAt": "response string"
  },
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "response string",
  "createdAt": "response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "response string",
  "ipCountryCode": "response string",
  "ipRecordedAt": "response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
```

```
  "progressStatus": "response string",
  "firstResponseAt": "response string",
  "queuedAt": "response string",
  "assignedAt": "response string",
  "transferReason": "response string",
  "satisfactionScore": 456,
  "satisfactionComment": "response string",
  "satisfactionSubmittedAt": "response string",
  "latestAnalysisSummary": "response string",
  "clientPlatform": {},
  "workingDirectory": "response string",
  "slideProjectId": "response string",
  "matchedField": "response string",
  "matchedSnippet": "response string",
  "matchedMessageId": "response string",
  "pinnedAt": "response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "displayContextUsage": false
}
```


Return to AI Replies

PATCH `/api/v1/conversations/{id}/return-to-ai/`

Clears the assigned member and restores automatic AI replies; sends a handoff.returned_to_ai Webhook event when complete.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Conversation.

Request Body

Request Parameters

Field	Type	Required	Description
contact	object	No	Lightweight Contact Serializer for the Conversation List view. Includes only fields required by the List view, removing inboxes, mcp_credentials, and api_credentials to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself, needed for display in the conversation sidebar, with no N+1 overhead.
contact.name	string	Yes	
contact.avatar	string	No	Stores our own media as a relative storage key, or an external CDN URL as-is.
contact.email	object	No	
contact.phoneNumber	string	No	
contact.metadata	object	No	Custom customer information shown in the customer panel of the conversation page
contact.sourceId	string	No	
inbox	object	No	Inbox summary embedded in a conversation payload. `unread_conversations_count` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS em...
inbox.name	string	Yes	
inbox.channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)	Yes	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` : Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
inbox.unreadConversationsCount	integer	No	
inbox.signAuth	object (with 8 properties: id,	No	

Field	Type	Required	Description
	signSource, signParams...)		
inbox.signAuth.signSource	string (uuid)	Yes	
inbox.signAuth.signParams	object	Yes	
inbox.signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
inbox.signAuth.signParams.line	object (with 1 property: liffId)	Yes	
inbox.signAuth.signParams.ad	object (with 1 property: saml)	Yes	
inbox.signAuth.signParams.google	object (with 1 property: enabled)	No	
inbox.signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
inbox.signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
inbox.signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
inbox.signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-to...

Field	Type	Required	Description
inbox.signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
inbox.enableAnalysis	boolean	No	
lastMessage	object	No	
lastMessage.type	string	No	
lastMessage.content	string	No	
enableGroupMention	boolean	No	
queryMetadata	object	No	Query metadata controlling which knowledge can be queried; see each endpoint description for details
queryMetadata.knowledgeBases	array[object]	No	List of queryable knowledge bases; an empty array or null means none can be queried (no access), while omitting it imposes no restriction
queryMetadata.labelRelations	object	No	Label filter criteria; an empty conditions array means no label filtering (not no access). Must be provided together with knowledgeBases; when provided alone, label filtering is not applied to knowledge base retrieval (equivalent to no restriction)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions, supporting nested operator/conditions combinations
toolMask	object	No	Enabled/disabled state per tool: {tool_id: bool}
connectorMask	object	No	Enabled/disabled state per connector: {connector_id: bool}
thinkingConfig	object	No	None = not set (inherit the chatbot setting); otherwise {"reasoning_effort": }
status	string (enum: open,	No	`open` : Open ; `resolved` : Resolved ; `queued` : Queued;

Field	Type	Required	Description
	resolved, queued)		
firstResponseAt	string (timestamp)	No	
queuedAt	string (timestamp)	No	
assignedAt	string (timestamp)	No	
transferReason	string	No	

Request Schema Example

```
{
  "contact"?: { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields needed by the List view, omitting inboxes, mcp_credentials, and
api_credentials
to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself,
needed for display in the frontend conversation sidebar, with no N+1 cost. (Optional)
  {
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // May have different types (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
  }
  }
  "inbox"?: { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``. (Optional)
  {
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
```

```
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
```

```
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "lastMessage"?: { // Optional
  {
    "type"?: string // Optional
    "content"?: string // Optional
  }
  }
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge available for
querying; see each endpoint description for details (Optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases available for querying; an empty array or
null = nothing can be queried (no permission), omitted = no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // File ID list (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // File ID list (the new name for chatbotFileIds; this field
takes precedence when both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // FAQ ID list (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the items in the lists; false = select only the items in the lists (Optional)
      }
    ]
    "labelRelations"?: { // Label filters; an empty conditions array means no label filtering
(not a lack of permissions). Must be provided together with knowledgeBases. When provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to no
restriction) (Optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are combined
      "conditions"?: [ // List of label conditions; nested operator/conditions combinations are
supported (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
```

```
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
}
```

Example Request Values


```
{
  "contact": {
    "name": "Example name",
    "avatar": "Example string",
    "email": null,
    "phoneNumber": "Example string",
    "metadata": null,
    "sourceId": "Example string"
  },
  "inbox": {
    "name": "Example name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Example string",
          "url": "Example string",
          "realm": "Example string"
        },
        "line": {
          "liffId": "Example string"
        },
        "ad": {
          "saml": "Example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "Example string"
      ]
    },
    "enableAnalysis": true
  },
  "lastMessage": {
    "type": "Example string",
    "content": "Hello, I would like to learn about your products."
```

```
  },
  "enableGroupMention": true,
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "status": "open",
  "firstResponseAt": "Example string",
  "queuedAt": "Example string",
  "assignedAt": "Example string",
  "transferReason": "Example string"
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/return-to-ai/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
    },
    "lastMessage": {
      "type": "Example string",
```

```
      "content": "Hello, I would like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  }'

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "contact": {
      "name": "Example name",
      "avatar": "Example string",
      "email": null,
      "phoneNumber": "Example string",
      "metadata": null,
      "sourceId": "Example string"
    },
    "inbox": {
      "name": "Example name",
      "channelType": "line",
      "unreadConversationsCount": 123,
      "signAuth": {
        "signSource": "550e8400-e29b-41d4-a716-446655440000",
        "signParams": {
          "keycloak": {
            "clientId": "Example string",
            "url": "Example string",
            "realm": "Example string"
          },
          "line": {
            "liffId": "Example string"
          },
          "ad": {
            "saml": "Example string"
          },
          "google": {
            "enabled": true
          }
        },
        "sourceIdAccessEnabled": true,
        "sourceIdVerifyMode": {},
        "sourceIdSignatureTtlMinutes": {},
        "keycloakTokenPassthrough": true,
        "keycloakTokenPassthroughAllowedHosts": [
          "Example string"
        ]
      },
      "enableAnalysis": true
```

```
    },
    "lastMessage": {
      "type": "Example string",
      "content": "Hello, I would like to learn about your products."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  };

axios.patch("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/return-to-ai/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-
446655440000/return-to-ai/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "contact": {
        "name": "Example name",
        "avatar": "Example string",
        "email": None,
        "phoneNumber": "Example string",
        "metadata": None,
        "sourceId": "Example string"
    },
    "inbox": {
        "name": "Example name",
        "channelType": "line",
        "unreadConversationsCount": 123,
        "signAuth": {
            "signSource": "550e8400-e29b-41d4-a716-446655440000",
            "signParams": {
                "keycloak": {
                    "clientId": "Example string",
                    "url": "Example string",
                    "realm": "Example string"
                },
                "line": {
                    "liffId": "Example string"
                },
                "ad": {
                    "saml": "Example string"
                },
                "google": {
                    "enabled": True
                }
            },
            "sourceIdAccessEnabled": True,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": True,
            "keycloakTokenPassthroughAllowedHosts": [
                "Example string"
            ]
        },
        "enableAnalysis": True
    },
```

```
    "lastMessage": {
        "type": "Example string",
        "content": "Hello, I would like to learn about your products."
    },
    "enableGroupMention": True,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": True
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                None
            ]
        }
    },
    "toolMask": None,
    "connectorMask": None,
    "thinkingConfig": None,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/return-to-ai/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'contact' => [
                'name' => 'Example name',
                'avatar' => 'Example string',
                'email' => null,
                'phoneNumber' => 'Example string',
                'metadata' => null,
                'sourceId' => 'Example string'
            ],
            'inbox' => [
                'name' => 'Example name',
                'channelType' => 'line',
                'unreadConversationsCount' => 123,
                'signAuth' => [
                    'signSource' => '550e8400-e29b-41d4-a716-446655440000',
                    'signParams' => [
                        'keycloak' => [
                            'clientId' => 'Example string',
                            'url' => 'Example string',
                            'realm' => 'Example string'
                        ],
                        'line' => [
                            'liffId' => 'Example string'
                        ],
                        'ad' => [
                            'saml' => 'Example string'
                        ],
                        'google' => [
                            'enabled' => true
                        ]
                    ],
                    'sourceIdAccessEnabled' => true,
                    'sourceIdVerifyMode' => new \stdClass(),
                    'sourceIdSignatureTtlMinutes' => new \stdClass(),
                    'keycloakTokenPassthrough' => true,
                    'keycloakTokenPassthroughAllowedHosts' => [
                        'Example string'
                    ]
                ],
```

```
                'enableAnalysis' => true
            ],
            'lastMessage' => [
                'type' => 'Example string',
                'content' => 'Hello, I would like to learn about your products.'
            ],
            'enableGroupMention' => true,
            'queryMetadata' => [
                'knowledgeBases' => [
                    [
                        'knowledgeBaseId' => '550e8400-e29b-41d4-a716-446655440000',
                        'chatbotFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'knowledgeBaseFileIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'faqIds' => [
                            '550e8400-e29b-41d4-a716-446655440000'
                        ],
                        'hasUserSelectedAll' => true
                    ]
                ],
                'labelRelations' => [
                    'operator' => 'AND',
                    'conditions' => [
                        null
                    ]
                ]
            ],
            'toolMask' => null,
            'connectorMask' => null,
            'thinkingConfig' => null,
            'status' => 'open',
            'firstResponseAt' => 'Example string',
            'queuedAt' => 'Example string',
            'assignedAt' => 'Example string',
            'transferReason' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields needed by the List view, omitting inboxes, mcp_credentials, and
api_credentials
to avoid N+1 queries. email / phone_number / metadata are scalar fields on Contact itself,
needed for display in the frontend conversation sidebar, with no N+1 cost.
  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
    "email"?: // Multiple types possible (Optional)
    string (email) // Optional
    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
```

```
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?:  // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // Optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
```

```
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // Optional
  }
  }
  "chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
  "title": string
  "lastMessage"?: { // Optional
  {
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
  }
  }
  "callRecord":
  {
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"?: boolean // Optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge available for
querying; see each endpoint description for details (Optional)
  {
    "knowledgeBases"?: [ // Knowledge bases available for querying; an empty array or null =
nothing can be queried (no permissions), omitted = no restriction (Optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // File ID list (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
```

```
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // File ID list (new name for chatbotFileIds; this field
takes precedence when both are provided) (Optional)
          string (uuid)
        ]
        "faqIds"?: [ // FAQ ID list (excluded or exclusively selected, depending on
hasUserSelectedAll) (Optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true selects all knowledge base content except the
listed items; false selects only the listed items (Optional)
      }
    ]
    "labelRelations"?: { // Label filters; an empty conditions array = no label filtering (not
a lack of permissions). Must be provided together with knowledgeBases; when provided alone,
label filtering is not applied to knowledge base retrieval (equivalent to no restriction)
(Optional)
    {
      "operator": string (enum: AND, OR) // How label conditions are combined
      "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
  "connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting); otherwise
{"reasoning_effort": <level>} (Optional)
  "thinkingConfigSchema": { // Thinking configuration form for this model; null means this
model has no thinking configuration
  {
    "type"?: string // Form type, always reasoning_effort (Optional)
    "fields"?: object // reasoning_effort field: options lists the thinking levels this model
accepts (from weakest to strongest); default is always null (Optional)
    "canDisable"?: boolean // Whether this model can disable thinking; when true, options
includes off (Optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior when neither the
conversation nor the assistant sets a thinking level: off = no thinking (fastest);
model_default = the model decides how to think, or the platform default level enables thinking
(Optional)
    "unsetLevel"?: string // The level the platform actually sends when none is set; null means
no thinking parameter is sent and the model default applies (Optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this conversation's
model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default level; None means
the model sends no thinking parameter, so the provider default applies.
  "llm": string (uuid) // The model the next reply runs on: the conversation's pin, else its
assistant's current model.
```

```
TC-0205: returning the stored ``conversation.llm`` would show a model an unpinned conversation
no
longer uses once its assistant switches model, while ``thinking_config_schema`` already follows
the
effective model. Read-only: displaying it never marks the conversation as explicitly selected;
only the update-llm endpoint does.
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // May have different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"?: [ // Optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"?: string (timestamp) // Optional
  "queuedAt"?: string (timestamp) // Optional
  "assignedAt"?: string (timestamp) // Optional
  "transferReason"?: string // Optional
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
  "satisfactionComment": string
```

```
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
  "clientPlatform":  // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
  "handoff": object // Handoff information: enabled = platform switch; a continuation
conversation carries source_conversation_id/source_title; the original handed-off conversation
carries continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage percentage bar
(conversation platform switch; always true for MaiGPT; independent of the handoff switch)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
```

```
    "phoneNumber": "Response string",
    "metadata": null,
    "sourceId": "Response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Response string",
          "url": "Response string",
          "realm": "Response string"
        },
        "line": {
          "liffId": "Response string"
        },
        "ad": {
          "saml": "Response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "Response string"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "title": "Response string",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "Response string",
    "content": "Response string",
    "createdAt": "Response string"
  },
```

```
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "Response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Response string",
  "createdAt": "Response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response string",
  "firstResponseAt": "Response string",
```

```
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
  "satisfactionSubmittedAt": "Response string",
  "latestAnalysisSummary": "Response string",
  "clientPlatform": {},
  "workingDirectory": "Response string",
  "slideProjectId": "Response string",
  "matchedField": "Response string",
  "matchedSnippet": "Response string",
  "matchedMessageId": "Response string",
  "pinnedAt": "Response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "displayContextUsage": false
}
```


List Conversation Output Files

GET `/api/v1/conversations/{conversationId}/outputs/`

For response fields (count, source, results[]), visibility scope, and sorting, see the technical manual: Conversation Outputs List API.

Parameters

Parameter Name	Required	Type	Description
`conversationId`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/{conversationId}/outputs/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/{conversationId}/outputs/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/conversations/{conversationId}/outputs/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/conversations/{conversationId}/outputs/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	No response body

Share Conversation

POST `/api/v1/conversations/{conversationPk}/share/`

Parameters

Parameter Name	Required	Type	Description
`conversationPk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
shareScope	string (enum: organization, group, member)	Yes	`organization` : Organization ; `group` : Group ; `member` : Member;
permission	object	No	
title	string	No	
description	string	No	
groupIds	array[string]	No	
memberIds	array[string]	No	

Example Request Schema

```
{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization
* `group` - Group
* `member` - Member
  "permission"?:  // Optional
  {
```

```
  }
  "title"?: string // Optional
  "description"?: string // Optional
  "groupIds"?: [ // Optional
    string (uuid)
  ]
  "memberIds"?: [ // Optional
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Example Name",
  "description": "Example String",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "shareScope": "organization",
    "permission": {},
    "title": "Example Name",
    "description": "Example String",
    "groupIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "shareScope": "organization",
    "permission": {},
    "title": "Example Name",
    "description": "Example String",
    "groupIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "shareScope": "organization",
    "permission": {},
    "title": "Example Name",
    "description": "Example String",
    "groupIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'shareScope' => 'organization',
            'permission' => new \stdClass(),
            'title' => 'Example Name',
            'description' => 'Example String',
            'groupIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'memberIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization
```

```
* `group` - Group
* `member` - Member
  "permission"?:  // Optional
  {
  }
  "title"?: string // Optional
  "description"?: string // Optional
  "groupIds"?: [ // Optional
    string (uuid)
  ]
  "memberIds"?: [ // Optional
    string (uuid)
  ]
}
```

Example Response Values

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Response String",
  "description": "Response String",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Quickly Share a Conversation

POST `/api/v1/conversations/{conversationPk}/share/quick/`

Parameters

Parameter Name	Required	Type	Description
`conversationPk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
shareScope	string (enum: organization, group, member)	Yes	`organization` : Organization ; `group` : Group ; `member` : Member;
permission	object	No	
title	string	No	
description	string	No	
groupIds	array[string]	No	
memberIds	array[string]	No	

Example Request Schema

```
{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization
* `group` - Group
* `member` - Member
  "permission"?:  // Optional
  {
  }
  "title"?: string // Optional
  "description"?: string // Optional
  "groupIds"?: [ // Optional
    string (uuid)
  ]
  "memberIds"?: [ // Optional
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Example Name",
  "description": "Example String",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "shareScope": "organization",
    "permission": {},
    "title": "Example Name",
    "description": "Example String",
    "groupIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "shareScope": "organization",
    "permission": {},
    "title": "Example Name",
    "description": "Example String",
    "groupIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "shareScope": "organization",
    "permission": {},
    "title": "Example Name",
    "description": "Example String",
    "groupIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'shareScope' => 'organization',
            'permission' => new \stdClass(),
            'title' => 'Example Name',
            'description' => 'Example String',
            'groupIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'memberIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization
```

```
* `group` - Group
* `member` - Member
  "permission"?:  // Optional
  {
  }
  "title"?: string // Optional
  "description"?: string // Optional
  "groupIds"?: [ // Optional
    string (uuid)
  ]
  "memberIds"?: [ // Optional
    string (uuid)
  ]
}
```

Example Response Values

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Response String",
  "description": "Response String",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

List Shared Conversations

GET `/api/v1/shared-conversations/`

Parameters

Parameter Name	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-conversations/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "title": string // Custom title for the shared conversation
      "description": string // Custom description for the shared conversation
      "shareScope":
      {
      }
      "permission":
      {
      }
      "sharedBy":
      {
        "id": string (uuid)
        "name": string
      }
      "messageCount": integer
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "title": "Response String",
      "description": "Response String",
      "shareScope": {},
      "permission": {},
      "sharedBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response String"
      },
      "messageCount": 456,
      "createdAt": "Response String"
    }
```

```
  ]
}
```

Retrieve a Specific Shared Conversation

GET `/api/v1/shared-conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Shared Conversation.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
  "createdAt": string (timestamp)
```

```
  "conversationSnapshot": object // Frozen conversation snapshot: {title, messages[]}
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "Response String",
  "description": "Response String",
  "permission": {},
  "sharedBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  },
  "createdAt": "Response String",
  "conversationSnapshot": null
}
```


Update a Shared Conversation

PATCH `/api/v1/shared-conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Shared Conversation.

Request Body

Request Parameters

Field	Type	Required	Description
title	string	No	
description	string	No	
permission	string (enum: readonly, copy)	No	`readonly` : Read Only ; `copy` : Copyable;

Example Request Schema

```
{
  "title"?: string // Optional
  "description"?: string // Optional
  "permission"?: string (enum: readonly, copy) // * `readonly` - Read Only
* `copy` - Copyable (Optional)
}
```

Example Request Values

```
{
  "title": "Example Name",
  "description": "Example String",
  "permission": "readonly"
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "title": "Example Name",
    "description": "Example String",
    "permission": "readonly"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "title": "Example Name",
    "description": "Example String",
    "permission": "readonly"
  };

axios.patch("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "title": "Example Name",
    "description": "Example String",
    "permission": "readonly"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'title' => 'Example Name',
            'description' => 'Example String',
            'permission' => 'readonly'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "title"?: string // Optional
  "description"?: string // Optional
  "permission"?: string (enum: readonly, copy) // * `readonly` - Read Only
* `copy` - Copyable (Optional)
}
```

Example Response Values

```
{
  "title": "Response String",
  "description": "Response String",
  "permission": "readonly"
}
```


Delete a Shared Conversation

DELETE `/api/v1/shared-conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Shared Conversation.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Fork a Shared Conversation

POST `/api/v1/shared-conversations/{id}/fork/`

Parameters

Parameter Name	Required	Type	Description

id	V	string	A UUID string identifying this Shared Conversation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-
a716-446655440000/fork/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{}'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {};

axios.post("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/fork/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-
446655440000/fork/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/fork/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => new \stdClass()
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
  "createdAt": string (timestamp)
```

```
  "conversationSnapshot": object // Frozen conversation snapshot: {title, messages[]}
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "response string",
  "description": "response string",
  "permission": {},
  "sharedBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "createdAt": "response string",
  "conversationSnapshot": null
}
```


Get My Shared Conversations

GET `/api/v1/shared-conversations/my-shares/`

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/my-shares/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/my-shares/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/my-shares/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-conversations/my-
shares/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "shareScope":
  {
  }
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
```

```
  "messageCount": integer
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "response string",
  "description": "response string",
  "shareScope": {},
  "permission": {},
  "sharedBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "messageCount": 456,
  "createdAt": "response string"
}
```

Create Message Feedback

`POST` `/api/v1/messages/{messagePk}/feedback/`

Parameters

Parameter Name	Required	Type	Description
`messagePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
type	object	Yes	
suggestion	string	No	

Example Request Schema

```
{
  "type":
  {
  }
  "suggestion"?: string // Optional
}
```

Example Request Values

```
{
  "type": {},
  "suggestion": "example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "type": {},
    "suggestion": "example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "type": {},
    "suggestion": "example string"
  };

axios.post("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "type": {},
    "suggestion": "example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'type' => new \stdClass(),
            'suggestion' => 'example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // Optional
  "updatedAt": string (timestamp)
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "suggestion": "response string",
  "updatedAt": "response string"
}
```


Status Code: 400 - This message already has feedback. Use PATCH to update it.

Update Message Feedback

PUT `/api/v1/messages/{messagePk}/feedback/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Message Feedback.
`messagePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
type	object	Yes	
suggestion	string	No	

Example Request Schema

```
{
  "type":
  {
  }
  "suggestion"?: string // Optional
}
```

Example Request Values

```
{
  "type": {},
  "suggestion": "example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "type": {},
    "suggestion": "example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "type": {},
    "suggestion": "example string"
  };

axios.put("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "type": {},
    "suggestion": "example string"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'type' => new \stdClass(),
            'suggestion' => 'example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // Optional
  "updatedAt": string (timestamp)
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "suggestion": "response string",
  "updatedAt": "response string"
}
```


Delete Message Feedback

DELETE `/api/v1/messages/{messagePk}/feedback/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Message Feedback.
`messagePk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	Feedback deleted successfully
400	This feedback does not belong to the specified message

List Conversation Records

GET /api/v1/records/

Parameters

Parameter Name	Required	Type	Description
`callerMember`	X	array	Multiple values may be separated by commas.
`chatbot`	X	string	Filter by a specific AI assistant (UUID)
`endDate`	X	string	End date (format: YYYY-MM-DD, e.g., 2025-01-31)
`endDatetime`	X	string	
`errorType`	X	array	Multiple values may be separated by commas. `system`: System ; `llm`: LLM ; `embedding`: Embedding ; `reranker`: Reranker ; `vector_db`: Vector DB ; `workflow`: Workflow ; `tool`: Tool ; `timeout`: Ti...
`feedbackType`	X	array	Multiple values may be separated by commas. `like`: Like ; `dislike`: Dislike;
`hasError`	X	boolean	
`hasFeedback`	X	boolean	
`keyword`	X	string	Keyword search (searches both user messages and bot responses)
`largeLanguageModel`	X	string	Filter by a specific large language model (UUID)
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`source`	X	string	Filter by record source, separated by commas. If omitted, both sources are returned. interactive = user conversations, scheduled = scheduled executions.
`startDate`	X	string	Start date (format: YYYY-MM-DD, e.g., 2025-01-01)
`startDatetime`	X	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "senderName": string // Returns the name of the user who sent the message
      "callerMemberId": string
      "callerMemberName": string
      "feedback":
      {
```

```
        "id": string (uuid)
        "type":
        {
        }
        "suggestion"?: string // Optional
        "updatedAt": string (timestamp)
      }
      "inputMessage": string
      "condenseMessage": string // User message rewritten from the chat history and context.
      "outputMessage": string
      "faithfulnessScore": number (double) // Whether the chatbot reply is grounded in the
knowledge base rather than made up.
      "displayFaithfulnessScore": integer
      "answerRelevancyScore": number (double) // Whether the chatbot reply is relevant to the
user's question.
      "displayAnswerRelevancyScore": integer
      "contextPrecisionScore": number (double) // Whether the chatbot reply is relevant to the
reference context.
      "displayContextPrecisionScore": integer
      "answerCorrectnessScore": number (double) // Whether the chatbot reply is correct
(requires a ground-truth answer).
      "displayAnswerCorrectnessScore": integer
      "answerSimilarityScore": number (double) // Whether the chatbot reply is similar to the
ground-truth answer.
      "displayAnswerSimilarityScore": integer
      "contextRecallScore": number (double) // Whether the chatbot reply recalls the relevant
information from the reference context.
      "displayContextRecallScore": integer
      "replyTime": string
      "createdAt": string (timestamp)
      "error": string // Error message raised during execution.
      "errorType": // May have different types
      string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Error source: system, LLM, embedding,
reranker, and so on.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other
      "processingTime": object // Returns timing statistics for each processing stage,
including the streaming_metrics waterfall
      "usage": object // Returns usage statistics, including character counts, token counts,
and token details for each LLM call
      "citationNodes": [
        {
          "id": string
          "text": string
```

```
          "chatbotFileId": string
          "fileName": string
          "url": string
        }
      ]
      "inputHookLogs": [
        {
          "id": string (uuid)
          "hook": string (uuid)
          "hookName": string
          "hookTypeLabel": string
          "hookAction": string
          "conversation": string (uuid)
          "message": string (uuid)
          "messageContent": string // Return the raw message text.
          "chatbotId": string
          "chatbotName": string
          "metadata": object
          "triggerPoint": string
          "createdAt": string (timestamp)
          "updatedAt": string (timestamp)
        }
      ]
      "outputHookLogs": [
        {
          "id": string (uuid)
          "hook": string (uuid)
          "hookName": string
          "hookTypeLabel": string
          "hookAction": string
          "conversation": string (uuid)
          "message": string (uuid)
          "messageContent": string // Return the raw message text.
          "chatbotId": string
          "chatbotName": string
          "metadata": object
          "triggerPoint": string
          "createdAt": string (timestamp)
          "updatedAt": string (timestamp)
        }
      ]
      "instructionId": string
      "llm":
      {
        "id": string (uuid)
        "name": string
      }
      "effectiveMaxLlmOutputTokens": integer // max_tokens actually used for this reply, from
Chatbot.custom_max_llm_output_tokens or LLM.max_tokens.
      "chatbot":
      {
        "id": string (uuid)
        "name": string
      }
      "context": string
      "conversationId": string (uuid)
```

```
      "inboxId": string (uuid)
      "botMessageId": string (uuid)
      "userMessageId": string (uuid)
      "creditCosts": [ // Return credit costs aggregated by item_code for credit-mode
organizations.
        object
      ]
      "cacheSavedCredits": number (double) // Net credits saved by prompt caching for this
reply; ``None`` outside credit mode.

Reuses the usage-statistics saving semantics (``compute_cache_saved_credits``)
on the reply's own billing batch, so the per-reply figure and the hourly
aggregate are defined by the same formula. The net value may be negative
(cache-creation premium exceeding read savings); the frontend hides
non-positive values by design.
      "trace": object // Return the round trace timeline for the detail view; ``None`` for
legacy records.
      "llmRawRequest": object // Raw messages array sent to the LLM on the final round,
including system prompt and memory
      "source": string // Origin discriminator for the unified /records/ response.
      "schedule": object // Shape parity with scheduled rows, which link their owning
ChatbotSchedule here.
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "senderName": "Response string",
      "callerMemberId": "Response string",
      "callerMemberName": "Response string",
      "feedback": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "type": {},
        "suggestion": "Response string",
        "updatedAt": "Response string"
      },
      "inputMessage": "Response string",
      "condenseMessage": "Response string",
      "outputMessage": "Response string",
      "faithfulnessScore": 456,
      "displayFaithfulnessScore": 456,
      "answerRelevancyScore": 456,
      "displayAnswerRelevancyScore": 456,
      "contextPrecisionScore": 456,
      "displayContextPrecisionScore": 456,
      "answerCorrectnessScore": 456,
      "displayAnswerCorrectnessScore": 456,
```

```
    "answerSimilarityScore": 456,
    "displayAnswerSimilarityScore": 456,
    "contextRecallScore": 456,
    "displayContextRecallScore": 456,
    "replyTime": "Response string",
    "createdAt": "Response string",
    "error": "Response string",
    "errorType": "system",
    "processingTime": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    },
    "usage": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    },
    "citationNodes": [
      {
        "id": "Response string",
        "text": "Response string",
        "chatbotFileId": "Response string",
        "fileName": "Response string",
        "url": "Response string"
      }
    ],
    "inputHookLogs": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "hook": "550e8400-e29b-41d4-a716-446655440000",
        "hookName": "Response string",
        "hookTypeLabel": "Response string",
        "hookAction": "Response string",
        "conversation": "550e8400-e29b-41d4-a716-446655440000",
        "message": "550e8400-e29b-41d4-a716-446655440000",
        "messageContent": "Response string",
        "chatbotId": "Response string",
        "chatbotName": "Response string",
        "metadata": null,
        "triggerPoint": "Response string",
        "createdAt": "Response string",
        "updatedAt": "Response string"
      }
    ],
    "outputHookLogs": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "hook": "550e8400-e29b-41d4-a716-446655440000",
        "hookName": "Response string",
        "hookTypeLabel": "Response string",
        "hookAction": "Response string",
        "conversation": "550e8400-e29b-41d4-a716-446655440000",
        "message": "550e8400-e29b-41d4-a716-446655440000",
        "messageContent": "Response string",
        "chatbotId": "Response string",
```

```
          "chatbotName": "Response string",
          "metadata": null,
          "triggerPoint": "Response string",
          "createdAt": "Response string",
          "updatedAt": "Response string"
        }
      ],
      "instructionId": "Response string",
      "llm": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "effectiveMaxLlmOutputTokens": 456,
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "context": "Response string",
      "conversationId": "550e8400-e29b-41d4-a716-446655440000",
      "inboxId": "550e8400-e29b-41d4-a716-446655440000",
      "botMessageId": "550e8400-e29b-41d4-a716-446655440000",
      "userMessageId": "550e8400-e29b-41d4-a716-446655440000",
      "creditCosts": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response name",
          "description": "Example response description"
        }
      ],
      "cacheSavedCredits": 456,
      "trace": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "llmRawRequest": null,
      "source": "Response string",
      "schedule": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    }
  ]
}
```


Retrieve a Specific Conversation Record

GET `/api/v1/records/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Chatbot Record.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "senderName": string // Returns the name of the user who sent the message
  "callerMemberId": string
  "callerMemberName": string
  "feedback":
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // Optional
    "updatedAt": string (timestamp)
  }
  "inputMessage": string
```

```
  "condenseMessage": string // User message rewritten from the chat history and context.
  "outputMessage": string
  "faithfulnessScore": number (double) // Whether the chatbot reply is grounded in the
knowledge base rather than made up.
  "displayFaithfulnessScore": integer
  "answerRelevancyScore": number (double) // Whether the chatbot reply is relevant to the
user's question.
  "displayAnswerRelevancyScore": integer
  "contextPrecisionScore": number (double) // Whether the chatbot reply is relevant to the
reference context.
  "displayContextPrecisionScore": integer
  "answerCorrectnessScore": number (double) // Whether the chatbot reply is correct (requires a
ground-truth answer).
  "displayAnswerCorrectnessScore": integer
  "answerSimilarityScore": number (double) // Whether the chatbot reply is similar to the
ground-truth answer.
  "displayAnswerSimilarityScore": integer
  "contextRecallScore": number (double) // Whether the chatbot reply recalls the relevant
information from the reference context.
  "displayContextRecallScore": integer
  "replyTime": string
  "createdAt": string (timestamp)
  "error": string // Error message raised during execution.
  "errorType": // May have different types
  string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Error source: system, LLM, embedding,
reranker, and so on.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other
  "processingTime": object // Returns timing statistics for each processing stage, including
the streaming_metrics waterfall
  "usage": object // Returns usage statistics, including character counts, token counts, and
token details for each LLM call
  "citationNodes": [
    {
      "id": string
      "text": string
      "chatbotFileId": string
      "fileName": string
      "url": string
    }
  ]
  "inputHookLogs": [
    {
      "id": string (uuid)
```

```
      "hook": string (uuid)
      "hookName": string
      "hookTypeLabel": string
      "hookAction": string
      "conversation": string (uuid)
      "message": string (uuid)
      "messageContent": string // Return the raw message text.
      "chatbotId": string
      "chatbotName": string
      "metadata": object
      "triggerPoint": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "outputHookLogs": [
    {
      "id": string (uuid)
      "hook": string (uuid)
      "hookName": string
      "hookTypeLabel": string
      "hookAction": string
      "conversation": string (uuid)
      "message": string (uuid)
      "messageContent": string // Return the raw message text.
      "chatbotId": string
      "chatbotName": string
      "metadata": object
      "triggerPoint": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "instructionId": string
  "llm":
  {
    "id": string (uuid)
    "name": string
  }
  "effectiveMaxLlmOutputTokens": integer // max_tokens actually used for this reply, from
Chatbot.custom_max_llm_output_tokens or LLM.max_tokens.
  "chatbot":
  {
    "id": string (uuid)
    "name": string
  }
  "context": string
  "conversationId": string (uuid)
  "inboxId": string (uuid)
  "botMessageId": string (uuid)
  "userMessageId": string (uuid)
  "creditCosts": [ // Return credit costs aggregated by item_code for credit-mode
organizations.
    object
  ]
  "cacheSavedCredits": number (double) // Net credits saved by prompt caching for this reply;
```

```
``None`` outside credit mode.

Reuses the usage-statistics saving semantics (``compute_cache_saved_credits``)
on the reply's own billing batch, so the per-reply figure and the hourly
aggregate are defined by the same formula. The net value may be negative
(cache-creation premium exceeding read savings); the frontend hides
non-positive values by design.
  "trace": object // Return the round trace timeline for the detail view; ``None`` for legacy
records.
  "llmRawRequest": object // Raw messages array sent to the LLM on the final round, including
system prompt and memory
  "source": string // Origin discriminator for the unified /records/ response.
  "schedule": object // Shape parity with scheduled rows, which link their owning
ChatbotSchedule here.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "senderName": "Response string",
  "callerMemberId": "Response string",
  "callerMemberName": "Response string",
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "Response string",
    "updatedAt": "Response string"
  },
  "inputMessage": "Response string",
  "condenseMessage": "Response string",
  "outputMessage": "Response string",
  "faithfulnessScore": 456,
  "displayFaithfulnessScore": 456,
  "answerRelevancyScore": 456,
  "displayAnswerRelevancyScore": 456,
  "contextPrecisionScore": 456,
  "displayContextPrecisionScore": 456,
  "answerCorrectnessScore": 456,
  "displayAnswerCorrectnessScore": 456,
  "answerSimilarityScore": 456,
  "displayAnswerSimilarityScore": 456,
  "contextRecallScore": 456,
  "displayContextRecallScore": 456,
  "replyTime": "Response string",
  "createdAt": "Response string",
  "error": "Response string",
  "errorType": "system",
  "processingTime": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "usage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Example response name",
    "description": "Example response description"
  },
  "citationNodes": [
    {
      "id": "Response string",
      "text": "Response string",
      "chatbotFileId": "Response string",
      "fileName": "Response string",
      "url": "Response string"
    }
  ],
  "inputHookLogs": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "hook": "550e8400-e29b-41d4-a716-446655440000",
      "hookName": "Response string",
      "hookTypeLabel": "Response string",
      "hookAction": "Response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000",
      "message": "550e8400-e29b-41d4-a716-446655440000",
      "messageContent": "Response string",
      "chatbotId": "Response string",
      "chatbotName": "Response string",
      "metadata": null,
      "triggerPoint": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "outputHookLogs": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "hook": "550e8400-e29b-41d4-a716-446655440000",
      "hookName": "Response string",
      "hookTypeLabel": "Response string",
      "hookAction": "Response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000",
      "message": "550e8400-e29b-41d4-a716-446655440000",
      "messageContent": "Response string",
      "chatbotId": "Response string",
      "chatbotName": "Response string",
      "metadata": null,
      "triggerPoint": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "instructionId": "Response string",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "effectiveMaxLlmOutputTokens": 456,
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Response string"
  },
  "context": "Response string",
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",
  "botMessageId": "550e8400-e29b-41d4-a716-446655440000",
  "userMessageId": "550e8400-e29b-41d4-a716-446655440000",
  "creditCosts": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "cacheSavedCredits": 456,
  "trace": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llmRawRequest": null,
  "source": "Response string",
  "schedule": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```


Status Code: 404 - The specified conversation record was not found

Create a Conversation Record Export Request

POST `/api/v1/records/export-excel-requests/`

Parameters

Parameter Name	Required	Type	Description
`endDate`	X	string	End date (format: YYYY-MM-DD, e.g., 2025-01-31)
`keyword`	X	string	Filter conversation records containing specific keywords (searches both user messages and bot replies)
`startDate`	X	string	Start date (format: YYYY-MM-DD, e.g., 2025-01-01)

Request Body

Request Parameters

Field	Type	Required	Description
chatbot	string	No	Specify the Chatbot UUID to export. Supports multiple comma-separated UUIDs. Leave blank to export all Chatbots in the organization that you have permission to access.
chatbotId	string (uuid)	No	Specify the Chatbot UUID to export (an alias for chatbot; supports only a single UUID).
largeLanguageModel	string	No	Specify the large language model UUID to export. Supports multiple comma-separated UUIDs.
startDatetime	string (timestamp)	No	Exact start time (ISO 8601, inclusive), with the same semantics as the list drill-down filters.
endDatetime	string (timestamp)	No	Exact end time (ISO 8601, exclusive), with the same semantics as the list drill-down filters.
hasError	boolean	No	Filter records by whether they contain errors (true includes only records with errors; false includes only records without errors).
errorType	string	No	Filter by error type. Supports multiple comma-separated values (OR logic), with the same allowed values as the conversation record list.
hasFeedback	boolean	No	Filter by whether records have user feedback (true includes only records with likes/dislikes; false includes only records without feedback).
feedbackType	string	No	Filter by user feedback type (like / dislike). Supports multiple comma-separated values (OR logic).
callerMember	string	No	Filter by the UUID of the speaking member. Supports multiple comma-separated values (OR logic).

Example Request Structure

```
{
  "chatbot"?: string // Specify the Chatbot UUID to export. Supports multiple comma-separated UUIDs. Leave blank to export all Chatbots in the organization that you have permission to access. (Optional)
  "chatbotId"?: string (uuid) // Specify the Chatbot UUID to export (an alias for chatbot; supports only a single UUID). (Optional)
  "largeLanguageModel"?: string // Specify the large language model UUID to export. Supports multiple comma-separated UUIDs. (Optional)
  "startDatetime"?: string (timestamp) // Exact start time (ISO 8601, inclusive), with the same semantics as the list drill-down filters. (Optional)
  "endDatetime"?: string (timestamp) // Exact end time (ISO 8601, exclusive), with the same semantics as the list drill-down filters. (Optional)
  "hasError"?: boolean // Filter records by whether they contain errors (true includes only records with errors; false includes only records without errors). (Optional)
```

```
  "errorType"?: string // Filter by error type. Supports multiple comma-separated values (OR logic), with the same allowed values as the conversation record list. (Optional)
  "hasFeedback"?: boolean // Filter by whether records have user feedback (true includes only records with likes/dislikes; false includes only records without feedback). (Optional)
  "feedbackType"?: string // Filter by user feedback type (like / dislike). Supports multiple comma-separated values (OR logic). (Optional)
  "callerMember"?: string // Filter by the UUID of the speaking member. Supports multiple comma-separated values (OR logic). (Optional)
}
```

Example Request Values

```
{
  "chatbot": "Example string",
  "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": "Example string",
  "startDatetime": "Example string",
  "endDatetime": "Example string",
  "hasError": true,
  "errorType": "Example string",
  "hasFeedback": true,
  "feedbackType": "Example string",
  "callerMember": "Example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "chatbot": "Example string",
    "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
    "keyword": "Example string",
    "largeLanguageModel": "Example string",
    "startDate": "Example string",
    "endDate": "Example string",
    "startDatetime": "Example string",
    "endDatetime": "Example string",
    "hasError": true,
    "errorType": "Example string",
    "hasFeedback": true,
    "feedbackType": "Example string",
    "callerMember": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "chatbot": "Example string",
    "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
    "keyword": "Example string",
    "largeLanguageModel": "Example string",
    "startDate": "Example string",
    "endDate": "Example string",
    "startDatetime": "Example string",
    "endDatetime": "Example string",
    "hasError": true,
    "errorType": "Example string",
    "hasFeedback": true,
    "feedbackType": "Example string",
    "callerMember": "Example string"
  };

axios.post("https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "chatbot": "Example string",
    "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
    "keyword": "Example string",
    "largeLanguageModel": "Example string",
    "startDate": "Example string",
    "endDate": "Example string",
    "startDatetime": "Example string",
    "endDatetime": "Example string",
    "hasError": True,
    "errorType": "Example string",
    "hasFeedback": True,
    "feedbackType": "Example string",
    "callerMember": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/records/export-excel-requests/?endDate=example&keyword=example&startDate=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'chatbot' => 'Example string',
            'chatbotId' => '550e8400-e29b-41d4-a716-446655440000',
            'keyword' => 'Example string',
            'largeLanguageModel' => 'Example string',
            'startDate' => 'Example string',
            'endDate' => 'Example string',
            'startDatetime' => 'Example string',
            'endDatetime' => 'Example string',
            'hasError' => true,
            'errorType' => 'Example string',
            'hasFeedback' => true,
            'feedbackType' => 'Example string',
            'callerMember' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	Excel file download

Get Conversation Record Export Status

GET `/api/v1/records/export-excel-requests/{exportId}`

Parameters

Parameter Name	Required	Type	Description
`exportId`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}
"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}
", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}
"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/export-excel-
requests/{exportId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "senderName": string // Returns the name of the user message sender
  "callerMemberId": string
  "callerMemberName": string
  "inputMessage": string
  "outputMessage": string
  "chatbot":
  {
    "id": string (uuid)
    "name": string
  }
  "condenseMessage": string // User message rewritten from the chat history and context.
  "feedback":
```

```
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // Optional
    "updatedAt": string (timestamp)
  }
  "displayFaithfulnessScore": integer
  "displayAnswerRelevancyScore": integer
  "displayContextPrecisionScore": integer
  "displayAnswerCorrectnessScore": integer
  "displayAnswerSimilarityScore": integer
  "displayContextRecallScore": integer
  "replyTime": string
  "processingTime": object // Returns processing time statistics for the list page, using the
same formulas as the detail page but excluding the streaming_metrics waterfall
  "usage": object // Returns usage statistics, including character counts, token counts, and
token details for each LLM call
  "llm":
  {
    "id": string (uuid)
    "name": string
  }
  "createdAt": string (timestamp)
  "conversationId": string (uuid)
  "errorType": // May have different types
  string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Error source: system, LLM, embedding,
reranker, and so on.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other
  "creditCosts": [
    object
  ]
  "source": string // Origin discriminator for the unified /records/ response.
  "schedule": object // Shape parity with scheduled rows, which link their owning
ChatbotSchedule here.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
  "senderName": "Response string",
  "callerMemberId": "Response string",
  "callerMemberName": "Response string",
  "inputMessage": "Response string",
  "outputMessage": "Response string",
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "condenseMessage": "Response string",
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "Response string",
    "updatedAt": "Response string"
  },
  "displayFaithfulnessScore": 456,
  "displayAnswerRelevancyScore": 456,
  "displayContextPrecisionScore": 456,
  "displayAnswerCorrectnessScore": 456,
  "displayAnswerSimilarityScore": 456,
  "displayContextRecallScore": 456,
  "replyTime": "Response string",
  "processingTime": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "usage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdAt": "Response string",
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "errorType": "system",
  "creditCosts": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "source": "Response string",
  "schedule": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```

Example API call (Shell)

目錄

1. List inboxes
2. Create an inbox
3. Retrieve a Specific Conversation Platform
4. Update a Conversation Platform
5. Partially Update a Conversation Platform
6. Delete a Conversation Platform
7. List Webhooks for a Conversation Platform
8. Create a Webhook
9. Retrieve a Specific Webhook
10. Update a Webhook
11. Partially Update a Webhook
12. Delete Webhook

Example API call (Shell)

An inbox is the unit to which conversations belong and where Webhooks are configured. Each inbox can have multiple Webhooks that send notifications to your system when a user message arrives (`incoming`) or an Agent replies (`outgoing`). Web Chat inboxes also have Web Chat endpoints. For configuration instructions and payload formats, see Webhook in the technical manual.

List inboxes

`GET` `/api/v1/inboxes/`

Parameters

Parameter name	Required	Type	Description
`accessType`	X	string	Filter by access type
`channelType`	X	string	Filter by channel type
`chatbot`	X	string	Filter by chatbot ID
`isActive`	X	boolean	Filter by enabled status
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`pagination`	X	string	Whether to paginate (true/false). When set to false, returns a simplified data format.
`query`	X	string	
`source`	X	string	`self_built`: self_built ; `built_in`: built_in ; `all`: all;

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/inboxes/?
accessType=example&channelType=example&chatbot=example&isActive=true&ordering=example&page=1&pageSize=
1&pagination=example&query=example&source=all"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/inboxes/?
accessType=example&channelType=example&chatbot=example&isActive=true&ordering=example&page=1&pageSize=
1&pagination=example&query=example&source=all", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/?
accessType=example&channelType=example&chatbot=example&isActive=true&ordering=example&page=1&pageSize=
1&pagination=example&query=example&source=all"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/inboxes/?
accessType=example&channelType=example&chatbot=example&isActive=true&ordering=example&page=1&pageSize=
1&pagination=example&query=example&source=all", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: `200` - Inbox list

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack,
team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
      "unreadConversationsCount"?: integer // Optional
      "signAuth"?:  // Optional
      {
        "id": string (uuid)
        "signSource": string (uuid)
        "signParams": {
        {
          "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
          "line":
          {
            "liffId": string
          }
          "ad":
          {
            "saml": string
          }
          "google"?:  // Optional
          {
            "enabled": boolean
          }
        }
        }
        "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID
(verified per the verify mode below) may chat without going through the login flow. Disabled keeps every
existing behavior unchanged. (Optional)
        "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC
signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact;
suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
        {
```

```
        }
        "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
        {
        }
        "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
        "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
          string
        ]
      }
      "enableAnalysis"?: boolean // Optional
    }
  ]
}
```

Example response values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "961dcaac-bd2e-44d3-9050-82596dbb3334",
      "chatbot": {
        "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b",
        "name": "Customer service bot",
        "groups": [],
        "largeLanguageModel": {
          "id": "933aa8b3-cdf4-47a7-86d5-b5dc27fbb044",
          "name": "llm"
        },
        "embeddingModel": {
          "id": "aefcc4f9-917d-4a8a-bd36-d97e3b7cecfe",
          "name": "embedding",
          "icon": null,
          "isDefault": false
        },
        "rerankerModel": {
          "id": "86c2ddfd-0df9-47b9-8f3b-0c42e3347da3",
          "name": "rerank",
          "icon": null,
          "isDefault": true
        },
        "knowledgeBases": [],
        "tools": [],
        "agentMode": "normal",
        "replyMode": "normal",
        "updatedAt": "1739519493000",
        "enableEvaluation": false,
        "enableInlineCitations": false,
        "enableKnowledgeBaseRag": true,
```

```
      "enableAgenticTextToSql": true,
      "enableToolSearching": true,
      "numberOfRetrievedChunks": 12
    },
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Administrator group"
      }
    ],
    "channelType": "web",
    "channelTypeLabel": "Web",
    "name": "1345",
    "autoReplySchedules": [
      {
        "id": "2f4f6383-d8b5-4d88-b4a1-6d02d731f2a6",
        "weekday": 0,
        "isEnabled": true,
        "timeSlots": [
          [
            "00:00:00",
            "00:00:00"
          ]
        ],
        "weekdayName": "Monday"
      },
      {
        "id": "2866f624-a5c3-4ffb-b980-0f8d6a9f0c93",
        "weekday": 1,
        "isEnabled": true,
        "timeSlots": [
          [
            "00:00:00",
            "00:00:00"
          ]
        ],
        "weekdayName": "Tuesday"
      },
      {
        "id": "3cf1e488-fb13-426d-bd59-e62bd71aca85",
        "weekday": 2,
        "isEnabled": true,
        "timeSlots": [
          [
            "00:00:00",
            "00:00:00"
          ]
        ],
        "weekdayName": "Wednesday"
      },
      {
        "id": "491776a9-fcf1-4e2e-892f-6d9ad5aebea4",
        "weekday": 3,
        "isEnabled": true,
        "timeSlots": [
          [
            "00:00:00",
            "00:00:00"
          ]
        ],
        "weekdayName": "Thursday"
      },
      {
```

```
      "id": "dfecb0a9-5001-4aa0-b2cc-5459b6d64351",
      "weekday": 4,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Friday"
    },
    {
      "id": "68b6b4d1-7ac8-450f-8877-41b72b499919",
      "weekday": 5,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Saturday"
    },
    {
      "id": "15228ea1-4513-4504-81f4-4e486703d8b9",
      "weekday": 6,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Sunday"
    }
  ],
  "unreadConversationsCount": 0,
  "totalConversationsCount": 42,
  "channel": {
    "id": "801a690a-1038-4fdd-8aba-bf003b12c3ef",
    "name": "1345",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-chatbot-avatar.png",
    "logo": null,
    "description": "",
    "cover": null,
    "backUrl": "",
    "isActive": true,
    "enableSpeech": false,
    "displayGreeting": null,
    "displayConversationStarters": [],
    "theme": {
      "primaryColor": "#3854d8",
      "navbarTextColor": "#ffffff",
      "conversationBackgroundColor": "#ffffff",
      "chatbotMessageTextColor": "#374151",
      "userMessageTextColor": "#374151",
      "chatbotMessageBackgroundColor": "#DCE9F6",
      "userMessageBackgroundColor": "#DDEAF6",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true
    },
    "darkTheme": {
      "primaryColor": "#404040",
```

```
          "navbarTextColor": "#ffffff",
          "conversationBackgroundColor": "#18181B",
          "chatbotMessageTextColor": "#ffffff",
          "userMessageTextColor": "#ffffff",
          "chatbotMessageBackgroundColor": "#171717",
          "userMessageBackgroundColor": "#333842",
          "chatbotMessageBackgroundShadowEnabled": false,
          "userMessageBackgroundShadowEnabled": false
        },
        "enableFileUpload": true,
        "enableDisplayCitations": true,
        "enableShareConversation": false,
        "enableLocation": false,
        "enableShowPoweredBy": true,
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": false,
        "conversationTimerDuration": null,
        "enableAutoNewConversationOnTimeout": false,
        "accessType": "api",
        "greetings": [],
        "greetingsByLocale": {
          "en": [
            "Hello, how can I help you?"
          ],
          "zh-hant": [
            "你好，我可以幫你什麼嗎？"
          ],
          "ja": [
            "こんにちは、何かお手伝いできますか？"
          ],
          "ko": [
            "안녕하세요, 무엇을 도와드릴까요?"
          ]
        },
        "conversationStarters": [],
        "conversationStartersByLocale": {
          "en": [
            "What can I help you with today?"
          ],
          "zh-hant": [
            "你今天需要什麼幫助？"
          ],
          "ja": [
            "今日は何かお手伝いできますか？"
          ],
          "ko": [
            "오늘 뭘 도와줄수있어？"
          ]
        }
      },
      "webhooks": [],
      "createdAt": "1739519493000",
      "updatedAt": "1739519493000",
      "organization": "24b619d9-e981-4a8b-96da-63dc4f96bda6",
      "signAuth": null
    }
  ]
}
```

Create an inbox

`POST` `/api/v1/inboxes/`

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	
channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)	Yes	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` : Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
unreadConversationsCount	integer	No	
signAuth	object (with 8 properties: id, signSource, signParams...)	No	
signAuth.signSource	string (uuid)	Yes	
signAuth.signParams	object	Yes	
signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
signAuth.signParams.keycloak.clientId	string	Yes	
signAuth.signParams.keycloak.url	string	Yes	
signAuth.signParams.keycloak.realm	string	Yes	
signAuth.signParams.line	object (with 1 property: liffId)	Yes	
signAuth.signParams.line.liffId	string	Yes	
signAuth.signParams.ad	object (with 1 property: saml)	Yes	
signAuth.signParams.ad.saml	string	Yes	
signAuth.signParams.google	object (with 1 property: enabled)	No	
signAuth.signParams.google.enabled	boolean	Yes	
signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login

Field	Type	Required	Description
			flow. Disabled keeps every existing behavior unchanged.
signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-to...
signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
enableAnalysis	boolean	No	

Example request schema

```
{
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack,
team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?:  // Optional
  {
    "signSource": string (uuid)
    "signParams": {
    {
      "keycloak":
      {
        "clientId": string
        "url": string
        "realm": string
      }
      "line":
      {
```

```
        "liffId": string
      }
      "ad":
      {
        "saml": string
      }
      "google"?:  // Optional
      {
        "enabled": boolean
      }
    }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified
per the verify mode below) may chat without going through the login flow. Disabled keeps every existing
behavior unchanged. (Optional)
    "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC
signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact;
suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }
    "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not
a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with
is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do
not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization
header a tool already resolves from its own credentials. The visitor identity is the organization member
bound to the conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of the tools the
visitor Keycloak token may be sent to (Optional)
      string
    ]
  }
  "enableAnalysis"?: boolean // Optional
}
```

Example request values

```
{
  "chatbot": {
    "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
  },
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "name": "Updated name"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/inboxes/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "chatbot": {
      "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "name": "Updated name"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "chatbot": {
      "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "name": "Updated name"
  };

axios.post("https://api.maiagent.ai/api/v1/inboxes/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "chatbot": {
        "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "name": "Updated name"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/inboxes/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'chatbot' => [
                'id' => '80f3170d-eed7-4651-a8da-6b7e6bdbef8b'
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'name' => 'Updated name'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Example response schema

```
{
  "id": string (uuid)
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack,
team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
```

```
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?:  // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
    {
      "keycloak":
      {
        "clientId": string
        "url": string
        "realm": string
      }
      "line":
      {
        "liffId": string
      }
      "ad":
      {
        "saml": string
      }
      "google"?:  // Optional
      {
        "enabled": boolean
      }
    }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
    "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }
    "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
      string
    ]
  }
```

```
  "enableAnalysis"?: boolean // Optional
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "channelType": "line",
  "unreadConversationsCount": 456,
  "signAuth": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "signSource": "550e8400-e29b-41d4-a716-446655440000",
    "signParams": {
      "keycloak": {
        "clientId": "Response string",
        "url": "Response string",
        "realm": "Response string"
      },
      "line": {
        "liffId": "Response string"
      },
      "ad": {
        "saml": "Response string"
      },
      "google": {
        "enabled": false
      }
    },
    "sourceIdAccessEnabled": false,
    "sourceIdVerifyMode": {},
    "sourceIdSignatureTtlMinutes": {},
    "keycloakTokenPassthrough": false,
    "keycloakTokenPassthroughAllowedHosts": [
      "Response string"
    ]
  },
  "enableAnalysis": false
}
```


Retrieve a Specific Conversation Platform

GET `/api/v1/inboxes/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Inbox.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: `200`

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack,
team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?:  // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
    {
      "keycloak":
      {
        "clientId": string
        "url": string
        "realm": string
      }
      "line":
      {
        "liffId": string
      }
      "ad":
      {
        "saml": string
      }
      "google"?:  // Optional
      {
        "enabled": boolean
      }
    }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified
per the verify mode below) may chat without going through the login flow. Disabled keeps every existing
behavior unchanged. (Optional)
    "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC
signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact;
suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }
    "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not
a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
```

```
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
      string
    ]
  }
  "enableAnalysis"?: boolean // Optional
}
```

Example Response Values


```
{
  "id": "961dcaac-bd2e-44d3-9050-82596dbb3334",
  "chatbot": {
    "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b",
    "name": "Customer Support Bot",
    "groups": [],
    "largeLanguageModel": {
      "id": "933aa8b3-cdf4-47a7-86d5-b5dc27fbb044",
      "name": "llm"
    },
    "embeddingModel": {
      "id": "aefcc4f9-917d-4a8a-bd36-d97e3b7cecfe",
      "name": "embedding",
      "icon": null,
      "isDefault": false
    },
    "rerankerModel": {
      "id": "86c2ddfd-0df9-47b9-8f3b-0c42e3347da3",
      "name": "rerank",
      "icon": null,
      "isDefault": true
    },
    "knowledgeBases": [],
    "tools": [],
    "agentMode": "normal",
    "replyMode": "normal",
    "updatedAt": "1739519493000",
    "enableEvaluation": false,
    "enableInlineCitations": false,
    "enableKnowledgeBaseRag": true,
    "enableAgenticTextToSql": true,
    "enableToolSearching": true,
    "numberOfRetrievedChunks": 12
  },
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Administrators Group"
    }
  ],
  "channelType": "web",
  "channelTypeLabel": "Web",
  "name": "1345",
  "autoReplySchedules": [
```

```
    {
      "id": "2f4f6383-d8b5-4d88-b4a1-6d02d731f2a6",
      "weekday": 0,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Monday"
    },
    {
      "id": "2866f624-a5c3-4ffb-b980-0f8d6a9f0c93",
      "weekday": 1,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Tuesday"
    },
    {
      "id": "3cf1e488-fb13-426d-bd59-e62bd71aca85",
      "weekday": 2,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Wednesday"
    },
    {
      "id": "491776a9-fcf1-4e2e-892f-6d9ad5aebea4",
      "weekday": 3,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Thursday"
    },
    {
      "id": "dfecb0a9-5001-4aa0-b2cc-5459b6d64351",
      "weekday": 4,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Friday"
    },
    {
      "id": "68b6b4d1-7ac8-450f-8877-41b72b499919",
      "weekday": 5,
```

```
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Saturday"
    },
    {
      "id": "15228ea1-4513-4504-81f4-4e486703d8b9",
      "weekday": 6,
      "isEnabled": true,
      "timeSlots": [
        [
          "00:00:00",
          "00:00:00"
        ]
      ],
      "weekdayName": "Sunday"
    }
  ],
  "unreadConversationsCount": 0,
  "totalConversationsCount": 42,
  "channel": {
    "id": "801a690a-1038-4fdd-8aba-bf003b12c3ef",
    "name": "1345",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-chatbot-avatar.png",
    "logo": null,
    "description": "",
    "cover": null,
    "backUrl": "",
    "isActive": true,
    "enableSpeech": false,
    "displayGreeting": null,
    "displayConversationStarters": [],
    "theme": {
      "primaryColor": "#3854d8",
      "navbarTextColor": "#ffffff",
      "conversationBackgroundColor": "#ffffff",
      "chatbotMessageTextColor": "#374151",
      "userMessageTextColor": "#374151",
      "chatbotMessageBackgroundColor": "#DCE9F6",
      "userMessageBackgroundColor": "#DDEAF6",
      "chatbotMessageBackgroundShadowEnabled": true,
      "userMessageBackgroundShadowEnabled": true
    },
    "darkTheme": {
      "primaryColor": "#404040",
      "navbarTextColor": "#ffffff",
      "conversationBackgroundColor": "#18181B",
      "chatbotMessageTextColor": "#ffffff",
      "userMessageTextColor": "#ffffff",
      "chatbotMessageBackgroundColor": "#171717",
      "userMessageBackgroundColor": "#333842",
      "chatbotMessageBackgroundShadowEnabled": false,
      "userMessageBackgroundShadowEnabled": false
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "enableShareConversation": false,
    "enableLocation": false,
    "enableShowPoweredBy": true,
```

```
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": false,
    "conversationTimerDuration": null,
    "enableAutoNewConversationOnTimeout": false,
    "accessType": "api",
    "greetings": [],
    "greetingsByLocale": {
      "en": [
        "Hello, how can I help you?"
      ],
      "zh-hant": [
        "Hello, how can I help you?"
      ],
      "ja": [
        "こんにちは、何かお手伝いできますか？"
      ],
      "ko": [
        "안녕하세요, 무엇을 도와드릴까요?"
      ]
    },
    "conversationStarters": [],
    "conversationStartersByLocale": {
      "en": [
        "What can I help you with today?"
      ],
      "zh-hant": [
        "What can I help you with today?"
      ],
      "ja": [
        "今日は何かお手伝いできますか？"
      ],
      "ko": [
        "오늘 뭘 도와줄수있어？"
      ]
    }
  },
  "webhooks": [],
  "createdAt": "1739519493000",
  "updatedAt": "1739519493000",
  "organization": "24b619d9-e981-4a8b-96da-63dc4f96bda6",
  "signAuth": null
}
```


Update a Conversation Platform

PUT `/api/v1/inboxes/{id}`

Parameters

Parameter Name	Required	Type	Description

`id`	V	string	A UUID string identifying this Inbox.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)	Yes	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` : Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
unreadConversationsCount	integer	No	
signAuth	object (containing 8 properties: id, signSource, signParams...)	No	
signAuth.signSource	string (uuid)	Yes	
signAuth.signParams	object	Yes	
signAuth.signParams.keycloak	object (containing 3 properties: clientId, url, realm)	Yes	
signAuth.signParams.keycloak.clientId	string	Yes	
signAuth.signParams.keycloak.url	string	Yes	
signAuth.signParams.keycloak.realm	string	Yes	
signAuth.signParams.line	object (containing 1 property: liffId)	Yes	
signAuth.signParams.line.liffId	string	Yes	
signAuth.signParams.ad	object (containing 1 property: saml)	Yes	
signAuth.signParams.ad.saml	string	Yes	
signAuth.signParams.google	object (containing 1 property: enabled)	No	
signAuth.signParams.google.enabled	boolean	Yes	
signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend.

Field	Type	Required	Description
			"Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-to...
signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
enableAnalysis	boolean	No	

Example Request Schema

```
{
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack,
team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?:  // Optional
  {
    "signSource": string (uuid)
    "signParams": {
    {
      "keycloak":
      {
        "clientId": string
        "url": string
        "realm": string
      }
      "line":
      {
        "liffId": string
      }
      "ad":
      {
        "saml": string
      }
```

```
      "google"?:  // Optional
      {
        "enabled": boolean
      }
    }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
    "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }
    "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
      string
    ]
  }
  "enableAnalysis"?: boolean // Optional
}
```

Example Request Values

```
{
  "chatbot": {
    "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
  },
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "name": "Updated name"
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "chatbot": {
      "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "name": "Updated name"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "chatbot": {
      "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "name": "Updated name"
  };

axios.put("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "chatbot": {
        "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "name": "Updated name"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'chatbot' => [
                'id' => '80f3170d-eed7-4651-a8da-6b7e6bdbef8b'
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'name' => 'Updated name'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack,
team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
```

```
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?:  // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
    {
      "keycloak":
      {
        "clientId": string
        "url": string
        "realm": string
      }
      "line":
      {
        "liffId": string
      }
      "ad":
      {
        "saml": string
      }
      "google"?:  // Optional
      {
        "enabled": boolean
      }
    }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
    "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }
    "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
      string
    ]
  }
```

```
  "enableAnalysis"?: boolean // Optional
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "channelType": "line",
  "unreadConversationsCount": 456,
  "signAuth": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "signSource": "550e8400-e29b-41d4-a716-446655440000",
    "signParams": {
      "keycloak": {
        "clientId": "Response string",
        "url": "Response string",
        "realm": "Response string"
      },
      "line": {
        "liffId": "Response string"
      },
      "ad": {
        "saml": "Response string"
      },
      "google": {
        "enabled": false
      }
    },
    "sourceIdAccessEnabled": false,
    "sourceIdVerifyMode": {},
    "sourceIdSignatureTtlMinutes": {},
    "keycloakTokenPassthrough": false,
    "keycloakTokenPassthroughAllowedHosts": [
      "Response string"
    ]
  },
  "enableAnalysis": false
}
```


Partially Update a Conversation Platform

`PATCH` `/api/v1/inboxes/{id}`

Parameters

Parameter	Required	Type	Description
`id`	V	string	A UUID string identifying this Inbox.

Request Body

Request Parameters

Field	Type	Required	Description
groups	array[IdName]	No	List of groups that can access this inbox
channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus, line_works, viber)	No	`line` : LINE ; `telegram` : Telegram ; `teams` : Teams ; `web` : Web ; `messenger` : Messenger ; `instagram` : Instagram ; `email` : Email ; `whatsapp` : WhatsApp ; `slack` : Slack ; `team_plus` : Team+ ; `lin...
name	string	No	
autoReplySchedules	array[AutoReplySchedule]	No	
channel	object	No	
organization	string (uuid)	No	
signAuth	object (containing 9 properties: id, signSource, signParams...)	No	
signAuth.signSource	string (uuid)	Yes	
signAuth.signParams	object	Yes	
signAuth.signParams.keycloak	object (containing 3 properties: clientId, url, realm)	Yes	
signAuth.signParams.keycloak.clientId	string	Yes	
signAuth.signParams.keycloak.url	string	Yes	
signAuth.signParams.keycloak.realm	string	Yes	
signAuth.signParams.line	object (containing 1 property: liffId)	Yes	
signAuth.signParams.line.liffId	string	Yes	
signAuth.signParams.ad	object (containing 1 property: saml)	Yes	
signAuth.signParams.ad.saml	string	Yes	
signAuth.signParams.google	object (containing 1 property: enabled)	No	
signAuth.signParams.google.enabled	boolean	Yes	
signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.

Field	Type	Required	Description
signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. `1` : 1 ; `5` : 5 ; `15` : 15;
signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-to...
signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
enableTimeAwareResponse	boolean	No	Enable time aware responses for this inbox
defaultTimezone	string	No	Default timezone for this inbox (e.g., Asia/Taipei, America/New_York)
enableSenderNameAwareResponse	boolean	No	Enable sender name aware responses for this inbox
enableGpsAwareResponse	boolean	No	Enable GPS aware responses for this inbox
enableOcr	boolean	No	Enable OCR parser, prioritize OCR-enabled parsers when uploading attachments
enableAgentAttachmentDecision	boolean	No	Kill switch for AGENT-mode attachment handling. When False (default), AGENT-mode chatbots skip auto-embedding on upload and let the agent handle attachments at runtime via ParseAttachmentContentTool /...
enableStreamMessage	boolean	No	
enableHumanHandoff	boolean	No	
handoffPrompt	string	No	
handoffLlm	string (uuid)	No	
handoffContextMessageLimit	integer	No	
handoffMessage	string	No	
queuingMessage	string	No	
assignedMessage	string	No	

Field	Type	Required	Description
enablePendingReplyMessage	boolean	No	Send a placeholder message to the user while the AI is processing.
pendingReplyMessage	string	No	
pendingReplyInitialDelaySeconds	integer	No	Delay before sending the first pending reply, so a fast AI response can pre-empt it.
pendingReplyIntervalSeconds	integer	No	Interval to repeat the pending reply while the AI is still processing.
enablePendingReplyCleanup	boolean	No	Delete pending reply messages after the AI reply is sent. Only effective on platforms that support message deletion (Teams, Telegram, Slack).
enableAnalysis	boolean	No	
analysisLlm	string (uuid)	No	
analysisPrompt	string	No	
analysisContextMessageLimit	integer	No	
enableServiceHours	boolean	No	
serviceHoursTimezone	string	No	
offlineBehavior	string (enum: show_message, leave_message)	No	`show_message` : Show Message ; `leave_message` : Leave Message;
offlineMessage	string	No	
serviceHoursSchedules	array[ServiceHoursSchedule]	No	
dispatchMode	object	No	Auto dispatch assigns transferred conversations to an agent automatically. Claim (handover) leaves them unassigned for online agents to pick up themselves; timeout reassignment does not apply and the ...
assignmentRoutingModel	string (enum: round_robin, least_active)	No	`round_robin` : Round Robin ; `least_active` : Least Active;
assignmentMaxConcurrent	integer	No	
assignmentQueueTimeout	integer	No	Queue timeout in seconds
queueTimeoutMessage	string	No	
maxQueueSize	integer	No	
queueFullMessage	string	No	
assignmentTargetMembers	array[string]	No	
enableTagRouting	boolean	No	
enableReassignOnTimeout	boolean	No	When on, a queued or assigned-but-unanswered conversation that times out is reassigned to the next available agent (skipping already-

Field	Type	Required	Description
			tried agents) instead of returning straight to the AI; it only retu...
enableQueueReminder	boolean	No	Reply with a throttled waiting reminder when the contact sends messages while queued.
queueReminderMessage	string	No	
queueReminderCooldown	integer	No	Minimum seconds between queue reminders in one conversation
enableQueueCancel	boolean	No	Allow the contact to cancel a queued handoff and return to the AI by themselves.
queueCancelButtonText	string	No	Label of the cancel quick-reply button. Its text also counts as a cancel keyword, so tapping the button always cancels even when the keyword list changes.
queueCancelKeywords	object	No	Messages matching one of these exactly (after stripping surrounding whitespace) cancel the queued handoff. Empty means no keyword matching (the cancel button text still matches); the serializer requir...
queueCancelMessage	string	No	Sent to the contact right after they cancel the queued handoff.
copilotEnabled	boolean	No	
copilotTriggerMode	string (enum: manual, every_incoming)	No	`manual` : Manual ; `every_incoming` : Every Incoming;
quotaExceededMessage	string	No	Custom message sent to the visitor when the inbox credit quota is exhausted. Used as the fallback when `quota_exceeded_message_by_locale` has no match for the conversation locale. Empty means the syst...
quotaExceededMessageByLocale	object	No	Locale-keyed quota-exceeded messages. A dict mapping normalized locale codes from CHAT_LANGUAGES (e.g. "en", "zh-hant", "ja") to a message string. Selected by the conversation locale; falls back to `q...

Example Request Schema

```
{
  "groups"?: [ // List of groups that can access this inbox (Optional)
    {
      "id": string (uuid)
    }
  ]
  "channelType"?: string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack,
team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
```

```
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber (Optional)
  "name"?: string // Optional
  "autoReplySchedules"?: [ // Optional
    {
      "isEnabled"?: boolean // Optional
      "timeSlots"?: [ // Optional
        array[string]
      ]
    }
  ]
  "channel"?: // May have different types (Optional)
  {
    "avatar"?: string (uri) // Optional
    "logo"?: string (uri) // Optional
    "description"?: string // Optional
    "cover"?: string (uri) // Optional
    "backUrl"?: string // Optional
    "isActive"?: boolean // Optional
    "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
    }
    "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
    }
    "enableFileUpload"?: boolean // Optional
    "enableDisplayCitations"?: boolean // Optional
    "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
    "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a button row
```

```
floating above the input box (the input box stays usable); when disabled, the current overlay panel is kept. Gradual-rollout switch that can be turned off at any time to roll back. (Optional)
    "enableConversationHandoff"?: boolean // When enabled, this conversation platform's webchat displays the handoff UI (notice bar, summary card, continue button) and registers the handoff tools; still governed by the global handoff switch in system settings (global off = always off). MaiGPT always displays it regardless of this switch. The context usage percentage bar at the bottom is controlled separately by "Display context usage". Disabled by default, so customers' public webchat shows no handoff UI; gradual-rollout switch: turning it off rolls back without a version rollback. (Optional)
    "enableDisplayContextUsage"?: boolean // When enabled, this conversation platform's webchat displays a context usage percentage bar below the input box. Independent of conversation handoff: it is not affected by the handoff switch (including the global handoff switch) and does not affect the handoff notice bar. MaiGPT always displays it regardless of this switch. Disabled by default; turning it off rolls back without a version rollback. (Optional)
    "enableDisplayThinking"?: boolean // Optional
    "enableUserThinkingControl"?: boolean // Optional
    "enableDisplayQueryMetadata"?: boolean // Optional
    "enableDisplayHookMarkers"?: boolean // Optional
    "hookMarkerTextInputModify"?: string // Optional
    "hookMarkerTextOutputModify"?: string // Optional
    "hookMarkerTextInputBlock"?: string // Optional
    "hookMarkerTextOutputBlock"?: string // Optional
    "enableShareConversation"?: boolean // Optional
    "enableLocation"?: boolean // Optional
    "enableShowPoweredBy"?: boolean // Optional
    "poweredByLogo"?: string (uri) // Optional
    "enableAnonymous"?: boolean // Optional
    "enableDisplayPreviousConversation"?: boolean // Optional
    "enableDisplayConversationHistory"?: boolean // Optional
    "enableConversationTimer"?: boolean // Optional
    "conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
    "enableAutoNewConversationOnTimeout"?: boolean // Optional
    "enableSatisfactionSurvey"?: boolean // Optional
    "satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey pops up, range 1-60 minutes (Optional)
    "satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend's localized default text (Optional)
    "enableThemeModeToggle"?: boolean // Optional
    "enableDisplayFontSizeSwitch"?: boolean // Optional
    "defaultThemeMode"?:  // Optional
    {
    }
    "enableDownloadCitations"?: boolean // Optional
    "customDomain"?:  // Optional
    {
      "domain": string
    }
    "enableDisplayChatbotAvatar"?: boolean // Optional
    "enableDisplayChatbotName"?: boolean // Optional
    "buttonIcon"?: string (uri) // Optional
    "conversationStartersDisplayCount"?: integer // Optional
    "includeGreetingInContext"?: boolean // When enabled, the conversation greeting and the numbered conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM context so the agent can resolve short replies like "1" to the matching starter option. (Optional)
    "preChatFormEnabled"?: boolean // Optional
    "preChatFormFields"?: object // Optional
    "preChatFormTitle"?: object // Optional
    "preChatFormSubtitle"?: object // Optional
    "loadingDisplayMode"?:  // Waiting-experience style. `narrative` shows per-stage copy; `brand_animation` plays the uploaded asset and hides every system narration line. Defaults to `narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
```

```
    {
    }
    "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a
partially filled dict is valid and never blocks a reply. (Optional)
    "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice for the
animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
    "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the long-
wait copy appears. (Optional)
    "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets, each
falling back to the shared asset when its own slot is empty. (Optional)
    "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
    "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only consulted
when `loading_animation_use_shared_asset` is False; a missing stage falls back to the shared asset.
(Optional)
    "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only
the animation with no text line. Thinking / analyzing / long-wait stages are unaffected. (Optional)
    "toolCardRunningIconUrl"?: string (uri) // Optional
    "toolCardCompletedIconUrl"?: string (uri) // Optional
    "greetings"?: [ // Default greeting messages shown as the AI first message when a new conversation
starts. One message is randomly selected per conversation. Used as the fallback when `greetings_by_locale`
has no match for the request locale. (Optional)
      string
    ]
    "greetingsByLocale"?: object // Locale-keyed greetings. A dict mapping locale codes to a list of
greeting strings (e.g. keys like "en", "zh-TW", "ja"). Selected by the conversation request Accept-Language
header; falls back to `greetings` when no locale matches. (Optional)
    "voiceGreetings"?: [ // Greeting messages spoken aloud when a voice session starts. One message is
randomly selected per session. The spoken greeting is ephemeral and NOT written back to conversation
history. Used as the fallback when `voice_greetings_by_locale` has no match for the request locale.
(Optional)
      string
    ]
    "voiceGreetingsByLocale"?: object // Locale-keyed voice greetings. A dict mapping locale codes to a list
of greeting strings. Selected by the Accept-Language header; falls back to `voice_greetings` when no locale
matches. (Optional)
    "conversationStarters"?: [ // Default suggested questions shown to the end-user as clickable prompts at
the start of a conversation. Used as the fallback when `conversation_starters_by_locale` has no match for
the request locale. (Optional)
      string
    ]
    "conversationStartersByLocale"?: object // Locale-keyed conversation starters. A dict mapping locale
codes to a list of starter strings (e.g. keys like "en", "zh-TW", "ja"). Selected by the Accept-Language
header; falls back to `conversation_starters` when no locale matches. (Optional)
    "allowedEmbedOrigins"?: [ // Origins allowed to embed this web chat and call the SDK contact-setup
endpoint. Each entry is either an exact origin (scheme://host[:port], e.g. https://example.com) or a
wildcard (*.example.com) that matches example.com and all of its subdomains on any scheme/port. An empty
list means no restriction. Validated against the embedding page origin forwarded by the web chat frontend;
bounds unauthorized-domain embedding. Browser-level enforcement (CSP frame-ancestors on the entry page)
additionally requires an AWS/CloudFront deployment; on GCP storage the allowlist is enforced by the API
alone. WARNING: leave empty until the web chat frontend forwards the embedding origin, otherwise setup calls
fall back to the web chat host origin and every request is rejected. (Optional)
      string
    ]
  }
  "organization"?: string (uuid) // Optional
  "signAuth"?:  // Optional
  {
    "signSource": string (uuid)
    "signParams": {
```

```
    {
      "keycloak":
      {
        "clientId": string
        "url": string
        "realm": string
      }
      "line":
      {
        "liffId": string
      }
      "ad":
      {
        "saml": string
      }
      "google"?:  // Optional
      {
        "enabled": boolean
      }
    }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
    "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }
    "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
      string
    ]
  }
  "enableTimeAwareResponse"?: boolean // Enable time aware responses for this inbox (Optional)
  "defaultTimezone"?: string // Default timezone for this inbox (e.g., Asia/Taipei, America/New_York) (Optional)
  "enableSenderNameAwareResponse"?: boolean // Enable sender name aware responses for this inbox (Optional)
  "enableGpsAwareResponse"?: boolean // Enable GPS aware responses for this inbox (Optional)
  "enableOcr"?: boolean // Enable OCR parser, prioritize OCR-enabled parsers when uploading attachments (Optional)
  "enableAgentAttachmentDecision"?: boolean // Kill switch for AGENT-mode attachment handling. When False (default), AGENT-mode chatbots skip auto-embedding on upload and let the agent handle attachments at runtime via ParseAttachmentContentTool / IngestAttachmentTool. When True, AGENT-mode chatbots fall back to auto-embedding chat attachments into the knowledge base on upload, same as non-AGENT modes. Has no effect on non-AGENT chatbot modes (they always auto-embed). (Optional)
  "enableStreamMessage"?: boolean // Optional
```

```
  "enableHumanHandoff"?: boolean // Optional
  "handoffPrompt"?: string // Optional
  "handoffLlm"?: string (uuid) // Optional
  "handoffContextMessageLimit"?: integer // Optional
  "handoffMessage"?: string // Optional
  "queuingMessage"?: string // Optional
  "assignedMessage"?: string // Optional
  "enablePendingReplyMessage"?: boolean // Send a placeholder message to the user while the AI is
processing. (Optional)
  "pendingReplyMessage"?: string // Optional
  "pendingReplyInitialDelaySeconds"?: integer // Delay before sending the first pending reply, so a fast AI
response can pre-empt it. (Optional)
  "pendingReplyIntervalSeconds"?: integer // Interval to repeat the pending reply while the AI is still
processing. (Optional)
  "enablePendingReplyCleanup"?: boolean // Delete pending reply messages after the AI reply is sent. Only
effective on platforms that support message deletion (Teams, Telegram, Slack). (Optional)
  "enableAnalysis"?: boolean // Optional
  "analysisLlm"?: string (uuid) // Optional
  "analysisPrompt"?: string // Optional
  "analysisContextMessageLimit"?: integer // Optional
  "enableServiceHours"?: boolean // Optional
  "serviceHoursTimezone"?: string // Optional
  "offlineBehavior"?: string (enum: show_message, leave_message) // * `show_message` - Show Message
* `leave_message` - Leave Message (Optional)
  "offlineMessage"?: string // Optional
  "serviceHoursSchedules"?: [ // Optional
    {
      "weekday": integer
      "startTime": string (time)
      "endTime": string (time)
      "isEnabled"?: boolean // Optional
    }
  ]
  "dispatchMode"?:  // Auto dispatch assigns transferred conversations to an agent automatically. Claim
(handover) leaves them unassigned for online agents to pick up themselves; timeout reassignment does not
apply and the queue timeout may be set to "no timeout".

* `auto` - Auto Dispatch
* `handover` - Claim (Handover) (Optional)
  {
  }
  "assignmentRoutingModel"?: string (enum: round_robin, least_active) // * `round_robin` - Round Robin
* `least_active` - Least Active (Optional)
  "assignmentMaxConcurrent"?: integer // Optional
  "assignmentQueueTimeout"?: integer // Queue timeout in seconds (Optional)
  "queueTimeoutMessage"?: string // Optional
  "maxQueueSize"?: integer // Optional
  "queueFullMessage"?: string // Optional
  "assignmentTargetMembers"?: [ // Optional
    string (uuid)
  ]
  "enableTagRouting"?: boolean // Optional
  "enableReassignOnTimeout"?: boolean // When on, a queued or assigned-but-unanswered conversation that
times out is reassigned to the next available agent (skipping already-tried agents) instead of returning
straight to the AI; it only returns to the AI once no agent is left to try. (Optional)
  "enableQueueReminder"?: boolean // Reply with a throttled waiting reminder when the contact sends messages
while queued. (Optional)
  "queueReminderMessage"?: string // Optional
  "queueReminderCooldown"?: integer // Minimum seconds between queue reminders in one conversation
(Optional)
  "enableQueueCancel"?: boolean // Allow the contact to cancel a queued handoff and return to the AI by
themselves. (Optional)
  "queueCancelButtonText"?: string // Label of the cancel quick-reply button. Its text also counts as a
```

```
cancel keyword, so tapping the button always cancels even when the keyword list changes. (Optional)
  "queueCancelKeywords"?: object // Messages matching one of these exactly (after stripping surrounding
whitespace) cancel the queued handoff. Empty means no keyword matching (the cancel button text still
matches); the serializer requires at least one keyword while queue cancel is enabled. (Optional)
  "queueCancelMessage"?: string // Sent to the contact right after they cancel the queued handoff.
(Optional)
  "copilotEnabled"?: boolean // Optional
  "copilotTriggerMode"?: string (enum: manual, every_incoming) // * `manual` - Manual
* `every_incoming` - Every Incoming (Optional)
  "quotaExceededMessage"?: string // Custom message sent to the visitor when the inbox credit quota is
exhausted. Used as the fallback when `quota_exceeded_message_by_locale` has no match for the conversation
locale. Empty means the system default message is used. (Optional)
  "quotaExceededMessageByLocale"?: object // Locale-keyed quota-exceeded messages. A dict mapping normalized
locale codes from CHAT_LANGUAGES (e.g. "en", "zh-hant", "ja") to a message string. Selected by the
conversation locale; falls back to `quota_exceeded_message`. (Optional)
}
```

Example request values

```
{
  "chatbot": {
    "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
  },
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "name": "Updated name"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "chatbot": {
      "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "name": "Updated name"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "chatbot": {
      "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "name": "Updated name"
  };

axios.patch("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "chatbot": {
        "id": "80f3170d-eed7-4651-a8da-6b7e6bdbef8b"
    },
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "name": "Updated name"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'chatbot' => [
                'id' => '80f3170d-eed7-4651-a8da-6b7e6bdbef8b'
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'name' => 'Updated name'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack,
team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
```

```
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?:  // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
    {
      "keycloak":
      {
        "clientId": string
        "url": string
        "realm": string
      }
      "line":
      {
        "liffId": string
      }
      "ad":
      {
        "saml": string
      }
      "google"?:  // Optional
      {
        "enabled": boolean
      }
    }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (Optional)
    "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }
    "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }
    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign source. Never overrides an Authorization header a tool already resolves from its own credentials. The visitor identity is the organization member bound to the conversation at login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to (Optional)
      string
    ]
  }
```

```
  "enableAnalysis"?: boolean // Optional
}
```

Example response values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "channelType": "line",
  "unreadConversationsCount": 456,
  "signAuth": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "signSource": "550e8400-e29b-41d4-a716-446655440000",
    "signParams": {
      "keycloak": {
        "clientId": "Response string",
        "url": "Response string",
        "realm": "Response string"
      },
      "line": {
        "liffId": "Response string"
      },
      "ad": {
        "saml": "Response string"
      },
      "google": {
        "enabled": false
      }
    },
    "sourceIdAccessEnabled": false,
    "sourceIdVerifyMode": {},
    "sourceIdSignatureTtlMinutes": {},
    "keycloakTokenPassthrough": false,
    "keycloakTokenPassthroughAllowedHosts": [
      "Response string"
    ]
  },
  "enableAnalysis": false
}
```


Delete a Conversation Platform

DELETE `/api/v1/inboxes/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this Inbox.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status Code	Description

204	No response body

List Webhooks for a Conversation Platform

GET `/api/v1/inboxes/{inboxPk}/webhooks/`

Parameters

Parameter name	Required	Type	Description
`inboxPk`	V	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/?
page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/?page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "inbox": string (uuid)
      "name": string // Webhook configuration name for identification
      "direction":  // When to trigger: incoming (user messages) or outgoing (bot replies)

* `incoming` - Incoming
* `outgoing` - Outgoing
* `receive` - Receive
      {
      }
      "directionLabel": string
      "isEnabled"?: boolean // Optional
      "url": string (uri)
      "method"?:  // Optional
      {
      }
```

```
      "headers"?: object // Custom HTTP headers as JSON object (Optional)
      "templatePreset"?: string (enum: maiagent_standard, genesys_cloud, custom) // * `maiagent_standard` -
MaiAgent Standard
* `genesys_cloud` - Genesys Cloud
* `custom` - Custom (Optional)
      "templatePresetLabel": string
      "customTemplate"?: string // Jinja2 template for custom payload format. Required when template_preset
is "custom". (Optional)
      "payloadPreset"?: // May have different types (Optional)
      string (enum: genesys_cloud) // Preset for parsing incoming webhook payloads (used with
direction=receive)

* `genesys_cloud` - Genesys Cloud (Optional)
      "payloadPresetLabel": string
      "receiveUrl": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example response values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "inbox": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "direction": {},
      "directionLabel": "Response string",
      "isEnabled": false,
      "url": "https://example.com/file.jpg",
      "method": {},
      "headers": null,
      "templatePreset": "maiagent_standard",
      "templatePresetLabel": "Response string",
      "customTemplate": "Response string",
      "payloadPreset": "genesys_cloud",
      "payloadPresetLabel": "Response string",
      "receiveUrl": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Create a Webhook

POST `/api/v1/inboxes/{inboxPk}/webhooks/`

direction is either incoming (triggered when a user message arrives) or outgoing (triggered when the Agent replies). Set template_preset to maiagent_standard to use the MaiAgent standard payload.

Parameters

Parameter name	Required	Type	Description
`inboxPk`	V	string	

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	Webhook configuration name for identification
direction	object	Yes	When to trigger: incoming (user messages) or outgoing (bot replies) `incoming` : Incoming ; `outgoing` : Outgoing ; `receive` : Receive;
isEnabled	boolean	No	
url	string (uri)	No	
method	object	No	
headers	object	No	Custom HTTP headers as JSON object
templatePreset	string (enum: maiagent_standard, genesys_cloud, custom)	No	`maiagent_standard` : MaiAgent Standard ; `genesys_cloud` : Genesys Cloud ; `custom` : Custom;
customTemplate	string	No	Jinja2 template for custom payload format. Required when template_preset is "custom".
secretToken	string	No	Secret token for webhook signature verification
payloadPreset	object	No	Preset for parsing incoming webhook payloads (used with direction=receive) `genesys_cloud` : Genesys Cloud;

Example request schema

```
{
  "name": string // Webhook configuration name for identification
  "direction":  // When to trigger: incoming (user messages) or outgoing (bot replies)

* `incoming` - Incoming
* `outgoing` - Outgoing
* `receive` - Receive
  {
  }
  "isEnabled"?: boolean // Optional
  "url"?: string (uri) // Optional
  "method"?:  // Optional
  {
  }
  "headers"?: object // Custom HTTP headers as JSON object (Optional)
  "templatePreset"?: string (enum: maiagent_standard, genesys_cloud, custom) // * `maiagent_standard` -
```

```
MaiAgent Standard
* `genesys_cloud` - Genesys Cloud
* `custom` - Custom (Optional)
  "customTemplate"?: string // Jinja2 template for custom payload format. Required when template_preset is
"custom". (Optional)
  "secretToken"?: string // Secret token for webhook signature verification (Optional)
  "payloadPreset"?: // May have different types (Optional)
  string (enum: genesys_cloud) // Preset for parsing incoming webhook payloads (used with direction=receive)

* `genesys_cloud` - Genesys Cloud (Optional)
}
```

Example Request Values

```
{
  "name": "Example Name",
  "direction": {},
  "isEnabled": true,
  "url": "https://example.com/file.jpg",
  "method": {},
  "headers": null,
  "templatePreset": "maiagent_standard",
  "customTemplate": "Example String",
  "secretToken": "Example String",
  "payloadPreset": null
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example Name",
    "direction": {},
    "isEnabled": true,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": null,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example Name",
    "direction": {},
    "isEnabled": true,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": null,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": null
  };

axios.post("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "direction": {},
    "isEnabled": True,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": None,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": None
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example Name',
            'direction' => new \stdClass(),
            'isEnabled' => true,
            'url' => 'https://example.com/file.jpg',
            'method' => new \stdClass(),
            'headers' => null,
            'templatePreset' => 'maiagent_standard',
            'customTemplate' => 'Example String',
            'secretToken' => 'Example String',
            'payloadPreset' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "inbox": string (uuid)
  "name": string // Webhook configuration name for identification
  "direction":  // When to trigger: incoming (user messages) or outgoing (bot replies)

* `incoming` - Incoming
* `outgoing` - Outgoing
* `receive` - Receive
```

```
  {
  }
  "directionLabel": string
  "isEnabled"?: boolean // Optional
  "url": string (uri)
  "method"?:  // Optional
  {
  }
  "headers"?: object // Custom HTTP headers as JSON object (Optional)
  "templatePreset"?: string (enum: maiagent_standard, genesys_cloud, custom) // * `maiagent_standard` -
MaiAgent Standard
* `genesys_cloud` - Genesys Cloud
* `custom` - Custom (Optional)
  "templatePresetLabel": string
  "customTemplate"?: string // Jinja2 template for custom payload format. Required when template_preset is
"custom". (Optional)
  "payloadPreset"?: // May have different types (Optional)
  string (enum: genesys_cloud) // Preset for parsing incoming webhook payloads (used with direction=receive)

* `genesys_cloud` - Genesys Cloud (Optional)
  "payloadPresetLabel": string
  "receiveUrl": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "direction": {},
  "directionLabel": "Response String",
  "isEnabled": false,
  "url": "https://example.com/file.jpg",
  "method": {},
  "headers": null,
  "templatePreset": "maiagent_standard",
  "templatePresetLabel": "Response String",
  "customTemplate": "Response String",
  "payloadPreset": "genesys_cloud",
  "payloadPresetLabel": "Response String",
  "receiveUrl": "Response String",
  "createdAt": "Response String",
  "updatedAt": "Response String"
}
```

Retrieve a Specific Webhook

GET `/api/v1/inboxes/{inboxPk}/webhooks/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Inbox Webhook.
`inboxPk`	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "inbox": string (uuid)
  "name": string // Webhook configuration name for identification
  "direction":  // When to trigger: incoming (user messages) or outgoing (bot replies)

* `incoming` - Incoming
* `outgoing` - Outgoing
* `receive` - Receive
  {
  }
  "directionLabel": string
  "isEnabled"?: boolean // Optional
  "url": string (uri)
  "method"?:  // Optional
  {
  }
  "headers"?: object // Custom HTTP headers as JSON object (Optional)
  "templatePreset"?: string (enum: maiagent_standard, genesys_cloud, custom) // * `maiagent_standard` -
MaiAgent Standard
* `genesys_cloud` - Genesys Cloud
* `custom` - Custom (Optional)
  "templatePresetLabel": string
  "customTemplate"?: string // Jinja2 template for custom payload format. Required when template_preset is
"custom". (Optional)
  "payloadPreset"?: // May have different types (Optional)
  string (enum: genesys_cloud) // Preset for parsing incoming webhook payloads (used with direction=receive)

* `genesys_cloud` - Genesys Cloud (Optional)
  "payloadPresetLabel": string
  "receiveUrl": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "direction": {},
  "directionLabel": "Response String",
  "isEnabled": false,
  "url": "https://example.com/file.jpg",
  "method": {},
  "headers": null,
  "templatePreset": "maiagent_standard",
  "templatePresetLabel": "Response String",
  "customTemplate": "Response String",
  "payloadPreset": "genesys_cloud",
  "payloadPresetLabel": "Response String",
  "receiveUrl": "Response String",
  "createdAt": "Response String",
  "updatedAt": "Response String"
}
```

Update a Webhook

PUT `/api/v1/inboxes/{inboxPk}/webhooks/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Inbox Webhook.
`inboxPk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Webhook configuration name for identification
direction	object	Yes	When to trigger: incoming (user messages) or outgoing (bot replies) `incoming` : Incoming ; `outgoing` : Outgoing ; `receive` : Receive;
isEnabled	boolean	No	
url	string (uri)	No	
method	object	No	
headers	object	No	Custom HTTP headers as JSON object
templatePreset	string (enum: maiagent_standard, genesys_cloud, custom)	No	`maiagent_standard` : MaiAgent Standard ; `genesys_cloud` : Genesys Cloud ; `custom` : Custom;
customTemplate	string	No	Jinja2 template for custom payload format. Required when template_preset is "custom".
secretToken	string	No	Secret token for webhook signature verification
payloadPreset	object	No	Preset for parsing incoming webhook payloads (used with direction=receive) `genesys_cloud` : Genesys Cloud;

Example Request Schema

```
{
  "name": string // Webhook configuration name for identification
  "direction":  // When to trigger: incoming (user messages) or outgoing (bot replies)

* `incoming` - Incoming
* `outgoing` - Outgoing
* `receive` - Receive
  {
```

```
  }
  "isEnabled"?: boolean // Optional
  "url"?: string (uri) // Optional
  "method"?:  // Optional
  {
  }
  "headers"?: object // Custom HTTP headers as JSON object (Optional)
  "templatePreset"?: string (enum: maiagent_standard, genesys_cloud, custom) // * `maiagent_standard` -
MaiAgent Standard
* `genesys_cloud` - Genesys Cloud
* `custom` - Custom (Optional)
  "customTemplate"?: string // Jinja2 template for custom payload format. Required when template_preset is
"custom". (Optional)
  "secretToken"?: string // Secret token for webhook signature verification (Optional)
  "payloadPreset"?: // May have different types (Optional)
  string (enum: genesys_cloud) // Preset for parsing incoming webhook payloads (used with direction=receive)

* `genesys_cloud` - Genesys Cloud (Optional)
}
```

Example Request Values

```
{
  "name": "Example Name",
  "direction": {},
  "isEnabled": true,
  "url": "https://example.com/file.jpg",
  "method": {},
  "headers": null,
  "templatePreset": "maiagent_standard",
  "customTemplate": "Example String",
  "secretToken": "Example String",
  "payloadPreset": null
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example Name",
    "direction": {},
    "isEnabled": true,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": null,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example Name",
    "direction": {},
    "isEnabled": true,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": null,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": null
  };

axios.put("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "direction": {},
    "isEnabled": True,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": None,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": None
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example Name',
            'direction' => new \stdClass(),
            'isEnabled' => true,
            'url' => 'https://example.com/file.jpg',
            'method' => new \stdClass(),
            'headers' => null,
            'templatePreset' => 'maiagent_standard',
            'customTemplate' => 'Example String',
            'secretToken' => 'Example String',
            'payloadPreset' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "inbox": string (uuid)
  "name": string // Webhook configuration name for identification
  "direction":  // When to trigger: incoming (user messages) or outgoing (bot replies)

* `incoming` - Incoming
* `outgoing` - Outgoing
* `receive` - Receive
```

```
  {
  }
  "directionLabel": string
  "isEnabled"?: boolean // Optional
  "url": string (uri)
  "method"?:  // Optional
  {
  }
  "headers"?: object // Custom HTTP headers as JSON object (Optional)
  "templatePreset"?: string (enum: maiagent_standard, genesys_cloud, custom) // * `maiagent_standard` -
MaiAgent Standard
* `genesys_cloud` - Genesys Cloud
* `custom` - Custom (Optional)
  "templatePresetLabel": string
  "customTemplate"?: string // Jinja2 template for custom payload format. Required when template_preset is
"custom". (Optional)
  "payloadPreset"?: // May have different types (Optional)
  string (enum: genesys_cloud) // Preset for parsing incoming webhook payloads (used with direction=receive)

* `genesys_cloud` - Genesys Cloud (Optional)
  "payloadPresetLabel": string
  "receiveUrl": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "direction": {},
  "directionLabel": "Response String",
  "isEnabled": false,
  "url": "https://example.com/file.jpg",
  "method": {},
  "headers": null,
  "templatePreset": "maiagent_standard",
  "templatePresetLabel": "Response String",
  "customTemplate": "Response String",
  "payloadPreset": "genesys_cloud",
  "payloadPresetLabel": "Response String",
  "receiveUrl": "Response String",
  "createdAt": "Response String",
  "updatedAt": "Response String"
}
```

Partially Update a Webhook

PATCH `/api/v1/inboxes/{inboxPk}/webhooks/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Inbox Webhook.
`inboxPk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	Webhook configuration name for identification
direction	object	No	When to trigger: incoming (user messages) or outgoing (bot replies) `incoming` : Incoming ; `outgoing` : Outgoing ; `receive` : Receive;
isEnabled	boolean	No	
url	string (uri)	No	
method	object	No	
headers	object	No	Custom HTTP headers as JSON object
templatePreset	string (enum: maiagent_standard, genesys_cloud, custom)	No	`maiagent_standard` : MaiAgent Standard ; `genesys_cloud` : Genesys Cloud ; `custom` : Custom;
customTemplate	string	No	Jinja2 template for custom payload format. Required when template_preset is "custom".
secretToken	string	No	Secret token for webhook signature verification
payloadPreset	object	No	Preset for parsing incoming webhook payloads (used with direction=receive) `genesys_cloud` : Genesys Cloud;

Example Request Schema

```
{
  "name"?: string // Webhook configuration name for identification (Optional)
  "direction"?:  // When to trigger: incoming (user messages) or outgoing (bot replies)

* `incoming` - Incoming
* `outgoing` - Outgoing
* `receive` - Receive (Optional)
  {
  }
  "isEnabled"?: boolean // Optional
  "url"?: string (uri) // Optional
  "method"?:  // Optional
  {
  }
  "headers"?: object // Custom HTTP headers as JSON object (Optional)
  "templatePreset"?: string (enum: maiagent_standard, genesys_cloud, custom) // * `maiagent_standard` -
MaiAgent Standard
```

```
* `genesys_cloud` - Genesys Cloud
* `custom` - Custom (Optional)
  "customTemplate"?: string // Jinja2 template for custom payload format. Required when template_preset is
"custom". (Optional)
  "secretToken"?: string // Secret token for webhook signature verification (Optional)
  "payloadPreset"?: // May have different types (Optional)
  string (enum: genesys_cloud) // Preset for parsing incoming webhook payloads (used with direction=receive)

* `genesys_cloud` - Genesys Cloud (Optional)
}
```

Example Request Values

```
{
  "name": "Example Name",
  "direction": {},
  "isEnabled": true,
  "url": "https://example.com/file.jpg",
  "method": {},
  "headers": null,
  "templatePreset": "maiagent_standard",
  "customTemplate": "Example String",
  "secretToken": "Example String",
  "payloadPreset": null
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example Name",
    "direction": {},
    "isEnabled": true,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": null,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example Name",
    "direction": {},
    "isEnabled": true,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": null,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": null
  };

axios.patch("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "direction": {},
    "isEnabled": True,
    "url": "https://example.com/file.jpg",
    "method": {},
    "headers": None,
    "templatePreset": "maiagent_standard",
    "customTemplate": "Example String",
    "secretToken": "Example String",
    "payloadPreset": None
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example Name',
            'direction' => new \stdClass(),
            'isEnabled' => true,
            'url' => 'https://example.com/file.jpg',
            'method' => new \stdClass(),
            'headers' => null,
            'templatePreset' => 'maiagent_standard',
            'customTemplate' => 'Example String',
            'secretToken' => 'Example String',
            'payloadPreset' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "inbox": string (uuid)
  "name": string // Webhook configuration name for identification
  "direction":  // When to trigger: incoming (user messages) or outgoing (bot replies)

* `incoming` - Incoming
* `outgoing` - Outgoing
* `receive` - Receive
```

```
  {
  }
  "directionLabel": string
  "isEnabled"?: boolean // Optional
  "url": string (uri)
  "method"?:  // Optional
  {
  }
  "headers"?: object // Custom HTTP headers as JSON object (Optional)
  "templatePreset"?: string (enum: maiagent_standard, genesys_cloud, custom) // * `maiagent_standard` -
MaiAgent Standard
* `genesys_cloud` - Genesys Cloud
* `custom` - Custom (Optional)
  "templatePresetLabel": string
  "customTemplate"?: string // Jinja2 template for custom payload format. Required when template_preset is
"custom". (Optional)
  "payloadPreset"?: // May have different types (Optional)
  string (enum: genesys_cloud) // Preset for parsing incoming webhook payloads (used with direction=receive)

* `genesys_cloud` - Genesys Cloud (Optional)
  "payloadPresetLabel": string
  "receiveUrl": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inbox": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "direction": {},
  "directionLabel": "Response String",
  "isEnabled": false,
  "url": "https://example.com/file.jpg",
  "method": {},
  "headers": null,
  "templatePreset": "maiagent_standard",
  "templatePresetLabel": "Response String",
  "customTemplate": "Response String",
  "payloadPreset": "genesys_cloud",
  "payloadPresetLabel": "Response String",
  "receiveUrl": "Response String",
  "createdAt": "Response String",
  "updatedAt": "Response String"
}
```

Delete Webhook

DELETE `/api/v1/inboxes/{inboxPk}/webhooks/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Inbox Webhook.
`inboxPk`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-446655440000/webhooks/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/inboxes/550e8400-e29b-41d4-a716-
446655440000/webhooks/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

API call example (Shell)

目錄

1. Create a tool
2. List tools
3. Retrieve a Specific Tool
4. Update a Tool
5. Partially update a tool
6. Delete a tool
7. List available tools
8. Create connector
9. List connectors
10. Retrieve a specific connector
11. Update Connector
12. Partially Update Connector
13. Delete Connector
14. List Authorized Connector Members
15. Remove Authorized Connector Member
16. List MCP Tool Registries
17. Retrieve a Specific MCP Tool Registry
18. List the Organization’ s Self-hosted MCP Servers
19. Add a Self-hosted MCP Server
20. Retrieve a Specific Self-hosted MCP Server
21. Partially Update a Self-hosted MCP Server
22. Delete a Self-hosted MCP Server
23. Test MCP Server Connection
24. List tool execution records
25. Retrieve a specific tool execution record
26. Get AI Assistant Filter Options for Execution Records
27. Get tool filter options for execution records
28. List execution records for a specific tool
29. Get a specific execution record for a specific tool
30. Get AI Assistant Filter Options for Execution Records of a Specific Tool
31. Get Tool Filter Options for a Specific Tool's Execution Records

API call example (Shell)

Create a tool

POST `/api/v1/tools/`

Request body

Request parameters

Field	Type	Required	Description
name	object	No	Tool name may only contain letters, digits, underscores (_) and hyphens (-). Not required for MCP tools; for API tools it is the tool name given to the LLM.
displayName	string	Yes	A name with any characters, used by users for configuration and management.
description	string	No	Tool description shown to users. Not required for MCP tools.
prompt	string	No	Tool description/prompt given to the LLM. If empty, the description field is used. Not required for MCP tools.
toolType	string (enum: api, function, mcp, desktop, flex, quick_reply, quick_menu)	No	`api` : API ; `function` : Function ; `mcp` : MCP ; `desktop` : Desktop ; `flex` : Flex Message ; `quick_reply` : Quick Reply ; `quick_menu` : Quick Menu;
apiUrl	string	No	
httpMethod	object	No	
rawHeaders	object	No	
sensitiveHeaders	array[string]	No	

Field	Type	Required	Description
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	Default headers used when the Contact has no matching MCP Credential.
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from

Field	Type	Required	Description
			flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
quickMenuConfig	object	No	Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a d...
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	`knowledge_base` : Knowledge base retrieval ; `web_search` : Web search ; `web_fetch` : Web page fetch ; `code_execution` : Code execution ; `file_processing` : File processing ; `image_video_generation` : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Example request structure

```
{
  "name"?: // May have different types (optional)
  string // Tool name may only contain letters, digits, underscores (_) and
hyphens (-). Not required for MCP tools; for API tools it is the tool name given
to the LLM. (optional)
  "displayName": string // A name with any characters, used by users for
configuration and management.
  "description"?: string // Tool description shown to users. Not required for MCP
tools. (optional)
  "prompt"?: string // Tool description/prompt given to the LLM. If empty, the
description field is used. Not required for MCP tools. (optional)
  "toolType"?: string (enum: api, function, mcp, desktop, flex, quick_reply,
quick_menu) // * `api` - API
* `function` - Function
* `mcp` - MCP
* `desktop` - Desktop
* `flex` - Flex Message
* `quick_reply` - Quick Reply
* `quick_menu` - Quick Menu (optional)
  "apiUrl"?: string // optional
  "httpMethod"?: // May have different types (optional)
  string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
  "sensitiveHeaders"?: [ // optional
    string
  ]
  "rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win —
use this field for API keys that must travel in the query string. (optional)
  "rawParametersSchema"?: object // optional
  "functionCode"?: string // optional
  "mcpUrl"?: string // optional
  "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (optional)
  "mcpAllowedTools"?: object // optional
  "rawMcpHeader"?: object // Default headers used when the Contact has no
matching MCP Credential. (optional)
  "authSubject"?:  // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
```

```
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (optional)
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // May have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (optional)
    {
      "id": string (uuid)
    }
  ]
}
```

Example request values

```
{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": "api",
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "sensitiveHeaders": [
    "Example string"
  ],
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```


Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/tools/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "sensitiveHeaders": [
      "Example string"
    ],
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "sensitiveHeaders": [
      "Example string"
    ],
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
```

```
  };

axios.post("https://api.maiagent.ai/api/v1/tools/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "name": None,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": None,
    "rawHeaders": None,
    "sensitiveHeaders": [
        "Example string"
    ],
    "rawQueryParams": None,
    "rawParametersSchema": None,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": None,
    "rawMcpHeader": None,
    "authSubject": {},
    "flexTemplateJson": None,
    "flexSchema": None,
    "quickReplyConfig": None,
    "quickMenuConfig": None,
    "category": "knowledge_base",
    "displayCopyByLocale": None,
    "outputRenderTemplateByLocale": None,
    "toolDisplayMode": None,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
response = requests.post(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/tools/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => null,
            'displayName' => 'Example string',
            'description' => 'Example string',
            'prompt' => 'Example string',
            'toolType' => 'api',
            'apiUrl' => 'Example string',
            'httpMethod' => null,
            'rawHeaders' => null,
            'sensitiveHeaders' => [
                'Example string'
            ],
            'rawQueryParams' => null,
            'rawParametersSchema' => null,
            'functionCode' => 'Example string',
            'mcpUrl' => 'Example string',
            'mcpRegistry' => '550e8400-e29b-41d4-a716-446655440000',
            'mcpAllowedTools' => null,
            'rawMcpHeader' => null,
            'authSubject' => new \stdClass(),
            'flexTemplateJson' => null,
            'flexSchema' => null,
            'quickReplyConfig' => null,
            'quickMenuConfig' => null,
            'category' => 'knowledge_base',
            'displayCopyByLocale' => null,
            'outputRenderTemplateByLocale' => null,
            'toolDisplayMode' => null,
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
```

```
            ]
        ]);

        $data = json_decode($response->getBody(), true);
        echo "Response retrieved successfully:\n";
        print_r($data);
    } catch (Exception $e) {
        echo 'An error occurred during the request: ' . $e->getMessage();
    }
    ?>
```

Response body

Status code: 201

Example response structure

```
{
  "id": string (uuid)
  "name"?: // May have different types (optional)
  string // Tool name may only contain letters, digits, underscores (_) and
hyphens (-). Not required for MCP tools; for API tools it is the tool name given
to the LLM. (optional)
  "displayName": string // A name with any characters, used by users for
configuration and management.
  "description"?: string // Tool description shown to users. Not required for MCP
tools. (optional)
  "prompt"?: string // Tool description/prompt given to the LLM. If empty, the
description field is used. Not required for MCP tools. (optional)
  "toolType"?: string (enum: api, function, mcp, desktop, flex, quick_reply,
quick_menu) // * `api` - API
* `function` - Function
* `mcp` - MCP
* `desktop` - Desktop
* `flex` - Flex Message
* `quick_reply` - Quick Reply
* `quick_menu` - Quick Menu (optional)
  "apiUrl"?: string // optional
  "httpMethod"?: // May have different types (optional)
  string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
```

```
  "rawHeadersMasked": object
  "sensitiveHeaders"?: [ // optional
    string
  ]
  "globalSensitiveHeaders": [ // Expose platform defaults as read-only authoring
guidance.
    string
  ]
  "rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win —
use this field for API keys that must travel in the query string. (optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // optional
  "functionCode"?: string // optional
  "organization": string (uuid)
  "createdBy": string (uuid)
  "isGlobal": boolean // If set as global tool, all organizations can use this
tool
  "mcpUrl"?: string // optional
  "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (optional)
  "mcpAllowedTools"?: object // optional
  "rawMcpHeader"?: object // Default headers used when the Contact has no
matching MCP Credential. (optional)
  "rawMcpHeaderMasked": object
  "authSubject"?:  // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (optional)
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
```

```
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // May have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Example response values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "prompt": "Response string",
  "toolType": "api",
  "apiUrl": "Response string",
  "httpMethod": "get",
```

```
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "sensitiveHeaders": [
    "Response string"
  ],
  "globalSensitiveHeaders": [
    "Response string"
  ],
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawParametersSchema": null,
  "functionCode": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "isGlobal": false,
  "mcpUrl": "Response string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "rawMcpHeaderMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": "full",
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string",
```

```
  "canUpdate": false,
  "canDelete": false
}
```

List tools

`GET` `/api/v1/tools/`

Parameters

Parameter name	Required	Type	Description
`clientPlatform`	X	string	Calling client platform. Pass "desktop" to include desktop-type tools; any other value, or omitting it, excludes desktop tools by default
`isGlobal`	X	boolean	Set to "true" to retrieve global tools; otherwise organization tools are retrieved
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`toolType`	X	string	Filter by tool type: api, function, mcp, desktop

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name"?: // May have different types (Optional)
```

```
      string // Tool name may only contain letters, digits, underscores (_) and
hyphens (-). Not required for MCP tools; for API tools it is the tool name given
to the LLM. (Optional)
      "displayName": string // A name with any characters, used by users for
configuration and management.
      "description"?: string // Tool description shown to users. Not required for
MCP tools. (Optional)
      "prompt"?: string // Tool description/prompt given to the LLM. If empty,
the description field is used. Not required for MCP tools. (Optional)
      "toolType"?: string (enum: api, function, mcp, desktop, flex, quick_reply,
quick_menu) // * `api` - API
* `function` - Function
* `mcp` - MCP
* `desktop` - Desktop
* `flex` - Flex Message
* `quick_reply` - Quick Reply
* `quick_menu` - Quick Menu (Optional)
      "apiUrl"?: string // Optional
      "httpMethod"?: // May have different types (Optional)
      string (enum: get, post, put, delete, patch) // Optional
      "rawHeaders"?: object // Optional
      "rawHeadersMasked": object
      "sensitiveHeaders"?: [ // Optional
        string
      ]
      "globalSensitiveHeaders": [ // Expose platform defaults as read-only
authoring guidance.
        string
      ]
      "rawQueryParams"?: object // Query string parameters appended to every
request of this API tool. Values support the same {{variable}} template rendering
as raw_headers. On key collision with LLM-generated arguments, these values win —
use this field for API keys that must travel in the query string. (Optional)
      "rawQueryParamsMasked": object
      "rawParametersSchema"?: object // Optional
      "functionCode"?: string // Optional
      "organization": string (uuid)
      "createdBy": string (uuid)
      "isGlobal": boolean // If set as global tool, all organizations can use
this tool
      "mcpUrl"?: string // Optional
      "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
      "mcpAllowedTools"?: object // Optional
      "rawMcpHeader"?: object // Default headers used when the Contact has no
matching MCP Credential. (Optional)
      "rawMcpHeaderMasked": object
      "authSubject"?:  // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
```

```
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
      {
      }
      "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
      "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON
object whose keys are field paths parsed from flex_template_json (e.g. text /
image url / button label / button uri / button message). Each value may be either
a plain string (the AI prompt) or an object carrying additional metadata such as
a field tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to
the template default. (Optional)
      "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
      "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools.
Structured JSON describing the persistent input-bar menu rendered by the web chat
only: layout (grid / list), default open state, and 2-6 items each carrying a
display label plus an action payload (send prompt / prefill text / open http(s)
link). (Optional)
      "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
      "displayCopyByLocale"?: object // Optional
      "outputRenderTemplateByLocale"?: object // Optional
      "toolDisplayMode"?: // May have different types (Optional)
      string (enum: full, compact, hidden) // Optional
      "groups"?: [ // Groups this tool is assigned to. Non-owner members must
pick at least one group they belong to. (Optional)
        {
          "id": string (uuid)
          "name": string
        }
```

```
      ]
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
      "canUpdate": boolean // Return whether the current user can update this
resource.
      "canDelete": boolean // Return whether the current user can delete this
resource.
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "displayName": "Response String",
      "description": "Response String",
      "prompt": "Response String",
      "toolType": "api",
      "apiUrl": "Response String",
      "httpMethod": "get",
      "rawHeaders": null,
      "rawHeadersMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "sensitiveHeaders": [
        "Response String"
      ],
      "globalSensitiveHeaders": [
        "Response String"
      ],
      "rawQueryParams": null,
      "rawQueryParamsMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "rawParametersSchema": null,
      "functionCode": "Response String",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "createdBy": "550e8400-e29b-41d4-a716-446655440000",
      "isGlobal": false,
      "mcpUrl": "Response String",
      "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
      "mcpAllowedTools": null,
      "rawMcpHeader": null,
      "rawMcpHeaderMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "authSubject": {},
      "flexTemplateJson": null,
      "flexSchema": null,
      "quickReplyConfig": null,
      "quickMenuConfig": null,
      "category": "knowledge_base",
      "displayCopyByLocale": null,
      "outputRenderTemplateByLocale": null,
      "toolDisplayMode": "full",
      "groups": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String"
        }
      ],
      "createdAt": "Response String",
      "updatedAt": "Response String",
      "canUpdate": false,
      "canDelete": false
    }
  ]
}
```


Retrieve a Specific Tool

`GET` `/api/v1/tools/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Tool.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
  string // Tool name may only contain letters, digits, underscores (_) and hyphens (-). Not required for MCP tools; for API tools it is the tool name given to the LLM. (Optional)
  "displayName": string // A name with any characters, used by users for configuration and management.
```

```
  "description"?: string // Tool description shown to users. Not required for MCP
tools. (Optional)
  "prompt"?: string // Tool description/prompt given to the LLM. If empty, the
description field is used. Not required for MCP tools. (Optional)
  "toolType"?: string (enum: api, function, mcp, desktop, flex, quick_reply,
quick_menu) // * `api` - API
* `function` - Function
* `mcp` - MCP
* `desktop` - Desktop
* `flex` - Flex Message
* `quick_reply` - Quick Reply
* `quick_menu` - Quick Menu (Optional)
  "apiUrl"?: string // Optional
  "httpMethod"?: // May have different types (Optional)
  string (enum: get, post, put, delete, patch) // Optional
  "rawHeaders"?: object // Optional
  "rawHeadersMasked": object
  "sensitiveHeaders"?: [ // Optional
    string
  ]
  "globalSensitiveHeaders": [ // Expose platform defaults as read-only authoring
guidance.
    string
  ]
  "rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win —
use this field for API keys that must travel in the query string. (Optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // Optional
  "functionCode"?: string // Optional
  "organization": string (uuid)
  "createdBy": string (uuid)
  "isGlobal": boolean // If set as global tool, all organizations can use this
tool
  "mcpUrl"?: string // Optional
  "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
  "mcpAllowedTools"?: object // Optional
  "rawMcpHeader"?: object // Default headers used when the Contact has no
matching MCP Credential. (Optional)
  "rawMcpHeaderMasked": object
  "authSubject"?:  // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
```

```
* `caller` - Caller (Optional)
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(Optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // Optional
  "toolDisplayMode"?: // May have different types (Optional)
  string (enum: full, compact, hidden) // Optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
```

```
  "canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "displayName": "Response String",
  "description": "Response String",
  "prompt": "Response String",
  "toolType": "api",
  "apiUrl": "Response String",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "sensitiveHeaders": [
    "Response String"
  ],
  "globalSensitiveHeaders": [
    "Response String"
  ],
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawParametersSchema": null,
  "functionCode": "Response String",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "isGlobal": false,
  "mcpUrl": "Response String",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "rawMcpHeaderMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "authSubject": {},
```

```
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": "full",
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "createdAt": "Response String",
  "updatedAt": "Response String",
  "canUpdate": false,
  "canDelete": false
}
```


Update a Tool

PUT `/api/v1/tools/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Tool.

Request Body

Request Parameters

Field	Type	Required	Description
name	object	No	Tool name may only contain letters, digits, underscores (_) and hyphens (-). Not required for MCP tools; for API tools it is the tool name given to the LLM.
displayName	string	Yes	A name with any characters, used by users for configuration and management.
description	string	No	Tool description shown to users. Not required for MCP tools.
prompt	string	No	Tool description/prompt given to the LLM. If empty, the description field is used. Not required for MCP tools.
toolType	string (enum: api, function, mcp, desktop, flex, quick_reply, quick_menu)	No	`api` : API ; `function` : Function ; `mcp` : MCP ; `desktop` : Desktop ; `flex` : Flex Message ; `quick_reply` : Quick Reply ; `quick_menu` : Quick Menu;
apiUrl	string	No	
httpMethod	object	No	
rawHeaders	object	No	
sensitiveHeaders	array[string]	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values ...

Field	Type	Required	Description
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	Default headers used when the Contact has no matching MCP Credential.
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip

Field	Type	Required	Description
			buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
quickMenuConfig	object	No	Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a d...
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	`knowledge_base` : Knowledge base retrieval ; `web_search` : Web search ; `web_fetch` : Web page fetch ; `code_execution` : Code execution ; `file_processing` : File processing ; `image_video_generation` : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Example Request Schema

```
{
  "name"?: // May have different types (Optional)
  string // Tool name may only contain letters, digits, underscores (_) and
hyphens (-). Not required for MCP tools; for API tools it is the tool name given
to the LLM. (Optional)
  "displayName": string // A name with any characters, used by users for
```

```
configuration and management.
  "description"?: string // Tool description shown to users. Not required for MCP tools. (Optional)
  "prompt"?: string // Tool description/prompt given to the LLM. If empty, the description field is used. Not required for MCP tools. (Optional)
  "toolType"?: string (enum: api, function, mcp, desktop, flex, quick_reply, quick_menu) // * `api` - API
* `function` - Function
* `mcp` - MCP
* `desktop` - Desktop
* `flex` - Flex Message
* `quick_reply` - Quick Reply
* `quick_menu` - Quick Menu (Optional)
  "apiUrl"?: string // Optional
  "httpMethod"?: // May have different types (Optional)
  string (enum: get, post, put, delete, patch) // Optional
  "rawHeaders"?: object // Optional
  "sensitiveHeaders"?: [ // Optional
    string
  ]
  "rawQueryParams"?: object // Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values win — use this field for API keys that must travel in the query string. (Optional)
  "rawParametersSchema"?: object // Optional
  "functionCode"?: string // Optional
  "mcpUrl"?: string // Optional
  "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for manual-URL MCP tools. (Optional)
  "mcpAllowedTools"?: object // Optional
  "rawMcpHeader"?: object // Default headers used when the Contact has no matching MCP Credential. (Optional)
  "authSubject"?:  // Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per contact credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.). (Optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be either a plain string (the AI prompt) or an object carrying additional metadata such as a field
```

```
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(Optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // Optional
  "toolDisplayMode"?: // May have different types (Optional)
  string (enum: full, compact, hidden) // Optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
    {
      "id": string (uuid)
    }
  ]
}
```

Example request values

```
{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": "api",
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "sensitiveHeaders": [
```

```
    "Example string"
  ],
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```


Code examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "sensitiveHeaders": [
      "Example string"
    ],
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'
```

```
# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "sensitiveHeaders": [
      "Example string"
    ],
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
```

```
  };

axios.put("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": None,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": None,
    "rawHeaders": None,
    "sensitiveHeaders": [
        "Example string"
    ],
    "rawQueryParams": None,
    "rawParametersSchema": None,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": None,
    "rawMcpHeader": None,
    "authSubject": {},
    "flexTemplateJson": None,
    "flexSchema": None,
    "quickReplyConfig": None,
    "quickMenuConfig": None,
    "category": "knowledge_base",
    "displayCopyByLocale": None,
    "outputRenderTemplateByLocale": None,
    "toolDisplayMode": None,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => null,
            'displayName' => 'Example string',
            'description' => 'Example string',
            'prompt' => 'Example string',
            'toolType' => 'api',
            'apiUrl' => 'Example string',
            'httpMethod' => null,
            'rawHeaders' => null,
            'sensitiveHeaders' => [
                'Example string'
            ],
            'rawQueryParams' => null,
            'rawParametersSchema' => null,
            'functionCode' => 'Example string',
            'mcpUrl' => 'Example string',
            'mcpRegistry' => '550e8400-e29b-41d4-a716-446655440000',
            'mcpAllowedTools' => null,
            'rawMcpHeader' => null,
            'authSubject' => new \stdClass(),
            'flexTemplateJson' => null,
            'flexSchema' => null,
            'quickReplyConfig' => null,
            'quickMenuConfig' => null,
            'category' => 'knowledge_base',
            'displayCopyByLocale' => null,
            'outputRenderTemplateByLocale' => null,
            'toolDisplayMode' => null,
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
```

```
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response structure

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
  string // Tool name may only contain letters, digits, underscores (_) and
hyphens (-). Not required for MCP tools; for API tools it is the tool name given
to the LLM. (Optional)
  "displayName": string // A name with any characters, used by users for
configuration and management.
  "description"?: string // Tool description shown to users. Not required for MCP
tools. (Optional)
  "prompt"?: string // Tool description/prompt given to the LLM. If empty, the
description field is used. Not required for MCP tools. (Optional)
  "toolType"?: string (enum: api, function, mcp, desktop, flex, quick_reply,
quick_menu) // * `api` - API
* `function` - Function
* `mcp` - MCP
* `desktop` - Desktop
* `flex` - Flex Message
* `quick_reply` - Quick Reply
* `quick_menu` - Quick Menu (Optional)
  "apiUrl"?: string // Optional
  "httpMethod"?: // May have different types (Optional)
  string (enum: get, post, put, delete, patch) // Optional
```

```
  "rawHeaders"?: object // Optional
  "rawHeadersMasked": object
  "sensitiveHeaders"?: [ // Optional
    string
  ]
  "globalSensitiveHeaders": [ // Expose platform defaults as read-only authoring guidance.
    string
  ]
  "rawQueryParams"?: object // Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values win — use this field for API keys that must travel in the query string. (Optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // Optional
  "functionCode"?: string // Optional
  "organization": string (uuid)
  "createdBy": string (uuid)
  "isGlobal": boolean // If set as global tool, all organizations can use this tool
  "mcpUrl"?: string // Optional
  "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for manual-URL MCP tools. (Optional)
  "mcpAllowedTools"?: object // Optional
  "rawMcpHeader"?: object // Default headers used when the Contact has no matching MCP Credential. (Optional)
  "rawMcpHeaderMasked": object
  "authSubject"?:  // Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per contact credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.). (Optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be either a plain string (the AI prompt) or an object carrying additional metadata such as a field tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the template default. (Optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt. (Optional)
```

```
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(Optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // Optional
  "toolDisplayMode"?: // May have different types (Optional)
  string (enum: full, compact, hidden) // Optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "prompt": "Response string",
  "toolType": "api",
  "apiUrl": "Response string",
```

```
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "sensitiveHeaders": [
    "Response string"
  ],
  "globalSensitiveHeaders": [
    "Response string"
  ],
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawParametersSchema": null,
  "functionCode": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "isGlobal": false,
  "mcpUrl": "Response string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "rawMcpHeaderMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": "full",
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string",
  "canUpdate": false,
  "canDelete": false
}
```


Partially update a tool

PATCH `/api/v1/tools/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this Tool.

Request body

Request parameters

Field	Type	Required	Description
name	object	No	Tool name may only contain letters, digits, underscores (_) and hyphens (-). Not required for MCP tools; for API tools it is the tool name given to the LLM.
displayName	string	No	A name with any characters, used by users for configuration and management.
description	string	No	Tool description shown to users. Not required for

Field	Type	Required	Description
			MCP tools.
prompt	string	No	Tool description/prompt given to the LLM. If empty, the description field is used. Not required for MCP tools.
toolType	string (enum: api, function, mcp, desktop, flex, quick_reply, quick_menu)	No	`api` : API ; `function` : Function ; `mcp` : MCP ; `desktop` : Desktop ; `flex` : Flex Message ; `quick_reply` : Quick Reply ; `quick_menu` : Quick Menu;
apiUrl	string	No	
httpMethod	object	No	
rawHeaders	object	No	
sensitiveHeaders	array[string]	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	Default headers used when the Contact has no matching MCP Credential.

Field	Type	Required	Description
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
quickMenuConfig	object	No	Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a d...

Field	Type	Required	Description
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	`knowledge_base` : Knowledge base retrieval ; `web_search` : Web search ; `web_fetch` : Web page fetch ; `code_execution` : Code execution ; `file_processing` : File processing ; `image_video_generation` : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Example request structure

```
{
  "name"?: // May have different types (optional)
  string // Tool name may only contain letters, digits, underscores (_) and
hyphens (-). Not required for MCP tools; for API tools it is the tool name given
to the LLM. (optional)
  "displayName"?: string // A name with any characters, used by users for
configuration and management. (optional)
  "description"?: string // Tool description shown to users. Not required for MCP
tools. (optional)
  "prompt"?: string // Tool description/prompt given to the LLM. If empty, the
description field is used. Not required for MCP tools. (optional)
  "toolType"?: string (enum: api, function, mcp, desktop, flex, quick_reply,
quick_menu) // * `api` - API
* `function` - Function
* `mcp` - MCP
* `desktop` - Desktop
* `flex` - Flex Message
* `quick_reply` - Quick Reply
* `quick_menu` - Quick Menu (optional)
  "apiUrl"?: string // optional
  "httpMethod"?: // May have different types (optional)
```

```
  string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
  "sensitiveHeaders"?: [ // optional
    string
  ]
  "rawQueryParams"?: object // Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values win — use this field for API keys that must travel in the query string. (optional)
  "rawParametersSchema"?: object // optional
  "functionCode"?: string // optional
  "mcpUrl"?: string // optional
  "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for manual-URL MCP tools. (optional)
  "mcpAllowedTools"?: object // optional
  "rawMcpHeader"?: object // Default headers used when the Contact has no matching MCP Credential. (optional)
  "authSubject"?:  // Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per contact credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.). (optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be either a plain string (the AI prompt) or an object carrying additional metadata such as a field tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the template default. (optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt. (optional)
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured JSON describing the persistent input-bar menu rendered by the web chat only: layout (grid / list), default open state, and 2-6 items each carrying a display label plus an action payload (send prompt / prefill text / open http(s) link). (optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other) // * `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
```

```
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // May have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (optional)
    {
      "id": string (uuid)
    }
  ]
}
```

Example request values

```
{
  "name": null,
  "displayName": "Example string",
  "description": "Example string",
  "prompt": "Example string",
  "toolType": "api",
  "apiUrl": "Example string",
  "httpMethod": null,
  "rawHeaders": null,
  "sensitiveHeaders": [
    "Example string"
  ],
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "Example string",
  "mcpUrl": "Example string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
```

```
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```


Code examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "sensitiveHeaders": [
      "Example string"
    ],
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'
```

```
# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "name": null,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": null,
    "rawHeaders": null,
    "sensitiveHeaders": [
      "Example string"
    ],
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "quickMenuConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
```

```
  };

axios.patch("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "name": None,
    "displayName": "Example string",
    "description": "Example string",
    "prompt": "Example string",
    "toolType": "api",
    "apiUrl": "Example string",
    "httpMethod": None,
    "rawHeaders": None,
    "sensitiveHeaders": [
        "Example string"
    ],
    "rawQueryParams": None,
    "rawParametersSchema": None,
    "functionCode": "Example string",
    "mcpUrl": "Example string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": None,
    "rawMcpHeader": None,
    "authSubject": {},
    "flexTemplateJson": None,
    "flexSchema": None,
    "quickReplyConfig": None,
    "quickMenuConfig": None,
    "category": "knowledge_base",
    "displayCopyByLocale": None,
    "outputRenderTemplateByLocale": None,
    "toolDisplayMode": None,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
response = requests.patch(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => null,
            'displayName' => 'Example string',
            'description' => 'Example string',
            'prompt' => 'Example string',
            'toolType' => 'api',
            'apiUrl' => 'Example string',
            'httpMethod' => null,
            'rawHeaders' => null,
            'sensitiveHeaders' => [
                'Example string'
            ],
            'rawQueryParams' => null,
            'rawParametersSchema' => null,
            'functionCode' => 'Example string',
            'mcpUrl' => 'Example string',
            'mcpRegistry' => '550e8400-e29b-41d4-a716-446655440000',
            'mcpAllowedTools' => null,
            'rawMcpHeader' => null,
            'authSubject' => new \stdClass(),
            'flexTemplateJson' => null,
            'flexSchema' => null,
            'quickReplyConfig' => null,
            'quickMenuConfig' => null,
            'category' => 'knowledge_base',
            'displayCopyByLocale' => null,
            'outputRenderTemplateByLocale' => null,
            'toolDisplayMode' => null,
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
```

```
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response structure

```
{
  "id": string (uuid)
  "name"?: // May have different types (optional)
  string // Tool name may only contain letters, digits, underscores (_) and
hyphens (-). Not required for MCP tools; for API tools it is the tool name given
to the LLM. (optional)
  "displayName": string // A name with any characters, used by users for
configuration and management.
  "description"?: string // Tool description shown to users. Not required for MCP
tools. (optional)
  "prompt"?: string // Tool description/prompt given to the LLM. If empty, the
description field is used. Not required for MCP tools. (optional)
  "toolType"?: string (enum: api, function, mcp, desktop, flex, quick_reply,
quick_menu) // * `api` - API
* `function` - Function
* `mcp` - MCP
* `desktop` - Desktop
* `flex` - Flex Message
* `quick_reply` - Quick Reply
* `quick_menu` - Quick Menu (optional)
  "apiUrl"?: string // optional
  "httpMethod"?: // May have different types (optional)
```

```
  string (enum: get, post, put, delete, patch) // optional
  "rawHeaders"?: object // optional
  "rawHeadersMasked": object
  "sensitiveHeaders"?: [ // optional
    string
  ]
  "globalSensitiveHeaders": [ // Expose platform defaults as read-only authoring
guidance.
    string
  ]
  "rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win —
use this field for API keys that must travel in the query string. (optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // optional
  "functionCode"?: string // optional
  "organization": string (uuid)
  "createdBy": string (uuid)
  "isGlobal": boolean // If set as global tool, all organizations can use this
tool
  "mcpUrl"?: string // optional
  "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (optional)
  "mcpAllowedTools"?: object // optional
  "rawMcpHeader"?: object // Default headers used when the Contact has no
matching MCP Credential. (optional)
  "rawMcpHeaderMasked": object
  "authSubject"?:  // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (optional)
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
```

```
generated) mode, each field either a fixed value or an LLM prompt. (optional)
  "quickMenuConfig"?: object // Authoring config for QUICK_MENU tools. Structured
JSON describing the persistent input-bar menu rendered by the web chat only:
layout (grid / list), default open state, and 2-6 items each carrying a display
label plus an action payload (send prompt / prefill text / open http(s) link).
(optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (optional)
  "displayCopyByLocale"?: object // optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // May have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "prompt": "Response string",
  "toolType": "api",
```

```
  "apiUrl": "Response string",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "sensitiveHeaders": [
    "Response string"
  ],
  "globalSensitiveHeaders": [
    "Response string"
  ],
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "rawParametersSchema": null,
  "functionCode": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "isGlobal": false,
  "mcpUrl": "Response string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "rawMcpHeaderMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "quickMenuConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": "full",
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
```

```
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "canUpdate": false,
  "canDelete": false
}
```


Delete a tool

DELETE `/api/v1/tools/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this Tool.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request payload
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
204	No response body

List available tools

GET `/api/v1/tools/available-tools/`

Parameters

Parameter name	Required	Type	Description
`availableTools`	V	array	List of all available MCP tools
`mcpAllowedTools`	X	array	List of allowed MCP tools
`mcpUrl`	V	string	MCP server URL (required)
`rawMcpHeader`	X		MCP Header (JSON format)
`toolId`	X	string	Tool ID for per-member OAuth token lookup

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "mcpUrl": string // MCP server URL (required)
  "rawMcpHeader"?: object // MCP Header (JSON format) (Optional)
  "mcpAllowedTools"?: [ // List of allowed MCP tools (Optional)
    string
  ]
```

```
  "toolId"?: string (uuid) // Tool ID for per-member OAuth token lookup
(Optional)
  "availableTools": [ // List of all available MCP tools
    string
  ]
}
```

Example response values

```
{
  "mcpUrl": "Response string",
  "rawMcpHeader": null,
  "mcpAllowedTools": [
    "Response string"
  ],
  "toolId": "550e8400-e29b-41d4-a716-446655440000",
  "availableTools": [
    "Response string"
  ]
}
```

Status code: 400

Example response schema

```
{
  "detail"?: string // Optional
}
```

Example response values

```
{
  "detail": "Response string"
}
```

Create connector

POST `/api/v1/connectors/`

Request body

Request parameters

Field	Type	Required	Description
mcpRegistryId	string (uuid)	Yes	
displayName	string	No	Custom display name for this connector (defaults to MCP Registry name)
description	string	No	Custom description for this connector (defaults to MCP Registry description)
defaultCategory	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category. `knowledge_base` : Knowledge base retrieval ; `web_search` : We...
defaultDisplayGroupNameByLocale	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"": str}. The headline for the conversatio...
defaultToolIcon	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon.
authHeaders	object	No	
allowedContextDataKeys	object	No	Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.}}. An empty list

Field	Type	Required	Description
			disables the feature for this connector: the placeholder is left verbati...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Example request schema

```
{
  "mcpRegistryId": string (uuid)
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
```

```
individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "authHeaders"?: object // Optional
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (Optional)
}
```

Example request values

```
{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/connectors/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
  };

axios.post("https://api.maiagent.ai/api/v1/connectors/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": None,
    "defaultDisplayGroupNameByLocale": None,
    "defaultToolIcon": "Example string",
    "authHeaders": None,
    "allowedContextDataKeys": None,
    "exposedMcpTools": None,
    "isEnabled": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/connectors/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'mcpRegistryId' => '550e8400-e29b-41d4-a716-446655440000',
            'displayName' => 'Example string',
            'description' => 'Example string',
            'defaultCategory' => null,
            'defaultDisplayGroupNameByLocale' => null,
            'defaultToolIcon' => 'Example string',
            'authHeaders' => null,
            'allowedContextDataKeys' => null,
            'exposedMcpTools' => null,
            'isEnabled' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Example response schema

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
alphanumeric, underscore and hyphen allowed.
    "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
    "description"?: string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
    "logoUrl": string (uri)
    "isActive"?: boolean // Whether this MCP service is currently available for
use (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
available to all organizations. (Optional)
    "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.
```

```
* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "effectiveDescription": string
  "remoteMcpUrl": string
  "oauthServerUrl": string
  "logoUrl": string (uri)
  "authType": string
  "authHeaders"?: object // Optional
  "authHeadersMasked": object
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)
  "organization": string (uuid)
  "createdBy": string (uuid)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List connectors

GET `/api/v1/connectors/`

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/connectors/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
[
  {
    "id": string (uuid)
    "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
    "mcpRegistryName": string
    "mcpRegistryId": string
    "mcpRegistryIsGlobal": boolean
    "logoUrl": string (uri)
```

```
    "defaultCategory"?: // May have different types (Optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
    "isEnabled"?: boolean // Whether this connector is enabled for contacts in
the organization (Optional)
    "authHeadersMasked": object
    "allowedContextDataKeys": object // Allowlist of webchat contextData keys
this connector may render into auth_headers through {{context_data.<key>}}. An
empty list disables the feature for this connector: the placeholder is left
verbatim, exactly as before the feature existed. Keys are matched literally and
are case-sensitive; at most 20 keys of letters, digits and underscores.
    "authorizedMembersCount": integer
    "createdBy":
    {
      "id": string (uuid)
      "name": string
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
```

Example response values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Response string",
    "mcpRegistryName": "Response string",
    "mcpRegistryId": "Response string",
    "mcpRegistryIsGlobal": false,
    "logoUrl": "https://example.com/file.jpg",
    "defaultCategory": "knowledge_base",
```

```
    "isEnabled": false,
    "authHeadersMasked": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    },
    "allowedContextDataKeys": null,
    "authorizedMembersCount": 456,
    "createdBy": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    },
    "createdAt": "Response string",
    "updatedAt": "Response string"
  }
]
```


Retrieve a specific connector

`GET` `/api/v1/connectors/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
alphanumeric, underscore and hyphen allowed.
```

```
    "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
    "description"?: string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
    "logoUrl": string (uri)
    "isActive"?: boolean // Whether this MCP service is currently available for
use (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
available to all organizations. (Optional)
    "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
```

```
  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "effectiveDescription": string
  "remoteMcpUrl": string
  "oauthServerUrl": string
  "logoUrl": string (uri)
  "authType": string
  "authHeaders"?: object // Optional
  "authHeadersMasked": object
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)
  "organization": string (uuid)
  "createdBy": string (uuid)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
```

```
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Update Connector

PUT `/api/v1/connectors/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
mcpRegistryId	string (uuid)	Yes	
displayName	string	No	Custom display name for this connector (defaults to MCP Registry name)
description	string	No	Custom description for this connector (defaults to MCP Registry description)
defaultCategory	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category. `knowledge_base` : Knowledge base retrieval ; `web_search` : We...
defaultDisplayGroupNameByLocale	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"": str}. The headline for the conversatio...
defaultToolIcon	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon.
authHeaders	object	No	

Field	Type	Required	Description
allowedContextDataKeys	object	No	Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.}}. An empty list disables the feature for this connector: the placeholder is left verbati...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Request Schema Example

```
{
  "mcpRegistryId": string (uuid)
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
```

```
without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "authHeaders"?: object // Optional
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)
}
```

Example Request Values

```
{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
  };

axios.put("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": None,
    "defaultDisplayGroupNameByLocale": None,
    "defaultToolIcon": "Example string",
    "authHeaders": None,
    "allowedContextDataKeys": None,
    "exposedMcpTools": None,
    "isEnabled": True
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'mcpRegistryId' => '550e8400-e29b-41d4-a716-446655440000',
            'displayName' => 'Example string',
            'description' => 'Example string',
            'defaultCategory' => null,
            'defaultDisplayGroupNameByLocale' => null,
            'defaultToolIcon' => 'Example string',
            'authHeaders' => null,
            'allowedContextDataKeys' => null,
            'exposedMcpTools' => null,
            'isEnabled' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
alphanumeric, underscore and hyphen allowed.
    "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
    "description"?: string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
    "logoUrl": string (uri)
    "isActive"?: boolean // Whether this MCP service is currently available for
use (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
available to all organizations. (Optional)
    "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.
```

```
* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "effectiveDescription": string
  "remoteMcpUrl": string
  "oauthServerUrl": string
  "logoUrl": string (uri)
  "authType": string
  "authHeaders"?: object // Optional
  "authHeadersMasked": object
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)
  "organization": string (uuid)
  "createdBy": string (uuid)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Partially Update Connector

`PATCH` `/api/v1/connectors/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
mcpRegistryId	string (uuid)	No	
displayName	string	No	Custom display name for this connector (defaults to MCP Registry name)
description	string	No	Custom description for this connector (defaults to MCP Registry description)
defaultCategory	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category. `knowledge_base` : Knowledge base retrieval ; `web_search` : We...
defaultDisplayGroupNameByLocale	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual

Field	Type	Required	Description
			Tool row). Shape: {"": str}. The headline for the conversatio...
defaultToolIcon	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon.
authHeaders	object	No	
allowedContextDataKeys	object	No	Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.}}. An empty list disables the feature for this connector: the placeholder is left verbati...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Request Schema Example

```
{
  "mcpRegistryId"?: string (uuid) // Optional
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
```

```
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "authHeaders"?: object // Optional
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)
}
```

Example Request Values

```
{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "Example string",
  "description": "Example string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
```

```
  "isEnabled": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
  };

axios.patch("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "Example string",
    "description": "Example string",
    "defaultCategory": None,
    "defaultDisplayGroupNameByLocale": None,
    "defaultToolIcon": "Example string",
    "authHeaders": None,
    "allowedContextDataKeys": None,
    "exposedMcpTools": None,
    "isEnabled": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'mcpRegistryId' => '550e8400-e29b-41d4-a716-446655440000',
            'displayName' => 'Example string',
            'description' => 'Example string',
            'defaultCategory' => null,
            'defaultDisplayGroupNameByLocale' => null,
            'defaultToolIcon' => 'Example string',
            'authHeaders' => null,
            'allowedContextDataKeys' => null,
            'exposedMcpTools' => null,
            'isEnabled' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
alphanumeric, underscore and hyphen allowed.
    "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
    "description"?: string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
    "logoUrl": string (uri)
    "isActive"?: boolean // Whether this MCP service is currently available for
use (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
available to all organizations. (Optional)
    "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.
```

```
* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "effectiveDescription": string
  "remoteMcpUrl": string
  "oauthServerUrl": string
  "logoUrl": string (uri)
  "authType": string
  "authHeaders"?: object // Optional
  "authHeadersMasked": object
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)
  "organization": string (uuid)
  "createdBy": string (uuid)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Delete Connector

DELETE `/api/v1/connectors/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

List Authorized Connector Members

GET `/api/v1/connectors/{id}/authorized-members/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/authorized-members/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/authorized-members/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/authorized-members/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
alphanumeric, underscore and hyphen allowed.
```

```
    "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
    "description"?: string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
    "logoUrl": string (uri)
    "isActive"?: boolean // Whether this MCP service is currently available for
use (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
available to all organizations. (Optional)
    "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
```

```
  "defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (Optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (Optional)
  "effectiveDescription": string
  "remoteMcpUrl": string
  "oauthServerUrl": string
  "logoUrl": string (uri)
  "authType": string
  "authHeaders"?: object // Optional
  "authHeadersMasked": object
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)
  "organization": string (uuid)
  "createdBy": string (uuid)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
```

```
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "Response string",
  "remoteMcpUrl": "Response string",
  "oauthServerUrl": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": "Response string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Remove Authorized Connector Member

`DELETE` `/api/v1/connectors/{id}/authorized-members/{memberId}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	
`memberId`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-
41d4-a716-446655440000/authorized-members/{memberId}
"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-
a716-446655440000/authorized-members/{memberId}
", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/authorized-members/{memberId}
"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/authorized-members/{memberId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status code	Description
204	No response body

List MCP Tool Registries

GET `/api/v1/mcp/registry/`

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/mcp/registry/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/mcp/registry/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/registry/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/mcp/registry/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status code: 200

Example Response Schema

```
[
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
alphanumeric, underscore and hyphen allowed.
    "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
    "description"?: string // Brief description of the MCP service (Optional)
```

```
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
    "logoUrl": string (uri)
    "isActive"?: boolean // Whether this MCP service is currently available for use (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and available to all organizations. (Optional)
    "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "Response string",
    "updatedAt": "Response string"
  }
]
```

Retrieve a Specific MCP Tool Registry

GET `/api/v1/mcp/registry/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-
a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric,
underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
```

```
  "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
  "oauthServerUrl"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
  "logoUrl": string (uri)
  "isActive"?: boolean // Whether this MCP service is currently available for use (Optional)
  "isGlobal"?: boolean // Global registries are maintained by the platform and available to all organizations. (Optional)
  "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
  {
  }
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": "https://example.com/file.jpg",
  "logoUrl": "https://example.com/file.jpg",
  "isActive": false,
  "isGlobal": false,
  "authType": {},
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List the Organization’ s Self-hosted MCP Servers

GET `/api/v1/mcp/private-registry/`

Connector management permission is required. For connection instructions, see Self-hosted MCP Server in the technical manual.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/mcp/private-registry/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/mcp/private-registry/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/private-registry/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/mcp/private-registry/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status code: 200

Example Response Schema

```
[
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
    "displayName"?: string // Human-readable display name (defaults to name if empty) (Optional)
    "description"?: string // Brief description of the MCP service (Optional)
```

```
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
    "resourceIdentifier"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discovery fails or the metadata is incorrect. (e.g., Asana uses https://mcp.asana.com, Supabase uses https://mcp.supabase.com/mcp) (Optional)
    "oauthClientId"?: string // Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration. (Optional)
    "oauthClientSecret"?: string // Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none). (Optional)
    "logoUrl"?: // May have different types (Optional)
    string (uri) // Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL. (Optional)
    "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "cspAllowedOrigins"?: object // Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by default; list only what the app genuinely needs. Empty = the host's strict default (self only). (Optional)
    "isActive": boolean // Whether this MCP service is currently available for use
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "displayName": "Response string",
```

```
    "description": "Response string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "resourceIdentifier": "https://example.com/file.jpg",
    "oauthClientId": "Response string",
    "oauthClientSecret": "Response string",
    "logoUrl": "https://example.com/file.jpg",
    "authType": {},
    "cspAllowedOrigins": null,
    "isActive": false,
    "createdAt": "Response string",
    "updatedAt": "Response string"
  }
]
```


Add a Self-hosted MCP Server

POST `/api/v1/mcp/private-registry/`

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
displayName	string	No	Human-readable display name (defaults to name if empty)
description	string	No	Brief description of the MCP service
remoteMcpUrl	string (uri)	Yes	The URL of the Remote MCP service
oauthServerUrl	object	No	Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server ad...

Field	Type	Required	Description
resourceIdentifier	object	No	Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discov...
oauthClientId	string	No	Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration.
oauthClientSecret	string	No	Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none).
logoUrl	object	No	Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL.
authType	object	No	How to authenticate with this MCP server. `oauth` : OAuth 2.0 ; `service_account` : Service Account ; `none` : No Authentication;
cspAllowedOrigins	object	No	Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by de...

Example Request Schema

```
{
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
  "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
  "oauthServerUrl"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
  "resourceIdentifier"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discovery fails or the metadata is incorrect. (e.g., Asana uses https://mcp.asana.com, Supabase uses
```

```
https://mcp.supabase.com/mcp) (Optional)
  "oauthClientId"?: string // Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration. (Optional)
  "oauthClientSecret"?: string // Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none). (Optional)
  "logoUrl"?: // May have different types (Optional)
  string (uri) // Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL. (Optional)
  "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
  {
  }
  "cspAllowedOrigins"?: object // Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by default; list only what the app genuinely needs. Empty = the host's strict default (self only). (Optional)
}
```

Example Request Values

```
{
  "name": "Example name",
  "displayName": "Example string",
  "description": "Example string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": null,
  "resourceIdentifier": null,
  "oauthClientId": "Example string",
  "oauthClientSecret": "Example string",
  "logoUrl": null,
  "authType": {},
  "cspAllowedOrigins": null
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/mcp/private-registry/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": null,
    "resourceIdentifier": null,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": null,
    "authType": {},
    "cspAllowedOrigins": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": null,
    "resourceIdentifier": null,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": null,
    "authType": {},
    "cspAllowedOrigins": null
  };

axios.post("https://api.maiagent.ai/api/v1/mcp/private-registry/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/private-registry/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": None,
    "resourceIdentifier": None,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": None,
    "authType": {},
    "cspAllowedOrigins": None
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/mcp/private-registry/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'displayName' => 'Example string',
            'description' => 'Example string',
            'remoteMcpUrl' => 'https://example.com/file.jpg',
            'oauthServerUrl' => null,
            'resourceIdentifier' => null,
            'oauthClientId' => 'Example string',
            'oauthClientSecret' => 'Example string',
            'logoUrl' => null,
            'authType' => new \stdClass(),
            'cspAllowedOrigins' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
  "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
  "oauthServerUrl"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
  "resourceIdentifier"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discovery fails or the metadata is incorrect. (e.g., Asana uses https://mcp.asana.com, Supabase uses https://mcp.supabase.com/mcp) (Optional)
  "oauthClientId"?: string // Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration. (Optional)
  "oauthClientSecret"?: string // Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none). (Optional)
  "logoUrl"?: // May have different types (Optional)
  string (uri) // Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL. (Optional)
  "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
  {
  }
  "cspAllowedOrigins"?: object // Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by default; list only what the app genuinely needs. Empty = the host's strict default (self only). (Optional)
  "isActive": boolean // Whether this MCP service is currently available for use
  "createdAt": string (timestamp)
```

```
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": "https://example.com/file.jpg",
  "resourceIdentifier": "https://example.com/file.jpg",
  "oauthClientId": "Response string",
  "oauthClientSecret": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": {},
  "cspAllowedOrigins": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Retrieve a Specific Self-hosted MCP Server

GET `/api/v1/mcp/private-registry/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this MCP Registry.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
  "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
```

```
  "oauthServerUrl"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
  "resourceIdentifier"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discovery fails or the metadata is incorrect. (e.g., Asana uses https://mcp.asana.com, Supabase uses https://mcp.supabase.com/mcp) (Optional)
  "oauthClientId"?: string // Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration. (Optional)
  "oauthClientSecret"?: string // Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none). (Optional)
  "logoUrl"?: // May have different types (Optional)
  string (uri) // Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL. (Optional)
  "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
  {
  }
  "cspAllowedOrigins"?: object // Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by default; list only what the app genuinely needs. Empty = the host's strict default (self only). (Optional)
  "isActive": boolean // Whether this MCP service is currently available for use
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": "https://example.com/file.jpg",
  "resourceIdentifier": "https://example.com/file.jpg",
```

```
  "oauthClientId": "Response string",
  "oauthClientSecret": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": {},
  "cspAllowedOrigins": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Partially Update a Self-hosted MCP Server

`PATCH` `/api/v1/mcp/private-registry/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this MCP Registry.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
displayName	string	No	Human-readable display name (defaults to name if empty)
description	string	No	Brief description of the MCP service
remoteMcpUrl	string (uri)	No	The URL of the Remote MCP service

Field	Type	Required	Description
oauthServerUrl	object	No	Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server ad...
resourceIdentifier	object	No	Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discov...
oauthClientId	string	No	Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration.
oauthClientSecret	string	No	Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none).
logoUrl	object	No	Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL.
authType	object	No	How to authenticate with this MCP server. `oauth` : OAuth 2.0 ; `service_account` : Service Account ; `none` : No Authentication;
cspAllowedOrigins	object	No	Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by de...

Example Request Schema

```
{
  "name"?: string // MCP service name (e.g., Supabase, Sentry). Only
alphanumeric, underscore and hyphen allowed. (Optional)
  "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
  "remoteMcpUrl"?: string (uri) // The URL of the Remote MCP service (Optional)
  "oauthServerUrl"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
```

```
  "resourceIdentifier"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discovery fails or the metadata is incorrect. (e.g., Asana uses https://mcp.asana.com, Supabase uses https://mcp.supabase.com/mcp) (Optional)
  "oauthClientId"?: string // Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration. (Optional)
  "oauthClientSecret"?: string // Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none). (Optional)
  "logoUrl"?: // May have different types (Optional)
  string (uri) // Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL. (Optional)
  "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
  {
  }
  "cspAllowedOrigins"?: object // Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by default; list only what the app genuinely needs. Empty = the host's strict default (self only). (Optional)
}
```

Example Request Values

```
{
  "name": "Example name",
  "displayName": "Example string",
  "description": "Example string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": null,
  "resourceIdentifier": null,
  "oauthClientId": "Example string",
  "oauthClientSecret": "Example string",
  "logoUrl": null,
  "authType": {},
  "cspAllowedOrigins": null
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": null,
    "resourceIdentifier": null,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": null,
    "authType": {},
    "cspAllowedOrigins": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": null,
    "resourceIdentifier": null,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": null,
    "authType": {},
    "cspAllowedOrigins": null
  };

axios.patch("https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": None,
    "resourceIdentifier": None,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": None,
    "authType": {},
    "cspAllowedOrigins": None
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'displayName' => 'Example string',
            'description' => 'Example string',
            'remoteMcpUrl' => 'https://example.com/file.jpg',
            'oauthServerUrl' => null,
            'resourceIdentifier' => null,
            'oauthClientId' => 'Example string',
            'oauthClientSecret' => 'Example string',
            'logoUrl' => null,
            'authType' => new \stdClass(),
            'cspAllowedOrigins' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
  "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
  "oauthServerUrl"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
  "resourceIdentifier"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discovery fails or the metadata is incorrect. (e.g., Asana uses https://mcp.asana.com, Supabase uses https://mcp.supabase.com/mcp) (Optional)
  "oauthClientId"?: string // Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration. (Optional)
  "oauthClientSecret"?: string // Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none). (Optional)
  "logoUrl"?: // May have different types (Optional)
  string (uri) // Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL. (Optional)
  "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
  {
  }
  "cspAllowedOrigins"?: object // Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by default; list only what the app genuinely needs. Empty = the host's strict default (self only). (Optional)
  "isActive": boolean // Whether this MCP service is currently available for use
  "createdAt": string (timestamp)
```

```
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": "https://example.com/file.jpg",
  "resourceIdentifier": "https://example.com/file.jpg",
  "oauthClientId": "Response string",
  "oauthClientSecret": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": {},
  "cspAllowedOrigins": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Delete a Self-hosted MCP Server

DELETE `/api/v1/mcp/private-registry/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this MCP Registry.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/mcp/private-
registry/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/mcp/private-registry/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Test MCP Server Connection

POST `/api/v1/mcp/private-registry/test-connection/`

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
displayName	string	No	Human-readable display name (defaults to name if empty)
description	string	No	Brief description of the MCP service
remoteMcpUrl	string (uri)	Yes	The URL of the Remote MCP service
oauthServerUrl	object	No	Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server ad...
resourceIdentifier	object	No	Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discov...
oauthClientId	string	No	Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration.
oauthClientSecret	string	No	Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none).
logoUrl	object	No	Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL.
authType	object	No	How to authenticate with this MCP server. `oauth` : OAuth 2.0 ; `service_account` : Service Account ; `none` : No Authentication;
cspAllowedOrigins	object	No	Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by de...

Example Request Schema

```
{
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
  "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
  "oauthServerUrl"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
  "resourceIdentifier"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discovery fails or the metadata is incorrect. (e.g., Asana uses https://mcp.asana.com, Supabase uses https://mcp.supabase.com/mcp) (Optional)
  "oauthClientId"?: string // Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration. (Optional)
  "oauthClientSecret"?: string // Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none). (Optional)
  "logoUrl"?: // May have different types (Optional)
  string (uri) // Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL. (Optional)
  "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
  {
  }
  "cspAllowedOrigins"?: object // Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by default; list only what the app genuinely needs. Empty = the host's strict default (self only). (Optional)
}
```

Example Request Values

```
{
  "name": "Example name",
  "displayName": "Example string",
```

```
  "description": "Example string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": null,
  "resourceIdentifier": null,
  "oauthClientId": "Example string",
  "oauthClientSecret": "Example string",
  "logoUrl": null,
  "authType": {},
  "cspAllowedOrigins": null
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/mcp/private-registry/test-
connection/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": null,
    "resourceIdentifier": null,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": null,
    "authType": {},
    "cspAllowedOrigins": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": null,
    "resourceIdentifier": null,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": null,
    "authType": {},
    "cspAllowedOrigins": null
  };

axios.post("https://api.maiagent.ai/api/v1/mcp/private-registry/test-
connection/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/private-registry/test-connection/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "displayName": "Example string",
    "description": "Example string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": None,
    "resourceIdentifier": None,
    "oauthClientId": "Example string",
    "oauthClientSecret": "Example string",
    "logoUrl": None,
    "authType": {},
    "cspAllowedOrigins": None
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/mcp/private-registry/test-connection/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'displayName' => 'Example string',
            'description' => 'Example string',
            'remoteMcpUrl' => 'https://example.com/file.jpg',
            'oauthServerUrl' => null,
            'resourceIdentifier' => null,
            'oauthClientId' => 'Example string',
            'oauthClientSecret' => 'Example string',
            'logoUrl' => null,
            'authType' => new \stdClass(),
            'cspAllowedOrigins' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric, underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
  "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
  "oauthServerUrl"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth authorization server URL. Leave empty to auto-discover from the MCP server (RFC 9728 protected resource metadata). Only fill in when auto-discovery fails or the server advertises incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at api.supabase.com) (Optional)
  "resourceIdentifier"?: // May have different types (Optional)
  string (uri) // Manual override for the OAuth resource identifier (RFC 9728 / RFC 8707). Leave empty to auto-discover from the MCP server's .well-known/oauth-protected-resource endpoint. Only fill in when auto-discovery fails or the metadata is incorrect. (e.g., Asana uses https://mcp.asana.com, Supabase uses https://mcp.supabase.com/mcp) (Optional)
  "oauthClientId"?: string // Pre-registered OAuth client ID for MCP servers that do not support dynamic client registration (RFC 7591). Leave empty if the server supports dynamic registration. (Optional)
  "oauthClientSecret"?: string // Pre-registered OAuth client secret. Required only if the MCP server requires a confidential client. Leave empty for public clients (token_endpoint_auth_method=none). (Optional)
  "logoUrl"?: // May have different types (Optional)
  string (uri) // Logo: the uploaded storage object key (media/...) or an external URL; read back as a signed URL. (Optional)
  "authType"?:  // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
  {
  }
  "cspAllowedOrigins"?: object // Extra origins a SEP-1865 MCP App card from this server may load from (script-src/img-src/connect-src etc.), e.g. ['https://viewer.diagrams.net']. The sandboxed iframe denies all external origins by default; list only what the app genuinely needs. Empty = the host's strict default (self only). (Optional)
  "isActive": boolean // Whether this MCP service is currently available for use
  "createdAt": string (timestamp)
```

```
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "displayName": "Response string",
  "description": "Response string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": "https://example.com/file.jpg",
  "resourceIdentifier": "https://example.com/file.jpg",
  "oauthClientId": "Response string",
  "oauthClientSecret": "Response string",
  "logoUrl": "https://example.com/file.jpg",
  "authType": {},
  "cspAllowedOrigins": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List tool execution records

`GET` `/api/v1/tool-execution-records/`

Parameters

Parameter name	Required	Type	Description
`chatbot`	X	string	Filter by AI assistant ID
`endDate`	X	string	End date (inclusive), format: YYYY-MM-DD
`page`	X	integer	A page number within the paginated result set.

`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	Search by AI assistant name, tool name, or message content
`startDate`	X	string	Start date (inclusive), format: YYYY-MM-DD
`status`	X	string	Filter by execution status: success, failure, pending, timeout
`tool`	X	string	Filter by tool ID
`toolType`	X	string	Filter by tool type: api, function, mcp

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/?chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
```

```
      "id": string (uuid)
      "tool": object // Tool information
      "chatbot": object // AI assistant information
      "messageContent": string // Message content
      "status"?: string (enum: success, failure, pending, timeout) // * `success`
- Success
* `failure` - Failed
* `pending` - Processing
* `timeout` - Timeout (Optional)
      "executionTime": integer // Execution time (milliseconds)
      "errorMessage"?: string // Optional
      "retryCount"?: integer // How many times the platform auto-retried this
tool call before the final outcome (0 = the first attempt was the final one). API
tools count tenacity retries inside APIRequestService; MCP tools count transient-
error retries (read-classified tools only). Function tools are never auto-
retried. (Optional)
      "createdAt": string (timestamp)
      "source": string // Scheduled runs' tool calls are attributed via
ChatbotScheduleExecution.user_message.
    }
  ]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tool": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "messageContent": "Response string",
      "status": "success",
      "executionTime": 456,
      "errorMessage": "Response string",
      "retryCount": 456,
      "createdAt": "Response string",
```

```
      "source": "Response string"
    }
  ]
}
```

Retrieve a specific tool execution record

GET `/api/v1/tool-execution-records/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this Tool Execution Record.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response structure

```
{
  "id": string (uuid)
  "tool": object // Tool information, including operated_tool_name
  "chatbot": object // AI assistant information
  "message":
  {
    "id": string (uuid)
    "content": string
```

```
  }
  "inputParameters": object
  "outputResult"?: object // Optional
  "errorMessage"?: string // Optional
  "status"?: string (enum: success, failure, pending, timeout) // * `success` -
Success
* `failure` - Failed
* `pending` - Processing
* `timeout` - Timeout (Optional)
  "executionTime": integer // Execution time (milliseconds)
  "retryCount"?: integer // How many times the platform auto-retried this tool
call before the final outcome (0 = the first attempt was the final one). API
tools count tenacity retries inside APIRequestService; MCP tools count transient-
error retries (read-classified tools only). Function tools are never auto-
retried. (Optional)
  "requestUrl"?: // Can be of different types (Optional)
  string (uri) // Final rendered URL used when calling an API tool. Null for non-
API tools. (Optional)
  "requestMethod"?: string // HTTP method (GET/POST/...) used when calling an API
tool. Null for non-API tools. (Optional)
  "requestHeaders": object // Return rendered request headers with sensitive
values masked.

Three sources are unioned, any hit masks:
``settings.TOOL_API_SENSITIVE_HEADERS``,
APICredential keys for the (contact, tool) pair, and the tool's current
``sensitive_headers``. The tool list is applied at read time against the stored
value, so editing it changes how existing records render without rewriting them.
  "requestBody"?: object // Final rendered body sent to the external API. Null
for non-API tools or read-only methods. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "source": string // Scheduled runs' tool calls are attributed via
ChatbotScheduleExecution.user_message.
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tool": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
```

```
    "description": "Example response description"
  },
  "message": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "content": "Response string"
  },
  "inputParameters": null,
  "outputResult": null,
  "errorMessage": "Response string",
  "status": "success",
  "executionTime": 456,
  "retryCount": 456,
  "requestUrl": "https://example.com/file.jpg",
  "requestMethod": "Response string",
  "requestHeaders": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "requestBody": null,
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "source": "Response string"
}
```


Get AI Assistant Filter Options for Execution Records

`GET` `/api/v1/tool-execution-records/chatbots/`

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-
records/chatbots/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Structure

```
[
  {
    "id"?: string (uuid) // Optional
    "name"?: string // Optional
  }
]
```

Example Response Values


```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
]
```


Get tool filter options for execution records

GET `/api/v1/tool-execution-records/tools/`

Parameters

Parameter name	Required	Type	Description
`chatbot`	X	string	
`endDate`	X	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`startDate`	X	string	
`status`	X	string	`success`: Success ; `failure`: Failed ; `pending`: Processing ; `timeout`: Timeout;
`tool`	X	string	

`toolType`	X	string	`api`: API ; `function`: Function ; `mcp`: MCP ; `desktop`: Desktop ; `flex`: Flex Message ; `quick_reply`: Quick Reply ; `quick_menu`: Quick Menu;

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/tools/?
chatbot=550e8400-e29b-41d4-a716-
446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=examp
le&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/tools/?
chatbot=550e8400-e29b-41d4-a716-
446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=examp
le&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
```

```
      "id": string (uuid)
      "displayName": string
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "displayName": "Response string"
    }
  ]
}
```

List execution records for a specific tool

GET `/api/v1/tools/{toolPk}/tool-execution-records/`

Parameters

Parameter name	Required	Type	Description
`toolPk`	V	string	
`chatbot`	X	string	Filter by AI assistant ID
`endDate`	X	string	End date (inclusive), format: YYYY-MM-DD
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

`query`	X	string	Search by AI assistant name, tool name, or message content
`startDate`	X	string	Start date (inclusive), format: YYYY-MM-DD
`status`	X	string	Filter by execution status: success, failure, pending, timeout
`tool`	X	string	Filter by tool ID
`toolType`	X	string	Filter by tool type: api, function, mcp

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/?chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/?chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
```

```
      "id": string (uuid)
      "tool": object // Tool information
      "chatbot": object // AI assistant information
      "messageContent": string // Message content
      "status"?: string (enum: success, failure, pending, timeout) // * `success`
- Success
* `failure` - Failed
* `pending` - Processing
* `timeout` - Timeout (Optional)
      "executionTime": integer // Execution time (milliseconds)
      "errorMessage"?: string // Optional
      "retryCount"?: integer // How many times the platform auto-retried this
tool call before the final outcome (0 = the first attempt was the final one). API
tools count tenacity retries inside APIRequestService; MCP tools count transient-
error retries (read-classified tools only). Function tools are never auto-
retried. (Optional)
      "createdAt": string (timestamp)
      "source": string // Scheduled runs' tool calls are attributed via
ChatbotScheduleExecution.user_message.
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tool": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "messageContent": "Response string",
      "status": "success",
      "executionTime": 456,
      "errorMessage": "Response string",
      "retryCount": 456,
      "createdAt": "Response string",
```

```
      "source": "Response string"
    }
  ]
}
```


Get a specific execution record for a specific tool

`GET` `/api/v1/tools/{toolPk}/tool-execution-records/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this Tool Execution Record.
`toolPk`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "tool": object // Tool information, including operated_tool_name
  "chatbot": object // AI assistant information
  "message":
  {
    "id": string (uuid)
```

```
    "content": string
  }
  "inputParameters": object
  "outputResult"?: object // Optional
  "errorMessage"?: string // Optional
  "status"?: string (enum: success, failure, pending, timeout) // * `success` -
Success
* `failure` - Failed
* `pending` - Processing
* `timeout` - Timeout (Optional)
  "executionTime": integer // Execution time (milliseconds)
  "retryCount"?: integer // How many times the platform auto-retried this tool
call before the final outcome (0 = the first attempt was the final one). API
tools count tenacity retries inside APIRequestService; MCP tools count transient-
error retries (read-classified tools only). Function tools are never auto-
retried. (Optional)
  "requestUrl"?: // May have different types (Optional)
  string (uri) // Final rendered URL used when calling an API tool. Null for non-
API tools. (Optional)
  "requestMethod"?: string // HTTP method (GET/POST/...) used when calling an API
tool. Null for non-API tools. (Optional)
  "requestHeaders": object // Return rendered request headers with sensitive
values masked.

Three sources are unioned, any hit masks:
``settings.TOOL_API_SENSITIVE_HEADERS``,
APICredential keys for the (contact, tool) pair, and the tool's current
``sensitive_headers``. The tool list is applied at read time against the stored
value, so editing it changes how existing records render without rewriting them.
  "requestBody"?: object // Final rendered body sent to the external API. Null
for non-API tools or read-only methods. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "source": string // Scheduled runs' tool calls are attributed via
ChatbotScheduleExecution.user_message.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tool": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Example response name",
    "description": "Example response description"
  },
  "message": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "content": "Response string"
  },
  "inputParameters": null,
  "outputResult": null,
  "errorMessage": "Response string",
  "status": "success",
  "executionTime": 456,
  "retryCount": 456,
  "requestUrl": "https://example.com/file.jpg",
  "requestMethod": "Response string",
  "requestHeaders": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "requestBody": null,
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "source": "Response string"
}
```


Get AI Assistant Filter Options for Execution Records of a Specific Tool

GET `/api/v1/tools/{toolPk}/tool-execution-records/chatbots/`

Parameters

Parameter Name	Required	Type	Description
`toolPk`	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/tool-execution-records/chatbots/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Response Schema Example

```
[
  {
    "id"?: string (uuid) // Optional
    "name"?: string // Optional
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
]
```

Get Tool Filter Options for a Specific Tool's Execution Records

`GET` `/api/v1/tools/{toolPk}/tool-execution-records/tools/`

Parameters

Parameter Name	Required	Type	Description
`toolPk`	V	string	
`chatbot`	X	string	
`endDate`	X	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`startDate`	X	string	
`status`	X	string	`success`: Success ; `failure`: Failed ; `pending`: Processing ; `timeout`: Timeout;
`tool`	X	string	
`toolType`	X	string	`api`: API ; `function`: Function ; `mcp`: MCP ; `desktop`: Desktop ; `flex`: Flex Message ;

`quick_reply`: Quick Reply ; `quick_menu`: Quick Menu;

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-
446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=examp
le&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
```

```
    {
      "id": string (uuid)
      "displayName": string
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "displayName": "Response string"
    }
  ]
}
```

API Call Example (Shell)

目錄

1. Upload a Skill
2. Create a Skill
3. List Skills
4. Retrieve a Specific Skill
5. Update Skill
6. Partially Update Skill
7. Delete Skill
8. Export Skill
9. Browse Skill Package Directory Structure
10. Reupload Skill
11. List Skill Resources
12. Upload Skill Resource
13. Delete Skill Resource
14. Download Skill Resource

API Call Example (Shell)

Upload a Skill

`POST` `/api/v1/skills/upload/`

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/skills/upload/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -F "file=@example_file.pdf"

  -F "attached_tool_ids=Example string"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with an actual file
formData.append('file',
* Actual file object *
, 'example_file.pdf');
formData.append('attached_tool_ids', 'Example string');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/skills/upload/",
formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/upload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
'application/pdf'),
    'attached_tool_ids': (None, 'Example string')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/skills/upload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ],
            [
                'name' => 'attached_tool_ids',
                'contents' => 'Example string'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
  "attached_tools"?: [ // Names of tools automatically attached based on
the skill package frontmatter (Optional)
    string
  ]
  "missing_tools"?: [ // Names of tools declared in the skill package
that are not found in this organization and must be attached manually
(Optional)
    string
  ]
  "message"?: string // Optional
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "attached_tools": [
    "Response string"
  ],
  "missing_tools": [
    "Response string"
  ],
  "message": "Response string"
}
```

Status Code: 400

Example Response Schema

```
{
  "detail"?: string // Error message (Optional)
}
```

Example Response Values

```
{
  "detail": "Response string"
}
```

Create a Skill

POST `/api/v1/skills/`

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
description	string	Yes	
whenToUse	string	No	Describe when the LLM should trigger this skill. Max 200 characters recommended.
instructions	string	No	Markdown text with instructions for the skill
attachedToolIds	array[string]	No	
groups	array[IdName]	No	Groups this skill is assigned to. Non-owner members must pick at least one

Field	Type	Required	Description
			group they belong to.
files	array[object]	No	Optional resource files to create with the skill. Each item must include path and content. content_encoding may be "text" or "base64".

Example Request Schema

```
{
  "name": string
  "description": string
  "whenToUse"?: string // Describe when the LLM should trigger this skill. Max 200 characters recommended. (Optional)
  "instructions"?: string // Markdown text with instructions for the skill (Optional)
  "attachedToolIds"?: [ // Optional
    string (uuid)
  ]
  "groups"?: [ // Groups this skill is assigned to. Non-owner members must pick at least one group they belong to. (Optional)
    {
      "id": string (uuid)
    }
  ]
  "files"?: [ // Optional resource files to create with the skill. Each item must include path and content. content_encoding may be "text" or "base64". (Optional)
    object
  ]
}
```

Example Request Values

```
{
  "name": "Example name",
  "description": "Example string",
  "whenToUse": "Example string",
  "instructions": "Example string",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
```

```
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "files": [
    {}
  ]
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/skills/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "files": [
      {}
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "Example name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "files": [
      {}
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/skills/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "files": [
        {}
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/skills/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string',
            'whenToUse' => 'Example string',
            'instructions' => 'Example string',
            'attachedToolIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'files' => [
                new \stdClass()
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
  "description": string
  "whenToUse": string // Describe when the LLM should trigger this skill.
Max 200 characters recommended.
  "instructions": string // Markdown text with instructions for the skill
  "compatibility": string // Optional environment requirements from the
Agent Skills Spec frontmatter (max 500 chars).
  "metadata": object // Optional arbitrary key-value mapping from the
Agent Skills Spec frontmatter.
  "allowedTools": object // Optional pre-approved tool list from the
Agent Skills Spec frontmatter (experimental).
  "source":  // How the skill was created: upload (.skill/.zip) or manual

* `upload` - Upload
* `manual` - Manual
  {
  }
  "skillPackagePath": string // S3 path for the uploaded .skill/.zip file
  "skillPackageName": string // Original filename of the uploaded skill
package
  "skillPackageSize": integer // Size of the uploaded skill package in
bytes
  "skillPackageUploadedAt": string (timestamp)
  "attachedTools": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
```

```
      "toolType": string
    }
  ]
  "resources": [
    {
      "id": string (uuid)
      "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
      "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
      "fileSize": integer // File size in bytes
      "mimeType": string
      "createdAt": string (timestamp)
    }
  ]
  "groups": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "lastUsedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "whenToUse": "Response string",
  "instructions": "Response string",
  "compatibility": "Response string",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response string",
  "skillPackageName": "Response string",
```

```
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response string",
  "attachedTools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "displayName": "Response string",
      "toolType": "Response string"
    }
  ],
  "resources": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "path": "Response string",
      "filePath": "Response string",
      "fileSize": 456,
      "mimeType": "Response string",
      "createdAt": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "lastUsedAt": "Response string",
  "canUpdate": false,
  "canDelete": false
}
```


List Skills

GET `/api/v1/skills/`

Parameters

Parameter Name	Required	Type	Description
`isActive`	X	boolean	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`search`	X	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/?
isActive=true&page=1&pageSize=1&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/?
isActive=true&page=1&pageSize=1&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/?
isActive=true&page=1&pageSize=1&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/?
isActive=true&page=1&pageSize=1&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "whenToUse": string // Describe when the LLM should trigger this
skill. Max 200 characters recommended.
      "source":  // How the skill was created: upload (.skill/.zip) or
manual

* `upload` - Upload
* `manual` - Manual
      {
      }
      "attachedTools": [
        {
          "id": string (uuid)
          "name": string
          "description": string
          "displayName": string
          "toolType": string
        }
      ]
      "groups": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "createdBy": object // Show 'System' as the creator for global
skills (uploaded by platform staff).
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
      "lastUsedAt": string (timestamp)
      "canUpdate": boolean // Return whether the current user can update
this resource.
      "canDelete": boolean // Return whether the current user can delete
this resource.
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "whenToUse": "Response string",
      "source": {},
      "attachedTools": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
          "description": "Response string",
          "displayName": "Response string",
          "toolType": "Response string"
        }
      ],
      "groups": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "createdAt": "Response string",
      "updatedAt": "Response string",
      "lastUsedAt": "Response string",
      "canUpdate": false,
      "canDelete": false
    }
  ]
}
```

Retrieve a Specific Skill

GET `/api/v1/skills/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
```

```
  "description": string
  "whenToUse": string // Describe when the LLM should trigger this skill.
Max 200 characters recommended.
  "instructions": string // Markdown text with instructions for the skill
  "compatibility": string // Optional environment requirements from the
Agent Skills Spec frontmatter (max 500 chars).
  "metadata": object // Optional arbitrary key-value mapping from the
Agent Skills Spec frontmatter.
  "allowedTools": object // Optional pre-approved tool list from the
Agent Skills Spec frontmatter (experimental).
  "source":  // How the skill was created: upload (.skill/.zip) or manual

* `upload` - Upload
* `manual` - Manual
  {
  }
  "skillPackagePath": string // S3 path for the uploaded .skill/.zip file
  "skillPackageName": string // Original filename of the uploaded skill
package
  "skillPackageSize": integer // Size of the uploaded skill package in
bytes
  "skillPackageUploadedAt": string (timestamp)
  "attachedTools": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "resources": [
    {
      "id": string (uuid)
      "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
      "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
      "fileSize": integer // File size in bytes
      "mimeType": string
      "createdAt": string (timestamp)
    }
  ]
  "groups": [
    {
```

```
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "lastUsedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "whenToUse": "Response string",
  "instructions": "Response string",
  "compatibility": "Response string",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response string",
  "skillPackageName": "Response string",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response string",
  "attachedTools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "displayName": "Response string",
      "toolType": "Response string"
    }
  ],
  "resources": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "path": "Response string",
      "filePath": "Response string",
      "fileSize": 456,
```

```
      "mimeType": "Response string",
      "createdAt": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "lastUsedAt": "Response string",
  "canUpdate": false,
  "canDelete": false
}
```


Update Skill

PUT `/api/v1/skills/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Skill.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
description	string	Yes	
whenToUse	string	No	Describe when the LLM should trigger this skill. Max 200 characters recommended.
instructions	string	No	Markdown text with instructions for the skill
attachedToolIds	array[string]	No	
groups	array[IdName]	No	Groups this skill is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name": string
  "description": string
  "whenToUse"?: string // Describe when the LLM should trigger this
skill. Max 200 characters recommended. (Optional)
  "instructions"?: string // Markdown text with instructions for the
skill (Optional)
  "attachedToolIds"?: [ // Optional
    string (uuid)
  ]
  "groups"?: [ // Groups this skill is assigned to. Non-owner members
must pick at least one group they belong to. (Optional)
    {
      "id": string (uuid)
    }
  ]
}
```

Example Request Values

```
{
  "name": "Example Name",
  "description": "Example string",
```

```
  "whenToUse": "Example string",
  "instructions": "Example string",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```


Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example Name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "Example Name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  };

axios.put("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example Name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example Name',
            'description' => 'Example string',
            'whenToUse' => 'Example string',
            'instructions' => 'Example string',
            'attachedToolIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "description": string
  "whenToUse": string // Describe when the LLM should trigger this skill.
Max 200 characters recommended.
  "instructions": string // Markdown text with instructions for the skill
  "compatibility": string // Optional environment requirements from the
Agent Skills Spec frontmatter (max 500 chars).
  "metadata": object // Optional arbitrary key-value mapping from the
Agent Skills Spec frontmatter.
  "allowedTools": object // Optional pre-approved tool list from the
Agent Skills Spec frontmatter (experimental).
  "source":  // How the skill was created: upload (.skill/.zip) or manual

* `upload` - Upload
* `manual` - Manual
  {
  }
  "skillPackagePath": string // S3 path for the uploaded .skill/.zip file
  "skillPackageName": string // Original filename of the uploaded skill
package
  "skillPackageSize": integer // Size of the uploaded skill package in
bytes
  "skillPackageUploadedAt": string (timestamp)
  "attachedTools": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "resources": [
```

```
    {
      "id": string (uuid)
      "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
      "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
      "fileSize": integer // File size in bytes
      "mimeType": string
      "createdAt": string (timestamp)
    }
  ]
  "groups": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "lastUsedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "whenToUse": "Response string",
  "instructions": "Response string",
  "compatibility": "Response string",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response string",
  "skillPackageName": "Response string",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response string",
  "attachedTools": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "displayName": "Response string",
      "toolType": "Response string"
    }
  ],
  "resources": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "path": "Response string",
      "filePath": "Response string",
      "fileSize": 456,
      "mimeType": "Response string",
      "createdAt": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "lastUsedAt": "Response string",
  "canUpdate": false,
  "canDelete": false
}
```


Partially Update Skill

PATCH `/api/v1/skills/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Skill.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	
description	string	No	
whenToUse	string	No	Describe when the LLM should trigger this skill. Max 200 characters recommended.
instructions	string	No	Markdown text with instructions for the skill
attachedToolIds	array[string]	No	
groups	array[IdName]	No	Groups this skill is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name"?: string // Optional
  "description"?: string // Optional
  "whenToUse"?: string // Describe when the LLM should trigger this
skill. Max 200 characters recommended. (Optional)
  "instructions"?: string // Markdown text with instructions for the
skill (Optional)
  "attachedToolIds"?: [ // Optional
    string (uuid)
  ]
  "groups"?: [ // Groups this skill is assigned to. Non-owner members
must pick at least one group they belong to. (Optional)
```

```
    {
      "id": string (uuid)
    }
  ]
}
```

Example Request Values

```
{
  "name": "Example Name",
  "description": "Example string",
  "whenToUse": "Example string",
  "instructions": "Example string",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example Name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "Example Name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  };

axios.patch("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example Name",
    "description": "Example string",
    "whenToUse": "Example string",
    "instructions": "Example string",
    "attachedToolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example Name',
            'description' => 'Example string',
            'whenToUse' => 'Example string',
            'instructions' => 'Example string',
            'attachedToolIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "description": string
  "whenToUse": string // Describe when the LLM should trigger this skill.
Max 200 characters recommended.
  "instructions": string // Markdown text with instructions for the skill
  "compatibility": string // Optional environment requirements from the
Agent Skills Spec frontmatter (max 500 chars).
  "metadata": object // Optional arbitrary key-value mapping from the
Agent Skills Spec frontmatter.
  "allowedTools": object // Optional pre-approved tool list from the
Agent Skills Spec frontmatter (experimental).
  "source":  // How the skill was created: upload (.skill/.zip) or manual

* `upload` - Upload
* `manual` - Manual
  {
  }
  "skillPackagePath": string // S3 path for the uploaded .skill/.zip file
  "skillPackageName": string // Original filename of the uploaded skill
package
  "skillPackageSize": integer // Size of the uploaded skill package in
bytes
  "skillPackageUploadedAt": string (timestamp)
  "attachedTools": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "resources": [
```

```
    {
      "id": string (uuid)
      "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
      "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
      "fileSize": integer // File size in bytes
      "mimeType": string
      "createdAt": string (timestamp)
    }
  ]
  "groups": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "lastUsedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "whenToUse": "Response string",
  "instructions": "Response string",
  "compatibility": "Response string",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response string",
  "skillPackageName": "Response string",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response string",
  "attachedTools": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "displayName": "Response string",
      "toolType": "Response string"
    }
  ],
  "resources": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "path": "Response string",
      "filePath": "Response string",
      "fileSize": 456,
      "mimeType": "Response string",
      "createdAt": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "lastUsedAt": "Response string",
  "canUpdate": false,
  "canDelete": false
}
```


Delete Skill

DELETE `/api/v1/skills/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Export Skill

GET `/api/v1/skills/{id}/export/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/export/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/export/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-
446655440000/export/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	.skill file download

Browse Skill Package Directory Structure

GET `/api/v1/skills/{id}/package-structure/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Skill.
`path`	X	string	Directory path to browse (defaults to the root directory)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/package-structure/?path=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/package-structure/?path=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/package-structure/?path=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-
446655440000/package-structure/?path=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "package_name"?: string // Skill package file name (Optional)
  "package_size"?: integer // Skill package size (bytes) (Optional)
```

```
  "uploaded_at"?: string (date-time) // Upload time (Optional)
  "current_path"?: string // Current browsing path (Optional)
  "items"?: [ // Directory contents (directories first, sorted
alphabetically) (Optional)
    {
      "name"?: string // File or directory name (Optional)
      "type"?: string (enum: file, dir) // Item type (Optional)
      "size"?: integer // File size (bytes), returned only when type=file
(Optional)
      "item_count"?: integer // Number of immediate children in the
directory, returned only when type=dir (Optional)
    }
  ]
}
```

Example Response Values


```
{
  "package_name": "response string",
  "package_size": 456,
  "uploaded_at": "2025-01-01T00:00:00.000Z",
  "current_path": "response string",
  "items": [
    {
      "name": "response string",
      "type": "file",
      "size": 456,
      "item_count": 456
    }
  ]
}
```


Status Code: 400

Example Response Schema

```
{
  "detail"?: string // Error message (Optional)
}
```

Example Response Values

```
{
  "detail": "response string"
}
```


Reupload Skill

POST `/api/v1/skills/{id}/reupload/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/reupload/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -F "file=@example_file.pdf"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with the actual file
formData.append('file',
* Actual file object *
, 'example_file.pdf');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-
41d4-a716-446655440000/reupload/", formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/reupload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Prepare files and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
'application/pdf')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
a716-446655440000/reupload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
  "message"?: string // Optional
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "description": "response string",
  "message": "response string"
}
```

Status Code: 400

Example Response Schema

```
{
  "detail"?: string // Error message (Optional)
}
```

Example Response Values

```
{
  "detail": "response string"
}
```

List Skill Resources

GET `/api/v1/skills/{skillPk}/resources/`

Parameters

Parameter Name	Required	Type	Description
`skillPk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources
/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
[
  {
    "id": string (uuid)
    "path": string // Relative path within the skill directory, e.g.
```

```
scripts/helper.py
    "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
    "fileSize": integer // File size in bytes
    "mimeType": string
    "createdAt": string (timestamp)
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "path": "response string",
    "filePath": "response string",
    "fileSize": 456,
    "mimeType": "response string",
    "createdAt": "response string"
  }
]
```

Upload Skill Resource

POST `/api/v1/skills/{skillPk}/resources/`

Parameters

Parameter Name	Required	Type	Description
`skillPk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -F "file=@example_file.pdf"

  -F "path=example string"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with the actual file
formData.append('file',
* Actual file object *
, 'example_file.pdf');
formData.append('path', 'example string');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/skills/{skillPk}/resource
s/", formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Prepare files and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
'application/pdf'),
    'path': (None, 'example string')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ],
            [
                'name' => 'path',
                'contents' => 'example string'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
  "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
  "fileSize": integer // File size in bytes
  "mimeType": string
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "path": "response string",
  "filePath": "response string",
  "fileSize": 456,
  "mimeType": "response string",
  "createdAt": "response string"
}
```

Status Code: 400

Example Response Schema

```
{
  "path"?: string // Path validation error (Optional)
  "file"?: string // File validation error (Optional)
}
```

Example Response Values

```
{
  "path": "response string",
  "file": "response string"
}
```


Delete Skill Resource

DELETE `/api/v1/skills/{skillPk}/resources/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`skillPk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/skills/{skillPk}/resour
ces/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url =
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/5
50e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Download Skill Resource

GET `/api/v1/skills/{skillPk}/resources/{id}/download/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`skillPk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/download/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources
/550e8400-e29b-41d4-a716-446655440000/download/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url =
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/download/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e
8400-e29b-41d4-a716-446655440000/download/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	File download

API call example (Shell)

目錄

1. List Meeting Records
2. Create Meeting Record
3. Retrieve a Specific Meeting Record
4. Partially Update a Meeting Record
5. Delete Meeting Record
6. Import Audio and Start Transcription
7. Retrieve the Meeting Transcript (JSON)
8. List Meeting Summaries
9. Generate a Meeting Summary
10. Retrieve a Specific Meeting Summary
11. Upload Transcript to Knowledge Base
12. Download SRT Subtitle File
13. Download MP3 Audio File

API call example (Shell)

The meeting record endpoints let your system create meetings, import audio files for transcription, retrieve transcripts and summaries, and store transcripts in a knowledge base. Before importing an audio file, upload it using `POST /api/v1/upload-presigned-url/` (`modelName` is `meeting-record` and `fieldName` is `audio`), as described in Attachments and Files. For feature details, see Meeting Records in the user manual.

List Meeting Records

`GET` `/api/v1/meeting-records/`

Parameters

Parameter Name	Required	Type	Description

`endDate`	X		
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X		
`startDate`	X		
`status`	X	string	`pending`: Pending; `recording`: Recording; `processing`: Processing; `completed`: Completed; `failed`: Failed;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/meeting-records/?
endDate=example&page=1&pageSize=1&query=example&startDate=example&st
atus=completed"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/meeting-records/?
endDate=example&page=1&pageSize=1&query=example&startDate=example&st
atus=completed", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/?
endDate=example&page=1&pageSize=1&query=example&startDate=example&st
atus=completed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/meeting-records/?
endDate=example&page=1&pageSize=1&query=example&startDate=example&st
atus=completed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "liveTranscriptionResult": object
      "eventContext"?: object // Optional
      "meetingRecordSttProvider":
      {
        "id": string (uuid)
        "name": string // Display name for this STT provider
        "provider": string (enum: whisperlive, azure, deepgram,
assemblyai, bronci, alibaba, localai, openai, rap) // * `whisperlive` -
Whisperlive
* `azure` - Azure
* `deepgram` - Deepgram
* `assemblyai` - Assemblyai
* `bronci` - Bronci
* `alibaba` - Alibaba
* `localai` - Localai
* `openai` - Openai
* `rap` - Rap
        "modelName": string
      }
      "offlineTranscriptProviderDetail":
      {
        "id": string (uuid)
        "name": string // Display name for this provider
        "provider": string (enum: whisper_openai, whisper_groq,
whisper_offline, localai, azure, bronci, alibaba, rap, deepgram,
assemblyai) // * `whisper_openai` - whisper_openai
* `whisper_groq` - whisper_groq
* `whisper_offline` - whisper_offline
* `localai` - localai
* `azure` - azure
* `bronci` - bronci
* `alibaba` - alibaba
* `rap` - rap
* `deepgram` - deepgram
* `assemblyai` - assemblyai
        "modelName": string // Free-form vendor model identifier; any
string the vendor deployment accepts is valid.
      }
      "meetingRecordSttProviderId"?: string (uuid) // Optional
      "speakerDiarizationProvider":
```

```
      {
        "id": string (uuid)
        "name": string // Display name for this provider
        "provider": string (enum: pyannoteai) // * `pyannoteai` -
pyannoteai
        "modelName": string // Free-form vendor model identifier; any
string the vendor deployment accepts is valid.
      }
      "enableTranslation"?: boolean // Optional
      "targetLanguages"?: [ // Target language codes for live transcript
translation. (Optional)
        string
      ]
      "translationLlmId"?: string (uuid) // Optional
      "knowledgeBaseId"?: string (uuid) // Optional
      "audioUrl": string
      "transcriptUrl": string
      "mp3Url": string
      "backupAudioUrl": string
      "backupSyncStatus": string
      "backupUnavailableReason": string
      "backupError": object
      "backupDurationSeconds": number (double)
      "backupTotalParts": integer
      "backupTotalChunks": integer
      "backupRecordingStartedAt": string (timestamp)
      "backupLastActivityAt": string (timestamp)
      "backupFinalizedAt": string (timestamp)
      "knowledgeBase": object
      "transcriptFile": object
      "qaConfig": object // Generate qa_config for frontend to use with
chatbot completion API.
      "availableChatbots": [ // Get chatbots that have this meeting's
knowledge base (detail view only).
        object
      ]
      "createdBy":
      {
        "id": string (uuid)
        "name": string
      }
      "members": [ // The meeting's access roster, visible to anyone who
can see the meeting.

Editing (add/remove) is gated separately on the members endpoint; here
```

```
the
list is read-only context so members can see who participated and who
else
was granted view access.
        object
      ]
      "myRole": string // The requesting user's role on this meeting, or
None if not a member.
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
      "meetingName": string
      "participants"?: object // List of meeting participants (Optional)
      "durationSeconds"?: number (double) // Optional
      "status"?: string (enum: pending, recording, processing, completed,
failed) // * `pending` - Pending
* `recording` - Recording
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed (Optional)
      "stopRequestedAt": string (timestamp) // When a user explicitly
requested to stop the recording. A finished recording without this set
ended on the server side (e.g. the client connection was lost).
      "recordingLimitReachedAt": string (timestamp) // When LiveKit ended
the egress at its maximum recording duration. The main audio file ends
here; the STT agent and the browser backup keep going until the room
closes, and the meeting completes on room end.
      "failureReason": // May have different types
      string (enum: egress_not_started, egress_failed, finalize_timeout)
// Why the live recording ended as failed. Empty for meetings that did
not fail.

* `` - None
* `egress_not_started` - Recording did not start
* `egress_failed` - Recording failed
* `finalize_timeout` - Recording was never finalized
      "clientSurface":  // Durable client entry point captured when
recording starts. Legacy records remain unknown.

* `unknown` - Unknown
* `web` - Web
* `desktop` - Desktop
      {
      }
      "requestedSourceMode":  // Audio source mode requested by the
client when recording starts.
```

```
* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
      {
      }
      "effectiveSourceMode":  // Server-resolved audio source mode used
for runtime and billing reconciliation.

* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
      {
      }
      "sourceLanguages"?: object // Source languages for transcription
(Optional)
      "speakerDiarizationConfig": object // Per-meeting non-secret
speaker diarization hints, such as numSpeakers.
      "offlineAsrRequestOptions": object // Per-request ASR payload for
the offline audio import, keyed by the selected provider row
request_option_specs contract. EMPTY MEANS: no extra parameters.
      "audioProcessingSnapshot": object // Immutable audio processing
policy snapshot frozen from the platform active profile at meeting start
(via MeetingAudioProcessingProfile.to_snapshot). Downlinked to the
LiveKit Agent with this meeting client/source facts and used to correlate
per-track usage.
      "translationInputTokens"?: integer (int64) // Cumulative LLM input
tokens consumed by realtime translation. Atomically incremented per
translate() call; settled at egress completion. (Optional)
      "translationOutputTokens"?: integer (int64) // Cumulative LLM
output tokens consumed by realtime translation. (Optional)
      "deletedAt"?: string (timestamp) // Optional
      "translationLlm"?: string (uuid) // LLM model to use for
translation when enabled (Optional)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
```

```
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "meeting_name": "Product Planning Meeting",
      "participants": [
        "Zhang San",
        "Li Si",
        "Wang Wu"
      ],
      "duration_seconds": 3600.5,
      "status": "completed",
      "audio_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890.ogg",
      "transcript_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4_transcript.json",
      "mp3_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-
e5f6-7890-abcd-ef1234567890_mp3.mp3",
      "source_languages": [
        "zh-TW"
      ],
      "enable_translation": true,
      "target_languages": [
        "en-US",
        "ja-JP"
      ],
      "meeting_record_stt_provider": {
        "id": "c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f",
        "provider": "deepgram",
        "model_name": "nova-2",
        "supported_languages": [
          "zh-TW",
          "en-US",
          "ja-JP"
        ],
        "config": {
          "punctuate": true,
          "diarize": true
        },
        "created_at": "2025-01-15T10:00:00Z",
        "updated_at": "2025-01-15T10:00:00Z"
      },
      "created_at": "2025-01-15T14:00:00Z",
      "updated_at": "2025-01-15T15:00:00Z"
```

```
    }
  ]
}
```

Create Meeting Record

`POST` `/api/v1/meeting-records/`

Request Body

Request Parameters

Field	Type	Required	Description
eventContext	object	No	
meetingRecordSttProviderId	string (uuid)	No	
enableTranslation	boolean	No	
targetLanguages	array[string]	No	Target language codes for live transcript translation.
translationLlmId	string (uuid)	No	
knowledgeBaseId	string (uuid)	No	
meetingName	string	Yes	
participants	object	No	List of meeting participants
durationSeconds	number (double)	No	
status	string (enum: pending, recording,	No	`pending` : Pending ; `recording` : Recording ; `processing` :

Field	Type	Required	Description
	processing, completed, failed)		Processing ; `completed` : Completed ; `failed` : Failed;
sourceLanguages	object	No	Source languages for transcription
translationInputTokens	integer (int64)	No	Cumulative LLM input tokens consumed by realtime translation. Atomically incremented per translate() call; settled at egress completion.
translationOutputTokens	integer (int64)	No	Cumulative LLM output tokens consumed by realtime translation.
deletedAt	string (timestamp)	No	
translationLlm	string (uuid)	No	LLM model to use for translation when enabled

Example Request Schema

```
{
  "eventContext"?: object // Optional
  "meetingRecordSttProviderId"?: string (uuid) // Optional
  "enableTranslation"?: boolean // Optional
  "targetLanguages"?: [ // Target language codes for live transcript
translation. (Optional)
    string
  ]
  "translationLlmId"?: string (uuid) // Optional
  "knowledgeBaseId"?: string (uuid) // Optional
  "meetingName": string
  "participants"?: object // List of meeting participants (Optional)
  "durationSeconds"?: number (double) // Optional
  "status"?: string (enum: pending, recording, processing, completed,
failed) // * `pending` - Pending
* `recording` - Recording
```

```
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed (Optional)
  "sourceLanguages"?: object // Source languages for transcription
(Optional)
  "translationInputTokens"?: integer (int64) // Cumulative LLM input
tokens consumed by realtime translation. Atomically incremented per
translate() call; settled at egress completion. (Optional)
  "translationOutputTokens"?: integer (int64) // Cumulative LLM output
tokens consumed by realtime translation. (Optional)
  "deletedAt"?: string (timestamp) // Optional
  "translationLlm"?: string (uuid) // LLM model to use for translation
when enabled (Optional)
}
```

Example Request Values

```
{
  "meeting_name": "Product Planning Meeting",
  "participants": [
    "Zhang San",
    "Li Si",
    "Wang Wu"
  ],
  "source_languages": [
    "zh-TW"
  ],
  "enable_translation": true,
  "target_languages": [
    "en-US",
    "ja-JP"
  ]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/meeting-records/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "meeting_name": "Product Planning Meeting",
    "participants": [
      "Zhang San",
      "Li Si",
      "Wang Wu"
    ],
    "source_languages": [
      "zh-TW"
    ],
    "enable_translation": true,
    "target_languages": [
      "en-US",
      "ja-JP"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "meeting_name": "Product Planning Meeting",
    "participants": [
      "Zhang San",
      "Li Si",
      "Wang Wu"
    ],
    "source_languages": [
      "zh-TW"
    ],
    "enable_translation": true,
    "target_languages": [
      "en-US",
      "ja-JP"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/meeting-records/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "meeting_name": "Product Planning Meeting",
    "participants": [
        "Zhang San",
        "Li Si",
        "Wang Wu"
    ],
    "source_languages": [
        "zh-TW"
    ],
    "enable_translation": True,
    "target_languages": [
        "en-US",
        "ja-JP"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/meeting-records/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'meeting_name' => 'Product Planning Meeting',
            'participants' => [
                'Zhang San',
                'Li Si',
                'Wang Wu'
            ],
            'source_languages' => [
                'zh-TW'
            ],
            'enable_translation' => true,
            'target_languages' => [
                'en-US',
                'ja-JP'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "liveTranscriptionResult": object
  "eventContext"?: object // Optional
  "meetingRecordSttProvider":
  {
    "id": string (uuid)
    "name": string // Display name for this STT provider
    "provider": string (enum: whisperlive, azure, deepgram, assemblyai,
bronci, alibaba, localai, openai, rap) // * `whisperlive` - Whisperlive
* `azure` - Azure
* `deepgram` - Deepgram
* `assemblyai` - Assemblyai
* `bronci` - Bronci
* `alibaba` - Alibaba
* `localai` - Localai
* `openai` - Openai
* `rap` - Rap
    "modelName": string
  }
  "offlineTranscriptProviderDetail":
  {
    "id": string (uuid)
    "name": string // Display name for this provider
    "provider": string (enum: whisper_openai, whisper_groq,
whisper_offline, localai, azure, bronci, alibaba, rap, deepgram,
assemblyai) // * `whisper_openai` - whisper_openai
* `whisper_groq` - whisper_groq
* `whisper_offline` - whisper_offline
* `localai` - localai
* `azure` - azure
```

```
* `bronci` - bronci
* `alibaba` - alibaba
* `rap` - rap
* `deepgram` - deepgram
* `assemblyai` - assemblyai
    "modelName": string // Free-form vendor model identifier; any string
the vendor deployment accepts is valid.
  }
  "meetingRecordSttProviderId"?: string (uuid) // Optional
  "speakerDiarizationProvider":
  {
    "id": string (uuid)
    "name": string // Display name for this provider
    "provider": string (enum: pyannoteai) // * `pyannoteai` - pyannoteai
    "modelName": string // Free-form vendor model identifier; any string
the vendor deployment accepts is valid.
  }
  "enableTranslation"?: boolean // Optional
  "targetLanguages"?: [ // Target language codes for live transcript
translation. (Optional)
    string
  ]
  "translationLlmId"?: string (uuid) // Optional
  "knowledgeBaseId"?: string (uuid) // Optional
  "audioUrl": string
  "transcriptUrl": string
  "mp3Url": string
  "backupAudioUrl": string
  "backupSyncStatus": string
  "backupUnavailableReason": string
  "backupError": object
  "backupDurationSeconds": number (double)
  "backupTotalParts": integer
  "backupTotalChunks": integer
  "backupRecordingStartedAt": string (timestamp)
  "backupLastActivityAt": string (timestamp)
  "backupFinalizedAt": string (timestamp)
  "knowledgeBase": object
  "transcriptFile": object
  "qaConfig": object // Generate qa_config for frontend to use with
chatbot completion API.
  "availableChatbots": [ // Get chatbots that have this meeting's
knowledge base (detail view only).
    object
  ]
```

```
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "members": [ // The meeting's access roster, visible to anyone who can
see the meeting.

Editing (add/remove) is gated separately on the members endpoint; here
the
list is read-only context so members can see who participated and who
else
was granted view access.
    object
  ]
  "myRole": string // The requesting user's role on this meeting, or None
if not a member.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "meetingName": string
  "participants"?: object // List of meeting participants (Optional)
  "durationSeconds"?: number (double) // Optional
  "status"?: string (enum: pending, recording, processing, completed,
failed) // * `pending` - Pending
* `recording` - Recording
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed (Optional)
  "stopRequestedAt": string (timestamp) // When a user explicitly
requested to stop the recording. A finished recording without this set
ended on the server side (e.g. the client connection was lost).
  "recordingLimitReachedAt": string (timestamp) // When LiveKit ended the
egress at its maximum recording duration. The main audio file ends here;
the STT agent and the browser backup keep going until the room closes,
and the meeting completes on room end.
  "failureReason": // May have different types
  string (enum: egress_not_started, egress_failed, finalize_timeout) //
Why the live recording ended as failed. Empty for meetings that did not
fail.

* `` - None
* `egress_not_started` - Recording did not start
* `egress_failed` - Recording failed
* `finalize_timeout` - Recording was never finalized
  "clientSurface":  // Durable client entry point captured when recording
```

```
starts. Legacy records remain unknown.

* `unknown` - Unknown
* `web` - Web
* `desktop` - Desktop
  {
  }
  "requestedSourceMode":  // Audio source mode requested by the client
when recording starts.

* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
  {
  }
  "effectiveSourceMode":  // Server-resolved audio source mode used for
runtime and billing reconciliation.

* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
  {
  }
  "sourceLanguages"?: object // Source languages for transcription
(Optional)
  "speakerDiarizationConfig": object // Per-meeting non-secret speaker
diarization hints, such as numSpeakers.
  "offlineAsrRequestOptions": object // Per-request ASR payload for the
offline audio import, keyed by the selected provider row
request_option_specs contract. EMPTY MEANS: no extra parameters.
  "audioProcessingSnapshot": object // Immutable audio processing policy
snapshot frozen from the platform active profile at meeting start (via
MeetingAudioProcessingProfile.to_snapshot). Downlinked to the LiveKit
Agent with this meeting client/source facts and used to correlate per-
track usage.
  "translationInputTokens"?: integer (int64) // Cumulative LLM input
tokens consumed by realtime translation. Atomically incremented per
translate() call; settled at egress completion. (Optional)
  "translationOutputTokens"?: integer (int64) // Cumulative LLM output
tokens consumed by realtime translation. (Optional)
  "deletedAt"?: string (timestamp) // Optional
  "translationLlm"?: string (uuid) // LLM model to use for translation
```

```
when enabled (Optional)
}
```

Response Example Value


```
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "meeting_name": "Product Planning Meeting",
  "participants": [
    "Zhang San",
    "Li Si",
    "Wang Wu"
  ],
  "duration_seconds": 3600.5,
  "status": "completed",
  "audio_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-
e5f6-7890-abcd-ef1234567890.ogg",
  "transcript_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4_transcript.json",
  "mp3_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-e5f6-
7890-abcd-ef1234567890_mp3.mp3",
  "source_languages": [
    "zh-TW"
  ],
  "enable_translation": true,
  "target_languages": [
    "en-US",
    "ja-JP"
  ],
  "meeting_record_stt_provider": {
    "id": "c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f",
    "provider": "deepgram",
    "model_name": "nova-2",
    "supported_languages": [
      "zh-TW",
      "en-US",
      "ja-JP"
    ],
    "config": {
      "punctuate": true,
      "diarize": true
    },
    "created_at": "2025-01-15T10:00:00Z",
    "updated_at": "2025-01-15T10:00:00Z"
```

```
  },
  "created_at": "2025-01-15T14:00:00Z",
  "updated_at": "2025-01-15T15:00:00Z"
}
```

Retrieve a Specific Meeting Record

GET `/api/v1/meeting-records/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Meeting Record.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/meeting-records/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "liveTranscriptionResult": object
```

```
  "eventContext"?: object // Optional
  "meetingRecordSttProvider":
  {
    "id": string (uuid)
    "name": string // Display name for this STT provider
    "provider": string (enum: whisperlive, azure, deepgram, assemblyai,
bronci, alibaba, localai, openai, rap) // * `whisperlive` - Whisperlive
* `azure` - Azure
* `deepgram` - Deepgram
* `assemblyai` - Assemblyai
* `bronci` - Bronci
* `alibaba` - Alibaba
* `localai` - Localai
* `openai` - Openai
* `rap` - Rap
    "modelName": string
  }
  "offlineTranscriptProviderDetail":
  {
    "id": string (uuid)
    "name": string // Display name for this provider
    "provider": string (enum: whisper_openai, whisper_groq,
whisper_offline, localai, azure, bronci, alibaba, rap, deepgram,
assemblyai) // * `whisper_openai` - whisper_openai
* `whisper_groq` - whisper_groq
* `whisper_offline` - whisper_offline
* `localai` - localai
* `azure` - azure
* `bronci` - bronci
* `alibaba` - alibaba
* `rap` - rap
* `deepgram` - deepgram
* `assemblyai` - assemblyai
    "modelName": string // Free-form vendor model identifier; any string
the vendor deployment accepts is valid.
  }
  "meetingRecordSttProviderId"?: string (uuid) // Optional
  "speakerDiarizationProvider":
  {
    "id": string (uuid)
    "name": string // Display name for this provider
    "provider": string (enum: pyannoteai) // * `pyannoteai` - pyannoteai
    "modelName": string // Free-form vendor model identifier; any string
the vendor deployment accepts is valid.
  }
```

```
  "enableTranslation"?: boolean // Optional
  "targetLanguages"?: [ // Target language codes for live transcript
translation. (Optional)
    string
  ]
  "translationLlmId"?: string (uuid) // Optional
  "knowledgeBaseId"?: string (uuid) // Optional
  "audioUrl": string
  "transcriptUrl": string
  "mp3Url": string
  "backupAudioUrl": string
  "backupSyncStatus": string
  "backupUnavailableReason": string
  "backupError": object
  "backupDurationSeconds": number (double)
  "backupTotalParts": integer
  "backupTotalChunks": integer
  "backupRecordingStartedAt": string (timestamp)
  "backupLastActivityAt": string (timestamp)
  "backupFinalizedAt": string (timestamp)
  "knowledgeBase": object
  "transcriptFile": object
  "qaConfig": object // Generate qa_config for frontend to use with
chatbot completion API.
  "availableChatbots": [ // Get chatbots that have this meeting's
knowledge base (detail view only).
    object
  ]
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "members": [ // The meeting's access roster, visible to anyone who can
see the meeting.

Editing (add/remove) is gated separately on the members endpoint; here
the
list is read-only context so members can see who participated and who
else
was granted view access.
    object
  ]
  "myRole": string // The requesting user's role on this meeting, or None
if not a member.
```

```
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "meetingName": string
  "participants"?: object // List of meeting participants (Optional)
  "durationSeconds"?: number (double) // Optional
  "status"?: string (enum: pending, recording, processing, completed,
failed) // * `pending` - Pending
* `recording` - Recording
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed (Optional)
  "stopRequestedAt": string (timestamp) // When a user explicitly
requested to stop the recording. A finished recording without this set
ended on the server side (e.g. the client connection was lost).
  "recordingLimitReachedAt": string (timestamp) // When LiveKit ended the
egress at its maximum recording duration. The main audio file ends here;
the STT agent and the browser backup keep going until the room closes,
and the meeting completes on room end.
  "failureReason": // May have different types
  string (enum: egress_not_started, egress_failed, finalize_timeout) //
Why the live recording ended as failed. Empty for meetings that did not
fail.

* `` - None
* `egress_not_started` - Recording did not start
* `egress_failed` - Recording failed
* `finalize_timeout` - Recording was never finalized
  "clientSurface":  // Durable client entry point captured when recording
starts. Legacy records remain unknown.

* `unknown` - Unknown
* `web` - Web
* `desktop` - Desktop
  {
  }
  "requestedSourceMode":  // Audio source mode requested by the client
when recording starts.

* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
  {
  }
  "effectiveSourceMode":  // Server-resolved audio source mode used for
```

```
runtime and billing reconciliation.

* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
  {
  }
  "sourceLanguages"?: object // Source languages for transcription
(Optional)
  "speakerDiarizationConfig": object // Per-meeting non-secret speaker
diarization hints, such as numSpeakers.
  "offlineAsrRequestOptions": object // Per-request ASR payload for the
offline audio import, keyed by the selected provider row
request_option_specs contract. EMPTY MEANS: no extra parameters.
  "audioProcessingSnapshot": object // Immutable audio processing policy
snapshot frozen from the platform active profile at meeting start (via
MeetingAudioProcessingProfile.to_snapshot). Downlinked to the LiveKit
Agent with this meeting client/source facts and used to correlate per-
track usage.
  "translationInputTokens"?: integer (int64) // Cumulative LLM input
tokens consumed by realtime translation. Atomically incremented per
translate() call; settled at egress completion. (Optional)
  "translationOutputTokens"?: integer (int64) // Cumulative LLM output
tokens consumed by realtime translation. (Optional)
  "deletedAt"?: string (timestamp) // Optional
  "translationLlm"?: string (uuid) // LLM model to use for translation
when enabled (Optional)
}
```

Response Example Value

```
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "meeting_name": "Product Planning Meeting",
  "participants": [
    "Zhang San",
    "Li Si",
    "Wang Wu"
  ],
  "duration_seconds": 3600.5,
  "status": "completed",
  "audio_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-
e5f6-7890-abcd-ef1234567890.ogg",
```

```
  "transcript_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4_transcript.json",
  "mp3_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-e5f6-
7890-abcd-ef1234567890_mp3.mp3",
  "source_languages": [
    "zh-TW"
  ],
  "enable_translation": true,
  "target_languages": [
    "en-US",
    "ja-JP"
  ],
  "meeting_record_stt_provider": {
    "id": "c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f",
    "provider": "deepgram",
    "model_name": "nova-2",
    "supported_languages": [
      "zh-TW",
      "en-US",
      "ja-JP"
    ],
    "config": {
      "punctuate": true,
      "diarize": true
    },
    "created_at": "2025-01-15T10:00:00Z",
    "updated_at": "2025-01-15T10:00:00Z"
  },
  "created_at": "2025-01-15T14:00:00Z",
  "updated_at": "2025-01-15T15:00:00Z"
}
```


Partially Update a Meeting Record

`PATCH` `/api/v1/meeting-records/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Meeting Record.

Request Body

Request Parameters

Field	Type	Required	Description
eventContext	object	No	
meetingRecordSttProviderId	string (uuid)	No	
enableTranslation	boolean	No	
targetLanguages	array[string]	No	Target language codes for live transcript translation.
translationLlmId	string (uuid)	No	
knowledgeBaseId	string (uuid)	No	
meetingName	string	No	
participants	object	No	List of meeting participants
durationSeconds	number (double)	No	
status	string (enum: pending, recording, processing, completed, failed)	No	`pending` : Pending ; `recording` : Recording ; `processing` : Processing ; `completed` : Completed ; `failed` : Failed;

Field	Type	Required	Description
sourceLanguages	object	No	Source languages for transcription
translationInputTokens	integer (int64)	No	Cumulative LLM input tokens consumed by realtime translation. Atomically incremented per translate() call; settled at egress completion.
translationOutputTokens	integer (int64)	No	Cumulative LLM output tokens consumed by realtime translation.
deletedAt	string (timestamp)	No	
translationLlm	string (uuid)	No	LLM model to use for translation when enabled

Request Schema Example

```
{
  "eventContext"?: object // Optional
  "meetingRecordSttProviderId"?: string (uuid) // Optional
  "enableTranslation"?: boolean // Optional
  "targetLanguages"?: [ // Target language codes for live transcript
translation. (Optional)
    string
  ]
  "translationLlmId"?: string (uuid) // Optional
  "knowledgeBaseId"?: string (uuid) // Optional
  "meetingName"?: string // Optional
  "participants"?: object // List of meeting participants (Optional)
  "durationSeconds"?: number (double) // Optional
  "status"?: string (enum: pending, recording, processing, completed,
failed) // * `pending` - Pending
* `recording` - Recording
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed (Optional)
```

```
  "sourceLanguages"?: object // Source languages for transcription
(Optional)
  "translationInputTokens"?: integer (int64) // Cumulative LLM input
tokens consumed by realtime translation. Atomically incremented per
translate() call; settled at egress completion. (Optional)
  "translationOutputTokens"?: integer (int64) // Cumulative LLM output
tokens consumed by realtime translation. (Optional)
  "deletedAt"?: string (timestamp) // Optional
  "translationLlm"?: string (uuid) // LLM model to use for translation
when enabled (Optional)
}
```

Request Example Value

```
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "meeting_name": "Product Planning Meeting",
  "participants": [
    "Zhang San",
    "Li Si",
    "Wang Wu"
  ],
  "duration_seconds": 3600.5,
  "status": "completed",
  "audio_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-
e5f6-7890-abcd-ef1234567890.ogg",
  "transcript_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4_transcript.json",
  "mp3_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-e5f6-
7890-abcd-ef1234567890_mp3.mp3",
  "source_languages": [
    "zh-TW"
  ],
  "enable_translation": true,
  "target_languages": [
    "en-US",
    "ja-JP"
  ],
  "meeting_record_stt_provider": {
    "id": "c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f",
    "provider": "deepgram",
    "model_name": "nova-2",
    "supported_languages": [
      "zh-TW",
```

```
      "en-US",
      "ja-JP"
    ],
    "config": {
      "punctuate": true,
      "diarize": true
    },
    "created_at": "2025-01-15T10:00:00Z",
    "updated_at": "2025-01-15T10:00:00Z"
  },
  "created_at": "2025-01-15T14:00:00Z",
  "updated_at": "2025-01-15T15:00:00Z"
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "meeting_name": "Product Planning Meeting",
    "participants": [
      "Zhang San",
      "Li Si",
      "Wang Wu"
    ],
    "duration_seconds": 3600.5,
    "status": "completed",
    "audio_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890.ogg",
    "transcript_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4_transcript.json",
    "mp3_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890_mp3.mp3",
    "source_languages": [
      "zh-TW"
    ],
    "enable_translation": true,
    "target_languages": [
      "en-US",
      "ja-JP"
    ],
    "meeting_record_stt_provider": {
      "id": "c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f",
      "provider": "deepgram",
      "model_name": "nova-2",
      "supported_languages": [
        "zh-TW",
        "en-US",
        "ja-JP"
```

```
      ],
      "config": {
        "punctuate": true,
        "diarize": true
      },
      "created_at": "2025-01-15T10:00:00Z",
      "updated_at": "2025-01-15T10:00:00Z"
    },
    "created_at": "2025-01-15T14:00:00Z",
    "updated_at": "2025-01-15T15:00:00Z"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "meeting_name": "Product Planning Meeting",
    "participants": [
      "Zhang San",
      "Li Si",
      "Wang Wu"
    ],
    "duration_seconds": 3600.5,
    "status": "completed",
    "audio_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890.ogg",
    "transcript_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4_transcript.json",
    "mp3_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890_mp3.mp3",
    "source_languages": [
      "zh-TW"
    ],
    "enable_translation": true,
    "target_languages": [
      "en-US",
      "ja-JP"
    ],
    "meeting_record_stt_provider": {
      "id": "c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f",
      "provider": "deepgram",
      "model_name": "nova-2",
      "supported_languages": [
```

```
        "zh-TW",
        "en-US",
        "ja-JP"
      ],
      "config": {
        "punctuate": true,
        "diarize": true
      },
      "created_at": "2025-01-15T10:00:00Z",
      "updated_at": "2025-01-15T10:00:00Z"
    },
    "created_at": "2025-01-15T14:00:00Z",
    "updated_at": "2025-01-15T15:00:00Z"
  };

axios.patch("https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "meeting_name": "Product Planning Meeting",
    "participants": [
        "Zhang San",
        "Li Si",
        "Wang Wu"
    ],
    "duration_seconds": 3600.5,
    "status": "completed",
    "audio_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890.ogg",
    "transcript_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4_transcript.json",
    "mp3_url": "https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890_mp3.mp3",
    "source_languages": [
        "zh-TW"
    ],
    "enable_translation": True,
    "target_languages": [
        "en-US",
        "ja-JP"
    ],
    "meeting_record_stt_provider": {
        "id": "c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f",
        "provider": "deepgram",
        "model_name": "nova-2",
        "supported_languages": [
            "zh-TW",
```

```
            "en-US",
            "ja-JP"
        ],
        "config": {
            "punctuate": True,
            "diarize": True
        },
        "created_at": "2025-01-15T10:00:00Z",
        "updated_at": "2025-01-15T10:00:00Z"
    },
    "created_at": "2025-01-15T14:00:00Z",
    "updated_at": "2025-01-15T15:00:00Z"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/meeting-records/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'id' => 'a1b2c3d4-e5f6-7890-abcd-ef1234567890',
            'meeting_name' => 'Product Planning Meeting',
            'participants' => [
                'Zhang San',
                'Li Si',
                'Wang Wu'
            ],
            'duration_seconds' => 3600.5,
            'status' => 'completed',
            'audio_url' => 'https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890.ogg',
            'transcript_url' =>
'https://cdn.example.com/media/meeting-
record/a1b2c3d4_transcript.json',
            'mp3_url' => 'https://cdn.example.com/media/meeting-
record/a1b2c3d4-e5f6-7890-abcd-ef1234567890_mp3.mp3',
            'source_languages' => [
                'zh-TW'
            ],
            'enable_translation' => true,
            'target_languages' => [
                'en-US',
                'ja-JP'
            ],
            'meeting_record_stt_provider' => [
                'id' => 'c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f',
```

```
                'provider' => 'deepgram',
                'model_name' => 'nova-2',
                'supported_languages' => [
                    'zh-TW',
                    'en-US',
                    'ja-JP'
                ],
                'config' => [
                    'punctuate' => true,
                    'diarize' => true
                ],
                'created_at' => '2025-01-15T10:00:00Z',
                'updated_at' => '2025-01-15T10:00:00Z'
            ],
            'created_at' => '2025-01-15T14:00:00Z',
            'updated_at' => '2025-01-15T15:00:00Z'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "liveTranscriptionResult": object
  "eventContext"?: object // Optional
```

```
  "meetingRecordSttProvider":
  {
    "id": string (uuid)
    "name": string // Display name for this STT provider
    "provider": string (enum: whisperlive, azure, deepgram, assemblyai,
bronci, alibaba, localai, openai, rap) // * `whisperlive` - Whisperlive
* `azure` - Azure
* `deepgram` - Deepgram
* `assemblyai` - Assemblyai
* `bronci` - Bronci
* `alibaba` - Alibaba
* `localai` - Localai
* `openai` - Openai
* `rap` - Rap
    "modelName": string
  }
  "offlineTranscriptProviderDetail":
  {
    "id": string (uuid)
    "name": string // Display name for this provider
    "provider": string (enum: whisper_openai, whisper_groq,
whisper_offline, localai, azure, bronci, alibaba, rap, deepgram,
assemblyai) // * `whisper_openai` - whisper_openai
* `whisper_groq` - whisper_groq
* `whisper_offline` - whisper_offline
* `localai` - localai
* `azure` - azure
* `bronci` - bronci
* `alibaba` - alibaba
* `rap` - rap
* `deepgram` - deepgram
* `assemblyai` - assemblyai
    "modelName": string // Free-form vendor model identifier; any string
the vendor deployment accepts is valid.
  }
  "meetingRecordSttProviderId"?: string (uuid) // Optional
  "speakerDiarizationProvider":
  {
    "id": string (uuid)
    "name": string // Display name for this provider
    "provider": string (enum: pyannoteai) // * `pyannoteai` - pyannoteai
    "modelName": string // Free-form vendor model identifier; any string
the vendor deployment accepts is valid.
  }
  "enableTranslation"?: boolean // Optional
```

```
  "targetLanguages"?: [ // Target language codes for live transcript
translation. (Optional)
    string
  ]
  "translationLlmId"?: string (uuid) // Optional
  "knowledgeBaseId"?: string (uuid) // Optional
  "audioUrl": string
  "transcriptUrl": string
  "mp3Url": string
  "backupAudioUrl": string
  "backupSyncStatus": string
  "backupUnavailableReason": string
  "backupError": object
  "backupDurationSeconds": number (double)
  "backupTotalParts": integer
  "backupTotalChunks": integer
  "backupRecordingStartedAt": string (timestamp)
  "backupLastActivityAt": string (timestamp)
  "backupFinalizedAt": string (timestamp)
  "knowledgeBase": object
  "transcriptFile": object
  "qaConfig": object // Generate qa_config for frontend to use with
chatbot completion API.
  "availableChatbots": [ // Get chatbots that have this meeting's
knowledge base (detail view only).
    object
  ]
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "members": [ // The meeting's access roster, visible to anyone who can
see the meeting.

Editing (add/remove) is gated separately on the members endpoint; here
the
list is read-only context so members can see who participated and who
else
was granted view access.
    object
  ]
  "myRole": string // The requesting user's role on this meeting, or None
if not a member.
  "createdAt": string (timestamp)
```

```
  "updatedAt": string (timestamp)
  "meetingName": string
  "participants"?: object // List of meeting participants (Optional)
  "durationSeconds"?: number (double) // Optional
  "status"?: string (enum: pending, recording, processing, completed,
failed) // * `pending` - Pending
* `recording` - Recording
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed (Optional)
  "stopRequestedAt": string (timestamp) // When a user explicitly
requested to stop the recording. A finished recording without this set
ended on the server side (e.g. the client connection was lost).
  "recordingLimitReachedAt": string (timestamp) // When LiveKit ended the
egress at its maximum recording duration. The main audio file ends here;
the STT agent and the browser backup keep going until the room closes,
and the meeting completes on room end.
  "failureReason": // May have different types
  string (enum: egress_not_started, egress_failed, finalize_timeout) //
Why the live recording ended as failed. Empty for meetings that did not
fail.

* `` - None
* `egress_not_started` - Recording did not start
* `egress_failed` - Recording failed
* `finalize_timeout` - Recording was never finalized
  "clientSurface":  // Durable client entry point captured when recording
starts. Legacy records remain unknown.

* `unknown` - Unknown
* `web` - Web
* `desktop` - Desktop
  {
  }
  "requestedSourceMode":  // Audio source mode requested by the client
when recording starts.

* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
  {
  }
  "effectiveSourceMode":  // Server-resolved audio source mode used for
runtime and billing reconciliation.
```

```
* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
  {
  }
  "sourceLanguages"?: object // Source languages for transcription
(Optional)
  "speakerDiarizationConfig": object // Per-meeting non-secret speaker
diarization hints, such as numSpeakers.
  "offlineAsrRequestOptions": object // Per-request ASR payload for the
offline audio import, keyed by the selected provider row
request_option_specs contract. EMPTY MEANS: no extra parameters.
  "audioProcessingSnapshot": object // Immutable audio processing policy
snapshot frozen from the platform active profile at meeting start (via
MeetingAudioProcessingProfile.to_snapshot). Downlinked to the LiveKit
Agent with this meeting client/source facts and used to correlate per-
track usage.
  "translationInputTokens"?: integer (int64) // Cumulative LLM input
tokens consumed by realtime translation. Atomically incremented per
translate() call; settled at egress completion. (Optional)
  "translationOutputTokens"?: integer (int64) // Cumulative LLM output
tokens consumed by realtime translation. (Optional)
  "deletedAt"?: string (timestamp) // Optional
  "translationLlm"?: string (uuid) // LLM model to use for translation
when enabled (Optional)
}
```

Example Response Values

```
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "meeting_name": "Product Planning Meeting",
  "participants": [
    "Zhang San",
    "Li Si",
    "Wang Wu"
  ],
  "duration_seconds": 3600.5,
  "status": "completed",
  "audio_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-
e5f6-7890-abcd-ef1234567890.ogg",
  "transcript_url": "https://cdn.example.com/media/meeting-
```

```
record/a1b2c3d4_transcript.json",
  "mp3_url": "https://cdn.example.com/media/meeting-record/a1b2c3d4-e5f6-
7890-abcd-ef1234567890_mp3.mp3",
  "source_languages": [
    "zh-TW"
  ],
  "enable_translation": true,
  "target_languages": [
    "en-US",
    "ja-JP"
  ],
  "meeting_record_stt_provider": {
    "id": "c8dc9f8a-2b5e-4a2d-8f93-1a2b3c4d5e6f",
    "provider": "deepgram",
    "model_name": "nova-2",
    "supported_languages": [
      "zh-TW",
      "en-US",
      "ja-JP"
    ],
    "config": {
      "punctuate": true,
      "diarize": true
    },
    "created_at": "2025-01-15T10:00:00Z",
    "updated_at": "2025-01-15T10:00:00Z"
  },
  "created_at": "2025-01-15T14:00:00Z",
  "updated_at": "2025-01-15T15:00:00Z"
}
```


Delete Meeting Record

DELETE `/api/v1/meeting-records/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Meeting Record.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/meeting-records/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Import Audio and Start Transcription

`POST` `/api/v1/meeting-records/import-audio/`

Transcribe an uploaded audio file asynchronously; check the status field of the meeting record for transcription progress.

Request Body

Request Parameters

Field	Type	Required	Description
offlineTranscriptProviderId	string (uuid)	Yes	
asrOptions	object	No	Nested ASR selection shared by every ASR entry point: languages (at least one), provider payload keys governed by the provider row request_option_specs contract, and the diarization selection (source ...
meetingName	string	Yes	
audioFilePath	string	Yes	
clientSurface	object	No	Client uploading the audio. Omitting it stays unknown instead of claiming web. `unknown` : Unknown ; `web` : Web ; `desktop` : Desktop;

Example Request Structure

```
{
  "offlineTranscriptProviderId": string (uuid)
  "asrOptions"?: object // Nested ASR selection shared by every ASR entry point: languages (at least one), provider payload keys governed by the
```

```
provider row request_option_specs contract, and the diarization selection
(source / provider_id / options). (Optional)
  "meetingName": string
  "audioFilePath": string
  "clientSurface"?:  // Client uploading the audio. Omitting it stays
unknown instead of claiming web.

* `unknown` - Unknown
* `web` - Web
* `desktop` - Desktop (Optional)
  {
  }
}
```

Example Request Values

```
{
  "meeting_name": "Customer Interview Recording",
  "audio_file_path": "media/chatbot-file/2026/06/customer-interview.m4a",
  "offline_transcript_provider_id": "c8dc9f8a-2b5e-4a2d-8f93-
1a2b3c4d5e6f",
  "asr_options": {
    "languages": [
      "zh-TW"
    ],
    "diarization": {
      "source": "model",
      "provider_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
    }
  }
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/meeting-records/import-
audio/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "meeting_name": "Customer Interview Recording",
    "audio_file_path": "media/chatbot-file/2026/06/customer-
interview.m4a",
    "offline_transcript_provider_id": "c8dc9f8a-2b5e-4a2d-8f93-
1a2b3c4d5e6f",
    "asr_options": {
      "languages": [
        "zh-TW"
      ],
      "diarization": {
        "source": "model",
        "provider_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
      }
    }
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "meeting_name": "Customer Interview Recording",
    "audio_file_path": "media/chatbot-file/2026/06/customer-
interview.m4a",
    "offline_transcript_provider_id": "c8dc9f8a-2b5e-4a2d-8f93-
1a2b3c4d5e6f",
    "asr_options": {
      "languages": [
        "zh-TW"
      ],
      "diarization": {
        "source": "model",
        "provider_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
      }
    }
  };

axios.post("https://api.maiagent.ai/api/v1/meeting-records/import-
audio/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/import-audio/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "meeting_name": "Customer Interview Recording",
    "audio_file_path": "media/chatbot-file/2026/06/customer-
interview.m4a",
    "offline_transcript_provider_id": "c8dc9f8a-2b5e-4a2d-8f93-
1a2b3c4d5e6f",
    "asr_options": {
        "languages": [
            "zh-TW"
        ],
        "diarization": {
            "source": "model",
            "provider_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
        }
    }
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/meeting-records/import-
audio/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'meeting_name' => 'Customer Interview Recording',
            'audio_file_path' => 'media/chatbot-
file/2026/06/customer-interview.m4a',
            'offline_transcript_provider_id' => 'c8dc9f8a-2b5e-4a2d-
8f93-1a2b3c4d5e6f',
            'asr_options' => [
                'languages' => [
                    'zh-TW'
                ],
                'diarization' => [
                    'source' => 'model',
                    'provider_id' => 'a1b2c3d4-e5f6-7890-abcd-
ef1234567890'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 202

Example Response Structure

```
{
  "id": string (uuid)
  "liveTranscriptionResult": object
  "eventContext"?: object // Optional
  "meetingRecordSttProvider":
  {
    "id": string (uuid)
    "name": string // Display name for this STT provider
    "provider": string (enum: whisperlive, azure, deepgram, assemblyai,
bronci, alibaba, localai, openai, rap) // * `whisperlive` - Whisperlive
* `azure` - Azure
* `deepgram` - Deepgram
* `assemblyai` - Assemblyai
* `bronci` - Bronci
* `alibaba` - Alibaba
* `localai` - Localai
* `openai` - Openai
* `rap` - Rap
    "modelName": string
  }
  "offlineTranscriptProviderDetail":
  {
    "id": string (uuid)
    "name": string // Display name for this provider
    "provider": string (enum: whisper_openai, whisper_groq,
whisper_offline, localai, azure, bronci, alibaba, rap, deepgram,
assemblyai) // * `whisper_openai` - whisper_openai
* `whisper_groq` - whisper_groq
* `whisper_offline` - whisper_offline
* `localai` - localai
* `azure` - azure
```

```
* `bronci` - bronci
* `alibaba` - alibaba
* `rap` - rap
* `deepgram` - deepgram
* `assemblyai` - assemblyai
    "modelName": string // Free-form vendor model identifier; any string
the vendor deployment accepts is valid.
  }
  "meetingRecordSttProviderId"?: string (uuid) // Optional
  "speakerDiarizationProvider":
  {
    "id": string (uuid)
    "name": string // Display name for this provider
    "provider": string (enum: pyannoteai) // * `pyannoteai` - pyannoteai
    "modelName": string // Free-form vendor model identifier; any string
the vendor deployment accepts is valid.
  }
  "enableTranslation"?: boolean // Optional
  "targetLanguages"?: [ // Target language codes for live transcript
translation. (Optional)
    string
  ]
  "translationLlmId"?: string (uuid) // Optional
  "knowledgeBaseId"?: string (uuid) // Optional
  "audioUrl": string
  "transcriptUrl": string
  "mp3Url": string
  "backupAudioUrl": string
  "backupSyncStatus": string
  "backupUnavailableReason": string
  "backupError": object
  "backupDurationSeconds": number (double)
  "backupTotalParts": integer
  "backupTotalChunks": integer
  "backupRecordingStartedAt": string (timestamp)
  "backupLastActivityAt": string (timestamp)
  "backupFinalizedAt": string (timestamp)
  "knowledgeBase": object
  "transcriptFile": object
  "qaConfig": object // Generate qa_config for frontend to use with
chatbot completion API.
  "availableChatbots": [ // Get chatbots that have this meeting's
knowledge base (detail view only).
    object
  ]
```

```
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "members": [ // The meeting's access roster, visible to anyone who can
see the meeting.

Editing (add/remove) is gated separately on the members endpoint; here
the
list is read-only context so members can see who participated and who
else
was granted view access.
    object
  ]
  "myRole": string // The requesting user's role on this meeting, or None
if not a member.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "meetingName": string
  "participants"?: object // List of meeting participants (Optional)
  "durationSeconds"?: number (double) // Optional
  "status"?: string (enum: pending, recording, processing, completed,
failed) // * `pending` - Pending
* `recording` - Recording
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed (Optional)
  "stopRequestedAt": string (timestamp) // When a user explicitly
requested to stop the recording. A finished recording without this set
ended on the server side (e.g. the client connection was lost).
  "recordingLimitReachedAt": string (timestamp) // When LiveKit ended the
egress at its maximum recording duration. The main audio file ends here;
the STT agent and the browser backup keep going until the room closes,
and the meeting completes on room end.
  "failureReason": // May have different types
  string (enum: egress_not_started, egress_failed, finalize_timeout) //
Why the live recording ended as failed. Empty for meetings that did not
fail.

* `` - None
* `egress_not_started` - Recording did not start
* `egress_failed` - Recording failed
* `finalize_timeout` - Recording was never finalized
  "clientSurface":  // Durable client entry point captured when recording
```

```
starts. Legacy records remain unknown.

* `unknown` - Unknown
* `web` - Web
* `desktop` - Desktop
  {
  }
  "requestedSourceMode":  // Audio source mode requested by the client
when recording starts.

* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
  {
  }
  "effectiveSourceMode":  // Server-resolved audio source mode used for
runtime and billing reconciliation.

* `unknown` - Unknown
* `microphone` - Microphone
* `system_audio` - System Audio
* `both` - Microphone and System Audio
  {
  }
  "sourceLanguages"?: object // Source languages for transcription
(Optional)
  "speakerDiarizationConfig": object // Per-meeting non-secret speaker
diarization hints, such as numSpeakers.
  "offlineAsrRequestOptions": object // Per-request ASR payload for the
offline audio import, keyed by the selected provider row
request_option_specs contract. EMPTY MEANS: no extra parameters.
  "audioProcessingSnapshot": object // Immutable audio processing policy
snapshot frozen from the platform active profile at meeting start (via
MeetingAudioProcessingProfile.to_snapshot). Downlinked to the LiveKit
Agent with this meeting client/source facts and used to correlate per-
track usage.
  "translationInputTokens"?: integer (int64) // Cumulative LLM input
tokens consumed by realtime translation. Atomically incremented per
translate() call; settled at egress completion. (Optional)
  "translationOutputTokens"?: integer (int64) // Cumulative LLM output
tokens consumed by realtime translation. (Optional)
  "deletedAt"?: string (timestamp) // Optional
  "translationLlm"?: string (uuid) // LLM model to use for translation
```

```
when enabled (Optional)
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "liveTranscriptionResult": null,
  "eventContext": null,
  "meetingRecordSttProvider": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": "whisperlive",
    "modelName": "Response string"
  },
  "offlineTranscriptProviderDetail": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": "whisper_openai",
    "modelName": "Response string"
  },
  "meetingRecordSttProviderId": "550e8400-e29b-41d4-a716-446655440000",
  "speakerDiarizationProvider": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": "pyannoteai",
    "modelName": "Response string"
  },
  "enableTranslation": false,
  "targetLanguages": [
    "Response string"
  ],
  "translationLlmId": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
  "audioUrl": "Response string",
  "transcriptUrl": "Response string",
  "mp3Url": "Response string",
  "backupAudioUrl": "Response string",
  "backupSyncStatus": "Response string",
  "backupUnavailableReason": "Response string",
  "backupError": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
```

```
  },
  "backupDurationSeconds": 456,
  "backupTotalParts": 456,
  "backupTotalChunks": 456,
  "backupRecordingStartedAt": "Response string",
  "backupLastActivityAt": "Response string",
  "backupFinalizedAt": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "transcriptFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "qaConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "availableChatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "members": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "myRole": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "meetingName": "Response string",
  "participants": null,
```

```
  "durationSeconds": 456,
  "status": "pending",
  "stopRequestedAt": "Response string",
  "recordingLimitReachedAt": "Response string",
  "failureReason": "egress_not_started",
  "clientSurface": {},
  "requestedSourceMode": {},
  "effectiveSourceMode": {},
  "sourceLanguages": null,
  "speakerDiarizationConfig": null,
  "offlineAsrRequestOptions": null,
  "audioProcessingSnapshot": null,
  "translationInputTokens": 456,
  "translationOutputTokens": 456,
  "deletedAt": "Response string",
  "translationLlm": "550e8400-e29b-41d4-a716-446655440000"
}
```


Retrieve the Meeting Transcript (JSON)

GET `/api/v1/meeting-records/{id}/transcript/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Meeting Record.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/transcript/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/meeting-records/550e8400-
e29b-41d4-a716-446655440000/transcript/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-41d4-a716-446655440000/transcript/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-
41d4-a716-446655440000/transcript/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
object
```

Example Response Values


```
[
  {
    "id": "9f4a0a46-1c67-45fc-b2d0-35b0f2bb5a2e",
    "type": "final",
    "text": "Let’s first confirm the meeting summary workflow today.",
    "startTime": 0,
    "endTime": 2.5,
    "language": "zh-TW"
  }
]
```


Status Code: 400 - No response body

List Meeting Summaries

GET `/api/v1/meeting-records/{meetingRecordPk}/summaries/`

Parameters

Parameter Name	Required	Type	Description
`meetingRecordPk`	V	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/?page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/?page=1&pageSize=1", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/?page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "content"?: string // Summary content; empty while a background
generation is in progress. (Optional)
      "status":  // Manual create/edit rows stay completed; background
generations walk from pending to processing.

* `pending` - Pending
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed
      {
      }
      "failureReason": string // Why the last save/generation failed, for
both AI-generated and manually-edited summaries.
      "completedSegments": integer
      "totalSegments": integer
      "transcriptOptimization": string (uuid) // Which re-transcribed
pass this summary was generated from. Null means the meeting's own
realtime transcript, which is the only other source a summary can have.
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "d1e2f3a4-b5c6-7890-abcd-ef1234567890",
      "content": "This meeting focused on product development progress,
and the team confirmed the milestone goals for next week...",
      "created_at": "2025-01-15T14:30:00Z",
      "updated_at": "2025-01-15T14:30:00Z"
    }
```

```
  ]
}
```

Generate a Meeting Summary

POST `/api/v1/meeting-records/{meetingRecordPk}/summaries/generate/`

Parameters

Parameter Name	Required	Type	Description
`meetingRecordPk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
transcriptOptimizationId	string (uuid)	No	Optional re-transcribed pass to summarize, from GET /meeting-records/{id}/transcript-optimizations/; it must be completed and belong to this meeting. Omit it to summarize the meeting's own transcript.
chatbotId	string (uuid)	No	Optional summary agent; when set, the agent generates the summary itself, reading the transcript through the transcript-reading tool.

Field	Type	Required	Description
largeLanguageModelId	string (uuid)	No	Optional LLM override for the base (no-agent) mode; mutually exclusive with chatbot_id, whose agent always uses its own configured model.
knowledgeBaseIds	array[string]	No	Optional RBAC-filtered knowledge base ids for the base (no-agent) mode; mutually exclusive with chatbot_id, whose agent always uses its own configured knowledge bases.
customInstructions	string	No	Optional extra instructions appended to the built-in summary prompt.
clientPlatform	object	No	Client platform triggering this generation; stamped on the summary conversation. `web` : Web ; `desktop` : Desktop;

Example Request Schema

```
{
  "transcriptOptimizationId"?: string (uuid) // Optional re-transcribed pass to summarize, from GET /meeting-records/{id}/transcript-optimizations/; it must be completed and belong to this meeting. Omit it to summarize the meeting's own transcript. (Optional)
  "chatbotId"?: string (uuid) // Optional summary agent; when set, the agent generates the summary itself, reading the transcript through the transcript-reading tool. (Optional)
  "largeLanguageModelId"?: string (uuid) // Optional LLM override for the base (no-agent) mode; mutually exclusive with chatbot_id, whose agent always uses its own configured model. (Optional)
  "knowledgeBaseIds"?: [ // Optional RBAC-filtered knowledge base ids for the base (no-agent) mode; mutually exclusive with chatbot_id, whose agent always uses its own configured knowledge bases. (Optional)
    string (uuid)
  ]
  "customInstructions"?: string // Optional extra instructions appended to the built-in summary prompt. (Optional)
```

```
  "clientPlatform"?:  // Client platform triggering this generation;
stamped on the summary conversation.

* `web` - Web
* `desktop` - Desktop (Optional)
  {
  }
}
```

Example Request Values

```
{
  "transcriptOptimizationId": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBaseIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "customInstructions": "Example string",
  "clientPlatform": {}
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/generate/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "transcriptOptimizationId": "550e8400-e29b-41d4-a716-
446655440000",
    "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
    "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "customInstructions": "Example string",
    "clientPlatform": {}
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "transcriptOptimizationId": "550e8400-e29b-41d4-a716-
446655440000",
    "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
    "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "customInstructions": "Example string",
    "clientPlatform": {}
  };

axios.post("https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/generate/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/generate/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "transcriptOptimizationId": "550e8400-e29b-41d4-a716-
446655440000",
    "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
    "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "customInstructions": "Example string",
    "clientPlatform": {}
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/generate/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'transcriptOptimizationId' => '550e8400-e29b-41d4-a716-
446655440000',
            'chatbotId' => '550e8400-e29b-41d4-a716-446655440000',
            'largeLanguageModelId' => '550e8400-e29b-41d4-a716-
446655440000',
            'knowledgeBaseIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'customInstructions' => 'Example string',
            'clientPlatform' => new \stdClass()
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "content"?: string // Summary content; empty while a background
generation is in progress. (Optional)
  "status":  // Manual create/edit rows stay completed; background
generations walk from pending to processing.

* `pending` - Pending
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed
  {
  }
  "failureReason": string // Why the last save/generation failed, for
both AI-generated and manually-edited summaries.
  "completedSegments": integer
  "totalSegments": integer
  "transcriptOptimization": string (uuid) // Which re-transcribed pass
this summary was generated from. Null means the meeting's own realtime
transcript, which is the only other source a summary can have.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "content": "Response string",
  "status": {},
  "failureReason": "Response string",
  "completedSegments": 456,
  "totalSegments": 456,
  "transcriptOptimization": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Status Code: 202

Example Response Schema

```
{
  "id": string (uuid)
  "content"?: string // Summary content; empty while a background
generation is in progress. (Optional)
  "status":  // Manual create/edit rows stay completed; background
generations walk from pending to processing.

* `pending` - Pending
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed
  {
  }
  "failureReason": string // Why the last save/generation failed, for
both AI-generated and manually-edited summaries.
  "completedSegments": integer
  "totalSegments": integer
  "transcriptOptimization": string (uuid) // Which re-transcribed pass
this summary was generated from. Null means the meeting's own realtime
transcript, which is the only other source a summary can have.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "content": "Response string",
  "status": {},
  "failureReason": "Response string",
  "completedSegments": 456,
  "totalSegments": 456,
  "transcriptOptimization": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Retrieve a Specific Meeting Summary

GET `/api/v1/meeting-records/{meetingRecordPk}/summaries/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`meetingRecordPk`	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/meeting-
records/{meetingRecordPk}/summaries/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
```

```
  "content"?: string // Summary content; empty while a background
generation is in progress. (optional)
  "status":  // Manual create/edit rows stay completed; background
generations walk from pending to processing.

* `pending` - Pending
* `processing` - Processing
* `completed` - Completed
* `failed` - Failed
  {
  }
  "failureReason": string // Why the last save/generation failed, for
both AI-generated and manually-edited summaries.
  "completedSegments": integer
  "totalSegments": integer
  "transcriptOptimization": string (uuid) // Which re-transcribed pass
this summary was generated from. Null means the meeting's own realtime
transcript, which is the only other source a summary can have.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response

```
{
  "id": "d1e2f3a4-b5c6-7890-abcd-ef1234567890",
  "content": "This meeting focused on product development progress, and
the team confirmed the milestone targets for next week...",
  "created_at": "2025-01-15T14:30:00Z",
  "updated_at": "2025-01-15T14:30:00Z"
}
```

Upload Transcript to Knowledge Base

POST `/api/v1/meeting-records/{id}/upload-transcript/`

Include knowledge_base_id for the first upload. If the transcript is deleted from the knowledge base, call this endpoint again to re-upload it.

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Meeting Record.

Request Body

Request Parameters

Field	Type	Required	Description
knowledgeBaseId	string (uuid)	No	Knowledge base ID to upload transcript to. If not provided, uses existing knowledge base.

Example Request Schema

```
{
  "knowledgeBaseId"?: string (uuid) // Knowledge base ID to upload transcript to. If not provided, uses existing knowledge base. (optional)
}
```

Example Request

```
{
  "knowledge_base_id": "b1c2d3e4-f5a6-7890-abcd-ef1234567890"
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/upload-transcript/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "knowledge_base_id": "b1c2d3e4-f5a6-7890-abcd-ef1234567890"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "knowledge_base_id": "b1c2d3e4-f5a6-7890-abcd-ef1234567890"
  };

axios.post("https://api.maiagent.ai/api/v1/meeting-records/550e8400-
e29b-41d4-a716-446655440000/upload-transcript/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-
41d4-a716-446655440000/upload-transcript/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "knowledge_base_id": "b1c2d3e4-f5a6-7890-abcd-ef1234567890"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-
41d4-a716-446655440000/upload-transcript/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'knowledge_base_id' => 'b1c2d3e4-f5a6-7890-abcd-
ef1234567890'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
202	No response body

400	No response body
404	No response body

Download SRT Subtitle File

GET `/api/v1/meeting-records/{id}/download-srt/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Meeting Record.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/download-srt/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/meeting-records/550e8400-
e29b-41d4-a716-446655440000/download-srt/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-41d4-a716-446655440000/download-srt/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-
41d4-a716-446655440000/download-srt/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: `200` - SRT subtitle file content

Example Response Schema

```
string (binary)
```

Example Response


```
"1\n00:00:00,000 --> 00:00:05,500\nWelcome to the
meeting\n\n2\n00:00:05,500 --> 00:00:10,200\nLet's begin the discussion"
```


Status Code: 400 - No response body

Download MP3 Audio File

GET `/api/v1/meeting-records/{id}/download-mp3/`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Meeting Record.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/meeting-
records/550e8400-e29b-41d4-a716-446655440000/download-mp3/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/meeting-records/550e8400-
e29b-41d4-a716-446655440000/download-mp3/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-41d4-a716-446655440000/download-mp3/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/meeting-records/550e8400-e29b-
41d4-a716-446655440000/download-mp3/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "downloadUrl": string (uri) // Presigned download URL
  "filename": string // Suggested filename
```

```
  "expiresIn": integer // URL expiration time in seconds
}
```

Example Response

```
{
  "download_url": "https://s3.amazonaws.com/bucket/media/meeting-
record/xxx_mp3.mp3?X-Amz-...",
  "filename": "產品規劃會議_20250115.mp3",
  "expires_in": 3600
}
```

Status Code: `400` - No response body

Example API Call (Shell)

目錄

1. Create a Knowledge Base
2. List All Knowledge Bases
3. Retrieve Details of a Specific Knowledge Base
4. Search the Contents of a Specific Knowledge Base
5. Update a Knowledge Base
6. Partially Update a Knowledge Base
7. Delete a Knowledge Base
8. Check RRF Availability
9. Export Labels to Excel
10. Import Labels from Excel
11. Export Metadata to Excel
12. Import Metadata from Excel
13. Create a New Label
14. List labels in the knowledge base
15. Get details of a specific label
16. Update a label
17. Partially update a label
18. Delete a label
19. Create a metadata field
20. List Metadata Fields
21. Retrieve a Specific Metadata Field
22. Update a Metadata Field
23. Upload Files to the Knowledge Base
24. Update File Metadata
25. Batch Delete Files
26. Batch Reparse Files
27. List Files in a Knowledge Base
28. Retrieve Details of a Specific File
29. Retrieve the Latest File Failure Reason
30. Retrieve File Transcription
31. Update File Information
32. Partially Update a File
33. Delete a File
34. Create a File Translation
35. List File Translations

36. Retrieve a Specific Translation
37. Delete a Translation
38. Download a Translated File
39. List Knowledge Base Documents
40. Update a Document
41. Partially Update a Document
42. Delete a Document
43. Create a new FAQ
44. Update FAQ metadata
45. Batch delete FAQs
46. List all FAQs in a specific knowledge base
47. Retrieve details of a specific FAQ
48. Update FAQ
49. Partially Update FAQ
50. Delete FAQ
51. Export FAQ Excel (or a Blank Template)
52. Import FAQ Excel (Asynchronous)
53. List FAQ Import Jobs
54. Retrieve a Specific FAQ Import Job
55. List Parsers
56. Retrieve a Specific Parser

Example API Call (Shell)

Create a Knowledge Base

`POST` `/api/v1/knowledge-bases/`

Request Body

Request Parameters

Field	Type	Required	Description
embeddingModel	string (uuid)	Yes	
rerankerModel	string (uuid)	Yes	
name	string	Yes	
description	string	No	
numberOfRetrievedChunks	integer	No	Number of reference chunks to retrieve; defaults to 12, with a minimum of 1
enableHyde	boolean	No	Enable HyDE; defaults to False
similarityCutoff	number (double)	No	Similarity threshold; defaults to 0.0, with a range of 0.0-1.0
enableRerank	boolean	No	Whether to enable reranking; defaults to True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license. If not provided, uses system default.
vectorDatabaseType	object	No	Type of vector database: elasticsearch, opensearch, oracle or synxdb. Defaults to the platform-managed engine. `elasticsearch` : Elasticsearch ; `opensearch` : OpenSearch ; `oracle` : Oracle AI Database ...
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For

Field	Type	Required	Description
			Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.
opensearchUsername	string	No	Username for OpenSearch Basic Auth.
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Request Schema Example

```
{
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to retrieve; defaults to 12, with a minimum of 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; defaults to 0.0, with a range of 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking; defaults to True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license. If not provided, uses system default. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch, opensearch, oracle or synxdb. Defaults to the platform-managed engine.
```

```
* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
}
```

Example Request Values

```
{
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example String",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
```

```
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example String",
  "vectorDatabaseApiKey": "Example String",
  "opensearchUsername": "Example String",
  "opensearchPassword": "Example String",
  "oracleUser": "Example String",
  "oraclePassword": "Example String",
  "synxdbUser": "Example String",
  "synxdbPassword": "Example String",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```


Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-bases/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example String",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example String",
    "vectorDatabaseApiKey": "Example String",
    "opensearchUsername": "Example String",
    "opensearchPassword": "Example String",
    "oracleUser": "Example String",
    "oraclePassword": "Example String",
    "synxdbUser": "Example String",
    "synxdbPassword": "Example String",
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example String",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example String",
    "vectorDatabaseApiKey": "Example String",
    "opensearchUsername": "Example String",
    "opensearchPassword": "Example String",
    "oracleUser": "Example String",
    "oraclePassword": "Example String",
    "synxdbUser": "Example String",
    "synxdbPassword": "Example String",
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
```

```
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/", data,
config)
  .then(response => {
    console.log('Response Received Successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request Error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example String",
    "numberOfRetrievedChunks": 123,
    "enableHyde": True,
    "similarityCutoff": 123,
    "enableRerank": True,
    "enableRrf": True,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example String",
    "vectorDatabaseApiKey": "Example String",
    "opensearchUsername": "Example String",
    "opensearchPassword": "Example String",
    "oracleUser": "Example String",
    "oraclePassword": "Example String",
    "synxdbUser": "Example String",
    "synxdbPassword": "Example String",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
response = requests.post(url, json=data, headers=headers)
try:
    print("Response Received Successfully:")
    print(response.json())
except Exception as e:
    print("Request Error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'embeddingModel' => '550e8400-e29b-41d4-a716-
446655440000',
            'rerankerModel' => '550e8400-e29b-41d4-a716-
446655440000',
            'name' => 'Example Name',
            'description' => 'Example String',
            'numberOfRetrievedChunks' => 123,
            'enableHyde' => true,
            'similarityCutoff' => 123,
            'enableRerank' => true,
            'enableRrf' => true,
            'vectorDatabaseType' => new \stdClass(),
            'vectorDatabaseUrl' => 'Example String',
            'vectorDatabaseApiKey' => 'Example String',
            'opensearchUsername' => 'Example String',
            'opensearchPassword' => 'Example String',
            'oracleUser' => 'Example String',
            'oraclePassword' => 'Example String',
            'synxdbUser' => 'Example String',
            'synxdbPassword' => 'Example String',
            'chatbots' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'groups' => [
                [
```

```
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response Received Successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request Error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization"?: string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
retrieve; defaults to 12, with a minimum of 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, with a range of 0.0-1.0 (Optional)
```

```
  "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that the knowledge base can access
(Optional)
    {
      "id": string (uuid)
      "name": string
```

```
    }
  ]
  "filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
  "vectorStorageSize": integer
  "chunksCount": integer
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Response String",
  "vectorDatabaseApiKey": "Response String",
  "opensearchUsername": "Response String",
  "opensearchPassword": "Response String",
  "oracleUser": "Response String",
  "oraclePassword": "Response String",
  "synxdbUser": "Response String",
  "synxdbPassword": "Response String",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "name": "Response String"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "filesCount": 456,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response String",
  "updatedAt": "Response String"
}
```


List All Knowledge Bases

GET `/api/v1/knowledge-bases/`

Parameters

Parameter Name	Required	Type	Description
`embeddingModel`	X	string	

`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`rerankerModel`	X	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set Request Headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000", config)
  .then(response => {
    console.log('Response Received Successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request Error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response Received Successfully:")
    print(response.json())
except Exception as e:
    print("Request Error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response Received Successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request Error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "createdBy":
      {
        "id": string (uuid)
        "name": string
      }
      "organization"?: string (uuid) // Optional
      "embeddingModel": string (uuid)
      "rerankerModel": string (uuid)
      "name": string
      "description"?: string // Optional
      "numberOfRetrievedChunks"?: integer // Number of reference chunks
to retrieve; defaults to 12, with a minimum of 1 (Optional)
      "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
      "similarityCutoff"?: number (double) // Similarity threshold;
defaults to 0.0, with a range of 0.0-1.0 (Optional)
      "enableRerank"?: boolean // Whether to enable reranking; defaults
to True (Optional)
      "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
      "vectorDatabaseType"?:  // Type of vector database: elasticsearch
or opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
      {
      }
      "vectorDatabaseUrl"?: string // Custom vector database endpoint URL
or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
      "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (Optional)
      "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (Optional)
```

```
      "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (Optional)
      "oracleUser"?: string // Oracle database username. (Optional)
      "oraclePassword"?: string // Oracle database password. (Optional)
      "synxdbUser"?: string // SynxDB database username. (Optional)
      "synxdbPassword"?: string // SynxDB database password. (Optional)
      "labels": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "chatbots"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "groups"?: [ // List of groups that the knowledge base can access
(Optional)
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
      "vectorStorageSize": integer
      "chunksCount": integer
      "canUpdate": boolean // Return whether the current user can update
this resource.
      "canDelete": boolean // Return whether the current user can delete
this resource.
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
```

```
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response String"
      },
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
      "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String",
      "numberOfRetrievedChunks": 456,
      "enableHyde": false,
      "similarityCutoff": 456,
      "enableRerank": false,
      "enableRrf": false,
      "vectorDatabaseType": {},
      "vectorDatabaseUrl": "Response String",
      "vectorDatabaseApiKey": "Response String",
      "opensearchUsername": "Response String",
      "opensearchPassword": "Response String",
      "oracleUser": "Response String",
      "oraclePassword": "Response String",
      "synxdbUser": "Response String",
      "synxdbPassword": "Response String",
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String"
        }
      ],
      "chatbots": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String"
        }
      ],
      "groups": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String"
        }
      ],
```

```
      "filesCount": 456,
      "vectorStorageSize": 456,
      "chunksCount": 456,
      "canUpdate": false,
      "canDelete": false,
      "createdAt": "Response String",
      "updatedAt": "Response String"
    }
  ]
}
```


Retrieve Details of a Specific Knowledge Base

GET `/api/v1/knowledge-bases/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```shell
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```javascript
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "createdBy":
```

```
  {
    "id": string (uuid)
    "name": string
  }
  "organization"?: string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
retrieve; defaults to 12, minimum 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, range 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
```

```
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
  "vectorStorageSize": integer
  "chunksCount": integer
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
```

```
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Response string",
  "vectorDatabaseApiKey": "Response string",
  "opensearchUsername": "Response string",
  "opensearchPassword": "Response string",
  "oracleUser": "Response string",
  "oraclePassword": "Response string",
  "synxdbUser": "Response string",
  "synxdbPassword": "Response string",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "filesCount": 456,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Search the Contents of a Specific Knowledge Base

POST `/api/v1/knowledge-bases/{id}/search/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.
`embeddingModel`	X	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`rerankerModel`	X	string	

Request Body

Request Parameters

Field	Type	Required	Description
searchMode	object	No	Search mode: hybrid (default, vector + BM25), semantic (vector only), keyword (BM25 only) `hybrid` : Hybrid (vector + BM25) ; `semantic` : Semantic (vector only) ; `keyword` : Keyword (BM25 only);
queryMetadata	object	No	Query metadata used to control the search scope and method

Example Request Schema

```
{
  "searchMode"?:  // Search mode: hybrid (default, vector + BM25),
semantic (vector only), keyword (BM25 only)

* `hybrid` - Hybrid (vector + BM25)
* `semantic` - Semantic (vector only)
* `keyword` - Keyword (BM25 only) (Optional)
  {
  }
  "queryMetadata"?: object // Query metadata used to control the search
scope and method (Optional)
}
```

Example Request Values

```
{
  "query": "How to use the API"
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/search/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "query": "How to use the API"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "query": "How to use the API"
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/search/?embeddingModel=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-e29b-41d4-a716-446655440000", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/search/?embeddingModel=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "query": "How to use the API"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/search/?embeddingModel=550e8400-e29b-41d4-
a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'query' => 'How to use the API'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "charactersCount": integer
      "hitsCount": integer
      "text": string
      "updatedAt": string (timestamp)
      "filename": string
      "chatbotFile": object // Return the complete ChatbotFile
information, including the file URL and type, to support frontend image
previews
      "knowledgeBaseFile": object // Same as get_chatbot_file, for
backward compatibility
      "pageNumber": integer // Backward-compatible alias of
``page_start``.
      "pageStart": integer
      "pageEnd": integer
      "citationTitle": string // Source title shown on the inline
citation hover card.
      "citationDescription": string // Source description shown on the
inline citation hover card.
      "citationQuote": string // Quoted text snippet shown on the inline
citation hover card.
      "hasImage": boolean // Determine whether an image is present (check
the file type or Markdown images in text)
      "imageUrl": string // Extract the image URL (prefer the file URL;
otherwise extract it from Markdown)
      "displayText": string // Return the full text with Markdown image
markup removed
      "displayTitle": string // Return the display title (fallback:
citation_title -> filename)
      "highlightedText": string // Return text with matched terms wrapped
in ``<mark>`` tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via
``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
      "labels": [
```

```
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Metadata keys and values
defined by the user. (Optional)
      "metadataEnabledForSearch": object // Map each user-defined
metadata key on this node to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is
True *and*
the node carries a value for it. Since this maps over
``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the
node's
knowledge base ``MetadataKey`` current setting, matching the AI Search
indicator.
      "startCharIdx": integer
      "endCharIdx": integer
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "Response string",
      "updatedAt": "Response string",
      "filename": "Response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
```

```
      "knowledgeBaseFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "pageNumber": 456,
      "pageStart": 456,
      "pageEnd": 456,
      "citationTitle": "Response string",
      "citationDescription": "Response string",
      "citationQuote": "Response string",
      "hasImage": false,
      "imageUrl": "Response string",
      "displayText": "Response string",
      "displayTitle": "Response string",
      "highlightedText": "Response string",
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "metadataEnabledForSearch": {
        "key1": "value1",
        "key2": "value2",
        "createdAt": "2025-01-01T00:00:00.000Z"
      },
      "startCharIdx": 456,
      "endCharIdx": 456
    }
  ]
}
```


Update a Knowledge Base

PUT `/api/v1/knowledge-bases/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.

Request Body

Request Parameters

Field	Type	Required	Description
organization	string (uuid)	No	
embeddingModel	string (uuid)	Yes	
rerankerModel	string (uuid)	Yes	
name	string	Yes	
description	string	No	
numberOfRetrievedChunks	integer	No	Number of reference chunks to retrieve; defaults to 12, minimum 1
enableHyde	boolean	No	Enable HyDE; defaults to False
similarityCutoff	number (double)	No	Similarity threshold; defaults to 0.0, range 0.0-1.0
enableRerank	boolean	No	Whether to enable reranking; defaults to True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license.

Field	Type	Required	Description
vectorDatabaseType	object	No	Type of vector database: elasticsearch or opensearch. `elasticsearch` : Elasticsearch ; `opensearch` : OpenSearch ; `oracle` : Oracle AI Database 26ai ; `synxdb` : SynxDB;
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.
opensearchUsername	string	No	Username for OpenSearch Basic Auth.
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	List of groups that can access the knowledge base

Example Request Schema

```
{
  "organization"?: string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
retrieve; defaults to 12, minimum 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, range 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
```

```
  "groups"?: [ // List of groups that can access the knowledge base
(Optional)
    {
      "id": string (uuid)
    }
  ]
}
```

Example Request Values

```
{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "description": "Example string",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example string",
    "vectorDatabaseApiKey": "Example string",
    "opensearchUsername": "Example string",
    "opensearchPassword": "Example string",
    "oracleUser": "Example string",
    "oraclePassword": "Example string",
    "synxdbUser": "Example string",
    "synxdbPassword": "Example string",
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'
```

```
# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "description": "Example string",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example string",
    "vectorDatabaseApiKey": "Example string",
    "opensearchUsername": "Example string",
    "opensearchPassword": "Example string",
    "oracleUser": "Example string",
    "oraclePassword": "Example string",
    "synxdbUser": "Example string",
    "synxdbPassword": "Example string",
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
```

```
    ]
  };

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example String",
    "numberOfRetrievedChunks": 123,
    "enableHyde": True,
    "similarityCutoff": 123,
    "enableRerank": True,
    "enableRrf": True,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example String",
    "vectorDatabaseApiKey": "Example String",
    "opensearchUsername": "Example String",
    "opensearchPassword": "Example String",
    "oracleUser": "Example String",
    "oraclePassword": "Example String",
    "synxdbUser": "Example String",
    "synxdbPassword": "Example String",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
```

```
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'organization' => '550e8400-e29b-41d4-a716-
446655440000',
            'embeddingModel' => '550e8400-e29b-41d4-a716-
446655440000',
            'rerankerModel' => '550e8400-e29b-41d4-a716-
446655440000',
            'name' => 'Example Name',
            'description' => 'Example String',
            'numberOfRetrievedChunks' => 123,
            'enableHyde' => true,
            'similarityCutoff' => 123,
            'enableRerank' => true,
            'enableRrf' => true,
            'vectorDatabaseType' => new \stdClass(),
            'vectorDatabaseUrl' => 'Example String',
            'vectorDatabaseApiKey' => 'Example String',
            'opensearchUsername' => 'Example String',
            'opensearchPassword' => 'Example String',
            'oracleUser' => 'Example String',
            'oraclePassword' => 'Example String',
            'synxdbUser' => 'Example String',
            'synxdbPassword' => 'Example String',
            'chatbots' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
```

```
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization"?: string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
```

```
retrieve; defaults to 12, with a minimum of 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, with a range of 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
```

```
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
  "vectorStorageSize": integer
  "chunksCount": integer
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Response String",
  "vectorDatabaseApiKey": "Response String",
  "opensearchUsername": "Response String",
  "opensearchPassword": "Response String",
  "oracleUser": "Response String",
  "oraclePassword": "Response String",
  "synxdbUser": "Response String",
```

```
  "synxdbPassword": "Response String",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "filesCount": 456,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response String",
  "updatedAt": "Response String"
}
```


Partially Update a Knowledge Base

PATCH `/api/v1/knowledge-bases/{id}`

Parameters

Parameter Name	Required	Type	Description

`id`	V	string	A UUID string identifying this Knowledge Base.

Request Body

Request Parameters

Field	Type	Required	Description
organization	string (uuid)	No	
embeddingModel	string (uuid)	No	
rerankerModel	string (uuid)	No	
name	string	No	
description	string	No	
numberOfRetrievedChunks	integer	No	Number of reference chunks to retrieve; defaults to 12, with a minimum of 1
enableHyde	boolean	No	Enable HyDE; defaults to False
similarityCutoff	number (double)	No	Similarity threshold; defaults to 0.0, with a range of 0.0-1.0
enableRerank	boolean	No	Whether to enable reranking; defaults to True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license.
vectorDatabaseType	object	No	Type of vector database: elasticsearch or opensearch. `elasticsearch` : Elasticsearch ; `opensearch` : OpenSearch ;

Field	Type	Required	Description
			`oracle` : Oracle AI Database 26ai ; `synxdb` : SynxDB;
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.
opensearchUsername	string	No	Username for OpenSearch Basic Auth.
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	List of groups that can access the knowledge base

Example Request Schema

```
{
  "organization"?: string (uuid) // Optional
  "embeddingModel"?: string (uuid) // Optional
  "rerankerModel"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
```

```
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
retrieve; defaults to 12, with a minimum of 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, with a range of 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
(Optional)
    {
      "id": string (uuid)
    }
```

```
  ]
}
```

Example Request Values

```
{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example String",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example String",
  "vectorDatabaseApiKey": "Example String",
  "opensearchUsername": "Example String",
  "opensearchPassword": "Example String",
  "oracleUser": "Example String",
  "oraclePassword": "Example String",
  "synxdbUser": "Example String",
  "synxdbPassword": "Example String",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example String",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example String",
    "vectorDatabaseApiKey": "Example String",
    "opensearchUsername": "Example String",
    "opensearchPassword": "Example String",
    "oracleUser": "Example String",
    "oraclePassword": "Example String",
    "synxdbUser": "Example String",
    "synxdbPassword": "Example String",
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'
```

```
# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example String",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example String",
    "vectorDatabaseApiKey": "Example String",
    "opensearchUsername": "Example String",
    "opensearchPassword": "Example String",
    "oracleUser": "Example String",
    "oraclePassword": "Example String",
    "synxdbUser": "Example String",
    "synxdbPassword": "Example String",
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
```

```
    ]
  };

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example String",
    "numberOfRetrievedChunks": 123,
    "enableHyde": True,
    "similarityCutoff": 123,
    "enableRerank": True,
    "enableRrf": True,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example String",
    "vectorDatabaseApiKey": "Example String",
    "opensearchUsername": "Example String",
    "opensearchPassword": "Example String",
    "oracleUser": "Example String",
    "oraclePassword": "Example String",
    "synxdbUser": "Example String",
    "synxdbPassword": "Example String",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
```

```
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'organization' => '550e8400-e29b-41d4-a716-
446655440000',
            'embeddingModel' => '550e8400-e29b-41d4-a716-
446655440000',
            'rerankerModel' => '550e8400-e29b-41d4-a716-
446655440000',
            'name' => 'Example Name',
            'description' => 'Example String',
            'numberOfRetrievedChunks' => 123,
            'enableHyde' => true,
            'similarityCutoff' => 123,
            'enableRerank' => true,
            'enableRrf' => true,
            'vectorDatabaseType' => new \stdClass(),
            'vectorDatabaseUrl' => 'Example String',
            'vectorDatabaseApiKey' => 'Example String',
            'opensearchUsername' => 'Example String',
            'opensearchPassword' => 'Example String',
            'oracleUser' => 'Example String',
            'oraclePassword' => 'Example String',
            'synxdbUser' => 'Example String',
            'synxdbPassword' => 'Example String',
            'chatbots' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
```

```
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization"?: string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
```

```
retrieve; defaults to 12, with a minimum of 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, with a range of 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
```

```
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
  "vectorStorageSize": integer
  "chunksCount": integer
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Response String",
  "vectorDatabaseApiKey": "Response String",
  "opensearchUsername": "Response String",
  "opensearchPassword": "Response String",
  "oracleUser": "Response String",
  "oraclePassword": "Response String",
  "synxdbUser": "Response String",
```

```
  "synxdbPassword": "Response String",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "filesCount": 456,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response String",
  "updatedAt": "Response String"
}
```


Delete a Knowledge Base

DELETE `/api/v1/knowledge-bases/{id}`

Parameters

Parameter Name	Required	Type	Description

id	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Check RRF Availability

GET `/api/v1/knowledge-bases/{id}/check-rrf-availability/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/check-rrf-availability/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/check-rrf-availability/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/check-rrf-availability/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/check-rrf-availability/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "createdBy":
```

```
  {
    "id": string (uuid)
    "name": string
  }
  "organization"?: string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
retrieve; default: 12, minimum: 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; default: False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; default:
0.0, range: 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking; default: True
(Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
```

```
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
  "vectorStorageSize": integer
  "chunksCount": integer
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
```

```
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Response string",
  "vectorDatabaseApiKey": "Response string",
  "opensearchUsername": "Response string",
  "opensearchPassword": "Response string",
  "oracleUser": "Response string",
  "oraclePassword": "Response string",
  "synxdbUser": "Response string",
  "synxdbPassword": "Response string",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "filesCount": 456,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Export Labels to Excel

GET `/api/v1/knowledge-bases/{id}/export-labels-excel/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/export-labels-excel/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/export-labels-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-labels-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/export-labels-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	Excel file download

Import Labels from Excel

POST `/api/v1/knowledge-bases/{id}/import-labels-excel/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/import-labels-excel/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -F "excelFile=@example_file.txt"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with an actual file
formData.append('excelFile',
* Actual file object *
, 'example_file.txt');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/import-labels-excel/", formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/import-labels-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Prepare files and form data
files = {
    'excelFile': ('example_file.txt', open('example_file.txt',
'rb'), 'text/plain')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/import-labels-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'excelFile',
                'contents' => fopen('example_file.txt', 'r'),
                'filename' => 'example_file.txt'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "success"?: boolean // Optional
  "updated_count"?: integer // Optional
  "total_rows"?: integer // Optional
  "errors"?: [ // Optional
    {
      "row"?: integer // Optional
      "file_id"?: string // Optional
      "error"?: string // Optional
    }
  ]
}
```

Example Response Values

```
{
  "success": false,
  "updated_count": 456,
  "total_rows": 456,
  "errors": [
    {
      "row": 456,
      "file_id": "Response string",
      "error": "Response string"
    }
  ]
}
```

Export Metadata to Excel

GET `/api/v1/knowledge-bases/{id}/export-metadata-excel/`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/export-metadata-excel/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/export-metadata-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-metadata-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/export-metadata-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	Excel file download

Import Metadata from Excel

POST `/api/v1/knowledge-bases/{id}/import-metadata-excel/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/import-metadata-excel/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -F "excelFile=@example_file.txt"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with an actual file
formData.append('excelFile',
* Actual file object *
, 'example_file.txt');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/import-metadata-excel/", formData,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/import-metadata-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Prepare files and form data
files = {
    'excelFile': ('example_file.txt', open('example_file.txt',
'rb'), 'text/plain')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/import-metadata-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'excelFile',
                'contents' => fopen('example_file.txt', 'r'),
                'filename' => 'example_file.txt'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "success"?: boolean // Optional
  "updated_count"?: integer // Optional
  "total_rows"?: integer // Optional
  "errors"?: [ // Optional
    {
      "row"?: integer // Optional
      "file_id"?: string // Optional
      "error"?: string // Optional
    }
  ]
}
```

Example Response Values


```
{
  "success": false,
  "updated_count": 456,
  "total_rows": 456,
  "errors": [
    {
      "row": 456,
      "file_id": "Response string",
      "error": "Response string"
    }
  ]
}
```


Create a New Label

POST `/api/v1/knowledge-bases/{knowledgeBasePk}/labels/`

Parameters

Parameter Name	Required	Type	Description
knowledgeBasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	

Request Schema Example

```
{
  "name": string
}
```

Example Request Values

```
{
  "name": "Example name"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name"
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Example response schema

```
{
  "id": string (uuid)
  "name"?: string // Optional
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
```

List labels in the knowledge base

`GET` `/api/v1/knowledge-bases/{knowledgeBasePk}/labels/`

Parameters

Parameter name	Required	Type	Description
`knowledgeBasePk`	V	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/?page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/?page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name"?: string // Optional
    }
  ]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ]
}
```

Get details of a specific label

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this knowledge base label.

`knowledgeBasePk`	V	string

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
```

```
  "name"?: string // Optional
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
```

Update a label

PUT `/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this knowledge base label.
`knowledgeBasePk`	V	string	

Request body

Request parameters

Field	Type	Required	Description
name	string	Yes	

Example request schema

```
{
  "name": string
}
```

Example request values

```
{
  "name": "Example name"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name"
  };

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "name"?: string // Optional
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
```

Partially update a label

`PATCH` `/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this knowledge base label.
`knowledgeBasePk`	V	string	

Request body

Request parameters

Field	Type	Required	Description
name	string	No	

Example request schema

```
{
  "name"?: string // Optional
}
```

Example request values

```
{
  "name": "Example name"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name"
  };

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "name"?: string // Optional
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
```

Delete a label

DELETE `/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this knowledge base label.
`knowledgeBasePk`	V	string	

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
204	No response body

Create a metadata field

POST `/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/`

Parameters

Parameter name	Required	Type	Description
knowledgeBasePk	V	string	

Request body

Request parameters

Field	Type	Required	Description
keyName	string	Yes	Metadata key name
description	string	No	Description of the purpose of this field
isRequired	boolean	No	Whether this field is required
defaultValue	string	No	Default value for new files
enabledForSearch	boolean	No	Whether to use this field during retrieval

Example request schema

```
{
  "keyName": string // Metadata key name
  "description"?: string // Description of the purpose of this field
(Optional)
  "isRequired"?: boolean // Whether this field is required (Optional)
  "defaultValue"?: string // Default value for new files (Optional)
  "enabledForSearch"?: boolean // Whether to use this field during
```

```
retrieval (Optional)
}
```

Example request values

```
{
  "keyName": "Example string",
  "description": "Example string",
  "isRequired": true,
  "defaultValue": "Example string",
  "enabledForSearch": true
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "keyName": "Example string",
    "description": "Example string",
    "isRequired": true,
    "defaultValue": "Example string",
    "enabledForSearch": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "keyName": "Example string",
    "description": "Example string",
    "isRequired": true,
    "defaultValue": "Example string",
    "enabledForSearch": true
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "keyName": "Example string",
    "description": "Example string",
    "isRequired": True,
    "defaultValue": "Example string",
    "enabledForSearch": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'keyName' => 'Example string',
            'description' => 'Example string',
            'isRequired' => true,
            'defaultValue' => 'Example string',
            'enabledForSearch' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "knowledgeBase":
  {
    "id": string (uuid)
    "name": string
  }
  "keyName": string // Metadata key name
  "keyType": string // Automatically determined from the field name
(default or custom field)
  "description"?: string // Describes the purpose of this field
(Optional)
  "isRequired"?: boolean // Whether this field is required (Optional)
  "defaultValue"?: string // Default value for new files (Optional)
  "enabledForSearch"?: boolean // Whether to use this field during
retrieval (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "keyName": "Response string",
  "keyType": "Response string",
  "description": "Response string",
  "isRequired": false,
  "defaultValue": "Response string",
  "enabledForSearch": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List Metadata Fields

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/`

Parameters

Parameter Name	Required	Type	Description
`knowledgeBasePk`	V	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`pagination`	X	boolean	Set to false to return all keys without pagination (defaults to true).

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/?
page=1&pageSize=1&pagination=true"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/metadata-keys/?
page=1&pageSize=1&pagination=true", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/?page=1&pageSize=1&pagination=true"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/?
page=1&pageSize=1&pagination=true", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "knowledgeBase":
      {
        "id": string (uuid)
        "name": string
      }
      "keyName": string // Metadata key name
      "keyType": string // Automatically determined from the field name
(default or custom field)
      "description"?: string // Describes the purpose of this field
(Optional)
      "isRequired"?: boolean // Whether this field is required (Optional)
      "defaultValue"?: string // Default value for new files (Optional)
      "enabledForSearch"?: boolean // Whether to use this field during
retrieval (Optional)
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "keyName": "Response string",
      "keyType": "Response string",
      "description": "Response string",
      "isRequired": false,
      "defaultValue": "Response string",
      "enabledForSearch": false,
```

```
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```

Retrieve a Specific Metadata Field

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Metadata Key.
`knowledgeBasePk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
```

```
  "knowledgeBase":
  {
    "id": string (uuid)
    "name": string
  }
  "keyName": string // Metadata key name
  "keyType": string // Automatically determined from the field name
(default or custom field)
  "description"?: string // Describes the purpose of this field
(Optional)
  "isRequired"?: boolean // Whether this field is required (Optional)
  "defaultValue"?: string // Default value for new files (Optional)
  "enabledForSearch"?: boolean // Whether to use this field during
retrieval (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "keyName": "Response string",
  "keyType": "Response string",
  "description": "Response string",
  "isRequired": false,
  "defaultValue": "Response string",
  "enabledForSearch": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Update a Metadata Field

PUT `/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Metadata Key.
`knowledgeBasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
keyName	string	Yes	Metadata key name
description	string	No	Describes the purpose of this field
isRequired	boolean	No	Whether this field is required
defaultValue	string	No	Default value for new files
enabledForSearch	boolean	No	Whether to use this field during retrieval

Example Request Schema

```
{
  "keyName": string // Metadata key name
  "description"?: string // Describes the purpose of this field
(Optional)
  "isRequired"?: boolean // Whether this field is required (Optional)
  "defaultValue"?: string // Default value for new files (Optional)
  "enabledForSearch"?: boolean // Whether to use this field during
```

```
retrieval (Optional)
}
```

Example Request Values

```
{
  "keyName": "Example string",
  "description": "Example string",
  "isRequired": true,
  "defaultValue": "Example string",
  "enabledForSearch": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "keyName": "Example string",
    "description": "Example string",
    "isRequired": true,
    "defaultValue": "Example string",
    "enabledForSearch": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "keyName": "Example string",
    "description": "Example string",
    "isRequired": true,
    "defaultValue": "Example string",
    "enabledForSearch": true
  };

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "keyName": "Example string",
    "description": "Example string",
    "isRequired": True,
    "defaultValue": "Example string",
    "enabledForSearch": True
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'keyName' => 'Example string',
            'description' => 'Example string',
            'isRequired' => true,
            'defaultValue' => 'Example string',
            'enabledForSearch' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "knowledgeBase":
  {
    "id": string (uuid)
    "name": string
  }
  "keyName": string // Metadata key name
  "keyType": string // Automatically determined from the field name
(default or custom field)
  "description"?: string // Describes the purpose of this field
(Optional)
  "isRequired"?: boolean // Whether this field is required (Optional)
  "defaultValue"?: string // Default value for new files (Optional)
  "enabledForSearch"?: boolean // Whether to use this field during
retrieval (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "keyName": "Response string",
  "keyType": "Response string",
  "description": "Response string",
  "isRequired": false,
  "defaultValue": "Response string",
  "enabledForSearch": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Upload Files to the Knowledge Base

POST `/api/v1/knowledge-bases/{knowledgeBasePk}/files/`

Parameters

Parameter Name	Required	Type	Description
`knowledgeBasePk`	V	string	
`fileType`	X	string	Filter by file type, such as pdf, docx, txt, xlsx, etc.
`knowledgeBase`	X	string	Knowledge base ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	Search keywords. Supports multiple criteria separated by spaces. Searchable fields: file ID, filename, label name, Metadata value
`status`	X	string	Filter by file status. Available values: initial (initializing), processing (processing), done (completed), deleting (deleting), deleted (deleted), failed (failed) `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Request Body

Request Parameters

Field	Type	Required	Description
files	array[KnowledgeBaseFileCreate]	Yes	Supports batch uploads of multiple files, up to 50 files per request

Example Request Schema

```
{
  "files": [ // Supports batch uploads of multiple files, up to 50 files per request
    {
      "filename": string // Filename
      "file": string (uri) // File to upload
      "parser"?: string (uuid) // Optional
      "labels"?: [ // Optional
        {
          "id": string (uuid)
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "asrOptions"?: object // Per-file canonical transcription options (languages, prompt, diarization). Empty means no per-file options: provider defaults apply. Completely decoupled from user-defined metadata; never indexed for search. (Optional)
    }
  ]
}
```

Example Request Values

```
{
  "files": [
    {
      "filename": "my-file.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "labels": [
        {
          "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
          "name": "my-label"
        }
      ],
```

```
      "rawUserDefineMetadata": {
        "key1": "value1",
        "key2": "value2"
      }
    },
    {
      "filename": "you-can-upload-multiple-files.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
    }
  ]
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "files": [
      {
        "filename": "my-file.pdf",
        "file": "string",
        "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "labels": [
          {
            "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
            "name": "my-label"
          }
        ],
        "rawUserDefineMetadata": {
          "key1": "value1",
          "key2": "value2"
        }
      },
      {
        "filename": "you-can-upload-multiple-files.pdf",
        "file": "string",
        "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "files": [
      {
        "filename": "my-file.pdf",
        "file": "string",
        "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "labels": [
          {
            "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
            "name": "my-label"
          }
        ],
        "rawUserDefineMetadata": {
          "key1": "value1",
          "key2": "value2"
        }
      },
      {
        "filename": "you-can-upload-multiple-files.pdf",
        "file": "string",
        "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
      }
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
```

```
  data, config)
    .then(response => {
      console.log('Response received successfully:');
      console.log(response.data);
    })
    .catch(error => {
      console.error('Request error:');
      console.error(error.response?.data || error.message);
    });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "files": [
        {
            "filename": "my-file.pdf",
            "file": "string",
            "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
            "labels": [
                {
                    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
                    "name": "my-label"
                }
            ],
            "rawUserDefineMetadata": {
                "key1": "value1",
                "key2": "value2"
            }
        },
        {
            "filename": "you-can-upload-multiple-files.pdf",
            "file": "string",
            "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
```

```
        print(response.json())
    except Exception as e:
        print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'files' => [
                [
                    'filename' => 'my-file.pdf',
                    'file' => 'string',
                    'parser' => '3fa85f64-5717-4562-b3fc-
2c963f66afa6',
                    'labels' => [
                        [
                            'id' => '3fa85f64-5717-4562-b3fc-
2c963f66afa6',
                            'name' => 'my-label'
                        ]
                    ],
                    'rawUserDefineMetadata' => [
                        'key1' => 'value1',
                        'key2' => 'value2'
                    ]
                ],
                [
                    'filename' => 'you-can-upload-multiple-
files.pdf',
                    'file' => 'string',
                    'parser' => '3fa85f64-5717-4562-b3fc-
```

```
    2c963f66afa6'
                    ]
                ]
            ]
        ]);

        $data = json_decode($response->getBody(), true);
        echo "Response received successfully:\n";
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error: ' . $e->getMessage();
    }
    ?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // Filename
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?:  // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
```

```
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
          "displayName": string
        }
      ]
      "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
      "chunksCount": integer // Number of chunks/nodes generated from
this file
      "waitingTime": number (double)
      "processingTime": number (double)
      "processingTimeDetails": object
      "previewUrl": string // For presentations (pptx/ppt), returns the
URL of a derived PDF file for preview (displayed in the frontend PDF
viewer); otherwise, None.

Generate URLs with the same CustomizedFileFieldSerializer as the `file`
field to ensure consistent formatting (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
generates incorrectly formatted URLs.
      "createdAt": string (timestamp)
    }
```

```
  ]
}
```

Example Response Values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "fileType": "response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "provider": {},
        "isTimestampSttProvider": false
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "speakerLabels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "originalLabel": "response string",
          "customName": "response string",
          "displayName": "response string"
        }
      ],
      "vectorStorageSize": 456,
      "chunksCount": 456,
```

```
      "waitingTime": 456,
      "processingTime": 456,
      "processingTimeDetails": null,
      "previewUrl": "response string",
      "createdAt": "response string"
    }
  ]
}
```


Update File Metadata

PATCH `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}/update-metadata/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base File.
`knowledgeBasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Request Schema Example

```
{
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
}
```

Example Request Values

```
{
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/update-metadata/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  };

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/update-metadata/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/update-metadata/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": None
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/update-metadata/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'labels' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'rawUserDefineMetadata' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "filename": string // Filename
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?:  // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "speakerLabels": [
    {
      "id": string (uuid)
      "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
      "customName"?: string // User-defined speaker name (e.g., John)
```

```
(Optional)
      "displayName": string
    }
  ]
  "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
  "chunksCount": integer // Number of chunks/nodes generated from this
file
  "waitingTime": number (double)
  "processingTime": number (double)
  "processingTimeDetails": object
  "previewUrl": string // For presentations (pptx/ppt), returns the URL
of a derived PDF file for preview (displayed in the frontend PDF viewer);
otherwise, None.

Generate URLs with the same CustomizedFileFieldSerializer as the `file`
field to ensure consistent formatting (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
generates incorrectly formatted URLs.
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "fileType": "response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "name": "response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "response string",
      "customName": "response string",
      "displayName": "response string"
    }
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "previewUrl": "response string",
  "createdAt": "response string"
}
```


Batch Delete Files

POST `/api/v1/knowledge-bases/{knowledgeBasePk}/files/batch-delete/`

Parameters

Parameter Name	Required	Type	Description
knowledgeBasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
ids	array[string]	Yes	

Request Schema Example

```
{
  "ids": [
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/batch-delete/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/batch-delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'ids' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "queued": [
    string (uuid)
  ]
  "skipped": [
    {
      "id": string (uuid)
      "reason": string (enum: deleting) // * `deleting` - deleting
    }
  ]
}
```

Example Response Values

```
{
  "queued": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "skipped": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "reason": "deleting"
    }
  ]
}
```

Batch Reparse Files

PATCH `/api/v1/knowledge-bases/{knowledgeBasePk}/files/batch-reparse/`

Parameters

Parameter Name	Required	Type	Description
`knowledgeBasePk`	V	string	
`fileType`	X	string	Filter by file type, such as pdf, docx, txt, xlsx, etc.
`knowledgeBase`	X	string	Knowledge base ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	Search keywords; supports multiple criteria separated by spaces. Searchable fields: file ID, filename, label name, Metadata value
`status`	X	string	Filter by file status. Available values: initial (initialized), processing (processing), done (completed), deleting (deleting), deleted (deleted), failed (failed) `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/batch-reparse/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '[]'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = [];

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/batch-reparse/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/batch-reparse/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = []

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/batch-reparse/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => []
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // Filename
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?:  // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
          "displayName": string
        }
```

```
      ]
      "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
      "chunksCount": integer // Number of chunks/nodes generated from
this file
      "waitingTime": number (double)
      "processingTime": number (double)
      "processingTimeDetails": object
      "previewUrl": string // For presentations (pptx/ppt), returns the
URL of a derived PDF for preview (displayed in the frontend PDF viewer);
otherwise, None.

Generate the URL using the same CustomizedFileFieldSerializer as the
`file` field to ensure a consistent format (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
produces an incorrectly formatted URL.
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "fileType": "Response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "provider": {},
        "isTimestampSttProvider": false
```

```
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "speakerLabels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "originalLabel": "Response string",
          "customName": "Response string",
          "displayName": "Response string"
        }
      ],
      "vectorStorageSize": 456,
      "chunksCount": 456,
      "waitingTime": 456,
      "processingTime": 456,
      "processingTimeDetails": null,
      "previewUrl": "Response string",
      "createdAt": "Response string"
    }
  ]
}
```


List Files in a Knowledge Base

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/files/`

Parameters

Parameter Name	Required	Type	Description
`knowledgeBasePk`	V	string	

`fileType`	X	string	Filter by file type, such as pdf, docx, txt, or xlsx
`knowledgeBase`	X	string	Knowledge base ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	Search keywords; supports multiple criteria separated by spaces. Searchable fields: file ID, filename, label name, Metadata value
`status`	X	string	Filter by file status. Available values: initial (initialized), processing (in progress), done (completed), deleting (being deleted), deleted (deleted), failed (failed) `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // Filename
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?:  // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
          "displayName": string
        }
```

```
      ]
      "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
      "chunksCount": integer // Number of chunks/nodes generated from
this file
      "waitingTime": number (double)
      "processingTime": number (double)
      "processingTimeDetails": object
      "previewUrl": string // For presentations (pptx/ppt), returns the
URL of a derived PDF for preview (displayed in the frontend PDF viewer);
otherwise, None.

Generate the URL using the same CustomizedFileFieldSerializer as the
`file` field to ensure a consistent format (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
produces an incorrectly formatted URL.
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "fileType": "Response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "provider": {},
        "isTimestampSttProvider": false
```

```
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "speakerLabels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "originalLabel": "Response string",
          "customName": "Response string",
          "displayName": "Response string"
        }
      ],
      "vectorStorageSize": 456,
      "chunksCount": 456,
      "waitingTime": 456,
      "processingTime": 456,
      "processingTimeDetails": null,
      "previewUrl": "Response string",
      "createdAt": "Response string"
    }
  ]
}
```


Retrieve Details of a Specific File

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter Name	Required	Type	Description

`id`	V	string	A UUID string identifying this Knowledge Base File.
`knowledgeBasePk`	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
```

```
  "filename": string // Filename
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?:  // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "speakerLabels": [
    {
      "id": string (uuid)
      "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
      "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
      "displayName": string
    }
  ]
  "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
  "chunksCount": integer // Number of chunks/nodes generated from this
file
  "waitingTime": number (double)
  "processingTime": number (double)
  "processingTimeDetails": object
```

```
  "previewUrl": string // For presentations (pptx/ppt), returns the URL
of a derived PDF for preview (displayed in the frontend PDF viewer);
otherwise, None.

Generate the URL using the same CustomizedFileFieldSerializer as the
`file` field to ensure a consistent format (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
produces an incorrectly formatted URL.
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "file": "https://example.com/file.jpg",
  "fileType": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "Response string",
      "customName": "Response string",
      "displayName": "Response string"
    }
```

```
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "previewUrl": "Response string",
  "createdAt": "Response string"
}
```


Retrieve the Latest File Failure Reason

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}/latest-failed-reason/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base File.
`knowledgeBasePk`	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/latest-failed-reason/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/latest-failed-reason/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/latest-failed-reason/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/latest-failed-reason/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
```

```
  "errorMessage": string
  "errorCode"?: string // Machine-readable code of a known failure type
(see FAILED_REASON_LOCALIZED_MESSAGES in whizchat/parsers/exceptions.py);
the serializer translates it into the request language on read. An empty
string means no translation, and error_message is shown as is. (Optional)
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "errorMessage": "Response string",
  "errorCode": "Response string",
  "createdAt": "Response string"
}
```

Retrieve File Transcription

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}/transcription/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base File.
`knowledgeBasePk`	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/transcription/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/transcription/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/transcription/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/transcription/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
```

```
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?:  // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "speakerLabels": [
    {
      "id": string (uuid)
      "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
      "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
      "displayName": string
    }
  ]
  "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
  "chunksCount": integer // Number of chunks/nodes generated from this
file
  "waitingTime": number (double)
  "processingTime": number (double)
  "processingTimeDetails": object
```

```
  "previewUrl": string // For presentations (pptx/ppt), returns the URL
of a derived PDF file for preview (displayed using the frontend PDF
viewer); otherwise, None.

Generate the URL using the same CustomizedFileFieldSerializer as the
`file` field to ensure consistent formatting (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
generates incorrectly formatted URLs.
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "fileType": "response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "response string",
      "customName": "response string",
      "displayName": "response string"
    }
```

```
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "previewUrl": "response string",
  "createdAt": "response string"
}
```


Update File Information

PUT `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base File.
`knowledgeBasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
filename	string	Yes	File name
file	string (uri)	Yes	File to upload

Field	Type	Required	Description
parser	string (uuid)	No	
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	
asrOptions	object	No	Per-file canonical transcription options (languages, prompt, diarization). Empty means no per-file options: provider defaults apply. Completely decoupled from user-defined metadata; never indexed for ...

Example Request Schema

```
{
  "filename": string // File name
  "file": string (uri) // File to upload
  "parser"?: string (uuid) // Optional
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "asrOptions"?: object // Per-file canonical transcription options
(languages, prompt, diarization). Empty means no per-file options:
provider defaults apply. Completely decoupled from user-defined metadata;
never indexed for search. (Optional)
}
```

Example Request Values

```
{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
```

```
    }
  ],
  "rawUserDefineMetadata": null,
  "asrOptions": null
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null,
    "asrOptions": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null,
    "asrOptions": null
  };

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": None,
    "asrOptions": None
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'filename' => 'document.pdf',
            'file' => 'https://example.com/file.jpg',
            'parser' => '550e8400-e29b-41d4-a716-446655440000',
            'labels' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'rawUserDefineMetadata' => null,
            'asrOptions' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?:  // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "speakerLabels": [
    {
      "id": string (uuid)
      "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
```

```
      "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
      "displayName": string
    }
  ]
  "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
  "chunksCount": integer // Number of chunks/nodes generated from this
file
  "waitingTime": number (double)
  "processingTime": number (double)
  "processingTimeDetails": object
  "previewUrl": string // For presentations (pptx/ppt), returns the URL
of a derived PDF file for preview (displayed using the frontend PDF
viewer); otherwise, None.

Generate the URL using the same CustomizedFileFieldSerializer as the
`file` field to ensure consistent formatting (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
generates incorrectly formatted URLs.
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "fileType": "response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "response string",
      "customName": "response string",
      "displayName": "response string"
    }
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "previewUrl": "response string",
  "createdAt": "response string"
}
```


Partially Update a File

PATCH `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base File.
`knowledgeBasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
filename	string	No	File name
file	string (uri)	No	File to upload
parser	string (uuid)	No	
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	
asrOptions	object	No	Per-file canonical transcription options (languages, prompt, diarization). Empty means no per-file options: provider defaults apply. Completely decoupled from user-defined metadata; never indexed for ...

Example Request Schema

```
{
  "filename"?: string // File name (Optional)
  "file"?: string (uri) // File to upload (Optional)
  "parser"?: string (uuid) // Optional
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "asrOptions"?: object // Per-file canonical transcription options
(languages, prompt, diarization). Empty means no per-file options:
provider defaults apply. Completely decoupled from user-defined metadata;
never indexed for search. (Optional)
}
```

Example Request Values

```
{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "asrOptions": null
}
```


Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null,
    "asrOptions": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null,
    "asrOptions": null
  };

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": None,
    "asrOptions": None
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'filename' => 'document.pdf',
            'file' => 'https://example.com/file.jpg',
            'parser' => '550e8400-e29b-41d4-a716-446655440000',
            'labels' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'rawUserDefineMetadata' => null,
            'asrOptions' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?:  // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "speakerLabels": [
    {
      "id": string (uuid)
      "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
```

```
      "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
      "displayName": string
    }
  ]
  "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
  "chunksCount": integer // Number of chunks/nodes generated from this
file
  "waitingTime": number (double)
  "processingTime": number (double)
  "processingTimeDetails": object
  "previewUrl": string // For presentations (pptx/ppt), returns the URL
of a derived PDF file for preview (displayed using the frontend PDF
viewer); otherwise, None.

Generate the URL using the same CustomizedFileFieldSerializer as the
`file` field to ensure consistent formatting (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
generates incorrectly formatted URLs.
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "fileType": "response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "provider": {},
    "isTimestampSttProvider": false
  },
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "response string",
      "customName": "response string",
      "displayName": "response string"
    }
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "previewUrl": "response string",
  "createdAt": "response string"
}
```


Delete a File

DELETE `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base File.
`knowledgeBasePk`	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 202

Example Response Schema

```
{
  "detail": string
```

```
}
```

Example Response Values

```
{
  "detail": "response string"
}
```

Status Code: 204 - No response body

Create a File Translation

POST `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/`

Parameters

Parameter name	Required	Type	Description
`knowledgeBaseFilePk`	V	string	
`knowledgeBasePk`	V	string	

Request body

Request parameters

Field	Type	Required	Description
sourceLanguage	string	No	
targetLanguage	string	Yes	

Field	Type	Required	Description
customPrompt	string	No	
largeLanguageModelId	string (uuid)	Yes	

Example request schema

```
{
  "sourceLanguage"?: string // Optional
  "targetLanguage": string
  "customPrompt"?: string // Optional
  "largeLanguageModelId": string (uuid)
}
```

Example request values

```
{
  "sourceLanguage": "Example string",
  "targetLanguage": "Example string",
  "customPrompt": "Example string",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "sourceLanguage": "Example string",
    "targetLanguage": "Example string",
    "customPrompt": "Example string",
    "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "sourceLanguage": "Example string",
    "targetLanguage": "Example string",
    "customPrompt": "Example string",
    "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "sourceLanguage": "Example string",
    "targetLanguage": "Example string",
    "customPrompt": "Example string",
    "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'sourceLanguage' => 'Example string',
            'targetLanguage' => 'Example string',
            'customPrompt' => 'Example string',
            'largeLanguageModelId' => '550e8400-e29b-41d4-a716-
446655440000'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Example response schema

```
{
  "sourceLanguage"?: string // Optional
  "targetLanguage": string
  "customPrompt"?: string // Optional
  "largeLanguageModelId": string (uuid)
}
```

Example response values

```
{
  "sourceLanguage": "Response string",
  "targetLanguage": "Response string",
  "customPrompt": "Response string",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}
```

List File Translations

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/`

Parameters

Parameter name	Required	Type	Description
`knowledgeBaseFilePk`	V	string	
`knowledgeBasePk`	V	string	

`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "chatbotFileId": string (uuid)
      "chatbotFileFilename": string
      "organization": string (uuid)
      "largeLanguageModel":
      {
        "id": string (uuid)
        "name": string
      }
      "sourceLanguage": string
      "targetLanguage": string
      "customPrompt": string // User-provided terminology/glossary rules
      "status":
      {
      }
      "errorMessage": string
      "translatedFileUrl": string
      "bilingualFileUrl": string
      "processDuration": string
      "processingDurationSeconds": number (double)
      "pageCount": integer
      "originalWordCount": integer
      "translatedWordCount": integer
      "inputTokensUsed": integer
      "outputTokensUsed": integer
      "totalTokensUsed": integer
      "originalFileSize": integer
      "translatedFileSize": integer
      "bilingualFileSize": integer
      "llmTranslationDuration": string // Total time spent on LLM API
calls
      "llmTranslationDurationSeconds": number (double)
      "llmCallCount": integer // Number of LLM API calls made
      "avgLlmCallDuration": number (double) // Average time per LLM call
in seconds
      "pdfParsingDuration": string // Time spent on PDF parsing and
processing
      "pdfParsingDurationSeconds": number (double)
      "fileSavingDuration": string // Time spent on saving output files
      "fileSavingDurationSeconds": number (double)
      "startedAt": string (timestamp)
```

```
      "completedAt": string (timestamp)
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileFilename": "Response string",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "largeLanguageModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "sourceLanguage": "Response string",
      "targetLanguage": "Response string",
      "customPrompt": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "translatedFileUrl": "Response string",
      "bilingualFileUrl": "Response string",
      "processDuration": "Response string",
      "processingDurationSeconds": 456,
      "pageCount": 456,
      "originalWordCount": 456,
      "translatedWordCount": 456,
      "inputTokensUsed": 456,
      "outputTokensUsed": 456,
      "totalTokensUsed": 456,
      "originalFileSize": 456,
      "translatedFileSize": 456,
      "bilingualFileSize": 456,
      "llmTranslationDuration": "Response string",
      "llmTranslationDurationSeconds": 456,
      "llmCallCount": 456,
```

```
      "avgLlmCallDuration": 456,
      "pdfParsingDuration": "Response string",
      "pdfParsingDurationSeconds": 456,
      "fileSavingDuration": "Response string",
      "fileSavingDurationSeconds": 456,
      "startedAt": "Response string",
      "completedAt": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Retrieve a Specific Translation

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this File Translation.
`knowledgeBaseFilePk`	V	string	
`knowledgeBasePk`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
```

```
  "chatbotFileId": string (uuid)
  "chatbotFileFilename": string
  "organization": string (uuid)
  "largeLanguageModel":
  {
    "id": string (uuid)
    "name": string
  }
  "sourceLanguage": string
  "targetLanguage": string
  "customPrompt": string // User-provided terminology/glossary rules
  "status":
  {
  }
  "errorMessage": string
  "translatedFileUrl": string
  "bilingualFileUrl": string
  "processDuration": string
  "processingDurationSeconds": number (double)
  "pageCount": integer
  "originalWordCount": integer
  "translatedWordCount": integer
  "inputTokensUsed": integer
  "outputTokensUsed": integer
  "totalTokensUsed": integer
  "originalFileSize": integer
  "translatedFileSize": integer
  "bilingualFileSize": integer
  "llmTranslationDuration": string // Total time spent on LLM API calls
  "llmTranslationDurationSeconds": number (double)
  "llmCallCount": integer // Number of LLM API calls made
  "avgLlmCallDuration": number (double) // Average time per LLM call in
seconds
  "pdfParsingDuration": string // Time spent on PDF parsing and
processing
  "pdfParsingDurationSeconds": number (double)
  "fileSavingDuration": string // Time spent on saving output files
  "fileSavingDurationSeconds": number (double)
  "startedAt": string (timestamp)
  "completedAt": string (timestamp)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileFilename": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "sourceLanguage": "Response string",
  "targetLanguage": "Response string",
  "customPrompt": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "translatedFileUrl": "Response string",
  "bilingualFileUrl": "Response string",
  "processDuration": "Response string",
  "processingDurationSeconds": 456,
  "pageCount": 456,
  "originalWordCount": 456,
  "translatedWordCount": 456,
  "inputTokensUsed": 456,
  "outputTokensUsed": 456,
  "totalTokensUsed": 456,
  "originalFileSize": 456,
  "translatedFileSize": 456,
  "bilingualFileSize": 456,
  "llmTranslationDuration": "Response string",
  "llmTranslationDurationSeconds": 456,
  "llmCallCount": 456,
  "avgLlmCallDuration": 456,
  "pdfParsingDuration": "Response string",
  "pdfParsingDurationSeconds": 456,
  "fileSavingDuration": "Response string",
  "fileSavingDurationSeconds": 456,
  "startedAt": "Response string",
  "completedAt": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Delete a Translation

DELETE `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this File Translation.
`knowledgeBaseFilePk`	V	string	
`knowledgeBasePk`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
204	No response body

Download a Translated File

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}/download/`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this File Translation.
`knowledgeBaseFilePk`	V	string	
`knowledgeBasePk`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/download/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/download/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/download/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/download/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "id": string (uuid)
```

```
  "chatbotFileId": string (uuid)
  "chatbotFileFilename": string
  "organization": string (uuid)
  "largeLanguageModel":
  {
    "id": string (uuid)
    "name": string
  }
  "sourceLanguage": string
  "targetLanguage": string
  "customPrompt": string // User-provided terminology/glossary rules
  "status":
  {
  }
  "errorMessage": string
  "translatedFileUrl": string
  "bilingualFileUrl": string
  "processDuration": string
  "processingDurationSeconds": number (double)
  "pageCount": integer
  "originalWordCount": integer
  "translatedWordCount": integer
  "inputTokensUsed": integer
  "outputTokensUsed": integer
  "totalTokensUsed": integer
  "originalFileSize": integer
  "translatedFileSize": integer
  "bilingualFileSize": integer
  "llmTranslationDuration": string // Total time spent on LLM API calls
  "llmTranslationDurationSeconds": number (double)
  "llmCallCount": integer // Number of LLM API calls made
  "avgLlmCallDuration": number (double) // Average time per LLM call in
seconds
  "pdfParsingDuration": string // Time spent on PDF parsing and
processing
  "pdfParsingDurationSeconds": number (double)
  "fileSavingDuration": string // Time spent on saving output files
  "fileSavingDurationSeconds": number (double)
  "startedAt": string (timestamp)
  "completedAt": string (timestamp)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileFilename": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "sourceLanguage": "Response string",
  "targetLanguage": "Response string",
  "customPrompt": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "translatedFileUrl": "Response string",
  "bilingualFileUrl": "Response string",
  "processDuration": "Response string",
  "processingDurationSeconds": 456,
  "pageCount": 456,
  "originalWordCount": 456,
  "translatedWordCount": 456,
  "inputTokensUsed": 456,
  "outputTokensUsed": 456,
  "totalTokensUsed": 456,
  "originalFileSize": 456,
  "translatedFileSize": 456,
  "bilingualFileSize": 456,
  "llmTranslationDuration": "Response string",
  "llmTranslationDurationSeconds": 456,
  "llmCallCount": 456,
  "avgLlmCallDuration": 456,
  "pdfParsingDuration": "Response string",
  "pdfParsingDurationSeconds": 456,
  "fileSavingDuration": "Response string",
  "fileSavingDurationSeconds": 456,
  "startedAt": "Response string",
  "completedAt": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List Knowledge Base Documents

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/documents/`

Parameters

Parameter Name	Required	Type	Description
`knowledgeBasePk`	V	string	
`chatbotFileId`	X	string	[To be deprecated] File ID (use knowledge_base_file_id)
`knowledgeBaseFileId`	X	string	Knowledge Base File ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/?
chatbotFileId=example&knowledgeBaseFileId=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/documents/?
chatbotFileId=example&knowledgeBaseFileId=example", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/documents/?
chatbotFileId=example&knowledgeBaseFileId=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/documents/?
chatbotFileId=example&knowledgeBaseFileId=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
[
  {
```

```
    "id": string (uuid)
    "fileName": string
    "pageNumber"?: integer // Optional
    "chatbotFile": string (uuid)
    "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (Optional)
    "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (Optional)
    "llamaIndexObjectJson"?: object // Optional
    "metadata"?: object // Optional
    "text": string
    "updatedAt": string (timestamp)
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "fileName": "Response string",
    "pageNumber": 456,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "Response string",
    "updatedAt": "Response string"
  }
]
```

Status Code: `404` - The knowledge base file does not exist or does not belong to the current organization

Update a Document

PUT `/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Document.
`knowledgeBasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
fileName	string	Yes	
pageNumber	integer	No	
chatbotFile	string (uuid)	Yes	
excludedEmbedMetadataKeys	object	No	List[str]: metadata keys to exclude from the embedding.
excludedLlmMetadataKeys	object	No	List[str]: metadata keys to exclude from the LLM context.
llamaIndexObjectJson	object	No	
metadata	object	No	
text	string	Yes	

Example Request Schema

```
{
  "fileName": string
  "pageNumber"?: integer // Optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (Optional)
  "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (Optional)
  "llamaIndexObjectJson"?: object // Optional
  "metadata"?: object // Optional
  "text": string
}
```

Example Request Values

```
{
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "This is an example message"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "This is an example message"
  };

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": None,
    "excludedLlmMetadataKeys": None,
    "llamaIndexObjectJson": None,
    "metadata": None,
    "text": "This is an example message"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'fileName' => 'Example string',
            'pageNumber' => 123,
            'chatbotFile' => '550e8400-e29b-41d4-a716-446655440000',
            'excludedEmbedMetadataKeys' => null,
            'excludedLlmMetadataKeys' => null,
            'llamaIndexObjectJson' => null,
            'metadata' => null,
            'text' => 'This is an example message'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "fileName": string
  "pageNumber"?: integer // Optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (Optional)
  "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (Optional)
  "llamaIndexObjectJson"?: object // Optional
  "metadata"?: object // Optional
  "text": string
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "fileName": "Response string",
  "pageNumber": 456,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "Response string",
  "updatedAt": "Response string"
}
```

Partially Update a Document

PATCH `/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Document.
`knowledgeBasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
fileName	string	No	
pageNumber	integer	No	
chatbotFile	string (uuid)	No	
excludedEmbedMetadataKeys	object	No	List[str]: metadata keys to exclude from the embedding.
excludedLlmMetadataKeys	object	No	List[str]: metadata keys to exclude from the LLM context.
llamaIndexObjectJson	object	No	
metadata	object	No	
text	string	No	

Example Request Schema

```
{
  "fileName"?: string // Optional
  "pageNumber"?: integer // Optional
  "chatbotFile"?: string (uuid) // Optional
  "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (Optional)
  "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (Optional)
  "llamaIndexObjectJson"?: object // Optional
  "metadata"?: object // Optional
  "text"?: string // Optional
}
```

Example Request Values

```
{
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "This is an example message"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "This is an example message"
  };

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": None,
    "excludedLlmMetadataKeys": None,
    "llamaIndexObjectJson": None,
    "metadata": None,
    "text": "This is an example message"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'fileName' => 'Example string',
            'pageNumber' => 123,
            'chatbotFile' => '550e8400-e29b-41d4-a716-446655440000',
            'excludedEmbedMetadataKeys' => null,
            'excludedLlmMetadataKeys' => null,
            'llamaIndexObjectJson' => null,
            'metadata' => null,
            'text' => 'This is an example message'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "fileName": string
  "pageNumber"?: integer // Optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys"?: object // List[str]: metadata keys to
exclude from the embedding. (Optional)
  "excludedLlmMetadataKeys"?: object // List[str]: metadata keys to
exclude from the LLM context. (Optional)
  "llamaIndexObjectJson"?: object // Optional
  "metadata"?: object // Optional
  "text": string
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "fileName": "Response string",
  "pageNumber": 456,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "Response string",
  "updatedAt": "Response string"
}
```

Delete a Document

DELETE `/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Chatbot Document.
`knowledgeBasePk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
204	No response body

Create a new FAQ

POST `/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/`

Parameters

Parameter name	Required	Type	Description
knowledgeBasePk	V	string	

Request body

Request parameters

Field	Type	Required	Description
question	string	Yes	
answer	string	Yes	
answerMediaUrls	object	No	URLs of media files, such as images and videos, in the answer.
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Example request schema

```
{
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (Optional)
```

```
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
}
```

Example request values

```
{
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": null,
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": null,
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faqs/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": None,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": None
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'question' => 'Example string',
            'answer' => 'Example string',
            'answerMediaUrls' => null,
            'labels' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'rawUserDefineMetadata' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Example response schema

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used to
create this FAQ.
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase":  // Always taken from the knowledge base in the URL; a
value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```


Update FAQ metadata

PATCH `/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}/update-metadata/`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this FAQ.
`knowledgeBasePk`	V	string	

Request body

Request parameters

Field	Type	Required	Description
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Example request schema

```
{
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
}
```

Example request values

```
{
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/update-metadata/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  };

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/update-metadata/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-
446655440000/update-metadata/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": None
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-
446655440000/update-metadata/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'labels' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'rawUserDefineMetadata' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: `200`

Example response schema

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used to
create this FAQ.
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase":  // Always taken from the knowledge base in the URL; a
value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example response values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```


Batch delete FAQs

POST `/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/batch-delete/`

Parameters

Parameter name	Required	Type	Description
knowledgeBasePk	V	string	

Request body

Request parameters

Field	Type	Required	Description
ids	array[string]	No	

Example request schema

```
{
  "ids"?: [ // Optional
    string (uuid)
  ]
}
```

Example request values

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faqs/batch-delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'ids' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description

204	No response body

List all FAQs in a specific knowledge base

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/`

Parameters

Parameter name	Required	Type	Description
`knowledgeBasePk`	V	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	Search keywords. Supports multiple search criteria separated by spaces. Searchable fields: FAQ ID, question, answer, label name, and Metadata value

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/?
page=1&pageSize=1&query=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "question": string
      "answer": string
      "answerMediaUrls"?: object // URLs of media files, such as images
and videos, in the answer. (Optional)
      "hitsCount": integer
      "embeddingTokensCount": integer // Number of embedding tokens used
to create this FAQ.
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "knowledgeBase":  // Always taken from the knowledge base in the
URL; a value in the request body is ignored.
      {
        "id": string (uuid)
        "name": string
      }
    }
  ]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "question": "Response string",
      "answer": "Response string",
      "answerMediaUrls": null,
      "hitsCount": 456,
      "embeddingTokensCount": 456,
      "labels": [
        {
```

```
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ],
        "rawUserDefineMetadata": null,
        "knowledgeBase": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      }
    ]
}
```


Retrieve details of a specific FAQ

GET `/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this FAQ.
`knowledgeBasePk`	V	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
```

```
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used to
create this FAQ.
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase":  // Always taken from the knowledge base in the URL; a
value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
```

```
    }
}
```

Update FAQ

PUT `/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this FAQ.
`knowledgeBasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
question	string	Yes	
answer	string	Yes	
answerMediaUrls	object	No	URLs of media files, such as images and videos, in the answer.
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Example Request Schema

```
{
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (Optional)
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
}
```

Example Request Values

```
{
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": null,
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": null,
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  };

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": None,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": None
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'question' => 'Example string',
            'answer' => 'Example string',
            'answerMediaUrls' => null,
            'labels' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'rawUserDefineMetadata' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used to
create this FAQ.
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase":  // Always taken from the knowledge base in the URL; a
value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```


Partially Update FAQ

PATCH `/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this FAQ.
`knowledgeBasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
question	string	No	
answer	string	No	

Field	Type	Required	Description
answerMediaUrls	object	No	URLs of media files, such as images and videos, in the answer.
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Example Request Schema

```
{
  "question"?: string // Optional
  "answer"?: string // Optional
  "answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (Optional)
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
}
```

Example Request Values

```
{
  "question": "Example string",
  "answer": "Example string",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": null,
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": null,
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "rawUserDefineMetadata": null
  };

axios.patch("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "question": "Example string",
    "answer": "Example string",
    "answerMediaUrls": None,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": None
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'question' => 'Example string',
            'answer' => 'Example string',
            'answerMediaUrls' => null,
            'labels' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'rawUserDefineMetadata' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs of media files, such as images and
videos, in the answer. (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used to
create this FAQ.
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase":  // Always taken from the knowledge base in the URL; a
value in the request body is ignored.
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```


Delete FAQ

DELETE `/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this FAQ.
`knowledgeBasePk`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Export FAQ Excel (or a Blank Template)

GET `/api/v1/knowledge-bases/{id}/export-faq-excel/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/export-faq-excel/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/export-faq-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-faq-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/export-faq-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "createdBy":
```

```
  {
    "id": string (uuid)
    "name": string
  }
  "organization"?: string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of reference chunks to
retrieve; defaults to 12, with a minimum of 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE; defaults to False (Optional)
  "similarityCutoff"?: number (double) // Similarity threshold; defaults
to 0.0, with a range of 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking; defaults to
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?:  // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
```

```
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access the knowledge base
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
  "vectorStorageSize": integer
  "chunksCount": integer
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "description": "response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
```

```
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "response string",
  "vectorDatabaseApiKey": "response string",
  "opensearchUsername": "response string",
  "opensearchPassword": "response string",
  "oracleUser": "response string",
  "oraclePassword": "response string",
  "synxdbUser": "response string",
  "synxdbPassword": "response string",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ],
  "filesCount": 456,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "response string",
  "updatedAt": "response string"
}
```


Import FAQ Excel (Asynchronous)

POST `/api/v1/knowledge-bases/{id}/import-faq-excel/`

Only one import job can run at a time for a knowledge base. Calling this endpoint while a job is running returns 409; use the FAQ import job endpoints below to check progress.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/import-faq-excel/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -F "excelFile=@example_file.txt"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with the actual file
formData.append('excelFile',
* Actual file object *
, 'example_file.txt');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/import-faq-excel/", formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/import-faq-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'excelFile': ('example_file.txt', open('example_file.txt',
'rb'), 'text/plain')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/import-faq-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'excelFile',
                'contents' => fopen('example_file.txt', 'r'),
                'filename' => 'example_file.txt'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 202

Example Response Schema

```
{
  "jobId": string (uuid)
  "status": string (enum: pending, processing, done, failed) // *
`pending` - Pending
* `processing` - Processing
* `done` - Done
* `failed` - Failed
}
```

Example Response Values

```
{
  "jobId": "550e8400-e29b-41d4-a716-446655440000",
  "status": "pending"
}
```

Status Code: 409

Example Response Schema

```
{
  "detail": string
  "jobId": string (uuid)
}
```

Example Response Values

```
{
  "detail": "response string",
  "jobId": "550e8400-e29b-41d4-a716-446655440000"
}
```

List FAQ Import Jobs

`GET` `/api/v1/knowledge-bases/{id}/faq-import-jobs/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.
`embeddingModel`	X	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`pagination`	X	boolean	Set to false to return all jobs without pagination (defaults to true).
`query`	X	string	
`rerankerModel`	X	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faq-import-jobs/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&pagination=true&query=example&reranke
rModel=550e8400-e29b-41d4-a716-446655440000"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faq-import-jobs/?embeddingModel=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&pagination=true&query=example&rerankerModel=550e8400-e29b-41d4-a716-446655440000", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faq-import-jobs/?embeddingModel=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&pagination=true&query=example&rerankerModel=550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faq-import-jobs/?embeddingModel=550e8400-
e29b-41d4-a716-
446655440000&page=1&pageSize=1&pagination=true&query=example&reranke
rModel=550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "status":
      {
      }
      "totalRows": integer
      "processedRows": integer
      "createdCount": integer
      "skippedDuplicateCount": integer // Reserved for content-hash
dedupe; always 0 until dedupe ships.
      "errors": object // Row-level parse errors and chunk-level import
errors.
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "status": {},
      "totalRows": 456,
      "processedRows": 456,
      "createdCount": 456,
      "skippedDuplicateCount": 456,
      "errors": null,
      "createdAt": "response string",
      "updatedAt": "response string"
    }
```

```
  ]
}
```

Retrieve a Specific FAQ Import Job

GET `/api/v1/knowledge-bases/{id}/faq-import-jobs/{jobId}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Knowledge Base.
`jobId`	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faq-import-jobs/{jobId}
"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faq-import-jobs/{jobId}
", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faq-import-jobs/{jobId}
"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faq-import-jobs/{jobId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
```

```
  "status":
  {
  }
  "totalRows": integer
  "processedRows": integer
  "createdCount": integer
  "skippedDuplicateCount": integer // Reserved for content-hash dedupe;
always 0 until dedupe ships.
  "errors": object // Row-level parse errors and chunk-level import
errors.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": {},
  "totalRows": 456,
  "processedRows": 456,
  "createdCount": 456,
  "skippedDuplicateCount": 456,
  "errors": null,
  "createdAt": "response string",
  "updatedAt": "response string"
}
```

List Parsers

GET `/api/v1/parsers/`

When uploading a file to a knowledge base, set the parser field to the parser's id, which you can retrieve from this endpoint.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/parsers/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/parsers/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/parsers/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/parsers/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
[
  {
    "id": string (uuid)
    "name": string
```

```
    "provider":
    {
    }
    "order"?: integer // Optional
    "isTimestampSttProvider": boolean
    "asrProvider": object // The provider spec, built once per provider
row per response.

The spec is platform-level, not per parser, and it is large — a
whisper-family row carries a 100-entry language catalog in its
``languages`` spec, as codes plus display names. This serializer is
nested per row in the knowledge-base file listing and in message
citations, so without memoizing, one response rebuilds the same
catalog for every row. ``self.context`` is the root serializer's shared
dict, which is also what keeps ``get_active_diarization_rows`` at one
query per response.
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "provider": {},
    "order": 456,
    "isTimestampSttProvider": false,
    "asrProvider": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  }
]
```

Retrieve a Specific Parser

GET `/api/v1/parsers/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this parser.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/parsers/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/parsers/550e8400-e29b-
41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/parsers/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/parsers/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string
```

```
  "provider":
  {
  }
  "order"?: integer // Optional
  "isTimestampSttProvider": boolean
  "asrProvider": object // The provider spec, built once per provider row
per response.

The spec is platform-level, not per parser, and it is large — a
whisper-family row carries a 100-entry language catalog in its
``languages`` spec, as codes plus display names. This serializer is
nested per row in the knowledge-base file listing and in message
citations, so without memoizing, one response rebuilds the same
catalog for every row. ``self.context`` is the root serializer's shared
dict, which is also what keeps ``get_active_diarization_rows`` at one
query per response.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "provider": {},
  "order": 456,
  "isTimestampSttProvider": false,
  "asrProvider": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```

Example API call (Shell)

目錄

1. List knowledge base files for an AI assistant
2. List knowledge base FAQs for an AI assistant
3. Retrieve text nodes from all files for an AI assistant
4. Retrieve a text node from a specific file for an AI assistant
5. Search Test
6. List Parsers Supported by File Type

Example API call (Shell)

List knowledge base files for an AI assistant

`GET` `/api/v1/chatbots/{id}/knowledge-bases-files/`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	
`largeLanguageModel`	X	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`replyMode`	X	string	
`source`	X	string	`self_built`: self_built ; `built_in`: built_in ; `all`: all;

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response content

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
```

```
    {
      "id": string (uuid)
      "filename": string // File name
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?:  // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser":
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "isTimestampSttProvider": boolean
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
          "displayName": string
        }
      ]
      "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
      "chunksCount": integer // Number of chunks/nodes generated from
this file
      "waitingTime": number (double)
```

```
      "processingTime": number (double)
      "processingTimeDetails": object
      "previewUrl": string // For presentations (pptx/ppt), return the
URL of a derived PDF file for preview (displayed with a PDF viewer in the
frontend); otherwise, return None.

Generate the URL with the same CustomizedFileFieldSerializer as the
`file` field to ensure a consistent format (including each
storage provider's presign/host rules). Do not use get_absolute_url()—it
generates incorrectly formatted URLs.
      "createdAt": string (timestamp)
    }
  ]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "fileType": "Response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "provider": {},
        "isTimestampSttProvider": false
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
```

```
      ],
      "rawUserDefineMetadata": null,
      "speakerLabels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "originalLabel": "Response string",
          "customName": "Response string",
          "displayName": "Response string"
        }
      ],
      "vectorStorageSize": 456,
      "chunksCount": 456,
      "waitingTime": 456,
      "processingTime": 456,
      "processingTimeDetails": null,
      "previewUrl": "Response string",
      "createdAt": "Response string"
    }
  ]
}
```


List knowledge base FAQs for an AI assistant

GET `/api/v1/chatbots/{id}/knowledge-bases-faqs/`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	
`largeLanguageModel`	X	string	
`page`	X	integer	A page number within the paginated result set.

`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`replyMode`	X	string	
`source`	X	string	`self_built`: self_built ; `built_in`: built_in ; `all`: all;

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response content

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
```

```
    {
      "id": string (uuid)
      "question": string
      "answer": string
      "answerMediaUrls"?: object // URLs of media files, such as images
and videos, in the answer. (Optional)
      "hitsCount": integer
      "embeddingTokensCount": integer // Number of embedding tokens used
to create this FAQ.
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "knowledgeBase":  // Always taken from the knowledge base in the
URL; a value in the request body is ignored.
      {
        "id": string (uuid)
        "name": string
      }
    }
  ]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "question": "Response string",
      "answer": "Response string",
      "answerMediaUrls": null,
      "hitsCount": 456,
      "embeddingTokensCount": 456,
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
```

```
        }
      ],
      "rawUserDefineMetadata": null,
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    }
  ]
}
```


Retrieve text nodes from all files for an AI assistant

GET `/api/v1/chatbot-text-nodes/`

Parameters

Parameter name	Required	Type	Description
`chatbotFile`	X	string	[To be deprecated] Chatbot file ID (use knowledge_base_file)
`knowledgeBaseFile`	X	string	Knowledge base file ID (recommended)
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/chatbot-text-nodes/?
chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/?
chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/chatbot-text-nodes/?
chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/?
chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response content

Status code: 200

Example response schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "charactersCount": integer
      "hitsCount": integer
      "text": string
      "updatedAt": string (timestamp)
      "filename": string
      "chatbotFile": object // Return complete ChatbotFile information,
including the file URL and type, to support image previews in the
frontend
      "knowledgeBaseFile": object // Same as get_chatbot_file, for
backward compatibility
      "pageNumber": integer // Backward-compatible alias of
``page_start``.
      "pageStart": integer
      "pageEnd": integer
      "citationTitle": string // Source title shown on the inline
citation hover card.
      "citationDescription": string // Source description shown on the
inline citation hover card.
      "citationQuote": string // Quoted text snippet shown on the inline
citation hover card.
      "hasImage": boolean // Determine whether images are present (check
the file type or Markdown images in text)
      "imageUrl": string // Extract the image URL (use the file URL if
available; otherwise, extract it from Markdown)
      "displayText": string // Return the full text with Markdown image
markup removed
      "displayTitle": string // Return the display title (fallback:
citation_title -> filename)
      "highlightedText": string // Return text with matched terms wrapped
in ``<mark>`` tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via
``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
      "labels": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
```

```
      "rawUserDefineMetadata"?: object // Metadata keys and values
defined by the user. (Optional)
      "metadataEnabledForSearch": object // Map each user-defined
metadata key on this node to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is
True *and*
the node carries a value for it. Since this maps over
``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the
node's
knowledge base ``MetadataKey`` current setting, matching the AI Search
indicator.
      "startCharIdx": integer
      "endCharIdx": integer
    }
  ]
}
```

Example response values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "Response string",
      "updatedAt": "Response string",
      "filename": "Response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "knowledgeBaseFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
```

```
      "pageNumber": 456,
      "pageStart": 456,
      "pageEnd": 456,
      "citationTitle": "Response string",
      "citationDescription": "Response string",
      "citationQuote": "Response string",
      "hasImage": false,
      "imageUrl": "Response string",
      "displayText": "Response string",
      "displayTitle": "Response string",
      "highlightedText": "Response string",
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "metadataEnabledForSearch": {
        "key1": "value1",
        "key2": "value2",
        "createdAt": "2025-01-01T00:00:00.000Z"
      },
      "startCharIdx": 456,
      "endCharIdx": 456
    }
  ]
}
```


Status code: 403 - You do not have permission to read text nodes from this file: knowledge base files require chatbot_knowledge_base_access and group access to the knowledge base; conversation attachments require chatbot_chatbot_access for the assistant; inbox files require group access to the inbox

Status code: 404 - The knowledge base file does not exist or does not belong to the current organization

Retrieve a text node from a specific file for an AI assistant

GET `/api/v1/chatbot-text-nodes/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this ChatbotTextNode.

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/chatbot-text-
nodes/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/chatbot-text-
nodes/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response content

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "charactersCount": integer
```

```
  "hitsCount": integer
  "text": string
  "updatedAt": string (timestamp)
  "filename": string
  "chatbotFile": object // Return complete ChatbotFile information,
including the file URL and type, to support image previews in the
frontend
  "knowledgeBaseFile": object // Same as get_chatbot_file, for backward
compatibility
  "pageNumber": integer // Backward-compatible alias of ``page_start``.
  "pageStart": integer
  "pageEnd": integer
  "citationTitle": string // Source title shown on the inline citation
hover card.
  "citationDescription": string // Source description shown on the inline
citation hover card.
  "citationQuote": string // Quoted text snippet shown on the inline
citation hover card.
  "hasImage": boolean // Determine whether images are present (check the
file type or Markdown images in text)
  "imageUrl": string // Extract the image URL (use the file URL if
available; otherwise, extract it from Markdown)
  "displayText": string // Return the full text with Markdown image
markup removed
  "displayTitle": string // Return the display title (fallback:
citation_title -> filename)
  "highlightedText": string // Return text with matched terms wrapped in
``<mark>`` tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via
``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Metadata keys and values defined by
the user. (Optional)
  "metadataEnabledForSearch": object // Map each user-defined metadata
key on this node to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is
```

```
True *and*
the node carries a value for it. Since this maps over
``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the
node's
knowledge base ``MetadataKey`` current setting, matching the AI Search
indicator.
  "startCharIdx": integer
  "endCharIdx": integer
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "charactersCount": 456,
  "hitsCount": 456,
  "text": "Response string",
  "updatedAt": "Response string",
  "filename": "Response string",
  "chatbotFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "knowledgeBaseFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "pageNumber": 456,
  "pageStart": 456,
  "pageEnd": 456,
  "citationTitle": "Response string",
  "citationDescription": "Response string",
  "citationQuote": "Response string",
  "hasImage": false,
  "imageUrl": "Response string",
  "displayText": "Response string",
  "displayTitle": "Response string",
  "highlightedText": "Response string",
  "labels": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "metadataEnabledForSearch": {
    "key1": "value1",
    "key2": "value2",
    "createdAt": "2025-01-01T00:00:00.000Z"
  },
  "startCharIdx": 456,
  "endCharIdx": 456
}
```


Search Test

POST `/api/v1/chatbots/{id}/search/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`largeLanguageModel`	X	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`replyMode`	X	string	

`source`	X	string	`self_built`: self_built ; `built_in`: built_in ; `all`: all;

Request Body

Request Parameters

Field	Type	Required	Description
queryMetadata	object	No	Not supported for chatbot-wide search; providing a value returns 400.

Example Request Schema

```
{
  "queryMetadata"?: object // Not supported for chatbot-wide search;
providing a value returns 400. (Optional)
}
```

Example Request Values

```
{
  "queryMetadata": null
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "query": "Example string",
    "queryMetadata": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "query": "Example string",
    "queryMetadata": null
  };

axios.post(" data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "query": "Example string",
    "queryMetadata": None
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'query' => 'Example string',
            'queryMetadata' => null
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "charactersCount": integer
      "hitsCount": integer
      "text": string
      "updatedAt": string (timestamp)
      "filename": string
      "chatbotFile": object // Returns complete ChatbotFile information,
including the file URL and type, to support image previews in the
frontend
      "knowledgeBaseFile": object // Same as get_chatbot_file; provides
backward compatibility
      "pageNumber": integer // Backward-compatible alias of
``page_start``.
      "pageStart": integer
      "pageEnd": integer
      "citationTitle": string // Source title shown on the inline
citation hover card.
      "citationDescription": string // Source description shown on the
inline citation hover card.
      "citationQuote": string // Quoted text snippet shown on the inline
citation hover card.
      "hasImage": boolean // Determines whether images are present by
checking the file type or Markdown images in text
      "imageUrl": string // Extracts the image URL, preferring the file
URL and otherwise extracting it from Markdown
      "displayText": string // Returns the full text with Markdown image
markup removed
      "displayTitle": string // Returns the display title (fallback:
citation_title -> filename)
      "highlightedText": string // Return text with matched terms wrapped
in ``<mark>`` tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via
``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
      "labels": [
        {
```

```
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Metadata keys and values
defined by the user. (Optional)
      "metadataEnabledForSearch": object // Map each user-defined
metadata key on this node to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is
True *and*
the node carries a value for it. Since this maps over
``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the
node's
knowledge base ``MetadataKey`` current setting, matching the AI Search
indicator.
      "startCharIdx": integer
      "endCharIdx": integer
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "Response string",
      "updatedAt": "Response string",
      "filename": "Response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "knowledgeBaseFile": {
```

```
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "pageNumber": 456,
      "pageStart": 456,
      "pageEnd": 456,
      "citationTitle": "Response string",
      "citationDescription": "Response string",
      "citationQuote": "Response string",
      "hasImage": false,
      "imageUrl": "Response string",
      "displayText": "Response string",
      "displayTitle": "Response string",
      "highlightedText": "Response string",
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "metadataEnabledForSearch": {
        "key1": "value1",
        "key2": "value2",
        "createdAt": "2025-01-01T00:00:00.000Z"
      },
      "startCharIdx": 456,
      "endCharIdx": 456
    }
  ]
}
```


List Parsers Supported by File Type

GET `/api/v1/parsers/supported-file-types/`

Parameters

Parameter Name	Required	Type	Description
`knowledgeBaseId`	X	string	Knowledge base ID. When provided and the knowledge base does not support multimodal embeddings, image file types will be filtered out.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/parsers/supported-file-types/?
knowledgeBaseId=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/parsers/supported-file-types/?
knowledgeBaseId=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
[
  {
    "fileType": string
```

```
    "parsers": [
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "order"?: integer // Optional
        "isTimestampSttProvider": boolean
        "asrProvider": object // The provider spec, built once per
provider row per response.

The spec is platform-level, not per parser, and it is large — a
whisper-family row carries a 100-entry language catalog in its
``languages`` spec, as codes plus display names. This serializer is
nested per row in the knowledge-base file listing and in message
citations, so without memoizing, one response rebuilds the same
catalog for every row. ``self.context`` is the root serializer's shared
dict, which is also what keeps ``get_active_diarization_rows`` at one
query per response.
      }
    ]
    "autoDetect": boolean // Whether files of this type get parser auto-
detection after upload
(the system auto-detects a parser when the user did not choose one).
  }
]
```

Example Response Values

```
[
  [
    {
      "fileType": ".pdf",
      "parsers": [
        {
          "id": "ab83f144-5026-4bce-993f-5c982cc19318",
          "name": "MaiAgent Parser",
          "provider": "maiagent",
          "order": 0,
          "isDefault": true
        },
        {
          "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
```

```
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
      }
    ]
  },
  {
    "fileType": ".doc",
    "parsers": [
      {
        "id": "ab83f144-5026-4bce-993f-5c982cc19318",
        "name": "MaiAgent Parser",
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
      },
      {
        "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
      }
    ]
  },
  {
    "fileType": ".docx",
    "parsers": [
      {
        "id": "ab83f144-5026-4bce-993f-5c982cc19318",
        "name": "MaiAgent Parser",
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
      },
      {
        "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
      }
    ]
  },
```

```
    {
      "fileType": ".ppt",
      "parsers": [
        {
          "id": "ab83f144-5026-4bce-993f-5c982cc19318",
          "name": "MaiAgent Parser",
          "provider": "maiagent",
          "order": 0,
          "isDefault": true
        },
        {
          "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
          "name": "MaiAgent Parser (Online)",
          "provider": "llama",
          "order": 2,
          "isDefault": false
        }
      ]
    },
    {
      "fileType": ".pptx",
      "parsers": [
        {
          "id": "ab83f144-5026-4bce-993f-5c982cc19318",
          "name": "MaiAgent Parser",
          "provider": "maiagent",
          "order": 0,
          "isDefault": true
        },
        {
          "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
          "name": "MaiAgent Parser (Online)",
          "provider": "llama",
          "order": 2,
          "isDefault": false
        }
      ]
    },
    {
      "fileType": ".xls",
      "parsers": [
        {
          "id": "ab83f144-5026-4bce-993f-5c982cc19318",
          "name": "MaiAgent Parser",
          "provider": "maiagent",
```

```
        "order": 0,
        "isDefault": true
      }
    ]
  },
  {
    "fileType": ".xlsx",
    "parsers": [
      {
        "id": "ab83f144-5026-4bce-993f-5c982cc19318",
        "name": "MaiAgent Parser",
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
      }
    ]
  },
  {
    "fileType": ".csv",
    "parsers": [
      {
        "id": "ab83f144-5026-4bce-993f-5c982cc19318",
        "name": "MaiAgent Parser",
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
      }
    ]
  },
  {
    "fileType": ".txt",
    "parsers": [
      {
        "id": "ab83f144-5026-4bce-993f-5c982cc19318",
        "name": "MaiAgent Parser",
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
      },
      {
        "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
```

```
      }
    ]
  },
  {
    "fileType": ".md",
    "parsers": [
      {
        "id": "ab83f144-5026-4bce-993f-5c982cc19318",
        "name": "MaiAgent Parser",
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
      }
    ]
  },
  {
    "fileType": ".json",
    "parsers": [
      {
        "id": "ab83f144-5026-4bce-993f-5c982cc19318",
        "name": "MaiAgent Parser",
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
      }
    ]
  },
  {
    "fileType": ".jsonl",
    "parsers": [
      {
        "id": "ab83f144-5026-4bce-993f-5c982cc19318",
        "name": "MaiAgent Parser",
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
      }
    ]
  },
  {
    "fileType": ".html",
    "parsers": [
      {
        "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
        "name": "MaiAgent Parser (Online)",
```

```
        "provider": "llama",
        "order": 2,
        "isDefault": false
      }
    ]
  },
  {
    "fileType": ".mp3",
    "parsers": [
      {
        "id": "4c47e305-2eb7-4438-8d3c-d91eb0b06cc0",
        "name": "Azure Speech",
        "provider": "azure",
        "order": 1,
        "isDefault": true
      },
      {
        "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
      }
    ]
  },
  {
    "fileType": ".wav",
    "parsers": [
      {
        "id": "4c47e305-2eb7-4438-8d3c-d91eb0b06cc0",
        "name": "Azure Speech",
        "provider": "azure",
        "order": 1,
        "isDefault": true
      },
      {
        "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
      }
    ]
  },
  {
```

```
      "fileType": ".mp4",
      "parsers": [
        {
          "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
          "name": "MaiAgent Parser (Online)",
          "provider": "llama",
          "order": 2,
          "isDefault": false
        }
      ]
    }
  ]
]
```

Example API call (Shell)

目錄

1. Register an Account
2. Change Password
3. Create a New Organization
4. Add a Member to the Specified Organization
5. List Organizations
6. Retrieve a specific organization
7. Get Current User Details
8. Get Current User Permissions
9. List Members of an Organization
10. Retrieve Member Details
11. Update Organization Information
12. Update Current User Details
13. Update Member Roles
14. List Organization API Keys
15. Issue an API Key for a Member
16. Get a Specific API Key
17. Update API Key Settings
18. Revoke API Key
19. Get the Organization an API Key Belongs To
20. Issue a Member Embed Token
21. List OAuth Applications
22. Register OAuth Application
23. Retrieve OAuth Application
24. Update OAuth Application
25. Delete OAuth Application and Revoke Its Tokens
26. Regenerate Client Secret
27. List OAuth applications I have authorized
28. Revoke my authorization for an application
29. Delete organization
30. Delete a specified member
31. Get member usage statistics
32. Get member usage statistics summary
33. Get AI Assistant Usage Statistics
34. Get AI Assistant Usage Statistics Summary
35. List Your Own Memories

36. Create or Update Your Own Memory
37. Update or Reactivate Your Own Memory
38. Delete Your Own Memory

Example API call (Shell)

Register an Account

`POST` `/api/v1/auth/registration/`

Request Body

Request Parameters

Field	Type	Required	Description
email	string (email)	Yes	
password1	string	Yes	
password2	string	Yes	
name	string	Yes	
company	string	Yes	
referralCode	string	No	
authSource	object (contains 2 properties: id, name)	No	
authSource.id	string (uuid)	Yes	

Request Schema Example

```
{
  "email": string (email)
  "password1": string
  "password2": string
```

```
  "name": string
  "company": string
  "referralCode"?: string // Optional
  "authSource"?:  // Optional
  {
    "id": string (uuid)
  }
}
```

Example Request Values

```
{
  "email": "user@example.com",
  "password1": "Example string",
  "password2": "Example string",
  "name": "Example name",
  "company": "Example string",
  "referralCode": "Example string",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/auth/registration/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "email": "user@example.com",
    "password1": "Example string",
    "password2": "Example string",
    "name": "Example name",
    "company": "Example string",
    "referralCode": "Example string",
    "authSource": {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "email": "user@example.com",
    "password1": "Example string",
    "password2": "Example string",
    "name": "Example name",
    "company": "Example string",
    "referralCode": "Example string",
    "authSource": {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  };

axios.post("https://api.maiagent.ai/api/v1/auth/registration/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/auth/registration/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "email": "user@example.com",
    "password1": "Example string",
    "password2": "Example string",
    "name": "Example name",
    "company": "Example string",
    "referralCode": "Example string",
    "authSource": {
        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/auth/registration/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'email' => 'user@example.com',
            'password1' => 'Example string',
            'password2' => 'Example string',
            'name' => 'Example name',
            'company' => 'Example string',
            'referralCode' => 'Example string',
            'authSource' => [
                'id' => '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "email": string (email)
  "password1": string
  "password2": string
  "name": string
  "company": string
  "referralCode"?: string // Optional
  "authSource"?:  // Optional
  {
    "id": string (uuid)
    "name": string
  }
}
```

Example Response Values

```
{
  "email": "response@example.com",
  "password1": "Response string",
  "password2": "Response string",
  "name": "Response string",
  "company": "Response string",
  "referralCode": "Response string",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```

Change Password

POST `/api/v1/auth/password/change/`

Request Body

Request Parameters

Field	Type	Required	Description
oldPassword	string	Yes	
newPassword1	string	Yes	
newPassword2	string	Yes	

Request Schema Example

```
{
  "oldPassword": string
  "newPassword1": string
  "newPassword2": string
}
```

Example Request Values

```
{
  "oldPassword": "Example string",
  "newPassword1": "Example string",
  "newPassword2": "Example string"
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/auth/password/change/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "oldPassword": "Example string",
    "newPassword1": "Example string",
    "newPassword2": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "oldPassword": "Example string",
    "newPassword1": "Example string",
    "newPassword2": "Example string"
  };

axios.post("https://api.maiagent.ai/api/v1/auth/password/change/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/auth/password/change/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "oldPassword": "Example string",
    "newPassword1": "Example string",
    "newPassword2": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/auth/password/change/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'oldPassword' => 'Example string',
            'newPassword1' => 'Example string',
            'newPassword2' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "detail": string
}
```

Example Response Values

```
{
  "detail": "Response string"
}
```

Create a New Organization

POST `/api/v1/organizations/`

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Organization name
compact_logo	string (binary)	No	Compact logo (optional, available only on the Enterprise plan)
full_logo	string (binary)	No	Full logo (optional, available only on the Enterprise plan)

Request Schema Example

```
{
  "name": string // Organization name
  "compact_logo"?: string (binary) // Compact logo (optional, available
```

```
only on the Enterprise plan) (Optional)
  "full_logo"?: string (binary) // Full logo (optional, available only on
the Enterprise plan) (Optional)
}
```

Example Request Values

```
{
  "name": "My Organization"
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "My Organization"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "My Organization"
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "My Organization"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'My Organization'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "createdAt": string (timestamp)
  "compactLogo"?: string (uri) // Optional
  "fullLogo"?: string (uri) // Optional
  "themePrimaryColor"?: // May have different types (Optional)
  string // UI theme color for elements such as buttons and links (HEX format, e.g., #0960bd); leaving it blank means it is unset, and the frontend falls back to the system’s primary brand color (Optional)
  "usageStatistics": object
  "organizationPlan": object
  "creditWallet": object // Return credit wallet with ID only.
  "maigptInboxId": string (uuid)
  "canUseCustomElasticsearch": boolean // Allow this organization to configure custom Elasticsearch endpoints for knowledge bases.
  "canUseVoiceAgent": boolean // Allow this organization to use voice agent features.
  "canUseMeetingRecords": boolean // Allow this organization to use meeting records features.
  "canUseCallCenter": boolean // Allow this organization to use call center features.
  "canUseHookSystem": boolean // Manual override to grant this organization the hook system regardless of plan. Enterprise-plan organizations get access automatically without this flag. Layered under the global Config.hook_system_enabled switch: it must also be enabled.
  "canUseAiGateway": boolean // Allow this organization to call the OpenAI-compatible `/api/v1/chat/completions` Gateway. On by default for license installs, otherwise off until a platform admin enables it.
  "canUseAgentTeams": boolean // Read-only projection of the platform-wide Agent Teams switch (Config.enable_agent_teams). Not a per-organization flag; member-level access is still governed by RBAC.
  "canUseFileLibrary": boolean // Allow this organization to use the file library. Off by default for gradual rollout.
  "canUseFileOnlineEditing": boolean // Enable online editing in the file library (P7): the text editor and upload-new-version entries. Requires can_use_file_library to also be on.
  "canUseOfficeOnlineEditing": boolean // Enable Office (xlsx/docx/pptx) online editing in the file library (P7 S9). Requires can_use_file_online_editing to also be on.
  "enableAgentMemory": boolean // Enable persistent cross-conversation memory for agents in this organization. When off, agents use only in-session memory. On by default on SaaS; license installs remain opt-in. Members can still turn it off for themselves in Profile > AI Memory.
```

```
  "hideChinaModels"?: boolean // When on, platform models whose developer is based in China (including ones added later) are not visible or usable in this organization: model menus, assistants, the API and AI Gateway all exclude them. (Optional)
  "gatewayServiceFeePercent": string (decimal) // Markup applied on top of the token price when billing MaiAgent-managed models called through the AI Gateway (/v1/chat/completions and /v1/embeddings). Managed gateway markup only; the organization's own models (BYOK / self-hosted) are billed via the AI Gateway traffic fee percent instead.
  "gatewayTrafficFeePercent": string (decimal) // Pass-through fee for calls that run on this organization's own AI Gateway models (BYOK / self-hosted): charged as the call's total tokens (input, output, and prompt-cache) x this percent in credits, with no LLM token pricing. Adjustable per organization; changes apply to new calls only.
  "platformProcessingFeePercent": string (decimal) // Surcharge on every credit consumption of this organization, billed as a separate "Platform Processing Fee" line equal to the consumed credits x this percent. 0 disables the fee. Applies to new consumption only; the percent is shown read-only on the customer pricing page.
  "platformProcessingFeeEnabled": boolean // Whether new consumption of this organization accrues the platform processing fee. The percent is exposed read-only as platform_processing_fee_percent.
  "isTrial": boolean // Whether the organization is in a trial period. True when the organization has exactly one trial plan and it is currently active
  "expirationDays": integer // Days since expiration. A positive value is the number of days since expiration; null means the plan has not expired or is a permanent plan
  "showTrialBanner": boolean // Controls whether the frontend displays the trial/free plan banner
  "noActivePlanDays": integer // Consecutive days without an active paid plan. null when cleanup is disabled.
  "cleanupRemainingDays": integer // Days remaining before organization data is permanently deleted. null when cleanup is disabled, org has an active paid plan, or no_active_plan_days is 0.
  "estimatedCleanupDate": string // ISO date when organization data will be permanently deleted. null when cleanup_remaining_days is null.
  "approvalStatus":  // Trial-eligibility review status. Only approved organizations can receive a trial plan when organization approval is enabled. Paid plan activation auto-approves the organization.

* `pending` - Pending Review
* `approved` - Approved
* `rejected` - Rejected
  {
```

```
  }
  "companyName"?: string // Applicant-reported company name collected at
organization creation for approval review. (Optional)
  "taxId"?: // May have different types (Optional)
  string // Applicant-reported business tax ID (8 digits, optional).
Stored as-is; not verified. (Optional)
  "contactPhone"?: string // Applicant-reported contact phone collected
at organization creation for approval review. (Optional)
  "applicationMetadata"?: object // Extra applicant-reported key-value
fields collected at organization creation. Lets the frontend collect
additional review fields without a schema migration; the core review
fields (company name, tax ID, contact phone) stay as named columns.
(Optional)
  "invitationCode"?: string // Optional
}
```

Example Response Values


```
{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "My Organization",
  "createdAt": "1748336288000",
  "compactLogo": null,
  "fullLogo": null,
  "usageStatistics": {
    "chatbotsCount": 0,
    "countableChatbotsCount": 0,
    "exemptChatbotsByMode": {},
    "canCreateChatbot": true,
    "createChatbotLimit": 1,
    "currentMonthWordsCountTotal": 1000000,
    "currentMonthUsedWordsCountTotal": 0,
    "currentMonthUsedConversationsCountTotal": 0,
    "availableUploadFilesSizeTotal": 104857600,
    "usedUploadFileSizeTotal": 0
  },
  "organizationPlan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "planName": "Free plan",
    "planType": "free",
    "expiredAt": null
  },
  "maigptInboxId": null,
  "canUseCustomElasticsearch": false,
```

```
  "isTrial": true,
  "expirationDays": null
}
```

Add a Member to the Specified Organization

POST `/api/v1/organizations/{organizationPk}/members/`

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this Organization ID

Request Body

Request Parameters

Field	Type	Required	Description
email	string (email)	Yes	
organization	string (uuid)	No	

Request Schema Example

```
{
  "email": string (email)
```

```
  "organization"?: string (uuid) // Optional
}
```

Example Request Values

```
{
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
    "is_owner": false
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
    "is_owner": false
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
    "is_owner": False
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'organization' => '613c86f7-a45f-4e29-b255-
29caff89de32',
            'is_owner' => false
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "organization": string (uuid) // Organization ID
  "is_owner": boolean // Whether the member is the organization owner
}
```

Example Response Values

```
{
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
}
```

List Organizations

GET `/api/v1/organizations/`

Parameters

Parameter	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`pagination`	X	string	Whether to paginate (true/false)

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "memberCreatedAt": string (timestamp)
      "approvalStatus": string
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "memberCreatedAt": "Response string",
      "approvalStatus": "Response string"
    }
  ]
}
```

Retrieve a specific organization

GET `/api/v1/organizations/{id}`

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this organization.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
  "name": string
  "createdAt": string (timestamp)
  "compactLogo"?: string (uri) // optional
  "fullLogo"?: string (uri) // optional
  "themePrimaryColor"?: // may have different types (optional)
  string // UI theme color used for buttons, links, and other elements
(HEX format, e.g. #0960bd); leave empty for unset, in which case the
frontend falls back to the system brand color (optional)
  "usageStatistics": object
  "organizationPlan": object
  "creditWallet": object // Return credit wallet with ID only.
  "maigptInboxId": string (uuid)
  "canUseCustomElasticsearch": boolean // Allow this organization to
configure custom Elasticsearch endpoints for knowledge bases.
  "canUseVoiceAgent": boolean // Allow this organization to use voice
agent features.
  "canUseMeetingRecords": boolean // Allow this organization to use
meeting records features.
  "canUseCallCenter": boolean // Allow this organization to use call
center features.
  "canUseHookSystem": boolean // Manual override to grant this
organization the hook system regardless of plan. Enterprise-plan
organizations get access automatically without this flag. Layered under
the global Config.hook_system_enabled switch: it must also be enabled.
  "canUseAiGateway": boolean // Allow this organization to call the
OpenAI-compatible `/api/v1/chat/completions` Gateway. On by default for
license installs, otherwise off until a platform admin enables it.
  "canUseAgentTeams": boolean // Read-only projection of the platform-
wide Agent Teams switch (Config.enable_agent_teams). Not a per-
organization flag; member-level access is still governed by RBAC.
  "canUseFileLibrary": boolean // Allow this organization to use the file
library. Off by default for gradual rollout.
  "canUseFileOnlineEditing": boolean // Enable online editing in the file
library (P7): the text editor and upload-new-version entries. Requires
can_use_file_library to also be on.
  "canUseOfficeOnlineEditing": boolean // Enable Office (xlsx/docx/pptx)
online editing in the file library (P7 S9). Requires
can_use_file_online_editing to also be on.
  "enableAgentMemory": boolean // Enable persistent cross-conversation
memory for agents in this organization. When off, agents use only in-
session memory. On by default on SaaS; license installs remain opt-in.
Members can still turn it off for themselves in Profile > AI Memory.
  "hideChinaModels"?: boolean // When on, platform models whose developer
is based in China (including ones added later) are not visible or usable
in this organization: model menus, assistants, the API and AI Gateway all
```

```
exclude them. (optional)
  "gatewayServiceFeePercent": string (decimal) // Markup applied on top
of the token price when billing MaiAgent-managed models called through
the AI Gateway (/v1/chat/completions and /v1/embeddings). Managed gateway
markup only; the organization's own models (BYOK / self-hosted) are
billed via the AI Gateway traffic fee percent instead.
  "gatewayTrafficFeePercent": string (decimal) // Pass-through fee for
calls that run on this organization's own AI Gateway models (BYOK / self-
hosted): charged as the call's total tokens (input, output, and prompt-
cache) x this percent in credits, with no LLM token pricing. Adjustable
per organization; changes apply to new calls only.
  "platformProcessingFeePercent": string (decimal) // Surcharge on every
credit consumption of this organization, billed as a separate "Platform
Processing Fee" line equal to the consumed credits x this percent. 0
disables the fee. Applies to new consumption only; the percent is shown
read-only on the customer pricing page.
  "platformProcessingFeeEnabled": boolean // Whether new consumption of
this organization accrues the platform processing fee. The percent is
exposed read-only as platform_processing_fee_percent.
  "isTrial": boolean // Whether the organization is in a trial period.
Logic: the organization has exactly one trial plan and it is currently
active
  "expirationDays": integer // Number of days expired. Positive = days
expired, null = not yet expired or permanent plan
  "showTrialBanner": boolean // Controls whether the frontend shows the
trial/free plan banner
  "noActivePlanDays": integer // Consecutive days without an active paid
plan. null when cleanup is disabled.
  "cleanupRemainingDays": integer // Days remaining before organization
data is permanently deleted. null when cleanup is disabled, org has an
active paid plan, or no_active_plan_days is 0.
  "estimatedCleanupDate": string // ISO date when organization data will
be permanently deleted. null when cleanup_remaining_days is null.
  "approvalStatus":  // Trial-eligibility review status. Only approved
organizations can receive a trial plan when organization approval is
enabled. Paid plan activation auto-approves the organization.

* `pending` - Pending Review
* `approved` - Approved
* `rejected` - Rejected
  {
  }
  "companyName"?: string // Applicant-reported company name collected at
organization creation for approval review. (optional)
  "taxId"?: // may have different types (optional)
```

```
  string // Applicant-reported business tax ID (8 digits, optional).
Stored as-is; not verified. (optional)
  "contactPhone"?: string // Applicant-reported contact phone collected
at organization creation for approval review. (optional)
  "applicationMetadata"?: object // Extra applicant-reported key-value
fields collected at organization creation. Lets the frontend collect
additional review fields without a schema migration; the core review
fields (company name, tax ID, contact phone) stay as named columns.
(optional)
  "invitationCode"?: string // optional
}
```

Response Example Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "createdAt": "Response string",
  "compactLogo": "https://example.com/file.jpg",
  "fullLogo": "https://example.com/file.jpg",
  "themePrimaryColor": "Response string",
  "usageStatistics": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "organizationPlan": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "creditWallet": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "maigptInboxId": "550e8400-e29b-41d4-a716-446655440000",
  "canUseCustomElasticsearch": false,
  "canUseVoiceAgent": false,
  "canUseMeetingRecords": false,
  "canUseCallCenter": false,
  "canUseHookSystem": false,
  "canUseAiGateway": false,
  "canUseAgentTeams": false,
```

```
  "canUseFileLibrary": false,
  "canUseFileOnlineEditing": false,
  "canUseOfficeOnlineEditing": false,
  "enableAgentMemory": false,
  "hideChinaModels": false,
  "gatewayServiceFeePercent": "Response string",
  "gatewayTrafficFeePercent": "Response string",
  "platformProcessingFeePercent": "Response string",
  "platformProcessingFeeEnabled": false,
  "isTrial": false,
  "expirationDays": 456,
  "showTrialBanner": false,
  "noActivePlanDays": 456,
  "cleanupRemainingDays": 456,
  "estimatedCleanupDate": "Response string",
  "approvalStatus": {},
  "companyName": "Response string",
  "taxId": "Response string",
  "contactPhone": "Response string",
  "applicationMetadata": null,
  "invitationCode": "Response string"
}
```


Get Current User Details

GET `/api/v1/users/current/`

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/users/current/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/users/current/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/users/current/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/users/current/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "avatar"?: string (uri) // Optional
  "originalAvatar"?: string (uri) // The original avatar image uploaded
```

```
by the user (long edge capped at 2048px). Every edit re-crops from this
image to avoid cumulative distortion from cropping an already-cropped
result. Legacy data from before this feature launched is empty and is not
backfilled (Optional)
  "avatarCropBox"?: object // Position and size of the last crop box on
the original image (x/y/width/height, with optional
rotate/scale_x/scale_y); coordinates are relative to the stored original
image (Optional)
  "name": string
  "email": string (email)
  "authSource":
  {
    "id": string (uuid)
    "name": string
  }
  "company"?: string // Optional
  "invitationCode"?: // Multiple types possible (Optional)
  string // Optional
  "apiKeys": [ // Get the list of the user's API keys in the specified
organization.

Flow:
1. Get the current organization instance from the request
2. If there is no organization instance (usually during third-party SSO
login), use the user's auth source
   to look up that auth source's default organization
(is_auth_source_default_organization=True)

Note:
<ul>
  <li>During third-party SSO login, the request does not contain
organization_instance</li>
  <li>The organization must be located via the auth_source and
is_auth_source_default_organization flags</li>
</ul>
    object
  ]
  "permissions": object // Get the user's permissions in the current
organization (nested parent-child structure).
  "resourcePermissions": object // Get the user's resource creation
permissions in the current organization.
  "isOwner": boolean // Determine whether the user is an Owner in the
current organization.
  "maigptCapabilities": object // The member's effective built-in feature
capabilities in MaiGPT (resolved from role permissions), used by the
```

```
frontend to show/hide features.

When there is no member context (not bound to an organization), the
resolved result is fully open, consistent with the resolver's "missing
data does not lock out" principle;
hiding in the frontend is only a UX layer, and the authoritative check is
the gate in the reply path.
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "avatar": "https://example.com/file.jpg",
  "originalAvatar": "https://example.com/file.jpg",
  "avatarCropBox": null,
  "name": "Response string",
  "email": "response@example.com",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "company": "Response string",
  "invitationCode": "Response string",
  "apiKeys": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "permissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "resourcePermissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "isOwner": false,
  "maigptCapabilities": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Example response name",
    "description": "Example response description"
  }
}
```


Get Current User Permissions

GET `/api/v1/permissions/`

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/permissions/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/permissions/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/permissions/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/permissions/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
[
  {
    "id": string (uuid)
    "name": string
```

```
    "value": string
    "description": string
    "children": [
      object
    ]
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "value": "Response string",
    "description": "Response string",
    "children": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    ]
  }
]
```

List Members of an Organization

GET `/api/v1/organizations/{organizationPk}/members/`

Parameters

Parameter	Required	Type	Description

`organizationPk`	V	string	A UUID string identifying this organization ID
`group`	X	string	
`ids`	X	string	
`inbox`	X	string	
`isActive`	X	boolean	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`pagination`	X	string	Whether to paginate (true/false)
`query`	X	string	
`search`	X	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-446655440000&isActive=true&page=1&pageSize=1&pagination=example&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "email": string
      "isActive": boolean
      "isOwner": boolean // Checks whether the member is an organization
owner (via the OWNER group)

Uses a cache to avoid duplicate queries within the same request
Cache lifetime: valid for as long as the member instance exists

Prefers the annotated result (if available) to avoid duplicate queries
      "permissions": [ // Compute the permission tree once per
(organization, custom-group set) per request.

The tree depends only on the member's organization (default groups,
feature
flags, plan whitelist), the member's custom groups, and the global
``Config``,
so members sharing a group set get the same result. A large role's
``pagination=false`` listing otherwise recomputes it per row — one Redis
``Config`` fetch plus the permission walk each time (issue #8436; the
largest
prod role has 3,837 members and a single group set). The memo lives in
the
serializer context, so it never outlives the request. Rows without the
eager
loaded ``_custom_groups_cache`` (single-object paths) keep computing
directly.
        object
      ]
      "groups": [
        object
      ]
      "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
      "activeQuota": object // Return the most recent quota summary
(active first), via prefetch to avoid N+1.
      "metadata": object // Organization-scoped member attributes,
```

```
injected into LLM system prompt
      "availabilityStatus":  // Self-declared availability for auto-
assignment. Paused members are skipped by the dispatch engine but can
still be manually assigned or claim conversations. Not touched by SCIM
provisioning.

* `available` - Available
* `paused` - Paused
      {
      }
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example string",
      "email": "Example string",
      "isActive": false,
      "isOwner": false,
      "permissions": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example name",
          "description": "Example description"
        }
      ],
      "groups": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example name",
          "description": "Example description"
        }
      ],
      "creditQuotaRequired": false,
      "activeQuota": {
```

```
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example name",
        "description": "Example description"
      },
      "metadata": null,
      "availabilityStatus": {},
      "createdAt": "Example string"
    }
  ]
}
```


Retrieve Member Details

GET `/api/v1/organizations/{organizationPk}/members/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this member.
`organizationPk`	V	string	A UUID string identifying this organization ID

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
  "name": string
  "email": string
  "isActive": boolean
  "isOwner": boolean // Checks whether the member is an organization
owner (via the OWNER group)

Uses a cache to avoid duplicate queries within the same request
Cache lifetime: valid for as long as the member instance exists

Prefers the annotated result (if available) to avoid duplicate queries
  "permissions": [ // Compute the permission tree once per (organization,
custom-group set) per request.

The tree depends only on the member's organization (default groups,
feature
flags, plan whitelist), the member's custom groups, and the global
``Config``,
so members sharing a group set get the same result. A large role's
``pagination=false`` listing otherwise recomputes it per row — one Redis
``Config`` fetch plus the permission walk each time (issue #8436; the
largest
prod role has 3,837 members and a single group set). The memo lives in
the
serializer context, so it never outlives the request. Rows without the
eager
loaded ``_custom_groups_cache`` (single-object paths) keep computing
directly.
    object
  ]
  "groups": [
    object
  ]
  "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
  "activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
  "metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
  "availabilityStatus":  // Self-declared availability for auto-
assignment. Paused members are skipped by the dispatch engine but can
still be manually assigned or claim conversations. Not touched by SCIM
provisioning.

* `available` - Available
* `paused` - Paused
```

```
  {
  }
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example string",
  "email": "Example string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example name",
      "description": "Example description"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example name",
      "description": "Example description"
    }
  ],
  "creditQuotaRequired": false,
  "activeQuota": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example name",
    "description": "Example description"
  },
  "metadata": null,
  "availabilityStatus": {},
  "createdAt": "Example string"
}
```

Update Organization Information

PUT `/api/v1/organizations/{id}`

Parameters

Parameter	Required	Type	Description
`id`	V	string	A UUID string identifying this organization.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
compactLogo	string (uri)	No	
fullLogo	string (uri)	No	
themePrimaryColor	object	No	UI theme color used for elements such as buttons and links (HEX format, e.g. #0960bd); leave empty to indicate it is not set, in which case the frontend falls back to the system brand primary color
hideChinaModels	boolean	No	When on, platform models whose developer is based in China (including ones added later) are not visible or usable in this organization: model menus, assistants, the API and AI Gateway all exclude them...

Field	Type	Required	Description
companyName	string	No	Applicant-reported company name collected at organization creation for approval review.
taxId	object	No	Applicant-reported business tax ID (8 digits, optional). Stored as-is; not verified.
contactPhone	string	No	Applicant-reported contact phone collected at organization creation for approval review.
applicationMetadata	object	No	Extra applicant-reported key-value fields collected at organization creation. Lets the frontend collect additional review fields without a schema migration; the core review fields (company name, tax I...
invitationCode	string	No	

Request Schema Example

```
{
  "name": string
  "compactLogo"?: string (uri) // Optional
  "fullLogo"?: string (uri) // Optional
  "themePrimaryColor"?: // May have different types (Optional)
  string // UI theme color used for elements such as buttons and links (HEX format, e.g. #0960bd); leave empty to indicate it is not set, in which case the frontend falls back to the system brand primary color (Optional)
  "hideChinaModels"?: boolean // When on, platform models whose developer is based in China (including ones added later) are not visible or usable in this organization: model menus, assistants, the API and AI Gateway all exclude them. (Optional)
  "companyName"?: string // Applicant-reported company name collected at organization creation for approval review. (Optional)
  "taxId"?: // May have different types (Optional)
  string // Applicant-reported business tax ID (8 digits, optional). Stored as-is; not verified. (Optional)
  "contactPhone"?: string // Applicant-reported contact phone collected at organization creation for approval review. (Optional)
```

```
  "applicationMetadata"?: object // Extra applicant-reported key-value
fields collected at organization creation. Lets the frontend collect
additional review fields without a schema migration; the core review
fields (company name, tax ID, contact phone) stay as named columns.
(Optional)
  "invitationCode"?: string // Optional
}
```

Example Request Values


```
{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "countable_chatbots_count": 0,
    "exempt_chatbots_by_mode": {},
    "allowed_chatbot_reply_modes": [],
    "can_create_chatbot": true,
    "create_chatbot_limit": 1,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free Plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
}
```


Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
    "name": "Updated Org Name",
    "created_at": "1748336288000",
    "usage_statistics": {
      "chatbots_count": 0,
      "countable_chatbots_count": 0,
      "exempt_chatbots_by_mode": {},
      "allowed_chatbot_reply_modes": [],
      "can_create_chatbot": true,
      "create_chatbot_limit": 1,
      "current_month_words_count_total": 1000000,
      "current_month_used_words_count_total": 0,
      "current_month_used_conversations_count_total": 0,
      "available_upload_files_size_total": 104857600,
      "used_upload_file_size_total": 0
    },
    "organization_plan": {
      "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
      "plan_name": "Free Plan",
      "expired_at": null
    },
    "is_trial": false,
    "expiration_days": null
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
    "name": "Updated Org Name",
    "created_at": "1748336288000",
    "usage_statistics": {
      "chatbots_count": 0,
      "countable_chatbots_count": 0,
      "exempt_chatbots_by_mode": {},
      "allowed_chatbot_reply_modes": [],
      "can_create_chatbot": true,
      "create_chatbot_limit": 1,
      "current_month_words_count_total": 1000000,
      "current_month_used_words_count_total": 0,
      "current_month_used_conversations_count_total": 0,
      "available_upload_files_size_total": 104857600,
      "used_upload_file_size_total": 0
    },
    "organization_plan": {
      "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
      "plan_name": "Free Plan",
      "expired_at": null
    },
    "is_trial": false,
    "expiration_days": null
  };

axios.put("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
```

```
  console.log('Response received successfully:');
  console.log(response.data);
})
.catch(error => {
  console.error('Request failed:');
  console.error(error.response?.data || error.message);
});
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
    "name": "Updated Org Name",
    "created_at": "1748336288000",
    "usage_statistics": {
        "chatbots_count": 0,
        "countable_chatbots_count": 0,
        "exempt_chatbots_by_mode": {},
        "allowed_chatbot_reply_modes": [],
        "can_create_chatbot": True,
        "create_chatbot_limit": 1,
        "current_month_words_count_total": 1000000,
        "current_month_used_words_count_total": 0,
        "current_month_used_conversations_count_total": 0,
        "available_upload_files_size_total": 104857600,
        "used_upload_file_size_total": 0
    },
    "organization_plan": {
        "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
        "plan_name": "Free Plan",
        "expired_at": None
    },
    "is_trial": False,
    "expiration_days": None
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
```

```
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'id' => '5b01e0b9-0b0e-4079-8150-d12015576d2c',
            'name' => 'Updated Org Name',
            'created_at' => '1748336288000',
            'usage_statistics' => [
                'chatbots_count' => 0,
                'countable_chatbots_count' => 0,
                'exempt_chatbots_by_mode' => new \stdClass(),
                'allowed_chatbot_reply_modes' => [],
                'can_create_chatbot' => true,
                'create_chatbot_limit' => 1,
                'current_month_words_count_total' => 1000000,
                'current_month_used_words_count_total' => 0,
                'current_month_used_conversations_count_total' => 0,
                'available_upload_files_size_total' => 104857600,
                'used_upload_file_size_total' => 0
            ],
            'organization_plan' => [
                'id' => 'cac825e8-cb9b-4a16-b440-4ff6b2e73911',
                'plan_name' => 'Free Plan',
                'expired_at' => null
            ],
            'is_trial' => false,
            'expiration_days' => null
        ]
    ]);
```

```
    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid) // Organization ID
  "name": string // Organization name
  "created_at": string (timestamp) // Creation timestamp
  "usage_statistics": { // Usage statistics
  {
    "chatbots_count"?: integer // Number of chatbots (Optional)
    "countable_chatbots_count"?: integer // Number of chatbots that count
toward the creation quota (modes not in the plan allowlist are excluded)
(Optional)
    "exempt_chatbots_by_mode"?: object // Number of chatbots exempt from
the quota, grouped by the mode that exempts them (Optional)
    "allowed_chatbot_reply_modes"?: [ // Allowlist of reply modes the
plan permits creating (an empty array means no restriction) (Optional)
      string
    ]
    "can_create_chatbot"?: boolean // Whether a chatbot can be created
(Optional)
    "create_chatbot_limit"?: integer // Maximum number of chatbots that
can be created (-1 means unlimited, 0 means creation is not allowed)
(Optional)
    "current_month_words_count_total"?: integer // Total word limit for
this month (Optional)
```

```
    "current_month_used_words_count_total"?: integer // Words used this
month (Optional)
    "current_month_used_conversations_count_total"?: integer //
Conversations used this month (Optional)
    "available_upload_files_size_total"?: integer // Total size of files
that can be uploaded (bytes) (Optional)
    "used_upload_file_size_total"?: integer // Size of uploaded files
used (bytes) (Optional)
  }
  }
  "organization_plan": { // Organization plan information
  {
    "id"?: string (uuid) // Plan ID (Optional)
    "plan_name"?: string // Plan name (Optional)
    "expired_at"?: string (date-time) // Expiration time (Optional)
  }
  }
  "is_trial"?: boolean // Whether the organization is in a trial period.
Logic: the organization has exactly one trial plan and it is currently
active (Optional)
  "expiration_days"?: integer // Number of days since expiration.
Positive = days expired, null = not yet expired or a permanent plan
(Optional)
}
```

Example Response Values


```
{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "countable_chatbots_count": 0,
    "exempt_chatbots_by_mode": {},
    "allowed_chatbot_reply_modes": [],
    "can_create_chatbot": true,
    "create_chatbot_limit": 1,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
```

```
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free Plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
}
```


Update Current User Details

PUT `/api/v1/users/current/`

Request Body

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
originalAvatar	string (uri)	No	Original avatar image uploaded by the user (long edge capped at 2048px). Each edit re-crops from this image to avoid cumulative distortion from cropping an already-cropped result. Legacy data from before this feature launched is empty and is not backfilled
avatarCropBox	object	No	Position and size of the last crop box on the original image (x/y/width/height, with optional rotate/scale_x/scale_y); coordinates are relative to the stored original image
name	string	Yes	

Field	Type	Required	Description
email	string (email)	Yes	
company	string	No	
invitationCode	object	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "originalAvatar"?: string (uri) // Original avatar uploaded by the user (long edge limited to 2048px). Every edit re-crops from this image to avoid cumulative distortion from cropping an already-cropped result. Legacy data created before this feature launched is empty and is not backfilled (Optional)
  "avatarCropBox"?: object // Position and size of the last crop box on the original image (x/y/width/height, with optional rotate/scale_x/scale_y); coordinates are relative to the stored original image (Optional)
  "name": string
  "email": string (email)
  "company"?: string // Optional
  "invitationCode"?: // Can be of different types (Optional)
  string // Optional
}
```

Example Request Values

```
{
  "avatar": "https://example.com/file.jpg",
  "originalAvatar": "https://example.com/file.jpg",
  "avatarCropBox": null,
  "name": "Example name",
  "email": "user@example.com",
  "company": "Example string",
  "invitationCode": null
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/users/current/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Add form fields

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.put("https://api.maiagent.ai/api/v1/users/current/", formData,
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/users/current/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example.pdf', open('example.pdf', 'rb'),
'application/pdf')
}

response = requests.put(url, files=files, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/users/current/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example.pdf', 'r'),
                'filename' => 'example.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "avatar"?: string (uri) // Optional
  "originalAvatar"?: string (uri) // Original avatar uploaded by the user
(long edge limited to 2048px). Every edit re-crops from this image to
avoid cumulative distortion from cropping an already-cropped result.
Legacy data created before this feature launched is empty and is not
backfilled (Optional)
  "avatarCropBox"?: object // Position and size of the last crop box on
the original image (x/y/width/height, with optional
rotate/scale_x/scale_y); coordinates are relative to the stored original
image (Optional)
  "name": string
  "email": string (email)
  "authSource":
  {
    "id": string (uuid)
    "name": string
  }
  "company"?: string // Optional
  "invitationCode"?: // Can be of different types (Optional)
  string // Optional
  "apiKeys": [ // Get the list of the user's API keys in the specified
organization.

Flow:
1. Get the current organization instance from the request
2. If there is no organization instance (usually during third-party SSO
login), use the user's authentication source
   to look up that authentication source's default organization
(is_auth_source_default_organization=True)

Note:
<ul>
  <li>The request does not contain organization_instance during third-
party SSO login</li>
  <li>The organization must be located via the auth_source and
is_auth_source_default_organization flags</li>
</ul>
    object
  ]
  "permissions": object // Get the user's permissions in the current
organization (nested parent-child structure).
```

```
  "resourcePermissions": object // Get the user's resource creation
permissions in the current organization.
  "isOwner": boolean // Determine whether the user is an Owner in the
current organization.
  "maigptCapabilities": object // Effective built-in feature capabilities
of the member in MaiGPT (role permission resolution result), used by the
frontend to show/hide features.

When there is no member context (not bound to an organization), the
resolved result is all enabled, consistent with the resolver's "do not
lock out on missing data"
behavior; hiding in the frontend is only a UX layer, and the
authoritative check is the gate on the reply path.
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "avatar": "https://example.com/file.jpg",
  "originalAvatar": "https://example.com/file.jpg",
  "avatarCropBox": null,
  "name": "Response string",
  "email": "response@example.com",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "company": "Response string",
  "invitationCode": "Response string",
  "apiKeys": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "permissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "resourcePermissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Example response name",
    "description": "Example response description"
  },
  "isOwner": false,
  "maigptCapabilities": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
```


Update Member Roles

POST `/api/v1/organizations/{organizationPk}/members/{id}/update-groups/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this member.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Request Body

Request Parameters

Field	Type	Required	Description
groups	array[string]	Yes	

Request Schema Example

```
{
  "groups": [
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "groups": [
    "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
    "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/update-groups/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "groups": [
      "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
      "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
    ]
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "groups": [
      "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
      "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/update-groups/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/update-groups/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "groups": [
        "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
        "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/update-groups/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'groups' => [
                '4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61',
                '5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "email": string
  "isActive": boolean
  "isOwner": boolean // Check whether the member is an organization owner
(via the OWNER Group)

Use a cache to avoid duplicate queries within the same request
Cache lifetime: valid for as long as the member instance exists

Prefer the annotation result (if available) to avoid duplicate queries
  "permissions": [ // Compute the permission tree once per (organization,
custom-group set) per request.

The tree depends only on the member's organization (default groups,
feature
flags, plan whitelist), the member's custom groups, and the global
``Config``,
so members sharing a group set get the same result. A large role's
``pagination=false`` listing otherwise recomputes it per row — one Redis
``Config`` fetch plus the permission walk each time (issue #8436; the
largest
prod role has 3,837 members and a single group set). The memo lives in
the
serializer context, so it never outlives the request. Rows without the
eager
loaded ``_custom_groups_cache`` (single-object paths) keep computing
directly.
    object
  ]
  "groups": [
    object
  ]
  "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
  "activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
  "metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
  "availabilityStatus":  // Self-declared availability for auto-
```

```
assignment. Paused members are skipped by the dispatch engine but can
still be manually assigned or claim conversations. Not touched by SCIM
provisioning.

* `available` - Available
* `paused` - Paused
  {
  }
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "creditQuotaRequired": false,
  "activeQuota": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "availabilityStatus": {},
  "createdAt": "Response string"
}
```

Status Code: 400

Response Schema Example

```
object
```

List Organization API Keys

`GET` `/api/v1/organizations/{organizationPk}/api-keys/`

API key management from the administrator's perspective; requires the "API Key management permission". A member can hold multiple valid keys at the same time, so you can issue a new key first and revoke the old one after deployment. See API Authentication for how to obtain keys.

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this Organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`source`	X	string	
`status`	X	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/api-keys/?
page=1&pageSize=1&query=example&source=example&status=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/api-keys/?
page=1&pageSize=1&query=example&source=example&status=example",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/?
page=1&pageSize=1&query=example&source=example&status=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/?
page=1&pageSize=1&query=example&source=example&status=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string
      "name"?: string // A free-form name for the API key. Need not be
unique. 50 characters max. (Optional)
      "prefix": string
      "status": string
      "source": string
      "isActive"?: boolean // Reversible enable switch. A disabled key is
rejected exactly like an unknown key (generic 401, no information leak),
but keeps its secret and usage history and can be re-enabled at any time
— unlike the irreversible revoked flag. (Optional)
      "rateLimitRpm"?: integer // Max requests per minute across every
platform API this key can call. Null = unlimited. (Optional)
      "rateLimitTpm"?: integer // Max LLM tokens per minute through the
AI Gateway inference endpoints. Null = unlimited. (Optional)
      "gatewayAllowedLlmCount": integer // Size of the gateway allow-
list, when the queryset annotated it.

0 means "no restriction" (see ``api_key_allowed_llm_ids``). ``None``
on surfaces that did not annotate (login payload, PAT list) so a
missing count is never mistaken for "unrestricted".
      "expiryDate"?: string (timestamp) // Once API key expires, clients
cannot use it anymore. (Optional)
      "createdAt": string (timestamp)
      "lastUsedAt"?: string (timestamp) // Optional
      "member": object
      "createdByName": string
      "revokedAt"?: string (timestamp) // Optional
      "revokedByName": string
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
```

```
      "id": "Response string",
      "name": "Response string",
      "prefix": "Response string",
      "status": "Response string",
      "source": "Response string",
      "isActive": false,
      "rateLimitRpm": 456,
      "rateLimitTpm": 456,
      "gatewayAllowedLlmCount": 456,
      "expiryDate": "Response string",
      "createdAt": "Response string",
      "lastUsedAt": "Response string",
      "member": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "createdByName": "Response string",
      "revokedAt": "Response string",
      "revokedByName": "Response string"
    }
  ]
}
```


Status Code: 403 - Insufficient permissions

Issue an API Key for a Member

POST `/api/v1/organizations/{organizationPk}/api-keys/`

Parameters

Parameter Name	Required	Type	Description

`organizationPk`	V	string	A UUID string identifying this Organization ID

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
expiryDate	string (timestamp)	No	
memberId	string (uuid)	Yes	
isActive	boolean	No	

Request Schema Example

```
{
  "name": string
  "expiryDate"?: string (timestamp) // Optional
  "memberId": string (uuid)
  "isActive"?: boolean // Optional
}
```

Example Request Values

```
{
  "name": "My API Key",
  "member_id": "00000000-0000-0000-0000-000000000000",
  "expiry_date": "2027-12-31T23:59:59Z"
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/api-keys/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "My API Key",
    "member_id": "00000000-0000-0000-0000-000000000000",
    "expiry_date": "2027-12-31T23:59:59Z"
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "My API Key",
    "member_id": "00000000-0000-0000-0000-000000000000",
    "expiry_date": "2027-12-31T23:59:59Z"
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/api-keys/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "My API Key",
    "member_id": "00000000-0000-0000-0000-000000000000",
    "expiry_date": "2027-12-31T23:59:59Z"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'My API Key',
            'member_id' => '00000000-0000-0000-0000-000000000000',
            'expiry_date' => '2027-12-31T23:59:59Z'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 201

Example Response Structure

```
{
  "id": string
  "name"?: string // A free-form name for the API key. Need not be
unique. 50 characters max. (Optional)
  "prefix": string
  "key": string // The full API key. Shown only in this response and
never stored, so it
cannot be retrieved again.
  "status": string
  "source": string
  "isActive"?: boolean // Reversible enable switch. A disabled key is
rejected exactly like an unknown key (generic 401, no information leak),
but keeps its secret and usage history and can be re-enabled at any time
— unlike the irreversible revoked flag. (Optional)
  "expiryDate"?: string (timestamp) // Once API key expires, clients
cannot use it anymore. (Optional)
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "Response string",
  "name": "Response string",
  "prefix": "Response string",
  "key": "Response string",
  "status": "Response string",
  "source": "Response string",
  "isActive": false,
  "expiryDate": "Response string",
  "createdAt": "Response string"
}
```

Status Code: `400` - Validation error (e.g., blank name, expiry date in the past)

Status Code: `403` - Insufficient permissions

Get a Specific API Key

GET `/api/v1/organizations/{organizationPk}/api-keys/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
{
  "id": string
```

```
  "name"?: string // A free-form name for the API key. Need not be
unique. 50 characters max. (Optional)
  "prefix": string
  "status": string
  "source": string
  "isActive"?: boolean // Reversible enable switch. A disabled key is
rejected exactly like an unknown key (generic 401, no information leak),
but keeps its secret and usage history and can be re-enabled at any time
— unlike the irreversible revoked flag. (Optional)
  "rateLimitRpm"?: integer // Max requests per minute across every
platform API this key can call. Null = unlimited. (Optional)
  "rateLimitTpm"?: integer // Max LLM tokens per minute through the AI
Gateway inference endpoints. Null = unlimited. (Optional)
  "gatewayAllowedLlmCount": integer // Size of the gateway allow-list,
when the queryset annotated it.

0 means "no restriction" (see ``api_key_allowed_llm_ids``). ``None``
on surfaces that did not annotate (login payload, PAT list) so a
missing count is never mistaken for "unrestricted".
  "expiryDate"?: string (timestamp) // Once API key expires, clients
cannot use it anymore. (Optional)
  "createdAt": string (timestamp)
  "lastUsedAt"?: string (timestamp) // Optional
  "member": object
  "createdByName": string
  "revokedAt"?: string (timestamp) // Optional
  "revokedByName": string
  "gatewayAllowedLlmIds": [
    string (uuid)
  ]
}
```

Example Response Values

```
{
  "id": "Response string",
  "name": "Response string",
  "prefix": "Response string",
  "status": "Response string",
  "source": "Response string",
  "isActive": false,
  "rateLimitRpm": 456,
  "rateLimitTpm": 456,
  "gatewayAllowedLlmCount": 456,
```

```
  "expiryDate": "Response string",
  "createdAt": "Response string",
  "lastUsedAt": "Response string",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "createdByName": "Response string",
  "revokedAt": "Response string",
  "revokedByName": "Response string",
  "gatewayAllowedLlmIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```


Update API Key Settings

PATCH `/api/v1/organizations/{organizationPk}/api-keys/{id}`

You can adjust the enabled status, requests per minute (RPM), and expiry time. Disabling is reversible; use DELETE to revoke.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	
`organizationPk`	V	string	A UUID string identifying this Organization ID

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	A free-form name for the API key. Need not be unique. 50 characters max.
isActive	boolean	No	Reversible enable switch. A disabled key is rejected exactly like an unknown key (generic 401, no information leak), but keeps its secret and usage history and can be re-enabled at any time — unlike t...
rateLimitRpm	integer	No	Max requests per minute across every platform API this key can call. Null = unlimited.
rateLimitTpm	integer	No	Max LLM tokens per minute through the AI Gateway inference endpoints. Null = unlimited.
expiryDate	string (timestamp)	No	Once API key expires, clients cannot use it anymore.
lastUsedAt	string (timestamp)	No	
revokedAt	string (timestamp)	No	

Request Schema Example

```
{
  "name"?: string // A free-form name for the API key. Need not be
unique. 50 characters max. (Optional)
  "isActive"?: boolean // Reversible enable switch. A disabled key is
rejected exactly like an unknown key (generic 401, no information leak),
but keeps its secret and usage history and can be re-enabled at any time
— unlike the irreversible revoked flag. (Optional)
  "rateLimitRpm"?: integer // Max requests per minute across every
platform API this key can call. Null = unlimited. (Optional)
  "rateLimitTpm"?: integer // Max LLM tokens per minute through the AI
```

```
Gateway inference endpoints. Null = unlimited. (Optional)
  "expiryDate"?: string (timestamp) // Once API key expires, clients
cannot use it anymore. (Optional)
  "lastUsedAt"?: string (timestamp) // Optional
  "revokedAt"?: string (timestamp) // Optional
}
```

Example Request Values

```
{
  "name": "Example name",
  "isActive": true,
  "rateLimitRpm": 123,
  "rateLimitTpm": 123,
  "expiryDate": "Example string",
  "lastUsedAt": "Example string",
  "revokedAt": "Example string"
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X PATCH
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/api-keys/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "isActive": true,
    "rateLimitRpm": 123,
    "rateLimitTpm": 123,
    "expiryDate": "Example string",
    "lastUsedAt": "Example string",
    "revokedAt": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "Example name",
    "isActive": true,
    "rateLimitRpm": 123,
    "rateLimitTpm": 123,
    "expiryDate": "Example string",
    "lastUsedAt": "Example string",
    "revokedAt": "Example string"
  };

axios.patch("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example name",
    "isActive": True,
    "rateLimitRpm": 123,
    "rateLimitTpm": 123,
    "expiryDate": "Example string",
    "lastUsedAt": "Example string",
    "revokedAt": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'isActive' => true,
            'rateLimitRpm' => 123,
            'rateLimitTpm' => 123,
            'expiryDate' => 'Example string',
            'lastUsedAt' => 'Example string',
            'revokedAt' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
{
  "id": string
  "name"?: string // A free-form name for the API key. Need not be
unique. 50 characters max. (Optional)
  "prefix": string
  "status": string
  "source": string
  "isActive"?: boolean // Reversible enable switch. A disabled key is
rejected exactly like an unknown key (generic 401, no information leak),
but keeps its secret and usage history and can be re-enabled at any time
— unlike the irreversible revoked flag. (Optional)
  "rateLimitRpm"?: integer // Max requests per minute across every
platform API this key can call. Null = unlimited. (Optional)
  "rateLimitTpm"?: integer // Max LLM tokens per minute through the AI
Gateway inference endpoints. Null = unlimited. (Optional)
  "gatewayAllowedLlmCount": integer // Size of the gateway allow-list,
when the queryset annotated it.

0 means "no restriction" (see ``api_key_allowed_llm_ids``). ``None``
on surfaces that did not annotate (login payload, PAT list) so a
missing count is never mistaken for "unrestricted".
  "expiryDate"?: string (timestamp) // Once API key expires, clients
cannot use it anymore. (Optional)
  "createdAt": string (timestamp)
  "lastUsedAt"?: string (timestamp) // Optional
  "member": object
  "createdByName": string
  "revokedAt"?: string (timestamp) // Optional
  "revokedByName": string
  "gatewayAllowedLlmIds": [
    string (uuid)
  ]
}
```

Example Response Values

```
{
  "id": "Response string",
  "name": "Response string",
  "prefix": "Response string",
  "status": "Response string",
  "source": "Response string",
  "isActive": false,
  "rateLimitRpm": 456,
  "rateLimitTpm": 456,
  "gatewayAllowedLlmCount": 456,
  "expiryDate": "Response string",
  "createdAt": "Response string",
  "lastUsedAt": "Response string",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "createdByName": "Response string",
  "revokedAt": "Response string",
  "revokedByName": "Response string",
  "gatewayAllowedLlmIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```


Revoke API Key

DELETE `/api/v1/organizations/{organizationPk}/api-keys/{id}`

Parameters

Parameter Name	Required	Type	Description

`id`	V	string	
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/api-keys/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/api-keys/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
204	API Key revoked successfully
400	API Key has already been revoked
403	Insufficient permissions

404	API Key not found

Get the Organization an API Key Belongs To

GET `/api/v1/users/api-key-context/`

Accepts only Authorization: Api-Key and returns the organization the key is bound to; it does not count toward the key's RPM quota. For usage, see Common Conventions: Confirm the Organization a Key Belongs To。

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/users/api-key-context/"
  -H "Authorization: Api-Key YOUR_API_KEY"
# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/users/api-key-context/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/users/api-key-context/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/users/api-key-context/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Example Response Structure

```
{
  "organizationId": string (uuid)
}
```

Example Response Values

```
{
  "organizationId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Status Code: 401 - Missing or invalid API Key, or a non-API-Key credential was used

Issue a Member Embed Token

POST `/api/v1/members/embed-tokens/`

Uses an organization API Key to issue a token valid for 15 minutes for a specified member, so your system can embed MaiGPT as that member. Only keys held by the organization owner or by members with the "Organization Overview" management permission can call this endpoint. For the flow, see the technical manual MaiGPT Member Identity Embedding。

Request Body

Request Parameters

Field	Type	Required	Description
email	string (email)	No	
memberId	string (uuid)	No	

Request Schema Example

```
{
  "email"?: string (email) // Optional
  "memberId"?: string (uuid) // Optional
}
```

Example Request Values

```
{
  "email": "user@example.com",
  "memberId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/members/embed-tokens/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "email": "user@example.com",
    "memberId": "550e8400-e29b-41d4-a716-446655440000"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "email": "user@example.com",
    "memberId": "550e8400-e29b-41d4-a716-446655440000"
  };

axios.post("https://api.maiagent.ai/api/v1/members/embed-tokens/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/members/embed-tokens/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "email": "user@example.com",
    "memberId": "550e8400-e29b-41d4-a716-446655440000"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/members/embed-tokens/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'email' => 'user@example.com',
            'memberId' => '550e8400-e29b-41d4-a716-446655440000'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "accessToken": string
  "organizationId": string (uuid)
  "expiresAt": string (timestamp)
  "member": {
  {
    "id": string (uuid)
    "userId": string (uuid)
    "email": string (email)
    "name": string
  }
  }
}
```

Example Response Values

```
{
  "accessToken": "Response string",
  "organizationId": "550e8400-e29b-41d4-a716-446655440000",
  "expiresAt": "Response string",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "userId": "550e8400-e29b-41d4-a716-446655440000",
    "email": "response@example.com",
    "name": "Response string"
  }
}
```

Status Code: `400` - Provide exactly one of email and memberId

Status Code: `401` - Not authenticated with an Api-Key

Status Code: `403` - The API Key holder is neither an organization owner nor has organization management permission

Status Code: `404` - The specified member does not exist, is disabled, or does not belong to this organization

List OAuth Applications

`GET` `/api/v1/oauth/applications/`

When MaiAgent acts as an OAuth 2.1 authorization server, your application registers here first. For the flow, see the technical manual MaiAgent as an OAuth Authorization Server.

Parameters

Parameter Name	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/oauth/applications/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/oauth/applications/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/oauth/applications/?
page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/oauth/applications/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
```

```
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "clientId": string
      "clientType":
      {
      }
      "redirectUris": [
        string
      ]
      "authorizedUserCount": integer
      "createdByName": string
      "created": string (timestamp)
      "updated": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "clientId": "Response string",
      "clientType": {},
      "redirectUris": [
        "Response string"
      ],
      "authorizedUserCount": 456,
      "createdByName": "Response string",
      "created": "Response string",
      "updated": "Response string"
    }
```

```
  ]
}
```

Register OAuth Application

POST `/api/v1/oauth/applications/`

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
clientType	object	No	
redirectUris	array[string]	Yes	

Request Schema Example

```
{
  "name": string
  "clientType"?:  // Optional
  {
  }
  "redirectUris": [
    string
  ]
}
```

Example Request Values

```
{
  "name": "Example name",
  "clientType": {},
  "redirectUris": [
    "Example string"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/oauth/applications/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "clientType": {},
    "redirectUris": [
      "Example string"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "Example name",
    "clientType": {},
    "redirectUris": [
      "Example string"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/oauth/applications/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/oauth/applications/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example name",
    "clientType": {},
    "redirectUris": [
        "Example string"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/oauth/applications/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'clientType' => new \stdClass(),
            'redirectUris' => [
                'Example string'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "clientId": string
  "clientType":
  {
  }
  "redirectUris": [
    string
  ]
  "authorizedUserCount": integer
  "createdByName": string
  "created": string (timestamp)
  "updated": string (timestamp)
  "clientSecret": string
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "clientId": "Response string",
  "clientType": {},
  "redirectUris": [
    "Response string"
  ],
  "authorizedUserCount": 456,
  "createdByName": "Response string",
  "created": "Response string",
  "updated": "Response string",
  "clientSecret": "Response string"
}
```

Retrieve OAuth Application

GET `/api/v1/oauth/applications/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this MaiAgent OAuth application.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/oauth/applications/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/oauth/applications/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/oauth/applications/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/oauth/applications/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
```

```
  "clientId": string
  "clientType":
  {
  }
  "redirectUris": [
    string
  ]
  "authorizedUserCount": integer
  "createdByName": string
  "created": string (timestamp)
  "updated": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "clientId": "Response string",
  "clientType": {},
  "redirectUris": [
    "Response string"
  ],
  "authorizedUserCount": 456,
  "createdByName": "Response string",
  "created": "Response string",
  "updated": "Response string"
}
```

Update OAuth Application

PATCH `/api/v1/oauth/applications/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this MaiAgent OAuth application.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	
redirectUris	array[string]	No	

Request Schema Example

```
{
  "name"?: string // Optional
  "redirectUris"?: [ // Optional
    string
  ]
}
```

Example Request Values

```
{
  "name": "Example name",
  "redirectUris": [
    "Example string"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X PATCH
"https://api.maiagent.ai/api/v1/oauth/applications/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "redirectUris": [
      "Example string"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "name": "Example name",
    "redirectUris": [
      "Example string"
    ]
  };

axios.patch("https://api.maiagent.ai/api/v1/oauth/applications/550e8
400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/oauth/applications/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "name": "Example name",
    "redirectUris": [
        "Example string"
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/oauth/applications/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'redirectUris' => [
                'Example string'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "clientId": string
  "clientType":
  {
  }
  "redirectUris": [
    string
  ]
  "authorizedUserCount": integer
  "createdByName": string
  "created": string (timestamp)
  "updated": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "clientId": "Response string",
  "clientType": {},
  "redirectUris": [
    "Response string"
  ],
  "authorizedUserCount": 456,
  "createdByName": "Response string",
  "created": "Response string",
  "updated": "Response string"
}
```

Delete OAuth Application and Revoke Its Tokens

DELETE `/api/v1/oauth/applications/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this MaiAgent OAuth application.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/oauth/applications/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/oauth/applications/550e
8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/oauth/applications/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/oauth/applications/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Regenerate Client Secret

POST `/api/v1/oauth/applications/{id}/regenerate-secret/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this MaiAgent OAuth application.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X POST
"https://api.maiagent.ai/api/v1/oauth/applications/550e8400-e29b-
41d4-a716-446655440000/regenerate-secret/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/oauth/applications/550e84
00-e29b-41d4-a716-446655440000/regenerate-secret/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/oauth/applications/550e8400-
e29b-41d4-a716-446655440000/regenerate-secret/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/oauth/applications/550e8400-
e29b-41d4-a716-446655440000/regenerate-secret/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
```

```
  "clientId": string
  "clientType":
  {
  }
  "redirectUris": [
    string
  ]
  "authorizedUserCount": integer
  "createdByName": string
  "created": string (timestamp)
  "updated": string (timestamp)
  "clientSecret": string
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "clientId": "Response string",
  "clientType": {},
  "redirectUris": [
    "Response string"
  ],
  "authorizedUserCount": 456,
  "createdByName": "Response string",
  "created": "Response string",
  "updated": "Response string",
  "clientSecret": "Response string"
}
```

List OAuth applications I have authorized

GET `/api/v1/oauth/authorized-applications/`

Parameters

Parameter Name	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/oauth/authorized-
applications/?page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/oauth/authorized-
applications/?page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/oauth/authorized-
applications/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/oauth/authorized-applications/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // optional
```

```
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "organization": string (uuid)
      "organizationName": string
      "tokenCount": integer
      "lastAuthorizedAt": string (timestamp)
      "latestExpiresAt": string (timestamp)
    }
  ]
}
```

Response Example

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "organizationName": "Response string",
      "tokenCount": 456,
      "lastAuthorizedAt": "Response string",
      "latestExpiresAt": "Response string"
    }
  ]
}
```

Revoke my authorization for an application

DELETE `/api/v1/oauth/authorized-applications/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/oauth/authorized-
applications/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/oauth/authorized-
applications/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/oauth/authorized-
applications/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/oauth/authorized-
applications/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Delete organization

DELETE `/api/v1/organizations/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Organization.

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Delete a specified member

`DELETE` `/api/v1/organizations/{organizationPk}/members/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Member.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Get member usage statistics

`GET` `/api/v1/organizations/{organizationPk}/usage-statistics/members/`

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this Organization ID
`chatbot`	X	string	Filter by a specific AI assistant ID
`chatbotId`	X	string	
`endDate`	X	string	End date, format: YYYY-MM-DD
`includeTime`	X	boolean	Whether to return time-series data aggregated by time only, for trend charts (true/false). When set to true, the response contains no entity information, only time fields and statistical values, and is not paginated.
`memberId`	X	string	
`page`	X	integer	Page number
`pageSize`	X	integer	Number of items per page
`search`	X	string	Search by member ID, name, or Email
`sortBy`	X	string	Sort field
`sortOrder`	X	string	Sort order (asc/desc)
`startDate`	V	string	Start date, format: YYYY-MM-DD

timeGranularity	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&memberId=550e8400-
e29b-41d4-a716-
446655440000&page=1&pageSize=1&search=example&sortBy=conversations_c
ount&sortOrder=asc&startDate=example&timeGranularity=day"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&memberId=550e8400-
e29b-41d4-a716-
446655440000&page=1&pageSize=1&search=example&sortBy=conversations_c
ount&sortOrder=asc&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&memberId=550e8400-
e29b-41d4-a716-
446655440000&page=1&pageSize=1&search=example&sortBy=conversations_c
ount&sortOrder=asc&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&memberId=550e8400-
e29b-41d4-a716-
446655440000&page=1&pageSize=1&search=example&sortBy=conversations_c
ount&sortOrder=asc&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "count"?: integer // Result count (optional)
      "results"?: [ // optional
        {
          "member"?: { // optional
          {
            "id"?: string (uuid) // Member ID (optional)
            "name"?: string // Member name (optional)
            "email"?: string // Member email (optional)
          }
          }
          "datetime"?: string (date-time) // Timestamp (only appears when
include_time=true and time_granularity=hour) (optional)
          "period"?: string (date) // Date (only appears when
include_time=true and time_granularity=day/month) (optional)
          "input_characters_count"?: integer // Input character count
(optional)
          "output_characters_count"?: integer // Output character count
(optional)
          "input_token_usage"?: integer // Input token count (optional)
          "output_token_usage"?: integer // Output token count (optional)
          "messages_count"?: integer // Message count (optional)
          "conversations_count"?: integer // Conversation count
(optional)
          "total_characters_count"?: integer // Total character count
(optional)
          "total_token_usage"?: integer // Total token count (optional)
        }
      ]
    }
  ]
}
```

Response Example

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
```

```
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "count": 2,
      "results": [
        {
          "member": {
            "id": "3a245511-d5e0-4e06-9dc6-1106915b9a8a",
            "name": "Wang Xiaoming",
            "email": "wang@example.com"
          },
          "input_characters_count": 10000,
          "output_characters_count": 15000,
          "input_token_usage": 2500,
          "output_token_usage": 3750,
          "messages_count": 100,
          "conversations_count": 50,
          "total_characters_count": 25000,
          "total_token_usage": 6250
        }
      ]
    }
  ]
}
```


Get member usage statistics summary

`GET` `/api/v1/organizations/{organizationPk}/usage-statistics/members/summary/`

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this Organization ID

`endDate`	X	string	End date, format: YYYY-MM-DD
`startDate`	V	string	Start date, format: YYYY-MM-DD
`timeGranularity`	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# Example API Call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/summary/?
endDate=example&startDate=example&timeGranularity=day"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/summary/?
endDate=example&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/members/summary/?
endDate=example&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/members/summary/?
endDate=example&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "total_input_characters"?: integer // Total input characters (Optional)
  "total_output_characters"?: integer // Total output characters
(Optional)
  "total_characters"?: integer // Total characters (Optional)
  "total_input_tokens"?: integer // Total input tokens (Optional)
  "total_output_tokens"?: integer // Total output tokens (Optional)
  "total_tokens"?: integer // Total tokens (Optional)
  "total_messages"?: integer // Total messages (Optional)
  "total_conversations"?: integer // Total conversations (Optional)
  "total_members"?: integer // Total members (Optional)
}
```

Example Response Values

```
{
  "total_input_characters": 456,
  "total_output_characters": 456,
  "total_characters": 456,
  "total_input_tokens": 456,
  "total_output_tokens": 456,
  "total_tokens": 456,
  "total_messages": 456,
  "total_conversations": 456,
  "total_members": 456
}
```

Get AI Assistant Usage Statistics

GET `/api/v1/organizations/{organizationPk}/usage-statistics/chatbots/`

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this Organization ID
`chatbotId`	X	string	
`endDate`	X	string	End date, format: YYYY-MM-DD
`includeTime`	X	boolean	Whether to return time-series data aggregated by time only, for trend charts (true/false). When set to true, the response contains no entity information, only time fields and statistics, and is not paginated.
`page`	X	integer	Page number
`pageSize`	X	integer	Number of results per page
`search`	X	string	Search by AI assistant ID or name
`sortBy`	X	string	Sort field
`sortOrder`	X	string	Sort order (asc/desc)
`startDate`	V	string	Start date, format: YYYY-MM-DD
`timeGranularity`	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/?
chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&page=1&pageSize=1&sear
ch=example&sortBy=conversations_count&sortOrder=asc&startDate=exampl
e&timeGranularity=day"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/?
chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&page=1&pageSize=1&sear
ch=example&sortBy=conversations_count&sortOrder=asc&startDate=exampl
e&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/chatbots/?
chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&page=1&pageSize=1&sear
ch=example&sortBy=conversations_count&sortOrder=asc&startDate=exampl
e&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/chatbots/?
chatbotId=550e8400-e29b-41d4-a716-
446655440000&endDate=example&includeTime=true&page=1&pageSize=1&sear
ch=example&sortBy=conversations_count&sortOrder=asc&startDate=exampl
e&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "count"?: integer // Number of results (Optional)
      "results"?: [ // Optional
        {
          "chatbot"?: { // Optional
          {
            "id"?: string (uuid) // AI assistant ID (Optional)
            "name"?: string // AI assistant name (Optional)
          }
          }
          "datetime"?: string (date-time) // Timestamp (appears only when
include_time=true and time_granularity=hour) (Optional)
          "period"?: string (date) // Date (appears only when
include_time=true and time_granularity=day/month) (Optional)
          "input_characters_count"?: integer // Input characters
(Optional)
          "output_characters_count"?: integer // Output characters
(Optional)
          "input_token_usage"?: integer // Input tokens (Optional)
          "output_token_usage"?: integer // Output tokens (Optional)
          "messages_count"?: integer // Messages (Optional)
          "conversations_count"?: integer // Conversations (Optional)
          "total_characters_count"?: integer // Total characters
(Optional)
          "total_token_usage"?: integer // Total tokens (Optional)
        }
      ]
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
```

```
    {
      "count": 2,
      "results": [
        {
          "chatbot": {
            "id": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
            "name": "Customer Service Assistant"
          },
          "input_characters_count": 10000,
          "output_characters_count": 15000,
          "input_token_usage": 2500,
          "output_token_usage": 3750,
          "messages_count": 100,
          "conversations_count": 50,
          "total_characters_count": 25000,
          "total_token_usage": 6250
        }
      ]
    }
  ]
}
```


Get AI Assistant Usage Statistics Summary

`GET` `/api/v1/organizations/{organizationPk}/usage-statistics/chatbots/summary/`

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this Organization ID
`endDate`	X	string	End date, format: YYYY-MM-DD
`startDate`	V	string	Start date, format: YYYY-MM-DD

`timeGranularity`	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?endDate=example&startDate=example&timeGranularity=day"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?
endDate=example&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/chatbots/summary/?
endDate=example&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/chatbots/summary/?
endDate=example&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "total_input_characters"?: integer // Total input characters (Optional)
  "total_output_characters"?: integer // Total output characters
(Optional)
  "total_characters"?: integer // Total characters (Optional)
  "total_input_tokens"?: integer // Total input tokens (Optional)
  "total_output_tokens"?: integer // Total output tokens (Optional)
  "total_tokens"?: integer // Total tokens (Optional)
  "total_messages"?: integer // Total messages (Optional)
  "total_conversations"?: integer // Total conversations (Optional)
  "total_chatbots"?: integer // Total AI assistants (Optional)
}
```

Example Response Values

```
{
  "total_input_characters": 456,
  "total_output_characters": 456,
  "total_characters": 456,
  "total_input_tokens": 456,
  "total_output_tokens": 456,
  "total_tokens": 456,
  "total_messages": 456,
  "total_conversations": 456,
  "total_chatbots": 456
}
```

List Your Own Memories

GET `/api/v1/me/memories/`

Include the X-Organization-Id header to specify the organization to operate on, even when calling with an API key. You can filter with the status (active
expired) and kind (fact
preference

instruction) query parameters; memories pending confirmation or rejected are not listed. For behavior details, see the technical manual Member Memory API.

Parameters

Parameter Name	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/me/memories/?
page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/me/memories/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/me/memories/?
page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/me/memories/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "topic": string // LLM-assigned topic slug for grouping related
memories (e.g. "language_preferences", "role", "communication_style").
Null means legacy memory without topic grouping.
      "summary": string // One-line summary of the memory topic.
      "details": object // Concrete facts for this topic, stored as a
list of strings.
      "kind":
      {
      }
      "source":  // How this memory was created: user explicitly stated,
agent extracted, or manually added.

* `user_stated` - User Stated
* `agent_extracted` - Agent Extracted
* `agent_stated` - Agent Stated
* `user_manual` - User Manual
      {
      }
      "requestedByUser": boolean // True when the user explicitly asked
the agent to remember this. Only affects the UI label (S12), not LRU
eviction priority (S6).
      "status":
      {
      }
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
      "topic": "Response string",
      "summary": "Response string",
      "details": null,
      "kind": {},
      "source": {},
      "requestedByUser": false,
      "status": {},
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Create or Update Your Own Memory

POST `/api/v1/me/memories/`

Include the X-Organization-Id header to specify the organization to operate on, even when calling with an API key. Request fields: topic (required, up to 100 characters, lowercase letters, digits, and inner underscores only), summary (required, up to 500 characters), details (required, array of strings, up to 20 items with up to 500 characters each), and kind (fact
preference
instruction, defaults to fact). If the same member already has an active memory with the same topic, it is updated and 200 is returned; a newly created memory returns 201.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/me/memories/"
  -H "Authorization: Api-Key YOUR_API_KEY"
  -H "Content-Type: application/json"
  -d '{}'
# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {};

axios.post("https://api.maiagent.ai/api/v1/me/memories/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/me/memories/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/me/memories/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => new \stdClass()
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "topic": string // LLM-assigned topic slug for grouping related
memories (e.g. "language_preferences", "role", "communication_style").
Null means legacy memory without topic grouping.
  "summary": string // One-line summary of the memory topic.
  "details": object // Concrete facts for this topic, stored as a list of
strings.
  "kind":
  {
  }
  "source":  // How this memory was created: user explicitly stated,
agent extracted, or manually added.

* `user_stated` - User Stated
* `agent_extracted` - Agent Extracted
* `agent_stated` - Agent Stated
* `user_manual` - User Manual
  {
  }
  "requestedByUser": boolean // True when the user explicitly asked the
agent to remember this. Only affects the UI label (S12), not LRU eviction
priority (S6).
  "status":
  {
  }
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "topic": "Response string",
  "summary": "Response string",
  "details": null,
  "kind": {},
  "source": {},
  "requestedByUser": false,
  "status": {},
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Update or Reactivate Your Own Memory

PATCH `/api/v1/me/memories/{id}`

Include the X-Organization-Id header to specify the organization to operate on, even when calling with an API key. You can send topic, summary, and details (same limits as creation) to modify the content, or send status (active
expired) to deactivate or reactivate the memory; at least one of these fields is required.

Parameters

Parameter Name	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/me/memories/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{}'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {};

axios.patch("https://api.maiagent.ai/api/v1/me/memories/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/me/memories/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/me/memories/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => new \stdClass()
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "topic": string // LLM-assigned topic slug for grouping related
memories (e.g. "language_preferences", "role", "communication_style").
Null means legacy memory without topic grouping.
  "summary": string // One-line summary of the memory topic.
  "details": object // Concrete facts for this topic, stored as a list of
strings.
  "kind":
  {
  }
  "source":  // How this memory was created: user explicitly stated,
agent extracted, or manually added.

* `user_stated` - User Stated
* `agent_extracted` - Agent Extracted
* `agent_stated` - Agent Stated
* `user_manual` - User Manual
  {
  }
  "requestedByUser": boolean // True when the user explicitly asked the
agent to remember this. Only affects the UI label (S12), not LRU eviction
priority (S6).
  "status":
  {
  }
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "topic": "Response string",
  "summary": "Response string",
  "details": null,
  "kind": {},
  "source": {},
  "requestedByUser": false,
  "status": {},
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Delete Your Own Memory

DELETE `/api/v1/me/memories/{id}`

Include the X-Organization-Id header to specify the organization to operate on, even when calling with an API key.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/me/memories/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/me/memories/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/me/memories/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/me/memories/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

API call example (Shell)

目錄

1. List Contacts
2. Create Contact
3. Get Contact Details
4. Update Contact
5. Partially Update Contact
6. Delete Contact
7. Get Contact's Conversation List
8. Get Contact's Latest Conversation
9. Broadcast Message
10. Create API Credentials for Contact
11. Update Contact's API Credentials
12. Delete Contact's API Credentials
13. Create MCP Credentials for Contact
14. Update Contact's MCP Credentials
15. Delete Contact's MCP Credentials
16. Download Contact Bulk Import Template
17. Bulk Import Contacts

API call example (Shell)

List Contacts

`GET` `/api/v1/contacts/`

Parameters

Parameter Name	Required	Type	Description
`channels`	X	array	Multiple values may be separated by commas.
`inbox`	X	string	Deprecated: Use inboxes instead

`inboxes`	X	string	Conversation platform ID (used to filter contacts in a specific inbox)
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	Search by name or source ID
`search`	X	string	Full-text search
`tags`	X	array	Multiple values may be separated by commas.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/?
channels=example&inbox=550e8400-e29b-41d4-a716-
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=exa
mple&tags=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/?
channels=example&inbox=550e8400-e29b-41d4-a716-
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=exa
mple&tags=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/?
channels=example&inbox=550e8400-e29b-41d4-a716-
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=exa
mple&tags=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/contacts/?channels=example&inbox=550e8400-e29b-41d4-a716-446655440000&inboxes=example&page=1&pageSize=1&query=example&search=example&tags=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
```

```
  "results": [
    {
      "id": string (uuid)
      "inboxes"?: [ // optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "inbox"?:  // Backward-compatible field: a single inbox; ignored if
inboxes is also provided (optional)
      {
        "id": string (uuid)
        "name": string
      }
      "name": string
      "avatar"?: // may be of different types (optional)
      string (uri) // Contact avatar URL; supports a full URL or a relative
path (optional)
      "phoneNumber"?: string // optional
      "email"?: // may be of different types (optional)
      string (email) // optional
      "sourceId"?: string // optional
      "queryMetadata"?: { // Query metadata that controls the scope of
knowledge that can be queried; see each endpoint's description (optional)
      {
        "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
          {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
              string (uuid)
            ]
            "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
              string (uuid)
            ]
            "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
              string (uuid)
            ]
            "hasUserSelectedAll"?: boolean // true = select all content of
the knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
          }
```

```
        ]
        "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
        {
          "operator": string (enum: AND, OR) // How the label conditions are
combined
          "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
            object
          ]
        }
        }
      }
      }
      "metadata"?: object // Custom customer information shown in the
customer panel of the conversation page (optional)
      "mcpCredentials": [ // List of the contact's MCP credentials
        object
      ]
      "apiCredentials": [ // List of the contact's API credentials
        object
      ]
      "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
        object
      ]
      "tags": [
        {
          "id": string (uuid)
          "name": string
          "createdAt": string (timestamp)
          "updatedAt": string (timestamp)
        }
      ]
      "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve UUID keys in metadata into display names
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Response Example Values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    [
      {
        "id": "user1_uuid",
        "inboxes": [
          {
            "id": "inbox_uuid",
            "name": "inbox_name"
          }
        ],
        "name": "user1",
        "avatar": "",
        "sourceId": null,
        "queryMetadata": null,
        "createdAt": "1751875361000",
        "updatedAt": "1751875361000"
      },
      {
        "id": "user2_uuid",
        "inboxes": [
          {
            "id": "inbox_uuid",
            "name": "inbox_name"
          }
        ],
        "name": "user2",
        "avatar": "avatar_url",
        "sourceId": "self defined source id",
        "queryMetadata": {
          "knowledgeBases": [
            {
              "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
              "hasUserSelectedAll": true
            }
          ],
          "labelRelations": {
            "operator": "OR",
            "conditions": [
              {
                "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
```

```
              }
            ]
          }
        },
        "createdAt": "1751875400000",
        "updatedAt": "1751875400000"
      }
    ]
  ]
}
```


Create Contact

POST `/api/v1/contacts/`

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/contacts/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "inboxes": [
      {
        "id": "inbox_uuid"
      }
    ],
    "name": "user1",
    "avatar": "",
    "phoneNumber": 912345678,
    "email": "user1@example.com",
    "sourceId": "self defined source id",
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "OR",
        "conditions": [
          {
            "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
          }
        ]
      }
    },
    "metadata": [
      {
        "key": "Customer Tier",
        "value": "VIP"
      },
      {
        "key": "Preferred Language",
        "value": "Traditional Chinese"
```

```
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "inboxes": [
      {
        "id": "inbox_uuid"
      }
    ],
    "name": "user1",
    "avatar": "",
    "phoneNumber": 912345678,
    "email": "user1@example.com",
    "sourceId": "self defined source id",
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "OR",
        "conditions": [
          {
            "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
          }
        ]
      }
    },
    "metadata": [
      {
        "key": "Customer Tier",
        "value": "VIP"
```

```
      },
      {
        "key": "Preferred Language",
        "value": "Traditional Chinese"
      }
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/contacts/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "inbox_uuid"
        }
    ],
    "name": "user1",
    "avatar": "",
    "phoneNumber": 912345678,
    "email": "user1@example.com",
    "sourceId": "self defined source id",
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-
fdeea327a3a2",
                "hasUserSelectedAll": True
            }
        ],
        "labelRelations": {
            "operator": "OR",
            "conditions": [
                {
                    "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
                }
            ]
        }
    },
    "metadata": [
        {
            "key": "Customer Tier",
            "value": "VIP"
        },
```

```
        {
            "key": "Preferred Language",
            "value": "Traditional Chinese"
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/contacts/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'inboxes' => [
                [
                    'id' => 'inbox_uuid'
                ]
            ],
            'name' => 'user1',
            'avatar' => '',
            'phoneNumber' => 912345678,
            'email' => 'user1@example.com',
            'sourceId' => 'self defined source id',
            'queryMetadata' => [
                'knowledgeBases' => [
                    [
                        'knowledgeBaseId' => '5f8c6a99-5515-43aa-8e83-
fdeea327a3a2',
                        'hasUserSelectedAll' => true
                    ]
                ],
                'labelRelations' => [
                    'operator' => 'OR',
                    'conditions' => [
                        [
                            'labelId' => '957998f9-1824-4872-becd-
b7f8a7962896'
                        ]
                    ]
                ]
            ],
            'metadata' => [
```

```
                [
                    'key' => 'Customer Tier',
                    'value' => 'VIP'
                ],
                [
                    'key' => 'Preferred Language',
                    'value' => 'Traditional Chinese'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes is also provided (optional)
  {
    "id": string (uuid)
```

```
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports a full URL or a relative path
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
```

```
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
```

```
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",
  "updatedAt": "1751875361000"
}
```


Get Contact Details

`GET` `/api/v1/contacts/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Contact.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
```

```
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports a full URL or a relative path
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
```

```
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
```

```
        "name": "inbox_name"
      }
    ],
    "name": "user1",
    "avatar": "",
    "phoneNumber": 912345678,
    "email": "user1@example.com",
    "sourceId": null,
    "queryMetadata": null,
    "createdAt": "1751875361000",
    "updatedAt": "1751875361000"
}
```


Update Contact

PUT `/api/v1/contacts/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Contact.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	Contact name
inboxes	array[object]	No	List of conversation platforms (new format)

Field	Type	Required	Description
inbox	object	No	Single conversation platform (legacy format, backward compatible)
inbox.id	string (uuid)	Yes	Conversation platform ID
avatar	string	No	Avatar URL
phoneNumber	string	No	Contact phone number
email	string (email)	No	Contact email address
sourceId	string	No	Source ID
queryMetadata	object	No	Query metadata that controls the scope of knowledge that can be queried. Not provided or null = not set (no restriction); empty object {} = treated as no restriction (select all), not no permission; knowledgeBases empty array or null = nothing can be queried (no permission); labelRelations.conditions empty array = no label filtering (this does not mean no permission). See this endpoint's description for details.
queryMetadata.knowledgeBases	array[object]	No	List of knowledge bases that can be queried; empty array or null = nothing can be queried (no permission); not provided = no restriction
queryMetadata.labelRelations	object	No	Label filter conditions; an empty conditions array = no label filtering (this does not mean no permission). Must be

Field	Type	Required	Description
			provided together with knowledgeBases—when provided alone, knowledge base retrieval does not apply label filtering (equivalent to no restriction)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How the label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
metadata	array[object]	No	List of custom contact attributes

Request Schema Example

```
{
  "name"?: string // Contact name (optional)
  "inboxes"?: [ // List of conversation platforms (new format) (optional)
    {
      "id": string (uuid) // Conversation platform ID
    }
  ]
  "inbox"?: { // Single conversation platform (legacy format, backward
compatible) (optional)
  {
    "id": string (uuid) // Conversation platform ID
  }
  }
  "avatar"?: string // Avatar URL (optional)
  "phoneNumber"?: string // Contact phone number (optional)
  "email"?: string (email) // Contact email address (optional)
  "sourceId"?: string // Source ID (optional)
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried. Not provided or null = not set (no restriction); empty
object {} = treated as no restriction (select all), not no permission;
knowledgeBases empty array or null = nothing can be queried (no permission);
labelRelations.conditions empty array = no label filtering (this does not
mean no permission). See this endpoint's description for details. (optional)
```

```
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: [ // List of custom contact attributes (optional)
    {
      "key": string // Attribute name
      "value": string // Attribute value
      "updatedAt"?: string // Update time (ISO format, optional; filled in
automatically if not provided) (optional)
    }
```

```
  ]
}
```

Request Example Values

```
{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": {
    "knowledgeBases": []
  },
  "metadata": [
    {
      "key": "Customer Tier",
      "value": "VVIP"
    },
    {
      "key": "Preferred Language",
      "value": "Japanese"
    }
  ]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "inboxes": [
      {
        "id": "new_inbox_uuid"
      }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": {
      "knowledgeBases": []
    },
    "metadata": [
      {
        "key": "Customer Tier",
        "value": "VVIP"
      },
      {
        "key": "Preferred Language",
        "value": "Japanese"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "inboxes": [
      {
        "id": "new_inbox_uuid"
      }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": {
      "knowledgeBases": []
    },
    "metadata": [
      {
        "key": "Customer Tier",
        "value": "VVIP"
      },
      {
        "key": "Preferred Language",
        "value": "Japanese"
      }
    ]
  };

axios.put("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
```

```
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "new_inbox_uuid"
        }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": {
        "knowledgeBases": []
    },
    "metadata": [
        {
            "key": "Customer Tier",
            "value": "VVIP"
        },
        {
            "key": "Preferred Language",
            "value": "Japanese"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
```

```
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'inboxes' => [
                [
                    'id' => 'new_inbox_uuid'
                ]
            ],
            'name' => 'updated_user_name',
            'avatar' => 'updated_avatar_url',
            'phoneNumber' => 987654321,
            'email' => 'updated@example.com',
            'sourceId' => 'updated_source_id',
            'queryMetadata' => [
                'knowledgeBases' => []
            ],
            'metadata' => [
                [
                    'key' => 'Customer Tier',
                    'value' => 'VVIP'
                ],
                [
                    'key' => 'Preferred Language',
                    'value' => 'Japanese'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
```

```
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error: ' . $e->getMessage();
    }
    ?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports a full URL or a relative path
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
```

```
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
```

```
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",
  "updatedAt": "1751875361000"
}
```

Partially Update Contact

PATCH `/api/v1/contacts/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Contact.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	Contact name
inboxes	array[object]	No	List of conversation platforms (new format)
inbox	object	No	Single conversation platform (legacy format, backward compatible)
inbox.id	string (uuid)	Yes	Conversation platform ID
avatar	string	No	Avatar URL
phoneNumber	string	No	Contact phone number
email	string (email)	No	Contact email address
sourceId	string	No	Source ID
queryMetadata	object	No	Query metadata that controls the scope of knowledge that can be queried. Not provided or null = not set (no restriction); empty object {} = treated as no

Field	Type	Required	Description
			restriction (select all), not no permission; knowledgeBases empty array or null = nothing can be queried (no permission); labelRelations.conditions empty array = no label filtering (this does not mean no permission). See this endpoint's description for details.
queryMetadata.knowledgeBases	array[object]	No	List of knowledge bases that can be queried; empty array or null = nothing can be queried (no permission); not provided = no restriction
queryMetadata.labelRelations	object	No	Label filter conditions; an empty conditions array = no label filtering (this does not mean no permission). Must be provided together with knowledgeBases—when provided alone, knowledge base retrieval does not apply label filtering (equivalent to no restriction)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How the label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
metadata	array[object]	No	List of custom contact attributes

Request Schema Example

```
{
  "name"?: string // Contact name (optional)
  "inboxes"?: [ // List of conversation platforms (new format) (optional)
    {
      "id": string (uuid) // Conversation platform ID
    }
  ]
  "inbox"?: { // Single conversation platform (legacy format, backward
compatible) (optional)
  {
    "id": string (uuid) // Conversation platform ID
  }
  }
  "avatar"?: string // Avatar URL (optional)
  "phoneNumber"?: string // Contact phone number (optional)
  "email"?: string (email) // Contact email address (optional)
  "sourceId"?: string // Source ID (optional)
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried. Not provided or null = not set (no restriction); empty
object {} = treated as no restriction (select all), not no permission;
knowledgeBases empty array or null = nothing can be queried (no permission);
labelRelations.conditions empty array = no label filtering (this does not
mean no permission). See this endpoint's description for details. (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
    ]
```

```
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: [ // List of custom contact attributes (optional)
    {
      "key": string // Attribute name
      "value": string // Attribute value
      "updatedAt"?: string // Update time (ISO format, optional; filled in
automatically if not provided) (optional)
    }
  ]
}
```

Request Example Values

```
{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": {
    "knowledgeBases": []
  },
  "metadata": [
    {
      "key": "Customer Tier",
```

```
      "value": "VVIP"
    },
    {
      "key": "Preferred Language",
      "value": "Japanese"
    }
  ]
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "inboxes": [
      {
        "id": "new_inbox_uuid"
      }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": {
      "knowledgeBases": []
    },
    "metadata": [
      {
        "key": "Customer Tier",
        "value": "VVIP"
      },
      {
        "key": "Preferred Language",
        "value": "Japanese"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "inboxes": [
      {
        "id": "new_inbox_uuid"
      }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": {
      "knowledgeBases": []
    },
    "metadata": [
      {
        "key": "Customer Tier",
        "value": "VVIP"
      },
      {
        "key": "Preferred Language",
        "value": "Japanese"
      }
    ]
  };

axios.patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
```

```
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "new_inbox_uuid"
        }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": {
        "knowledgeBases": []
    },
    "metadata": [
        {
            "key": "Customer Tier",
            "value": "VVIP"
        },
        {
            "key": "Preferred Language",
            "value": "Japanese"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
```

```
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'inboxes' => [
                [
                    'id' => 'new_inbox_uuid'
                ]
            ],
            'name' => 'updated_user_name',
            'avatar' => 'updated_avatar_url',
            'phoneNumber' => 987654321,
            'email' => 'updated@example.com',
            'sourceId' => 'updated_source_id',
            'queryMetadata' => [
                'knowledgeBases' => []
            ],
            'metadata' => [
                [
                    'key' => 'Customer Tier',
                    'value' => 'VVIP'
                ],
                [
                    'key' => 'Preferred Language',
                    'value' => 'Japanese'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
```

```
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports a full URL or a relative path
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an
```

```
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
```

```
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve the UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example

```
{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",
  "updatedAt": "1751875361000"
}
```

Delete Contact

DELETE `/api/v1/contacts/{id}`

Parameters

Parameter	Required	Type	Description
`id`	V	string	A UUID string identifying this Contact.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request payload
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body
400	The contact still has associated conversations and cannot be deleted

Get Contact's Conversation List

`GET` `/api/v1/contacts/{id}/conversations/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Contact.
`channels`	X	array	Multiple values may be separated by commas.
`inbox`	X	string	Filter by a single inbox ID (UUID format)
`inboxes`	X	string	Filter by multiple inbox IDs (comma-separated, e.g., uuid1,uuid2, or use multiple inboxes parameters)
`keyword`	X	string	Search message content in conversations (case-insensitive)
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`tags`	X	array	Multiple values may be separated by commas.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/conversations/?
channels=example&inbox=example&inboxes=example&keyword=example&page=1&p
ageSize=1&query=example&tags=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before
running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/conversations/?
channels=example&inbox=example&inboxes=example&keyword=example&page=1&p
ageSize=1&query=example&tags=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/?channels=example&inbox=example&inboxes=example&keyword=example&page=1&pageSize=1&query=example&tags=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/conversations/?
channels=example&inbox=example&inboxes=example&keyword=example&page=1&p
ageSize=1&query=example&tags=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // optional
```

```
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "contact": { // Lightweight Contact Serializer used for the
Conversation List view

Only includes the fields required by the List view; removes inboxes,
mcp_credentials, and api_credentials
to avoid N+1 query problems. email / phone_number / metadata are scalar
fields of the Contact itself,
and the frontend conversation sidebar needs to display them, with no N+1
cost.
      {
        "id": string (uuid)
        "name": string
        "avatar"?: string // Stores our own media as a relative storage key,
or an external CDN URL as-is. (optional)
        "email"?: // may be of different types (optional)
        string (email) // optional
        "phoneNumber"?: string // optional
        "metadata"?: object // Custom customer information shown in the
customer panel of the conversation page (optional)
        "sourceId"?: string // optional
        "tags": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
      }
      }
      "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
      {
        "id": string (uuid)
        "name": string
        "channelType": string (enum: line, telegram, teams, web, messenger,
instagram, email, whatsapp, slack, team_plus, line_works, viber) // * `line`
- LINE
* `telegram` - Telegram
* `teams` - Teams
```

```
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
        "unreadConversationsCount"?: integer // optional
        "signAuth"?:  // optional
        {
          "id": string (uuid)
          "signSource": string (uuid)
          "signParams": {
          {
            "keycloak":
            {
              "clientId": string
              "url": string
              "realm": string
            }
            "line":
            {
              "liffId": string
            }
            "ad":
            {
              "saml": string
            }
            "google"?:  // optional
            {
              "enabled": boolean
            }
          }
          }
          "sourceIdAccessEnabled"?: boolean // When enabled, embedded
visitors presenting a Source ID (verified per the verify mode below) may chat
without going through the login flow. Disabled keeps every existing behavior
unchanged. (optional)
          "sourceIdVerifyMode"?:  // "Require signature verification": the
embedding site must send an HMAC signature computed on its backend. "Source
ID only": anyone knowing a Source ID can act as that contact; suitable only
for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (optional)
```

```
          {
          }
          "sourceIdSignatureTtlMinutes"?:  // How long a signature can be
used to obtain the contact identity. Not a session lifetime: once exchanged,
the session follows the existing WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (optional)
          {
          }
          "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak
access token a visitor logged in with is forwarded as the Authorization
header when the AI calls MCP and API tools on their behalf, so tools do not
require a separate per-tool login. Only for the Keycloak sign source. Never
overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the
conversation at login, never the contact Source ID. (optional)
          "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or
"*.example.com" wildcards) of the tools the visitor Keycloak token may be
sent to (optional)
            string
          ]
        }
        "enableAnalysis"?: boolean // optional
      }
      }
      "chatbot": object // Chatbot (Agent) summary from the conversation's
inbox, already eager-loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon,
feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
      "title": string
      "lastMessage"?: { // optional
      {
        "id": string (uuid)
        "type"?: string // optional
        "content"?: string // optional
        "createdAt": string (timestamp)
      }
      }
      "callRecord":
      {
        "id": string (uuid)
        "status"?: string (enum: initiated, ringing, in-progress, completed,
failed, busy, no-answer, canceled) // * `initiated` - Initiated
```

```
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (optional)
        "direction": string (enum: inbound, outbound) // * `inbound` -
Inbound
* `outbound` - Outbound
        "isActive": boolean
        "canHangup": boolean
      }
      "lastMessageCreatedAt": string (timestamp)
      "unreadMessagesCount": integer
      "autoReplyEnabled": boolean
      "isAutoReplyNow": boolean
      "lastReadAt": string (timestamp)
      "createdAt": string (timestamp)
      "isGroupChat": boolean
      "enableGroupMention"?: boolean // optional
      "queryMetadata"?: { // Query metadata that controls the scope of
knowledge that can be queried; see each endpoint's description for details
(optional)
      {
        "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
          {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
              string (uuid)
            ]
            "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; when both are provided, this field takes precedence)
(optional)
              string (uuid)
            ]
            "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
              string (uuid)
            ]
            "hasUserSelectedAll"?: boolean // true = select all knowledge
base content and exclude the items in the list; false = select only the items
in the list (optional)
          }
```

```
        ]
        "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases — when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
        {
          "operator": string (enum: AND, OR) // How the label conditions are
combined
          "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
            object
          ]
        }
        }
      }
      }
      "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool}
(optional)
      "connectorMask"?: object // Enabled/disabled state per connector:
{connector_id: bool} (optional)
      "thinkingConfig"?: object // None = not set (inherit the chatbot
setting); otherwise {"reasoning_effort": <level>} (optional)
      "thinkingConfigSchema": { // Thinking configuration form for this
model; null means this model has no thinking configuration
      {
        "type"?: string // Form type, always reasoning_effort (optional)
        "fields"?: object // reasoning_effort field: options lists the
thinking levels this model accepts (from weakest to strongest); default is
always null (optional)
        "canDisable"?: boolean // Whether this model can disable thinking;
when true, options includes off (optional)
        "unsetBehavior"?: string (enum: off, model_default) // Actual
behavior when neither the conversation nor the assistant sets a thinking
level: off = no thinking (fastest); model_default = the model thinks on its
own, or the platform default level enables thinking (optional)
        "unsetLevel"?: string // The level the platform actually sends when
no level is set; null means no thinking parameter is sent and the model's own
default applies (optional)
      }
      }
      "effectiveThinkingConfig": object // The level the next reply sends on
this conversation's model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default
level; None means
the model sends no thinking parameter, so the provider default applies.
```

```
      "llm": string (uuid) // The model the next reply runs on: the
conversation's pin, else its assistant's current model.

TC-0205: returning the stored ``conversation.llm`` would show a model an
unpinned conversation no
longer uses once its assistant switches model, while
``thinking_config_schema`` already follows the
effective model. Read-only: displaying it never marks the conversation as
explicitly selected;
only the update-llm endpoint does.
      "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
      {
      }
      "deepResearchOutputFormat": // may be of different types
      string (enum: canvas, chat, file) // Chosen delivery form for the
finished report. Set only when the user clicks a format in the pre-research
choice card; reset to NULL each time Deep Research is (re-)armed. NULL means
no explicit choice (or a plan-skipping request) and falls back to Canvas at
render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas
behavior, force this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
      "ipAddress": string
      "ipCountryCode": string
      "ipRecordedAt": string (timestamp)
      "assignee":
      {
        "id": string (uuid)
        "name": string
        "userId": string (uuid)
      }
      "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (optional)
      "tags": [
        {
          "id": string (uuid)
          "name": string
```

```
          "memberIds"?: [ // optional
            string (uuid)
          ]
        }
      ]
      "progressStatus": string
      "firstResponseAt"?: string (timestamp) // optional
      "queuedAt"?: string (timestamp) // optional
      "assignedAt"?: string (timestamp) // optional
      "transferReason"?: string // optional
      "satisfactionScore": integer // Conversation satisfaction rating on a
5-point scale (1=very dissatisfied, 5=very satisfied)
      "satisfactionComment": string
      "satisfactionSubmittedAt": string (timestamp)
      "latestAnalysisSummary": string // The conversation's current analysis
summary (its active summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
      "clientPlatform":  // Client platform that created this conversation
(web, desktop)

* `web` - Web
* `desktop` - Desktop
      {
      }
      "workingDirectory": string // Absolute path of the local folder bound
to a desktop conversation. Only set for desktop conversations; null for web
conversations.
      "slideProjectId": string
      "matchedField": string // Cached per instance:
``get_matched_message_id`` / ``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
      "matchedSnippet": string // Keyword-centred excerpt of the matched
message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
      "matchedMessageId": string
      "pinnedAt": string (timestamp) // When the conversation was pinned.
Null means unpinned.
      "folder": string (uuid) // The user-defined sidebar folder this
conversation is grouped into. Null means unclassified (shown under the time
groups). Deleting the folder resets this to null without deleting the
```

```
conversation.
      "hasActiveBrowserSession": boolean // Whether a browser_use session
snapshot exists to resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
      "callerOrganizationId": string // Bound caller organization (stamped at
conversation start); null when unbound
      "handoff": object // Handoff information: enabled = platform switch;
the continuation conversation carries source_conversation_id/source_title;
the original handed-off conversation carries
continued_to_conversation_id/continued_to_title and latest_summary_message_id
      "displayContextUsage": boolean // Whether to display the context usage
percentage bar (conversation platform switch; always true for MaiGPT;
independent of the handoff switch)
    }
  ]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "contact": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "avatar": "Response string",
        "email": "response@example.com",
        "phoneNumber": "Response string",
        "metadata": null,
        "sourceId": "Response string",
        "tags": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ]
      },
```

```
      "inbox": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "channelType": "line",
        "unreadConversationsCount": 456,
        "signAuth": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "signSource": "550e8400-e29b-41d4-a716-446655440000",
          "signParams": {
            "keycloak": {
              "clientId": "Response string",
              "url": "Response string",
              "realm": "Response string"
            },
            "line": {
              "liffId": "Response string"
            },
            "ad": {
              "saml": "Response string"
            },
            "google": {
              "enabled": false
            }
          },
          "sourceIdAccessEnabled": false,
          "sourceIdVerifyMode": {},
          "sourceIdSignatureTtlMinutes": {},
          "keycloakTokenPassthrough": false,
          "keycloakTokenPassthroughAllowedHosts": [
            "Response string"
          ]
        },
        "enableAnalysis": false
      },
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response example name",
        "description": "Response example description"
      },
      "title": "Response string",
      "lastMessage": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "type": "Response string",
        "content": "Response string",
        "createdAt": "Response string"
      },
      "callRecord": {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "Response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "Response string",
  "createdAt": "Response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
          "name": "Response string",
          "memberIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ]
        }
      ],
      "progressStatus": "Response string",
      "firstResponseAt": "Response string",
      "queuedAt": "Response string",
      "assignedAt": "Response string",
      "transferReason": "Response string",
      "satisfactionScore": 456,
      "satisfactionComment": "Response string",
      "satisfactionSubmittedAt": "Response string",
      "latestAnalysisSummary": "Response string",
      "clientPlatform": {},
      "workingDirectory": "Response string",
      "slideProjectId": "Response string",
      "matchedField": "Response string",
      "matchedSnippet": "Response string",
      "matchedMessageId": "Response string",
      "pinnedAt": "Response string",
      "folder": "550e8400-e29b-41d4-a716-446655440000",
      "hasActiveBrowserSession": false,
      "callerOrganizationId": "Response string",
      "handoff": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response example name",
        "description": "Response example description"
      },
      "displayContextUsage": false
    }
  ]
}
```


Get Contact's Latest Conversation

GET `/api/v1/contacts/{id}/conversations/latest/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Contact.
`inbox`	X	string	Filter by a single inbox ID (UUID format)
`inboxes`	X	string	Filter by multiple inbox IDs (comma-separated, e.g., uuid1,uuid2, or use multiple inboxes parameters)
`keyword`	X	string	Search message content in conversations (case-insensitive)

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/conversations/latest/?
inbox=example&inboxes=example&keyword=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before
running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/conversations/latest/?
inbox=example&inboxes=example&keyword=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/conversations/latest/?
inbox=example&inboxes=example&keyword=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/conversations/latest/?
inbox=example&inboxes=example&keyword=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact serializer used by the Conversation
List view
```

```
Contains only the fields required by the List view, with inboxes,
mcp_credentials and api_credentials removed
to avoid N+1 query problems. email / phone_number / metadata are scalar
fields on the Contact itself,
which the frontend conversation sidebar needs to display, so they add no N+1
cost.
  {
    "id": string (uuid)
    "name": string
    "avatar"?: string // Stores our own media as a relative storage key, or
an external CDN URL as-is. (optional)
    "email"?: // may be of different types (optional)
    string (email) // optional
    "phoneNumber"?: string // optional
    "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
    "sourceId"?: string // optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger,
instagram, email, whatsapp, slack, team_plus, line_works, viber) // * `line`
- LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
```

```
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // optional
    "signAuth"?:  // optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
          "realm": string
        }
        "line":
        {
          "liffId": string
        }
        "ad":
        {
          "saml": string
        }
        "google"?:  // optional
        {
          "enabled": boolean
        }
      }
      }
      "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged. (optional)
      "sourceIdVerifyMode"?:  // "Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (optional)
      {
      }
      "sourceIdSignatureTtlMinutes"?:  // How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism.
```

```
* `1` - 1
* `5` - 5
* `15` - 15 (optional)
      {
      }
      "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak
access token a visitor logged in with is forwarded as the Authorization
header when the AI calls MCP and API tools on their behalf, so tools do not
require a separate per-tool login. Only for the Keycloak sign source. Never
overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the
conversation at login, never the contact Source ID. (optional)
      "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or
"*.example.com" wildcards) of the tools the visitor Keycloak token may be
sent to (optional)
        string
      ]
    }
    "enableAnalysis"?: boolean // optional
  }
  }
  "chatbot": object // Chatbot (Agent) summary from the conversation's inbox,
already eager-loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon,
feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
  "title": string
  "lastMessage"?: { // optional
  {
    "id": string (uuid)
    "type"?: string // optional
    "content"?: string // optional
    "createdAt": string (timestamp)
  }
  }
  "callRecord":
  {
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed,
failed, busy, no-answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
```

```
* `canceled` - Canceled (optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
  }
  "lastMessageCreatedAt": string (timestamp)
  "unreadMessagesCount": integer
  "autoReplyEnabled": boolean
  "isAutoReplyNow": boolean
  "lastReadAt": string (timestamp)
  "createdAt": string (timestamp)
  "isGroupChat": boolean
  "enableGroupMention"?: boolean // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission), omitted = no
restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (treated as exclusions or as
the only selection depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; if both are provided, this field takes precedence) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (treated as exclusions or as the only
selection depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the lists; false = select only the
items in the lists (optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (not a lack of permission). Must be provided
together with knowledgeBases — when provided alone, knowledge base retrieval
does not apply label filtering (equivalent to no restriction) (optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
```

```
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool}
(optional)
  "connectorMask"?: object // Enabled/disabled state per connector:
{connector_id: bool} (optional)
  "thinkingConfig"?: object // None = not set (inherit the chatbot setting);
otherwise {"reasoning_effort": <level>} (optional)
  "thinkingConfigSchema": { // Thinking configuration form for this model;
null means this model has no thinking configuration
  {
    "type"?: string // Form type; always reasoning_effort (optional)
    "fields"?: object // reasoning_effort field: options lists the thinking
levels this model accepts (from weakest to strongest); default is always null
(optional)
    "canDisable"?: boolean // Whether this model can turn thinking off; when
true, options includes off (optional)
    "unsetBehavior"?: string (enum: off, model_default) // Actual behavior
when neither the conversation nor the assistant sets a thinking level: off =
no thinking (fastest); model_default = the model thinks on its own, or the
platform default level enables thinking (optional)
    "unsetLevel"?: string // The level the platform actually sends when no
level is set; null means no thinking parameter is sent and the model's own
default applies (optional)
  }
  }
  "effectiveThinkingConfig": object // The level the next reply sends on this
conversation's model, after gating and clamping.

The stored choice resolves conversation > chatbot > the LLM row's default
level; None means
the model sends no thinking parameter, so the provider default applies.
  "llm": string (uuid) // The model the next reply runs on: the
conversation's pin, else its assistant's current model.

TC-0205: returning the stored ``conversation.llm`` would show a model an
unpinned conversation no
longer uses once its assistant switches model, while
``thinking_config_schema`` already follows the
effective model. Read-only: displaying it never marks the conversation as
explicitly selected;
only the update-llm endpoint does.
```

```
  "deepResearchStatus":  // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
  {
  }
  "deepResearchOutputFormat": // may be of different types
  string (enum: canvas, chat, file) // Chosen delivery form for the finished
report. Set only when the user clicks a format in the pre-research choice
card; reset to NULL each time Deep Research is (re-)armed. NULL means no
explicit choice (or a plan-skipping request) and falls back to Canvas at
render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas
behavior, force this to NULL/canvas — no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
  "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (optional)
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "memberIds"?: [ // optional
        string (uuid)
      ]
    }
  ]
  "progressStatus": string
  "firstResponseAt"?: string (timestamp) // optional
  "queuedAt"?: string (timestamp) // optional
  "assignedAt"?: string (timestamp) // optional
  "transferReason"?: string // optional
```

```
  "satisfactionScore": integer // Conversation satisfaction rating on a 5-
point scale (1=very dissatisfied, 5=very satisfied)
  "satisfactionComment": string
  "satisfactionSubmittedAt": string (timestamp)
  "latestAnalysisSummary": string // The conversation's current analysis
summary (its active summary revision).

Reads the `latest_analysis_summary` annotation added by `setup_eager_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-load,
so the annotation is present; it falls back to '' only for instances built
outside `setup_eager_loading`.
  "clientPlatform":  // Client platform that created this conversation (web,
desktop)

* `web` - Web
* `desktop` - Desktop
  {
  }
  "workingDirectory": string // Absolute path of the local folder bound to a
desktop conversation. Only set for desktop conversations; null for web
conversations.
  "slideProjectId": string
  "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
  "matchedSnippet": string // Keyword-centred excerpt of the matched message,
with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
  "matchedMessageId": string
  "pinnedAt": string (timestamp) // When the conversation was pinned. Null
means unpinned.
  "folder": string (uuid) // The user-defined sidebar folder this
conversation is grouped into. Null means unclassified (shown under the time
groups). Deleting the folder resets this to null without deleting the
conversation.
  "hasActiveBrowserSession": boolean // Whether a browser_use session
snapshot exists to resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
  "callerOrganizationId": string // Bound caller organization (stamped at
conversation start); null when unbound
```

```
  "handoff": object // Handoff information: enabled = platform switch; the
continuing conversation carries source_conversation_id/source_title; the
original handed-off conversation carries
continued_to_conversation_id/continued_to_title and latest_summary_message_id
  "displayContextUsage": boolean // Whether to show the context usage
percentage bar (conversation platform switch; always true for MaiGPT;
independent of the handoff switch)
}
```

Response Example Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "avatar": "response string",
    "email": "response@example.com",
    "phoneNumber": "response string",
    "metadata": null,
    "sourceId": "response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "response string",
          "url": "response string",
          "realm": "response string"
        },
        "line": {
          "liffId": "response string"
        },
```

```
        "ad": {
          "saml": "response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "response string"
      ]
    },
    "enableAnalysis": false
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Sample response name",
    "description": "Sample response description"
  },
  "title": "response string",
  "lastMessage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "response string",
    "content": "response string",
    "createdAt": "response string"
  },
  "callRecord": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "initiated",
    "direction": "inbound",
    "isActive": false,
    "canHangup": false
  },
  "lastMessageCreatedAt": "response string",
  "unreadMessagesCount": 456,
  "autoReplyEnabled": false,
  "isAutoReplyNow": false,
  "lastReadAt": "response string",
  "createdAt": "response string",
  "isGroupChat": false,
  "enableGroupMention": false,
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Sample response name",
```

```
    "description": "Sample response description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Sample response name",
    "description": "Sample response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Sample response name",
    "description": "Sample response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "response string",
  "ipCountryCode": "response string",
  "ipRecordedAt": "response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "response string",
  "firstResponseAt": "response string",
  "queuedAt": "response string",
  "assignedAt": "response string",
  "transferReason": "response string",
  "satisfactionScore": 456,
  "satisfactionComment": "response string",
  "satisfactionSubmittedAt": "response string",
  "latestAnalysisSummary": "response string",
  "clientPlatform": {},
  "workingDirectory": "response string",
```

```
  "slideProjectId": "response string",
  "matchedField": "response string",
  "matchedSnippet": "response string",
  "matchedMessageId": "response string",
  "pinnedAt": "response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Sample response name",
    "description": "Sample response description"
  },
  "displayContextUsage": false
}
```


Status Code: 404 - No response body

Broadcast Message

POST `/api/v1/contacts/broadcast-message/`

Request Body

Request Parameters

Field	Type	Required	Description
contactIds	array[string]	No	[Explicit mode] List of contact IDs to send the message to
inboxId	string (uuid)	No	[Exclusion/send-to-all mode] Inbox ID; the message is sent to all contacts under this inbox
excludeContactIds	array[string]	No	[Exclusion mode] List of contact IDs to exclude; must be used together with inbox_id

Field	Type	Required	Description
message	string	Yes	Content of the message to send

Request Schema Example

```
{
  "contactIds"?: [ // [Explicit mode] List of contact IDs to send the message to (optional)
    string (uuid)
  ]
  "inboxId"?: string (uuid) // [Exclusion/send-to-all mode] Inbox ID; the message is sent to all contacts under this inbox (optional)
  "excludeContactIds"?: [ // [Exclusion mode] List of contact IDs to exclude; must be used together with inbox_id (optional)
    string (uuid)
  ]
  "message": string // Content of the message to send
}
```

Request Example Values

```
{
  "contactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",
  "excludeContactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "message": "This is a sample message"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/contacts/broadcast-
message/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "contactIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "inboxId": "550e8400-e29b-41d4-a716-446655440000",
    "excludeContactIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "message": "This is a sample message"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "contactIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "inboxId": "550e8400-e29b-41d4-a716-446655440000",
    "excludeContactIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "message": "This is a sample message"
  };

axios.post("https://api.maiagent.ai/api/v1/contacts/broadcast-
message/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/broadcast-message/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "contactIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "inboxId": "550e8400-e29b-41d4-a716-446655440000",
    "excludeContactIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "message": "This is a sample message"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/contacts/broadcast-message/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'contactIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'inboxId' => '550e8400-e29b-41d4-a716-446655440000',
            'excludeContactIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'message' => 'This is a sample message'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "results"?: [ // optional
    {
      "contact_id"?: string (uuid) // optional
      "conversation_id"?: string (uuid) // optional
      "message_id"?: string (uuid) // optional
      "status"?: string // optional
    }
  ]
  "errors"?: [ // optional
    {
      "contact_id"?: string (uuid) // optional
      "error"?: string // optional
    }
  ]
  "total_contacts"?: integer // optional
  "success_count"?: integer // optional
  "error_count"?: integer // optional
}
```

Response Example Values


```
{
  "results": [
    {
      "contact_id": "550e8400-e29b-41d4-a716-446655440000",
      "conversation_id": "550e8400-e29b-41d4-a716-446655440000",
      "message_id": "550e8400-e29b-41d4-a716-446655440000",
      "status": "success"
    }
  ],
  "errors": [
    {
      "contact_id": "550e8400-e29b-41d4-a716-446655440000",
      "error": "Response string"
    }
  ],
  "total_contacts": 456,
  "success_count": 456,
```

```
  "error_count": 456
}
```

Create API Credentials for Contact

POST `/api/v1/contacts/{id}/api-credentials/`

Let the Agent call your API tools on behalf of this contact. For the workflow, see the technical manual: Contact Credentials Sync ◦

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Contact.

Request Body

Request Parameters

Field	Type	Required	Description
tool	string (uuid)	Yes	ID of the API tool
headers	object	No	Dictionary of authentication headers

Request Schema Example

```
{
  "tool": string (uuid) // ID of the API tool
  "headers"?: object // Dictionary of authentication headers (optional)
}
```

Request Example Values

```
{
  "tool": "550e8400-e29b-41d4-a716-446655440000",
  "headers": {}
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/api-credentials/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "tool": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "tool": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  };

axios.post("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/api-credentials/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/api-credentials/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "tool": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/api-credentials/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'tool' => '550e8400-e29b-41d4-a716-446655440000',
            'headers' => new \stdClass()
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports full URLs or relative paths
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
```

```
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inboxes": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "phoneNumber": "Response string",
  "email": "response@example.com",
  "sourceId": "Response string",
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "mcpCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "apiCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "contactConnectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "tags": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "preChatFieldIndex": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Status Code: 400 - No response body

Update Contact's API Credentials

PATCH `/api/v1/contacts/{id}/api-credentials/{credentialId}`

Parameters

Parameter Name	Required	Type	Description
`credentialId`	V	string	
`id`	V	string	A UUID string identifying this Contact.

Request Body

Request Parameters

Field	Type	Required	Description
headers	object	No	Dictionary of authentication headers

Request Schema Example

```
{
  "headers"?: object // Dictionary of authentication headers (optional)
}
```

Request Example Values

```
{
  "headers": {}
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/api-credentials/{credentialId}
"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "headers": {}
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "headers": {}
  };

axios.patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/api-credentials/{credentialId}
", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/api-credentials/{credentialId}
"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "headers": {}
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/api-credentials/{credentialId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'headers' => new \stdClass()
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports full URLs or relative paths
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
```

```
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inboxes": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "phoneNumber": "Response string",
  "email": "response@example.com",
  "sourceId": "Response string",
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "mcpCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "apiCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "contactConnectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "tags": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "preChatFieldIndex": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Status Code: 404 - No response body

Delete Contact's API Credentials

DELETE `/api/v1/contacts/{id}/api-credentials/{credentialId}`

Parameters

Parameter Name	Required	Type	Description
`credentialId`	V	string	
`id`	V	string	A UUID string identifying this Contact.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/api-credentials/{credentialId}
"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/api-credentials/{credentialId}
", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/api-credentials/{credentialId}
"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/api-credentials/{credentialId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body
404	No response body

Create MCP Credentials for Contact

`POST` `/api/v1/contacts/{id}/mcp-credentials/`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Contact.

Request Body

Request Parameters

Field	Type	Required	Description
tool	string (uuid)	Yes	ID of the MCP tool
headers	object	No	Dictionary of authentication headers

Request Schema Example

```
{
  "tool": string (uuid) // ID of the MCP tool
  "headers"?: object // Dictionary of authentication headers (optional)
}
```

Request Example Values

```
{
  "tool": "550e8400-e29b-41d4-a716-446655440000",
```

```
  "headers": {}
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/mcp-credentials/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "tool": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "tool": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  };

axios.post("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/mcp-credentials/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/mcp-credentials/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "tool": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/mcp-credentials/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'tool' => '550e8400-e29b-41d4-a716-446655440000',
            'headers' => new \stdClass()
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports full URLs or relative paths
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an
empty array or null = none can be queried (no permission), not provided = no
restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
```

```
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve the UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inboxes": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "phoneNumber": "Response string",
  "email": "response@example.com",
  "sourceId": "Response string",
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "mcpCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "apiCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "contactConnectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "tags": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "preChatFieldIndex": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Status Code: 400 - No response body

Update Contact's MCP Credentials

PATCH `/api/v1/contacts/{id}/mcp-credentials/{credentialId}`

Parameters

Parameter Name	Required	Type	Description
`credentialId`	V	string	
`id`	V	string	A UUID string identifying this Contact.

Request Body

Request Parameters

Field	Type	Required	Description
headers	object	No	Dictionary of authentication headers

Request Schema Example

```
{
  "headers"?: object // Dictionary of authentication headers (optional)
}
```

Request Example Values

```
{
  "headers": {}
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/mcp-credentials/{credentialId}
"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "headers": {}
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "headers": {}
  };

axios.patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/mcp-credentials/{credentialId}
", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/mcp-credentials/{credentialId}
"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "headers": {}
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/mcp-credentials/{credentialId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'headers' => new \stdClass()
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports full URLs or relative paths
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried; an
empty array or null = none can be queried (no permission), not provided = no
restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
```

```
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve the UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inboxes": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "phoneNumber": "Response string",
  "email": "response@example.com",
  "sourceId": "Response string",
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "mcpCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "apiCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "contactConnectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "tags": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "preChatFieldIndex": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Status Code: 404 - No response body

Delete Contact's MCP Credentials

DELETE `/api/v1/contacts/{id}/mcp-credentials/{credentialId}`

Parameters

Parameter Name	Required	Type	Description
`credentialId`	V	string	
`id`	V	string	A UUID string identifying this Contact.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/mcp-credentials/{credentialId}
"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-
41d4-a716-446655440000/mcp-credentials/{credentialId}
", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-
446655440000/mcp-credentials/{credentialId}
"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/mcp-credentials/{credentialId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body
404	No response body

Download Contact Bulk Import Template

GET `/api/v1/contacts/bulk-import-template/`

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/bulk-import-
template/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/bulk-import-
template/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/bulk-import-template/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/bulk-import-template/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
```

```
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports a full URL or a relative path
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
```

```
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inboxes": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
```

```
    }
  ],
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "phoneNumber": "Response string",
  "email": "response@example.com",
  "sourceId": "Response string",
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "metadata": null,
  "mcpCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "apiCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "contactConnectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "preChatFieldIndex": {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Bulk Import Contacts

POST `/api/v1/contacts/bulk-import/`

Upload an Excel file filled in according to the template; with dry_run=true, only reports the number of records that would be created/updated without writing anything (returns 200 instead of 201).

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/contacts/bulk-import/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before
running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Add form fields

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/contacts/bulk-import/",
formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/bulk-import/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example.pdf', open('example.pdf', 'rb'),
'application/pdf')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/contacts/bulk-import/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example.pdf', 'r'),
                'filename' => 'example.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?:  // Backward-compatible field: a single inbox; ignored if inboxes
is also provided (optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // may be of different types (optional)
  string (uri) // Contact avatar URL; supports a full URL or a relative path
(optional)
  "phoneNumber"?: string // optional
  "email"?: // may be of different types (optional)
  string (email) // optional
  "sourceId"?: string // optional
  "queryMetadata"?: { // Query metadata that controls the scope of knowledge
that can be queried; see each endpoint's description (optional)
  {
    "knowledgeBases"?: [ // List of knowledge bases that can be queried;
empty array or null = nothing can be queried (no permission); not provided =
no restriction (optional)
      {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively
selected depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name of
chatbotFileIds; this field takes precedence if both are provided) (optional)
          string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected
depending on hasUserSelectedAll) (optional)
          string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content of the
knowledge base and exclude the items in the list; false = select only the
items in the list (optional)
      }
```

```
    ]
    "labelRelations"?: { // Label filter conditions; an empty conditions
array = no label filtering (this does not mean no permission). Must be
provided together with knowledgeBases—when provided alone, knowledge base
retrieval does not apply label filtering (equivalent to no restriction)
(optional)
    {
      "operator": string (enum: AND, OR) // How the label conditions are
combined
      "conditions"?: [ // List of label conditions; supports nested
operator/conditions combinations (optional)
        object
      ]
    }
    }
  }
  }
  "metadata"?: object // Custom customer information shown in the customer
panel of the conversation page (optional)
  "mcpCredentials": [ // List of the contact's MCP credentials
    object
  ]
  "apiCredentials": [ // List of the contact's API credentials
    object
  ]
  "contactConnectors": [ // List of the contact's Connector OAuth
authorizations (excluding the token itself)
    object
  ]
  "tags": [
    {
      "id": string (uuid)
      "name": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "preChatFieldIndex": object // Pre-chat form field settings from the
inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}};
the frontend uses it to resolve UUID keys in metadata into display names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inboxes": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "phoneNumber": "Response string",
  "email": "response@example.com",
  "sourceId": "Response string",
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "metadata": null,
  "mcpCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "apiCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "contactConnectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "tags": [
    {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "preChatFieldIndex": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Example API call (Shell)

目錄

1. Create Role
2. Bulk Add Role Members
3. Bulk Assign AI Assistants to Role
4. Bulk Assign Inboxes to Role
5. Bulk Assign Knowledge Bases to a Role
6. Bulk Assign Databases to a Role
7. List Permissions
8. Get role list
9. Get role details
10. List Role Members
11. Get a Specific Role Member
12. List Role AI Assistants
13. List Role Inboxes
14. List Role Knowledge Bases
15. List Role Databases
16. Update Role Permissions
17. Partially Update Role Permissions
18. Delete Role
19. Remove Role Member
20. Remove AI Assistant from Role
21. Remove Inbox from Role
22. Remove Knowledge Base from Role
23. Remove Database from Role

Example API call (Shell)

Create Role

`POST` `/api/v1/organizations/{organizationPk}/groups/`

Parameters

Parameter Name	Required	Type	Description
`organizationPk`	V	string	A UUID string identifying this organization ID

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
organization	string (uuid)	No	
permissions	array[string]	Yes	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	
maigptCanCodeInterpreter	boolean	No	Whether members of this role may use the code interpreter built-in tool in MaiGPT.
maigptCanRag	boolean	No	Whether members of this role may use the knowledge base RAG built-in tool in MaiGPT.

Field	Type	Required	Description
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.
maigptAllLlms	boolean	No	Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none.
maigptAllTools	boolean	No	Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none.
maigptAllConnectors	boolean	No	Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none.
maigptAllSkills	boolean	No	Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none.
allowedLlms	array[string]	No	
allowedTools	array[string]	No	
allowedConnectors	array[string]	No	

Field	Type	Required	Description
allowedSkills	array[string]	No	

Example Request Structure

```
{
  "name": string
  "organization"?: string (uuid) // optional
  "permissions": [
    string (uuid)
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
  "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
  "allowedLlms"?: [ // optional
    string (uuid)
  ]
```

```
  "allowedTools"?: [ // optional
    string (uuid)
  ]
  "allowedConnectors"?: [ // optional
    string (uuid)
  ]
  "allowedSkills"?: [ // optional
    string (uuid)
  ]
}
```

Example Request Values


```
{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
```

```
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedSkills": [
      "550e8400-e29b-41d4-a716-446655440000"
```

```
    ]
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
      "550e8400-e29b-41d4-a716-446655440000"
```

```
    ],
    "allowedSkills": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": True,
    "canCreateKnowledgeBase": True,
    "canCreateInbox": True,
    "canCreateDatabase": True,
    "canCreateTool": True,
    "canCreateAgentUiTool": True,
    "canCreateSkill": True,
    "maigptCanCodeInterpreter": True,
    "maigptCanRag": True,
    "maigptCanDeepResearch": True,
    "maigptCanBrowserTool": True,
    "maigptAllLlms": True,
    "maigptAllTools": True,
    "maigptAllConnectors": True,
    "maigptAllSkills": True,
    "allowedLlms": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
```

```
    "allowedSkills": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'organization' => '550e8400-e29b-41d4-a716-
446655440000',
            'permissions' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'canCreateChatbot' => true,
            'canCreateKnowledgeBase' => true,
            'canCreateInbox' => true,
            'canCreateDatabase' => true,
            'canCreateTool' => true,
            'canCreateAgentUiTool' => true,
            'canCreateSkill' => true,
            'maigptCanCodeInterpreter' => true,
            'maigptCanRag' => true,
            'maigptCanDeepResearch' => true,
            'maigptCanBrowserTool' => true,
            'maigptAllLlms' => true,
            'maigptAllTools' => true,
            'maigptAllConnectors' => true,
            'maigptAllSkills' => true,
            'allowedLlms' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'allowedTools' => [
                '550e8400-e29b-41d4-a716-446655440000'
```

```
                ],
                'allowedConnectors' => [
                    '550e8400-e29b-41d4-a716-446655440000'
                ],
                'allowedSkills' => [
                    '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]);

        $data = json_decode($response->getBody(), true);
        echo "Response received successfully:\n";
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error: ' . $e->getMessage();
    }
    ?>
```

Response Content

Status code: 201

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
  "type":
  {
  }
  "permissions": [ // Returns the permission list of the role

Processing flow:
1. Split the role permissions into parent permissions and child
permissions
2. Group child permissions by parent permission ID
3. If a child permission is missing its parent permission, automatically
```

```
load the missing parent permission from the database
4. Serialize the parent permissions and their child permissions, and sort
them by order
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Returns the remaining member count (after
deducting the 3 shown in the list), used for the frontend +N badge

Unlike resource counts:
<ul>
  <li>members_count: returns total - 3 (used for the +N badge); None
means the total is <= 3</li>
  <li>chatbots_count/knowledge_bases_count/inboxes_count: return the
total count</li>
</ul>
  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]
  "databasesCount": integer
  "databasesPreview": [
    string
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
```

```
  "maigptCanDeepResearch"?: boolean // Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this switch too. (optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none. (optional)
  "maigptAllTools"?: boolean // Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none. (optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none. (optional)
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "membersPreview": [
    "Example response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "Example response string"
  ],
  "knowledgeBasesCount": 456,
  "knowledgeBasesPreview": [
```

```
    "Example response string"
  ],
  "inboxesCount": 456,
  "inboxesPreview": [
    "Example response string"
  ],
  "databasesCount": 456,
  "databasesPreview": [
    "Example response string"
  ],
  "canCreateChatbot": false,
  "canCreateKnowledgeBase": false,
  "canCreateInbox": false,
  "canCreateDatabase": false,
  "canCreateTool": false,
  "canCreateAgentUiTool": false,
  "canCreateSkill": false,
  "maigptCanCodeInterpreter": false,
  "maigptCanRag": false,
  "maigptCanDeepResearch": false,
  "maigptCanBrowserTool": false,
  "maigptAllLlms": false,
  "maigptAllTools": false,
  "maigptAllConnectors": false,
  "maigptAllSkills": false,
  "createdAt": "Example response string"
}
```


Bulk Add Role Members

POST `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/bulk-create/`

Parameters

Parameter Name	Required	Type	Description
`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID

Request Body

Request Parameters

Field	Type	Required	Description
members	array[string]	Yes	

Example Request Structure

```
{
  "members": [
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "members": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/bulk-create/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "members": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "members": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/bulk-create/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/bulk-create/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "members": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/bulk-create/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'members' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Example Response Structure

```
[
  {
    "id"?: string (uuid) // Role member ID (optional)
    "member"?: { // optional
    {
      "id"?: string (uuid) // Member ID (optional)
      "name"?: string // Member name (optional)
      "email"?: string (email) // Member email (optional)
      "organization"?: { // optional
      {
        "id"?: string (uuid) // Organization ID (optional)
        "name"?: string // Organization name (optional)
        "createdAt"?: string // Organization creation timestamp
(optional)
        "usageStatistics"?: { // optional
        {
          "chatbotsCount"?: integer // Number of chatbots (optional)
          "countableChatbotsCount"?: integer // Number of chatbots that
count toward the creation quota (chatbots whose mode is not in the plan
allowlist are excluded) (optional)
          "exemptChatbotsByMode"?: object // Number of chatbots exempt
from the quota, grouped by the mode that exempts them (optional)
          "canCreateChatbot"?: boolean // Whether chatbots can be created
(optional)
          "createChatbotLimit"?: integer // Maximum number of chatbots
that can be created (-1 means unlimited, 0 means creation is disabled)
(optional)
          "currentMonthWordsCountTotal"?: integer // Total word limit for
the current month (optional)
          "currentMonthUsedWordsCountTotal"?: integer // Words used in
the current month (optional)
          "currentMonthUsedConversationsCountTotal"?: integer //
Conversations used in the current month (optional)
          "availableUploadFilesSizeTotal"?: integer // Total size of
files that can be uploaded (optional)
          "usedUploadFileSizeTotal"?: integer // Size of uploaded files
used (optional)
        }
        }
        "organizationPlan"?: { // optional
        {
          "id"?: string (uuid) // Plan ID (optional)
          "planName"?: string // Plan name (optional)
```

```
          "expiredAt"?: string // Plan expiration time (optional)
        }
        }
      }
      }
      "isOwner"?: boolean // Whether the member is the organization owner
(optional)
      "permissions"?: { // optional
      {
        "hasOrganizationAccessPermission"?: boolean // Whether the member
has organization access permission (optional)
        "hasChatAccessPermission"?: boolean // Whether the member has
chat access permission (optional)
        "hasConversationAccessPermission"?: boolean // Whether the member
has conversation access permission (optional)
        "hasChatbotAccessPermission"?: boolean // Whether the member has
chatbot access permission (optional)
      }
      }
      "createdAt"?: string // Member creation timestamp (optional)
    }
    }
    "createdAt"?: string // Role member creation timestamp (optional)
  }
]
```

Example Response Values


```
[
  {
    "id": "c1e54f04-01fc-45e9-b40c-9d8b37bd0ac0",
    "member": {
      "id": "af4e472d-3f9b-446b-bab8-05de4112d086",
      "name": "h",
      "email": "h@gmail.com",
      "organization": {
        "id": "670e3d30-9d24-47b4-b2cc-6bf224b2f3bb",
        "name": "Updated Org Name",
        "createdAt": "1745288290000",
        "usageStatistics": {
          "chatbotsCount": 1,
          "countableChatbotsCount": 1,
          "exemptChatbotsByMode": {},
          "canCreateChatbot": false,
```

```
          "createChatbotLimit": 1,
          "currentMonthWordsCountTotal": 1000000,
          "currentMonthUsedWordsCountTotal": 0,
          "currentMonthUsedConversationsCountTotal": 0,
          "availableUploadFilesSizeTotal": 104857600,
          "usedUploadFileSizeTotal": 0
        },
        "organizationPlan": {
          "id": "16136181-ae0d-4b3f-aaeb-9a6ed251459f",
          "planName": "Free Plan",
          "expiredAt": null
        }
      },
      "isOwner": false,
      "permissions": {
        "hasMaigptAccessPermission": false,
        "hasChatbotAccessPermission": false,
        "hasAgentopsAccessPermission": false,
        "hasConversationAccessPermission": false,
        "hasChatAccessPermission": false,
        "hasDeveloperAccessPermission": false,
        "hasOrganizationAccessPermission": true
      },
      "createdAt": "1748335331000"
    },
    "createdAt": "1748341252000"
  }
]
```


Bulk Assign AI Assistants to Role

POST `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/bulk-create/`

Parameters

Parameter	Required	Type	Description
`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Request Content

Request Parameters

Field	Type	Required	Description
chatbots	array[string]	Yes	

Example Request Structure

```
{
  "chatbots": [
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "chatbots": [
    "550e8400-e29b-41d4-a716-446655440000"
```

```
  ]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/bulk-create/?
page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "chatbots": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "chatbots": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/bulk-create/?
page=1&pageSize=1&query=example&search=example", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/bulk-create/?
page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "chatbots": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/bulk-create/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'chatbots' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "chatbot":
      {
        "id": string (uuid)
        "name": string // Chatbot name. In Agent mode it carries semantic
meaning; in other modes it only distinguishes chatbots.
        "largeLanguageModel": string (uuid) // The large language model
used by the bot to generate answers (including organization-owned AI
Gateway models)
        "llm":  // ID, name, and retirement information of the bound
model, read directly from the FK; returned even if the model has been
disabled by the organization
        {
          "id": string (uuid)
          "name": string
          "retiresAt": string (date) // Provider retirement date; once it
passes, a daily job switches the assistant to the replacement and
disables this model. null means no retirement is scheduled
          "replacement":  // Model that takes over after retirement (id
and name); null means none is specified and nothing is migrated
automatically on expiry
          {
            "id": string (uuid)
            "name": string
          }
        }
        "embeddingModel"?: string (uuid) // Embedding model used to
vectorize text; optional (optional)
        "rerankerModel"?: string (uuid) // Model used for reranking;
optional (optional)
        "instructions"?: string // Role instructions describing the
chatbot's role and behavior. (optional)
        "knowledgeBases"?: [ // List of knowledge bases the bot can
access (optional)
```

```
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "databases"?: [ // List of SQL databases the bot can access (for
the Text-to-SQL feature) (optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "updatedAt": string (timestamp)
        "organization"?: string (uuid) // Organization the bot belongs
to; if empty, it is a personal bot (optional)
        "replyMode"?:  // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (optional)
        {
        }
        "template"?: string // Template used by Template and Hybrid
modes. (optional)
        "unanswerableTemplate"?: string // Template used by Template and
Hybrid modes when the question cannot be answered. (optional)
        "totalWordsCount"?: integer (int64) // Accumulated total word
count used. (optional)
        "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (optional)
        {
        }
        "rawOutputFormat"?: object // JSON structure definition for the
custom output format (optional)
        "groups"?: [ // List of groups the bot can access (optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "tools"?: [ // List of tools the bot can use (optional)
          {
```

```
            "id": string (uuid)
            "name": string
            "description": string
            "displayName": string
            "toolType": string
          }
        ]
        "skills"?: [ // List of skills associated with the bot (optional)
          {
            "id": string (uuid)
            "name": string
            "description": string
          }
        ]
        "connectors"?: [ // Connectors associated with this chatbot,
gating per-contact OAuth authorization (optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (optional)
        {
        }
        "numberOfRetrievedChunks"?: integer // Number of reference chunks
to retrieve; default is 12, minimum is 1 (optional)
        "defaultLabelRelations"?: object // Assistant-level default
knowledge base label filter, in the same format as
query_metadata.label_relations; null means no filtering. Applied as the
lowest-priority fallback only when the caller does not provide
query_metadata. (optional)
        "useFullRetrievalRange"?: boolean // When enabled, ignores the
LLM top_k truncation and returns the full reranker results (optional)
        "enableEvaluation"?: boolean // optional
        "enableInlineCitations"?: boolean // Enables inline citations,
inserting citation markers such as [1][2] into responses. (optional)
        "enableCodeInterpreter"?: boolean // When enabled, provides the
Code Interpreter tool to run code in an isolated container. (optional)
        "enableBrowserTool"?: boolean // Enable browser automation tool
for web tasks (optional)
        "enableImageSearch"?: boolean // Provide the image_search tool
```

```
when any attached knowledge base uses a multimodal embedding model.
Disable to remove the tool entirely. (optional)
        "enableClaudeStreamRetry"?: boolean // When a Claude streaming
call stalls (first-chunk or mid-stream), abort and re-issue the call once
in non-streaming mode. Mid-stream retries send a reset signal so the
client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (optional)
        "customMaxLlmOutputTokens"?: integer // Custom maximum number of
LLM output tokens; minimum is 512 (optional)
        "voiceConfig"?: { // optional
        {
          "enabled"?: boolean // Whether this chatbot can be used as a
voice agent. Checked by every conversation platform (WebChat, SIP phone,
admin/marketplace voice test) instead of a per-platform toggle.
(optional)
          "voiceAgentType"?: string (uuid) // Voice agent mode type.
(optional)
          "sttProvider"?: string (uuid) // Speech-to-Text service
provider. (optional)
          "sttConfig"?: object // STT configuration parameters (JSON).
(optional)
          "ttsProvider"?: string (uuid) // Text-to-Speech service
provider. (optional)
          "ttsConfig"?: object // TTS configuration parameters (JSON).
(optional)
          "realtimeProvider"?: string (uuid) // Realtime end-to-end voice
model provider. (optional)
          "realtimeConfig"?: object // Realtime configuration parameters
(JSON). (optional)
          "realtimeInstructions"?: string // Prompt dedicated to the
realtime voice session; falls back to chatbot instructions when empty
(optional)
          "turnHandling"?: object // Turn-taking and interruption
settings for the voice agent. (optional)
          "enableVoiceReply"?: boolean // When disabled, the voice agent
replies in text only and does not speak (optional)
        }
        }
        "thinkingConfig"?: object // Thinking level: null (model default)
or {"reasoning_effort": "off|minimal|low|medium|high|xhigh|max"}
(optional)
        "enableToolSearching"?: boolean // Enables dynamic tool search,
allowing the agent to discover and load tools dynamically via
tool_searching_tool (optional)
        "maxAgentIterations"?: integer // Maximum number of iterations
```

```
for AgentWorkflow (1-100, default: 20) (optional)
        "agentTimeoutSeconds"?: integer // Timeout in seconds for
AgentWorkflow execution (30-1200, default: 600) (optional)
        "agentTimeoutMessage"?: string // Custom user-facing message
shown when AgentWorkflow times out (max 500 chars). Leave empty to use
the system default. (optional)
        "requireTextWithFlexCards"?: boolean // When enabled, a turn that
would end with only Flex Message cards and no text for the user is re-
issued to the LLM once, asking it to write the text first and then send
the card. Each retry incurs a second billing for the same prompt.
(optional)
        "isMultimodalLlm": boolean // Whether the chatbot LLM supports
multimodal (read-only)
        "enableSizeBasedToolResultTruncation"?: boolean // When enabled,
tool results exceeding the character threshold are kept as head+tail
(with the identifiers found in the cut part) when written to conversation
history; knowledge base, SQL and conversation search results keep their
100-character cut either way. Reduces memory bloat from large tool
outputs (code interpreter bash stdout, web search results, etc.).
(optional)
        "toolResultMaxChars"?: integer // Character threshold for size-
based tool result truncation (500-50000). Only effective when
enable_size_based_tool_result_truncation is True. (optional)
        "enableVectorMemory"?: boolean // When enabled, archived messages
are vectorized and processed by fact extraction for semantic retrieval
(existing behavior). When disabled, archived messages are discarded and
only the active chat history window is used for multi-turn context.
(optional)
        "longTermMemoryTopK"?: integer // Number of memory nodes
retrieved per long-term memory lookup (similarity_top_k of the vector
search); default is 10, range 1–50 (optional)
        "enableCrossConversationSearch"?: boolean // When enabled, the
conversation history tools also reach this contact's other conversations
in the same inbox, within the lookback window. When disabled, they only
reach the current conversation. (optional)
        "crossConversationLookbackDays"?: integer // How far back cross-
conversation search may reach, measured from each conversation's last
message (1-365 days). Leave empty to fall back to 90 days. Only effective
when enable_cross_conversation_search is True. (optional)
        "useGatewayFallback"?: boolean // When enabled, use the fallback
group instead of the single LLM (optional)
        "gatewayFallbackGroup"?: string (uuid) // optional
        "enableSearchMessagesTool"?: boolean // Allow the AI to search
past messages in the current conversation. (optional)
        "enableGetMessageContextTool"?: boolean // Allow the AI to
```

```
retrieve messages surrounding a specific message for context. (optional)
        "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to
read the full text of plain-text attachments. AGENT mode only. (optional)
        "enableParseAttachmentContentTool"?: boolean // Allow the AI to
parse PDF, Office, audio, and other attachments. AGENT mode only.
(optional)
        "enableListAvailableParsersTool"?: boolean // Allow the AI to
list available parsers and their capabilities. AGENT mode only.
(optional)
        "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate
shareable download links for produced content. AGENT mode only.
(optional)
        "isLocked": boolean // Locked instance: config follows the
official version; only the dedicated KB can be modified
        "sourceBuildInAgent": string (uuid) // Points to the official
Build-in Agent blueprint when instantiated by a rental
        "canUpdate": boolean // Return whether the current user can
update this resource.
        "canDelete": boolean // Return whether the current user can
delete this resource.
      }
      "canRead"?: boolean // optional
      "canUpdate"?: boolean // optional
      "canDelete"?: boolean // optional
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response string",
        "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
        "llm": {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response string",
      "retiresAt": "Example response string",
      "replacement": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response string"
      }
    },
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example response string",
    "knowledgeBases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response string"
      }
    ],
    "databases": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response string"
      }
    ],
    "updatedAt": "Example response string",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example response string",
    "unanswerableTemplate": "Example response string",
    "totalWordsCount": 456,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response string"
      }
    ],
    "tools": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response string",
        "description": "Example response string",
        "displayName": "Example response string",
        "toolType": "Example response string"
      }
```

```
        ],
        "skills": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response string",
            "description": "Example response string"
          }
        ],
        "connectors": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response string"
          }
        ],
        "agentMode": {},
        "numberOfRetrievedChunks": 456,
        "defaultLabelRelations": null,
        "useFullRetrievalRange": false,
        "enableEvaluation": false,
        "enableInlineCitations": false,
        "enableCodeInterpreter": false,
        "enableBrowserTool": false,
        "enableImageSearch": false,
        "enableClaudeStreamRetry": false,
        "customMaxLlmOutputTokens": 456,
        "voiceConfig": {
          "enabled": false,
          "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
          "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
          "sttConfig": null,
          "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
          "ttsConfig": null,
          "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
          "realtimeConfig": null,
          "realtimeInstructions": "Example response string",
          "turnHandling": null,
          "enableVoiceReply": false
        },
        "thinkingConfig": null,
        "enableToolSearching": false,
        "maxAgentIterations": 456,
        "agentTimeoutSeconds": 456,
        "agentTimeoutMessage": "Example response string",
        "requireTextWithFlexCards": false,
        "isMultimodalLlm": false,
```

```
        "enableSizeBasedToolResultTruncation": false,
        "toolResultMaxChars": 456,
        "enableVectorMemory": false,
        "longTermMemoryTopK": 456,
        "enableCrossConversationSearch": false,
        "crossConversationLookbackDays": 456,
        "useGatewayFallback": false,
        "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
        "enableSearchMessagesTool": false,
        "enableGetMessageContextTool": false,
        "enableReadAttachmentFullTextTool": false,
        "enableParseAttachmentContentTool": false,
        "enableListAvailableParsersTool": false,
        "enableGetDownloadLinkTool": false,
        "isLocked": false,
        "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
        "canUpdate": false,
        "canDelete": false
      },
      "canRead": false,
      "canUpdate": false,
      "canDelete": false,
      "createdAt": "Example response string"
    }
  ]
}
```


Bulk Assign Inboxes to Role

POST `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/bulk-create/`

Parameters

Parameter	Required	Type	Description

`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/bulk-create/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/bulk-create/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/bulk-create/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/bulk-create/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	No response body

Bulk Assign Knowledge Bases to a Role

POST `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/bulk-create/`

Parameters

Parameter	Required	Type	Description
`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this Organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Request

Request Parameters

Field	Type	Required	Description
knowledgeBases	array[string]	Yes	

Request Schema

```
{
  "knowledgeBases": [
```

```
    string (uuid)
  ]
}
```

Request Example

```
{
  "knowledgeBases": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/bulk-create/?
page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "knowledgeBases": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "knowledgeBases": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/bulk-create/?
page=1&pageSize=1&query=example&search=example", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/bulk-create/?
page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "knowledgeBases": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/bulk-create/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'knowledgeBases' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "knowledgeBase":
      {
        "id": string (uuid)
        "createdBy":
        {
          "id": string (uuid)
          "name": string
        }
        "organization"?: string (uuid) // optional
        "embeddingModel": string (uuid)
        "rerankerModel": string (uuid)
        "name": string
        "description"?: string // optional
        "numberOfRetrievedChunks"?: integer // Number of retrieved
reference chunks. Default is 12, minimum is 1 (optional)
        "enableHyde"?: boolean // Enable HyDE. Default is False
(optional)
        "similarityCutoff"?: number (double) // Similarity threshold.
Default is 0.0, range 0.0-1.0 (optional)
        "enableRerank"?: boolean // Whether to enable reranking. Default
is True (optional)
        "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
        "vectorDatabaseType"?:  // Type of vector database: elasticsearch
or opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
        {
        }
```

```
        "vectorDatabaseUrl"?: string // Custom vector database endpoint
URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (optional)
        "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (optional)
        "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (optional)
        "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (optional)
        "oracleUser"?: string // Oracle database username. (optional)
        "oraclePassword"?: string // Oracle database password. (optional)
        "synxdbUser"?: string // SynxDB database username. (optional)
        "synxdbPassword"?: string // SynxDB database password. (optional)
        "labels": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "chatbots"?: [ // optional
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "groups"?: [ // List of groups that can access the knowledge base
(optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
        "vectorStorageSize": integer
        "chunksCount": integer
        "canUpdate": boolean // Return whether the current user can
update this resource.
        "canDelete": boolean // Return whether the current user can
delete this resource.
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
      }
      "canRead"?: boolean // optional
```

```
      "canUpdate"?: boolean // optional
      "canDelete"?: boolean // optional
      "createdAt": string (timestamp)
    }
  ]
}
```

Response Example


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "createdBy": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        },
        "organization": "550e8400-e29b-41d4-a716-446655440000",
        "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
        "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "description": "response string",
        "numberOfRetrievedChunks": 456,
        "enableHyde": false,
        "similarityCutoff": 456,
        "enableRerank": false,
        "enableRrf": false,
        "vectorDatabaseType": {},
        "vectorDatabaseUrl": "response string",
        "vectorDatabaseApiKey": "response string",
        "opensearchUsername": "response string",
        "opensearchPassword": "response string",
        "oracleUser": "response string",
        "oraclePassword": "response string",
        "synxdbUser": "response string",
        "synxdbPassword": "response string",
        "labels": [
          {
```

```
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "response string"
          }
        ],
        "chatbots": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "response string"
          }
        ],
        "groups": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "response string"
          }
        ],
        "filesCount": 456,
        "vectorStorageSize": 456,
        "chunksCount": 456,
        "canUpdate": false,
        "canDelete": false,
        "createdAt": "response string",
        "updatedAt": "response string"
      },
      "canRead": false,
      "canUpdate": false,
      "canDelete": false,
      "createdAt": "response string"
    }
  ]
}
```


Bulk Assign Databases to a Role

POST `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/bulk-create/`

Parameters

Parameter	Required	Type	Description
`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this Organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Request

Request Parameters

Field	Type	Required	Description
databases	array[string]	Yes	

Request Schema

```
{
  "databases": [
    string (uuid)
  ]
}
```

Request Example

```
{
  "databases": [
```

```
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/bulk-create/?
page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "databases": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "databases": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/bulk-create/?
page=1&pageSize=1&query=example&search=example", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/bulk-create/?
page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "databases": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/bulk-create/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'databases' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "database":
      {
        "id": string (uuid)
        "name": string // Display name for this database connection
        "description": string // Description of the database and its
purpose
        "databaseType":  // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
        {
        }
        "includeTables": object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files.
        "isActive": boolean // When disabled, this database will not be
used in Text-to-SQL queries
        "status":
        {
        }
        "deletedAt": string (timestamp)
        "tables": [
          {
            "id": string (uuid)
            "tableName": string // Name of the table in the database
            "sheetName": string // Name of the source worksheet/sheet
this table was created from. Empty for tables created before this field
was added.
```

```
            "description"?: string // Description of the table and its
data for better Text-to-SQL understanding (optional)
            "status":
            {
            }
            "errorMessage": string // Error details if table creation or
deletion failed
            "sourceFile":
            {
              "id": string (uuid)
              "name": string
              "isFromKnowledgeBase": boolean // Determine if the source
file originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
            }
            "columns": [
              {
                "id": string (uuid)
                "columnName": string
                "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
                "description"?: string // Description of the column for
better Text-to-SQL understanding (optional)
                "isPrimaryKey"?: boolean // optional
                "isNullable"?: boolean // optional
                "createdAt": string (timestamp)
                "updatedAt": string (timestamp)
              }
            ]
            "createdAt": string (timestamp)
            "updatedAt": string (timestamp)
          }
        ]
        "chatbots": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "groups": [
          {
            "id": string (uuid)
            "name": string
```

```
          }
        ]
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
      }
      "canRead"?: boolean // optional
      "canUpdate"?: boolean // optional
      "canDelete"?: boolean // optional
      "createdAt": string (timestamp)
    }
  ]
}
```

Response Example

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "database": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "description": "response string",
        "databaseType": {},
        "includeTables": null,
        "isActive": false,
        "status": {},
        "deletedAt": "response string",
        "tables": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "tableName": "response string",
            "sheetName": "response string",
            "description": "response string",
            "status": {},
            "errorMessage": "response string",
            "sourceFile": {
              "id": "550e8400-e29b-41d4-a716-446655440000",
              "name": "response string",
              "isFromKnowledgeBase": false
```

```
            },
            "columns": [
              {
                "id": "550e8400-e29b-41d4-a716-446655440000",
                "columnName": "response string",
                "columnType": "response string",
                "description": "response string",
                "isPrimaryKey": false,
                "isNullable": false,
                "createdAt": "response string",
                "updatedAt": "response string"
              }
            ],
            "createdAt": "response string",
            "updatedAt": "response string"
          }
        ],
        "chatbots": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "response string"
          }
        ],
        "groups": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "response string"
          }
        ],
        "createdAt": "response string",
        "updatedAt": "response string"
      },
      "canRead": false,
      "canUpdate": false,
      "canDelete": false,
      "createdAt": "response string"
    }
  ]
}
```

List Permissions

GET `/api/v1/permissions/`

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/permissions/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/permissions/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/permissions/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/permissions/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema

```
[
  {
    "id": string (uuid)
    "name": string
```

```
    "value": string
    "description": string
    "children": [
      object
    ]
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "value": "Response string",
    "description": "Response string",
    "children": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response example name",
        "description": "Response example description"
      }
    ]
  }
]
```

Get role list

GET `/api/v1/organizations/{organizationPk}/groups/`

Parameters

Parameter name	Required	Type	Description

`organizationPk`	V	string	A UUID string identifying this Organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/?
page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/?
page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "type":
      {
      }
      "permissions": [ // Return the list of group permissions

Processing steps:
1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission has no parent permission loaded, automatically
load the missing parent permission from the database
4. Serialize parent permissions and their corresponding child
permissions, then sort by order
        object
      ]
      "membersPreview": [
        string
      ]
      "membersCount": integer // Return the remaining member count after
subtracting the 3 members shown in the list, for the frontend +N badge

Unlike resource counts:
<ul>
  <li>members_count: Return total - 3 (for the +N badge); None indicates
a total <= 3</li>
  <li>chatbots_count/knowledge_bases_count/inboxes_count: Return the
total count</li>
</ul>
      "chatbotsCount": integer
      "chatbotsPreview": [
        string
      ]
      "knowledgeBasesCount": integer
      "knowledgeBasesPreview": [
        string
      ]
      "inboxesCount": integer
      "inboxesPreview": [
        string
      ]
```

```
      "databasesCount": integer
      "databasesPreview": [
        string
      ]
      "canCreateChatbot"?: boolean // Optional
      "canCreateKnowledgeBase"?: boolean // Optional
      "canCreateInbox"?: boolean // Optional
      "canCreateDatabase"?: boolean // Optional
      "canCreateTool"?: boolean // Optional
      "canCreateAgentUiTool"?: boolean // Optional
      "canCreateSkill"?: boolean // Optional
      "maigptCanCodeInterpreter"?: boolean // Whether members of this
role may use the code interpreter built-in tool in MaiGPT. (Optional)
      "maigptCanRag"?: boolean // Whether members of this role may use
the knowledge base RAG built-in tool in MaiGPT. (Optional)
      "maigptCanDeepResearch"?: boolean // Whether members of this role
may use Deep Research in MaiGPT. Deep Research reaches the public
internet through its own built-in search, so a role that revokes the
official web-search tools needs this switch too. (Optional)
      "maigptCanBrowserTool"?: boolean // Whether members of this role
may use the browser built-in tool in MaiGPT. (Optional)
      "maigptAllLlms"?: boolean // Whether members of this role may use
every organization-available LLM in MaiGPT. Off = only the bound LLMs; an
empty binding then means none. (Optional)
      "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (Optional)
      "maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (Optional)
      "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (Optional)
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
```

```
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "type": {},
      "permissions": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response example name",
          "description": "Response example description"
        }
      ],
      "membersPreview": [
        "Response string"
      ],
      "membersCount": 456,
      "chatbotsCount": 456,
      "chatbotsPreview": [
        "Response string"
      ],
      "knowledgeBasesCount": 456,
      "knowledgeBasesPreview": [
        "Response string"
      ],
      "inboxesCount": 456,
      "inboxesPreview": [
        "Response string"
      ],
      "databasesCount": 456,
      "databasesPreview": [
        "Response string"
      ],
      "canCreateChatbot": false,
      "canCreateKnowledgeBase": false,
      "canCreateInbox": false,
      "canCreateDatabase": false,
      "canCreateTool": false,
      "canCreateAgentUiTool": false,
      "canCreateSkill": false,
      "maigptCanCodeInterpreter": false,
      "maigptCanRag": false,
      "maigptCanDeepResearch": false,
      "maigptCanBrowserTool": false,
      "maigptAllLlms": false,
      "maigptAllTools": false,
```

```
        "maigptAllConnectors": false,
        "maigptAllSkills": false,
        "createdAt": "Response string"
      }
    ]
}
```


Get role details

GET `/api/v1/organizations/{organizationPk}/groups/{id}`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this Group.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "id": string (uuid)
```

```
  "name": string
  "type":
  {
  }
  "permissions": [ // Return the list of group permissions

Processing steps:
1. Separate group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If a child permission has no parent permission loaded, automatically
load the missing parent permission from the database
4. Serialize parent permissions and their corresponding child
permissions, then sort by order
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Return the remaining member count after
subtracting the 3 members shown in the list, for the frontend +N badge

Unlike resource counts:
<ul>
  <li>members_count: Return total - 3 (for the +N badge); None indicates
a total <= 3</li>
  <li>chatbots_count/knowledge_bases_count/inboxes_count: Return the
total count</li>
</ul>
  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]
  "databasesCount": integer
  "databasesPreview": [
    string
  ]
  "canCreateChatbot"?: boolean // Optional
```

```
  "canCreateKnowledgeBase"?: boolean // Optional
  "canCreateInbox"?: boolean // Optional
  "canCreateDatabase"?: boolean // Optional
  "canCreateTool"?: boolean // Optional
  "canCreateAgentUiTool"?: boolean // Optional
  "canCreateSkill"?: boolean // Optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (Optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (Optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (Optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (Optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (Optional)
  "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (Optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (Optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (Optional)
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
```

```
  "membersPreview": [
    "Response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "Response string"
  ],
  "knowledgeBasesCount": 456,
  "knowledgeBasesPreview": [
    "Response string"
  ],
  "inboxesCount": 456,
  "inboxesPreview": [
    "Response string"
  ],
  "databasesCount": 456,
  "databasesPreview": [
    "Response string"
  ],
  "canCreateChatbot": false,
  "canCreateKnowledgeBase": false,
  "canCreateInbox": false,
  "canCreateDatabase": false,
  "canCreateTool": false,
  "canCreateAgentUiTool": false,
  "canCreateSkill": false,
  "maigptCanCodeInterpreter": false,
  "maigptCanRag": false,
  "maigptCanDeepResearch": false,
  "maigptCanBrowserTool": false,
  "maigptAllLlms": false,
  "maigptAllTools": false,
  "maigptAllConnectors": false,
  "maigptAllSkills": false,
  "createdAt": "Response string"
}
```


List Role Members

GET `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/`

Parameters

Parameter name	Required	Type	Description
`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this Organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "member": {
      {
        "id": string (uuid)
        "name": string
        "email": string
        "isActive": boolean
        "isOwner": boolean // Check whether the member is an organization
owner (via the OWNER Group)

Use caching to avoid repeated queries within the same request
Cache lifetime: valid for the lifetime of the member instance

Prefer annotation results (if available) to avoid repeated queries
        "permissions": [ // Compute the permission tree once per
(organization, custom-group set) per request.

The tree depends only on the member's organization (default groups,
feature
flags, plan whitelist), the member's custom groups, and the global
``Config``,
so members sharing a group set get the same result. A large role's
``pagination=false`` listing otherwise recomputes it per row — one Redis
``Config`` fetch plus the permission walk each time (issue #8436; the
largest
prod role has 3,837 members and a single group set). The memo lives in
the
serializer context, so it never outlives the request. Rows without the
eager
loaded ``_custom_groups_cache`` (single-object paths) keep computing
directly.
          object
        ]
        "groups": [
          object
        ]
        "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
```

```
        "activeQuota": object // Return the most recent quota summary
(active first), via prefetch to avoid N+1.
        "metadata": object // Organization-scoped member attributes,
injected into LLM system prompt
        "availabilityStatus":  // Self-declared availability for auto-
assignment. Paused members are skipped by the dispatch engine but can
still be manually assigned or claim conversations. Not touched by SCIM
provisioning.

* `available` - Available
* `paused` - Paused
        {
        }
        "createdAt": string (timestamp)
      }
      }
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "member": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "email": "Response string",
        "isActive": false,
        "isOwner": false,
        "permissions": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response example name",
            "description": "Response example description"
          }
        ],
        "groups": [
```

```
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response example name",
            "description": "Response example description"
          }
        ],
        "creditQuotaRequired": false,
        "activeQuota": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response example name",
          "description": "Response example description"
        },
        "metadata": null,
        "availabilityStatus": {},
        "createdAt": "Response string"
      },
      "createdAt": "Response string"
    }
  ]
}
```


Get a Specific Role Member

GET `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/{id}`

Parameters

Parameter name	Required	Type	Description
`groupPk`	V	string	
`id`	V	string	A UUID string identifying this Group Member.

`organizationPk`	V	string	A UUID string identifying this Organization ID

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "id": string (uuid)
```

```
  "member": {
  {
    "id": string (uuid)
    "name": string
    "email": string
    "isActive": boolean
    "isOwner": boolean // Checks whether the member is an organization
owner(via the OWNER Group)

Use a cache to avoid duplicate queries within the same request
Cache lifetime: valid for as long as the member instance exists

Prefer the annotation result (if available) to avoid duplicate queries
    "permissions": [ // Compute the permission tree once per
(organization, custom-group set) per request.

The tree depends only on the member's organization (default groups,
feature
flags, plan whitelist), the member's custom groups, and the global
``Config``,
so members sharing a group set get the same result. A large role's
``pagination=false`` listing otherwise recomputes it per row — one Redis
``Config`` fetch plus the permission walk each time (issue #8436; the
largest
prod role has 3,837 members and a single group set). The memo lives in
the
serializer context, so it never outlives the request. Rows without the
eager
loaded ``_custom_groups_cache`` (single-object paths) keep computing
directly.
      object
    ]
    "groups": [
      object
    ]
    "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
    "activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
    "metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
    "availabilityStatus":  // Self-declared availability for auto-
assignment. Paused members are skipped by the dispatch engine but can
still be manually assigned or claim conversations. Not touched by SCIM
provisioning.
```

```
* `available` - Available
* `paused` - Paused
    {
    }
    "createdAt": string (timestamp)
  }
  }
  "createdAt": string (timestamp)
}
```

Response example values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "member": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "email": "Response string",
    "isActive": false,
    "isOwner": false,
    "permissions": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      }
    ],
    "creditQuotaRequired": false,
    "activeQuota": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    },
    "metadata": null,
    "availabilityStatus": {},
    "createdAt": "Response string"
```

```
  },
  "createdAt": "Response string"
}
```

List Role AI Assistants

GET `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/`

Parameters

Parameter name	Required	Type	Description
`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this Organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response schema example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "chatbot":
      {
        "id": string (uuid)
        "name": string // Chatbot name. In Agent mode it carries semantic meaning; in other modes it only distinguishes chatbots.
        "largeLanguageModel": string (uuid) // Large language model used by the chatbot to generate answers (including organization-owned AI Gateway models)
        "llm":  // ID, name, and retirement information of the bound model, read directly from the FK; returned even if the model has been disabled by the organization
        {
          "id": string (uuid)
          "name": string
          "retiresAt": string (date) // Provider retirement date; once it passes, a daily job switches the assistant to the replacement and disables this model. null means no retirement is scheduled
          "replacement":  // Model that takes over after retirement (id and name); null means none is specified and nothing is migrated automatically on expiry
          {
            "id": string (uuid)
            "name": string
          }
        }
        "embeddingModel"?: string (uuid) // Embedding model used to vectorize text; optional (Optional)
        "rerankerModel"?: string (uuid) // Model used for reranking; optional (Optional)
        "instructions"?: string // Role instructions describing the chatbot's role and behavior. (Optional)
        "knowledgeBases"?: [ // List of knowledge bases the chatbot can access (Optional)
          {
            "id": string (uuid)
            "name": string
```

```
          }
        ]
        "databases"?: [ // List of SQL databases the chatbot can access
(used by the Text-to-SQL feature) (Optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "updatedAt": string (timestamp)
        "organization"?: string (uuid) // Organization the chatbot
belongs to; if empty, it is a personal chatbot (Optional)
        "replyMode"?:  // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `agent` - Agent (Optional)
        {
        }
        "template"?: string // Template used by Template and Hybrid
modes. (Optional)
        "unanswerableTemplate"?: string // Template used by Template and
Hybrid modes when the question cannot be answered. (Optional)
        "totalWordsCount"?: integer (int64) // Accumulated total word
count used. (Optional)
        "outputMode"?:  // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
        {
        }
        "rawOutputFormat"?: object // JSON structure definition of the
custom output format (Optional)
        "groups"?: [ // List of groups the chatbot can access (Optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "tools"?: [ // List of tools the chatbot can use (Optional)
          {
            "id": string (uuid)
            "name": string
            "description": string
```

```
            "displayName": string
            "toolType": string
          }
        ]
        "skills"?: [ // List of skills associated with the chatbot
(Optional)
          {
            "id": string (uuid)
            "name": string
            "description": string
          }
        ]
        "connectors"?: [ // Connectors associated with this chatbot,
gating per-contact OAuth authorization (Optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "agentMode"?:  // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
        {
        }
        "numberOfRetrievedChunks"?: integer // Number of reference chunks
to retrieve; default is 12, minimum is 1 (Optional)
        "defaultLabelRelations"?: object // Assistant-level default
knowledge base label filter, in the same format as
query_metadata.label_relations; null means no filtering. Applied as the
lowest-priority fallback only when the caller does not supply
query_metadata. (Optional)
        "useFullRetrievalRange"?: boolean // When enabled, ignores the
LLM's top_k truncation and returns the full reranker results (Optional)
        "enableEvaluation"?: boolean // Optional
        "enableInlineCitations"?: boolean // Enables inline citations,
inserting citation markers such as [1][2] into responses. (Optional)
        "enableCodeInterpreter"?: boolean // When enabled, provides the
Code Interpreter tool to run code in an isolated container. (Optional)
        "enableBrowserTool"?: boolean // Enable browser automation tool
for web tasks (Optional)
        "enableImageSearch"?: boolean // Provide the image_search tool
when any attached knowledge base uses a multimodal embedding model.
Disable to remove the tool entirely. (Optional)
```

```
        "enableClaudeStreamRetry"?: boolean // When a Claude streaming
call stalls (first-chunk or mid-stream), abort and re-issue the call once
in non-streaming mode. Mid-stream retries send a reset signal so the
client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
        "customMaxLlmOutputTokens"?: integer // Custom maximum number of
LLM output tokens; minimum is 512 (Optional)
        "voiceConfig"?: { // Optional
        {
          "enabled"?: boolean // Whether this chatbot can be used as a
voice agent. Checked by every conversation platform (WebChat, SIP phone,
admin/marketplace voice test) instead of a per-platform toggle.
(Optional)
          "voiceAgentType"?: string (uuid) // Voice agent mode type.
(Optional)
          "sttProvider"?: string (uuid) // Speech-to-Text service
provider. (Optional)
          "sttConfig"?: object // STT configuration parameters (JSON).
(Optional)
          "ttsProvider"?: string (uuid) // Text-to-Speech service
provider. (Optional)
          "ttsConfig"?: object // TTS configuration parameters (JSON).
(Optional)
          "realtimeProvider"?: string (uuid) // Realtime end-to-end voice
model provider. (Optional)
          "realtimeConfig"?: object // Realtime configuration parameters
(JSON). (Optional)
          "realtimeInstructions"?: string // Prompt dedicated to the
realtime voice session; falls back to chatbot instructions when empty
(Optional)
          "turnHandling"?: object // Turn-taking and interruption
settings for the voice agent. (Optional)
          "enableVoiceReply"?: boolean // When disabled, the voice agent
replies in text only and does not speak (Optional)
        }
        }
        "thinkingConfig"?: object // Thinking level: null (model default)
or {"reasoning_effort": "off|minimal|low|medium|high|xhigh|max"}
(Optional)
        "enableToolSearching"?: boolean // Enables dynamic tool search,
allowing the agent to discover and load tools dynamically via
tool_searching_tool (Optional)
        "maxAgentIterations"?: integer // Maximum number of iterations
for AgentWorkflow (1-100, default: 20) (Optional)
        "agentTimeoutSeconds"?: integer // Timeout in seconds for
```

```
AgentWorkflow execution (30-1200, default: 600) (Optional)
        "agentTimeoutMessage"?: string // Custom user-facing message
shown when AgentWorkflow times out (max 500 chars). Leave empty to use
the system default. (Optional)
        "requireTextWithFlexCards"?: boolean // When enabled, a turn that
would end with only Flex Message cards and no text for the user is re-
issued to the LLM once, asking it to write the text first and then send
the card. Each retry incurs a second billing for the same prompt.
(Optional)
        "isMultimodalLlm": boolean // Whether the chatbot LLM supports
multimodal (read-only)
        "enableSizeBasedToolResultTruncation"?: boolean // When enabled,
tool results exceeding the character threshold are kept as head+tail
(with the identifiers found in the cut part) when written to conversation
history; knowledge base, SQL and conversation search results keep their
100-character cut either way. Reduces memory bloat from large tool
outputs (code interpreter bash stdout, web search results, etc.).
(Optional)
        "toolResultMaxChars"?: integer // Character threshold for size-
based tool result truncation (500-50000). Only effective when
enable_size_based_tool_result_truncation is True. (Optional)
        "enableVectorMemory"?: boolean // When enabled, archived messages
are vectorized and processed by fact extraction for semantic retrieval
(existing behavior). When disabled, archived messages are discarded and
only the active chat history window is used for multi-turn context.
(Optional)
        "longTermMemoryTopK"?: integer // Number of memory nodes
retrieved per long-term memory lookup (similarity_top_k of the vector
search); default is 10, range 1–50 (Optional)
        "enableCrossConversationSearch"?: boolean // When enabled, the
conversation history tools also reach this contact's other conversations
in the same inbox, within the lookback window. When disabled, they only
reach the current conversation. (Optional)
        "crossConversationLookbackDays"?: integer // How far back cross-
conversation search may reach, measured from each conversation's last
message (1-365 days). Leave empty to fall back to 90 days. Only effective
when enable_cross_conversation_search is True. (Optional)
        "useGatewayFallback"?: boolean // When enabled, use the fallback
group instead of the single LLM (Optional)
        "gatewayFallbackGroup"?: string (uuid) // Optional
        "enableSearchMessagesTool"?: boolean // Allow the AI to search
past messages in the current conversation. (Optional)
        "enableGetMessageContextTool"?: boolean // Allow the AI to
retrieve messages surrounding a specific message for context. (Optional)
        "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to
```

```
read the full text of plain-text attachments. AGENT mode only. (Optional)
        "enableParseAttachmentContentTool"?: boolean // Allow the AI to
parse PDF, Office, audio, and other attachments. AGENT mode only.
(Optional)
        "enableListAvailableParsersTool"?: boolean // Allow the AI to
list available parsers and their capabilities. AGENT mode only.
(Optional)
        "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate
shareable download links for produced content. AGENT mode only.
(Optional)
        "isLocked": boolean // Locked instance: config follows the
official version; only the dedicated KB can be modified
        "sourceBuildInAgent": string (uuid) // Points to the official
Build-in Agent blueprint when instantiated by a rental
        "canUpdate": boolean // Return whether the current user can
update this resource.
        "canDelete": boolean // Return whether the current user can
delete this resource.
      }
      "canRead"?: boolean // Optional
      "canUpdate"?: boolean // Optional
      "canDelete"?: boolean // Optional
      "createdAt": string (timestamp)
    }
  ]
}
```

Response example values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
        "llm": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string",
```

```
    "retiresAt": "Response string",
    "replacement": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response string",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "updatedAt": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response string",
  "unanswerableTemplate": "Response string",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "displayName": "Response string",
      "toolType": "Response string"
    }
  ],
  "skills": [
```

```
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "description": "Response string"
      }
    ],
    "connectors": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 456,
    "defaultLabelRelations": null,
    "useFullRetrievalRange": false,
    "enableEvaluation": false,
    "enableInlineCitations": false,
    "enableCodeInterpreter": false,
    "enableBrowserTool": false,
    "enableImageSearch": false,
    "enableClaudeStreamRetry": false,
    "customMaxLlmOutputTokens": 456,
    "voiceConfig": {
      "enabled": false,
      "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
      "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
      "sttConfig": null,
      "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
      "ttsConfig": null,
      "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
      "realtimeConfig": null,
      "realtimeInstructions": "Response string",
      "turnHandling": null,
      "enableVoiceReply": false
    },
    "thinkingConfig": null,
    "enableToolSearching": false,
    "maxAgentIterations": 456,
    "agentTimeoutSeconds": 456,
    "agentTimeoutMessage": "Response string",
    "requireTextWithFlexCards": false,
    "isMultimodalLlm": false,
    "enableSizeBasedToolResultTruncation": false,
    "toolResultMaxChars": 456,
```

```
        "enableVectorMemory": false,
        "longTermMemoryTopK": 456,
        "enableCrossConversationSearch": false,
        "crossConversationLookbackDays": 456,
        "useGatewayFallback": false,
        "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
        "enableSearchMessagesTool": false,
        "enableGetMessageContextTool": false,
        "enableReadAttachmentFullTextTool": false,
        "enableParseAttachmentContentTool": false,
        "enableListAvailableParsersTool": false,
        "enableGetDownloadLinkTool": false,
        "isLocked": false,
        "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
        "canUpdate": false,
        "canDelete": false
      },
      "canRead": false,
      "canUpdate": false,
      "canDelete": false,
      "createdAt": "Response string"
    }
  ]
}
```


List Role Inboxes

`GET` `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/`

Parameters

Parameter Name	Required	Type	Description
`groupPk`	V	string	

`organizationPk`	V	string	A UUID string identifying this organization ID
`channelType`	X	string	`line`: LINE ; `telegram`: Telegram ; `teams`: Teams ; `web`: Web ; `messenger`: Messenger ; `instagram`: Instagram ; `email`: Email ; `whatsapp`: WhatsApp ; `slack`: Slack ; `team_plus`: Team+ ; `lin...
`chatbot`	X	string	
`isActive`	X	boolean	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-41d4-a716-446655440000&isActive=true&page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-
41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&query=example&search=ex
ample", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-41d4-a716-446655440000&isActive=true&page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-
41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&query=example&search=ex
ample", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "inbox":
      {
        "id": string (uuid)
        "channelType": string (enum: line, telegram, teams, web,
messenger, instagram, email, whatsapp, slack, team_plus, line_works,
viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
        "name": string
        "unreadConversationsCount": integer
        "accessType": string
      }
      "canRead"?: boolean // optional
      "canUpdate"?: boolean // optional
      "canDelete"?: boolean // optional
      "canViewConversations"?: boolean // Whether group members can
browse all conversations of this inbox in the conversations page.
Independent from can_read, which controls using the inbox for chatting.
(optional)
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "inbox": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "channelType": "line",
        "name": "Example response string",
        "unreadConversationsCount": 456,
        "accessType": "Example response string"
      },
      "canRead": false,
      "canUpdate": false,
      "canDelete": false,
      "canViewConversations": false,
      "createdAt": "Example response string"
    }
  ]
}
```


List Role Knowledge Bases

`GET` `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/`

Parameters

Parameter Name	Required	Type	Description
`groupPk`	V	string	

`organizationPk`	V	string	A UUID string identifying this organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/?
page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/?
page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "knowledgeBase":
      {
        "id": string (uuid)
        "createdBy":
        {
          "id": string (uuid)
          "name": string
        }
        "organization"?: string (uuid) // optional
        "embeddingModel": string (uuid)
        "rerankerModel": string (uuid)
        "name": string
        "description"?: string // optional
        "numberOfRetrievedChunks"?: integer // Number of retrieved
reference chunks. Default is 12, minimum is 1 (optional)
        "enableHyde"?: boolean // Enable HyDE. Default is False
(optional)
        "similarityCutoff"?: number (double) // Similarity threshold.
Default is 0.0, range 0.0-1.0 (optional)
        "enableRerank"?: boolean // Whether to enable reranking. Default
is True (optional)
        "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (optional)
        "vectorDatabaseType"?:  // Type of vector database: elasticsearch
or opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
        {
        }
        "vectorDatabaseUrl"?: string // Custom vector database endpoint
URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
```

```
"host:1521/service_name"). (optional)
        "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (optional)
        "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (optional)
        "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (optional)
        "oracleUser"?: string // Oracle database username. (optional)
        "oraclePassword"?: string // Oracle database password. (optional)
        "synxdbUser"?: string // SynxDB database username. (optional)
        "synxdbPassword"?: string // SynxDB database password. (optional)
        "labels": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "chatbots"?: [ // optional
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "groups"?: [ // List of groups that can access the knowledge base
(optional)
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
        "vectorStorageSize": integer
        "chunksCount": integer
        "canUpdate": boolean // Return whether the current user can
update this resource.
        "canDelete": boolean // Return whether the current user can
delete this resource.
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
      }
      "canRead"?: boolean // optional
      "canUpdate"?: boolean // optional
      "canDelete"?: boolean // optional
      "createdAt": string (timestamp)
```

```
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "createdBy": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Example response string"
        },
        "organization": "550e8400-e29b-41d4-a716-446655440000",
        "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
        "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response string",
        "description": "Example response string",
        "numberOfRetrievedChunks": 456,
        "enableHyde": false,
        "similarityCutoff": 456,
        "enableRerank": false,
        "enableRrf": false,
        "vectorDatabaseType": {},
        "vectorDatabaseUrl": "Example response string",
        "vectorDatabaseApiKey": "Example response string",
        "opensearchUsername": "Example response string",
        "opensearchPassword": "Example response string",
        "oracleUser": "Example response string",
        "oraclePassword": "Example response string",
        "synxdbUser": "Example response string",
        "synxdbPassword": "Example response string",
        "labels": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response string"
          }
```

```
        ],
        "chatbots": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response string"
          }
        ],
        "groups": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response string"
          }
        ],
        "filesCount": 456,
        "vectorStorageSize": 456,
        "chunksCount": 456,
        "canUpdate": false,
        "canDelete": false,
        "createdAt": "Example response string",
        "updatedAt": "Example response string"
      },
      "canRead": false,
      "canUpdate": false,
      "canDelete": false,
      "createdAt": "Example response string"
    }
  ]
}
```


List Role Databases

`GET` `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/`

Parameters

Parameter Name	Required	Type	Description
`groupPk`	V	string	
`organizationPk`	V	string	A UUID string identifying this organization ID
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "group": string (uuid)
      "database":
      {
        "id": string (uuid)
        "name": string // Display name for this database connection
        "description": string // Description of the database and its
purpose
        "databaseType":  // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
        {
        }
        "includeTables": object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files.
        "isActive": boolean // When disabled, this database will not be
used in Text-to-SQL queries
        "status":
        {
        }
        "deletedAt": string (timestamp)
        "tables": [
          {
            "id": string (uuid)
            "tableName": string // Name of the table in the database
            "sheetName": string // Name of the source worksheet/sheet
this table was created from. Empty for tables created before this field
was added.
            "description"?: string // Description of the table and its
data for better Text-to-SQL understanding (optional)
            "status":
```

```
            {
            }
            "errorMessage": string // Error details if table creation or
deletion failed
            "sourceFile":
            {
              "id": string (uuid)
              "name": string
              "isFromKnowledgeBase": boolean // Determine if the source
file originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
            }
            "columns": [
              {
                "id": string (uuid)
                "columnName": string
                "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
                "description"?: string // Description of the column for
better Text-to-SQL understanding (optional)
                "isPrimaryKey"?: boolean // optional
                "isNullable"?: boolean // optional
                "createdAt": string (timestamp)
                "updatedAt": string (timestamp)
              }
            ]
            "createdAt": string (timestamp)
            "updatedAt": string (timestamp)
          }
        ]
        "chatbots": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "groups": [
          {
            "id": string (uuid)
            "name": string
          }
        ]
        "createdAt": string (timestamp)
```

```
        "updatedAt": string (timestamp)
      }
      "canRead"?: boolean // optional
      "canUpdate"?: boolean // optional
      "canDelete"?: boolean // optional
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "group": "550e8400-e29b-41d4-a716-446655440000",
      "database": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response string",
        "description": "Example response string",
        "databaseType": {},
        "includeTables": null,
        "isActive": false,
        "status": {},
        "deletedAt": "Example response string",
        "tables": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "tableName": "Example response string",
            "sheetName": "Example response string",
            "description": "Example response string",
            "status": {},
            "errorMessage": "Example response string",
            "sourceFile": {
              "id": "550e8400-e29b-41d4-a716-446655440000",
              "name": "Example response string",
              "isFromKnowledgeBase": false
            },
            "columns": [
              {
```

```
                "id": "550e8400-e29b-41d4-a716-446655440000",
                "columnName": "Example response string",
                "columnType": "Example response string",
                "description": "Example response string",
                "isPrimaryKey": false,
                "isNullable": false,
                "createdAt": "Example response string",
                "updatedAt": "Example response string"
              }
            ],
            "createdAt": "Example response string",
            "updatedAt": "Example response string"
          }
        ],
        "chatbots": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response string"
          }
        ],
        "groups": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example response string"
          }
        ],
        "createdAt": "Example response string",
        "updatedAt": "Example response string"
      },
      "canRead": false,
      "canUpdate": false,
      "canDelete": false,
      "createdAt": "Example response string"
    }
  ]
}
```


Update Role Permissions

PUT `/api/v1/organizations/{organizationPk}/groups/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Group.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
organization	string (uuid)	No	
permissions	array[string]	Yes	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	

Field	Type	Required	Description
maigptCanCodeInterpreter	boolean	No	Whether members of this role may use the code interpreter built-in tool in MaiGPT.
maigptCanRag	boolean	No	Whether members of this role may use the knowledge base RAG built-in tool in MaiGPT.
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.
maigptAllLlms	boolean	No	Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none.
maigptAllTools	boolean	No	Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none.
maigptAllConnectors	boolean	No	Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none.
maigptAllSkills	boolean	No	Whether members of this role may use every official (global) skill in MaiGPT. Off = only the

Field	Type	Required	Description
			bound skills; an empty binding then means none.
allowedLlms	array[string]	No	
allowedTools	array[string]	No	
allowedConnectors	array[string]	No	
allowedSkills	array[string]	No	

Request Structure Example

```
{
  "name": string
  "organization"?: string (uuid) // optional
  "permissions": [
    string (uuid)
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
  "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may
```

```
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
  "allowedLlms"?: [ // optional
    string (uuid)
  ]
  "allowedTools"?: [ // optional
    string (uuid)
  ]
  "allowedConnectors"?: [ // optional
    string (uuid)
  ]
  "allowedSkills"?: [ // optional
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
```

```
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedSkills": [
```

```
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
      "550e8400-e29b-41d4-a716-446655440000"
```

```
    ],
    "allowedSkills": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.put("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": True,
    "canCreateKnowledgeBase": True,
    "canCreateInbox": True,
    "canCreateDatabase": True,
    "canCreateTool": True,
    "canCreateAgentUiTool": True,
    "canCreateSkill": True,
    "maigptCanCodeInterpreter": True,
    "maigptCanRag": True,
    "maigptCanDeepResearch": True,
    "maigptCanBrowserTool": True,
    "maigptAllLlms": True,
    "maigptAllTools": True,
    "maigptAllConnectors": True,
    "maigptAllSkills": True,
    "allowedLlms": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
```

```
    "allowedSkills": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'organization' => '550e8400-e29b-41d4-a716-
446655440000',
            'permissions' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'canCreateChatbot' => true,
            'canCreateKnowledgeBase' => true,
            'canCreateInbox' => true,
            'canCreateDatabase' => true,
            'canCreateTool' => true,
            'canCreateAgentUiTool' => true,
            'canCreateSkill' => true,
            'maigptCanCodeInterpreter' => true,
            'maigptCanRag' => true,
            'maigptCanDeepResearch' => true,
            'maigptCanBrowserTool' => true,
            'maigptAllLlms' => true,
            'maigptAllTools' => true,
            'maigptAllConnectors' => true,
            'maigptAllSkills' => true,
            'allowedLlms' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'allowedTools' => [
```

```
                    '550e8400-e29b-41d4-a716-446655440000'
                ],
                'allowedConnectors' => [
                    '550e8400-e29b-41d4-a716-446655440000'
                ],
                'allowedSkills' => [
                    '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]);

        $data = json_decode($response->getBody(), true);
        echo "Response received successfully:\n";
        print_r($data);
    } catch (Exception $e) {
        echo 'Request failed: ' . $e->getMessage();
    }
    ?>
```

Response

Status code: 200

Response Structure Example

```
{
  "id": string (uuid)
  "name": string
  "type":
  {
  }
  "permissions": [ // Permissions list of the returned group

Processing flow:
1. Split the group permissions into parent permissions and child
permissions
2. Group child permissions by parent permission ID
```

```
3. If any child permission is missing its parent permission,
automatically load the missing parent permission from the database
4. Serialize the parent permissions and their child permissions, and sort
by order
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Returns the remaining member count (after
subtracting the 3 members shown in the list) for the frontend +N badge

Unlike resource counts:
<ul>
  <li>members_count: returns total - 3 (for the +N badge); None means the
total is <= 3</li>
  <li>chatbots_count/knowledge_bases_count/inboxes_count: return the
total count</li>
</ul>
  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]
  "databasesCount": integer
  "databasesPreview": [
    string
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
```

```
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
  "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
  "createdAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example response name",
      "description": "Example response description"
    }
  ],
  "membersPreview": [
    "Example response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "Example response string"
  ],
  "knowledgeBasesCount": 456,
```

```
  "knowledgeBasesPreview": [
    "Example response string"
  ],
  "inboxesCount": 456,
  "inboxesPreview": [
    "Example response string"
  ],
  "databasesCount": 456,
  "databasesPreview": [
    "Example response string"
  ],
  "canCreateChatbot": false,
  "canCreateKnowledgeBase": false,
  "canCreateInbox": false,
  "canCreateDatabase": false,
  "canCreateTool": false,
  "canCreateAgentUiTool": false,
  "canCreateSkill": false,
  "maigptCanCodeInterpreter": false,
  "maigptCanRag": false,
  "maigptCanDeepResearch": false,
  "maigptCanBrowserTool": false,
  "maigptAllLlms": false,
  "maigptAllTools": false,
  "maigptAllConnectors": false,
  "maigptAllSkills": false,
  "createdAt": "Example response string"
}
```


Partially Update Role Permissions

PATCH `/api/v1/organizations/{organizationPk}/groups/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Group.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	
organization	string (uuid)	No	
permissions	array[string]	No	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	
maigptCanCodeInterpreter	boolean	No	Whether members of this role may use the code interpreter built-in tool in MaiGPT.
maigptCanRag	boolean	No	Whether members of this role may use the knowledge base RAG built-in tool in MaiGPT.
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research

Field	Type	Required	Description
			reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.
maigptAllLlms	boolean	No	Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none.
maigptAllTools	boolean	No	Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none.
maigptAllConnectors	boolean	No	Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none.
maigptAllSkills	boolean	No	Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none.
allowedLlms	array[string]	No	
allowedTools	array[string]	No	
allowedConnectors	array[string]	No	
allowedSkills	array[string]	No	

Request Structure Example

```
{
  "name"?: string // optional
  "organization"?: string (uuid) // optional
  "permissions"?: [ // optional
    string (uuid)
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
  "maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every
organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty
binding then means none. (optional)
  "maigptAllTools"?: boolean // Whether members of this role may call
every official (global) tool in MaiGPT. Off = only the bound tools; an
empty binding then means none. (optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may
call every official (global) MCP connector in MaiGPT. Off = only the
bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use
every official (global) skill in MaiGPT. Off = only the bound skills; an
empty binding then means none. (optional)
  "allowedLlms"?: [ // optional
    string (uuid)
  ]
  "allowedTools"?: [ // optional
    string (uuid)
  ]
  "allowedConnectors"?: [ // optional
    string (uuid)
  ]
```

```
  "allowedSkills"?: [ // optional
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "name": "Example name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "maigptAllLlms": true,
  "maigptAllTools": true,
  "maigptAllConnectors": true,
  "maigptAllSkills": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X PATCH
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedSkills": [
```

```
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "maigptAllLlms": true,
    "maigptAllTools": true,
    "maigptAllConnectors": true,
    "maigptAllSkills": true,
    "allowedLlms": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
      "550e8400-e29b-41d4-a716-446655440000"
```

```
    ],
    "allowedSkills": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.patch("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Successfully received response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "name": "Example name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": True,
    "canCreateKnowledgeBase": True,
    "canCreateInbox": True,
    "canCreateDatabase": True,
    "canCreateTool": True,
    "canCreateAgentUiTool": True,
    "canCreateSkill": True,
    "maigptCanCodeInterpreter": True,
    "maigptCanRag": True,
    "maigptCanDeepResearch": True,
    "maigptCanBrowserTool": True,
    "maigptAllLlms": True,
    "maigptAllTools": True,
    "maigptAllConnectors": True,
    "maigptAllSkills": True,
    "allowedLlms": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
```

```
    "allowedSkills": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Successfully received response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'organization' => '550e8400-e29b-41d4-a716-
446655440000',
            'permissions' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'canCreateChatbot' => true,
            'canCreateKnowledgeBase' => true,
            'canCreateInbox' => true,
            'canCreateDatabase' => true,
            'canCreateTool' => true,
            'canCreateAgentUiTool' => true,
            'canCreateSkill' => true,
            'maigptCanCodeInterpreter' => true,
            'maigptCanRag' => true,
            'maigptCanDeepResearch' => true,
            'maigptCanBrowserTool' => true,
            'maigptAllLlms' => true,
            'maigptAllTools' => true,
            'maigptAllConnectors' => true,
            'maigptAllSkills' => true,
            'allowedLlms' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'allowedTools' => [
```

```
                    '550e8400-e29b-41d4-a716-446655440000'
                ],
                'allowedConnectors' => [
                    '550e8400-e29b-41d4-a716-446655440000'
                ],
                'allowedSkills' => [
                    '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]);

        $data = json_decode($response->getBody(), true);
        echo "Successfully received response:\n";
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error: ' . $e->getMessage();
    }
    ?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "type":
  {
  }
  "permissions": [ // Returns the group's permission list

Processing flow:
1. Split the group's permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If any child permission is missing its parent, automatically load the
```

```
missing parent permissions from the database
4. Serialize the parent permissions and their child permissions, sorted
by order
    object
  ]
  "membersPreview": [
    string
  ]
  "membersCount": integer // Returns the number of remaining members
(after the 3 shown in the list), used for the frontend +N badge

Unlike the resource counts:
<ul>
  <li>members_count: returns total - 3 (for the +N badge); None means the
total is <= 3</li>
  <li>chatbots_count/knowledge_bases_count/inboxes_count: return the
total count</li>
</ul>
  "chatbotsCount": integer
  "chatbotsPreview": [
    string
  ]
  "knowledgeBasesCount": integer
  "knowledgeBasesPreview": [
    string
  ]
  "inboxesCount": integer
  "inboxesPreview": [
    string
  ]
  "databasesCount": integer
  "databasesPreview": [
    string
  ]
  "canCreateChatbot"?: boolean // optional
  "canCreateKnowledgeBase"?: boolean // optional
  "canCreateInbox"?: boolean // optional
  "canCreateDatabase"?: boolean // optional
  "canCreateTool"?: boolean // optional
  "canCreateAgentUiTool"?: boolean // optional
  "canCreateSkill"?: boolean // optional
  "maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (optional)
  "maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (optional)
```

```
  "maigptCanDeepResearch"?: boolean // Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this switch too. (optional)
  "maigptCanBrowserTool"?: boolean // Whether members of this role may use the browser built-in tool in MaiGPT. (optional)
  "maigptAllLlms"?: boolean // Whether members of this role may use every organization-available LLM in MaiGPT. Off = only the bound LLMs; an empty binding then means none. (optional)
  "maigptAllTools"?: boolean // Whether members of this role may call every official (global) tool in MaiGPT. Off = only the bound tools; an empty binding then means none. (optional)
  "maigptAllConnectors"?: boolean // Whether members of this role may call every official (global) MCP connector in MaiGPT. Off = only the bound connectors; an empty binding then means none. (optional)
  "maigptAllSkills"?: boolean // Whether members of this role may use every official (global) skill in MaiGPT. Off = only the bound skills; an empty binding then means none. (optional)
  "createdAt": string (timestamp)
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "type": {},
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "membersPreview": [
    "response string"
  ],
  "membersCount": 456,
  "chatbotsCount": 456,
  "chatbotsPreview": [
    "response string"
  ],
  "knowledgeBasesCount": 456,
  "knowledgeBasesPreview": [
```

```
    "response string"
  ],
  "inboxesCount": 456,
  "inboxesPreview": [
    "response string"
  ],
  "databasesCount": 456,
  "databasesPreview": [
    "response string"
  ],
  "canCreateChatbot": false,
  "canCreateKnowledgeBase": false,
  "canCreateInbox": false,
  "canCreateDatabase": false,
  "canCreateTool": false,
  "canCreateAgentUiTool": false,
  "canCreateSkill": false,
  "maigptCanCodeInterpreter": false,
  "maigptCanRag": false,
  "maigptCanDeepResearch": false,
  "maigptCanBrowserTool": false,
  "maigptAllLlms": false,
  "maigptAllTools": false,
  "maigptAllConnectors": false,
  "maigptAllSkills": false,
  "createdAt": "response string"
}
```


Delete Role

DELETE `/api/v1/organizations/{organizationPk}/groups/{id}`

Parameters

Parameter	Required	Type	Description

`id`	V	string	A UUID string identifying this Group.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request payload
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Successfully received response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Successfully received response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully received response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Remove Role Member

DELETE `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/{id}`

Parameters

Parameter	Required	Type	Description
`groupPk`	V	string	
`id`	V	string	A UUID string identifying this Group Member.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
members/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request payload
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Successfully received response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Successfully received response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully received response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Remove AI Assistant from Role

DELETE `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/{id}`

Parameters

Parameter	Required	Type	Description
`groupPk`	V	string	
`id`	V	string	A UUID string identifying this Group Chatbot.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
chatbots/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request payload
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Successfully received response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Successfully received response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully received response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Remove Inbox from Role

DELETE `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/{id}`

Parameters

Parameter	Required	Type	Description
`groupPk`	V	string	
`id`	V	string	A UUID string identifying this Group Inbox.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
inboxes/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request payload
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Successfully received response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Successfully received response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Remove Knowledge Base from Role

DELETE `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/{id}`

Parameters

Parameter Name	Required	Type	Description
`groupPk`	V	string	
`id`	V	string	A UUID string identifying this Group KnowledgeBase.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Remove Database from Role

DELETE `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`groupPk`	V	string	
`id`	V	string	A UUID string identifying this Group Database.
`organizationPk`	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
databases/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

API call example (Shell)

目錄

1. List test sets
2. Create an empty test set
3. Upload CSV or Excel to create a test set
4. Generate a test set from a knowledge base file
5. List knowledge base files available for test case generation
6. Retrieve a specific test set
7. Retrieve a test set with its single-turn test cases
8. Update a test set
9. Partially update a test set
10. Delete a test set
11. List single-turn test cases
12. Create a Single-Turn Test Case
13. Upload a CSV or Excel File to Add Test Cases
14. Generate Single-Turn Test Cases from Knowledge Base Files
15. List Test Case Generation Jobs
16. Get Test Case Generation Job Progress
17. Update Single-Turn Test Case
18. Partially Update Single-Turn Test Case
19. Delete Single-Turn Test Case
20. Bulk delete single-turn test cases
21. List multi-turn test cases
22. Create a multi-turn test case
23. Generate multi-turn test cases from a knowledge base file
24. Retrieve a specific multi-turn test case
25. Update Multi-turn Test Case
26. Partially Update Multi-turn Test Case
27. Delete Multi-turn Test Case
28. List Available Evaluation Models
29. Run Single-turn Evaluation
30. Run Multi-Turn Evaluation
31. List Evaluations
32. Get Evaluation Result
33. Get Per-Question Results of a Single-Turn Evaluation
34. Get Per-Conversation Results of a Multi-Turn Evaluation
35. Cancel a Running Evaluation

36. Resume an Unfinished Evaluation
37. Export Evaluation Report
38. Get Latest Export Status
39. List Export Records
40. Edit Evaluation Name and Description
41. Delete Evaluation

API call example (Shell)

The evaluation and test set endpoints correspond to "Test Sets" and "Automated Testing" in the AgentOps back office, letting you batch-test Agents from your own systems. The full flow is:

1. **Create a test set**: Use `POST /api/v1/dataset/` to create an empty test set, or use `POST /api/v1/dataset/upload/` to upload a CSV/Excel file and create one in a single step.
2. **Add test cases**: For single-turn Q&A, use `/api/v1/test-case/` (create one by one, upload a file, or generate from a knowledge base file); for multi-turn conversation scenarios, use `/api/v1/multi-turn-test-case/`. Generating from a knowledge base file runs in the background and returns `202` with a `jobId`; track it with Retrieve test case generation job progress.
3. **Run an evaluation**: Use `POST /api/v1/evaluations/evaluate/` (single-turn) or `POST /api/v1/evaluations/multi-turn-evaluate/` (multi-turn) to specify the test set and Agent. The response is `202` with an `evaluationId`.
4. **Read results and export**: Poll `GET /api/v1/evaluations/{id}/` until `status` becomes `completed`, `partial_completed`, or `failed`. Per-question results for single-turn evaluations are in `details`, and per-conversation results for multi-turn evaluations are in `multi-turn-results`. A completed single-turn evaluation can be exported as an Excel report with `export`, and the download URL is available from `export-status`.

Required permissions: The API Key calls the API as the member who created it. Listing and all write operations on `/dataset/`, `/test-case/`, and `/multi-turn-test-case/` require a role with the "Test Set permission"; listing and all write operations on `/evaluations/` require the "Automated Testing permission" (the AgentOps permissions in Role Permissions; owners always have them). Insufficient permissions return `403`. Members other than the owner can only see and evaluate Agents that their role is allowed to use.

Credit: Generating test cases from knowledge base files deducts the pricing item `agentops_test_gen` based on the number of questions actually generated. Running an evaluation deducts `agentops_test_run` and `agentops_deepeval` when it finishes, based on the number of completed test cases, and the evaluation model calls used for scoring are billed separately according to that model's usage. Before a generation or evaluation request is submitted, the system checks the organization's Credit and quota, and returns `402` if they are insufficient. Manual creation, file upload, and reading results do not deduct Credit; for unit prices, see List pricing items.

For features and back-office operations, see the user manual pages Test Set Management and Automated Evaluation and Agent Monitoring.

List test sets

`GET` `/api/v1/dataset/`

Parameters

Parameter Name	Required	Type	Description
`isActive`	X	boolean	
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	A search term.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/dataset/?
isActive=true&ordering=example&page=1&pageSize=1&query=example&searc
h=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/dataset/?
isActive=true&ordering=example&page=1&pageSize=1&query=example&searc
h=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/?
isActive=true&ordering=example&page=1&pageSize=1&query=example&searc
h=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/dataset/?
isActive=true&ordering=example&page=1&pageSize=1&query=example&searc
h=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "description"?: string // Optional
      "organization":
      {
        "id": string (uuid)
        "name": string
      }
      "createdBy":
      {
        "id": string (uuid)
        "name": string
      }
      "isActive"?: boolean // Optional
      "singleTurnTestCaseCount": integer
      "multiTurnTestCaseCount": integer
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "organization": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
```

```
      },
      "isActive": false,
      "singleTurnTestCaseCount": 456,
      "multiTurnTestCaseCount": 456,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Create an empty test set

POST `/api/v1/dataset/`

Test set names must be unique within an organization; a duplicate name returns 400.

Request Content

Request Parameters

Field	Type	Required	Description
name	string	Yes	
description	string	No	

Example Request Structure


```
{
  "name": string
  "description"?: string // Optional
}
```


Example Request Values

```
{
  "name": "Example name",
  "description": "Example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/dataset/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string"
  };

axios.post("https://api.maiagent.ai/api/v1/dataset/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/dataset/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
  "description"?: string // Optional
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "isActive"?: boolean // Optional
  "singleTurnTestCaseCount": integer
  "multiTurnTestCaseCount": integer
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "isActive": false,
  "singleTurnTestCaseCount": 456,
  "multiTurnTestCaseCount": 456,
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Upload CSV or Excel to create a test set

POST `/api/v1/dataset/upload/`

Upload as multipart/form-data: file and name are required, and description is optional. The file format is the same as Upload CSV or Excel to add test cases; test set names must be unique within an organization.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/dataset/upload/"
  -H "Authorization: Api-Key YOUR_API_KEY"
  -F "file=@example_file.pdf"
  -F "name=Example string"
  -F "description=Example string"
# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with an actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');
formData.append('name', 'Example string');
formData.append('description', 'Example string');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/dataset/upload/",
formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/upload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
'application/pdf'),
    'name': (None, 'Example string'),
    'description': (None, 'Example string')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/dataset/upload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ],
            [
                'name' => 'name',
                'contents' => 'Example string'
            ],
            [
                'name' => 'description',
                'contents' => 'Example string'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
  "description"?: string // Optional
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "isActive"?: boolean // Optional
  "singleTurnTestCaseCount": integer
  "multiTurnTestCaseCount": integer
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Response string"
  },
  "isActive": false,
  "singleTurnTestCaseCount": 456,
  "multiTurnTestCaseCount": 456,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Generate a test set from a knowledge base file

POST `/api/v1/dataset/generate-from-file/`

Creates a new test set from a single knowledge base file (knowledgeBaseFileId) and generates single-turn test cases; numTestCases ranges from 1 to 50 (default 10). Generation runs in the background, and the actual response is 202 with a taskId; it does not include the test set ID, and there is no progress to query. Once generation finishes, the new test set appears in the list. If you need to track progress, first create an empty test set and then use Generate single-turn test cases from a knowledge base file. Credit: The organization's Credit is checked before submission, and 402 is returned if it is insufficient; agentops_test_gen is deducted based on the number of questions actually generated.

Request Content

Request Parameters

Field	Type	Required	Description
name	string	Yes	
description	string	No	
chatbotFileId	string (uuid)	No	
knowledgeBaseFileId	string (uuid)	No	
numTestCases	integer	No	

Example Request Structure

```
{
  "name": string
  "description"?: string // Optional
  "chatbotFileId"?: string (uuid) // Optional
  "knowledgeBaseFileId"?: string (uuid) // Optional
  "numTestCases"?: integer // Optional
}
```

Example Request Values

```
{
  "name": "Example name",
  "description": "Example string",
  "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBaseFileId": "550e8400-e29b-41d4-a716-446655440000",
  "numTestCases": 123
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/dataset/generate-from-
file/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string",
    "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseFileId": "550e8400-e29b-41d4-a716-446655440000",
    "numTestCases": 123
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string",
    "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseFileId": "550e8400-e29b-41d4-a716-446655440000",
    "numTestCases": 123
  };

axios.post("https://api.maiagent.ai/api/v1/dataset/generate-from-
file/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/dataset/generate-from-file/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseFileId": "550e8400-e29b-41d4-a716-446655440000",
    "numTestCases": 123
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/dataset/generate-from-file/",
[
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string',
            'chatbotFileId' => '550e8400-e29b-41d4-a716-
446655440000',
            'knowledgeBaseFileId' => '550e8400-e29b-41d4-a716-
446655440000',
            'numTestCases' => 123
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
  "description"?: string // Optional
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "isActive"?: boolean // Optional
  "singleTurnTestCaseCount": integer
  "multiTurnTestCaseCount": integer
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
```

```
  "isActive": false,
  "singleTurnTestCaseCount": 456,
  "multiTurnTestCaseCount": 456,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


List knowledge base files available for test case generation

GET `/api/v1/dataset/knowledge-base-files/`

Grouped by knowledge base; only files that have finished processing and have the type txt, docx, pdf, md, markdown, or mdx are listed. pageSize is the maximum number of files returned per knowledge base (default 5, maximum 100), and search filters by file name.

Parameters

Parameter Name	Required	Type	Description
`pageSize`	X	integer	Number of files per knowledge base (default: 5, max: 100)
`search`	X	string	Filter files by filename

Code Examples

Shell/Bash

```shell
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/dataset/knowledge-base-files/?pageSize=1&search=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```javascript
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/dataset/knowledge-base-files/?pageSize=1&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/knowledge-base-files/?
pageSize=1&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/dataset/knowledge-base-files/?
pageSize=1&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
object
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
```


Retrieve a specific test set

GET `/api/v1/dataset/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
```

```
  "description"?: string // Optional
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "isActive"?: boolean // Optional
  "singleTurnTestCaseCount": integer
  "multiTurnTestCaseCount": integer
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "isActive": false,
  "singleTurnTestCaseCount": 456,
  "multiTurnTestCaseCount": 456,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Retrieve a test set with its single-turn test cases

`GET` `/api/v1/dataset/{id}/details/`

In addition to the test set fields, the response includes a testCases array that contains single-turn test cases only; for multi-turn test cases, use GET /api/v1/multi-turn-test-case/?datasetId={id}.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/details/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/details/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/details/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/details/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
```

```
  "description"?: string // Optional
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "isActive"?: boolean // Optional
  "singleTurnTestCaseCount": integer
  "multiTurnTestCaseCount": integer
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "isActive": false,
  "singleTurnTestCaseCount": 456,
  "multiTurnTestCaseCount": 456,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Update a test set

PUT `/api/v1/dataset/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	
description	string	No	
isActive	boolean	No	

Example Request Structure

```
{
  "name": string
  "description"?: string // Optional
  "isActive"?: boolean // Optional
}
```

Example Request Values

```
{
  "name": "Example name",
  "description": "Example string",
  "isActive": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string",
    "isActive": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string",
    "isActive": true
  };

axios.put("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "isActive": True
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string',
            'isActive' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
  "description"?: string // Optional
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "isActive"?: boolean // Optional
  "singleTurnTestCaseCount": integer
  "multiTurnTestCaseCount": integer
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "isActive": false,
  "singleTurnTestCaseCount": 456,
  "multiTurnTestCaseCount": 456,
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Partially update a test set

PATCH `/api/v1/dataset/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Dataset.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	
description	string	No	
isActive	boolean	No	

Example Request Structure

```
{
  "name"?: string // Optional
  "description"?: string // Optional
```

```
  "isActive"?: boolean // Optional
}
```

Example Request Values

```
{
  "name": "Example name",
  "description": "Example string",
  "isActive": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string",
    "isActive": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string",
    "isActive": true
  };

axios.patch("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "isActive": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string',
            'isActive' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
  "description"?: string // Optional
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "isActive"?: boolean // Optional
  "singleTurnTestCaseCount": integer
  "multiTurnTestCaseCount": integer
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "isActive": false,
  "singleTurnTestCaseCount": 456,
  "multiTurnTestCaseCount": 456,
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Delete a test set

DELETE `/api/v1/dataset/{id}`

Deletion cannot be undone. It also deletes all test cases in the test set, as well as the evaluations and results that were run with this test set.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/dataset/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/dataset/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
204	No response body

List single-turn test cases

`GET` `/api/v1/test-case/`

Use datasetId to filter by a single test set; you can also filter by status and creationMethod (manual, file, generate).

Parameters

Parameter Name	Required	Type	Description
`creationMethod`	X	string	`manual`: Manual ; `file`: File import ; `generate`: Generated;
`datasetId`	X	string	
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	A search term.
`status`	X	string	`active`: Active ; `inactive`: Inactive ; `pending`: Pending Review;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/test-case/?
creationMethod=file&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=active"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/test-case/?
creationMethod=file&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=active", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/?
creationMethod=file&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=active"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/test-case/?creationMethod=file&datasetId=550e8400-e29b-41d4-a716-446655440000&ordering=example&page=1&pageSize=1&query=example&search=example&status=active", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "dataset":
      {
        "id": string (uuid)
        "name": string
      }
      "question": string
      "groundTruth"?: string // Optional
      "status"?: string (enum: active, inactive, pending) // * `active` -
Active
* `inactive` - Inactive
* `pending` - Pending Review (Optional)
      "creationMethod"?: string (enum: manual, file, generate) // *
`manual` - Manual
* `file` - File import
* `generate` - Generated (Optional)
      "sourceFile"?: string // Name of the CSV, Excel or knowledge base
file this test case came from (Optional)
      "flaky"?: boolean // Flaky test cases still run and keep their
results, but are excluded from the pass rate. (Optional)
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "dataset": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "question": "Response string",
```

```
        "groundTruth": "Response string",
        "status": "active",
        "creationMethod": "manual",
        "sourceFile": "Response string",
        "flaky": false,
        "createdAt": "Response string",
        "updatedAt": "Response string"
      }
    ]
}
```


Create a Single-Turn Test Case

POST `/api/v1/test-case/`

datasetId, question, and groundTruth are required. status defaults to active; evaluations only run test cases that are active.

Request Content

Request Parameters

Field	Type	Required	Description
datasetId	string (uuid)	Yes	
question	string	Yes	
groundTruth	string	Yes	
status	string	No	
flaky	boolean	No	

Example Request Structure

```
{
  "datasetId": string (uuid)
  "question": string
  "groundTruth": string
  "status"?: string // Optional
  "flaky"?: boolean // Optional
}
```

Example Request Values

```
{
  "datasetId": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Example string",
  "groundTruth": "Example string",
  "status": "Example string",
  "flaky": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/test-case/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "Example string",
    "flaky": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "Example string",
    "flaky": true
  };

axios.post("https://api.maiagent.ai/api/v1/test-case/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "Example string",
    "flaky": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/test-case/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'datasetId' => '550e8400-e29b-41d4-a716-446655440000',
            'question' => 'Example string',
            'groundTruth' => 'Example string',
            'status' => 'Example string',
            'flaky' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Example Response Structure

```
{
  "id": string (uuid)
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "question": string
  "groundTruth"?: string // Optional
  "status"?: string (enum: active, inactive, pending) // * `active` -
Active
* `inactive` - Inactive
* `pending` - Pending Review (Optional)
  "creationMethod"?: string (enum: manual, file, generate) // * `manual`
- Manual
* `file` - File import
* `generate` - Generated (Optional)
  "sourceFile"?: string // Name of the CSV, Excel or knowledge base file
this test case came from (Optional)
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "question": "Response string",
  "groundTruth": "Response string",
  "status": "active",
  "creationMethod": "manual",
  "sourceFile": "Response string",
  "flaky": false,
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Upload a CSV or Excel File to Add Test Cases

POST `/api/v1/test-case/upload/`

Upload as multipart/form-data; both file and dataset_id are required. The file must be .csv, .xlsx, or .xls, and the header row must contain the question and ground_truth columns. Each row adds one single-turn test case; encode CSV files as UTF-8. The response is an array of the newly created test cases.

Parameters

Parameter Name	Required	Type	Description
`creationMethod`	X	string	`manual`: Manual ; `file`: File import ; `generate`: Generated;
`datasetId`	X	string	
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	A search term.

`status`		X	string	`active`: Active ; `inactive`: Inactive ; `pending`: Pending Review;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/test-case/upload/?
creationMethod=file&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=active"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -F "file=@example_file.pdf"

  -F "dataset_id=Example string"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create a FormData object
const formData = new FormData();
// Replace with the actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');
formData.append('dataset_id', 'Example string');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/test-case/upload/?
creationMethod=file&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=active", formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/upload/?
creationMethod=file&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=active"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
'application/pdf'),
    'dataset_id': (None, 'Example string')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/test-case/upload/?creationMethod=file&datasetId=550e8400-e29b-41d4-a716-446655440000&ordering=example&page=1&pageSize=1&query=example&search=example&status=active", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ],
            [
                'name' => 'dataset_id',
                'contents' => 'Example string'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "dataset":
      {
        "id": string (uuid)
        "name": string
      }
      "question": string
      "groundTruth"?: string // Optional
      "status"?: string (enum: active, inactive, pending) // * `active` -
Active
* `inactive` - Inactive
* `pending` - Pending Review (Optional)
      "creationMethod"?: string (enum: manual, file, generate) // *
`manual` - Manual
* `file` - File import
* `generate` - Generated (Optional)
      "sourceFile"?: string // Name of the CSV, Excel or knowledge base
file this test case came from (Optional)
      "flaky"?: boolean // Flaky test cases still run and keep their
results, but are excluded from the pass rate. (Optional)
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "dataset": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "question": "Response string",
      "groundTruth": "Response string",
      "status": "active",
      "creationMethod": "manual",
      "sourceFile": "Response string",
      "flaky": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Generate Single-Turn Test Cases from Knowledge Base Files

POST `/api/v1/test-case/generate-from-file/`

Specify the source with knowledgeBaseIds (expanded to all processed files of a supported type in the knowledge base) or knowledgeBaseFileIds; the two can be combined. Supported formats are txt, docx, pdf, md, markdown, and mdx. numTestCases is the total number of questions to generate in this run (default 10). If the number of files or questions exceeds the platform limit, the API returns 400 and the message states the limit. Generation runs in the background and the response is 202 with a jobId; if the same test set already has a generation job in progress, the existing job's jobId is returned and no duplicate is created. Credit: The organization's Credit is checked before submission, and 402 is returned if it is insufficient; agentops_test_gen is deducted based on the number of questions actually generated.

Request Content

Request Parameters

Field	Type	Required	Description
chatbotFileIds	array[string]	No	List of KnowledgeBaseFile IDs to generate test cases from (deprecated)
knowledgeBaseFileIds	array[string]	No	List of KnowledgeBaseFile IDs to generate test cases from
knowledgeBaseIds	array[string]	No	List of KnowledgeBase IDs to generate test cases from all their files
datasetId	string (uuid)	Yes	
numTestCases	integer	No	

Example Request Structure

```
{
  "chatbotFileIds"?: [ // List of KnowledgeBaseFile IDs to generate test cases from (deprecated) (Optional)
    string (uuid)
  ]
  "knowledgeBaseFileIds"?: [ // List of KnowledgeBaseFile IDs to generate test cases from (Optional)
    string (uuid)
  ]
  "knowledgeBaseIds"?: [ // List of KnowledgeBase IDs to generate test cases from all their files (Optional)
    string (uuid)
  ]
  "datasetId": string (uuid)
  "numTestCases"?: integer // Optional
}
```

Example Request Values

```
{
  "chatbotFileIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "knowledgeBaseFileIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "knowledgeBaseIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "datasetId": "550e8400-e29b-41d4-a716-446655440000",
  "numTestCases": 123
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/test-case/generate-
from-file/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "chatbotFileIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "knowledgeBaseFileIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "knowledgeBaseIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "numTestCases": 123
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "chatbotFileIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "knowledgeBaseFileIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "knowledgeBaseIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "numTestCases": 123
  };

axios.post("https://api.maiagent.ai/api/v1/test-case/generate-from-
file/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/test-case/generate-from-file/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "chatbotFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "knowledgeBaseFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "knowledgeBaseIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "numTestCases": 123
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/test-case/generate-from-file/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'chatbotFileIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'knowledgeBaseFileIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'knowledgeBaseIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'datasetId' => '550e8400-e29b-41d4-a716-446655440000',
            'numTestCases' => 123
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 202

Example Response Structure

```
object
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
```

List Test Case Generation Jobs

`GET` `/api/v1/test-case/generation-jobs/`

Lists the organization's single-turn and multi-turn generation jobs, sorted from newest to oldest. You can filter by datasetId and status.

Parameters

Parameter Name	Required	Type	Description
`creationMethod`	X	string	`manual`: Manual ; `file`: File import ; `generate`: Generated;

`datasetId`	X	string	Only lists generation jobs for this dataset
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	A search term.
`status`	X	string	Filter by status: pending running completed partial_completed failed

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/test-case/generation-
jobs/?
creationMethod=file&datasetId=example&ordering=example&page=1&pageSi
ze=1&query=example&search=example&status=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/test-case/generation-
jobs/?
creationMethod=file&datasetId=example&ordering=example&page=1&pageSi
ze=1&query=example&search=example&status=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/generation-jobs/?
creationMethod=file&datasetId=example&ordering=example&page=1&pageSi
ze=1&query=example&search=example&status=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/test-case/generation-jobs/?creationMethod=file&datasetId=example&ordering=example&page=1&pageSize=1&query=example&search=example&status=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "dataset":
      {
        "id": string (uuid)
        "name": string
      }
      "status"?: string (enum: pending, running, completed,
partial_completed, failed) // * `pending` - Pending
* `running` - Generating
* `completed` - Completed
* `partial_completed` - Partially completed
* `failed` - Generation failed (Optional)
      "jobType"?:  // Single-turn Q&A test data or multi-turn
conversation test data

* `single_turn` - Single turn
* `multi_turn` - Multi turn (Optional)
      {
      }
      "requestedNumTestCases"?: integer // Optional
      "totalFiles"?: integer // Optional
      "processedFiles"?: integer // Optional
      "generatedCount"?: integer // Optional
      "progressPercentage": number (double)
      "errorMessage"?: string // Error message recorded when the whole
batch fails (Optional)
      "fileResults"?: object // Result for each file, such as
[{'file_id', 'filename', 'status', 'generated', 'error'}] (Optional)
      "startedAt"?: string (timestamp) // Optional
      "completedAt"?: string (timestamp) // Optional
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
```

```
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "dataset": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "status": "pending",
      "jobType": {},
      "requestedNumTestCases": 456,
      "totalFiles": 456,
      "processedFiles": 456,
      "generatedCount": 456,
      "progressPercentage": 456,
      "errorMessage": "Response string",
      "fileResults": null,
      "startedAt": "Response string",
      "completedAt": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Get Test Case Generation Job Progress

`GET` `/api/v1/test-case/generation-jobs/{jobId}`

status is one of pending, running, completed, partial_completed, or failed. generatedCount is the number of questions generated so far, and fileResults lists the result and failure reason for each file. Both single-turn and multi-turn generation jobs are queried through this endpoint; use jobType to tell them apart.

Parameters

Parameter Name	Required	Type	Description
jobId	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/test-case/generation-
jobs/{jobId}
"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/test-case/generation-
jobs/{jobId}
", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/generation-
jobs/{jobId}
"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/test-
case/generation-jobs/{jobId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "dataset":
```

```
  {
    "id": string (uuid)
    "name": string
  }
  "status"?: string (enum: pending, running, completed,
partial_completed, failed) // * `pending` - Pending
* `running` - Generating
* `completed` - Completed
* `partial_completed` - Partially completed
* `failed` - Generation failed (Optional)
  "jobType"?:  // Single-turn Q&A test data or multi-turn conversation
test data

* `single_turn` - Single turn
* `multi_turn` - Multi turn (Optional)
  {
  }
  "requestedNumTestCases"?: integer // Optional
  "totalFiles"?: integer // Optional
  "processedFiles"?: integer // Optional
  "generatedCount"?: integer // Optional
  "progressPercentage": number (double)
  "errorMessage"?: string // Error message recorded when the whole batch
fails (Optional)
  "fileResults"?: object // Result for each file, such as [{'file_id',
'filename', 'status', 'generated', 'error'}] (Optional)
  "startedAt"?: string (timestamp) // Optional
  "completedAt"?: string (timestamp) // Optional
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "status": "pending",
  "jobType": {},
  "requestedNumTestCases": 456,
  "totalFiles": 456,
```

```
  "processedFiles": 456,
  "generatedCount": 456,
  "progressPercentage": 456,
  "errorMessage": "Response string",
  "fileResults": null,
  "startedAt": "Response string",
  "completedAt": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Update Single-Turn Test Case

PUT `/api/v1/test-case/{id}`

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Q&A pair.

Request Content

Request Parameters

Field	Type	Required	Description
question	string	Yes	
groundTruth	string	No	

Field	Type	Required	Description
status	string (enum: active, inactive, pending)	No	`active` : Active ; `inactive` : Inactive ; `pending` : Pending Review;
flaky	boolean	No	Flaky test cases still run and keep their results, but are excluded from the pass rate.

Example Request Structure

```
{
  "question": string
  "groundTruth"?: string // Optional
  "status"?: string (enum: active, inactive, pending) // * `active` -
Active
* `inactive` - Inactive
* `pending` - Pending Review (Optional)
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
}
```

Example Request Values

```
{
  "question": "Example string",
  "groundTruth": "Example string",
  "status": "active",
  "flaky": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "active",
    "flaky": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "active",
    "flaky": true
  };

axios.put("https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "active",
    "flaky": True
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'question' => 'Example string',
            'groundTruth' => 'Example string',
            'status' => 'active',
            'flaky' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "question": string
  "groundTruth"?: string // Optional
  "status"?: string (enum: active, inactive, pending) // * `active` -
Active
* `inactive` - Inactive
* `pending` - Pending Review (Optional)
  "creationMethod"?: string (enum: manual, file, generate) // * `manual`
- Manual
* `file` - File import
* `generate` - Generated (Optional)
  "sourceFile"?: string // Name of the CSV, Excel or knowledge base file
this test case came from (Optional)
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "question": "Response string",
  "groundTruth": "Response string",
  "status": "active",
  "creationMethod": "manual",
  "sourceFile": "Response string",
  "flaky": false,
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Partially Update Single-Turn Test Case

PATCH `/api/v1/test-case/{id}`

To temporarily skip a question, set status to inactive. Evaluations will skip it, and past evaluation results are kept.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Q&A pair.

Request Content

Request Parameters

Field	Type	Required	Description
question	string	No	
groundTruth	string	No	
status	string (enum: active, inactive, pending)	No	`active` : Active ; `inactive` : Inactive ; `pending` : Pending Review;
flaky	boolean	No	Flaky test cases still run and keep their results, but are excluded from the pass rate.

Example Request Structure

```
{
  "question"?: string // Optional
  "groundTruth"?: string // Optional
  "status"?: string (enum: active, inactive, pending) // * `active` -
Active
* `inactive` - Inactive
* `pending` - Pending Review (Optional)
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
}
```

Example Request Values

```
{
  "question": "Example string",
  "groundTruth": "Example string",
  "status": "active",
  "flaky": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/test-case/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "active",
    "flaky": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "active",
    "flaky": true
  };

axios.patch("https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "Example string",
    "groundTruth": "Example string",
    "status": "active",
    "flaky": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'question' => 'Example string',
            'groundTruth' => 'Example string',
            'status' => 'active',
            'flaky' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "question": string
  "groundTruth"?: string // Optional
  "status"?: string (enum: active, inactive, pending) // * `active` -
Active
* `inactive` - Inactive
* `pending` - Pending Review (Optional)
  "creationMethod"?: string (enum: manual, file, generate) // * `manual`
- Manual
* `file` - File import
* `generate` - Generated (Optional)
  "sourceFile"?: string // Name of the CSV, Excel or knowledge base file
this test case came from (Optional)
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "question": "Response string",
  "groundTruth": "Response string",
  "status": "active",
  "creationMethod": "manual",
  "sourceFile": "Response string",
  "flaky": false,
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string"
}
```

Delete Single-Turn Test Case

DELETE `/api/v1/test-case/{id}`

This also deletes this test case's per-question results from past evaluations. To only pause testing, use status instead.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Q&A pair.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/test-case/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/test-case/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-41d4-
a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/test-case/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description
204	No response body

Bulk delete single-turn test cases

`POST` `/api/v1/test-case/bulk-delete/`

Pass an array of test case IDs to delete as testCaseIds; success returns 204. The impact of deletion is the same as Delete a single-turn test case.

Request

Request Parameters

Field	Type	Required	Description
test_case_ids	array[string]	Yes	List of test case IDs to delete

Request Schema Example

```
{
  "test_case_ids": [ // List of test case IDs to delete
    string (uuid)
  ]
}
```

Request Example Values

```
{
  "test_case_ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/test-case/bulk-delete/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "test_case_ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "test_case_ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/test-case/bulk-delete/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/test-case/bulk-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "test_case_ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/test-case/bulk-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'test_case_ids' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code	Description

204	No response body

List multi-turn test cases

GET `/api/v1/multi-turn-test-case/`

Use datasetId to filter by a single test set.

Parameters

Parameter Name	Required	Type	Description
`dataset`	X	string	
`datasetId`	X	string	
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`search`	X	string	A search term.
`status`	X	string	`active`: Active ; `inactive`: Inactive;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/multi-turn-test-case/?
dataset=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-
e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&search=example&statu
s=active"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/multi-turn-test-case/?
dataset=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-
e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&search=example&statu
s=active", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/multi-turn-test-case/?
dataset=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-
e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&search=example&statu
s=active"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/multi-turn-test-case/?dataset=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-e29b-41d4-a716-446655440000&ordering=example&page=1&pageSize=1&search=example&status=active", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "dataset":
      {
        "id": string (uuid)
        "name": string
      }
      "scenario": string // Task scenario the simulated user should
accomplish
      "expectedOutcome": string // Outcome a successful conversation
should reach, also used as the simulation stop condition
      "userDescription"?: string // Persona of the simulated user, such
as tone, background and level of expertise (Optional)
      "status"?: string (enum: active, inactive) // * `active` - Active
* `inactive` - Inactive (Optional)
      "creationMethod":
      {
      }
      "sourceFile": string // Name of the knowledge base file this multi-
turn test case was generated from
      "flaky"?: boolean // Flaky test cases still run and keep their
results, but are excluded from the pass rate. (Optional)
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "dataset": {
```

```
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "scenario": "Response string",
      "expectedOutcome": "Response string",
      "userDescription": "Response string",
      "status": "active",
      "creationMethod": {},
      "sourceFile": "Response string",
      "flaky": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Create a multi-turn test case

POST `/api/v1/multi-turn-test-case/`

datasetId, scenario (the task scenario the simulated user should accomplish), and expectedOutcome (the outcome a successful conversation should reach, which is also the stop condition for the simulated conversation) are required; userDescription is optional and describes the simulated user's tone, background, and level of expertise.

Request

Request Parameters

Field	Type	Required	Description
datasetId	string (uuid)	Yes	
scenario	string	Yes	
expectedOutcome	string	Yes	

Field	Type	Required	Description
userDescription	string	No	
status	object	No	
flaky	boolean	No	

Request Schema Example

```
{
  "datasetId": string (uuid)
  "scenario": string
  "expectedOutcome": string
  "userDescription"?: string // Optional
  "status"?:  // Optional
  {
  }
  "flaky"?: boolean // Optional
}
```

Request Example Values

```
{
  "datasetId": "550e8400-e29b-41d4-a716-446655440000",
  "scenario": "Example string",
  "expectedOutcome": "Example string",
  "userDescription": "Example string",
  "status": {},
  "flaky": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/multi-turn-test-case/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": {},
    "flaky": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": {},
    "flaky": true
  };

axios.post("https://api.maiagent.ai/api/v1/multi-turn-test-case/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/multi-turn-test-case/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": {},
    "flaky": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/multi-turn-test-case/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'datasetId' => '550e8400-e29b-41d4-a716-446655440000',
            'scenario' => 'Example string',
            'expectedOutcome' => 'Example string',
            'userDescription' => 'Example string',
            'status' => new \stdClass(),
            'flaky' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "scenario": string // Task scenario the simulated user should
accomplish
  "expectedOutcome": string // Outcome a successful conversation should
reach, also used as the simulation stop condition
  "userDescription"?: string // Persona of the simulated user, such as
tone, background and level of expertise (Optional)
  "status"?: string (enum: active, inactive) // * `active` - Active
* `inactive` - Inactive (Optional)
  "creationMethod":
  {
  }
  "sourceFile": string // Name of the knowledge base file this multi-turn
test case was generated from
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "scenario": "Response string",
  "expectedOutcome": "Response string",
  "userDescription": "Response string",
  "status": "active",
  "creationMethod": {},
```

```
  "sourceFile": "Response string",
  "flaky": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Generate multi-turn test cases from a knowledge base file

POST `/api/v1/multi-turn-test-case/generate-from-file/`

Specify the source files with knowledgeBaseFileIds; numTestCases defaults to 5 and is at most 20. Generation runs in the background and the response is 202 with a jobId; track it with Retrieve test case generation job progress. Credit: Same as single-turn generation; agentops_test_gen is deducted based on the number of questions actually generated.

Request

Request Parameters

Field	Type	Required	Description
datasetId	string (uuid)	Yes	
knowledgeBaseFileIds	array[string]	Yes	List of KnowledgeBaseFile IDs to generate multi-turn test cases from
numTestCases	integer	No	

Request Schema Example

```
{
  "datasetId": string (uuid)
  "knowledgeBaseFileIds": [ // List of KnowledgeBaseFile IDs to generate
multi-turn test cases from
    string (uuid)
```

```
  ]
  "numTestCases"?: integer // Optional
}
```

Request Example Values

```
{
  "datasetId": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBaseFileIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "numTestCases": 123
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/multi-turn-test-
case/generate-from-file/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseFileIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "numTestCases": 123
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseFileIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ],
    "numTestCases": 123
  };

axios.post("https://api.maiagent.ai/api/v1/multi-turn-test-
case/generate-from-file/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/multi-turn-test-case/generate-
from-file/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "datasetId": "550e8400-e29b-41d4-a716-446655440000",
    "knowledgeBaseFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "numTestCases": 123
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/multi-turn-test-case/generate-from-file/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'datasetId' => '550e8400-e29b-41d4-a716-446655440000',
            'knowledgeBaseFileIds' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ],
            'numTestCases' => 123
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status code: 202

Response Schema Example

```
object
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
}
```

Retrieve a specific multi-turn test case

GET `/api/v1/multi-turn-test-case/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Multi-turn test case.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/multi-turn-test-case/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/multi-turn-test-case/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "dataset":
  {
```

```
    "id": string (uuid)
    "name": string
  }
  "scenario": string // Task scenario the simulated user should
accomplish
  "expectedOutcome": string // Outcome a successful conversation should
reach, also used as the simulation stop condition
  "userDescription"?: string // Persona of the simulated user, such as
tone, background and level of expertise (Optional)
  "status"?: string (enum: active, inactive) // * `active` - Active
* `inactive` - Inactive (Optional)
  "creationMethod":
  {
  }
  "sourceFile": string // Name of the knowledge base file this multi-turn
test case was generated from
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "scenario": "Response string",
  "expectedOutcome": "Response string",
  "userDescription": "Response string",
  "status": "active",
  "creationMethod": {},
  "sourceFile": "Response string",
  "flaky": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Update Multi-turn Test Case

PUT `/api/v1/multi-turn-test-case/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Multi-turn test case.

Request Content

Request Parameters

Field	Type	Required	Description
scenario	string	Yes	Task scenario the simulated user should accomplish
expectedOutcome	string	Yes	Outcome a successful conversation should reach, also used as the simulation stop condition
userDescription	string	No	Persona of the simulated user, such as tone, background and level of expertise
status	string (enum: active, inactive)	No	`active` : Active ; `inactive` : Inactive;
flaky	boolean	No	Flaky test cases still run and keep their results, but are excluded from the pass rate.

Example Request Structure

```
{
  "scenario": string // Task scenario the simulated user should
accomplish
  "expectedOutcome": string // Outcome a successful conversation should
reach, also used as the simulation stop condition
  "userDescription"?: string // Persona of the simulated user, such as
tone, background and level of expertise (Optional)
  "status"?: string (enum: active, inactive) // * `active` - Active
* `inactive` - Inactive (Optional)
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
}
```

Example Request Values

```
{
  "scenario": "Example string",
  "expectedOutcome": "Example string",
  "userDescription": "Example string",
  "status": "active",
  "flaky": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": "active",
    "flaky": true
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": "active",
    "flaky": true
  };

axios.put("https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/multi-turn-test-case/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": "active",
    "flaky": True
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/multi-turn-test-case/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'scenario' => 'Example string',
            'expectedOutcome' => 'Example string',
            'userDescription' => 'Example string',
            'status' => 'active',
            'flaky' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "scenario": string // Task scenario the simulated user should
accomplish
  "expectedOutcome": string // Outcome a successful conversation should
reach, also used as the simulation stop condition
  "userDescription"?: string // Persona of the simulated user, such as
tone, background and level of expertise (Optional)
  "status"?: string (enum: active, inactive) // * `active` - Active
* `inactive` - Inactive (Optional)
  "creationMethod":
  {
  }
  "sourceFile": string // Name of the knowledge base file this multi-turn
test case was generated from
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "scenario": "Response string",
  "expectedOutcome": "Response string",
  "userDescription": "Response string",
  "status": "active",
  "creationMethod": {},
  "sourceFile": "Response string",
  "flaky": false,
```

```
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Partially Update Multi-turn Test Case

PATCH `/api/v1/multi-turn-test-case/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Multi-turn test case.

Request Content

Request Parameters

Field	Type	Required	Description
scenario	string	No	Task scenario the simulated user should accomplish
expectedOutcome	string	No	Outcome a successful conversation should reach, also used as the simulation stop condition
userDescription	string	No	Persona of the simulated user, such as tone, background and level of expertise

Field	Type	Required	Description
status	string (enum: active, inactive)	No	`active` : Active ; `inactive` : Inactive;
flaky	boolean	No	Flaky test cases still run and keep their results, but are excluded from the pass rate.

Example Request Structure

```
{
  "scenario"?: string // Task scenario the simulated user should accomplish (Optional)
  "expectedOutcome"?: string // Outcome a successful conversation should reach, also used as the simulation stop condition (Optional)
  "userDescription"?: string // Persona of the simulated user, such as tone, background and level of expertise (Optional)
  "status"?: string (enum: active, inactive) // * `active` - Active * `inactive` - Inactive (Optional)
  "flaky"?: boolean // Flaky test cases still run and keep their results, but are excluded from the pass rate. (Optional)
}
```

Example Request Values

```
{
  "scenario": "Example string",
  "expectedOutcome": "Example string",
  "userDescription": "Example string",
  "status": "active",
  "flaky": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": "active",
    "flaky": true
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": "active",
    "flaky": true
  };

axios.patch("https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/multi-turn-test-case/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "scenario": "Example string",
    "expectedOutcome": "Example string",
    "userDescription": "Example string",
    "status": "active",
    "flaky": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'scenario' => 'Example string',
            'expectedOutcome' => 'Example string',
            'userDescription' => 'Example string',
            'status' => 'active',
            'flaky' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "scenario": string // Task scenario the simulated user should
accomplish
  "expectedOutcome": string // Outcome a successful conversation should
reach, also used as the simulation stop condition
  "userDescription"?: string // Persona of the simulated user, such as
tone, background and level of expertise (Optional)
  "status"?: string (enum: active, inactive) // * `active` - Active
* `inactive` - Inactive (Optional)
  "creationMethod":
  {
  }
  "sourceFile": string // Name of the knowledge base file this multi-turn
test case was generated from
  "flaky"?: boolean // Flaky test cases still run and keep their results,
but are excluded from the pass rate. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "scenario": "Response string",
  "expectedOutcome": "Response string",
  "userDescription": "Response string",
  "status": "active",
  "creationMethod": {},
```

```
  "sourceFile": "Response string",
  "flaky": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Delete Multi-turn Test Case

DELETE `/api/v1/multi-turn-test-case/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Multi-turn test case.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/multi-turn-test-case/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/multi-turn-test-
case/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
204	No response body

List Available Evaluation Models

`GET` `/api/v1/evaluations/evaluation-llms/`

The model whose isDefault is true is the default evaluation model used when no evaluationLlmId is specified for an evaluation run.

Parameters

Parameter Name	Required	Type	Description
`chatbotId`	X	string	
`datasetId`	X	string	
`ordering`	X	string	Which field to use when ordering the results.
`query`	X	string	
`search`	X	string	A search term.
`status`	X	string	`pending`: Pending evaluation ; `running`: Evaluating ; `stopping`: Stopping ; `completed`: Completed ; `partial_completed`: Partially completed ; `failed`: Evaluation failed;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/evaluations/evaluation-llms/?chatbotId=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-e29b-41d4-a716-446655440000&ordering=example&query=example&search=example&status=completed"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/evaluations/evaluation-
llms/?chatbotId=550e8400-e29b-41d4-a716-
446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&query=example&search=example&status=co
mpleted", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/evaluation-llms/?chatbotId=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-e29b-41d4-a716-446655440000&ordering=example&query=example&search=example&status=completed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/evaluations/evaluation-llms/?
chatbotId=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-
e29b-41d4-a716-
446655440000&ordering=example&query=example&search=example&status=co
mpleted", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
[
  {
    "id": string (uuid)
    "name": string
    "value": string
    "provider": string
    "isDefault": boolean
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "value": "Response string",
    "provider": "Response string",
    "isDefault": false
  }
]
```

Run Single-turn Evaluation

POST `/api/v1/evaluations/evaluate/`

datasetId and chatbotId are required; if evaluationLlmId is omitted, the default evaluation model is used. metricConfig toggles the built-in metrics with boolean values, and customMetrics accepts up to 10 custom metrics. Only test cases with status active are evaluated. The evaluation runs in the background and responds with 202 and an evaluationId. Credit: The organization's Credit is checked before submission, and 402 is returned if it is insufficient; when the evaluation ends, agentops_test_run and agentops_deepeval are deducted based on the number of completed test cases, and calls to the evaluation model are billed separately.

Request Content

Request Parameters

Field	Type	Required	Description
datasetId	string (uuid)	Yes	
chatbotId	string (uuid)	Yes	
evaluationLlmId	string (uuid)	No	
name	string	No	
description	string	No	
metricConfig	object	No	
customMetrics	array[DatasetEvaluationCustomMetric]	No	

Example Request Structure

```
{
  "datasetId": string (uuid)
  "chatbotId": string (uuid)
  "evaluationLlmId"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
  "metricConfig"?: object // Optional
  "customMetrics"?: [ // Optional
    {
      "name": string
      "criteria"?: string // Optional
      "evaluationSteps"?: [ // Optional
        string
      ]
      "threshold"?: number (double) // Optional
    }
  ]
}
```

Example Request Values

```
{
  "dataset_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "chatbot_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "evaluation_llm_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "name": "Evaluation Name",
  "description": "Evaluation Description",
  "metric_config": {
    "ground_truth": true,
    "task_completion": false,
    "tool_correctness": false,
    "guardrail": true,
    "summarize": false,
    "hallucination": true
  },
  "custom_metrics": [
    {
      "name": "Support Tone",
      "criteria": "The answer should be polite, empathetic, and
professional in tone.",
      "threshold": 0.6
    }
  ]
}
```


Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/evaluations/evaluate/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "dataset_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "chatbot_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "evaluation_llm_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "name": "Evaluation Name",
    "description": "Evaluation Description",
    "metric_config": {
      "ground_truth": true,
      "task_completion": false,
      "tool_correctness": false,
      "guardrail": true,
      "summarize": false,
      "hallucination": true
    },
    "custom_metrics": [
      {
        "name": "Support Tone",
        "criteria": "The answer should be polite, empathetic, and
professional in tone.",
        "threshold": 0.6
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "dataset_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "chatbot_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "evaluation_llm_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "name": "Evaluation Name",
    "description": "Evaluation Description",
    "metric_config": {
      "ground_truth": true,
      "task_completion": false,
      "tool_correctness": false,
      "guardrail": true,
      "summarize": false,
      "hallucination": true
    },
    "custom_metrics": [
      {
        "name": "Support Tone",
        "criteria": "The answer should be polite, empathetic, and
professional in tone.",
        "threshold": 0.6
      }
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/evaluations/evaluate/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
```

```
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/evaluate/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "dataset_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "chatbot_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "evaluation_llm_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "name": "Evaluation Name",
    "description": "Evaluation Description",
    "metric_config": {
        "ground_truth": True,
        "task_completion": False,
        "tool_correctness": False,
        "guardrail": True,
        "summarize": False,
        "hallucination": True
    },
    "custom_metrics": [
        {
            "name": "Support Tone",
            "criteria": "The answer should be polite, empathetic,
and professional in tone.",
            "threshold": 0.6
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/evaluations/evaluate/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'dataset_id' => '3fa85f64-5717-4562-b3fc-2c963f66afa6',
            'chatbot_id' => '3fa85f64-5717-4562-b3fc-2c963f66afa6',
            'evaluation_llm_id' => '3fa85f64-5717-4562-b3fc-
2c963f66afa6',
            'name' => 'Evaluation Name',
            'description' => 'Evaluation Description',
            'metric_config' => [
                'ground_truth' => true,
                'task_completion' => false,
                'tool_correctness' => false,
                'guardrail' => true,
                'summarize' => false,
                'hallucination' => true
            ],
            'custom_metrics' => [
                [
                    'name' => 'Support Tone',
                    'criteria' => 'The answer should be polite,
empathetic, and professional in tone.',
                    'threshold' => 0.6
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
```

```
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 202

Example Response Structure

```
object
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example name",
  "description": "Example description"
}
```

Run Multi-Turn Evaluation

POST `/api/v1/evaluations/multi-turn-evaluate/`

Simulates a conversation between a user and the Agent using the multi-turn test cases in the test set, then scores it. maxTurns is the maximum number of turns the simulated user speaks (2–10, default 6),

and language is the language of the simulated user (Traditional Chinese, Simplified Chinese, English, Japanese, Korean; default Traditional Chinese). Returns 400 if the test set has no active multi-turn test cases. The response and Credit usage are the same as Run Single-Turn Evaluation.

Request Body

Request Parameters

Field	Type	Required	Description
datasetId	string (uuid)	Yes	
chatbotId	string (uuid)	Yes	
evaluationLlmId	string (uuid)	No	
name	string	No	
description	string	No	
maxTurns	integer	No	
minTurns	integer	No	
turnFaithfulness	boolean	No	
turnRelevancy	boolean	No	
conversationCompleteness	boolean	No	
language	object	No	
customMetrics	array[MultiTurnCustomMetric]	No	

Example Request Schema

```
{
  "datasetId": string (uuid)
  "chatbotId": string (uuid)
  "evaluationLlmId"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
  "maxTurns"?: integer // Optional
  "minTurns"?: integer // Optional
  "turnFaithfulness"?: boolean // Optional
```

```
  "turnRelevancy"?: boolean // Optional
  "conversationCompleteness"?: boolean // Optional
  "language"?:  // Optional
  {
  }
  "customMetrics"?: [ // Optional
    {
      "name": string
      "criteria"?: string // Optional
      "evaluationSteps"?: [ // Optional
        string
      ]
      "threshold"?: number (double) // Optional
    }
  ]
}
```

Example Request Values

```
{
  "dataset_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "chatbot_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "max_turns": 6,
  "min_turns": 2,
  "custom_metrics": [
    {
      "name": "Identity Check",
      "criteria": "The assistant confirms the user identity before
sharing account details.",
      "threshold": 0.6
    }
  ]
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/evaluations/multi-turn-evaluate/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "dataset_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "chatbot_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "max_turns": 6,
    "min_turns": 2,
    "custom_metrics": [
      {
        "name": "Identity Check",
        "criteria": "The assistant confirms the user identity before sharing account details.",
        "threshold": 0.6
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
    "dataset_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "chatbot_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "max_turns": 6,
    "min_turns": 2,
    "custom_metrics": [
      {
        "name": "Identity Check",
        "criteria": "The assistant confirms the user identity before
sharing account details.",
        "threshold": 0.6
      }
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/evaluations/multi-turn-
evaluate/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/multi-turn-
evaluate/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "dataset_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "chatbot_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "max_turns": 6,
    "min_turns": 2,
    "custom_metrics": [
        {
            "name": "Identity Check",
            "criteria": "The assistant confirms the user identity
before sharing account details.",
            "threshold": 0.6
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/evaluations/multi-turn-
evaluate/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'dataset_id' => '3fa85f64-5717-4562-b3fc-2c963f66afa6',
            'chatbot_id' => '3fa85f64-5717-4562-b3fc-2c963f66afa6',
            'max_turns' => 6,
            'min_turns' => 2,
            'custom_metrics' => [
                [
                    'name' => 'Identity Check',
                    'criteria' => 'The assistant confirms the user
identity before sharing account details.',
                    'threshold' => 0.6
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 202

Example Response Schema

```
object
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
}
```

List Evaluations

GET `/api/v1/evaluations/`

Filter by datasetId, chatbotId, or status.

Parameters

Parameter Name	Required	Type	Description
`chatbotId`	X	string	
`datasetId`	X	string	

`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	A search term.
`status`	X	string	`pending`: Pending evaluation ; `running`: Evaluating ; `stopping`: Stopping ; `completed`: Completed ; `partial_completed`: Partially completed ; `failed`: Evaluation failed;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/evaluations/?
chatbotId=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-
e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/evaluations/?
chatbotId=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-
e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/?
chatbotId=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-
e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/evaluations/?
chatbotId=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-
e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "chatbot":
      {
        "id": string (uuid)
        "name": string
      }
      "dataset":
      {
        "id": string (uuid)
        "name": string
      }
      "organization":
      {
        "id": string (uuid)
        "name": string
      }
      "evaluationLlm":
      {
        "id": string (uuid)
        "name": string
      }
      "status":
      {
      }
      "startedAt": string (timestamp)
      "completedAt": string (timestamp)
      "totalTestCases": integer
      "completedTestCases": integer
      "passedTestCases": integer // Number of test cases that passed
      "metricScores": object // Detailed score per evaluation metric,
such as ground_truth, task_completion and tool_correctness
      "evaluationConfig": object
      "progressPercentage": number (double)
      "overallScore": number (double)
      "passRate": number (double)
      "averageExecutionTimeMs": integer
```

```
      "totalExecutionTimeMs": integer
      "chatbotConfig": string
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
      "insights":
      {
        "summary": string
        "metricStats"?: object // Optional
        "recommendations"?: object // Optional
      }
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "dataset": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "organization": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "evaluationLlm": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "status": {},
      "startedAt": "Response string",
      "completedAt": "Response string",
```

```
      "totalTestCases": 456,
      "completedTestCases": 456,
      "passedTestCases": 456,
      "metricScores": null,
      "evaluationConfig": null,
      "progressPercentage": 456,
      "overallScore": 456,
      "passRate": 456,
      "averageExecutionTimeMs": 456,
      "totalExecutionTimeMs": 456,
      "chatbotConfig": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string",
      "insights": {
        "summary": "Response string",
        "metricStats": null,
        "recommendations": null
      }
    }
  ]
}
```


Get Evaluation Result

`GET` `/api/v1/evaluations/{id}`

status is one of pending, running, stopping, completed, partial_completed, or failed. While running, check progressPercentage; once finished, check passRate, overallScore, and the metricScores of each metric. If no test case completes, the evaluation ends as failed.

Parameters

Parameter Name	Required	Type	Description

`id`	V	string	A UUID string identifying this Dataset evaluation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
```

```
  "description": string
  "chatbot":
  {
    "id": string (uuid)
    "name": string
  }
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "evaluationLlm":
  {
    "id": string (uuid)
    "name": string
  }
  "status":
  {
  }
  "startedAt": string (timestamp)
  "completedAt": string (timestamp)
  "totalTestCases": integer
  "completedTestCases": integer
  "passedTestCases": integer // Number of test cases that passed
  "metricScores": object // Detailed score per evaluation metric, such as
ground_truth, task_completion and tool_correctness
  "evaluationConfig": object
  "progressPercentage": number (double)
  "overallScore": number (double)
  "passRate": number (double)
  "averageExecutionTimeMs": integer
  "totalExecutionTimeMs": integer
  "chatbotConfig": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "insights":
  {
    "summary": string
    "metricStats"?: object // Optional
    "recommendations"?: object // Optional
```

```
  }
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "evaluationLlm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "status": {},
  "startedAt": "Response string",
  "completedAt": "Response string",
  "totalTestCases": 456,
  "completedTestCases": 456,
  "passedTestCases": 456,
  "metricScores": null,
  "evaluationConfig": null,
  "progressPercentage": 456,
  "overallScore": 456,
  "passRate": 456,
  "averageExecutionTimeMs": 456,
  "totalExecutionTimeMs": 456,
  "chatbotConfig": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "insights": {
    "summary": "Response string",
    "metricStats": null,
```

```
    "recommendations": null
  }
}
```

Get Per-Question Results of a Single-Turn Evaluation

GET `/api/v1/evaluations/{id}/details/`

Only applies to single-turn evaluations; multi-turn evaluations return 400. The response is an array of all test case results and is not paginated.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.
`chatbotId`	X	string	
`datasetId`	X	string	
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	A search term.
`status`	X	string	`pending`: Pending evaluation ; `running`: Evaluating ; `stopping`: Stopping ; `completed`: Completed ;

			`partial_completed`: Partially completed ; `failed`: Evaluation failed;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-41d4-a716-446655440000/details/?chatbotId=550e8400-e29b-41d4-a716-446655440000&datasetId=550e8400-e29b-41d4-a716-446655440000&ordering=example&page=1&pageSize=1&query=example&search=example&status=completed"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/details/?chatbotId=550e8400-e29b-41d4-a716-
446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/details/?chatbotId=550e8400-e29b-41d4-a716-
446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-41d4-
a716-446655440000/details/?chatbotId=550e8400-e29b-41d4-a716-
446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "datasetEvaluation":
      {
        "id": string (uuid)
        "name": string
      }
      "testCase":
      {
        "id": string (uuid)
        "dataset":
        {
          "id": string (uuid)
          "name": string
        }
        "question": string
        "groundTruth"?: string // Optional
        "status"?: string (enum: active, inactive, pending) // * `active`
- Active
* `inactive` - Inactive
* `pending` - Pending Review (Optional)
        "creationMethod"?: string (enum: manual, file, generate) // *
`manual` - Manual
* `file` - File import
* `generate` - Generated (Optional)
        "sourceFile"?: string // Name of the CSV, Excel or knowledge base
file this test case came from (Optional)
        "flaky"?: boolean // Flaky test cases still run and keep their
results, but are excluded from the pass rate. (Optional)
        "createdAt": string (timestamp)
        "updatedAt": string (timestamp)
      }
      "status":
      {
      }
      "chatbotResponse": string
      "context": [ // Enrich each context item with is_deleted status for
referenced knowledge base files.
        object
```

```
      ]
      "metricScores": object
      "executionTimeMs": integer
      "executionTimeSeconds": number (double)
      "passed": boolean
      "errorMessage": string // Error message recorded when the
evaluation fails
      "startedAt": string (timestamp)
      "completedAt": string (timestamp)
      "retryCount": integer // Number of times this test case has been
retried
      "lastAttemptAt": string (timestamp) // Time of the last evaluation
attempt
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "datasetEvaluation": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "testCase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "dataset": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        },
        "question": "Response string",
        "groundTruth": "Response string",
        "status": "active",
        "creationMethod": "manual",
        "sourceFile": "Response string",
        "flaky": false,
```

```
        "createdAt": "Response string",
        "updatedAt": "Response string"
      },
      "status": {},
      "chatbotResponse": "Response string",
      "context": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response example name",
          "description": "Response example description"
        }
      ],
      "metricScores": null,
      "executionTimeMs": 456,
      "executionTimeSeconds": 456,
      "passed": false,
      "errorMessage": "Response string",
      "startedAt": "Response string",
      "completedAt": "Response string",
      "retryCount": 456,
      "lastAttemptAt": "Response string",
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Get Per-Conversation Results of a Multi-Turn Evaluation

`GET` `/api/v1/evaluations/{id}/multi-turn-results/`

Only applies to multi-turn evaluations; single-turn evaluations return 400. The response is paginated.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.
`chatbotId`	X	string	
`datasetId`	X	string	
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	A search term.
`status`	X	string	`pending`: Pending evaluation ; `running`: Evaluating ; `stopping`: Stopping ; `completed`: Completed ; `partial_completed`: Partially completed ; `failed`: Evaluation failed;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/multi-turn-results/?chatbotId=550e8400-
e29b-41d4-a716-446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/multi-turn-results/?chatbotId=550e8400-e29b-
41d4-a716-446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/multi-turn-results/?chatbotId=550e8400-e29b-
41d4-a716-446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-41d4-
a716-446655440000/multi-turn-results/?chatbotId=550e8400-e29b-41d4-
a716-446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "multiTurnTestCase": string (uuid)
      "scenario": string
      "expectedOutcome": string
      "flaky": boolean
      "status":
      {
      }
      "turns": object // The whole simulated conversation; each turn is
{role, content, retrieval_context}
      "metricScores": object
      "passed": boolean
      "errorMessage": string
      "startedAt": string (timestamp)
      "completedAt": string (timestamp)
      "createdAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "multiTurnTestCase": "550e8400-e29b-41d4-a716-446655440000",
      "scenario": "Response string",
      "expectedOutcome": "Response string",
      "flaky": false,
      "status": {},
      "turns": null,
      "metricScores": null,
      "passed": false,
```

```
      "errorMessage": "Response string",
      "startedAt": "Response string",
      "completedAt": "Response string",
      "createdAt": "Response string"
    }
  ]
}
```


Cancel a Running Evaluation

POST `/api/v1/evaluations/{id}/cancel/`

Only evaluations whose status is running can be canceled; otherwise 400 is returned. After cancellation, status first becomes stopping and ends as partial_completed once stopped. Scores already calculated are kept, and completed test cases are still charged Credit as usual. You can then use Resume an Unfinished Evaluation to run the rest.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/cancel/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/cancel/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/cancel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/cancel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Example Response Structure

```
{
  "id": string (uuid)
  "name": string
```

```
  "description": string
  "chatbot":
  {
    "id": string (uuid)
    "name": string
  }
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "evaluationLlm":
  {
    "id": string (uuid)
    "name": string
  }
  "status":
  {
  }
  "startedAt": string (timestamp)
  "completedAt": string (timestamp)
  "totalTestCases": integer
  "completedTestCases": integer
  "passedTestCases": integer // Number of test cases that passed
  "metricScores": object // Detailed score per evaluation metric, such as
ground_truth, task_completion and tool_correctness
  "evaluationConfig": object
  "progressPercentage": number (double)
  "overallScore": number (double)
  "passRate": number (double)
  "averageExecutionTimeMs": integer
  "totalExecutionTimeMs": integer
  "chatbotConfig": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "insights":
  {
    "summary": string
    "metricStats"?: object // Optional
    "recommendations"?: object // Optional
```

```
  }
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "evaluationLlm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "status": {},
  "startedAt": "Response string",
  "completedAt": "Response string",
  "totalTestCases": 456,
  "completedTestCases": 456,
  "passedTestCases": 456,
  "metricScores": null,
  "evaluationConfig": null,
  "progressPercentage": 456,
  "overallScore": 456,
  "passRate": 456,
  "averageExecutionTimeMs": 456,
  "totalExecutionTimeMs": 456,
  "chatbotConfig": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string",
  "insights": {
    "summary": "Response string",
    "metricStats": null,
```

```
    "recommendations": null
  }
}
```

Resume an Unfinished Evaluation

POST `/api/v1/evaluations/{id}/resume/`

Only evaluations whose status is partial_completed can be resumed; otherwise 400 is returned. Failed and not-yet-run test cases are re-run while completed results are kept, and the response is 202. Credit is charged only for completed cases that were not previously billed.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/resume/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/resume/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/resume/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/resume/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 202

Response Schema Example

```
object
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
}
```

Export Evaluation Report

POST `/api/v1/evaluations/{id}/export/`

Only single-round evaluations whose status is completed can be exported; multi-round evaluations are not currently supported for export and return 400. The report is generated as an Excel file in the background, and the response is 201 with an export record. Then use Get Latest Export Status to get the download URL.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/export/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/export/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/export/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.post(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/export/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "userEmail": string (email)
```

```
  "datasetEvaluationId": string (uuid)
  "datasetEvaluationName": string
  "status":
  {
  }
  "statusDisplay": string
  "fileUrl": string // Returns the file URL
  "fileSize": integer // File size in bytes
  "errorMessage": string
  "processingDurationSeconds": number (double) // Calculated processing
duration (seconds)
  "createdAt": string (timestamp)
  "startedAt": string (timestamp)
  "completedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "userEmail": "response@example.com",
  "datasetEvaluationId": "550e8400-e29b-41d4-a716-446655440000",
  "datasetEvaluationName": "Response string",
  "status": {},
  "statusDisplay": "Response string",
  "fileUrl": "Response string",
  "fileSize": 456,
  "errorMessage": "Response string",
  "processingDurationSeconds": 456,
  "createdAt": "Response string",
  "startedAt": "Response string",
  "completedAt": "Response string"
}
```

Get Latest Export Status

GET `/api/v1/evaluations/{id}/export-status/`

Returns the most recent export record for this evaluation: status is pending, processing, completed, or failed; when completed, download the file from fileUrl. Returns 404 if this evaluation has no export records.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/export-status/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/export-status/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/export-status/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-41d4-
a716-446655440000/export-status/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "userEmail": string (email)
```

```
  "datasetEvaluationId": string (uuid)
  "datasetEvaluationName": string
  "status":
  {
  }
  "statusDisplay": string
  "fileUrl": string // Returns the file URL
  "fileSize": integer // File size in bytes
  "errorMessage": string
  "processingDurationSeconds": number (double) // Calculated processing
duration (seconds)
  "createdAt": string (timestamp)
  "startedAt": string (timestamp)
  "completedAt": string (timestamp)
}
```

Response Example Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "userEmail": "response@example.com",
  "datasetEvaluationId": "550e8400-e29b-41d4-a716-446655440000",
  "datasetEvaluationName": "Response string",
  "status": {},
  "statusDisplay": "Response string",
  "fileUrl": "Response string",
  "fileSize": 456,
  "errorMessage": "Response string",
  "processingDurationSeconds": 456,
  "createdAt": "Response string",
  "startedAt": "Response string",
  "completedAt": "Response string"
}
```


List Export Records

GET `/api/v1/evaluations/{id}/exports/`

The response is an array of all export records for this evaluation, sorted from newest to oldest, without pagination.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.
`chatbotId`	X	string	
`datasetId`	X	string	
`ordering`	X	string	Which field to use when ordering the results.
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`search`	X	string	A search term.
`status`	X	string	`pending`: Pending evaluation ; `running`: Evaluating ; `stopping`: Stopping ; `completed`: Completed ; `partial_completed`: Partially completed ; `failed`: Evaluation failed;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/exports/?chatbotId=550e8400-e29b-41d4-
a716-446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/exports/?chatbotId=550e8400-e29b-41d4-a716-
446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/exports/?chatbotId=550e8400-e29b-41d4-a716-
446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-41d4-
a716-446655440000/exports/?chatbotId=550e8400-e29b-41d4-a716-
446655440000&datasetId=550e8400-e29b-41d4-a716-
446655440000&ordering=example&page=1&pageSize=1&query=example&search
=example&status=completed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "userEmail": string (email)
      "datasetEvaluationId": string (uuid)
      "datasetEvaluationName": string
      "status":
      {
      }
      "statusDisplay": string
      "fileUrl": string // Returns the file URL
      "fileSize": integer // File size in bytes
      "processingDurationSeconds": number (double) // Calculated
processing duration (seconds)
      "createdAt": string (timestamp)
      "startedAt": string (timestamp)
      "completedAt": string (timestamp)
    }
  ]
}
```

Response Example Values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "userEmail": "response@example.com",
      "datasetEvaluationId": "550e8400-e29b-41d4-a716-446655440000",
      "datasetEvaluationName": "Response string",
      "status": {},
      "statusDisplay": "Response string",
      "fileUrl": "Response string",
      "fileSize": 456,
      "processingDurationSeconds": 456,
      "createdAt": "Response string",
```

```
        "startedAt": "Response string",
        "completedAt": "Response string"
      }
    ]
}
```


Edit Evaluation Name and Description

PATCH `/api/v1/evaluations/{id}`

Only name and description are accepted; other fields are ignored. An evaluation can be edited in any status.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.

Request Content

Request Parameters

Field	Type	Required	Description
name	string	No	
description	string	No	

Request Schema Example

```
{
  "name"?: string // Optional
  "description"?: string // Optional
}
```

Request Example Values

```
{
  "name": "Example name",
  "description": "Example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string"
  };

axios.patch("https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "description": string
  "chatbot":
  {
    "id": string (uuid)
    "name": string
  }
  "dataset":
  {
    "id": string (uuid)
    "name": string
  }
  "organization":
  {
    "id": string (uuid)
    "name": string
  }
  "evaluationLlm":
  {
    "id": string (uuid)
    "name": string
  }
  "status":
  {
  }
  "startedAt": string (timestamp)
  "completedAt": string (timestamp)
  "totalTestCases": integer
  "completedTestCases": integer
  "passedTestCases": integer // Number of test cases that passed
  "metricScores": object // Detailed score per evaluation metric, such as
ground_truth, task_completion and tool_correctness
  "evaluationConfig": object
  "progressPercentage": number (double)
  "overallScore": number (double)
  "passRate": number (double)
  "averageExecutionTimeMs": integer
  "totalExecutionTimeMs": integer
  "chatbotConfig": string
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "insights":
```

```
  {
    "summary": string
    "metricStats"?: object // Optional
    "recommendations"?: object // Optional
  }
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "dataset": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "evaluationLlm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "status": {},
  "startedAt": "Response string",
  "completedAt": "Response string",
  "totalTestCases": 456,
  "completedTestCases": 456,
  "passedTestCases": 456,
  "metricScores": null,
  "evaluationConfig": null,
  "progressPercentage": 456,
  "overallScore": 456,
  "passRate": 456,
  "averageExecutionTimeMs": 456,
  "totalExecutionTimeMs": 456,
  "chatbotConfig": "Response string",
  "createdAt": "Response string",
```

```
  "updatedAt": "Response string",
  "insights": {
    "summary": "Response string",
    "metricStats": null,
    "recommendations": null
  }
}
```


Delete Evaluation

DELETE `/api/v1/evaluations/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Dataset evaluation.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/evaluations/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/evaluations/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code	Description
204	No response body

API Call Example (Shell)

目錄

1. Create SQL Database Connection
2. List SQL Databases
3. Get Specific SQL Database
4. Update SQL Database
5. Partially Update SQL Database
6. Delete SQL Database
7. Test database connection
8. Upload files to a database
9. Batch delete SQL databases
10. List database tables
11. Create a database table
12. Get Specific Table
13. Update Table
14. Partially Update Table
15. Delete Table
16. Batch delete tables
17. List table columns
18. Get a specific table column
19. Update a table column
20. Partially update a table column
21. Test Database Connection (General)
22. List Database Connection String Examples
23. Retrieve a Specific Connection String Example
24. List Database Configurations of an AI Assistant
25. Add Database Configuration to an AI Assistant
26. Get Specific Database Configuration
27. Update Database Configuration
28. Partially Update Database Configuration
29. Delete Database Configuration

API Call Example (Shell)

Create SQL Database Connection

`POST` `/api/v1/sql-databases/`

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseType	object	Yes	Type of database: PostgreSQL, MySQL, MSSQL, Oracle, SynxDB, or MaiAgent `postgresql` : PostgreSQL ; `mysql` : MySQL ; `maiagent` : MaiAgent ; `oracle` : Oracle ; `mssql` : MSSQL ; `synxdb` : SynxDB;
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Example Request Schema

```
{
  "name": string // Display name for this database connection
  "description"?: string // Description of the database and its purpose
(Optional)
  "databaseType":  // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl"?: string // Connection string for external databases. Not
required for MaiAgent type as it uses the system default. (Optional)
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (Optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (Optional)
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
}
```

Example Request Values

```
{
  "name": "Example name",
  "description": "Example string",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
```

```
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```


Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/sql-databases/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
```

```
  console.error(error.response?.data || error.message);
});
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": None,
    "isActive": True,
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-databases/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string',
            'databaseType' => new \stdClass(),
            'databaseUrl' => 'Example string',
            'includeTables' => null,
            'isActive' => true,
            'chatbots' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Display name for this database connection
  "description"?: string // Description of the database and its purpose
(Optional)
  "databaseType":  // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrlMasked": string
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (Optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (Optional)
  "status":
  {
  }
  "deletedAt": string (timestamp)
  "tables": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
```

```
table was created from. Empty for tables created before this field was
added.
      "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
      "status":
      {
      }
      "errorMessage": string // Error details if table creation or
deletion failed
      "sourceFile":
      {
        "id": string (uuid)
        "name": string
        "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
      }
      "columns": [
        {
          "id": string (uuid)
          "columnName": string
          "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (Optional)
          "description"?: string // Description of the column for better
Text-to-SQL understanding (Optional)
          "isPrimaryKey"?: boolean // Optional
          "isNullable"?: boolean // Optional
          "createdAt": string (timestamp)
          "updatedAt": string (timestamp)
        }
      ]
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "chatbots": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups": [
    {
```

```
      "id": string (uuid)
      "name": string
    }
  ]
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "databaseType": {},
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "status": {},
  "deletedAt": "Response string",
  "tables": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tableName": "Response string",
      "sheetName": "Response string",
      "description": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "sourceFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isFromKnowledgeBase": false
      },
      "columns": [
        {
```

```
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "columnName": "Response string",
          "columnType": "Response string",
          "description": "Response string",
          "isPrimaryKey": false,
          "isNullable": false,
          "createdAt": "Response string",
          "updatedAt": "Response string"
        }
      ],
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


List SQL Databases

GET `/api/v1/sql-databases/`

Parameters

Parameter Name	Required	Type	Description
`databaseType`	X	string	Type of database: PostgreSQL, MySQL, MSSQL, Oracle, SynxDB, or MaiAgent `postgresql`: PostgreSQL ; `mysql`: MySQL ; `maiagent`: MaiAgent ; `oracle`: Oracle ; `mssql`: MSSQL ; `synxdb`: SynxDB;
`isActive`	X	boolean	
`isExternal`	X	boolean	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-databases/?databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=1&query=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
```

```
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string // Display name for this database connection
      "description"?: string // Description of the database and its
purpose (Optional)
      "databaseType":  // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
      {
      }
      "databaseUrlMasked": string
      "includeTables"?: object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files. (Optional)
      "isActive"?: boolean // When disabled, this database will not be
used in Text-to-SQL queries (Optional)
      "status":
      {
      }
      "deletedAt": string (timestamp)
      "tables": [
        {
          "id": string (uuid)
          "tableName": string // Name of the table in the database
        }
      ]
      "tablesCount": integer
      "chatbots": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "groups": [
        {
          "id": string (uuid)
```

```
          "name": string
        }
      ]
      "createdBy":
      {
        "id": string (uuid)
        "name": string
      }
      "canUpdate": boolean // Return whether the current user can update
this resource.
      "canDelete": boolean // Return whether the current user can delete
this resource.
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "databaseType": {},
      "databaseUrlMasked": "Response string",
      "includeTables": null,
      "isActive": false,
      "status": {},
      "deletedAt": "Response string",
      "tables": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "tableName": "Response string"
        }
      ],
      "tablesCount": 456,
      "chatbots": [
        {
```

```
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ],
        "groups": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ],
        "createdBy": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        },
        "canUpdate": false,
        "canDelete": false,
        "createdAt": "Response string",
        "updatedAt": "Response string"
      }
    ]
}
```


Get Specific SQL Database

`GET` `/api/v1/sql-databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this SQL Database.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Display name for this database connection
  "description"?: string // Description of the database and its purpose
```

```
(Optional)
  "databaseType":  // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrlMasked": string
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (Optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (Optional)
  "status":
  {
  }
  "deletedAt": string (timestamp)
  "tables": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
table was created from. Empty for tables created before this field was
added.
      "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
      "status":
      {
      }
      "errorMessage": string // Error details if table creation or
deletion failed
      "sourceFile":
      {
        "id": string (uuid)
        "name": string
        "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
```

```
      }
      "columns": [
        {
          "id": string (uuid)
          "columnName": string
          "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (Optional)
          "description"?: string // Description of the column for better
Text-to-SQL understanding (Optional)
          "isPrimaryKey"?: boolean // Optional
          "isNullable"?: boolean // Optional
          "createdAt": string (timestamp)
          "updatedAt": string (timestamp)
        }
      ]
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "chatbots": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "databaseType": {},
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "status": {},
  "deletedAt": "Response string",
  "tables": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tableName": "Response string",
      "sheetName": "Response string",
      "description": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "sourceFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isFromKnowledgeBase": false
      },
      "columns": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "columnName": "Response string",
          "columnType": "Response string",
          "description": "Response string",
          "isPrimaryKey": false,
          "isNullable": false,
          "createdAt": "Response string",
          "updatedAt": "Response string"
        }
      ],
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
```

```
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Update SQL Database

PUT `/api/v1/sql-databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this SQL Database.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Example Request Schema

```
{
  "name": string // Display name for this database connection
  "description"?: string // Description of the database and its purpose
(Optional)
  "databaseUrl"?: string // Connection string for external databases. Not
required for MaiAgent type as it uses the system default. (Optional)
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (Optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (Optional)
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // Optional
    {
      "id": string (uuid)
    }
```

```
  ]
}
```

Example Request Values

```
{
  "name": "Example name",
  "description": "Example string",
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string",
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string",
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  };

axios.put("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "databaseUrl": "Example string",
    "includeTables": None,
    "isActive": True,
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string',
            'databaseUrl' => 'Example string',
            'includeTables' => null,
            'isActive' => true,
            'chatbots' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Display name for this database connection
  "description"?: string // Description of the database and its purpose
(Optional)
  "databaseType":  // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrlMasked": string
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (Optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (Optional)
  "status":
  {
  }
  "deletedAt": string (timestamp)
  "tables": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
```

```
table was created from. Empty for tables created before this field was
added.
      "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
      "status":
      {
      }
      "errorMessage": string // Error details if table creation or
deletion failed
      "sourceFile":
      {
        "id": string (uuid)
        "name": string
        "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
      }
      "columns": [
        {
          "id": string (uuid)
          "columnName": string
          "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (Optional)
          "description"?: string // Description of the column for better
Text-to-SQL understanding (Optional)
          "isPrimaryKey"?: boolean // Optional
          "isNullable"?: boolean // Optional
          "createdAt": string (timestamp)
          "updatedAt": string (timestamp)
        }
      ]
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "chatbots": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups": [
    {
```

```
      "id": string (uuid)
      "name": string
    }
  ]
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "databaseType": {},
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "status": {},
  "deletedAt": "Response string",
  "tables": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tableName": "Response string",
      "sheetName": "Response string",
      "description": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "sourceFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isFromKnowledgeBase": false
      },
      "columns": [
        {
```

```
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "columnName": "Response string",
          "columnType": "Response string",
          "description": "Response string",
          "isPrimaryKey": false,
          "isNullable": false,
          "createdAt": "Response string",
          "updatedAt": "Response string"
        }
      ],
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Partially Update SQL Database

PATCH `/api/v1/sql-databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this SQL Database.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Example Request Schema

```
{
  "name"?: string // Display name for this database connection (Optional)
  "description"?: string // Description of the database and its purpose
(Optional)
  "databaseUrl"?: string // Connection string for external databases. Not
required for MaiAgent type as it uses the system default. (Optional)
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (Optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (Optional)
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
}
```

Example Request Values

```
{
  "name": "Example name",
  "description": "Example string",
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true,
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
```

```
  ]
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "description": "Example string",
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "description": "Example string",
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true,
    "chatbots": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ],
    "groups": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000"
      }
    ]
  };

axios.patch("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "description": "Example string",
    "databaseUrl": "Example string",
    "includeTables": None,
    "isActive": True,
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'description' => 'Example string',
            'databaseUrl' => 'Example string',
            'includeTables' => null,
            'isActive' => true,
            'chatbots' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ],
            'groups' => [
                [
                    'id' => '550e8400-e29b-41d4-a716-446655440000'
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Display name for this database connection
  "description"?: string // Description of the database and its purpose
(Optional)
  "databaseType":  // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrlMasked": string
  "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (Optional)
  "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (Optional)
  "status":
  {
  }
  "deletedAt": string (timestamp)
  "tables": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
```

```
table was created from. Empty for tables created before this field was
added.
      "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
      "status":
      {
      }
      "errorMessage": string // Error details if table creation or
deletion failed
      "sourceFile":
      {
        "id": string (uuid)
        "name": string
        "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
      }
      "columns": [
        {
          "id": string (uuid)
          "columnName": string
          "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (Optional)
          "description"?: string // Description of the column for better
Text-to-SQL understanding (Optional)
          "isPrimaryKey"?: boolean // Optional
          "isNullable"?: boolean // Optional
          "createdAt": string (timestamp)
          "updatedAt": string (timestamp)
        }
      ]
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "chatbots": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups": [
    {
```

```
      "id": string (uuid)
      "name": string
    }
  ]
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "databaseType": {},
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "status": {},
  "deletedAt": "Response string",
  "tables": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tableName": "Response string",
      "sheetName": "Response string",
      "description": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "sourceFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isFromKnowledgeBase": false
      },
      "columns": [
        {
```

```
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "columnName": "Response string",
          "columnType": "Response string",
          "description": "Response string",
          "isPrimaryKey": false,
          "isNullable": false,
          "createdAt": "Response string",
          "updatedAt": "Response string"
        }
      ],
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Delete SQL Database

DELETE `/api/v1/sql-databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this SQL Database.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running
this example.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description
204	No response body

Test database connection

POST `/api/v1/sql-databases/{id}/test-connection/`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this SQL Database.

Request body

Request parameters

Field	Type	Required	Description
databaseType	string (enum: postgresql, mysql, maiagent, oracle, mssql, synxdb)	Yes	`postgresql` : PostgreSQL ; `mysql` : MySQL ; `maiagent` : MaiAgent ; `oracle` : Oracle ; `mssql` : MSSQL ; `synxdb` : SynxDB;
databaseUrl	string	Yes	

Request schema example

```
{
  "databaseType": string (enum: postgresql, mysql, maiagent, oracle,
mssql, synxdb) // * `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  "databaseUrl": string
}
```

Request example value

```
{
  "databaseType": "postgresql",
  "databaseUrl": "Example string"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/test-connection/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "databaseType": "postgresql",
    "databaseUrl": "Example string"
  }'

# Replace YOUR_API_KEY and check the request data before running
this example.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "databaseType": "postgresql",
    "databaseUrl": "Example string"
  };

axios.post("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/test-connection/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-
41d4-a716-446655440000/test-connection/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "databaseType": "postgresql",
    "databaseUrl": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/test-connection/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'databaseType' => 'postgresql',
            'databaseUrl' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "success"?: boolean // optional
  "message"?: string // optional
}
```

Response example value

```
{
  "success": false,
  "message": "Response string"
}
```

Upload files to a database

POST `/api/v1/sql-databases/{id}/upload-files/`

Parameters

Parameter name	Required	Type	Description
`id`	V	string	A UUID string identifying this SQL Database.

Request body

Request parameters

Field	Type	Required	Description
files	array[object]	Yes	

Request schema example

```
{
  "files": [
    {
      "file": string // File path from presigned URL upload (e.g.,
media/chatbotfiles/xxx/file.csv)
      "filename": string // Original filename
      "size": integer // File size in bytes
    }
  ]
}
```

Request example value

```
{
  "files": [
    {
      "file": "media/chatbotfiles/xxx/data.csv",
      "filename": "data.csv",
      "size": 1024
    },
    {
      "file": "media/chatbotfiles/xxx/report.xlsx",
      "filename": "report.xlsx",
      "size": 2048
    }
  ]
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/upload-files/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "files": [
      {
        "file": "media/chatbotfiles/xxx/data.csv",
        "filename": "data.csv",
        "size": 1024
      },
      {
        "file": "media/chatbotfiles/xxx/report.xlsx",
        "filename": "report.xlsx",
        "size": 2048
      }
    ]
  }'

# Replace YOUR_API_KEY and check the request data before running
this example.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "files": [
      {
        "file": "media/chatbotfiles/xxx/data.csv",
        "filename": "data.csv",
        "size": 1024
      },
      {
        "file": "media/chatbotfiles/xxx/report.xlsx",
        "filename": "report.xlsx",
        "size": 2048
      }
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/upload-files/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-
41d4-a716-446655440000/upload-files/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "files": [
        {
            "file": "media/chatbotfiles/xxx/data.csv",
            "filename": "data.csv",
            "size": 1024
        },
        {
            "file": "media/chatbotfiles/xxx/report.xlsx",
            "filename": "report.xlsx",
            "size": 2048
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/upload-files/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'files' => [
                [
                    'file' => 'media/chatbotfiles/xxx/data.csv',
                    'filename' => 'data.csv',
                    'size' => 1024
                ],
                [
                    'file' => 'media/chatbotfiles/xxx/report.xlsx',
                    'filename' => 'report.xlsx',
                    'size' => 2048
                ]
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 201

Response schema example

```
{
  "message"?: string // optional
  "filesCount"?: integer // optional
  "fileIds"?: [ // optional
    string (uuid)
  ]
}
```

Response example value

```
{
  "message": "Response string",
  "filesCount": 456,
  "fileIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Batch delete SQL databases

POST `/api/v1/sql-databases/batch-delete/`

Request body

Request parameters

Field	Type	Required	Description
ids	array[string]	Yes	

Request schema example

```
{
  "ids": [
    string (uuid)
  ]
}
```

Request example value

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/batch-
delete/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and check the request data before running
this example.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/sql-databases/batch-
delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'ids' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code	Description

204	No response body

List database tables

GET `/api/v1/sql-databases/{sqlDatabasePk}/tables/`

Parameters

Parameter name	Required	Type	Description
`sqlDatabasePk`	V	string	
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.
`query`	X	string	
`status`	X	string	`pending`: Pending ; `processing`: Processing ; `success`: Success ; `failed`: Failed ; `dropping`: Dropping ; `drop_failed`: Drop Failed;

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and check the request data before running
this example.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/?page=1&pageSize=1&query=example&status=drop_failed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response body

Status code: 200

Response schema example

```
{
  "count": integer
  "next"?: string (uri) // optional
```

```
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "tableName": string // Name of the table in the database
      "sheetName": string // Name of the source worksheet/sheet this
table was created from. Empty for tables created before this field was
added.
      "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
      "status":
      {
      }
      "errorMessage": string // Error details if table creation or
deletion failed
      "sourceFile":
      {
        "id": string (uuid)
        "name": string
        "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
      }
      "columns": [
        {
          "id": string (uuid)
          "columnName": string
          "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
          "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
          "isPrimaryKey"?: boolean // optional
          "isNullable"?: boolean // optional
          "createdAt": string (timestamp)
          "updatedAt": string (timestamp)
        }
      ]
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Response example value


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tableName": "Response string",
      "sheetName": "Response string",
      "description": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "sourceFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "isFromKnowledgeBase": false
      },
      "columns": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "columnName": "Response string",
          "columnType": "Response string",
          "description": "Response string",
          "isPrimaryKey": false,
          "isNullable": false,
          "createdAt": "Response string",
          "updatedAt": "Response string"
        }
      ],
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ]
}
```


Create a database table

POST `/api/v1/sql-databases/{sqlDatabasePk}/tables/`

Parameters

Parameter name	Required	Type	Description
sqlDatabasePk	V	string	

Request body

Request parameters

Field	Type	Required	Description
tableName	string	Yes	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

Request schema example

```
{
  "tableName": string // Name of the table in the database
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
}
```

Request example value

```
{
  "tableName": "Example string",
```

```
  "description": "Example string"
}
```

Code examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "tableName": "Example string",
    "description": "Example string"
  }'

# Replace YOUR_API_KEY and check the request data before running
this example.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "tableName": "Example string",
    "description": "Example string"
  };

axios.post("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "tableName": "Example string",
    "description": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'tableName' => 'Example string',
            'description' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "tableName": string // Name of the table in the database
  "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
  "status":
  {
  }
  "errorMessage": string // Error details if table creation or deletion
failed
  "sourceFile":
  {
    "id": string (uuid)
    "name": string
    "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
  }
  "columns": [
    {
      "id": string (uuid)
      "columnName": string
      "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
      "description"?: string // Description of the column for better
Text-to-SQL understanding (Optional)
      "isPrimaryKey"?: boolean // Optional
      "isNullable"?: boolean // Optional
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tableName": "Response string",
  "sheetName": "Response string",
  "description": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "sourceFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isFromKnowledgeBase": false
  },
  "columns": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "columnName": "Response string",
      "columnType": "Response string",
      "description": "Response string",
      "isPrimaryKey": false,
      "isNullable": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Get Specific Table

`GET` `/api/v1/sql-databases/{sqlDatabasePk}/tables/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Database Table.
`sqlDatabasePk`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "tableName": string // Name of the table in the database
```

```
  "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
  "status":
  {
  }
  "errorMessage": string // Error details if table creation or deletion
failed
  "sourceFile":
  {
    "id": string (uuid)
    "name": string
    "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
  }
  "columns": [
    {
      "id": string (uuid)
      "columnName": string
      "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
      "description"?: string // Description of the column for better
Text-to-SQL understanding (Optional)
      "isPrimaryKey"?: boolean // Optional
      "isNullable"?: boolean // Optional
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tableName": "Response string",
  "sheetName": "Response string",
  "description": "Response string",
```

```
  "status": {},
  "errorMessage": "Response string",
  "sourceFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isFromKnowledgeBase": false
  },
  "columns": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "columnName": "Response string",
      "columnType": "Response string",
      "description": "Response string",
      "isPrimaryKey": false,
      "isNullable": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Update Table

PUT `/api/v1/sql-databases/{sqlDatabasePk}/tables/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Database Table.
`sqlDatabasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
tableName	string	Yes	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

Example Request Schema

```
{
  "tableName": string // Name of the table in the database
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
}
```

Example Request Values

```
{
  "tableName": "Example string",
  "description": "Example string"
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "tableName": "Example string",
    "description": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "tableName": "Example string",
    "description": "Example string"
  };

axios.put("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "tableName": "Example string",
    "description": "Example string"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'tableName' => 'Example string',
            'description' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "tableName": string // Name of the table in the database
  "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
  "status":
  {
  }
  "errorMessage": string // Error details if table creation or deletion
failed
  "sourceFile":
  {
    "id": string (uuid)
    "name": string
    "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
  }
  "columns": [
    {
      "id": string (uuid)
      "columnName": string
      "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
      "description"?: string // Description of the column for better
Text-to-SQL understanding (Optional)
      "isPrimaryKey"?: boolean // Optional
      "isNullable"?: boolean // Optional
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tableName": "Response string",
  "sheetName": "Response string",
  "description": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "sourceFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isFromKnowledgeBase": false
  },
  "columns": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "columnName": "Response string",
      "columnType": "Response string",
      "description": "Response string",
      "isPrimaryKey": false,
      "isNullable": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Partially Update Table

PATCH `/api/v1/sql-databases/{sqlDatabasePk}/tables/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Database Table.
`sqlDatabasePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
tableName	string	No	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

Example Request Schema

```
{
  "tableName"?: string // Name of the table in the database (Optional)
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
}
```

Example Request Values

```
{
  "tableName": "Example string",
  "description": "Example string"
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "tableName": "Example string",
    "description": "Example string"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "tableName": "Example string",
    "description": "Example string"
  };

axios.patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "tableName": "Example string",
    "description": "Example string"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'tableName' => 'Example string',
            'description' => 'Example string'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "tableName": string // Name of the table in the database
  "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
  "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (Optional)
  "status":
  {
  }
  "errorMessage": string // Error details if table creation or deletion
failed
  "sourceFile":
  {
    "id": string (uuid)
    "name": string
    "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
  }
  "columns": [
    {
      "id": string (uuid)
      "columnName": string
      "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
      "description"?: string // Description of the column for better
Text-to-SQL understanding (Optional)
      "isPrimaryKey"?: boolean // Optional
      "isNullable"?: boolean // Optional
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tableName": "Response string",
  "sheetName": "Response string",
  "description": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "sourceFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "isFromKnowledgeBase": false
  },
  "columns": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "columnName": "Response string",
      "columnType": "Response string",
      "description": "Response string",
      "isPrimaryKey": false,
      "isNullable": false,
      "createdAt": "Response string",
      "updatedAt": "Response string"
    }
  ],
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```


Delete Table

DELETE `/api/v1/sql-databases/{sqlDatabasePk}/tables/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Database Table.
`sqlDatabasePk`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status code	Description
204	No response body

Batch delete tables

POST `/api/v1/sql-databases/{sqlDatabasePk}/tables/batch-delete/`

Parameters

Parameter Name	Required	Type	Description
sqlDatabasePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
ids	array[string]	Yes	

Example Request Schema

```
{
  "ids": [
    string (uuid)
  ]
}
```

Example Request Values

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
```

```
  ]
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/batch-delete/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "ids": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  };

axios.post("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/batch-delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'ids' => [
                '550e8400-e29b-41d4-a716-446655440000'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status code	Description

204	No response body

List table columns

GET `/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/`

Parameters

Parameter Name	Required	Type	Description
`sqlDatabasePk`	V	string	
`tablePk`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
[
  {
    "id": string (uuid)
    "columnName": string
```

```
    "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
    "description"?: string // Description of the column for better Text-
to-SQL understanding (Optional)
    "isPrimaryKey"?: boolean // Optional
    "isNullable"?: boolean // Optional
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
```

Example Response Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "columnName": "Response string",
    "columnType": "Response string",
    "description": "Response string",
    "isPrimaryKey": false,
    "isNullable": false,
    "createdAt": "Response string",
    "updatedAt": "Response string"
  }
]
```

Get a specific table column

GET `/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}`

Parameters

Parameter Name	Required	Type	Description

`id`	V	string	A UUID string identifying this Database Table Column.
`sqlDatabasePk`	V	string	
`tablePk`	V	string	

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "columnName": string
```

```
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
  "description"?: string // Description of the column for better Text-to-
SQL understanding (Optional)
  "isPrimaryKey"?: boolean // Optional
  "isNullable"?: boolean // Optional
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "columnName": "Response string",
  "columnType": "Response string",
  "description": "Response string",
  "isPrimaryKey": false,
  "isNullable": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Update a table column

PUT `/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Database Table Column.

`sqlDatabasePk`	V	string	
`tablePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
columnName	string	Yes	
columnType	string	No	Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP)
description	string	No	Description of the column for better Text-to-SQL understanding
isPrimaryKey	boolean	No	
isNullable	boolean	No	

Example Request Schema

```
{
  "columnName": string
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
  "description"?: string // Description of the column for better Text-to-
SQL understanding (Optional)
  "isPrimaryKey"?: boolean // Optional
  "isNullable"?: boolean // Optional
}
```

Example Request Values

```
{
  "columnName": "Example string",
  "columnType": "Example string",
  "description": "Example string",
```

```
  "isPrimaryKey": true,
  "isNullable": true
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "columnName": "Example string",
    "columnType": "Example string",
    "description": "Example string",
    "isPrimaryKey": true,
    "isNullable": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "columnName": "Example string",
    "columnType": "Example string",
    "description": "Example string",
    "isPrimaryKey": true,
    "isNullable": true
  };

axios.put("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "columnName": "Example string",
    "columnType": "Example string",
    "description": "Example string",
    "isPrimaryKey": True,
    "isNullable": True
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'columnName' => 'Example string',
            'columnType' => 'Example string',
            'description' => 'Example string',
            'isPrimaryKey' => true,
            'isNullable' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "columnName": string
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
  "description"?: string // Description of the column for better Text-to-
SQL understanding (Optional)
  "isPrimaryKey"?: boolean // Optional
  "isNullable"?: boolean // Optional
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "columnName": "Response string",
  "columnType": "Response string",
  "description": "Response string",
  "isPrimaryKey": false,
  "isNullable": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Partially update a table column

PATCH `/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}`

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	A UUID string identifying this Database Table Column.
`sqlDatabasePk`	V	string	
`tablePk`	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
columnName	string	No	
columnType	string	No	Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP)
description	string	No	Description of the column for better Text-to-SQL understanding
isPrimaryKey	boolean	No	
isNullable	boolean	No	

Example Request Schema

```
{
  "columnName"?: string // Optional
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
  "description"?: string // Description of the column for better Text-to-
SQL understanding (Optional)
  "isPrimaryKey"?: boolean // Optional
```

```
  "isNullable"?: boolean // Optional
}
```

Example Request Values

```
{
  "columnName": "Example string",
  "columnType": "Example string",
  "description": "Example string",
  "isPrimaryKey": true,
  "isNullable": true
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "columnName": "Example string",
    "columnType": "Example string",
    "description": "Example string",
    "isPrimaryKey": true,
    "isNullable": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "columnName": "Example string",
    "columnType": "Example string",
    "description": "Example string",
    "isPrimaryKey": true,
    "isNullable": true
  };

axios.patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "columnName": "Example string",
    "columnType": "Example string",
    "description": "Example string",
    "isPrimaryKey": True,
    "isNullable": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'columnName' => 'Example string',
            'columnType' => 'Example string',
            'description' => 'Example string',
            'isPrimaryKey' => true,
            'isNullable' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "columnName": string
  "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (Optional)
  "description"?: string // Description of the column for better Text-to-
SQL understanding (Optional)
  "isPrimaryKey"?: boolean // Optional
  "isNullable"?: boolean // Optional
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "columnName": "Response string",
  "columnType": "Response string",
  "description": "Response string",
  "isPrimaryKey": false,
  "isNullable": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Test Database Connection (General)

POST `/api/v1/database-connection/test/`

Request Body

Request Parameters

Field	Type	Required	Description
databaseType	object	Yes	Database type (excluding the type used internally by MaiAgent) `postgresql` : PostgreSQL ; `mysql` : MySQL ; `oracle` : Oracle ; `mssql` : MSSQL ; `synxdb` : SynxDB;
databaseUrl	string	Yes	Database connection string

Example Request Schema

```
{
  "databaseType":  // Database type (excluding the type used internally
by MaiAgent)

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string
}
```

Example Request Values

```
{
  "database_type": "postgresql",
  "database_url": "postgresql://user:password@localhost:5432/dbname"
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/database-
connection/test/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "database_type": "postgresql",
    "database_url":
"postgresql://user:password@localhost:5432/dbname"
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "database_type": "postgresql",
    "database_url":
"postgresql://user:password@localhost:5432/dbname"
  };

axios.post("https://api.maiagent.ai/api/v1/database-
connection/test/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/database-connection/test/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "database_type": "postgresql",
    "database_url":
"postgresql://user:password@localhost:5432/dbname"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/database-connection/test/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'database_type' => 'postgresql',
            'database_url' =>
'postgresql://user:password@localhost:5432/dbname'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "status":  // Connection status

* `success` - success
* `failed` - failed
  {
  }
  "responseTime"?: number (double) // Response time (seconds) (Optional)
  "message"?: string // Success message (Optional)
  "errorMessage"?: string // Error message (Optional)
}
```

Example Response Values

```
{
  "status": "success",
  "response_time": 0.123,
  "message": "Successfully connected to PostgreSQL database"
}
```

Status Code: 400

Example Response Schema

```
{
  "detail"?: string // Optional
}
```

Example Response Values

```
{
  "status": "success",
  "response_time": 0.123,
  "message": "Successfully connected to PostgreSQL database"
}
```

List Database Connection String Examples

GET `/api/v1/database-examples/`

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/database-examples/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/database-examples/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/database-examples/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/database-examples/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
[
  {
    "databaseType"?:  // Database type option
```

```
* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB (Optional)
    {
    }
    "databaseTypeExample": string // For example:
postgresql://db_user:db_password@db_ip:db_port/db_name
  }
]
```

Example Response Values

```
[
  {
    "databaseType": {},
    "databaseTypeExample": "Response string"
  }
]
```

Status Code: 401

Example Response Schema

```
{
  "detail"?: string // Optional
}
```

Example Response Values

```
{
  "detail": "Response string"
}
```

Retrieve a Specific Connection String Example

GET `/api/v1/database-examples/{id}`

Parameters

Parameter	Required	Type	Description
`id`	V	string	A UUID string identifying this Database Connection Example.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/database-
examples/550e8400-e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/database-
examples/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/database-examples/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/database-examples/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "databaseType"?:  // Database type option
```

```
* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB (Optional)
  {
  }
  "databaseTypeExample": string // For example:
postgresql://db_user:db_password@db_ip:db_port/db_name
}
```

Example Response Values

```
{
  "databaseType": {},
  "databaseTypeExample": "Response string"
}
```

List Database Configurations of an AI Assistant

GET `/api/v1/chatbots/{chatbotPk}/databases/`

Parameters

Parameter Name	Required	Type	Description
`chatbotPk`	V	string	A UUID string identifying this Chatbot ID
`page`	X	integer	A page number within the paginated result set.

`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
```

```
    {
      "id": string (uuid)
      "name": string // Display name, for example: Main Business DB,
Analytics DB.
      "databaseType":  // Database type: MySQL, PostgreSQL, MSSQL,
Oracle, SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
      {
      }
      "databaseUrl": string // Database connection string.
      "databaseUrlMasked": string
      "includeTables"?: object // Tables to query, formatted as
{"tables": ["table1", "table2"]}. The MaiAgent type reads them from
uploaded files automatically, so no manual setup is needed. (Optional)
      "isActive"?: boolean // When inactive, this database is not used
for Text-to-SQL queries. (Optional)
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Example Response Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "databaseType": {},
      "databaseUrl": "Response string",
      "databaseUrlMasked": "Response string",
      "includeTables": null,
      "isActive": false,
      "createdAt": "Response string",
```

```
        "updatedAt": "Response string"
      }
    ]
  }
```


Add Database Configuration to an AI Assistant

POST `/api/v1/chatbots/{chatbotPk}/databases/`

Parameters

Parameter Name	Required	Type	Description
`chatbotPk`	V	string	A UUID string identifying this Chatbot ID

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Display name, for example: Main Business DB, Analytics DB.
databaseType	object	Yes	Database type: MySQL, PostgreSQL, MSSQL, Oracle, SynxDB, or MaiAgent. `postgresql` : PostgreSQL ; `mysql` : MySQL ; `maiagent` : MaiAgent ; `oracle` : Oracle ; `mssql` : MSSQL ; `synxdb` : SynxDB;
databaseUrl	string	Yes	Database connection string.

Field	Type	Required	Description
includeTables	object	No	Tables to query, formatted as {"tables": ["table1", "table2"]}. The MaiAgent type reads them from uploaded files automatically, so no manual setup is needed.
isActive	boolean	No	When inactive, this database is not used for Text-to-SQL queries.

Example Request Schema

```
{
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType":  // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (Optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (Optional)
}
```

Example Request Values

```
{
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
```

```
  "isActive": true
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X POST "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
  };

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": None,
    "isActive": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'databaseType' => new \stdClass(),
            'databaseUrl' => 'Example string',
            'includeTables' => null,
            'isActive' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType":  // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (Optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "databaseType": {},
  "databaseUrl": "Response string",
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Get Specific Database Configuration

GET `/api/v1/chatbots/{chatbotPk}/databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`chatbotPk`	V	string	A UUID string identifying this Chatbot ID
`id`	V	string	A UUID string identifying this Chatbot Database Configuration.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X GET "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType":  // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
```

```
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (Optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "databaseType": {},
  "databaseUrl": "Response string",
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Update Database Configuration

PUT `/api/v1/chatbots/{chatbotPk}/databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`chatbotPk`	V	string	A UUID string identifying this Chatbot ID
`id`	V	string	A UUID string identifying this Chatbot Database Configuration.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	Yes	Display name, for example: Main Business DB, Analytics DB.
databaseType	object	Yes	Database type: MySQL, PostgreSQL, MSSQL, Oracle, SynxDB, or MaiAgent. `postgresql` : PostgreSQL ; `mysql` : MySQL ; `maiagent` : MaiAgent ; `oracle` : Oracle ; `mssql` : MSSQL ; `synxdb` : SynxDB;
databaseUrl	string	Yes	Database connection string.
includeTables	object	No	Tables to query, formatted as {"tables": ["table1", "table2"]}. The MaiAgent type reads them from uploaded files automatically, so no manual setup is needed.
isActive	boolean	No	When inactive, this database is not used for Text-to-SQL queries.

Example Request Schema

```
{
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType":  // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (Optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (Optional)
}
```

Example Request Values

```
{
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PUT "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
  };

axios.put(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": None,
    "isActive": True
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'databaseType' => new \stdClass(),
            'databaseUrl' => 'Example string',
            'includeTables' => null,
            'isActive' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType":  // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (Optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "databaseType": {},
  "databaseUrl": "Response string",
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Partially Update Database Configuration

`PATCH` `/api/v1/chatbots/{chatbotPk}/databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`chatbotPk`	V	string	A UUID string identifying this Chatbot ID
`id`	V	string	A UUID string identifying this Chatbot Database Configuration.

Request Body

Request Parameters

Field	Type	Required	Description
name	string	No	Display name, for example: Main Business DB, Analytics DB.
databaseType	object	No	Database type: MySQL, PostgreSQL, MSSQL, Oracle, SynxDB, or MaiAgent. `postgresql` : PostgreSQL ; `mysql` : MySQL ; `maiagent` : MaiAgent ; `oracle` : Oracle ; `mssql` : MSSQL ; `synxdb` : SynxDB;
databaseUrl	string	No	Database connection string.
includeTables	object	No	Tables to query, formatted as {"tables": ["table1", "table2"]}. The MaiAgent type reads them from uploaded files automatically, so no manual setup is needed.

Field	Type	Required	Description
isActive	boolean	No	When inactive, this database is not used for Text-to-SQL queries.

Example Request Schema

```
{
  "name"?: string // Display name, for example: Main Business DB, Analytics DB. (Optional)
  "databaseType"?:  // Database type: MySQL, PostgreSQL, MSSQL, Oracle, SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB (Optional)
  {
  }
  "databaseUrl"?: string // Database connection string. (Optional)
  "includeTables"?: object // Tables to query, formatted as {"tables": ["table1", "table2"]}. The MaiAgent type reads them from uploaded files automatically, so no manual setup is needed. (Optional)
  "isActive"?: boolean // When inactive, this database is not used for Text-to-SQL queries. (Optional)
}
```

Example Request Values

```
{
  "name": "Example name",
  "databaseType": {},
  "databaseUrl": "Example string",
  "includeTables": null,
  "isActive": true
}
```

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X PATCH "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
  }'

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": null,
    "isActive": true
  };

axios.patch(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example name",
    "databaseType": {},
    "databaseUrl": "Example string",
    "includeTables": None,
    "isActive": True
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'name' => 'Example name',
            'databaseType' => new \stdClass(),
            'databaseUrl' => 'Example string',
            'includeTables' => null,
            'isActive' => true
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "name": string // Display name, for example: Main Business DB,
Analytics DB.
  "databaseType":  // Database type: MySQL, PostgreSQL, MSSQL, Oracle,
SynxDB, or MaiAgent.

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
  {
  }
  "databaseUrl": string // Database connection string.
  "databaseUrlMasked": string
  "includeTables"?: object // Tables to query, formatted as {"tables":
["table1", "table2"]}. The MaiAgent type reads them from uploaded files
automatically, so no manual setup is needed. (Optional)
  "isActive"?: boolean // When inactive, this database is not used for
Text-to-SQL queries. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Example Response Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "databaseType": {},
  "databaseUrl": "Response string",
  "databaseUrlMasked": "Response string",
  "includeTables": null,
  "isActive": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Delete Database Configuration

DELETE `/api/v1/chatbots/{chatbotPk}/databases/{id}`

Parameters

Parameter Name	Required	Type	Description
`chatbotPk`	V	string	A UUID string identifying this Chatbot ID
`id`	V	string	A UUID string identifying this Chatbot Database Configuration.

Code Examples

Shell/Bash

```
# API Call Example (Shell)
curl -X DELETE "

  -H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request failed:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request failed:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request failed: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
204	No response body

Example API call (Shell)

目錄

1. Create New Attachment (Integrated)
2. Get Presigned URL for File Upload
3. Upload Attachment via Presigned URL
4. Get Specific Attachment
5. Create Conversation Attachment
6. Get Allowed File Specifications
7. List Conversation Attachments
8. Delete Conversation Attachment

Example API call (Shell)

Create New Attachment (Integrated)

`POST` `/api/v1/attachments-upload/`

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/attachments-upload/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -F "file=@example_file.pdf"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Replace with an actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/attachments-upload/",
formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/attachments-upload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
'application/pdf')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/attachments-upload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description

200	No response body

Get Presigned URL for File Upload

POST `/api/v1/upload-presigned-url/`

Parameters

Parameter	Required	Type	Description
`fieldName`	X	string	Field name
`filename`	X	string	File name
`modelName`	X	string	Model name

Request Body

Request Parameters

Field	Type	Required	Description
fileSize	integer	Yes	

Request Schema Example

```
{
  "fileSize": integer
}
```

Request Example

```
{
  "fileSize": 123
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "modelName": "example string",
    "fieldName": "example string",
    "filename": "document.pdf",
    "fileSize": 123
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "modelName": "example string",
    "fieldName": "example string",
    "filename": "document.pdf",
    "fileSize": 123
  };

axios.post("https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "modelName": "example string",
    "fieldName": "example string",
    "filename": "document.pdf",
    "fileSize": 123
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'modelName' => 'example string',
            'fieldName' => 'example string',
            'filename' => 'document.pdf',
            'fileSize' => 123
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "url": string (uri) // Upload target URL (bucket URL for S3/GCS; blob
URL with SAS for Azure)
  "fields": object // POST form fields; Azure includes only key
  "method"?:  // POST if omitted; Azure Blob returns PUT

* `POST` - POST
* `PUT` - PUT (Optional)
  {
  }
  "headers"?: object // Request headers required for PUT (Optional)
}
```

Response Example

```
{
  "url": "https://s3.example.com/bucket-name",
  "fields": {
    "key": "media/chat/attachment/filename.pdf",
    "x-amz-algorithm": "AWS4-HMAC-SHA256",
    "x-amz-credential":
"AKIAXXXXXXXXXXXXXXX/YYYYMMDD/region/s3/aws4_request",
    "x-amz-date": "YYYYMMDDTHHMMSSZ",
    "policy": "base64-encoded-policy-document",
    "x-amz-signature": "generated-signature-hash"
  }
}
```

Upload Attachment via Presigned URL

POST `/api/v1/attachments/`

Request Body

Request Parameters

Field	Type	Required	Description
type	object	No	
filename	string	Yes	
file	string (uri)	Yes	

Request Schema Example

```
{
  "type"?:  // Optional
  {
  }
  "filename": string
  "file": string (uri)
}
```

Request Example

```
{
  "type": {},
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg"
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/attachments/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "type": {},
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg"
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "type": {},
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg"
  };

axios.post("https://api.maiagent.ai/api/v1/attachments/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```python
import requests

url = "https://api.maiagent.ai/api/v1/attachments/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "type": {},
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/attachments/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'type' => new \stdClass(),
            'filename' => 'document.pdf',
            'file' => 'https://example.com/file.jpg'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "type"?:  // Optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

For conversation-bound attachments: conversation.last_message_created_at + retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
  "downloadStatus": string
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "expiresAt": "response string",
  "downloadStatus": "response string"
}
```

Get Specific Attachment

GET `/api/v1/attachments/{id}`

Parameters

Parameter	Required	Type	Description
`id`	V	string	A UUID string identifying this attachment.

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/attachments/550e8400-
e29b-41d4-a716-446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/attachments/550e8400-e29b-
41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/attachments/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/attachments/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
  "type"?:  // Optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

For conversation-bound attachments: conversation.last_message_created_at + retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
  "downloadStatus": string
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "expiresAt": "response string",
  "downloadStatus": "response string"
}
```

Create Conversation Attachment

POST `/api/v1/conversations/{conversationPk}/attachments/`

Parameters

Parameter	Required	Type	Description

conversationPk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
type	object	No	
filename	string	Yes	
file	string (uri)	Yes	
conversation	string (uuid)	No	

Request Schema Example

```
{
  "type"?:  // Optional
  {
  }
  "filename": string
  "file": string (uri)
  "conversation"?: string (uuid) // Optional
}
```

Request Example

```
{
  "type": {},
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "conversation": "550e8400-e29b-41d4-a716-446655440000"
}
```

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/attachments/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "type": {},
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "conversation": "550e8400-e29b-41d4-a716-446655440000"
  }'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request payload
const data = {
    "type": {},
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "conversation": "550e8400-e29b-41d4-a716-446655440000"
  };

axios.post("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/attachments/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-
41d4-a716-446655440000/attachments/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request payload
data = {
    "type": {},
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "conversation": "550e8400-e29b-41d4-a716-446655440000"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-
41d4-a716-446655440000/attachments/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => [
            'type' => new \stdClass(),
            'filename' => 'document.pdf',
            'file' => 'https://example.com/file.jpg',
            'conversation' => '550e8400-e29b-41d4-a716-446655440000'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "type"?:  // Optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

For conversation-bound attachments: conversation.last_message_created_at + retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
  "conversation"?: string (uuid) // Optional
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "expiresAt": "response string",
  "conversation": "550e8400-e29b-41d4-a716-446655440000"
}
```

Get Allowed File Specifications

GET `/api/v1/allowed-file-specs/`

Code Examples

Shell/Bash

```
# Example API call (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/allowed-file-specs/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/allowed-file-specs/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/allowed-file-specs/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/allowed-file-specs/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "webChat": object
  "organization": object
```

```
  "attachment": object
  "chatbotFile": object
  "knowledgeBaseFile": object
  "batchQa": object
  "meetingRecord": object
}
```

Response Example

```
{
  "web_chat": {
    "logo": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 5242880
      }
    ],
    "avatar": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 2097152
      }
    ],
    "cover": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 10485760
      }
    ],
```

```
    "powered-by-logo": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 5242880
      }
    ]
  },
  "organization": {
    "compact_logo": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 2097152
      }
    ],
    "full_logo": [
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 2097152
      }
    ]
  },
  "attachment": {
    "file": [
      {
        "content_type": {
          ".pdf": "application/pdf",
          ".doc": "application/msword",
          ".docx": "application/vnd.openxmlformats-
officedocument.wordprocessingml.document",
          ".txt": "text/plain"
```

```
        },
        "max_size": 20971520
      },
      {
        "content_type": {
          ".jpg": "image/jpeg",
          ".jpeg": "image/jpeg",
          ".png": "image/png",
          ".gif": "image/gif"
        },
        "max_size": 5242880
      }
    ]
  },
  "chatbot_file": {
    "file": [
      {
        "content_type": {
          ".pdf": "application/pdf",
          ".doc": "application/msword",
          ".docx": "application/vnd.openxmlformats-
officedocument.wordprocessingml.document",
          ".xlsx": "application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet",
          ".ppt": "application/vnd.ms-powerpoint",
          ".pptx": "application/vnd.openxmlformats-
officedocument.presentationml.presentation",
          ".txt": "text/plain"
        },
        "max_size": 104857600
      }
    ]
  },
  "batch_qa": {
    "file": [
      {
        "content_type": null,
        "max_size": 104857600
      }
    ]
  },
  "faq": {
    "answer": [
      {
        "content_type": {
```

```
            ".jpg": "image/jpeg",
            ".jpeg": "image/jpeg",
            ".png": "image/png",
            ".gif": "image/gif",
            ".webp": "image/webp",
            ".tiff": "image/tiff",
            ".mp4": "video/mp4"
          },
          "max_size": 104857600
        }
      ]
    }
}
```

List Conversation Attachments

`GET` `/api/v1/file-library/conversation-attachments/`

Required permissions: The organization has the file library enabled, and the caller's role has any file library permission. For the visible scope and how expiresAt is calculated, see the technical manual Conversation Attachment Management API。

Parameters

Parameter Name	Required	Type	Description
`page`	X	integer	A page number within the paginated result set.
`pageSize`	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/file-
library/conversation-attachments/?page=1&pageSize=1"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/file-library/conversation-attachments/?page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/file-library/conversation-
attachments/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/file-
library/conversation-attachments/?page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
```

```
  "results": [
    {
      "id": string (uuid)
      "filename": string
      "fileSize": integer
      "type":
      {
      }
      "conversationId": string (uuid)
      "expiresAt": string
      "createdAt": string (timestamp)
    }
  ]
}
```

Response Example Values


```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "fileSize": 456,
      "type": {},
      "conversationId": "550e8400-e29b-41d4-a716-446655440000",
      "expiresAt": "Response string",
      "createdAt": "Response string"
    }
  ]
}
```


Delete Conversation Attachment

DELETE `/api/v1/file-library/conversation-attachments/{id}`

Required permissions: Same as "List Conversation Attachments". You can only delete attachments visible to the caller; other attachments return 404. Deleting also removes the stored file if no other attachment or file shares it.

Parameters

Parameter Name	Required	Type	Description
`id`	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/file-
library/conversation-attachments/550e8400-e29b-41d4-a716-
446655440000/"

  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/file-
library/conversation-attachments/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/file-library/conversation-
attachments/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}


response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

PHP

```php
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/file-library/conversation-
attachments/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body